Cortex Cloud Platform APIs

Confidential - Copyright © Palo Alto Networks

1. Cortex Cloud APIs

1.1. Understand Cortex Cloud licenses

1.2. Get started with Cortex Cloud 1.3 APIs

1.3. Create a new API key

1.4. Get your Cortex API key ID

1.5. Get your FQDN

1.6. Make your first API call

1.7. Run XQL query APIs

1.8. Whats-new-in-this-release

2. APIs

2.1. Cortex Cloud Platform APIs

2.1.1. Preface

2.1.1.1. Servers

2.1.2. Cortex CLI

2.1.2.1. Get the latest version of the Cortex CLI

2.1.3. XQL query

2.1.3.1. Start an XQL query

2.1.3.2. Get XQL query results

2.1.3.3. Get XQL query Quota

2.1.3.4. Get XQL query results Stream

2.1.4. Endpoint Management

2.1.4.1. Get Distribution version

2.1.4.2. Get all Endpoints

2.1.4.3. Get Policy

2.1.4.4. Delete Endpoints

2.1.4.5. Create distributions

2.1.4.6. Get Violations

2.1.4.7. Get Distribution status

2.1.4.8. Get Distribution URL

2.1.4.9. Set an Endpoint Alias

2.1.4.10. Assign Tags

2.1.4.11. Remove Tags

2.1.4.12. Get Endpoint

2.1.4.13. Initiate Forensics Triage

2.1.5. Response Action

2.1.5.1. Restore File

2.1.5.2. File Retrieval Details

2.1.5.3. Allow List Files

2.1.5.4. Get Quarantine Status

2.1.5.5. Quarantine Files

2.1.5.6. Block List Files

2.1.5.7. Unisolate Endpoints

2.1.5.8. Cancel Scan Endpoints

2.1.5.9. Scan Endpoints

2.1.5.10. Get Action Status

2.1.5.11. Retrieve File

2.1.5.12. Isolate Endpoints

2.1.5.13. Get triage presets

2.1.6. Script execution

2.1.6.1. Run Snippet Code Script

2.1.6.2. Run Script

2.1.6.3. Get Script Metadata

2.1.6.4. Get Script Execution Status

2.1.6.5. Get Scripts

2.1.6.6. Get Script Execution Results

2.1.6.7. Get Script Execution Result Files

2.1.6.8. Get Script Code

2.1.7. Audit log

2.1.7.1. Get Audit Management Log

2.1.7.2. Get Audit Agent Report

2.1.8. System management

2.1.8.1. System Health Check

2.1.8.2. Get Tenant Info

2.1.8.3. Get Users

2.1.8.4. Get Roles

2.1.8.5. Get User Groups

2.1.8.6. Set a User Role

2.1.8.7. Get Risk Score

2.1.8.8. Get Risky Users

2.1.8.9. Get Risky Hosts

2.1.9. API keys

2.1.9.1. Get existing API keys

2.1.9.2. Generate an API key

2.1.9.3. Delete API keys

2.1.10. Attack surface management

2.1.10.1. Get External Service

2.1.10.2. Get All Services
2.1.10.3. Get vulnerability tests
2.1.10.4. Bulk Update Vulnerability Tests
2.1.11. XQL user datasets
2.1.11.1. Define an XQL user dataset
2.1.11.2. Get created XQL user datasets
2.1.11.3. Delete an XQL user dataset
2.1.12. Dataset Management
2.1.12.1. Add Dataset
2.1.12.2. Delete a dataset
2.1.12.3. Get all datasets
2.1.13. Lookup Datasets
2.1.13.1. Add or update data in a lookup dataset
2.1.13.2. Remove data from a lookup dataset
2.1.13.3. Get data from a lookup dataset
2.1.14. Authentication settings
2.1.14.1. Create authentication settings for IdP SSO or metadata URL
2.1.14.2. Update authentication settings
2.1.14.3. Delete authentication settings by domain
2.1.14.4. Get authentication settings for all configured domains
2.1.14.5. Get IdP metadata
2.1.15. Syslog servers
2.1.15.1. Create a syslog integration
2.1.15.2. Get all or filtered syslog servers
2.1.15.3. Update a syslog integration
2.1.15.4. Delete all or filtered syslog integrations
2.1.15.5. Test syslog integration
2.1.16. Scheduled Queries
2.1.16.1. Get scheduled queries
2.1.16.2. Insert or update scheduled queries
2.1.16.3. Delete a scheduled query
2.1.17. Query Library
2.1.17.1. Get XQL Queries
2.1.17.2. Insert or update XQL queries
2.1.17.3. Delete XQL Queries
2.1.18. IOCs
2.1.18.1. Get Indicators (IOCs)
2.1.18.2. Insert or update IOCs
2.1.18.3. Delete Indicators (IOCs)
2.1.19. BIOCs
2.1.19.1. Get BIOCs
2.1.19.2. Insert or update BIOCs
2.1.19.3. Delete BIOCs
2.1.20. Correlation Rules
2.1.20.1. Get Correlation Rules
2.1.20.2. Insert or update Correlation Rules
2.1.20.3. Delete Correlation Rules
2.1.21. Playbooks
2.1.21.1. Get a playbook
2.1.21.2. Insert or update playbooks
2.1.21.3. Delete a playbook
2.1.22. Scripts
2.1.22.1. Get a script
2.1.22.2. Insert or update a script
2.1.22.3. Delete a script
2.1.23. Dashboards
2.1.23.1. Get dashboards
2.1.23.2. Insert or update dashboards
2.1.23.3. Delete dashboards
2.1.24. Widgets
2.1.24.1. Get widgets
2.1.24.2. Insert or update widgets
2.1.24.3. Delete widgets
2.1.25. Issues
2.1.25.1. Create a new issue
2.1.25.2. Retrieve issues based on filters
2.1.25.3. Update existing issue
2.1.26. Cases
2.1.26.1. Retrieve cases based on filters
2.1.26.2. Update existing case
2.1.26.3. Retrieve Case Artifacts by Case Id
2.1.26.4. Get War Room entries
2.1.26.5. Add War Room entries
2.1.27. Asset inventory
2.1.27.1. Get all or filtered assets
2.1.27.2. Get asset by ID
2.1.27.3. Get raw fields of asset by ID
2.1.27.4. Get schema of asset inventory
2.1.27.5. Get enum values of specified field
2.1.28. Asset groups
2.1.28.1. Get all or filtered asset groups
2.1.28.2. Create an Asset Group
2.1.28.3. Update an Asset Group
2.1.28.4. Delete an Asset Group
2.2. Cortex Platform IAM APIs

2.2.1. Preface
2.2.1.1. Servers
2.2.1.2. Authentication methods
2.2.2. Roles
2.2.2.1. List all roles
2.2.2.2. Create a new role
2.2.2.3. Delete an existing role
2.2.2.4. List all permission configs
2.2.3. User Groups
2.2.3.1. List all user groups
2.2.3.2. Create a new user group
2.2.3.3. Edit an existing user group
2.2.3.4. Delete an existing user group
2.2.4. Scopes
2.2.4.1. Retrieve an existing scope
2.2.4.2. Edit an existing scope
2.2.5. User
2.2.5.1. List all users
2.2.5.2. Get user
2.2.5.3. Edit an existing user
2.2.6. API Keys
2.2.6.1. Get API Key
2.2.6.2. Edit an API key
2.3. Cloud Onboarding APIs
2.3.1. Preface
2.3.1.1. Servers
2.3.1.2. Authentication methods
2.3.2. Cloud integration instance management
2.3.2.1. Create a cloud onboarding integration template
2.3.2.2. Get integration instance details
2.3.2.3. Get all or filtered integration instances
2.3.2.4. Edit an integration instance
2.3.2.5. Enable or disable cloud integration instances
2.3.2.6. Delete the specified integration instances
2.3.2.7. Get identifier roles for integration instance
2.3.3. Cloud account management
2.3.3.1. Get accounts in the specified integration instances
2.3.3.2. Enable or disable cloud accounts
2.3.4. Outpost management
2.3.4.1. Create an outpost template
2.3.4.2. Edit an outpost
2.3.4.3. Get all or filtered outposts
2.3.5. General
2.3.5.1. Get available regions for a CSP
2.3.5.2. Get approved Azure tenants
2.4. ASPM, CICD and Application Security APIs
2.4.1. Preface
2.4.1.1. Servers
2.4.1.2. Authentication methods
2.4.1.2.1. Default authentication
2.4.2. Applications
2.4.2.1. Get an application configuration
2.4.2.2. Create an application
2.4.2.3. Get applications
2.4.2.4. Get an application
2.4.2.5. Update an application
2.4.2.6. Delete an application
2.4.3. Policies
2.4.3.1. Create an AppSec policy
2.4.3.1.1. Create an Application Security policy.
2.4.3.1.1.1. How to access Application Security policies in the UI:
2.4.3.1.1.2. Limitations
2.4.3.1.1.2.1. Trigger and Action Constraints by Finding Type
2.4.3.1.1.2.2. General Policy Constraints
2.4.3.2. List AppSec policies
2.4.3.2.1. List all or filtered Application Security policies.
2.4.3.2.1.1. How to access Application Security policies in the UI:
2.4.3.3. Get an AppSec policy
2.4.3.3.1. Get the details of the Application Security policy specified by its policy ID.
2.4.3.3.1.1. How to access Application Security policies in the UI:
2.4.3.4. Delete an AppSec policy
2.4.3.4.1. Delete an Application Security policy by specifying its policy ID.
2.4.3.4.1.1. How to access Application Security policies in the UI:
2.4.3.5. Update an AppSec policy
2.4.3.5.1. Update the details of the Application Security policy specified by its policy ID.
2.4.3.5.1.1. How to access Application Security policies in the UI:
2.4.3.6. Supported Condition Fields
2.4.3.7. Supported Scope Fields
2.4.3.8. Supported Condition Fields
2.4.3.9. Supported Scope Fields
2.4.4. Rules
2.4.4.1. Get AppSec rules
2.4.4.2. Create or clone an AppSec rule
2.4.4.3. Get an AppSec rule
2.4.4.4. Modify an AppSec rule
2.4.4.5. Delete an AppSec rule
2.4.4.6. Get AppSec rule labels

2.4.4.7. Create an AppSec rule validation
2.4.5. Repositories
2.4.5.1. Get repositories
2.4.5.2. Get a repository
2.4.5.3. Get a repository scan configuration
2.4.5.4. Update a repository scan configuration
2.4.5.5. Get AppSec repository branches
2.4.5.6. Update an AppSec repository branch
2.4.6. Data Sources
2.4.6.1. Get Data Sources
2.4.6.2. Get an AppSec Data Source
2.4.6.3. Update an AppSec Data Source
2.4.6.4. Delete an AppSec Data Source
2.4.7. Scan management
2.4.7.1. Get unscanned AppSec scan management repositories
2.4.7.2. Get AppSec branch periodic scans
2.4.7.3. Get AppSec Pull Request scans
2.4.7.4. Get AppSec CI scans
2.4.7.5. List AppSec scan issues
2.4.7.6. List AppSec scan findings
2.4.7.7. Rerun a repository scan
2.4.8. SBOM management
2.4.8.1. Get an SBOM for the specified repository
2.4.8.2. Get all SBOMs for the specified organization
2.4.9. Remediations
2.4.9.1. Get Fix Suggestion
2.4.9.2. Trigger Fix Pull Request
2.4.9.3. Get Fix Status
2.4.10. Criteria
2.4.10.1. Get all criteria
2.4.10.2. Get a Criteria by ID
2.4.10.3. Delete a Criteria
2.4.10.4. Create a Criteria
2.5. Vulnerability Intelligence public APIs
2.5.1. Preface
2.5.1.1. Servers
2.5.1.2. Authentication methods
2.5.2. Vulnerabilities
2.5.2.1. Get list of vulnerabilities
2.5.2.2. Get vulnerability details
2.5.3. Affected software
2.5.3.1. Get affected software
2.6. Vulnerability Management APIs
2.6.1. Preface
2.6.1.1. Servers
2.6.2. Vulnerability Management
2.6.2.1. Get Policies List
2.6.2.2. Create Policy Public
2.6.2.3. Update Policy
2.6.2.4. Get Policy By ID
2.6.2.5. Delete Policy
2.7. Cloud Infrastructure Entitlement Management APIs
2.7.1. Preface
2.7.1.1. Servers
2.7.2. CIEM access
2.7.2.1. Search CIEM Access
2.8. Compliance Controls API
2.8.1. Preface
2.8.1.1. Servers
2.8.1.2. Authentication methods
2.8.1.2.1. Default authentication
2.8.2. Assessment profiles
2.8.2.1. Get assessment profiles
2.8.2.2. Get assessment profile by ID
2.8.2.3. Add assessment profile
2.8.2.4. Edit assessment profile
2.8.2.5. Delete assessment profile
2.8.3. Assessment results
2.8.3.1. Get assessment profile results
2.8.4. Controls
2.8.4.1. Get compliance controls
2.8.4.2. Get compliance control by ID
2.8.4.3. Add new control
2.8.4.4. Edit existing control
2.8.4.5. Delete control
2.8.4.6. Get control by revision
2.8.5. Rules
2.8.5.1. Add compliance rules to a compliance control
2.8.5.2. Delete rules from control
2.8.6. Categories
2.8.6.1. Get categories and subcategories
2.8.7. Results
2.8.7.1. Get control failed results

2.8.7.2. Get rule failed results
2.8.8. Reports
2.8.8.1. Get compliance reports
2.8.9. Standards
2.8.9.1. Get compliance standards
2.8.9.2. Get single standard by ID
2.8.9.3. Add new standard
2.8.9.4. Edit existing standard
2.8.9.5. Delete standard
2.8.10. Compliance Assets
2.8.10.1. Get compliance assets
2.9. Cloud Workload Protection APIs
2.9.1. Preface
2.9.1.1. Servers
2.9.1.2. Authentication methods
2.9.2. SBOM
2.9.2.1. Get the SBOM of the specified asset
2.9.3. Policies
2.9.3.1. Get CWP policies
2.9.3.2. Add CWP policies
2.9.3.3. Get a CWP policy by ID
2.9.3.4. Delete a CWP policy by ID
2.9.3.5. Update a CWP policy by ID
2.10. Data Security Posture Management APIs
2.10.1. Preface
2.10.1.1. Servers
2.10.2. Inventory
2.10.2.1. Get data pattern inventory
2.10.2.2. Get field inventory details
2.10.2.3. Get file inventory details
2.11. Trusted Images APIs
2.11.1. Preface
2.11.1.1. Servers
2.11.2. Rules Management Service
2.11.2.1. Delete Policies
2.11.2.2. Get Policies
2.11.2.3. Add Policies
2.11.2.4. Update Policies
2.11.2.5. Disable Policies
2.11.2.6. Enable Policies
2.11.2.7. Get a Policy by ID
2.11.2.8. Add (custom) Compliance rules
2.11.2.9. Delete (custom) CWP Rules
2.11.2.10. Get rules by IDs
2.11.2.11. Get the Compliance Rules
2.11.2.12. Get a rule by ID
2.11.2.13. rego script linter
2.11.2.14. Update (custom) Compliance rules
2.11.2.15. Get Policies
2.11.2.16. Add Policies
2.11.2.17. Update Policies
2.11.2.18. Get a Policy by ID
2.11.2.19. Get Policies.
2.11.2.20. Add Policy.
2.11.2.21. Delete a Policy by ID
2.11.2.22. Get a Policy by ID.
2.11.2.23. Update a Policy by ID.

1 | Cortex Cloud APIs

1.1 | Understand Cortex Cloud licenses

Version 1.3

This Cortex Cloud 1.3 API documentation is relevant for Cortex Cloud Runtime Security and Cortex Cloud Posture Management. It is important to check the license requirements for each API. Each API includes information on which licenses and add-ons are necessary to run the API.

To read more about Cortex Cloud licensing, see Understand license plans in Cloud Runtime Security and Understand license plans in Cloud Posture Management.

1.2 | Get started with Cortex Cloud 1.3 APIs

Before you can begin using Cortex Cloud APIs, you must generate the following items in Cortex Cloud:

Value	Description
API Key	The API Key is your unique identifier used as the `Authorization:{key}` header required for authenticating API calls. Depending on your desired security level, you can generate two types of API keys, Advanced or Standard, from Cortex.
API Key ID	The API Key ID is your unique token used to authenticate the API Key. The header used when running an API call is `x-xdr-auth-id:{key_id}`.
FQDN	The FQDN is a unique host and domain name associated with each tenant.

Cortex Cloud API URIs are made up of your tenant's FQDN, the API name, and endpoint path. For example, `https://api-{fqdn}/xsiam/public/v1/{endpoint_path}/.`

The following steps describe how to generate the necessary key values and run your first API call:

1. Create a new API key.
2. Get your Cortex Cloud API key ID.
3. Get your FQDN.
4. Make your first API call.

1.3 | Create a new API key

1. In Cortex, navigate to Settings > Configurations > Integrations > API Keys > New Key.
2. In the Role tab, perform the following:
    1. Under Security Level, select the type of API Key you want to generate: Advanced or Standard. The Advanced API key hashes the key using a nonce, a random string, and a timestamp to prevent replay attacks. cURL does not support this but it is suitable with scripts.
    2. Under Role, select the desired level of access for this key. You can select from predefined roles or custom roles. Roles are available according to what was defined in either the Cortex Gateway or Cortex XSIAM Access Management. You can view the configuration of the role selected by expanding the sections under Components. For more information, see Assign user roles and groups.
    3. (Optional) Under Comment, provide a comment that describes the purpose of the API key.
    4. (Optional) If you want to define a time limit on the API key authentication, select Enable Expiration Date, and select the expiration date and time. You can track the expiration date of each API key in the API Keys page. In addition, Cortex Cloud displays a API Key Expiration notification in the Notification Center one week and one day prior to the defined expiration date.
3. (Optional) To configure and manage granular scoping for Scope-Based Access Control (SBAC), click the Scope tab, and under Scope Definition, expand the scoping areas that you want to grant the user role access to for this API by clicking the chevron icon (>) beside the scoping area title. For more information, see Manage API keys.
4. Generate the API Key.
5. Copy the API key, and then click Close. This value represents your unique `Authorization:{key}`.

    warning

    You will not be able to view the API Key again after you complete this step. Ensure that you copy it before closing the notification.

1.4 | Get your Cortex API key ID

1. In the API Keys table, locate the ID field.
2. Note your corresponding ID number. This value represents the `x-xdr-auth-id:{key_id}` token.

1.5 | Get your FQDN

1. In Cortex Cloud, navigate to Settings > Configurations > Integrations > API Keys.
2. Click Copy API URL.
3. Your FQDN is saved in the clipboard. You must use the FQDN as part of the URL in every API call. For example: `https://api-company.us.com/xsoar/public/v1/{endpoint_path}/`.

You can use the CURL Example URL to run the APIs.

1.6 | Make your first API call

The following examples vary depending on the type of key you select.

You can test authentication with Advanced API keys using the provided Python 3 example. With Standard API keys, use either the cURL example or the Python 3 example. Don't forget to replace the example variables with your unique API key, API key ID, and FQDN tenant. After you verify authentication, you can begin making API calls.

Standard Key cURL Example

```
curl -X POST https://api-{fqdn}/public_api/v1/{name of api}/{name of call}/
-H "x-xdr-auth-id:{key_id}"
-H "Authorization:{key}"
-H "Content-Type:application/json"
-d '{}'
```

Standard Key Python 3 Example

```
import requests
    def test_standard_authentication(api_key_id, api_key):
    headers = {
        "x-xdr-auth-id": str(api_key_id),
        "Authorization": api_key
    }
    parameters = {}
    res = requests.post(url="https://api-{fqdn}/public_api/v1/{name of api}/{name of call}",
                                            headers=headers,
                                            json=parameters)
    return res
```

Advanced Key Python 3 Example

```
import requests

from datetime import datetime, timezone
import secrets
import string
import hashlib
import requests

def test_advanced_authentication(api_key_id, api_key):
   # Generate a 64 bytes random string
    nonce = "".join([secrets.choice(string.ascii_letters + string.digits) for _ in range(64)])
    # Get the current timestamp as milliseconds.
    timestamp = int(datetime.now(timezone.utc).timestamp()) * 1000
    # Generate the auth key:
    auth_key = "%s%s%s" % (api_key, nonce, timestamp)
    # Convert to bytes object
    auth_key = auth_key.encode("utf-8")
    # Calculate sha256:
    api_key_hash = hashlib.sha256(auth_key).hexdigest()
    # Generate HTTP call headers
    headers = {
        "x-xdr-timestamp": str(timestamp),
        "x-xdr-nonce": nonce,
        "x-xdr-auth-id": str(api_key_id),
        "Authorization": api_key_hash
    }
    parameters = {}
    res = requests.post(url="https://api-{fqdn}/public_api/v1/{name of api}/{name of call}",
                                            headers=headers,
                                            json=parameters)
    return res
```

1.7 | Run XQL query APIs

Cortex Cloud enables you to run XQL queries on your data sources using a series of APIs. To execute XQL APIs you must have:

- Valid API Key and API Key ID that include the Instance Administrator role permissions.
- Available query quota.

Query quota is made up of query units that enable you to run XQL APIs. Each XQL API query entails a cost of query units calculated according to the complexity and number of search results. The query cost for each API query is displayed in the **Get Query Results** API. You can also track the query cost per XQL API search, overall usage, and remaining quota in Cortex Cloud or by running a **Get XQL Query Quota** API. Cortex Cloud provides a free daily quota relative to your license size for you to run XQL API queries.

info

Note: You will be able to purchase additional query units in future Cortex Cloud versions.

To execute a XQL API, you need to run a series of APIs. Each API requires a response value from the previous API to continue. This allows you to track the number of XQL queries you want to run, which in turn helps you manage your daily quota. Queries called without enough quota will fail. To ensure you don't surpass your quota, Cortex Cloud allows you to run up to four API queries in parallel.

Run the following APIs to call an XQL query:

1. **Start an XQL Query**—Run an XQL query. Response returns a unique execution ID used to retrieve the results by the **Get XQL Query Results** API.
2. **Get XQL Query Results**—Retrieve XQL query results. API displays up to 1,000 results. If query generated more than 1,000 results, the response returns a unique stream ID used to retrieve additional results by the **Get XQL Query Results Stream** API.
3. **Get XQL Query Results Stream**—Retrieve XQL query with more than 1,000 results.

1.8 | Whats-new-in-this-release

What's new

Cortex Cloud version 1.3 includes the following new APIs.

Product Name	API Details

Application Security	Remediation APIs
	`/public_api/appsec/v1/issues/fix/trigger_fix_pull_request`
	`/public_api/appsec/v1/issues/fix/{issueId}/fix_suggestion`
	`/public_api/appsec/v1/issues/fix/{remediationId}`
	
	Criteria APIs
	`/public_api/appsec/v1/application/criteria`
	`/public_api/appsec/v1/application/criteria/all`
	`/public_api/appsec/v1/application/criteria/{criteriaId}`

Cloud Onboarding	`/public_api/v1/cloud_onboarding/get_identifiers`
Compliance	`/public_api/v1/compliance/get_assets`
Cortex Cloud Platform	Identity and Access Management (IAM) APIs `/platform/iam/v1/role` (GET, POST) `/platform/iam/v1/role/{role_id}` (DELETE) `/platform/iam/v1/role/permission-config` (GET) `/platform/iam/v1/user-group` (GET, POST) `/platform/iam/v1/user-group/{group_id}` (PATCH, DELETE) `/platform/iam/v1/scope/{entity_type}/{entity_id}` (GET, PUT) `/platform/iam/v1/user` (GET) `/platform/iam/v1/user/{user_email}` (GET, PATCH) `/platform/iam/v1/api-key/{api_key_id}` (GET, PUT)
Cloud CIEM	No new APIs in this release.
Compute (CWP)	No new APIs in this release.
DSPM	No new APIs in this release.
UVEM	No new APIs in this release.
Vulnerability	No new APIs in this release.

What's changed with version 1.3

Cortex Cloud version 1.3 includes the following updates.

Product Name	API And Schema Changes
Application Security	Schema Additions - ApplicationMetadataCloud - AppsecPolicyTriggersandActions - CloudCriteriaConfig - CreateCriteriaRequest - CreateCriteriaResponse - CriteriaCreationMethod - CriteriaDTOCloud - CriteriaDefinition - CriteriaDefinitionType - CriteriaMetadata - CriteriaType - CriteriaType.Cloud - FixStatus - GetIssueFixSuggestionResponse - PaginationResponse_CriteriaDTO_ - PolicyCondition - PolicyScope - RefreshStatus - SaveFixResult - SectionProvider - TriggerFixPrPayload - TriggerFixPrResponse - TriggeredPr Schema Updates - ConditionOperators: Updated enum `WILDCARD_NOT` → `WILDCARDNOT` - CortexCondition: `SEARCH_FIELD`, `SEARCH_TYPE`, `SEARCH_VALUE` made required - CreateRequest: Removed `enabled`, `overrideIssueSeverity`; added `assetGroupIds` - ExtendedFields: Properties and required fields updated - PolicyConditions & PolicyScope: Updated with available fields - PolicyTriggers: Updated with `details` property
Cloud Onboarding	Schema Additions - GetIdentifiersRequestData - GetIdentifiersResponse Schema Updates - InstanceAuditLogsConfig: Added new properties and updated required fields - InstanceAdditionalCapabilities: Added new properties and updated required fields - CreateInstanceTemplateRequestData: Added enum `OCI` to `cloud_provider`, added `scan_env_id`, updated required fields - EditInstanceRequestData, CreateOutpostTemplateRequestData, ListCloudProviderRegionsRequestData, ListCloudProviderRegionsResponse: Added enum `OCI` to `cloud_provider`
Cortex Cloud Platform	- Endpoint category renamed to Endpoint Management. - Issues endpoints updated to include `/public_api/` prefix for consistency with other API endpoints
Cloud CIEM	No updates in this release.
Compliance	Schema Additions - AssessmentProfileSortObject - AssessmentResultFilterObject - AssetFilterObject - AssetSortObject - AssetsResponse - ComplianceAsset - GetAssetsRequest Schemas Removed - FilterObject - SortObject
Compute (CWP)	No updates in this release.
DSPM	No updates in this release.
UVEM	No updates in this release.

Product Name	API And Schema Changes

Vulnerability	Added pagination properties to the following POST requests:
	`/public_api/uvem/v1/get_vulnerabilities`
	`/public_api/uvem/v1/get_affected_software`

2 | APIs

2.1 | Cortex Cloud Platform APIs

2.1.1 | Preface

You can export the OpenAPI specs in the Cortex Stoplight instance.

Using the Cortex Platform APIs, you can configure and manage authentication settings, datasets, cases, issues, and script executions. Additionally, the APIs allow you to run XQL queries and perform various other operations across the platform.

The license requirements for each API are listed individually.

Version: Cortex Cloud 1.3

2.1.1.1 | Servers

https://api-yourfqdn

2.1.2 | Cortex CLI

APIs for managing the Cortex CLI

2.1.2.1 | Get the latest version of the Cortex CLI

get /public_api/v1/cli/releases/version

Get the latest version of Cortex CLI

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/cli/releases/version'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE" } conn.request("GET",
"/public_api/v1/cli/releases/version", headers=headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cli/releases/version") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' response = http.request(request) puts
response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/v1/cli/releases/version");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/v1/cli/releases/version")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE" ] let
request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/cli/releases/version")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields =
headers let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cli/releases/version", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/v1/cli/releases/version"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cli/releases/version"); var request = new
RestRequest(Method.GET); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); IRestResponse response = client.Execute(request);
```

Responses

Success

Body
application/json
▼ versionstring

Cortex CLI version

Example:`"v1.2.3"`
RESPONSE

```
{ "version": "v1.2.3" }
```

Service had unexpected internal error

2.1.3 | XQL query

Run XQL queries on your data sources using a series of APIs.

2.1.3.1 | Start an XQL query

post /public_api/v1/xql/start_xql_query

Execute an XQL query.

For more information on how to run XQL queries, see *Running XQL query APIs*.

> Note
>
> To ensure you don't surpass your quota, Cortex Cloud allows you to run up to four API queries in parallel.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/xql/start_xql_query'
```

```
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"query\":\"dataset=xdr_data | fields event_id, event_type, event_sub_type | limit 3\",\"tenants\":[],\"timeframe\":
{\"from\":1598907600000,\"to\":1599080399000}}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/xql/start_xql_query",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/xql/start_xql_query") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"query\":\"dataset=xdr_data | fields event_id, event_type, event_sub_type | limit
3\",\"tenants\":[],\"timeframe\":{\"from\":1598907600000,\"to\":1599080399000}}}" response = http.request(request)
puts response.read_body
const data = JSON.stringify({ "request_data": { "query": "dataset=xdr_data | fields event_id, event_type,
event_sub_type | limit 3", "tenants": [], "timeframe": { "from": 1598907600000, "to": 1599080399000 } } }); const xhr
= new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/xql/start_xql_query"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/xql/start_xql_query")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"query\":\"dataset=xdr_data | fields event_id, event_type,
event_sub_type | limit 3\",\"tenants\":[],\"timeframe\":{\"from\":1598907600000,\"to\":1599080399000}}}")
.asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "query": "dataset=xdr_data | fields
event_id, event_type, event_sub_type | limit 3", "tenants": [], "timeframe": [ "from": 1598907600000, "to":
1599080399000 ] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: [])
let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/xql/start_xql_query")! as
URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST"
request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/xql/start_xql_query", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"query\":\"dataset=xdr_data | fields event_id, event_type,
event_sub_type | limit 3\",\"tenants\":[],\"timeframe\":{\"from\":1598907600000,\"to\":1599080399000}}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/xql/start_xql_query"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"query\":\"dataset=xdr_data | fields event_id, event_type, event_sub_type | limit 3\",\"tenants\":[],\"timeframe\":
{\"from\":1598907600000,\"to\":1599080399000}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/xql/start_xql_query"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"query\":\"dataset=xdr_data | fields event_id, event_type, event_sub_type | limit
3\",\"tenants\":[],\"timeframe\":{\"from\":1598907600000,\"to\":1599080399000}}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

▶ querystring

String of the XQL query.

▶ tenantsarray[string]

Note: This is only used when querying tenants managed by Managed Security Services Providers (MSSP).

List of strings used for running APIs on local and Managed Security tenants. Valid values:

- For single tenant (local tenant) query, enter a single-item list with your tenant_id. Additional valid values are, empty list ([]) or null (default).
- For multi-tenant investigations (Managed Security parent who investigate children and/or local), enter multi-item list with the required tenant_id. List of IDs can contain the parent, children, or both parent and children.

▶ timeframeobject

Integer in timestamp epoch milliseconds. Valid values include:

- Absolute Unix timestamp representing a date period: {"from" : 1598907600000, "to" : 1599080399000} = date period: 31/08/20 09:00:00 PM UTC - 02/09/20 8:59:59 PM UTC
- Relative Unix timestamp representing the last 24 hours: {"relativeTime": 86400000} = (24*60*60*1000 = 86400000).

REQUEST default

```
{ "request_data": { "query": "dataset=xdr_data | fields event_id, event_type, event_sub_type | limit 3", "tenants": [], "timeframe": { "from": 1598907600000, "to": 1599080399000 } } }
{ "request_data": { "query": "dataset=xdr_data | fields event_id, event_type, event_sub_type | limit 3", "tenants": [ "431509831", "401387390" ], "timeframe": { "from": 1598907600000, "to": 1599080399000 } } }
```

Responses

Successful response

Body
application/json
▼ replystring
RESPONSE Example 1

```
{ "reply": "ad21c1e1492d4c_667_inv" }
```

Bad Request. Invalid JSON.

Body
application/json
▼ replyobject
▶ err_codeinteger
▶ err_msgstring
▶ err_extraobject
RESPONSE Example 1

```
{ "reply": { "err_code": 500, "err_msg": "An error occurred while processing XDR - XQL query", "err_extra": { "err_msg": "reached max allowed amount of parallel running queries. please wait for some queries to finish and submit your query again", "query_cost": 0, "remaining_quota": 5, "total_daily_running_queries": 4, "total_daily_concurrent_rejected_queries": 1 } } }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json
▼ replyobject
▶ err_codeinteger
▶ err_msgstring
▶ err_extraobject
RESPONSE Example 1

```
{ "reply": { "err_code": 500, "err_msg": "An error occurred while processing XDR - XQL query", "err_extra": { "err_msg": "reached max allowed amount of parallel running queries. please wait for some queries to finish and submit your query again", "query_cost": 0, "remaining_quota": 5, "total_daily_running_queries": 4, "total_daily_concurrent_rejected_queries": 1 } } }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json
▼ replyobject
▶ err_codeinteger
▶ err_msgstring
▶ err_extraobject
RESPONSE Example 1

```
{ "reply": { "err_code": 500, "err_msg": "An error occurred while processing XDR - XQL query", "err_extra": { "err_msg": "reached max allowed amount of parallel running queries. please wait for some queries to finish and submit your query again", "query_cost": 0, "remaining_quota": 5, "total_daily_running_queries": 4, "total_daily_concurrent_rejected_queries": 1 } } }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json
▼ replyobject

▶ err_codeinteger
▶ err_msgstring
▶ err_extraobject
RESPONSE Example 1

```
{ "reply": { "err_code": 500, "err_msg": "An error occurred while processing XDR - XQL query", "err_extra": {
"err_msg": "reached max allowed amount of parallel running queries. please wait for some queries to finish and submit
your query again", "query_cost": 0, "remaining_quota": 5, "total_daily_running_queries": 4,
"total_daily_concurrent_rejected_queries": 1 } } }
```


Internal server error. A unified status for API communication type errors.

Body
application/json
▼ replyobject
▶ err_codeinteger
▶ err_msgstring
▶ err_extraobject
RESPONSE Example 1

```
{ "reply": { "err_code": 500, "err_msg": "An error occurred while processing XDR - XQL query", "err_extra": {
"err_msg": "reached max allowed amount of parallel running queries. please wait for some queries to finish and submit
your query again", "query_cost": 0, "remaining_quota": 5, "total_daily_running_queries": 4,
"total_daily_concurrent_rejected_queries": 1 } } }
```

2.1.3.2 | Get XQL query results

post /public_api/v1/xql/get_query_results

Retrieve results of an executed XQL query API.

Note: This endpoint only works on XQL queries initiated by `/public_api/v1/xql/start_xql_query/`.

Maximum result set size is 1000. The API does not support pagination, therefore, you can set values to determine the result size limitation and how to wait for the results. To view response with greater than 1000 results you must call Get XQL query results Stream.

For more information on how to run XQL queries, see *Running XQL query APIs*.

> Note
>
> To ensure you don't surpass your quota, Cortex Cloud allows you to run up to four API queries in parallel.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/xql/get_query_results'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"query_id\":\"061880b4867446_4356_inv\",\"pending_flag\":true,\"limit\":100,\"format\":\"json\"}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/xql/get_query_results", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/xql/get_query_results") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":
```

```
{\"query_id\":\"061880b4867446_4356_inv\",\"pending_flag\":true,\"limit\":100,\"format\":\"json\"}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "query_id": "061880b4867446_4356_inv", "pending_flag": true, "limit":
100, "format": "json" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/xql/get_query_results");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/xql/get_query_results")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":
{\"query_id\":\"061880b4867446_4356_inv\",\"pending_flag\":true,\"limit\":100,\"format\":\"json\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "query_id": "061880b4867446_4356_inv",
"pending_flag": true, "limit": 100, "format": "json" ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/xql/get_query_results")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/xql/get_query_results", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"query_id\":\"061880b4867446_4356_inv\",\"pending_flag\":true,\"limit\":100,\"format\":\"json\"}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/xql/get_query_results"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-
id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"query_id\":\"061880b4867446_4356_inv\",\"pending_flag\":true,\"limit\":100,\"format\":\"json\"}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/xql/get_query_results"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":
{\"query_id\":\"061880b4867446_4356_inv\",\"pending_flag\":true,\"limit\":100,\"format\":\"json\"}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

▶ query_idstring

String representing the unique execution ID generated by the response to Start an XQL query API. You can also enter the execution ID of a query generated in Cortex XDR and listed in the Query Center table.

▶ pending_flagboolean

Boolean flag indicating whether the API call should operate in synchronous/blocking mode, or in asynchronous/non-blocking mode. Valid Values:

- True (default): The call returns immediately with one of the following options:

1. PENDING status indicating query hasn't yet completed or results are not yet ready to be returned. Need to execute the API call again.
2. SUCCESS/FAIL status

- False: The API will block until query completes and results are ready to be returned.

▶ limitinteger

Integer representing the maximum number of results to return. If the 'limit' is not specified or if 'limit' is greater than 1000 and the query yields more than 1000 valid results, a `stream id` will be generated for use in the *Get XQL query results Stream** API. In the context of multi-tenant investigations, when you specify the parameter value (x), it will return x results across all tenants combined, rather than x results for each individual tenant. For example, if there are y tenants participating in the investigation, the maximum number of results returned can be x*y (up to the limit of 1,000,000).

▶ formatstring (Enum)

The type of response output.

REQUEST default

```
{ "request_data": { "query_id": "061880b4867446_4356_inv", "pending_flag": true, "limit": 100, "format": "json" } }
```


```
{ "request_data": { "query_id": "061880b4867446_4356_inv", "pending_flag": true, "limit": 100, "format": "json" } }
```

Responses

Successful response

Body
application/json
▼ replyobject
▶ statusstring
▶ number_of_resultsinteger
▶ query_costobject
▶ remaining_quotanumber
▶ resultsobject
RESPONSE pending_flag=true

```
{ "reply": { "status": "PENDING" } }
```
```
{ "reply": { "status": "SUCCESS", "number_of_results": 3, "query_cost": { "tenant_id_1": 0.001596388888888889 },
"remaining_quota": 4.998403611111111, "results": { "data": [ { "event_id": "eventID1", "_vendor": "PANW", "_product":
"Fusion", "insert_timestamp": 1621541825324, "_time": 1621541523000, "event_type": "STORY", "event_sub_type": "NULL"
}, { "event_id": "eventID2", "_vendor": "PANW", "_product": "Fusion", "insert_timestamp": 1621541825326, "_time":
1621541528000, "event_type": "STORY", "event_sub_type": "NULL" }, { "event_id": "eventID3", "_vendor": "PANW",
"_product": "Fusion", "insert_timestamp": 1621541825325, "_time": 1621541517000, "event_type": "STORY",
"event_sub_type": "NULL" } ] } } }
```
```
{ "reply": { "status": "SUCCESS", "number_of_results": 6, "query_cost": { "tenant_id_1": 0.001596388888888889,
"tenant_id_2": 0.00179989 }, "remaining_quota": 4.995007332222222, "results": { "data": [ { "event_id": "eventID1",
"_vendor": "PANW", "_product": "Fusion", "insert_timestamp": 1621541825324, "_time": 1621541523000, "event_type":
"STORY", "event_sub_type": "NULL", "tenant": "1723879655" }, { "event_id": "eventID2", "_vendor": "PANW", "_product":
"Fusion", "insert_timestamp": 1621541825326, "_time": 1621541528000, "event_type": "STORY", "event_sub_type": "NULL",
"tenant": "1723879655" }, { "event_id": "eventID3", "_vendor": "PANW", "_product": "Fusion", "insert_timestamp":
1621541825325, "_time": 1621541517000, "event_type": "STORY", "event_sub_type": "NULL", "tenant": "1723879655" }, {
"event_id": "eventID4", "_vendor": "PANW", "_product": "Fusion", "insert_timestamp": 1621541825324, "_time":
1621541523000, "event_type": "STORY", "event_sub_type": "NULL", "tenant": "1705396706" }, { "event_id": "eventID5",
"_vendor": "PANW", "_product": "Fusion", "insert_timestamp": 1621541825326, "_time": 1621541528000, "event_type":
"STORY", "event_sub_type": "NULL", "tenant": "1705396706" }, { "event_id": "eventID6", "_vendor": "PANW", "_product":
"Fusion", "insert_timestamp": 1621541825325, "_time": 1621541517000, "event_type": "STORY", "event_sub_type": "NULL",
"tenant": "1705396706" } ] } } }
```
```
{ "reply": { "status": "SUCCESS", "number_of_results": 1000000, "query_cost": { "tenant_id_1": 0.011742777777777777
}, "remaining_quota": 4.984442777777778, "results": { "stream_id": "streamID" } } }
```


Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.3.3 | Get XQL query Quota

post /public_api/v1/xql/get_quota

Retrieve the amount of query quota available and used.

Note: This endpoint only works on XQL queries initiated by `/public_api/v1/xql/start_xql_query/`.

For more information on how to run XQL queries, see *Running XQL query APIs*.

> Note
>
> To ensure you don't surpass your quota, Cortex Cloud allows you to run up to four API queries in parallel.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/xql/get_quota'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/xql/get_quota", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/xql/get_quota") http
= Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": {} }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/xql/get_quota");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/xql/get_quota")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": []] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/xql/get_quota")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/xql/get_quota", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-
type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/xql/get_quota"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/xql/get_quota"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

Object

application/json

REQUEST default

```
{ "request_data": {} }
```

Responses

Successful response

Body

application/json

▼ replyobject

▶ license_quotainteger

▶ additional_purchased_quotainteger

▶ used_quotanumber

▶ eval_quotainteger

▶ total_daily_running_queriesinteger

The number of daily active queries. This value is reset nightly.

▶ total_daily_concurrent_rejected_queriesinteger

The number of daily queries rejected due to too many concurrent XQL queries being run through the API. This value is reset nightly.

▶ current_concurrent_active_queriesobject

Currently running XQL queries with their current duration.

▶ current_concurrent_active_queries_countinteger

The number of active queries currently running.

▶ max_daily_concurrent_active_query_countinteger

The maximum number of queries that ran concurrently today on this tenant. This value is reset nightly.

RESPONSE example-1

```
{ "reply": { "license_quota": 5, "additional_purchased_quota": 0, "used_quota": 0, "eval_quota": 0,
"total_daily_running_queries": 4, "total_daily_concurrent_rejected_queries": 8, "current_concurrent_active_queries":
{ "debee6b0c41f47_911_inv": { "xql": "config timeframe = 1mo | dataset=xdr_data | limit 1000000", "duration": 61 } },
"current_concurrent_active_queries_count": 1, "max_daily_concurrent_active_query_count": 4 } }
```

Bad Request. Got an invalid JSON.

Body

application/json

▼ replyobject

▶ err_codeinteger

▶ err_msgstring

▶ err_extraobject

RESPONSE Example 1

```
{ "reply": { "err_code": 500, "err_msg": "An error occurred while processing XDR - XQL query", "err_extra": {
"err_msg": "reached max allowed amount of parallel running queries. please wait for some queries to finish and submit
your query again", "query_cost": 0, "remaining_quota": 5, "total_daily_running_queries": 4,
"total_daily_concurrent_rejected_queries": 1 } } }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.
▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.3.4 | Get XQL query results Stream

post /public_api/v1/xql/get_query_results_stream

Retrieve XQL query results with more than 1000 results.

Note: This endpoint only works on XQL queries initiated by `/public_api/v1/xql/start_xql_query/`.

Response is returned as chunked (Transfer-Encoding: chunked). To retrieve a compressed gzipped response (Content-Encoding: gzip), in your header add Accept-Encoding: gzip.

For more information on how to run XQL queries, see *Running XQL query APIs*.

> Note
>
> To ensure you don't surpass your quota, Cortex Cloud allows you to run up to four API queries in parallel.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
▼ Accept-Encoding String

For retrieving a compressed gzipped response

Example: `acceptEncoding_example`
Default: `gzip`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example' -H 'Accept-Encoding: acceptEncoding_example'
'https://api-yourfqdn/public_api/v1/xql/get_query_results_stream'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\": {\"stream_id\":\"563c5e24-====-9a1f8139d3c5\",\"is_gzip_compressed\":true}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'Accept-Encoding': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/xql/get_query_results_stream", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/xql/get_query_results_stream") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["Accept-Encoding"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"stream_id\":\"563c5e24-===-9a1f8139d3c5\",\"is_gzip_compressed\":true}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "stream_id": "563c5e24-===-9a1f8139d3c5", "is_gzip_compressed": true } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/xql/get_query_results_stream"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("Accept-Encoding", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/xql/get_query_results_stream") .header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("Accept-Encoding", "SOME_STRING_VALUE") .header("content-type", "application/json") .body("{\"request_data\": {\"stream_id\":\"563c5e24-===-9a1f8139d3c5\",\"is_gzip_compressed\":true}}") .asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE", "Accept-Encoding": "SOME_STRING_VALUE", "content-type": "application/json" ] let parameters = ["request_data": [ "stream_id": "563c5e24-===-9a1f8139d3c5", "is_gzip_compressed": true ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/xql/get_query_results_stream")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/xql/get_query_results_stream", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"stream_id\":\"563c5e24-===-9a1f8139d3c5\",\"is_gzip_compressed\":true}}", CURLOPT_HTTPHEADER => [ "Accept-Encoding: SOME_STRING_VALUE", "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd, CURLOPT_URL, "https://api-yourfqdn/public_api/v1/xql/get_query_results_stream"); struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "Accept-Encoding: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"stream_id\":\"563c5e24-===-9a1f8139d3c5\",\"is_gzip_compressed\":true}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/xql/get_query_results_stream"); var request = new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); request.AddHeader("Accept-Encoding", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{\"request_data\":{\"stream_id\":\"563c5e24-===-9a1f8139d3c5\",\"is_gzip_compressed\":true}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

▶ stream_idstring

String representing the unique ID generate by the response to Get XQL query results API.

▶ is_gzip_compressedboolean

A boolean flag.

REQUEST default

```
{ "request_data": { "stream_id": "563c5e24-===-9a1f8139d3c5", "is_gzip_compressed": true } }
```

Responses

Successful response

Bad Request. Got an invalid JSON.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.4 | Endpoint Management

APIs for managing endpoints

2.1.4.1 | Get Distribution version

post /public_api/v1/distributions/get_versions

Get a list of all the agent versions to use for creating a distribution list.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/distributions/get_versions'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/distributions/get_versions", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/distributions/get_versions") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{}" response = http.request(request) puts response.read_body
const data = JSON.stringify({}); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/distributions/get_versions"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/distributions/get_versions") .header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type", "application/json") .body("{}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE", "content-type": "application/json" ] let parameters = [] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/distributions/get_versions")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/distributions/get_versions", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
```

```
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type:
application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/distributions/get_versions"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/distributions/get_versions"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

Object

application/json

REQUEST

```
{}
```

Responses

Successful response

Body

application/json

▼ replyobject

▶ windowsarray[string]

List of Windows agent versions.

▶ linuxarray[string]

List of Linux agent versions.

▶ macosarray[string]

List of Mac agent versions.

RESPONSE

```
{ "reply": { "windows": [ "example" ], "linux": [ "example" ], "macos": [ "example" ] } }
```

Bad Request. Got an invalid JSON.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.4.2 | Get all Endpoints

post /public_api/v1/endpoints/get_endpoints

Gets a list of all of your endpoints. The response is concatenated using AND condition (OR is not supported).

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/endpoints/get_endpoints'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/endpoints/get_endpoints", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/endpoints/get_endpoints") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{}" response = http.request(request) puts response.read_body
const data = JSON.stringify({}); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/endpoints/get_endpoints"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/endpoints/get_endpoints") .header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type", "application/json") .body("{}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE", "content-type": "application/json" ] let parameters = [] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/endpoints/get_endpoints")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/endpoints/get_endpoints", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd, CURLOPT_URL, "https://api-yourfqdn/public_api/v1/endpoints/get_endpoints"); struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/endpoints/get_endpoints"); var request = new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
```

```
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
Object
application/json
REQUEST

```
{}
```

Responses

Successful response

Body
application/json
▼ replyarray
[
▶ agent_idstring
▶ agent_statusstring
▶ operational_statusstring
▶ host_namestring
▶ agent_typestring
▶ iparray[string]
▶ last_seeninteger
▶ tagsobject
▶ usersarray[string]
]
RESPONSE example-1

```
{ "reply": [ { "agent_id": "<agent_id>", "agent_status": "DISCONNECTED", "operational_status": "PROTECTED",
"host_name": "<hostname>", "agent_type": "Workstation", "ip": [ "<ip_address>" ], "last_seen": 1678012587521, "tags":
{ "server_tags": [], "endpoint_tags": [] }, "users": [ "user" ] } ] }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.4.3 | Get Policy

post /public_api/v1/endpoints/get_policy

Get the policy name for a specific endpoint.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/endpoints/get_policy'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"endpoint_id\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/endpoints/get_policy",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/endpoints/get_policy") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"endpoint_id\":\"string\"}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "endpoint_id": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/endpoints/get_policy");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/endpoints/get_policy")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"endpoint_id\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["endpoint_id": "string"]] as [String : Any]
let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/endpoints/get_policy")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/endpoints/get_policy", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"endpoint_id\":\"string\"}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/endpoints/get_policy"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"endpoint_id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/endpoints/get_policy"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"endpoint_id\":\"string\"}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```
Body parameters

application/json
▼ request_dataobjectrequired
▶ endpoint_idstring

Endpoint ID.

REQUEST example-1
```
{ "request_data": { "endpoint_id": "<endpoint ID>" } }
```
Responses

Successful response

Body
application/json
▼ replyobject
▶ policy_namestring

Name of the policy allocated with the endpoint.

RESPONSE example-1
```
{ "reply": { "policy_name": "Windows Default" } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.
▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.4.4 | Delete Endpoints

post /public_api/v1/endpoints/delete

Delete selected endpoints in Cortex Cloud. You can delete up to 1000 endpoints.

Note: Endpoints are deleted from Cortex Cloud UI, however they still exist in the database.

When filtering by multiple fields:

- Response is concatenated using AND condition (OR is not supported).
- Maximum result set size is 1000.
- Offset is the zero-based number of cases from the start of the result set.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/endpoints/delete'
-d ''
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"string\",\"operator\":\"in\",\"value\":[\"string\"]}]}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/endpoints/delete", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
```

```
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/endpoints/delete")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"string\",\"operator\":\"in\",\"value\":[\"string\"]}]}}" response = http.request(request) puts
response.read_body
```

```
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "string", "operator": "in", "value": [
"string" ] } ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/endpoints/delete");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/endpoints/delete")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":[{\"field\":\"string\",\"operator\":\"in\",\"value\":
[\"string\"]}]}}") .asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["filters": [ [ "field": "string", "operator":
"in", "value": ["string"] ] ]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/endpoints/delete")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/endpoints/delete", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":[{\"field\":\"string\",\"operator\":\"in\",\"value\":
[\"string\"]}]}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-
xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if
($err) { echo "cURL Error #:" . $err; } else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/endpoints/delete"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"string\",\"operator\":\"in\",\"value\":[\"string\"]}]}}"); CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/endpoints/delete"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"string\",\"operator\":\"in\",\"value\":[\"string\"]}]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ filtersarray

Array of filter fields.

REQUEST example-1

```
{ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in", "value": [ "<endpoint ID>" ] } ] }
}
```

Responses

Successful response

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.4.5 | Create distributions

post /public_api/v1/distributions/create

Create an installation package. This is an async call that returns the distribution ID; it does not mean that the creation succeeded. To confirm the package has been created, check the status of the distribution by running the **Get Distribution Status** API.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/distributions/create'
-d ''
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"name\":\"string\",\"platform\":\"windows\",\"package_type\":\"string\",\"agent_version\":\"string\",\"windows_version\":\"st
{\"property1\":null,\"property2\":null},\"proxy\":
[\"string\"],\"cluster_name\":\"string\",\"run_on_master_node\":true,\"run_on_all_nodes\":true}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/distributions/create", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/distributions/create") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":
{\"name\":\"string\",\"platform\":\"windows\",\"package_type\":\"string\",\"agent_version\":\"string\",\"windows_version\":\"st
{\"property1\":null,\"property2\":null},\"proxy\":
[\"string\"],\"cluster_name\":\"string\",\"run_on_master_node\":true,\"run_on_all_nodes\":true}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "name": "string", "platform": "windows", "package_type": "string",
"agent_version": "string", "windows_version": "string", "linux_version": "string", "macos_version": "string",
"deployment_platform": "string", "default_namespace": "string", "node_selector": { "property1": null, "property2":
null }, "proxy": [ "string" ], "cluster_name": "string", "run_on_master_node": true, "run_on_all_nodes": true } });
const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () {
if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/distributions/create"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/distributions/create")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":
{\"name\":\"string\",\"platform\":\"windows\",\"package_type\":\"string\",\"agent_version\":\"string\",\"windows_version\":\"st
{\"property1\":null,\"property2\":null},\"proxy\":
[\"string\"],\"cluster_name\":\"string\",\"run_on_master_node\":true,\"run_on_all_nodes\":true}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "name": "string", "platform": "windows",
"package_type": "string", "agent_version": "string", "windows_version": "string", "linux_version": "string",
"macos_version": "string", "deployment_platform": "string", "default_namespace": "string", "node_selector": [
"property1": , "property2": ], "proxy": ["string"], "cluster_name": "string", "run_on_master_node": true,
"run_on_all_nodes": true ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/distributions/create")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/distributions/create", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"name\":\"string\",\"platform\":\"windows\",\"package_type\":\"string\",\"agent_version\":\"string\",\"windows_version\":\"st
{\"property1\":null,\"property2\":null},\"proxy\":
[\"string\"],\"cluster_name\":\"string\",\"run_on_master_node\":true,\"run_on_all_nodes\":true}}", CURLOPT_HTTPHEADER
=> [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/distributions/create"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"name\":\"string\",\"platform\":\"windows\",\"package_type\":\"string\",\"agent_version\":\"string\",\"windows_version\":\"st
{\"property1\":null,\"property2\":null},\"proxy\":
[\"string\"],\"cluster_name\":\"string\",\"run_on_master_node\":true,\"run_on_all_nodes\":true}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/distributions/create"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":
{\"name\":\"string\",\"platform\":\"windows\",\"package_type\":\"string\",\"agent_version\":\"string\",\"windows_version\":\"st
{\"property1\":null,\"property2\":null},\"proxy\":
```

```
[\"string\"],\"cluster_name\":\"string\",\"run_on_master_node\":true,\"run_on_all_nodes\":true}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

▶ namestring

The name of the installation package.

▶ platformstring (Enum)

The installation platform.

▶ package_typestring

A string representing the type of package to create. Each JSON object must contain *one* of the following keywords:

- `standalone`: Installation for a new agent. When using this, you must include the `platform` field with one of the following values: `windows`, `linux`, `macos`, `android`, `kubernetes`, `helm`.
- `upgrade`: Upgrade of an agent from ESM. When using this, you must include the `agent_version` field with one of the following values: `windows_version`, `linux_version`, or `macos_version`.

▶ agent_versionstring

Use `agent_version` when creating a standalone installer. The value should be the agent version number.

▶ windows_versionstring

Use `windows_version` when creating an upgrade package. The value is the relevant version number.

▶ linux_versionstring

Use `linux_version` when creating an upgrade package. The value is the relevant version number.

▶ macos_versionstring

Use `macos_version` when creating an upgrade package. The value is the relevant version number.

▶ deployment_platformstring

When the `package_type` is `kubernetes` or `helm`, use the `deployment_platform` to indicate the type of platform. Valid values include:

- `standard`
- `openshift`
- `gcos`
- `bottlerocket`
- `gke_autopilot`

▶ default_namespacestring

The default namespace

▶ node_selectorobject

The node selector in the following format: `"node_selector": {"key": "val"}'

▶ proxyarray[string]

▶ cluster_namestring

Cluster name

▶ run_on_master_nodeboolean

Whether or not to run on the master node.

▶ run_on_all_nodesboolean

Whether or not to run on all nodes.

REQUEST New Installation example

```
{ "request_data": { "name": "<installation package name>", "platform": "windows", "package_type": "standalone",
"agent_version": "windows_version" } }
{ "request_data": { "name": "<installation package name>", "package_type": "upgrade", "agent_version":
"windows_version" } }
{ "request_data": { "name": "PAPI Dist K8s", "description": "Created using PAPI", "endpoint_tags": [ "new-tag" ],
"package_type": "kubernetes", "platform": "linux", "agent_version": "8.8.0.10594", "deployment_platform": "standard",
"default_namespace": "cortex-xdr", "node_selector": { "key": "val" }, "proxy": [ "10.10.10.1:8080" ], "cluster_name":
"some_name", "run_on_master_node": true, "run_on_all_nodes": false } }
```

Responses

Successful response

Body
application/json
▼ replyobject
▶ distribution_idstring

Installation package ID.

RESPONSE
{ "reply": { "distribution_id": "example" } }

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.4.6 | Get Violations

post /public_api/v1/device_control/get_violations

Gets a list of device control violations filtered by selected fields. You can retrieve up to 100 violations.

When filtering by multiple fields:

- Response is concatenated using AND condition (OR is not supported).
- Maximum result set size is 100.
- Offset is the zero-based number of cases from the start of the result set.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: xXdrAuthId_example

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/device_control/get_violations'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[0]}],\"search_from\":0,\"search_to\":0,\"sort\":
{\"field\":\"endpoint_id_list\",\"value\":\"asc\"}}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-
id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/device_control/get_violations", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/device_control/get_violations") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[0]}],\"search_from\":0,\"search_to\":0,\"sort\":{\"field\":\"endpoint_id_list\",\"value\":\"asc\"}}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in",
"value": [ 0 ] } ], "search_from": 0, "search_to": 0, "sort": { "field": "endpoint_id_list", "value": "asc" } } });
const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () {
if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/device_control/get_violations"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/device_control/get_violations")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[0]}],\"search_from\":0,\"search_to\":0,\"sort\":
{\"field\":\"endpoint_id_list\",\"value\":\"asc\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "endpoint_id_list",
"operator": "in", "value": [0] ] ], "search_from": 0, "search_to": 0, "sort": [ "field": "endpoint_id_list", "value":
"asc" ] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let
request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/device_control/get_violations")!
as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST"
request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/device_control/get_violations", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[0]}],\"search_from\":0,\"search_to\":0,\"sort\":
{\"field\":\"endpoint_id_list\",\"value\":\"asc\"}}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE",
"content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/device_control/get_violations"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[0]}],\"search_from\":0,\"search_to\":0,\"sort\":{\"field\":\"endpoint_id_list\",\"value\":\"asc\"}}}"); CURLcode ret
= curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/device_control/get_violations"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[0]}],\"search_from\":0,\"search_to\":0,\"sort\":{\"field\":\"endpoint_id_list\",\"value\":\"asc\"}}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

An empty object returns all results.

▶ filtersarray

Provides an array of filter fields.

▶ search_frominteger

Integer representing the starting offset within the query result set from which you want violations returned. Violations are returned as a zero-based list. Any violation indexed less than this value is not returned in the final result set and defaults to zero.

▶ search_tointeger

An integer representing the end of offset within the result set after which you do not want violations returned. Violations in the violation list that are indexed higher than this value are not returned in the final results set. Defaults to zero, which returns all alerts to the end of the list.

▶ sortobjectrequired

Identifies the sort order for the result set.

REQUEST Request all results

```
{ "request_data": {} }
{ "request_data": { "filters": [ { "field": "type", "operator": "in", "value": [ "disk drive" ] } ], "search_to": 1 }
}
```

Responses

Successful response

Body
application/json

▼ replyobject

▶ total_countinteger

Number of total results of this filter without paging.

▶ result_countinteger

Number of alerts actually returned as a result.

▶ violationsarray

RESPONSE

```
{ "reply": { "total_count": 0, "result_count": 0, "violations": [ { "hostname": "example", "username": "example",
"ip": "example", "timestamp": 0, "violation_id": 0, "type": "example", "vendor_id": "example", "vendor": "example",
"product_id": "example", "product": "example", "serial": "example", "endpoint_id": "example" } ] } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

2.1.4.7 | Get Distribution status

post /public_api/v1/distributions/get_status

Check the status of the installation package.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/distributions/get_status'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"distribution_id\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/distributions/get_status", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/distributions/get_status") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"distribution_id\":\"string\"}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "distribution_id": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/distributions/get_status"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/distributions/get_status")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"distribution_id\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["distribution_id": "string"]] as [String :
Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/distributions/get_status")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/distributions/get_status", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"distribution_id\":\"string\"}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/distributions/get_status"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
```

```
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"distribution_id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/distributions/get_status"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"distribution_id\":\"string\"}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters
application/json
▼ request_dataobjectrequired
▶ distribution_idstring

The installation package ID.

REQUEST
```
{ "request_data": { "distribution_id": "example" } }
```
Responses

Successful response

Body
application/json
▼ replyobject
▶ statusstring

The status of the installation package.

RESPONSE
```
{ "reply": { "status": "example" } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.4.8 | Get Distribution URL

post /public_api/v1/distributions/get_dist_url

Get the distribution URL for downloading the installation package.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/distributions/get_dist_url'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"distribution_id\":\"string\",\"package_type\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-
xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/distributions/get_dist_url", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/distributions/get_dist_url") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"distribution_id\":\"string\",\"package_type\":\"string\"}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "distribution_id": "string", "package_type": "string" } }); const xhr
= new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/distributions/get_dist_url"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/distributions/get_dist_url")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"distribution_id\":\"string\",\"package_type\":\"string\"}}")
.asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "distribution_id": "string", "package_type":
"string" ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let
request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/distributions/get_dist_url")! as
URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST"
request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/distributions/get_dist_url", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"distribution_id\":\"string\",\"package_type\":\"string\"}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/distributions/get_dist_url"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"distribution_id\":\"string\",\"package_type\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/distributions/get_dist_url"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"distribution_id\":\"string\",\"package_type\":\"string\"}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters

application/json
▼ request_dataobjectrequired
▶ distribution_idstring

Installation package ID.

▶ package_typestring

A string representing the type of installation package. Select *one* of the following valid keywords and values:

- `upgrade` Package type should match the distribution type or platform:
- `sh`: x86_64 Linux SH installer
- `rpm`: x86_64 Linux RPM installer
- `deb`: x86_64 Linux DEB installer
- `aarch64_sh`: aarch64 Linux SH installer
- `aarch64_rpm`: aarch64 Linux RPM installer
- `aarch64_deb`: aarch64 Linux DEB installer
- `pkg`: Mac
- `x86`: Windows
- `x64`: Windows

REQUEST example-1
```
{ "request_data": { "distribution_id": "<distribution ID>", "package_type": "x86" } }
```
Responses

Successful response

Body
application/json
▼ replyobject
▶ distribution_urlstring

URL for downloading the installation package.

RESPONSE example-1
```
{ "reply": { "distribution_url": "<DOWNLOAD_URL>" } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.
RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.4.9 | Set an Endpoint Alias

post /public_api/v1/endpoints/update_agent_name

Set or modify an Alias field for your endpoints.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/endpoints/update_agent_name'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"alias\":\"string\"}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/endpoints/update_agent_name", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/endpoints/update_agent_name") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"alias\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in",
"value": [ "string" ] } ], "alias": "string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/endpoints/update_agent_name"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/endpoints/update_agent_name")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"alias\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "endpoint_id_list",
"operator": "in", "value": ["string"] ] ], "alias": "string" ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/endpoints/update_agent_name")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/endpoints/update_agent_name", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"alias\":\"string\"}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/endpoints/update_agent_name"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
```

```
{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"alias\":\"string\"}}");
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/endpoints/update_agent_name"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"alias\":\"string\"}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
application/json
▼ request_dataobject
▶ filtersarray

An array of filter fields.

▶ aliasstring

The alias name you want to set or modify.

Note: If you send an empty field, the current alias name is deleted.

REQUEST example-1

```
{ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in", "value": [ "<distribution_id>" ] }
], "alias": "<alias_name>" } }
```

Responses

Successful response

Body
application/json

true=The alias name was set or modified successfully.

▼ boolean

true=The alias name was set or modified successfully.

RESPONSE

```
false
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.4.10 | Assign Tags

post /public_api/v1/tags/agents/assign

Assign one or more tags to one or more endpoints.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/tags/agents/assign'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id\",\"operator\":\"in\",\"value\":[\"string\"]}],\"tag\":\"string\"}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/tags/agents/assign", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/tags/agents/assign")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id\",\"operator\":\"in\",\"value\":[\"string\"]}],\"tag\":\"string\"}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "endpoint_id", "operator": "in", "value": [
"string" ] } ], "tag": "string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/tags/agents/assign");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/tags/agents/assign")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"tag\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "endpoint_id",
"operator": "in", "value": ["string"] ] ], "tag": "string" ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/tags/agents/assign")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/tags/agents/assign", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"tag\":\"string\"}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type:
application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/tags/agents/assign"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
```

```
[{\"field\":\"endpoint_id\",\"operator\":\"in\",\"value\":[\"string\"]}],\"tag\":\"string\"}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/tags/agents/assign"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"tag\":\"string\"}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
application/json
▼ request_dataobject
▶ filtersarray

An array of filter fields.

▶ tagstring

The tag you want to assign.

REQUEST example-1

```
{ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in", "value": [ "<distribution_id>" ] }
], "tag": "<tag_name>" } }
```

Responses

Successful response

Body
application/json

true=The tag name was assigned successfully.

▼ boolean

true=The tag name was assigned successfully.

RESPONSE

```
false
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

x Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.4.11 | Remove Tags

post /public_api/v1/tags/agents/remove

Remove one or more tags from one or more endpoints.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/tags/agents/remove'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"tag\":\"string\"}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/tags/agents/remove", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/tags/agents/remove")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"tag\":\"string\"}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in",
"value": [ "string" ] } ], "tag": "string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/tags/agents/remove");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/tags/agents/remove")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"tag\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "endpoint_id_list",
"operator": "in", "value": ["string"] ] ], "tag": "string" ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/tags/agents/remove")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/tags/agents/remove", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"tag\":\"string\"}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type:
application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/tags/agents/remove"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"tag\":\"string\"}}"); CURLcode ret =
curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/tags/agents/remove"); var request = new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"tag\":\"string\"}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
application/json

▼ request_dataobject

A dictionary containing the following API request fields.

▶ filtersarray

Array of filter fields.

▶ tagstring

The tag you want to remove.

REQUEST example-1

```
{ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in", "value": [ "<distribution_id>" ] } ], "tag": "<tag_name>" } }
```

Responses

Successful response

Body
application/json

true=tag name removed successfully.

▼ boolean

true=tag name removed successfully.

RESPONSE

```
false
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.4.12 | Get Endpoint

post /public_api/v1/endpoints/get_endpoint

Gets a list of filtered endpoints.

- The response is concatenated using AND condition (OR is not supported).
- The maximum result set size is 100.
- Offset is the zero-based number of endpoints from the start of the result set.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/endpoints/get_endpoint'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0,\"sort\":
{\"field\":\"endpoint_id\",\"keyword\":\"ASC\"}}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/endpoints/get_endpoint", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/endpoints/get_endpoint") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0,\"sort\":
{\"field\":\"endpoint_id\",\"keyword\":\"ASC\"}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in",
"value": "string" } ], "search_from": 0, "search_to": 0, "sort": { "field": "endpoint_id", "keyword": "ASC" } } });
const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () {
if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/endpoints/get_endpoint"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/endpoints/get_endpoint")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0,\"sort\":
{\"field\":\"endpoint_id\",\"keyword\":\"ASC\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "endpoint_id_list",
"operator": "in", "value": "string" ] ], "search_from": 0, "search_to": 0, "sort": [ "field": "endpoint_id",
"keyword": "ASC" ] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options:
[]) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/endpoints/get_endpoint")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/endpoints/get_endpoint", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0,\"sort\":
{\"field\":\"endpoint_id\",\"keyword\":\"ASC\"}}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE",
```

```
"content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/endpoints/get_endpoint"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-
id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0,\"sort\":
{\"field\":\"endpoint_id\",\"keyword\":\"ASC\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/endpoints/get_endpoint"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0,\"sort\":
{\"field\":\"endpoint_id\",\"keyword\":\"ASC\"}}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

A dictionary containing the API request fields.

An empty dictionary returns all results.

▶ filtersarray

Array of filter fields.

▶ search_frominteger

Represents the start offset within the query result set from which you want endpoints returned.

Endpoints are returned as a zero-based list. Any endpoint indexed less than this value is not returned in the final result set and defaults to zero.

▶ search_tointeger

Represents the end offset within the result set after which you do not want endpoints returned.

Endpoint in the endpoint list that is indexed higher than this value is not returned in the final results set. Defaults to 100, which returns all endpoints to the end of the list.

▶ sortobjectrequired

Identifies the sort order for the result set.

REQUEST Request filtered results

```
{ "request_data": { "search_from": 0, "search_to": 1, "sort": { "field": "endpoint_id", "keyword": "asc" },
"filters": [ { "field": "group_name", "operator": "in", "value": [ "Test-Group-01" ] }, { "field": "endpoint_status",
"operator": "in", "value": [ "disconnected" ] }, { "field": "dist_name", "operator": "in", "value": [ "papi-test" ]
}, { "field": "scan_status", "operator": "in", "value": [ "none", "pending", "in_progress", "pending_cancellation",
"aborted", "success", "canceled", "error" ] } ] } }
{ "request_data": {} }
```

Responses

OK

Body

application/json

▼ replyobject

JSON object containing the query result.

▶ total_countinteger

Number of total results of this filter without paging.

▶ result_countinteger

Number of endpoints actually returned as result.

▶ endpointsarray

A list of endpoints.

RESPONSE Example 1

{ "reply": { "total_count": 1, "result_count": 1, "endpoints": [{ "endpoint_id": "<endpoint ID>", "endpoint_name": "<endpoint name>", "endpointTags": "<tag name>", "endpoint_type": "<endpoint type>", "endpoint_status": "CONNECTED", "operational_status_details": [{ "title": "XDR Data Collection not running or not sent", "reason": "Linux kernel version is not supported" }, { "title": "BTP not working", "reason": "Linux kernel version is not supported" }, { "title": "Antimalware flow is asynchronous", "reason": "Linux kernel version is not supported" }, { "title": "Local privilege escalation", "reason": "Linux kernel version is not supported" }], "os_type": "AGENT_OS_WINDOWS", "os_version": "8.0.xxx", "ip": ["<IP address>"], "ipv6": [], "public_ip": "<IP address>", "users": ["XDR"], "domain": "WORKGROUP", "alias": "", "first_seen": 1606218761377, "last_seen": 1606218769163, "content_version": "", "installation_package": "XDR", "active_directory": null, "install_date": 1606218762089, "endpoint_version": "<version>", "is_isolated": "AGENT_UNISOLATED", "isolated_date": null, "group_name": [], "operational_status": "PARTIALLY_PROTECTED", "scan_status": "SCAN_STATUS_NONE", "content_release_timestamp": 1636285746000, "last_content_update_time": 1636381954285, "content_status": "up_to_date", "operating_system": "Debian 10.11", "mac_address": ["42:00:00:00:00:00"], "assigned_prevention_policy": "Linux Default", "assigned_extensions_policy": "" }] } }

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.4.13 | Initiate Forensics Triage

post /public_api/v1/triage_endpoint

Initiate forensics triage for the specified agents.

- Maximum of 10 concurrent triage actions at a time.
- Specified agents must have Forensics License enabled.
- Specified agents must be the same OS, Windows or macOS, but not a mixture of both.
- Specified configuration must have type "Online = True".

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/triage_endpoint'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"agent_ids\":
[\"string\"],\"collector_uuid\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/triage_endpoint",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/triage_endpoint")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"agent_ids\":
[\"string\"],\"collector_uuid\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "agent_ids": [ "string" ], "collector_uuid": "string" } }); const xhr
= new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/triage_endpoint"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/triage_endpoint")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"agent_ids\":[\"string\"],\"collector_uuid\":\"string\"}}")
.asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "agent_ids": ["string"], "collector_uuid":
"string" ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let
request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/triage_endpoint")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/triage_endpoint", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"agent_ids\":[\"string\"],\"collector_uuid\":\"string\"}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/triage_endpoint"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"agent_ids\":
[\"string\"],\"collector_uuid\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/triage_endpoint"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"agent_ids\":[\"string\"],\"collector_uuid\":\"string\"}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

▶ agent_idsarray[string]

List of agents to run forensics triage on.

▶ collector_uuidstring

UUID of the triage configuration. If none is specified, the default configuration is used for this action.

REQUEST

```
{ "request_data": { "agent_ids": [ "example" ], "collector_uuid": "example" } }
```

Responses

OK

Body
application/json
▼ replyobject
▶ group_action_idinteger

Unique ID for triage action.

▶ successful_agent_idsarray[string]

List of agent IDs that successfully received the triage action.

▶ unsuccessful_agent_idsarray

List of agent IDs that did not successfully receive the triage action.

RESPONSE Example 1

```
{ "reply": { "group_action_id": 325, "successful_agent_ids": [ "5111c5eb93944e2f97674db4f36b4211" ],
"unsuccessful_agent_ids": [] } }
```


Bad Request

Unauthorized

Payment Required

Forbidden

Internal Server Error

2.1.5 | Response Action

APIs for response actions

2.1.5.1 | Restore File

post /public_api/v1/endpoints/restore

Restore a quarantined file on a requested endpoints. When filtering by multiple fields:

- Response is concatenated using AND condition (OR is not supported).
- Maximum result set size is 100.
- Offset is the zero-based number of cases from the start of the result set.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/endpoints/restore'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"file_hash\":\"string\",\"endpoint_id\":\"string\",\"incident_id\":0}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/endpoints/restore", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/endpoints/restore")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
```

```
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"file_hash\":\"string\",\"endpoint_id\":\"string\",\"incident_id\":0}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "file_hash": "string", "endpoint_id": "string", "incident_id": 0 }
}); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function
() { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/endpoints/restore"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/endpoints/restore")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":
{\"file_hash\":\"string\",\"endpoint_id\":\"string\",\"incident_id\":0}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "file_hash": "string", "endpoint_id":
"string", "incident_id": 0 ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/endpoints/restore")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/endpoints/restore", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"file_hash\":\"string\",\"endpoint_id\":\"string\",\"incident_id\":0}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/endpoints/restore"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"file_hash\":\"string\",\"endpoint_id\":\"string\",\"incident_id\":0}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/endpoints/restore"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"file_hash\":\"string\",\"endpoint_id\":\"string\",\"incident_id\":0}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ file_hashstring

String that represents the file in hash. Hash must be a valid SHA256.

▶ endpoint_idstring

String that represents the endpoint ID. Note: if it is not specified, the request will run restore on all endpoints which relate to the quarantined file you defined.

▶ incident_idinteger

String representing the case ID. When included in the request, the Restore File action will appear in the Cortex Cloud Case View Timeline tab.

REQUEST example-1

```
{ "request_data": { "file_hash": "<hash value>", "incident_id": 302 } }
```

Responses

Successful response

Body

application/json

▼ replyobject

▶ action_idstring

▶ endpoints_countstring

RESPONSE example-1

```
{ "reply": { "action_id": "<action ID>", "status": 1, "endpoints_count": "673" } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.5.2 | File Retrieval Details

post /public_api/v1/actions/file_retrieval_details

View the API required to call in order to download the file retrieved by the **Retrieve File** API request according to the action ID.

The response contains a file hash you need to download and then unzip to view:

1. Download the file.

```
curl -XPOST "https://api-{fqdn}/public_api/v1/download/<api_value>"
-H "x-xdr-auth-id:{API_KEY_ID}"
-H "Authorization:{API_KEY}"
-H 'Content-Type:application/json'
--output /tmp/file.zip
```

2. Unzip the file: `unzip /tmp/file.zip`

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
```

```
'https://api-yourfqdn/public_api/v1/actions/file_retrieval_details'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"group_action_id\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/actions/file_retrieval_details", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/actions/file_retrieval_details") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"group_action_id\":\"string\"}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "group_action_id": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/actions/file_retrieval_details"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/actions/file_retrieval_details")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"group_action_id\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["group_action_id": "string"]] as [String :
Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/actions/file_retrieval_details")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/actions/file_retrieval_details", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"group_action_id\":\"string\"}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/actions/file_retrieval_details"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"group_action_id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/actions/file_retrieval_details"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"group_action_id\":\"string\"}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ group_action_idstring

The action ID of the Retrieve File API response.

REQUEST example-1

```
{ "request_data": { "group_action_id": "<action ID>" } }
```

Responses

Successful response

Body

application/json

▼ replyobject

▶ dataobject

RESPONSE Example 1

```
{ "reply": { "data": { "<endpoint_ID>": "https://api-{fqdn}/public_api/v1/download/<api_value>" } } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

2.1.5.3 | Allow List Files

post /public_api/v1/hash_exceptions/allowlist

Add files which do not exist in the allow or block lists to an allow list.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/hash_exceptions/allowlist'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"hash_list\":
[\"string\"],\"comment\":\"string\",\"incident_id\":0}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-
auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/hash_exceptions/allowlist", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/hash_exceptions/allowlist") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
```

```
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"hash_list\":[\"string\"],\"comment\":\"string\",\"incident_id\":0}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "hash_list": [ "string" ], "comment": "string", "incident_id": 0 }
}); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function
() { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/hash_exceptions/allowlist"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/hash_exceptions/allowlist")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"hash_list\":[\"string\"],\"comment\":\"string\",\"incident_id\":0}}")
.asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "hash_list": ["string"], "comment":
"string", "incident_id": 0 ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/hash_exceptions/allowlist")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/hash_exceptions/allowlist", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"hash_list\":
[\"string\"],\"comment\":\"string\",\"incident_id\":0}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE",
"content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/hash_exceptions/allowlist"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"hash_list\":[\"string\"],\"comment\":\"string\",\"incident_id\":0}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/hash_exceptions/allowlist"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"hash_list\":[\"string\"],\"comment\":\"string\",\"incident_id\":0}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ hash_listarray[string]

A list of hashed files you want to add to the allow list. Hash must be a valid SH256.

▶ commentstring

Additional information regarding the action.

▶ incident_idinteger

The case ID related to the hash. When included in the request, the **Allow List** action will appear in the **Cortex Cloud Case View Timeline** tab.

REQUEST example-1

```
{ "request_data": { "hash_list": [ "032196FB1A---DFCF69E5D553F0", "365296EB1B---FCF29E5D553E4", "365296EB1B---
FCF69E3D553E4", "365296EB1B---FCF69E5D553D4", "365296EB1B---FCF79E5D553D4" ], "comment": "test", "incident_id": 5 } }
```

Responses

Successful response

Body

application/json

true=File successfully added to the allow list.

▼ boolean

true=File successfully added to the allow list.

RESPONSE
```
false
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.5.4 | Get Quarantine Status

post /public_api/v1/quarantine/status

Retrieve the quarantine status for specified files.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/quarantine/status'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"files\":
[{\"endpoint_id\":\"string\",\"file_path\":\"string\",\"file_hash\":\"string\"}]}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/quarantine/status", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
```

```
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/quarantine/status")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"files\":
[{\"endpoint_id\":\"string\",\"file_path\":\"string\",\"file_hash\":\"string\"}]}}" response = http.request(request)
puts response.read_body
```

```
const data = JSON.stringify({ "request_data": { "files": [ { "endpoint_id": "string", "file_path": "string",
"file_hash": "string" } ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/quarantine/status");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/quarantine/status")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"files\":
[{\"endpoint_id\":\"string\",\"file_path\":\"string\",\"file_hash\":\"string\"}]}}") .asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["files": [ [ "endpoint_id": "string",
"file_path": "string", "file_hash": "string" ] ]]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/quarantine/status")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/quarantine/status", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"files\":
[{\"endpoint_id\":\"string\",\"file_path\":\"string\",\"file_hash\":\"string\"}]}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/quarantine/status"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"files\":
[{\"endpoint_id\":\"string\",\"file_path\":\"string\",\"file_hash\":\"string\"}]}}"); CURLcode ret =
curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/quarantine/status"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"files\":[{\"endpoint_id\":\"string\",\"file_path\":\"string\",\"file_hash\":\"string\"}]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

▶ filesarray

Array of endpoint IDs, filepaths, and file hash.

REQUEST example-1

```
{ "request_data": { "files": [ { "endpoint_id": "<endpoint ID>", "file_path": "C:\\<file path>\\test_x64.msi",
"file_hash": "<hash value>" } ] } }
```

Responses

Successful response

Body

application/json

▼ replyarray

[

▶ endpoint_idstring

Endpoint ID.

▶ file_pathstring

File path.

▶ file_hashstring

File hash.

▶ statusboolean

The file's status. True: The file is quarantined. False: The file is not quarantined.

]
RESPONSE example-1

```
{ "reply": [ { "endpoint_id": "<endpoint ID>", "file_path": "C:\\<file path>\\test_x64.msi", "file_hash": "<hash value>", "status": false } ] }
```


Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.5.5 | Quarantine Files

post /public_api/v1/endpoints/quarantine

Quarantine file on selected endpoints. You can select up to 1000 endpoints.

Note: A success response means that the request reached the defined endpoints, however if the file was not found there, no quarantine action will take place. To ensure if the file has been quarantined, check the Cortex XDR Action Center.

When filtering by multiple fields:

- Response is concatenated using AND condition (OR is not supported).
- Maximum result set size is 1000.
- Offset is the zero-based number of cases from the start of the result set.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/endpoints/quarantine'
-d ''
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"file_path\":\"string\",\"file_hash\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE",
'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/endpoints/quarantine", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
```

```
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/endpoints/quarantine") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"file_path\":\"string\",\"file_hash\":\"string\"}}" response = http.request(request) puts
response.read_body
```

```
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in",
"value": [ "string" ] } ], "file_path": "string", "file_hash": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/endpoints/quarantine");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/endpoints/quarantine")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"file_path\":\"string\",\"file_hash\":\"string\"}}") .asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "endpoint_id_list",
"operator": "in", "value": ["string"] ] ], "file_path": "string", "file_hash": "string" ]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/endpoints/quarantine")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/endpoints/quarantine", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"file_path\":\"string\",\"file_hash\":\"string\"}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/endpoints/quarantine"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"file_path\":\"string\",\"file_hash\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/endpoints/quarantine"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"file_path\":\"string\",\"file_hash\":\"string\"}}", ParameterType.RequestBody); IRestResponse
response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ filtersarray

An array of filter fields.

▶ file_pathstring

The path of the file you want to quarantine. You must enter a proper path and not symbolic links.

▶ file_hashstring

Case ID. When included in the request, the **Quarantine File** action will appear in the **Cortex Cloud Case View Timeline** tab.

REQUEST example-1
```
{ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in", "value": [ "<endpoint ID>" ] } ],
"file_path": "C:\\<file path>\\test_x64.msi", "file_hash": "<hash value>" } }
```
Responses

Successful response

Body
application/json
▼ replyobject

JSON object containing the query result.

▶ action_idstring

ID of action to quarantine selected endpoints. Response only indicates the request was successfully sent to the endpoint. To track if the file quarantine succeeded either:

- In Cortex XDR console, navigate to **Response** > **Action Center** and search for the action ID. Make sure the **Action ID** field is selected in the table **Layout** settings.
- Send a **Get Action Status** API request.

▶ statusstring

Integer representing whether the action:

- 1: succeeded
- 0: failed

▶ endpoints_countstring

Number of endpoints included in the request.

RESPONSE example-1
```
{ "reply": { "action_id": "[ID value]", "status": "1", "endpoints_count": "673" } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.5.6 | Block List Files

post /public_api/v1/hash_exceptions/blocklist

Add files which do not exist in the allow or block lists to a block list. You can view the block list in the UI at Investigation & Response > Response > Action Center > Block List.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/hash_exceptions/blocklist'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"hash_list\":
[\"string\"],\"comment\":\"string\",\"incident_id\":0}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-
auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/hash_exceptions/blocklist", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/hash_exceptions/blocklist") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"hash_list\":[\"string\"],\"comment\":\"string\",\"incident_id\":0}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "hash_list": [ "string" ], "comment": "string", "incident_id": 0 }
}); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function
() { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/hash_exceptions/blocklist"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/hash_exceptions/blocklist")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"hash_list\":[\"string\"],\"comment\":\"string\",\"incident_id\":0}}")
.asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "hash_list": ["string"], "comment":
"string", "incident_id": 0 ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/hash_exceptions/blocklist")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/hash_exceptions/blocklist", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"hash_list\":
[\"string\"],\"comment\":\"string\",\"incident_id\":0}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE",
"content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/hash_exceptions/blocklist"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"hash_list\":[\"string\"],\"comment\":\"string\",\"incident_id\":0}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/hash_exceptions/blocklist"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"hash_list\":[\"string\"],\"comment\":\"string\",\"incident_id\":0}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ hash_listarray[string]

A list of hashed files you want add to a block list. Hash must be a valid SH256.

▶ commentstring

Additional information regarding the action.

▶ incident_idinteger

The case ID related to the hash. When included in the request, the Block List action appears in the Cortex Cloud Case View Timeline tab.

REQUEST example-1

```
{ "request_data": { "hash_list": [ "032196FB1A---DFCF69E5D553F0", "365296EB1B---FCF69E7D553E4", "365296EB1B---
FCF69E5D523E4", "365296EB1B---FCF69E5D553D4", "365296EB1B---FCF63E5D553D4" ], "comment": "test", "incident_id": 5 } }
```

Responses

Successful response

Body

application/json

true=File successfully added to block list.

▼ boolean

true=File successfully added to block list.

RESPONSE

```
false
```

Bad Request. Got an invalid JSON.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.5.7 | Unisolate Endpoints

post /public_api/v1/endpoints/unisolate

Reverse the isolation of one or more endpoints in single request.

Note: You can only send a request with either `endpoint_id` to unisolate one endpoint or with filters to unisolate more than one endpoint. An error is raised if you try to use both `endpoint_id` and the filters.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/endpoints/unisolate'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"endpoint_id\":\"string\",\"incident_id\":\"string\"}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/endpoints/unisolate", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/endpoints/unisolate") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"endpoint_id\":\"string\",\"incident_id\":\"string\"}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in",
"value": [ "string" ] } ], "endpoint_id": "string", "incident_id": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/endpoints/unisolate");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/endpoints/unisolate")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
```

```
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"endpoint_id\":\"string\",\"incident_id\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "endpoint_id_list",
"operator": "in", "value": ["string"] ] ], "endpoint_id": "string", "incident_id": "string" ]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/endpoints/unisolate")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/endpoints/unisolate", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"endpoint_id\":\"string\",\"incident_id\":\"string\"}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/endpoints/unisolate"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"endpoint_id\":\"string\",\"incident_id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/endpoints/unisolate"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"endpoint_id\":\"string\",\"incident_id\":\"string\"}}", ParameterType.RequestBody); IRestResponse
response = client.Execute(request);
```


Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ filtersarray

An array of filter fields for unisolating a number of endpoints at once. Note: This field is only required if unisolating more than one endpoint.

▶ endpoint_idstring

The ID of the endpoint to unisolate.

Note: this field is only required if unisolating one endpoint.

▶ incident_idstring

Case ID. When included in the request, the **Unisolate Endpoints** action will appear in the **Cortex Cloud Case View Timeline** tab.

REQUEST Unisolate one endpoint


```
{ "request_data": { "endpoint_id": "<endpoint ID>" } }
{ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "IN", "value": [ "<endpoint_id_1>", "
<endpoint_id_2>", "<endpoint_id_3>" ] } ] } }
```


Responses

Successful response

Body

application/json

▼ replyobject

JSON object containing the query result.

▶ action_idstring

ID of the action to unisolate selected endpoints. Response only indicates the request was successfully sent to the endpoint. To track if the endpoint was restored either:

- In the Cortex XDR console, navigate to **Response** > **Action Center** > **Isolation** and search for the action ID. Make sure the **Action ID** field is selected in the table **Layout** settings.
- Send a **Get Action Status** API request.

▶ endpoints_countstring

Number of endpoints included in the request.

RESPONSE example-1
```
{ "reply": { "action_id": "<action ID>", "status": "1", "endpoints_count": "673" } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.5.8 | Cancel Scan Endpoints

post /public_api/v1/endpoints/abort_scan

Cancel the scan of selected endpoints. A scan can only be aborted if the selected endpoints are in **Pending** or in **Progress** status.

When filtering by multiple fields:

- Response is concatenated using AND condition (OR is not supported).
- Offset is the zero-based number of endpoints from the start of the result set.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/endpoints/abort_scan'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"incident_id\":\"string\"}}" headers =
{ 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/endpoints/abort_scan", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/endpoints/abort_scan") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"incident_id\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in",
"value": [ "string" ] } ], "incident_id": "string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials =
true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/endpoints/abort_scan");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/endpoints/abort_scan")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"incident_id\":\"string\"}}")
.asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "endpoint_id_list",
"operator": "in", "value": ["string"] ] ], "incident_id": "string" ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/endpoints/abort_scan")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/endpoints/abort_scan", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"incident_id\":\"string\"}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/endpoints/abort_scan"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"incident_id\":\"string\"}}");
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/endpoints/abort_scan"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"incident_id\":\"string\"}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ filtersobject

An array of filter fields to filter which endpoints to cancel scanning. To cancel scan of all endpoints, use the value "all".

▶ incident_idstring

Case ID. When included in the request, the **Cancel Scan Endpoints** action will appear in the **Cortex Cloud Case View Timeline** tab.

REQUEST To cancel scan of all endpoints

```
{ "request_data": { "filters": "all" } }
{ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in", "value": [ "<endpoint ID>" ] }, { "field": "dist_name", "operator": "in", "value": [ "WinInstaller" ] }, { "field": "group_name", "operator": "in", "value": [ "test" ] }, { "field": "scan_status", "operator": "in", "value": [ "none", "pending", "in_progress", "pending_cancellation", "aborted", "success" ] }, { "field": "group_name", "operator": "in", "value": [ "test" ] } ] } }
```

Responses

Successful response

Body
application/json
▼ replyobject

JSON object containing the query result.

▶ action_idstring

ID of action to cancel scan selected endpoints. Response only indicates the request was successfully sent to the endpoint. To track if the scan succeeded either:

- In Cortex XDR console, navigate to **Response** > **Action Center** and search for the action ID. Make sure the **Action ID** field is selected in the table Layout settings.
- Send a **Get Action Status** API request.

▶ endpoints_countstring

Number of endpoints included in the request.

RESPONSE

```
{ "reply": { "action_id": "example", "endpoints_count": "example" } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.5.9 | Scan Endpoints

post /public_api/v1/endpoints/scan

Run a scan on selected endpoints.

- Response is concatenated using AND condition (OR is not supported).
- Offset is the zero-based number of cases from the start of the result set.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/endpoints/scan'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"filters\":\"all\",\"incident_id\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/endpoints/scan",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/endpoints/scan")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"filters\":\"all\",\"incident_id\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": "all", "incident_id": "string" } }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/endpoints/scan"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/endpoints/scan")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":\"all\",\"incident_id\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": "all", "incident_id": "string" ]]
as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/endpoints/scan")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/endpoints/scan", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":\"all\",\"incident_id\":\"string\"}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/endpoints/scan"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"filters\":\"all\",\"incident_id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/endpoints/scan"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
```

```
"{\"request_data\":{\"filters\":\"all\",\"incident_id\":\"string\"}}", ParameterType.RequestBody); IRestResponse
response = client.Execute(request);
```

Body parameters
application/json
▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ filtersstring (Enum)

An array of filter fields. To scan all endpoints, use the value `all`.

▶ incident_idstring

Case ID. When included in the request, the Scan Endpoints action will appear in the Cortex Cloud Case View Timeline tab.

REQUEST Scan all endpoints

```
{ "request_data": { "filters": "all" } }
{ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in", "value": [ "<endpoint ID>" ] }, {
"field": "dist_name", "operator": "in", "value": [ "WinInstaller" ] }, { "field": "group_name", "operator": "in",
"value": [ "test\"" ] }, { "field": "scan_status", "operator": "in", "value": [ "none", "pending", "in_progress",
"pending_cancellation", "aborted", "success" ] }, { "field": "group_name", "operator": "in", "value": [ "test" ] } ]
} }
```

Responses

Successful response

Body
application/json
▼ replyobject

JSON object containing the query result.

▶ action_idstring

ID of action to scan selected endpoints. Response only indicates the request was successfully sent to the endpoint. To track if the scan was successful either:

- In Cortex XDR console, navigate to Response > Action Center > All Actions and search for the action ID. Make sure the Action ID field is selected in the table Layout settings.
- Send a Get Action Status API request.

▶ endpoints_countstring

Number of endpoints included in the request.

RESPONSE

```
{ "reply": { "action_id": "example", "endpoints_count": "example" } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.5.10 | Get Action Status

post /public_api/v1/actions/get_action_status

Retrieve the status of the requested actions according to the action ID.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/actions/get_action_status'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"group_action_id\":0}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE",
'content-type': "application/json" } conn.request("POST", "/public_api/v1/actions/get_action_status", payload,
headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/actions/get_action_status") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"group_action_id\":0}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "group_action_id": 0 } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/actions/get_action_status"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/actions/get_action_status")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"group_action_id\":0}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["group_action_id": 0]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/actions/get_action_status")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/actions/get_action_status", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"group_action_id\":0}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/actions/get_action_status"); struct curl_slist *headers = NULL;
```

```
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\": {\"group_action_id\":0}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/actions/get_action_status"); var request = new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{\"request_data\":{\"group_action_id\":0}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ group_action_idinteger

Action ID of the selected request.

REQUEST example-1

```
{ "request_data": { "group_action_id": 123456789 } }
```

Responses

Successful response

Body

application/json

▼ replyobject

JSON object containing the query result.

▶ dataobject

RESPONSE example-1

```
{ "reply": { "data": { "<agent ID>": "COMPLETED_SUCCESSFULLY" } } }
```

Bad Request. Got an invalid JSON.

Body

application/json

▼ replyobject

▶ dataobject

JSON object containing the query result.

▶ errorReasonsobject

Returns all error messages the agent returns to allow for easier analysis.

RESPONSE Example 1

```
{ "reply": { "data": { "5a763600a3e44928a94ba84b6088380f": "FAILED" }, "errorReasons": { "5a763600a3e44928a94ba84b6088380f": { "errorData": "{\"reportIds\": [\"0ca3c1ace7694fcd8b44b735e5ba6c04\"],\"errorText\":\"\"}", "terminated_by": "instance_id", "errorDescription": "Element not found.\r\n", "terminate_result": [ { "path": null, "status": 1 } ] } } } }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body

application/json

▼ replyobject

▶ dataobject

JSON object containing the query result.

▶ errorReasonsobject

Returns all error messages the agent returns to allow for easier analysis.

RESPONSE Example 1

```
{ "reply": { "data": { "5a763600a3e44928a94ba84b6088380f": "FAILED" }, "errorReasons": { "5a763600a3e44928a94ba84b6088380f": { "errorData": "{\"reportIds\": [\"0ca3c1ace7694fcd8b44b735e5ba6c04\"],\"errorText\":\"\"}", "terminated_by": "instance_id", "errorDescription": "Element not found.\r\n", "terminate_result": [ { "path": null, "status": 1 } ] } } } }
```

Unauthorized access. User does not have the required license type to run this API.

Body

application/json
▼ replyobject
▶ dataobject

JSON object containing the query result.

▶ errorReasonsobject

Returns all error messages the agent returns to allow for easier analysis.

RESPONSE Example 1


```
{ "reply": { "data": { "5a763600a3e44928a94ba84b6088380f": "FAILED" }, "errorReasons": {
"5a763600a3e44928a94ba84b6088380f": { "errorData": "{\"reportIds\":
[\"0ca3c1ace7694fcd8b44b735e5ba6c04\"],\"errorText\":\"\"}", "terminated_by": "instance_id", "errorDescription":
"Element not found.\r\n", "terminate_result": [ { "path": null, "status": 1 } ] } } } }
```


Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json
▼ replyobject
▶ dataobject

JSON object containing the query result.

▶ errorReasonsobject

Returns all error messages the agent returns to allow for easier analysis.

RESPONSE Example 1


```
{ "reply": { "data": { "5a763600a3e44928a94ba84b6088380f": "FAILED" }, "errorReasons": {
"5a763600a3e44928a94ba84b6088380f": { "errorData": "{\"reportIds\":
[\"0ca3c1ace7694fcd8b44b735e5ba6c04\"],\"errorText\":\"\"}", "terminated_by": "instance_id", "errorDescription":
"Element not found.\r\n", "terminate_result": [ { "path": null, "status": 1 } ] } } } }
```


Internal server error. A unified status for API communication type errors.

Body
application/json
▼ replyobject
▶ dataobject

JSON object containing the query result.

▶ errorReasonsobject

Returns all error messages the agent returns to allow for easier analysis.

RESPONSE Example 1


```
{ "reply": { "data": { "5a763600a3e44928a94ba84b6088380f": "FAILED" }, "errorReasons": {
"5a763600a3e44928a94ba84b6088380f": { "errorData": "{\"reportIds\":
[\"0ca3c1ace7694fcd8b44b735e5ba6c04\"],\"errorText\":\"\"}", "terminated_by": "instance_id", "errorDescription":
"Element not found.\r\n", "terminate_result": [ { "path": null, "status": 1 } ] } } } }
```

2.1.5.11 | Retrieve File

post /public_api/v1/endpoints/file_retrieval

Retrieve files from selected endpoints. You can retrieve up to 20 files, from no more than 10 endpoints.

- Response is concatenated using AND condition (OR is not supported).
- Offset is the zero-based number of cases from the start of the result set.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: xXdrAuthId_example

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/endpoints/file_retrieval'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"files\":{\"windows\":
[\"string\"],\"linux\":[\"string\"],\"macos\":[\"string\"]},\"incident_id\":\"string\"}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/endpoints/file_retrieval", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/endpoints/file_retrieval") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"files\":{\"windows\":[\"string\"],\"linux\":[\"string\"],\"macos\":
[\"string\"]},\"incident_id\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in",
"value": [ "string" ] } ], "files": { "windows": [ "string" ], "linux": [ "string" ], "macos": [ "string" ] },
"incident_id": "string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/endpoints/file_retrieval"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/endpoints/file_retrieval")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"files\":{\"windows\":
[\"string\"],\"linux\":[\"string\"],\"macos\":[\"string\"]},\"incident_id\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "endpoint_id_list",
"operator": "in", "value": ["string"] ] ], "files": [ "windows": ["string"], "linux": ["string"], "macos": ["string"]
], "incident_id": "string" ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/endpoints/file_retrieval")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/endpoints/file_retrieval", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"files\":{\"windows\":
[\"string\"],\"linux\":[\"string\"],\"macos\":[\"string\"]},\"incident_id\":\"string\"}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/endpoints/file_retrieval"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":[\"string\"]}],\"files\":{\"windows\":
[\"string\"],\"linux\":[\"string\"],\"macos\":[\"string\"]},\"incident_id\":\"string\"}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/endpoints/file_retrieval"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"files\":{\"windows\":[\"string\"],\"linux\":[\"string\"],\"macos\":
[\"string\"]},\"incident_id\":\"string\"}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters
application/json
▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ filtersarray

An array of filter fields.

▶ filesobject

One of the operating system types must be included.

▶ incident_idstring

Case ID. When included in the request, the Retrieve File action will appear in the Cortex Cloud Case View Timeline tab.

REQUEST

```
{ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in", "value": [ "example" ] } ],
"files": { "windows": [ "example" ], "linux": [ "example" ], "macos": [ "example" ] }, "incident_id": "example" } }
```

Responses

OK

Body
application/json
▼ replyobject

JSON object containing the query result.

▶ action_idstring

ID of action to retrieve files from selected endpoints. Response only indicates the request was successfully sent to the endpoint. To track if the file was retrieved successfully either: in the Cortex XDR console, navigate to Response > Action Center > Isolation and search for the action ID. Make sure the Action ID field is selected in the table Layout settings by selecting the three vertical dots. To view the file, send a File Retrieval Details request.

▶ statusstring
▶ endpoints_countstring

Number of endpoints included in the request.

RESPONSE

```
{ "reply": { "action_id": "example", "status": "example", "endpoints_count": "example" } }
```


Bad Request

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Unauthorized access. User does not have the required license type to run this API.

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Internal server error. A unified status for API communication type errors.

2.1.5.12 | Isolate Endpoints

post /public_api/v1/endpoints/isolate

Isolate one or more endpoints in a single request. Request is limited to 1000 endpoints.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Security, you need the Cortex Cloud Runtime Security add-on.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/endpoints/isolate'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"endpoint_id\":\"string\",\"incident_id\":\"string\"}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/endpoints/isolate", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/endpoints/isolate")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"endpoint_id\":\"string\",\"incident_id\":\"string\"}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in",
"value": [ "string" ] } ], "endpoint_id": "string", "incident_id": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/endpoints/isolate");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/endpoints/isolate")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"endpoint_id\":\"string\",\"incident_id\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "endpoint_id_list",
"operator": "in", "value": ["string"] ] ], "endpoint_id": "string", "incident_id": "string" ]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/endpoints/isolate")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/endpoints/isolate", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"endpoint_id\":\"string\",\"incident_id\":\"string\"}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/endpoints/isolate"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"endpoint_id\":\"string\",\"incident_id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/endpoints/isolate"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"endpoint_id\":\"string\",\"incident_id\":\"string\"}}", ParameterType.RequestBody); IRestResponse
response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ filtersarray

Array of filtered fields for isolating a number of endpoints at once. Note: Only required if isolating more than one endpoint.

▶ endpoint_idstring

Identifies the endpoint to isolate. Note: Only required if isolating one endpoint.

▶ incident_idstring

The case ID. When included in the request, the **Isolate Endpoints action** will appear in the Cortex Cloud Case View Timeline tab.

REQUEST Isolate one endpoint

```
{ "request_data": { "endpoint_id": "<endpoint ID>" } }
{ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in", "value": [ "<endpoint ID 1>", "
<endpoint ID 2>", "<endpoint ID 3>" ] } ] } }
```

Responses

OK

Body
application/json
▼ replyobject

JSON object containing the query result.

▶ action_idstring

Action ID to scan selected endpoints. The response only indicates the request was successfully sent to the endpoint. To track if the isolation succeeded either:

- In the Cortex XDR console, navigate to **Response** > **Action Center** > **Isolation** and search for the action ID. Make sure the Action ID field is selected in the table Layout settings by selecting the vertical ellipses.
- Send a **Get Action Status** request.

▶ endpoints_countstring

Number of endpoints included in the request.

RESPONSE Example 1

```
{ "reply": { "action_id": "<action ID>", "status": "1", "endpoints_count": "673" } }
```

Bad Request. Got an invalid JSON.

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, ID, or other invalid authentication parameters.

Unauthorized access. User does not have the required license type to run this API.

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Internal server error. A unified status for API communication type errors.

2.1.5.13 | Get triage presets

post /public_api/v1/get_triage_presets

Get all triage preset information including triage name, platform, description, created by, and triage type.

Required license: In Cortex Cloud Runtime Security, requires the Forensics add-on. Not supported in Cortex Cloud Posture Management.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/get_triage_presets'
-d ''
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/get_triage_presets", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/get_triage_presets") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": {} }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/get_triage_presets"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/get_triage_presets") .header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type", "application/json") .body("{\"request_data\":{}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE", "content-type": "application/json" ] let parameters = ["request_data": []] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/get_triage_presets")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/get_triage_presets", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd, CURLOPT_URL, "https://api-yourfqdn/public_api/v1/get_triage_presets"); struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/get_triage_presets"); var request = new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{\"request_data\":{}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

REQUEST

```
{ "request_data": {} }
```

Responses

OK

Body

application/json

▼ replyobject

▶ triage_presetsarray

RESPONSE Example 1

```
{ "reply": { "triage_presets": [ { "uuid": "374ecf0457c944c39791e5a60d1a6d24", "name": "XDR Default", "os": "MACOS", "description": "Default macOS Triage configuration included with the XDR Forensics add-on", "created_by": "user@company.com", "type": "Online / Offline" }, { "uuid": "ea7a5d3ff52d41629e96fbd1d5f68535", "name": "XDR Default", "os": "WINDOWS", "description": "Default Triage configuration included with XDR Forensics add-on", "created_by": "user@company.com", "type": "Online / Offline" } ] } }
```

2.1.6 | Script execution

APIs executing script

2.1.6.1 | Run Snippet Code Script

post /public_api/v1/scripts/run_snippet_code_script

Initiate a new endpoint script execution action using provided snippet code. Cortex XDR supports sending your request in Base64.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/scripts/run_snippet_code_script'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"timeout\":600,\"snippet_code\":\"string\",\"incident_id\":\"string\"}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/scripts/run_snippet_code_script", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/scripts/run_snippet_code_script") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"timeout\":600,\"snippet_code\":\"string\",\"incident_id\":\"string\"}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in",
"value": [ "string" ] } ], "timeout": 600, "snippet_code": "string", "incident_id": "string" } }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/scripts/run_snippet_code_script"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/scripts/run_snippet_code_script")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"timeout\":600,\"snippet_code\":\"string\",\"incident_id\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "endpoint_id_list",
"operator": "in", "value": ["string"] ] ], "timeout": 600, "snippet_code": "string", "incident_id": "string" ]] as
[String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/scripts/run_snippet_code_script")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/scripts/run_snippet_code_script", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"timeout\":600,\"snippet_code\":\"string\",\"incident_id\":\"string\"}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/scripts/run_snippet_code_script"); struct curl_slist *headers =
```

```
NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers,
"x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"timeout\":600,\"snippet_code\":\"string\",\"incident_id\":\"string\"}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/scripts/run_snippet_code_script"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"timeout\":600,\"snippet_code\":\"string\",\"incident_id\":\"string\"}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters
application/json
▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ filtersarray

An array of filter fields for running the script on a number of endpoints at once.

▶ timeoutinteger

The timeout in seconds for this execution. Default value is 600.

▶ snippet_codestring

Section of a script you want to initiate on an endpoint.

▶ incident_idstring

Case ID. When included in the request, the **Run Snippet Code Script** action will appear in the **Cortex Cloud Case View Timeline** tab.

REQUEST example-1

```
{ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in", "value": [ "<endpoint ID>" ] } ],
"snippet_code": "print (\"7\")" } }
```

Responses

Successful response

Body
application/json
▼ replyobject

JSON object containing the query result.

▶ action_idstring

ID of the action initiated. ID will be used as a reference to track in the action center.

▶ endpoints_countinteger

Number of endpoints the action was initiated on.

RESPONSE example-1

```
{ "reply": { "action_id": "<action ID>", "endpoints_count": 21 } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.6.2 | Run Script

post /public_api/v1/scripts/run_script

Initiate a new endpoint script execution action using a script from the script library. The script can be run on up to 1000 endpoints.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/scripts/run_script'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"script_uid\":\"string\",\"parameters_values\":
{\"x\":\"string\",\"y\":0},\"timeout\":600,\"incident_id\":\"string\"}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/scripts/run_script", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/scripts/run_script")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"script_uid\":\"string\",\"parameters_values\":
{\"x\":\"string\",\"y\":0},\"timeout\":600,\"incident_id\":\"string\"}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in",
"value": [ "string" ] } ], "script_uid": "string", "parameters_values": { "x": "string", "y": 0 }, "timeout": 600,
"incident_id": "string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/scripts/run_script");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/scripts/run_script")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
```

```
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"script_uid\":\"string\",\"parameters_values\":
{\"x\":\"string\",\"y\":0},\"timeout\":600,\"incident_id\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "endpoint_id_list",
"operator": "in", "value": ["string"] ] ], "script_uid": "string", "parameters_values": [ "x": "string", "y": 0 ],
"timeout": 600, "incident_id": "string" ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject:
parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/scripts/run_script")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/scripts/run_script", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"script_uid\":\"string\",\"parameters_values\":
{\"x\":\"string\",\"y\":0},\"timeout\":600,\"incident_id\":\"string\"}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/scripts/run_script"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"script_uid\":\"string\",\"parameters_values\":
{\"x\":\"string\",\"y\":0},\"timeout\":600,\"incident_id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/scripts/run_script"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id_list\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"script_uid\":\"string\",\"parameters_values\":
{\"x\":\"string\",\"y\":0},\"timeout\":600,\"incident_id\":\"string\"}}", ParameterType.RequestBody); IRestResponse
response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ filtersarray

Array of filter fields for running the script on a number of endpoints at once.

▶ script_uidstring

GUID, unique identifier of the script, returned by the Get Scripts API per script.

▶ parameters_valuesobjectrequired

Dictionary containing the parameter name, `key`, and its value for this execution, `value`.

You can obtain these values by running Get Script Metadata API.

▶ timeoutinteger

Timeout in seconds for this execution. Default value is 600.

▶ incident_idstring

Case ID. When included in the request, the Run Script action will appear in the Cortex Cloud Case View Timeline tab.

REQUEST example-1

```
{ "request_data": { "filters": [ { "field": "endpoint_id_list", "operator": "in", "value": [ "<endpoint ID>" ] } ],
"script_uid": "<unique ID>", "parameters_values": { "x": "param input as returned in Get Script Metadata", "y": 4 } }
}
```

Responses

Successful response

Body
application/json
▼ replyobject

JSON object containing the query result.

▶ action_idinteger

ID of the action initiated. ID will be used as a reference to track in the action center.

▶ endpoints_countinteger

Number of endpoints the action was initiated on.

▶ statusinteger

Integer representing whether the action:

- 1: succeeded
- 0: failed

RESPONSE example-1

```
{ "reply": { "action_id": 22519813685366, "status": 1, "endpoints_count": 1 } }
```


Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.6.3 | Get Script Metadata

post /public_api/v1/scripts/get_script_metadata

Get the full definitions of a specific script in the scripts library.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/scripts/get_script_metadata'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"script_uid\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE",
'content-type': "application/json" } conn.request("POST", "/public_api/v1/scripts/get_script_metadata", payload,
headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/scripts/get_script_metadata") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"script_uid\":\"string\"}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "script_uid": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/scripts/get_script_metadata"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/scripts/get_script_metadata")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"script_uid\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["script_uid": "string"]] as [String : Any]
let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/scripts/get_script_metadata")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/scripts/get_script_metadata", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"script_uid\":\"string\"}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/scripts/get_script_metadata"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"script_uid\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/scripts/get_script_metadata"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"script_uid\":\"string\"}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ script_uidstring

Unique identifier of the script, returned by the Get Scripts API per script.

REQUEST example-1

```
{ "request_data": { "script_uid": "<unique ID>" } }
```

Responses

Successful response

Body
application/json
▼ replyobject

JSON object containing the query result.

▶ script_idstring

Script ID.

▶ namestring

Name of script.

▶ descriptionstring

Description of script.

▶ modification_dateinteger

Timestamp of when the script was last modified.

▶ created_bystring

Name of the user who created the script.

▶ is_high_riskboolean

Whether the script has a high-risk outcome.

▶ windows_supportedboolean

Whether the script can be executed on Windows OS.

▶ linux_supportedboolean

Whether the script can be executed on Linux OS.

▶ macos_supportedboolean

Whether the script can be executed on macOS.

▶ script_uidstring

GUID, global ID of the script, used toidentify the script when executing.

▶ entry_pointstring

name of the entry point selected for the script defined as `run`.

▶ script_inputarray
▶ script_output_typestring (Enum)

Type of output.

▶ script_output_dictionary_definitionsarray

When the `script_output_type` is a dictionary an array with `friendly_name`, `name`, and `type` for each output is returned. The field is empty in all other cases.

RESPONSE When entry_point is returned as run


```
{ "reply": { "script_id": "<script ID>", "name": "list_directories", "description": "List all directories under path", "modification_date": 1585074627259, "created_by": "Palo Alto Networks", "is_high_risk": false, "windows_supported": true, "linux_supported": true, "macos_supported": true, "script_uid": "<unique ID>", "entry_point": "run", "script_input": [ { "name": "path", "type": "string" }, { "friendly_name": "Number of levels", "name": "num_levels", "type": "integer" } ], "script_output_type": "dictionary", "script_output_dictionary_definitions": [ { "friendly_name": "Number Of Processes", "name": "output_2", "type": "integer" }, { "friendly_name": "Name", "name": "output_1", "type": "string" } ] } }
{ "reply": { "script_id": "<script ID>", "name": "list_directories", "description": "List all directories under path", "modification_date": 1585074627259, "created_by": "Palo Alto Networks", "is_high_risk": false, "windows_supported": true, "linux_supported": true, "macos_supported": true, "script_uid": "<unique ID>" } }
```


Bad Request. Got an invalid JSON.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.6.4 | Get Script Execution Status

post /public_api/v1/scripts/get_script_execution_status

Retrieve the status of a script execution action.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/scripts/get_script_execution_status'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\": {\"action_id\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/scripts/get_script_execution_status", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/scripts/get_script_execution_status") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"action_id\":\"string\"}}" response = http.request(request) puts response.read_body
```

```
const data = JSON.stringify({ "request_data": { "action_id": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/scripts/get_script_execution_status"); xhr.setRequestHeader("Authorization",
"SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-
type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-
yourfqdn/public_api/v1/scripts/get_script_execution_status") .header("Authorization", "SOME_STRING_VALUE")
.header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"action_id\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["action_id": "string"]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/scripts/get_script_execution_status")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/scripts/get_script_execution_status", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"action_id\":\"string\"}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/scripts/get_script_execution_status"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers,
"x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"action_id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/scripts/get_script_execution_status"); var request =
new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-
id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"action_id\":\"string\"}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ action_idstring

Identifier of the action, can be found in Cortex XDR console Response > Action Center > Action ID field.

REQUEST example-1

```
{ "request_data": { "action_id": "<action ID>" } }
```

Responses

Successful response

Body

application/json

▼ replyobject

JSON object containing the query result.

▶ general_statusstring

General status of the action, considering the status of all the endpoints.

▶ endpoints_pendinginteger

Number of endpoints in pending status.

▶ endpoints_canceledinteger

Number of endpoints in 'canceled' status.

▶ endpoints_in_progressinteger

Number of endpoints in 'in progress' status.

▶ endpoints_timeoutinteger

Number of endpoints in 'timeout' status.

▶ endpoints_failedinteger

Number of endpoints in 'failed' status.

▶ endpoints_completed_successfullyinteger

Number of endpoints in 'completed successfully' status.

▶ endpoints_pending_abortinteger

Number of endpoints in 'pending abort' status.

▶ endpoints_abortedinteger

Number of endpoints in 'aborted' status.

▶ endpoints_expiredinteger

Number of endpoints in 'expired' status.

▶ error_messagestring

Error message regarding permissions for running APIs or stating that the action doesn't exist.

RESPONSE example-1

```
{ "reply": { "general_status": "PENDING", "endpoints_pending": 1, "endpoints_canceled": 0, "endpoints_in_progress":
0, "endpoints_timeout": 0, "endpoints_failed": 0, "endpoints_completed_successfully": 0, "endpoints_pending_abort":
0, "endpoints_aborted": 0, "endpoints_expired": 0 } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.6.5 | Get Scripts

post /public_api/v1/scripts/get_scripts

Get a list of scripts available in the scripts library.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management
Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/scripts/get_scripts'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"in\",\"value\":[\"string\"]}]}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/scripts/get_scripts", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/scripts/get_scripts") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"name\",\"operator\":\"in\",\"value\":[\"string\"]}]}}"
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "name", "operator": "in", "value": [ "string"
] } ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange",
function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST",
"https://api-yourfqdn/public_api/v1/scripts/get_scripts"); xhr.setRequestHeader("Authorization",
"SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-
type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/scripts/get_scripts")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":[{\"field\":\"name\",\"operator\":\"in\",\"value\":
[\"string\"]}]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["filters": [ [ "field": "name", "operator":
"in", "value": ["string"] ] ]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/scripts/get_scripts")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/scripts/get_scripts", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":[{\"field\":\"name\",\"operator\":\"in\",\"value\":
[\"string\"]}]}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-
xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if
($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/scripts/get_scripts"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"in\",\"value\":[\"string\"]}]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/scripts/get_scripts"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"name\",\"operator\":\"in\",\"value\":[\"string\"]}]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json
▼ request_dataobjectrequired

A dictionary containing the API request fields. An empty dictionary returns all results.

▶ filtersarray

An array of filter fields.

REQUEST Request all results
```
{ "request_data": {} }
{ "request_data": { "filters": [ { "field": "is_high_risk", "operator": "in", "value": [ "false" ] } ] } }
```
Responses

Successful response

Body
application/json
▼ replyobject

JSON object containing the query result.

▶ total_countinteger

Number of total results of this filter without paging.

▶ result_countinteger

Number of scripts returned as result.

▶ scriptsarray

An array of scripts.

RESPONSE example-1
```
{ "reply": { "total_count": 129, "result_count": 24, "scripts": [ { "script_id": "<script ID>", "name":
"list_directories", "description": "List all directories under path", "modification_date": 1585074627259,
"created_by": "Palo Alto Networks", "is_high_risk": false, "windows_supported": true, "linux_supported": true,
"macos_supported": true, "script_uid": "<unique ID>" }, { "script_id": "<script ID>", "name": "test 1",
"description": "test", "modification_date": 1583052236449, "created_by": "User 1", "is_high_risk": false,
"windows_supported": true, "linux_supported": false, "macos_supported": false, "script_uid": "<unique ID>" }, {
"script_id": "<script ID>", "name": "test 2", "description": "test 2", "modification_date": 1582709343498,
"created_by": "User 2", "is_high_risk": false, "windows_supported": true, "linux_supported": true, "macos_supported":
true, "script_uid": "<unique ID>" } ] } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.
▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.6.6 | Get Script Execution Results

post /public_api/v1/scripts/get_script_execution_results

Retrieve the results of a script execution action.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/scripts/get_script_execution_results'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\": {\"action_id\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/scripts/get_script_execution_results", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/scripts/get_script_execution_results") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"action_id\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "action_id": "string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/scripts/get_script_execution_results"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/scripts/get_script_execution_results") .header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type", "application/json") .body("{\"request_data\": {\"action_id\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE", "content-type": "application/json" ] let parameters = ["request_data": ["action_id": "string"]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/scripts/get_script_execution_results")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/scripts/get_script_execution_results", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"action_id\":\"string\"}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd, CURLOPT_URL, "https://api-yourfqdn/public_api/v1/scripts/get_script_execution_results"); struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers,
```

```
"x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"action_id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/scripts/get_script_execution_results"); var request =
new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-
id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"action_id\":\"string\"}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ action_idstring

Action ID. This can be found in the Cortex XDR console Response > Action Center > Action ID field.

REQUEST example-1

```
{ "request_data": { "action_id": "<action_id>" } }
```

Responses

Successful response

Body

application/json

▼ replyobject

JSON object containing the query result.

▶ script_namestring

Name of the script executed.

▶ script_descriptionstring

Description of the script executed.

▶ script_parametersarray

For each input parameter used in this execution, an array of `name` and `value`.

▶ date_createdstring

Timestamp in which the action was initiated.

▶ scopestring

Number of endpoints included in this action according to the filter used to select them.

▶ error_messagestring

Error message regarding permissions for running APIs.

▶ resultsarray

For each endpoint Cortex XDR displays any returned value by the script. The number of the results and their name are dynamic per script.

RESPONSE example-1

```
{ "reply": { "script_name": "snippet script", "script_description": "", "script_parameters": [], "date_created":
"2020-03-29 13:21:59", "scope": "win_10and 21 other endpoints", "error_message": "", "results": [ { "endpoint_name":
"<name>", "endpoint_ip_address": [ "<IP address>" ], "endpoint_status": "LOST", "domain": "aaaa", "endpoint_id": "
<endpoint ID>", "execution_status": "PENDING", "standard_output": null, "retrieved_files": 0, "failed_files": 0,
"retention_date": null }, { "endpoint_name": "<name>", "endpoint_ip_address": [ "<IP address>" ], "endpoint_status":
"LOST", "domain": "<domain name>", "endpoint_id": "<endpoint ID>", "execution_status": "PENDING", "standard_output":
null, "retrieved_files": 0, "failed_files": 0, "retention_date": null }, { "endpoint_name": "<name>",
"endpoint_ip_address": [ "<IP address>" ], "endpoint_status": "DISCONNECTED", "domain": "WORKGROUP", "endpoint_id": "
<endpoint ID>", "execution_status": "PENDING", "standard_output": null, "retrieved_files": 0, "failed_files": 0,
"retention_date": null } ] } }
```

Bad Request. Got an invalid JSON.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.6.7 | Get Script Execution Result Files

post /public_api/v1/scripts/get_script_execution_results_files

Get the files retrieved from a specific endpoint during a script execution.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/scripts/get_script_execution_results_files'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\": {\"action_id\":\"string\",\"endpoint_id\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/scripts/get_script_execution_results_files", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/scripts/get_script_execution_results_files") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"action_id\":\"string\",\"endpoint_id\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "action_id": "string", "endpoint_id": "string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
```

```
yourfqdn/public_api/v1/scripts/get_script_execution_results_files"); xhr.setRequestHeader("Authorization",
"SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-
type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-
yourfqdn/public_api/v1/scripts/get_script_execution_results_files") .header("Authorization", "SOME_STRING_VALUE")
.header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"action_id\":\"string\",\"endpoint_id\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "action_id": "string", "endpoint_id":
"string" ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let
request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/scripts/get_script_execution_results_files")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/scripts/get_script_execution_results_files", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING
=> "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"action_id\":\"string\",\"endpoint_id\":\"string\"}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE",
"content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/scripts/get_script_execution_results_files"); struct curl_slist
*headers = NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers =
curl_slist_append(headers, "x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":{\"action_id\":\"string\",\"endpoint_id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/scripts/get_script_execution_results_files"); var
request = new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE");
request.AddHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"action_id\":\"string\",\"endpoint_id\":\"string\"}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
application/json
▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ action_idstring

Identifier of the action, can be found in Cortex XDR console Response > Action Center > Action ID field.

▶ endpoint_idstring

Endpoint ID.

REQUEST

```
{ "request_data": { "action_id": "example", "endpoint_id": "example" } }
```

Responses

Successful response

Body
application/json
▼ replyobject

JSON object containing the query result.

▶ datastring

A signed public link to a zip file containing the retrieved files. Link expires after 10 minutes.

RESPONSE example-1

```
{ "reply": { "DATA": "https://example-link" } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.6.8 | Get Script Code

post /public_api/v1/scripts/get_script_code

Get the code of a specific script in the script library.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/scripts/get_script_code'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\": {\"script_uid\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/scripts/get_script_code", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/scripts/get_script_code") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"script_uid\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "script_uid": "string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/scripts/get_script_code"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
```

```
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/scripts/get_script_code")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"script_uid\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["script_uid": "string"]] as [String : Any]
let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/scripts/get_script_code")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/scripts/get_script_code", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"script_uid\":\"string\"}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/scripts/get_script_code"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"script_uid\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/scripts/get_script_code"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"script_uid\":\"string\"}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ script_uidstring

Unique identifier of the script, returned by the Get Scripts API per script.

REQUEST example-1

```
{ "request_data": { "script_uid": "<unique ID>" } }
```

Responses

Successful response

Body

application/json

▼ replystring

JSON object containing the query result.

RESPONSE example-1

```
{ "reply": "import os\nimport sys\nimport traceback\n\n\ndef run(path, num_levels):\n\tpath =
os.path.expanduser(path)\n\tpath = os.path.expandvars(path)\n\treturn scan_directory_recursive(path,
num_levels)\n\n\ndef scan_directory_recursive(directory, level):\n\tif level == 0:\n\t\treturn []\n\n\tsubfolders =
[]\n\ttry:\n\t\twith os.scandir(directory) as entries:\n\t\t\tfor f in entries:\n\t\t\t\ttry:\n\t\t\t\t\tif
f.is_dir():\n\t\t\t\t\t\tsubfolders.append(f.path)\n\t\t\t\texcept
(OSError,PermissionError):\n\t\t\t\t\tsys.stderr.write(f\"Failed access <{f.path}>\\n\")\n\n\tfor folder in
list(subfolders):\n\t\t subfolders.extend(scan_directory_recursive(folder, level-1))\n\texcept
Exception:\n\t\tsys.stderr.write(f\"Failed scanning directory: <{directory}>, error:
{traceback.format_exc()}\")\n\n\treturn subfolders\n" }
```

Bad Request. Got an invalid JSON.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.
▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.7 | Audit log

APIs for audit logs

2.1.7.1 | Get Audit Management Log

post /public_api/v1/audits/management_logs

Get audit management logs.

- Response is concatenated using AND condition (OR is not supported).
- Maximum result set size is 100.
- Offset is the zero-based number of cases from the start of the result set.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json,application/xml'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/audits/management_logs'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"email\",\"operator\":\"in\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":100,\"sort\":
{\"field\":\"type\",\"keyword\":\"asc\"}}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/audits/management_logs", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/audits/management_logs") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
```

```
request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"email\",\"operator\":\"in\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":100,\"sort\":
{\"field\":\"type\",\"keyword\":\"asc\"}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "email", "operator": "in", "value": "string"
} ], "search_from": 0, "search_to": 100, "sort": { "field": "type", "keyword": "asc" } } }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/audits/management_logs"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/audits/management_logs")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"email\",\"operator\":\"in\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":100,\"sort\":
{\"field\":\"type\",\"keyword\":\"asc\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "email", "operator":
"in", "value": "string" ] ], "search_from": 0, "search_to": 100, "sort": [ "field": "type", "keyword": "asc" ] ]] as
[String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/audits/management_logs")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/audits/management_logs", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"email\",\"operator\":\"in\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":100,\"sort\":
{\"field\":\"type\",\"keyword\":\"asc\"}}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-
type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/audits/management_logs"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-
id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"filters\":
[{\"field\":\"email\",\"operator\":\"in\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":100,\"sort\":
{\"field\":\"type\",\"keyword\":\"asc\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/audits/management_logs"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":
[{\"field\":\"email\",\"operator\":\"in\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":100,\"sort\":
{\"field\":\"type\",\"keyword\":\"asc\"}}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

A dictionary containing the API request fields. An empty dictionary returns all results.

▶ filtersarray

Array of filter fields.

▶ search_frominteger

An integer representing the starting offset within the query result set from which you want management logs returned. Management logs are returned as a zero-based list. Any log indexed less than this value is not returned in the final result set and defaults to zero.

▶ search_tointeger

An integer representing the end offset within the result set after which you do not want management logs returned. Logs in the management log list that are indexed higher than this value are not returned in the final results set. Defaults to 100, which returns all logs to the end of the list.

▶ sortobjectrequired

Identifies the sort order for the result set. By default the sort is defined as creation-time and desc.

REQUEST Request all management logs from older to newer

{ "request_data": { "search_from": 0, "search_to": 100, "sort": { "field": "timestamp", "keyword": "asc" } } }
{ "request_data": {} }
{ "request_data": { "search_from": 0, "search_to": 100, "sort": { "field": "timestamp", "keyword": "asc" },
"filters": [{ "field": "type", "operator": "in", "value": ["AUTH"] }, { "field": "sub_type", "operator": "in",
"value": ["login"] }, { "field": "result", "operator": "in", "value": ["SUCCESS"] }, { "field": "timestamp",
"operator": "gte", "value": 1565074114053 }] } }

Responses

Successful response

Body
application/json

▼ replyobject

JSON object containing the query result.

▶ total_countinteger

Number of total results of this filter without paging.

▶ result_countinteger

Number of returned items.

▶ dataarray

List of audit items.

RESPONSE example-1

{ "reply": { "total_count": 1, "result_count": 1, "data": [{ "AUDIT_ID": 1, "AUDIT_OWNER_NAME": "User Name",
"AUDIT_OWNER_EMAIL": "username@paloaltonetworks.com", "AUDIT_ASSET_JSON": "{}", "AUDIT_ASSET_NAMES": "",
"AUDIT_HOSTNAME": "", "AUDIT_RESULT": "SUCCESS", "AUDIT_REASON": "", "AUDIT_DESCRIPTION": "", "AUDIT_ENTITY": "AUTH",
"AUDIT_ENTITY_SUBTYPE": "Login", "AUDIT_SESSION_ID": 382303947890, "AUDIT_CASE_ID": 473829372, "AUDIT_INSERT_TIME":
1565074114053, "AUDIT_SEVERITY": "SEV_020_LOW", "AUDIT_LINK": null, "AUDIT_SOURCE_IP": "31.174.156.148",
"AUDIT_USER_AGENT": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/121.0.0.0 Safari/537.36", "AUDIT_USER_ROLES": ["Account Admin"], "AUDIT_ADDITIONAL_INFORMATION": {
"endpoint_names": ["WIN-fgo6762G"], "endpoint_count": 1 }, "AUDIT_OBJECT_ID": null }] } }

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.
RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
RESPONSE

2.1.7.2 | Get Audit Agent Report

post /public_api/v1/audits/agents_reports

Get agent event reports.

- Response is concatenated using AND condition (OR is not supported).
- Maximum result set size is 100.
- Offset is the zero-based number of cases from the start of the result set.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/audits/agents_reports'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"search_from\":0,\"search_to\":100,\"sort\":{\"field\":\"type\",\"keyword\":\"asc\"}}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/audits/agents_reports", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/audits/agents_reports") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"search_from\":0,\"search_to\":100,\"sort\":{\"field\":\"type\",\"keyword\":\"asc\"}}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "endpoint_id", "operator": "in", "value": [
"string" ] } ], "search_from": 0, "search_to": 100, "sort": { "field": "type", "keyword": "asc" } } }); const xhr =
new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/audits/agents_reports"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/audits/agents_reports")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"search_from\":0,\"search_to\":100,\"sort\":{\"field\":\"type\",\"keyword\":\"asc\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "endpoint_id",
"operator": "in", "value": ["string"] ] ], "search_from": 0, "search_to": 100, "sort": [ "field": "type", "keyword":
"asc" ] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let
request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/audits/agents_reports")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/audits/agents_reports", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"endpoint_id\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"search_from\":0,\"search_to\":100,\"sort\":{\"field\":\"type\",\"keyword\":\"asc\"}}}",
```

```
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/audits/agents_reports"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-
id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"filters\":[{\"field\":\"endpoint_id\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"search_from\":0,\"search_to\":100,\"sort\":{\"field\":\"type\",\"keyword\":\"asc\"}}}"); CURLcode
ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/audits/agents_reports"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"endpoint_id\",\"operator\":\"in\",\"value\":
[\"string\"]}],\"search_from\":0,\"search_to\":100,\"sort\":{\"field\":\"type\",\"keyword\":\"asc\"}}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

A dictionary containing the API request fields. An empty dictionary returns all results.

▶ filtersarray

An array of filter fields.

▶ search_frominteger

An integer representing the starting offset within the query result set from which you want agent reports returned. Reports are returned as a zero-based list. Any report indexed less than this value is not returned in the final result set and defaults to zero.

▶ search_tointeger

An integer representing the end offset within the result set after which you do not want agent reports returned. Reports in the agent report list that are indexed higher than this value are not returned in the final results set. Defaults to 100, which returns all reports to the end of the list.

▶ sortobjectrequired

Identifies the sort order for the result set.

REQUEST Request all results

```
{ "request_data": {} }
{ "request_data": { "filters": [ { "field": "trapsversion", "operator": "in", "value": [ "<version value>", "<version
value>" ] }, { "field": "timestamp", "operator": "gte", "value": 0 }, { "field": "domain", "operator": "in", "value":
[ "WORKGROUP" ] } ], "sort": { "field": "timestamp", "keyword": "asc" } } }
```

Responses

OK

Body

application/json

▼ replyobject

JSON object containing the query result.

▶ total_countinteger

Number of total results of this filter without paging.

▶ result_countinteger

Number of returned items.

▶ dataarray

List of audit items.

RESPONSE

```
{ "reply": { "total_count": 0, "result_count": 0, "data": [ { "TIMESTAMP": 0.1, "RECEIVEDTIME": 0.1, "ENDPOINTID":
"example", "ENDPOINTNAME": "example", "DOMAIN": "example", "TRAPSVERSION": "example", "CATEGORY": "example", "TYPE":
"example", "SUBTYPE": "example", "RESULT": "example", "REASON": "example", "DESCRIPTION": "example" } ] } }
```

Bad Request. Got an invalid JSON.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, ID, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.8 | System management

APIs for system management

2.1.8.1 | System Health Check

get /public_api/v1/healthcheck

Perform a health check of your Cortex Cloud environment.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/healthcheck'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE" } conn.request("GET", "/public_api/v1/healthcheck", headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/healthcheck") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/v1/healthcheck");
```

```
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/v1/healthcheck")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE" ] let
request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/healthcheck")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields =
headers let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/healthcheck", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/v1/healthcheck"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/healthcheck"); var request = new
RestRequest(Method.GET); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); IRestResponse response = client.Execute(request);
```

Responses

OK

Body
application/json
▼ statusstring

The condition of your Cortex environment.

RESPONSE
```
{ "status": "example" }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.
RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.8.2 | Get Tenant Info

post /public_api/v1/system/get_tenant_info

Get your tenant license information.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/system/get_tenant_info'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/system/get_tenant_info", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/system/get_tenant_info") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": {} }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/system/get_tenant_info");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/system/get_tenant_info")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": []] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/system/get_tenant_info")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/system/get_tenant_info", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE",
"content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/system/get_tenant_info"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-
id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/system/get_tenant_info"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```
Body parameters
application/json
▼ request_dataobject
REQUEST

```
{ "request_data": {} }
```

Responses

OK

Body
application/json
▼ replyobject

List of available licenses, number of devices, and purchased add-ons on your tenant.

- ▶ xsiam_premium_expirationinteger
- ▶ purchased_xsiam_premiumobject
- ▶ pro_per_endpoint_expirationinteger
- ▶ purchased_pro_per_endpointobject
- ▶ data_enabled_pro_per_endpointinteger
- ▶ prevent_expirationinteger
- ▶ purchased_preventinteger
- ▶ installed_preventinteger
- ▶ pro_gb_expirationinteger
- ▶ purchased_pro_gbobject
- ▶ installed_pro_gbinteger
- ▶ compute_unit_expirationinteger
- ▶ purchased_compute_unitinteger
- ▶ host_insights_expirationinteger
- ▶ enabled_host_insightsinteger
- ▶ purchased_host_insightsinteger
- ▶ forensics_expirationinteger
- ▶ enabled_forensicsinteger
- ▶ pro_cloud_expirationinteger
- ▶ purchased_pro_cloudobject
- ▶ installed_pro_cloudinteger
- ▶ data_enabled_pro_cloudinteger
- ▶ identity_threat_expirationinteger
- ▶ xth_expirationinteger
- ▶ purchased_xthinteger
- ▶ threat_intelligence_management_expirationinteger
- ▶ purchased_threat_intelligence_managementinteger
- ▶ attack_surface_management_expirationinteger
- ▶ purchased_attack_surface_managementinteger
- ▶ xsiam_ep_hot_expirationstring
- ▶ purchased_xsiam_ep_hotinteger
- ▶ xsiam_ep_cold_expirationstring
- ▶ purchased_xsiam_ep_coldinteger
- ▶ xsiam_gb_hot_expirationstring
- ▶ purchased_xsiam_gb_hotinteger
- ▶ xsiam_gb_cold_expirationstring
- ▶ purchased_xsiam_gb_coldinteger

RESPONSE Example 1


```
{ "reply": { "xsiam_premium_expiration": 12478046378, "purchased_xsiam_premium": { "users": 100, "gb": 100, "agents":
500 }, "pro_per_endpoint_expiration": 12478046378, "purchased_pro_per_endpoint": { "agents": 200 },
"data_enabled_pro_per_endpoint": 26, "prevent_expiration": 0, "purchased_prevent": 0, "installed_prevent": 27,
"pro_tb_expiration": 12478046378, "purchased_pro_gb": { "gb": 1 }, "installed_pro_tb": 0, "compute_unit_expiration":
0, "purchased_compute_unit": 16, "host_insights_expiration": 12478046378, "enabled_host_insights": 26,
"purchased_host_insights": 400, "forensics_expiration": 0, "enabled_forensics": 12 } }
{ "reply": { "pro_cloud_expiration": 0, "purchased_pro_cloud": 0, "installed_pro_cloud": 4, "data_enabled_pro_cloud":
4, "pro_per_endpoint_expiration": "Nov 7th 2025 07:59:59", "purchased_pro_per_endpoint": { "agents": 200 },
"data_enabled_pro_per_endpoint": 4, "prevent_expiration": 0, "purchased_prevent": 0, "installed_prevent": 4,
"pro_gb_expiration": "Nov 8th 2027 07:59:59", "purchased_pro_gb": { "gb": 33 }, "identity_threat_expiration": "Aug
13th 2023 11:03:41", "compute_unit_expiration": "Nov 7th 2025 07:59:59", "purchased_compute_unit": 50,
"host_insights_expiration": "Nov 8th 2027 07:59:59", "enabled_host_insights": 8, "purchased_host_insights": 250,
"forensics_expiration": 0, "enabled_forensics": 4, "xsiam_ep_hot_expiration": "Jan 2nd 2024 23:59:59",
"purchased_xsiam_ep_hot": 2 } }
```


Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal Server Error

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.8.3 | Get Users

post /public_api/v1/rbac/get_users

Retrieve a list of the current users in your environment.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/rbac/get_users'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/rbac/get_users", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/rbac/get_users") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{}" response = http.request(request) puts response.read_body
const data = JSON.stringify({}); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/rbac/get_users"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/rbac/get_users")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = [] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/rbac/get_users")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/rbac/get_users", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type:
application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/rbac/get_users"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/rbac/get_users"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

Object

application/json

REQUEST

```
{}
```

Responses

OK

Body

application/json

▼ replyarray

JSON object containing the query result.

[

▶ user_emailstring

Email address of the user.

▶ user_first_namestring

First name of the user.

▶ user_last_namestring

Last name of the user.

▶ role_namestring

Role name associated with the user.

▶ last_logged_ininteger

Timestamp of when the user last logged in.

▶ user_typestring

Type of user.

▶ groupsarray

Name of user groups associated with the user, if applicable.

▶ scopearray

Name of scope associated with the user, if applicable.

]

RESPONSE Example 1


```
{ "reply": [ { "user_email": "user1@acme.com", "user_first_name": "<first name>", "user_last_name": "<last name>",
"role_name": "Account Admin", "last_logged_in": 1640024700241, "user_type": "CSP", "groups": [], "scope": [] }, {
"user_email": "user2@acme.com", "user_first_name": "<first name>", "user_last_name": "<last name>", "role_name":
"Account Admin", "last_logged_in": null, "user_type": "CSP", "groups": [], "scope": [] }, { "user_email":
"user3@acme.com", "user_first_name": "<first name>", "user_last_name": "<last name>", "role_name": "Investigator",
"last_logged_in": null, "user_type": "CSP", "groups": [], "scope": [] } ] }
```


Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE


```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE


```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.8.4 | Get Roles

post /public_api/v1/rbac/get_roles

Retrieve information about one or more roles created in your environment.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
```

```
'https://api-yourfqdn/public_api/v1/rbac/get_roles'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"role_names\":
[\"string\"]}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-
type': "application/json" } conn.request("POST", "/public_api/v1/rbac/get_roles", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/rbac/get_roles")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"role_names\":
[\"string\"]}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "role_names": [ "string" ] } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/rbac/get_roles");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/rbac/get_roles")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"role_names\":[\"string\"]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["role_names": ["string"]]] as [String : Any]
let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/rbac/get_roles")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/rbac/get_roles", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"role_names\":[\"string\"]}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/rbac/get_roles"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"role_names\":
[\"string\"]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/rbac/get_roles"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"role_names\":[\"string\"]}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

A dictionary containing the API request fields.

▶ role_namesarray[string]

List of one or more role names in your environment for which you want detailed information.

REQUEST Example 1

```
{ "request_data": { "role_names": [ "Role1", "Role2" ] } }
```

Responses

OK

Body

application/json

▼ replyarray

JSON object containing the query result.

[

▶ pretty_namestring

Name of the role as it appears in the Management Console.

▶ permissionsarray[string]

List of permissions associated with this role.

▶ insert_timeinteger

Timestamp of when the Role was created.

▶ update_timeinteger

Timestamp of when the Role was last updated.

▶ created_bystring

Email of the user who created the Role.

▶ descriptionstring

Description of the Role, if available.

▶ groupsarray[string]

Group names associated with the Role.

▶ usersarray[string]

Email addresses of users associated with the Role.

]
RESPONSE Example 1

```
{ "reply": [ [ { "pretty_name": "Role1", "permissions": [ "Reports", "Playbooks", "Datasets Access Control",
"Dashboards", "Scripts" ], "insert_time": 1658315576844, "update_time": null, "created_by": "user1@acme.com",
"description": "", "groups": [ "group1", "group2" ], "users": [] } ], [ { "pretty_name": "Role2", "permissions": [
"Dashboards", "Datasets Access Control" ], "insert_time": 1661435660656, "update_time": null, "created_by":
"user1@acme.com", "description": "", "groups": [], "users": [] } ] ] }
```


Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE


```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE


```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE


```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE


```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.8.5 | Get User Groups

post /public_api/v1/rbac/get_user_group

Retrieve a list of the current user emails associated with one or more user groups in your environment.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/rbac/get_user_group'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"group_names\":
[\"string\"]}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-
type': "application/json" } conn.request("POST", "/public_api/v1/rbac/get_user_group", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/rbac/get_user_group") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"group_names\":[\"string\"]}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "group_names": [ "string" ] } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/rbac/get_user_group");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/rbac/get_user_group")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"group_names\":[\"string\"]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["group_names": ["string"]]] as [String : Any]
let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/rbac/get_user_group")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/rbac/get_user_group", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"group_names\":[\"string\"]}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/rbac/get_user_group"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"group_names\":
[\"string\"]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/rbac/get_user_group"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"group_names\":[\"string\"]}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```
Body parameters

application/json
▼ request_dataobject

A dictionary containing the API request fields.

▶ group_namesarray[string]

List of one or more user group names for which you want the associated users.

REQUEST Example 1

```
{ "request_data": { "group_names": [ "Group1", "Group2" ] } }
```

Responses

OK

Body
application/json
▼ replyarray

JSON object containing the query result.

[
▶ group_namestring

Name of the User Group.

▶ descriptionobject

Description of the User Group, if available.

▶ pretty_namestring

Name of the User Group as it appears in the Management Console.

▶ insert_timeinteger

Timestamp of when the User Group was created.

▶ update_timeinteger

Timestamp of when the User Group was last updated.

▶ user_emailarray[string]

List of email addresses belonging to the users associated with the User Group.

▶ sourcestring

Type of User Group.

]
RESPONSE Example 1

```
{ "reply": [ { "group_name": "Group1", "description": null, "pretty_name": "Investigator", "insert_time":
1661170832341, "update_time": 1661171650679, "user_email": [ "user1@acme.com", "user2@pacme.com", "user3@acme.com",
"user4@acme.com", "user5@acme.com" ], "source": "Custom" }, { "group_name": "Group2", "description": null,
"pretty_name": "Instance Administrator", "insert_time": 1660830450590, "update_time": 1661171631589, "user_email": [
"user1@acme.com", "user2@acme.com", "user3@acme.com", "user4@acme.com" ], "source": "Custom" } ] }
```


Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

2.1.8.6 | Set a User Role

post /public_api/v1/rbac/set_user_role

Add or remove one or more users from a role.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/rbac/set_user_role'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"user_emails\":
[\"string\"],\"role_name\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/rbac/set_user_role",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/rbac/set_user_role")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"user_emails\":
[\"string\"],\"role_name\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "user_emails": [ "string" ], "role_name": "string" } }); const xhr =
new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/rbac/set_user_role"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/rbac/set_user_role")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"user_emails\":[\"string\"],\"role_name\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "user_emails": ["string"], "role_name":
"string" ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let
request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/rbac/set_user_role")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/rbac/set_user_role", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"user_emails\":[\"string\"],\"role_name\":\"string\"}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/rbac/set_user_role"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"user_emails\":
[\"string\"],\"role_name\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/rbac/set_user_role"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"user_emails\":[\"string\"],\"role_name\":\"string\"}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```
Body parameters

application/json
▼ request_dataobject

A dictionary containing the API request fields.

▶ user_emailsarray[string]

List of one or more user emails of users you want to add to or remove from a role.

▶ role_namestring

Name of the role you want to add a user to. Send an empty field to remove the user.

REQUEST Example 1

```
{ "request_data": { "user_emails": [ "user1@acme.com", "user2@acme.com" ], "role_name": "Role1" } }
```

Responses

OK

Body
application/json
▼ replyobject

JSON object containing the query result.

▶ update_countstring

Number of updated users.

RESPONSE Example 1

```
{ "reply": { "update_count": "2" } }
```


Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal Server Error

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.8.7 | Get Risk Score

post /public_api/v1/get_risk_score

Retrieve the risk score of a specific user or endpoint in your environment, along with the reason for the score.

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/get_risk_score'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"id\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE",
'content-type': "application/json" } conn.request("POST", "/public_api/v1/get_risk_score", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/get_risk_score")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"id\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "id": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/get_risk_score");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/get_risk_score")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"id\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["id": "string"]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/get_risk_score")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/get_risk_score", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"id\":\"string\"}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/get_risk_score"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"id\":\"string\"}}");
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/get_risk_score"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"id\":\"string\"}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ idstring

Unique ID of a specific user or endpoint.

- User ID should be in the following format: `netBIOS/samAccount`
- Endpoint ID is the Cortex Agent ID.

You can only request one ID at a time.
REQUEST Example 1
```
{ "request_data": { "id": "<user or endpoint ID>" } }
```
Responses

OK

Body
application/json
▼ replyobject

JSON object containing the query result.

▶ typestring

Form of identification element.

▶ idstring

Identification value of the type field.

▶ scoreinteger

The score assigned to the type.

▶ norm_risk_scoreinteger

Normalization of the risk score.

▶ risk_levelstring (Enum)

The risk level.

▶ reasonsarray

Details describing when and which case name affected the score.

▶ emailstring

Email address.

RESPONSE Example 1
```
{ "reply": { "type": "user", "id": "user2_9fa235", "score": 100, "norm_risk_score": 800, "risk_level": "HIGH",
"reasons": [ { "date created": "2023-01-31", "description": "'Encoded VBScript executed' generated by XDR BIOC
detected on host acme3 involving user user2_9fa235", "severity": "SEV_050_CRITICAL", "status": "STATUS_010_NEW",
"points": 95 } ], "email": "user@company.com" } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.
Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.8.8 | Get Risky Users

post /public_api/v1/get_risky_users

Retrieve a list of users with the highest risk score in your environment along with the reason affecting each score.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/get_risky_users'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/get_risky_users", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/get_risky_users")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{}" response = http.request(request)
puts response.read_body
const data = JSON.stringify({}); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/get_risky_users");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/get_risky_users")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type", "application/json") .body("{}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE", "content-type": "application/json" ] let parameters = [] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/get_risky_users")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/get_risky_users", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd, CURLOPT_URL, "https://api-yourfqdn/public_api/v1/get_risky_users"); struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{}"); CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/get_risky_users"); var request = new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
Object
application/json
REQUEST

```
{}
```

Responses

OK

Body
application/json
▼ replyarray

JSON object containing the query result.

[
▶ typestring

Form of identification element.

▶ idstring

Identification value of the type field.

▶ scoreinteger

The score assigned to the user.

▶ norm_risk_scoreinteger

Normalization of the risk score.

▶ risk_levelstring (Enum)

The risk level.

▶ reasonsarray

Details describing when and which case name affected the user score.

▶ emailstring

Email address.

]
RESPONSE Example 1

```
{ "reply": [ { "type": "user", "id": "acme1_9fa235", "score": 100, "norm_risk_score": 800, "risk_level": "HIGH", "reasons": [ { "date created": "2023-01-31", "description": "'Encoded VBScript executed' generated by XDR BIOC detected on host acme_agent-6a7b involving user acme1_9fa235", "severity": "SEV_050_CRITICAL", "status": "STATUS_010_NEW", "points": 95 } ], "email": "user@company.com" }, { "type": "user", "id": "acme2_d40b59", "score": 100, "norm_risk_score": 800, "risk_level": "HIGH", "reasons": [ { "date created": "2023-01-26", "description": "'Kernel Privilege Escalation' along with 4 other alerts generated by XDR Analytics BIOC, XDR Analytics and XDR Agent detected on 3 hosts involving 3 users", "severity": "SEV_040_HIGH", "status": "STATUS_010_NEW", "points": 95 } ] }, { "type": "user", "id": "acme3_3a509d", "score": 15, "norm_risk_score": 100, "risk_level": "LOW", "reasons": [ { "date created": "2023-02-13", "description": "'First successful SSO connection from a country in organization' generated by XDR Analytics BIOC involving user acme3", "severity": "SEV_020_LOW", "status": "STATUS_010_NEW", "points": 15 } ], "email": "user@company.com" }, { "type": "user", "id": "acme4_207a4e", "score": 0, "norm_risk_score": 0, "risk_level": "LOW", "reasons": [], "email": "user@company.com" } ] }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.8.9 | Get Risky Hosts

post /public_api/v1/get_risky_hosts

Retrieve a list of endpoints with the highest risk score in your environment along with the reason for each score.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/get_risky_hosts'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/get_risky_hosts", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/get_risky_hosts")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{}" response = http.request(request)
puts response.read_body
const data = JSON.stringify({}); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/get_risky_hosts");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/get_risky_hosts")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type", "application/json") .body("{}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE", "content-type": "application/json" ] let parameters = [] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/get_risky_hosts")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
```

```
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/get_risky_hosts", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type:
application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/get_risky_hosts"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/get_risky_hosts"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

Object

application/json

REQUEST

```
{}
```

Responses

OK

Body

application/json

▼ replyarray

JSON object containing the query result.

[

▶ typestring

Form of identification element.

▶ idstring

Identification value of the type field.

▶ scoreinteger

The score assigned to the endpoint.

▶ norm_risk_scoreinteger

Normalization of the risk score.

▶ risk_levelstring (Enum)

The risk level.

▶ reasonsarray

Details describing when and which case name affected the endpoint score.

]

RESPONSE Example 1

```
{ "reply": [ { "type": "host", "id": "host1", "score": 100, "norm_risk_score": 800, "risk_level": "HIGH", "reasons":
[ { "date created": "2023-01-26", "description": "'Kernel Privilege Escalation' along with 4 other alerts generated
by XDR Analytics BIOC, XDR Analytics and XDR Agent detected on 3 hosts involving 3 users", "severity":
"SEV_040_HIGH", "status": "STATUS_010_NEW", "points": 95 } ] } ] }
```

Bad Request. Got an invalid JSON.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.
Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.9 | API keys

APIs for managing API keys

2.1.9.1 | Get existing API keys

post /public_api/v1/api_keys/get_api_keys

Get a list of API keys filtered by expiration date, role, or ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ authorization String required

api_key

Example:
DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7Gr
▼ x-xdr-auth-id String required

api_key_id

Example: 2841
▼ x-xdr-timestamp String required

timestamp in milliseconds

Example: xXdrTimestamp_example
▼ x-xdr-nonce String required

64 byte random string

Example: xXdrNonce_example
▼ x-child-tenant-id String required

child tenant ID

Example: xChildTenantId_example
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'authorization:
DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7Gr
-H 'x-xdr-auth-id: 2841' -H 'x-xdr-timestamp: xXdrTimestamp_example' -H 'x-xdr-nonce: xXdrNonce_example' -H 'x-child-
tenant-id: xChildTenantId_example'
'https://api-yourfqdn/public_api/v1/api_keys/get_api_keys'
```

```
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"expiration\",\"operator\":\"gte\",\"value\":0}]}}" headers = { 'authorization':
"DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7G
'x-xdr-auth-id': "2841", 'x-xdr-timestamp': "SOME_STRING_VALUE", 'x-xdr-nonce': "SOME_STRING_VALUE", 'x-child-tenant-
id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/api_keys/get_api_keys", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/api_keys/get_api_keys") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["authorization"] =
'DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7G
request["x-xdr-auth-id"] = '2841' request["x-xdr-timestamp"] = 'SOME_STRING_VALUE' request["x-xdr-nonce"] =
'SOME_STRING_VALUE' request["x-child-tenant-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"expiration\",\"operator\":\"gte\",\"value\":0}]}}"
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "expiration", "operator": "gte", "value": 0 }
] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange",
function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST",
"https://api-yourfqdn/public_api/v1/api_keys/get_api_keys"); xhr.setRequestHeader("authorization",
"DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7G
xhr.setRequestHeader("x-xdr-auth-id", "2841"); xhr.setRequestHeader("x-xdr-timestamp", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-nonce", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-child-tenant-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/api_keys/get_api_keys")
.header("authorization",
"DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7G
.header("x-xdr-auth-id", "2841") .header("x-xdr-timestamp", "SOME_STRING_VALUE") .header("x-xdr-nonce",
"SOME_STRING_VALUE") .header("x-child-tenant-id", "SOME_STRING_VALUE") .header("content-type", "application/json")
.body("{\"request_data\":{\"filters\":[{\"field\":\"expiration\",\"operator\":\"gte\",\"value\":0}]}}") .asString();
import Foundation let headers = [ "authorization":
"DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7G
"x-xdr-auth-id": "2841", "x-xdr-timestamp": "SOME_STRING_VALUE", "x-xdr-nonce": "SOME_STRING_VALUE", "x-child-tenant-
id": "SOME_STRING_VALUE", "content-type": "application/json" ] let parameters = ["request_data": ["filters": [ [
"field": "expiration", "operator": "gte", "value": 0 ] ]]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/api_keys/get_api_keys")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/api_keys/get_api_keys", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"expiration\",\"operator\":\"gte\",\"value\":0}]}}", CURLOPT_HTTPHEADER => [ "authorization:
DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7Gm
"content-type: application/json", "x-child-tenant-id: SOME_STRING_VALUE", "x-xdr-auth-id: 2841", "x-xdr-nonce:
SOME_STRING_VALUE", "x-xdr-timestamp: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/api_keys/get_api_keys"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "authorization:
DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7Gm
headers = curl_slist_append(headers, "x-xdr-auth-id: 2841"); headers = curl_slist_append(headers, "x-xdr-timestamp:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-nonce: SOME_STRING_VALUE"); headers =
curl_slist_append(headers, "x-child-tenant-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-
type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"expiration\",\"operator\":\"gte\",\"value\":0}]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/api_keys/get_api_keys"); var request = new
RestRequest(Method.POST); request.AddHeader("authorization",
"DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7G
request.AddHeader("x-xdr-auth-id", "2841"); request.AddHeader("x-xdr-timestamp", "SOME_STRING_VALUE");
request.AddHeader("x-xdr-nonce", "SOME_STRING_VALUE"); request.AddHeader("x-child-tenant-id", "SOME_STRING_VALUE");
request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{\"request_data\":
{\"filters\":[{\"field\":\"expiration\",\"operator\":\"gte\",\"value\":0}]}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters
application/json
▼ request_dataobject
▶ filtersarray

An array of filter fields.

REQUEST Get API keys and filter by expiration
```
{ "request_data": { "filters": [ { "field": "expiration", "operator": "gte", "value": 1721149909250 } ] } }
{ "request_data": { "filters": [ { "field": "roles", "operator": "contains", "value": [ "Public API Action" ] } ] } }
{ "request_data": { "filters": [ { "field": "id", "operator": "in", "value": [ 85 ] } ] } }
```
Responses

OK

Body
application/json
▼ replyobject
▶ dataarray
▶ filter_countinteger
▶ total_countinteger

Note: The `total_count` value contains all API Keys, including ones that have expired.

RESPONSE Example 1

```
{ "reply": { "DATA": [ { "id": 25, "creation_time": 1709544947832, "created_by": "N/A", "user_name": "Public API - 21", "roles": [ "default" ], "security_level": "standard", "comment": null, "expiration": 1710149747184 }, { "id": 24, "creation_time": 1709544787367, "created_by": "N/A", "user_name": "Public API - 21", "roles": [ "default" ], "security_level": "standard", "comment": null, "expiration": 1710149586852 }, { "id": 23, "creation_time": 1709544227320, "created_by": "N/A", "user_name": "Public API - 21", "roles": [ "default" ], "security_level": "standard", "comment": null, "expiration": 1710149026834 } ], "FILTER_COUNT": 3, "TOTAL_COUNT": 70 } }
```

2.1.9.2 | Generate an API key

post /public_api/v1/api_keys/generate

Generate a new API key and define the roles assigned to it and whether the security level is standard or advanced.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ authorization String required

api_key

Example:

`DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7G`

▼ x-xdr-auth-id String required

api_key_id

Example: `2841`
▼ x-xdr-timestamp String required

timestamp in milliseconds

Example: `xXdrTimestamp_example`
▼ x-xdr-nonce String required

64 byte random string

Example: `xXdrNonce_example`
▼ x-child-tenant-id String required

child tenant ID

Example: `xChildTenantId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'authorization:
DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7G
```

```
-H 'x-xdr-auth-id: 2841' -H 'x-xdr-timestamp: xXdrTimestamp_example' -H 'x-xdr-nonce: xXdrNonce_example' -H 'x-child-
tenant-id: xChildTenantId_example'
'https://api-yourfqdn/public_api/v1/api_keys/generate'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"roles\":
[\"string\"],\"security_level\":\"standard\",\"expiration\":0,\"comment\":\"string\"}}" headers = { 'authorization':
"DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7
'x-xdr-auth-id': "2841", 'x-xdr-timestamp': "SOME_STRING_VALUE", 'x-xdr-nonce': "SOME_STRING_VALUE", 'x-child-tenant-
id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/api_keys/generate", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/api_keys/generate")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["authorization"] =
'DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7
request["x-xdr-auth-id"] = '2841' request["x-xdr-timestamp"] = 'SOME_STRING_VALUE' request["x-xdr-nonce"] =
'SOME_STRING_VALUE' request["x-child-tenant-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"roles\":
[\"string\"],\"security_level\":\"standard\",\"expiration\":0,\"comment\":\"string\"}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "roles": [ "string" ], "security_level": "standard", "expiration": 0,
"comment": "string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/api_keys/generate");
xhr.setRequestHeader("authorization",
"DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7
xhr.setRequestHeader("x-xdr-auth-id", "2841"); xhr.setRequestHeader("x-xdr-timestamp", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-nonce", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-child-tenant-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/api_keys/generate")
.header("authorization",
"DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7
.header("x-xdr-auth-id", "2841") .header("x-xdr-timestamp", "SOME_STRING_VALUE") .header("x-xdr-nonce",
"SOME_STRING_VALUE") .header("x-child-tenant-id", "SOME_STRING_VALUE") .header("content-type", "application/json")
.body("{\"request_data\":{\"roles\":
[\"string\"],\"security_level\":\"standard\",\"expiration\":0,\"comment\":\"string\"}}") .asString();
import Foundation let headers = [ "authorization":
"DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7
"x-xdr-auth-id": "2841", "x-xdr-timestamp": "SOME_STRING_VALUE", "x-xdr-nonce": "SOME_STRING_VALUE", "x-child-tenant-
id": "SOME_STRING_VALUE", "content-type": "application/json" ] let parameters = ["request_data": [ "roles":
["string"], "security_level": "standard", "expiration": 0, "comment": "string" ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/api_keys/generate")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/api_keys/generate", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"roles\":
[\"string\"],\"security_level\":\"standard\",\"expiration\":0,\"comment\":\"string\"}}", CURLOPT_HTTPHEADER => [
"authorization:
DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7Gm
"content-type: application/json", "x-child-tenant-id: SOME_STRING_VALUE", "x-xdr-auth-id: 2841", "x-xdr-nonce:
SOME_STRING_VALUE", "x-xdr-timestamp: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/api_keys/generate"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "authorization:
DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7Gm
headers = curl_slist_append(headers, "x-xdr-auth-id: 2841"); headers = curl_slist_append(headers, "x-xdr-timestamp:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-nonce: SOME_STRING_VALUE"); headers =
curl_slist_append(headers, "x-child-tenant-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-
type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":{\"roles\":
[\"string\"],\"security_level\":\"standard\",\"expiration\":0,\"comment\":\"string\"}}"); CURLcode ret =
curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/api_keys/generate"); var request = new
RestRequest(Method.POST); request.AddHeader("authorization",
"DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7G
request.AddHeader("x-xdr-auth-id", "2841"); request.AddHeader("x-xdr-timestamp", "SOME_STRING_VALUE");
request.AddHeader("x-xdr-nonce", "SOME_STRING_VALUE"); request.AddHeader("x-child-tenant-id", "SOME_STRING_VALUE");
request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{\"request_data\":
{\"roles\":[\"string\"],\"security_level\":\"standard\",\"expiration\":0,\"comment\":\"string\"}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

▶ rolesarray[string]

A list of roles to be assigned to the API key.

▶ security_levelobject (Enum)

The security level of the API key. API keys with advanced security are hashed with a nonce and timestamp, which is useful for proprietary scripts and are intended to prevent replay attacks. Standard security API keys can be used as-is and are suitable for curl.

▶ expirationinteger

Integer in timestamp epoch milliseconds. Default value is one week from the time of the API call. Maximum expiration date is six months from the time of the API call.

▶ commentstring

REQUEST Generate a standard API key

```
{ "request_data": { "roles": [ "Public API Action" ], "security_level": "standard", "comment": "Api key for John" } }
{ "request_data": { "roles": [ "Public API Action" ], "security_level": "standard", "expiration": 1725802080000,
"comment": "API key for Joe" } }
```

Responses

OK

Body

application/json

▼ replyobject

▶ idinteger

▶ keystring

RESPONSE Standard API key

```
{ "reply": { "id": 4267, "key": "oZFaXmXCLCdoWIXTb9ITSqUvl4OBLsUQgCeuZc5FdwMpmpY5QBn8OUePZiLMb" } }
```

2.1.9.3 | Delete API keys

post /public_api/v1/api_keys/delete

Delete API keys by ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers

▼ authorization String required

api_key

Example: DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7G

▼ x-xdr-auth-id String required

api_key_id

Example: 2841

▼ x-xdr-timestamp String required

timestamp in milliseconds

Example: xXdrTimestamp_example

▼ x-xdr-nonce String required

64 byte random string

Example: xXdrNonce_example

▼ x-child-tenant-id String required

child tenant ID
Example: `xChildTenantId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'authorization:
DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7Gr
-H 'x-xdr-auth-id: 2841' -H 'x-xdr-timestamp: xXdrTimestamp_example' -H 'x-xdr-nonce: xXdrNonce_example' -H 'x-child-
tenant-id: xChildTenantId_example'
'https://api-yourfqdn/public_api/v1/api_keys/delete'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"id\",\"operator\":\"in\",\"value\":[0]}]}}" headers = { 'authorization':
"DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7G
'x-xdr-auth-id': "2841", 'x-xdr-timestamp': "SOME_STRING_VALUE", 'x-xdr-nonce': "SOME_STRING_VALUE", 'x-child-tenant-
id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/api_keys/delete",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/api_keys/delete")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["authorization"] =
'DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7G
request["x-xdr-auth-id"] = '2841' request["x-xdr-timestamp"] = 'SOME_STRING_VALUE' request["x-xdr-nonce"] =
'SOME_STRING_VALUE' request["x-child-tenant-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"id\",\"operator\":\"in\",\"value\":[0]}]}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "id", "operator": "in", "value": [ 0 ] } ] }
}); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function
() { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/api_keys/delete"); xhr.setRequestHeader("authorization",
"DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7G
xhr.setRequestHeader("x-xdr-auth-id", "2841"); xhr.setRequestHeader("x-xdr-timestamp", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-nonce", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-child-tenant-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/api_keys/delete")
.header("authorization",
"DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7G
.header("x-xdr-auth-id", "2841") .header("x-xdr-timestamp", "SOME_STRING_VALUE") .header("x-xdr-nonce",
"SOME_STRING_VALUE") .header("x-child-tenant-id", "SOME_STRING_VALUE") .header("content-type", "application/json")
.body("{\"request_data\":{\"filters\":[{\"field\":\"id\",\"operator\":\"in\",\"value\":[0]}]}}") .asString();
import Foundation let headers = [ "authorization":
"DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7G
"x-xdr-auth-id": "2841", "x-xdr-timestamp": "SOME_STRING_VALUE", "x-xdr-nonce": "SOME_STRING_VALUE", "x-child-tenant-
id": "SOME_STRING_VALUE", "content-type": "application/json" ] let parameters = ["request_data": ["filters": [ [
"field": "id", "operator": "in", "value": [0] ] ]]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/api_keys/delete")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/api_keys/delete", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":[{\"field\":\"id\",\"operator\":\"in\",\"value\":[0]}]}}",
CURLOPT_HTTPHEADER => [ "authorization:
DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7Gr
"content-type: application/json", "x-child-tenant-id: SOME_STRING_VALUE", "x-xdr-auth-id: 2841", "x-xdr-nonce:
SOME_STRING_VALUE", "x-xdr-timestamp: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/api_keys/delete"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "authorization:
DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7Gr
headers = curl_slist_append(headers, "x-xdr-auth-id: 2841"); headers = curl_slist_append(headers, "x-xdr-timestamp:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-nonce: SOME_STRING_VALUE"); headers =
curl_slist_append(headers, "x-child-tenant-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-
type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
```

```
CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":[{\"field\":\"id\",\"operator\":\"in\",\"value\":[0]}]}}");
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/api_keys/delete"); var request = new
RestRequest(Method.POST); request.AddHeader("authorization",
"DCdIeow0xm73EwnxjPza1tdHTfZQv2eH7bTKlTPkgBHLj8aSjFzjgTE9bQUK1DidlWLrnYRhaYQ4PCIyNrNJbMUC6DOWi8ANIn1JWpMTE2neGvoDIRsKUbj6pJ1z7
request.AddHeader("x-xdr-auth-id", "2841"); request.AddHeader("x-xdr-timestamp", "SOME_STRING_VALUE");
request.AddHeader("x-xdr-nonce", "SOME_STRING_VALUE"); request.AddHeader("x-child-tenant-id", "SOME_STRING_VALUE");
request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{\"request_data\":
{\"filters\":[{\"field\":\"id\",\"operator\":\"in\",\"value\":[0]}]}}", ParameterType.RequestBody); IRestResponse
response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ filtersarray

An array of filter fields.

REQUEST Example 1

```
{ "request_data": { "filters": [ { "field": "id", "operator": "in", "value": [ 121, 134 ] } ] } }
```

Responses

OK

Body

application/json

▼ replyobject

▶ update_countinteger

The number of API keys deleted.

RESPONSE Example 1

```
{ "reply": { "update_count": 2 } }
```

2.1.10 | Attack surface management

APIs for attack surface management

2.1.10.1 | Get External Service

post /public_api/v1/assets/get_external_service

Get service details according to the service ID. You can send up to 20 IDs.

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/assets/get_external_service'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"service_id_list\":[\"string\"]}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/assets/get_external_service", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/assets/get_external_service") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
```

```
request.body = "{\"request_data\":{\"service_id_list\":[\"string\"]}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "service_id_list": [ "string" ] } }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/assets/get_external_service"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/assets/get_external_service")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"service_id_list\":[\"string\"]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["service_id_list": ["string"]]] as [String :
Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/assets/get_external_service")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/assets/get_external_service", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"service_id_list\":[\"string\"]}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/assets/get_external_service"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"service_id_list\":[\"string\"]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/assets/get_external_service"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"service_id_list\":[\"string\"]}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

A dictionary containing the API request fields.

▶ service_id_listarray[string]

Represents the service ID you want to get details for.

REQUEST

```
{ "request_data": { "service_id_list": [ "example" ] } }
```

Responses

OK

Body

application/json

▼ replyobject

JSON object containing the query result.

▶ detailsarray

Service details according to the service ID.

RESPONSE

```
{ "reply": { "details": [ { "service_id": "example", "service_name": "example", "service_type": "example",
"ip_address": [ "example" ], "domain": [ {} ], "externally_detected_providers": [ "example" ], "is_active":
"example", "first_observed": 0, "last_observed": 0, "port": 0, "protocol": "example", "active_classifications": [
"example" ], "inactive_classifications": [ {} ], "discovery_type": "example", "business_units": [ "example" ],
"externally_inferred_vulnerability_score": {}, "externally_inferred_cves": [ {} ], "details": { "serviceKey":
"example", "serviceKeyType": "example", "businessUnits": [ { "name": "example" } ], "providerDetails": [ { "name":
"example", "firstObserved": 0, "lastObserved": 0 } ], "certificates": [ { "certificate": { "issuer": "example",
"issuerAlternativeNames": "example", "issuerCountry": "example", "issuerEmail": {}, "issuerLocality": "example",
```

"issuerName": "example", "issuerOrg": "example", "formattedIssuerOrg": "example", "issuerOrgUnit": "example", "issuerState": "example", "publicKey": "example", "publicKeyAlgorithm": "example", "publicKeyRsaExponent": 0, "signatureAlgorithm": "example", "subject": "example", "subjectAlternativeNames": "example", "subjectCountry": "example", "subjectEmail": "example", "subjectLocality": "example", "subjectName": "example", "subjectOrg": "example", "subjectOrgUnit": "example", "subjectState": "example", "serialNumber": "example", "validNotBefore": 0, "validNotAfter": 0, "version": "example", "publicKeyBits": 0, "publicKeyModulus": "example", "publicKeySpki": "example", "sha1Fingerprint": "example", "sha256Fingerprint": "example", "md5Fingerprint": "example" }, "activityStatus": "example", "lastObserved": 0, "firstObserved": 0 }], "domains": [{}], "ips": [{ "ip": "example", "protocol": "example", "provider": "example", "geolocation": { "latitude": 0, "longitude": 0, "countryCode": "example", "city": "example", "regionCode": "example", "timeZone": {} }, "activityStatus": "example", "lastObserved": 0, "firstObserved": 0 }], "classifications": [{ "name": "example", "activityStatus": "example", "values": [{ "jsonValue": "example", "firstObserved": 0, "lastObserved": 0 }], "firstObserved": 0, "lastObserved": 0 }], "tlsVersions": [{ "tlsVersion": "example", "cipherSuite": "example", "firstObserved": 0, "lastObserved": 0, "activityStatus": "example" }], "inferredCvesObserved": [{}], "enrichedObservationSource": "example", "ip_ranges": {} } }] } }

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.
Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.10.2 | Get All Services

post /public_api/v1/assets/get_external_services

Get a complete or filtered list of all your external services.

The maximum result limit is 500.

Request headers
▼ authorization String required

api-key

Example: `authorization_example`
▼ x-xdr-auth-id String required

api-key-id

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/assets/get_external_services'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
{\"field\":\"active_classifications\",\"operator\":\"contains\",\"value\":\"string\"},\"vulnerability_test_results\":true,\"sea
{\"keyword\":\"string\",\"field\":\"string\"}}}" headers = { 'authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/assets/get_external_services", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/assets/get_external_services") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":
{\"field\":\"active_classifications\",\"operator\":\"contains\",\"value\":\"string\"},\"vulnerability_test_results\":true,\"sea
{\"keyword\":\"string\",\"field\":\"string\"}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": { "field": "active_classifications", "operator":
"contains", "value": "string" }, "vulnerability_test_results": true, "search_from": 0, "search_to": 5000, "sort": {
"keyword": "string", "field": "string" } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/assets/get_external_services"); xhr.setRequestHeader("authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/assets/get_external_services")
.header("authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
{\"field\":\"active_classifications\",\"operator\":\"contains\",\"value\":\"string\"},\"vulnerability_test_results\":true,\"sea
{\"keyword\":\"string\",\"field\":\"string\"}}}") .asString();
import Foundation let headers = [ "authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ "field":
"active_classifications", "operator": "contains", "value": "string" ], "vulnerability_test_results": true,
"search_from": 0, "search_to": 5000, "sort": [ "keyword": "string", "field": "string" ] ]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/assets/get_external_services")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/assets/get_external_services", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
{\"field\":\"active_classifications\",\"operator\":\"contains\",\"value\":\"string\"},\"vulnerability_test_results\":true,\"sea
{\"keyword\":\"string\",\"field\":\"string\"}}}", CURLOPT_HTTPHEADER => [ "authorization: SOME_STRING_VALUE",
"content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/assets/get_external_services"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"filters\":
{\"field\":\"active_classifications\",\"operator\":\"contains\",\"value\":\"string\"},\"vulnerability_test_results\":true,\"sea
{\"keyword\":\"string\",\"field\":\"string\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/assets/get_external_services"); var request = new
RestRequest(Method.POST); request.AddHeader("authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":
{\"field\":\"active_classifications\",\"operator\":\"contains\",\"value\":\"string\"},\"vulnerability_test_results\":true,\"sea
{\"keyword\":\"string\",\"field\":\"string\"}}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject
▶ filtersobject

An array of filter fields.

▶ vulnerability_test_resultsboolean (Enum)

Use this field with the value `true` to get vulnerability test results for the last 14 days for each service. Using this field will slow down the endpoint.

▶ search_frominteger

An integer representing the start offset index of results.

▶ search_tointeger

An integer representing the start offset index of results. Use this field to specify the number of results on a page when using page token pagination.

▶ sortobject

Identifies the sort order for the result set.

REQUEST Example with page token

```
{ "request_data": { "filters": [ { "field": "string", "operator": "string", "value": "string" } ], "use_page_token": true } }
{ "request_data": { "filters": [ { "field": "discovery_type", "operator": "in", "value": [ "colocated_on_ip", "directly_discovery" ] }, { "field": "service_name", "operator": "contains", "value": "apache" } ], "search_from": 0, "search_to": 500 } }
```

Responses

OK

Body
application/json
▼ replyobject
▶ total_countinteger
▶ result_countinteger
▶ external_servicesarray

RESPONSE Example 1

```
{ "reply": { "total_count": 0, "result_count": 0, "external_services": [ { "service_id": "string", "service_name": "string", "service_type": "string", "ip_address": [ "string" ], "domain": [ "string" ], "externally_detected_providers": [ "string" ], "is_active": "string", "first_observed": 0, "last_observed": 0, "port": 0, "protocol": "string", "active_classifications": [ "string" ], "inactive_classifications": [ "string" ], "discovery_type": "string", "business_units": [ "string" ], "externally_inferred_vulnerability_score": "null", "externally_inferred_cves": [ {} ], "tls_versions": [ {} ], "inferred_cves_observed": [ {} ], "cloud_management_status": null } ] } }
{ "reply": { "total_count": 2, "result_count": 2, "external_services": [ { "service_id": "<service_id>", "service_name": "Server", "service_type": "AServer", "ip_address": [ "<ip_address>" ], "domain": [], "externally_detected_providers": [ "On Prem" ], "is_active": "Active", "first_observed": 1647152340000, "last_observed": 1649499420000, "port": 8009, "protocol": "TCP", "active_classifications": [ "AServer" ], "inactive_classifications": [], "discovery_type": "DirectlyDiscovered", "business_units": [ [ { "creation_time": 1684197662636, "family": "business_units", "family_alias": "BU", "id": "BU:<id>", "name": "Business Unit 1", "parent_id": null, "update_time": 1684197662636 } ] ], "externally_inferred_vulnerability_score": "null", "externally_inferred_cves": [], "tls_versions": [], "inferred_cves_observed": [] }, { "service_id": "<service_id>", "service_name": "HTTP Server", "service_type": "HttpServer", "ip_address": "ip_address", "domain": [ "email.test.org" ], "externally_detected_providers": [ "Demo" ], "is_active": "Inactive", "first_observed": 1647087420000, "last_observed": 1647087420000, "port": 80, "protocol": "TCP", "active_classifications": [], "inactive_classifications": [ "HttpServer", "NginxWebServer", "ServerSoftware" ], "discovery_type": "ColocatedOnIp", "business_units": [ "Test - Import-Export" ], "externally_inferred_vulnerability_score": null, "externally_inferred_cves": [], "cloud_management_status": null } ] } }
```

Bad Request. Got invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.10.3 | Get vulnerability tests

post /public_api/v1/assets/get_vulnerability_tests

Get a complete or filtered list of vulnerability tests. Results include details about each test, including the number of services confirmed vulnerable.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/assets/get_vulnerability_tests'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"string\",\"value\":\"string\"}]}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/assets/get_vulnerability_tests", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/assets/get_vulnerability_tests") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"name\",\"operator\":\"string\",\"value\":\"string\"}]}}"
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "name", "operator": "string", "value":
"string" } ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/assets/get_vulnerability_tests"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/assets/get_vulnerability_tests")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"string\",\"value\":\"string\"}]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["filters": [ [ "field": "name", "operator":
"string", "value": "string" ] ]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/assets/get_vulnerability_tests")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/assets/get_vulnerability_tests", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"string\",\"value\":\"string\"}]}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/assets/get_vulnerability_tests"); struct curl_slist *headers = NULL;
```

```
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\": {\"filters\":[{\"field\":\"name\",\"operator\":\"string\",\"value\":\"string\"}]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/assets/get_vulnerability_tests"); var request = new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{\"request_data\":{\"filters\":[{\"field\":\"name\",\"operator\":\"string\",\"value\":\"string\"}]}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ filtersarray

An array of filter fields.

REQUEST Example

```
{ "request_data": { "filters": [ { "field": "name", "operator": "contains", "value": "apache" } ] } }
```

Responses

OK

Body

application/json

▼ replyobject

▶ total_countinteger

▶ result_countinteger

▶ vulnerability_testsarray

RESPONSE Example

```
{ "reply": { "total_count": 1, "result_count": 1, "vulnerability_tests": [ { "id": "69527826-e1c5-42d8-b8d8-2c2005b75cbe", "name": "Apache Solr DataImportHandler Code Injection Vulnerability", "vulnerability_ids": [ "CVE-2019-0193" ], "description": "Apache Solr, a popular open-source search platform built on Apache Lucene, is affected by a remote code execution vulnerability. Solr's DataImportHandler (DIH), an optional module widely used to import data from databases and other sources, allows the entire DIH configuration to come from a request's \"dataConfig\" parameter. The debug mode of the DIH admin screen uses this feature for convenient debugging and development of a DIH configuration. However, since a DIH configuration can contain scripts, this parameter poses a security risk. The affected products can potentially be exposed to the public internet, making them vulnerable to exploitation.\n", "status": "DISABLED", "vendor_names": [ "apache" ], "affected_software": [ { "NAME": "cpe:2.3:a:apache:solr:*:*:*:*:*:*:*:*", "VERSION_START_INCLUDING": null, "VERSION_START_EXCLUDING": null, "VERSION_END_INCLUDING": null, "VERSION_END_EXCLUDING": "8.2.0", "VENDOR": "apache", "PRODUCT": "solr", "VERSION": "*" } ], "severity_score": 7.2, "cwe_ids": [ "CWE-94" ], "epss_score": 0.9605, "references": [ "https://issues.apache.org/jira/browse/SOLR-13669" ], "remediation_guidance": "Exploiting this vulnerability may lead to remote code execution, which could compromise the security and integrity of the affected system. To address this issue, follow these steps:\n\n1. Upgrade to Apache Solr 8.2.0 or later, which is secure by default.\n2. Alternatively, edit the solrconfig.xml to configure all DataImportHandler usages with an \"invariants\" section listing the \"dataConfig\" parameter set to an empty string.\n3. Ensure your network settings are configured so that only trusted traffic communicates with Solr, especially to the DIH request handler. This is a best practice for all Solr installations.\n\nBy implementing these fixes and mitigations, you can protect your Apache Solr installation from the remote code execution vulnerability described in CVE-2019-0193.\n", "first_published": 1699326060000, "created": 1711058940000, "count_vulnerable_services": null } ] } }
```

2.1.10.4 | Bulk Update Vulnerability Tests

post /public_api/v1/assets/bulk_update_vulnerability_tests

Enable or disable vulnerability tests.

To view vulnerability test results, use the Get All Services or Get Service Details endpoints.

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: xXdrAuthId_example

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/assets/bulk_update_vulnerability_tests'
-d ''
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"test_names\":
[\"string\"],\"status\":\"Enabled\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/assets/bulk_update_vulnerability_tests", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
```

```
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/assets/bulk_update_vulnerability_tests") http = Net::HTTP.new(url.host, url.port) http.use_ssl
= true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"test_names\":[\"string\"],\"status\":\"Enabled\"}}" response =
http.request(request) puts response.read_body
```

```
const data = JSON.stringify({ "request_data": { "test_names": [ "string" ], "status": "Enabled" } }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/assets/bulk_update_vulnerability_tests"); xhr.setRequestHeader("Authorization",
"SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-
type", "application/json"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.post("https://api-
yourfqdn/public_api/v1/assets/bulk_update_vulnerability_tests") .header("Authorization", "SOME_STRING_VALUE")
.header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"test_names\":[\"string\"],\"status\":\"Enabled\"}}") .asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "test_names": ["string"], "status":
"Enabled" ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let
request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/assets/bulk_update_vulnerability_tests")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/assets/bulk_update_vulnerability_tests", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING =>
"", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"test_names\":
[\"string\"],\"status\":\"Enabled\"}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type:
application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/assets/bulk_update_vulnerability_tests"); struct curl_slist *headers
= NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers =
curl_slist_append(headers, "x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":{\"test_names\":[\"string\"],\"status\":\"Enabled\"}}"); CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/assets/bulk_update_vulnerability_tests"); var request
= new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-
auth-id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"test_names\":[\"string\"],\"status\":\"Enabled\"}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ test_namesarray[string]

Names of tests, for example ["test1", "test2", "test3"]

▶ statusstring (Enum)

REQUEST Example 1

```
{ "request_data": { "test_names": [ "Apache Solr DataImportHandler Code Injection Vulnerability" ], "status":
"Enabled" } }
```

Responses

OK

2.1.11 | XQL user datasets

APIs for managing XQL user datasets.

2.1.11.1 | Define an XQL user dataset

post /public_api/v1/dataset/define_dataset

Define an XQL user dataset based on an existing BigQuery table created by the user.

Note: BigQuery table must be an existing table under public_access_user.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

These APIs are only applicable from within the XSIAM Notebook environment.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/dataset/define_dataset'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"table_name\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE",
'content-type': "application/json" } conn.request("POST", "/public_api/v1/dataset/define_dataset", payload, headers)
res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/dataset/define_dataset") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"table_name\":\"string\"}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "table_name": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/dataset/define_dataset"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/dataset/define_dataset")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"table_name\":\"string\"}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["table_name": "string"]] as [String : Any]
let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/dataset/define_dataset")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/dataset/define_dataset", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"table_name\":\"string\"}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/dataset/define_dataset"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-
id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
```

```
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"table_name\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/dataset/define_dataset"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"table_name\":\"string\"}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

▶ table_namestring

An existing BigQuery table name that was created by the user.

REQUEST

```
{ "request_data": { "table_name": "example" } }
```

Responses

OK

2.1.11.2 | Get created XQL user datasets

post /public_api/v1/dataset/get_created_datasets

Retrieve a list of all XQL user datasets created using the Cortex SDK.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

These APIs are only applicable from within the XSIAM Notebook environment.

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/dataset/get_created_datasets'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/dataset/get_created_datasets", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/dataset/get_created_datasets") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{}" response = http.request(request) puts response.read_body
const data = JSON.stringify({}); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/dataset/get_created_datasets"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/dataset/get_created_datasets")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = [] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/dataset/get_created_datasets")! as URL, cachePolicy: .useProtocolCachePolicy,
```

```
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/dataset/get_created_datasets", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type:
application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/dataset/get_created_datasets"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/dataset/get_created_datasets"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

Object

application/json

REQUEST

```
{}
```

Responses

OK

Body

application/json

▼ datasetsarray[string]

A list of created datasets.

RESPONSE

```
{ "datasets": [ "example" ] }
```

2.1.11.3 | Delete an XQL user dataset

post /public_api/v1/dataset/delete_dataset

Delete an XQL user dataset that was created by the Cortex SDK.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

These APIs are only applicable from within the XSIAM Notebook environment.

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/dataset/delete_dataset'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"dataset_name\":\"string\",\"delete_underlying_bq_table\":false}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
```

```
"/public_api/v1/dataset/delete_dataset", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/dataset/delete_dataset") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"dataset_name\":\"string\",\"delete_underlying_bq_table\":false}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "dataset_name": "string", "delete_underlying_bq_table": false } });
const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () {
if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/dataset/delete_dataset"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/dataset/delete_dataset")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"dataset_name\":\"string\",\"delete_underlying_bq_table\":false}}")
.asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "dataset_name": "string",
"delete_underlying_bq_table": false ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject:
parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/dataset/delete_dataset")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/dataset/delete_dataset", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"dataset_name\":\"string\",\"delete_underlying_bq_table\":false}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/dataset/delete_dataset"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-
id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"dataset_name\":\"string\",\"delete_underlying_bq_table\":false}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/dataset/delete_dataset"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"dataset_name\":\"string\",\"delete_underlying_bq_table\":false}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

▶ dataset_namestring

The dataset name to be deleted.

▶ delete_underlying_bq_tableboolean

Define whether or not to delete the BigQuery table related to the dataset.

REQUEST

```
{ "request_data": { "dataset_name": "example", "delete_underlying_bq_table": false } }
```

Responses

OK

Body

application/json

RESPONSE

```
{}
```

2.1.12 | Dataset Management

APIs for managing datasets

2.1.12.1 | Add Dataset

post /public_api/v1/xql/add_dataset

Add a dataset of type `lookup` with the specified name and schema.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/xql/add_dataset'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"dataset_name\":\"string\",\"dataset_type\":\"string\",\"dataset_schema\":
{\"property1\":\"text\",\"property2\":\"text\"}}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/xql/add_dataset",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/xql/add_dataset")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"dataset_name\":\"string\",\"dataset_type\":\"string\",\"dataset_schema\":
{\"property1\":\"text\",\"property2\":\"text\"}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "dataset_name": "string", "dataset_type": "string", "dataset_schema":
{ "property1": "text", "property2": "text" } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/xql/add_dataset");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/xql/add_dataset")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":
{\"dataset_name\":\"string\",\"dataset_type\":\"string\",\"dataset_schema\":
{\"property1\":\"text\",\"property2\":\"text\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "dataset_name": "string", "dataset_type":
"string", "dataset_schema": [ "property1": "text", "property2": "text" ] ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/xql/add_dataset")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/xql/add_dataset", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"dataset_name\":\"string\",\"dataset_type\":\"string\",\"dataset_schema\":
{\"property1\":\"text\",\"property2\":\"text\"}}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE",
"content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/xql/add_dataset"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
```

```
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"dataset_name\":\"string\",\"dataset_type\":\"string\",\"dataset_schema\":
{\"property1\":\"text\",\"property2\":\"text\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/xql/add_dataset"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"dataset_name\":\"string\",\"dataset_type\":\"string\",\"dataset_schema\":
{\"property1\":\"text\",\"property2\":\"text\"}}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters
application/json
▼ request_dataobjectrequired
▶ dataset_namestring

The designated name of the dataset.

▶ dataset_typestring

Dataset type. Currently only `lookup` is supported.

▶ dataset_schemaobject

The schema of the dataset in a comma-separated list of JSON pairs where the key is the field name and the value is the field type.

REQUEST Example 1
```
{ "request_data": { "dataset_name": "users", "dataset_schema": { "uid": "text", "username": "text", "zipcode":
"number", "salary": "number", "is_admin": "bool", "birthday": "datetime" }, "dataset_type": "lookup" } }
```
Responses

OK

Body
application/json
▼ dataset_namestring

Name of the dataset added.

RESPONSE
```
{ "dataset_name": "example" }
```

2.1.12.2 | Delete a dataset

post /public_api/v2/xql/delete_dataset

Delete a dataset with the specified name. The following dataset types can be deleted: Lookup, Raw, User, Snapshot, and Correlation. You can only delete a dataset with dependencies by setting `force` to TRUE.

Note: The System dataset and other protected datasets cannot be deleted.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v2/xql/delete_dataset'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"dataset_name\":\"string\",\"force\":true}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
```

```
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v2/xql/delete_dataset",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v2/xql/delete_dataset")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"dataset_name\":\"string\",\"force\":true}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "dataset_name": "string", "force": true } }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v2/xql/delete_dataset"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v2/xql/delete_dataset")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"dataset_name\":\"string\",\"force\":true}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "dataset_name": "string", "force": true ]]
as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v2/xql/delete_dataset")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v2/xql/delete_dataset", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"dataset_name\":\"string\",\"force\":true}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v2/xql/delete_dataset"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"dataset_name\":\"string\",\"force\":true}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v2/xql/delete_dataset"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"dataset_name\":\"string\",\"force\":true}}", ParameterType.RequestBody); IRestResponse response
= client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

▶ dataset_namestring

The name of the dataset to be deleted.

▶ forceboolean

Warning: Setting this to `True` forces deletion even when there are dependencies.

REQUEST Example 1

```
{ "request_data": { "dataset_name": "users", "force": "yes" } }
```

Responses

OK

2.1.12.3 | **Get all datasets**

post /public_api/v1/xql/get_datasets

Retrieve a list of all the datasets and their properties.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers

▼ Authorization String required

{api_key}
Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/xql/get_datasets'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/xql/get_datasets", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/xql/get_datasets")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": {} }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/xql/get_datasets");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/xql/get_datasets")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": []] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/xql/get_datasets")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/xql/get_datasets", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-
type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/xql/get_datasets"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/xql/get_datasets"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```
Body parameters
application/json
▼ request_dataobject
REQUEST
```
{ "request_data": {} }
```
Responses

OK

Body
application/json
▼ replyarray
[
▶ dataset_namestring

Dataset name.

▶ typestring

Dataset type. Can be one of the following: `System`, `Lookup`, `Raw`, `User`, `Snapshot`, `Correlation`, `System Audit`.

▶ log_update_typestring

Log update type. Can be one of the following: `Logs` (event logs are updated continuously), `State` (the current state is updated periodically).

▶ last_updatedinteger

Integer in timestamp epoch milliseconds. When the data in the dataset was last updated.

▶ total_days_storedinteger

Number of dats the data is stored in the tenant, which is comprised of `hot_range` + `cold_range`.

▶ hot_rangeobject

The time period of the hot storage from the start date to the end date.

▶ cold_rangeobject

The time period of the cold storage from the start date to the end date.

▶ total_size_storedinteger

Actual size of the data (in bytes) that is stored in the tenant. This number is dependent on the events stored in the hot storage. For the xdr_data dataset, where the first 31 days of storage are included with your license, the first 31 days are not included in the `total_size_stored` number.

▶ average_daily_sizeinteger

Average daily amount stored (in bytes) in the tenant. This number is dependent on the events stored in the hot storage.

▶ total_eventsinteger

Number of total events/logs that are stored in the tenant. This number is dependent on the events stored in the hot storage.

▶ average_event_sizeinteger

Average size (in bytes) of a single event in the dataset (`total_size_stored` divided by the `total_events`). This number is dependent on the events stored in the hot storage.

▶ ttlinteger

Time to live. Defines when lookup entries expire and are removed automatically from the lookup dataset.

▶ default_query_targetboolean

whether the dataset is configured to use as your default query target in XQL Search, so when you write your queries you do not need to define a dataset. Can be one of the following: `True`, `False`.

]

RESPONSE Example 1


```
{ "reply": [ { "Dataset Name": "xdr_data", "Type": "SYSTEM", "Log Update Type": "LOGS", "Last Updated": null, "Total Days Stored": null, "Hot Range": { "from": 1715299200000, "to": 1716595200000 }, "Cold Range": {}, "Total Size Stored": null, "Average Daily Size": null, "Total Events": null, "Average Event Size": null, "TTL": null, "Default Query Target": "FALSE" }, { "Dataset Name": "host_inventory", "Type": "SYSTEM", "Log Update Type": "LOGS", "Last Updated": null, "Total Days Stored": null, "Hot Range": {}, "Cold Range": {}, "Total Size Stored": null, "Average Daily Size": null, "Total Events": null, "Average Event Size": null, "TTL": null, "Default Query Target": "FALSE" }, { "Dataset Name": "host_users_to_groups", "Type": "SYSTEM", "Log Update Type": "LOGS", "Last Updated": null, "Total Days Stored": null, "Hot Range": {}, "Cold Range": {}, "Total Size Stored": null, "Average Daily Size": null, "Total Events": null, "Average Event Size": null, "TTL": null, "Default Query Target": "FALSE" } ] }
```

2.1.13 | Lookup Datasets

APIs for lookup datasets

2.1.13.1 | Add or update data in a lookup dataset

post /public_api/v1/xql/lookups/add_data

Add or update data in a lookup dataset.

When updating data, any field not specified in the `data` field, but specified on at least one of the rows, will be set to `None`.

The `/public_api/xql/lookups/add_data/` endpoint does not support concurrent edits. Sending concurrent calls to this endpoint can cause data to be unintentionally overwritten or deleted. To allow sufficient time for each API call to complete its operation before initiating another one, assume that 1000 entries can be added per API every 10 seconds.

**Note: **

- The maximum size of a lookup dataset is 50 MB. Attempting to exceed this limit will fail.
- Requests time out after three minutes.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management
Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/xql/lookups/add_data'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"dataset_name\":\"string\",\"key_fields\":[\"string\"],\"data\":
{\"property1\":\"string\",\"property2\":\"string\"}}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-
id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/xql/lookups/add_data", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/xql/lookups/add_data") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"dataset_name\":\"string\",\"key_fields\":[\"string\"],\"data\":
{\"property1\":\"string\",\"property2\":\"string\"}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "dataset_name": "string", "key_fields": [ "string" ], "data": {
"property1": "string", "property2": "string" } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/xql/lookups/add_data");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/xql/lookups/add_data")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"dataset_name\":\"string\",\"key_fields\":[\"string\"],\"data\":
{\"property1\":\"string\",\"property2\":\"string\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "dataset_name": "string", "key_fields":
["string"], "data": [ "property1": "string", "property2": "string" ] ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/xql/lookups/add_data")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/xql/lookups/add_data", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"dataset_name\":\"string\",\"key_fields\":
[\"string\"],\"data\":{\"property1\":\"string\",\"property2\":\"string\"}}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/xql/lookups/add_data"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
```

```
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"dataset_name\":\"string\",\"key_fields\":[\"string\"],\"data\":
{\"property1\":\"string\",\"property2\":\"string\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/xql/lookups/add_data"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"dataset_name\":\"string\",\"key_fields\":[\"string\"],\"data\":
{\"property1\":\"string\",\"property2\":\"string\"}}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters
application/json
▼ request_dataobjectrequired
▶ dataset_namestring

Unique dataset name

▶ key_fieldsarray[string]

The fields used to identify existing records. If there is not an exact match to the key_fields specified, a new row is created. When you specify `key_fields`, these fields are mandatory in data entries. When `key_fields` are not specified, existing data entries are not updated, and new entries are added with the specified data.

▶ dataobject

Key-value pairs of data entries.

REQUEST Example 1
```
{ "request_data": { "dataset_name": "users", "key_fields": [ "uid", "username" ], "data": [ { "uid": "123abc",
"username": "john", "zipcode": 58672, "salary": 5.1, "is_admin": false, "birthday": "31-05-1982T10:22:45Z" }, {
"uid": "124abc", "username": "jane", "zipcode": 58642, "salary": 5000000, "is_admin": true, "birthday": "31-03-
1982T10:22:45Z" } ] } }
```
Responses

OK

Body
application/json
▼ addedinteger
▼ updatedinteger
▼ skippedinteger
RESPONSE
```
{ "added": 0, "updated": 0, "skipped": 0 }
```

2.1.13.2 | Remove data from a lookup dataset

post /public_api/v1/xql/lookups/remove_data

Remove data from a dataset based on the specified parameters. If any one of the filter sets are not found, the API does not delete any data.

The `/public_api/xql/lookups/remove_data/` endpoint does not support concurrent edits. Sending concurrent calls to this endpoint can cause data to be unintentionally overwritten or deleted. To allow sufficient time for each API call to complete its operation before initiating another one, assume that 1000 entries can be added per API every 10 seconds.

Note:

- All lookup entries matching any of the filter blocks are deleted. To match a filter block, a lookup entry must match all the specified fields as if there were an **AND** operator between them.
- Requests time out after three minutes.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/xql/lookups/remove_data'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"dataset_name\":\"string\",\"filters\":{\"property1\":\"string\",\"property2\":\"string\"}}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/xql/lookups/remove_data", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/xql/lookups/remove_data") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"dataset_name\":\"string\",\"filters\":
{\"property1\":\"string\",\"property2\":\"string\"}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "dataset_name": "string", "filters": { "property1": "string",
"property2": "string" } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/xql/lookups/remove_data");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/xql/lookups/remove_data")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"dataset_name\":\"string\",\"filters\":
{\"property1\":\"string\",\"property2\":\"string\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "dataset_name": "string", "filters": [
"property1": "string", "property2": "string" ] ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/xql/lookups/remove_data")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/xql/lookups/remove_data", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"dataset_name\":\"string\",\"filters\":
{\"property1\":\"string\",\"property2\":\"string\"}}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE",
"content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/xql/lookups/remove_data"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"dataset_name\":\"string\",\"filters\":{\"property1\":\"string\",\"property2\":\"string\"}}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/xql/lookups/remove_data"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"dataset_name\":\"string\",\"filters\":{\"property1\":\"string\",\"property2\":\"string\"}}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

▶ dataset_namestring

The name of the dataset to delete.

▶ filtersobject

Key-value pairs of fields to query in datasets. A lookup entry must match all the specified fields as if there were an **AND** operator between them. You can use one or more fields, up to the number of fields in the schema.

REQUEST Example 1

```
{ "request_data": { "dataset_name": "users", "filters": [ { "uid": "123", "username": "john" }, { "uid": "124",
"zipcode": 58672 } ] } }
```

Responses

OK

Body
application/json
▼ deletedinteger

Number of entries deleted successfully.

RESPONSE
```
{ "deleted": 0 }
```

2.1.13.3 | Get data from a lookup dataset

post /public_api/v1/xql/lookups/get_data

Get data from a lookup dataset according to the specified filter fields. All lookup entries matching any of the filter blocks are returned. To match a filter block, a lookup entry must match all the specified fields as if there were an **AND** operator between them. If no filters are specified, return all lookup entries.

Note:

- The maximum number of entries returned is 10,000.
- Requests time out after three minutes.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/xql/lookups/get_data'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"dataset_name\":\"string\",\"filters\":[{\"property1\":\"string\",\"property2\":\"string\"}],\"limit\":0}}" headers
= { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/xql/lookups/get_data", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/xql/lookups/get_data") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"dataset_name\":\"string\",\"filters\":
[{\"property1\":\"string\",\"property2\":\"string\"}],\"limit\":0}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "dataset_name": "string", "filters": [ { "property1": "string",
"property2": "string" } ], "limit": 0 } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/xql/lookups/get_data");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/xql/lookups/get_data")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"dataset_name\":\"string\",\"filters\":
[{\"property1\":\"string\",\"property2\":\"string\"}],\"limit\":0}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "dataset_name": "string", "filters": [ [
```

```
"property1": "string", "property2": "string" ] ], "limit": 0 ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/xql/lookups/get_data")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/xql/lookups/get_data", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"dataset_name\":\"string\",\"filters\":
[{\"property1\":\"string\",\"property2\":\"string\"}],\"limit\":0}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/xql/lookups/get_data"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"dataset_name\":\"string\",\"filters\":[{\"property1\":\"string\",\"property2\":\"string\"}],\"limit\":0}}");
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/xql/lookups/get_data"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"dataset_name\":\"string\",\"filters\":
[{\"property1\":\"string\",\"property2\":\"string\"}],\"limit\":0}}", ParameterType.RequestBody); IRestResponse
response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

▶ dataset_namestring

Name of the dataset to query.

▶ filtersarray

Key-value pairs of fields to query in a dataset. A lookup entry must match all the specified fields as if there were an **AND** operator between them. You can use one or more fields, up to the number of fields in the schema.

▶ limitinteger

The maximum number of results to return. If this is not specified, return all lookup entries that match the filter criteria.

REQUEST Example 1

```
{ "request_data": { "dataset_name": "users", "filters": [ { "uid": "123", "username": "john" }, { "department":
"dev", "zipcode": "58674" } ], "limit": 20 } }
```

Responses

OK

Body

application/json

▼ dataobject

▶ Additional propertiesstring

▼ filter_countinteger

Number of entries that match the filter.

▼ total_countinteger

Total number of entries.

RESPONSE Example 1

```
{ "reply": { "data": [ { "uid": "uid5", "salary": 5.1, "zipcode": 70005, "birthday": 386418165000, "is_admin": true,
"username": "username5", "_insert_time": 1718807765000, "_update_time": 1718807765000, "_collector_name": "Console",
"_collector_type": "Console" }, { "uid": "uid6", "salary": 6.1, "zipcode": 70006, "birthday": 386418165000,
"is_admin": true, "username": "username6", "_insert_time": 1718807765000, "_update_time": 1718807765000,
"_collector_name": "Console", "_collector_type": "Console" } ], "filter count": 2, "total count": 10 } }
```

2.1.14 | Authentication settings

APIs for authentication settings, such as IdP and SSO

2.1.14.1 | Create authentication settings for IdP SSO or metadata URL

post /public_api/v1/authentication-settings/create

Create authentication settings for IdP SSO or metadata URL. You must include either the `metadata_url` field or all of the following fields: `idp_sso_url`, `idp_issuer`, and `idp_certificate`.

You must have **Instance Administrator** permissions to run this endpoint.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/authentication-settings/create'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"name\":\"string\",\"default_role\":\"string\",\"is_account_role\":false,\"domain\":\"string\",\"mappings\":
{\"email\":\"string\",\"firstname\":\"string\",\"lastname\":\"string\",\"group_name\":\"string\"},\"advanced_settings\":
{\"relay_state\":\"string\",\"idp_single_logout_url\":\"string\",\"service_provider_public_cert\":\"string\",\"service_provide
headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type':
"application/json" } conn.request("POST", "/public_api/v1/authentication-settings/create", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/authentication-
settings/create") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE'
request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "
{\"request_data\":
{\"name\":\"string\",\"default_role\":\"string\",\"is_account_role\":false,\"domain\":\"string\",\"mappings\":
{\"email\":\"string\",\"firstname\":\"string\",\"lastname\":\"string\",\"group_name\":\"string\"},\"advanced_settings\":
{\"relay_state\":\"string\",\"idp_single_logout_url\":\"string\",\"service_provider_public_cert\":\"string\",\"service_provide
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "name": "string", "default_role": "string", "is_account_role": false,
"domain": "string", "mappings": { "email": "string", "firstname": "string", "lastname": "string", "group_name":
"string" }, "advanced_settings": { "relay_state": "string", "idp_single_logout_url": "string",
"service_provider_public_cert": "string", "service_provider_private_key": "string", "authn_context_enabled": false,
"force_authn": false }, "idp_sso_url": "string", "idp_certificate": "string", "idp_issuer": "string", "metadata_url":
"string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange",
function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST",
"https://api-yourfqdn/public_api/v1/authentication-settings/create"); xhr.setRequestHeader("Authorization",
"SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-
type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/authentication-settings/create")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":
{\"name\":\"string\",\"default_role\":\"string\",\"is_account_role\":false,\"domain\":\"string\",\"mappings\":
{\"email\":\"string\",\"firstname\":\"string\",\"lastname\":\"string\",\"group_name\":\"string\"},\"advanced_settings\":
{\"relay_state\":\"string\",\"idp_single_logout_url\":\"string\",\"service_provider_public_cert\":\"string\",\"service_provide
.asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "name": "string", "default_role": "string",
"is_account_role": false, "domain": "string", "mappings": [ "email": "string", "firstname": "string", "lastname":
"string", "group_name": "string" ], "advanced_settings": [ "relay_state": "string", "idp_single_logout_url":
"string", "service_provider_public_cert": "string", "service_provider_private_key": "string",
"authn_context_enabled": false, "force_authn": false ], "idp_sso_url": "string", "idp_certificate": "string",
```

```
"idp_issuer": "string", "metadata_url": "string" ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/authentication-settings/create")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/authentication-settings/create", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"name\":\"string\",\"default_role\":\"string\",\"is_account_role\":false,\"domain\":\"string\",\"mappings\":
{\"email\":\"string\",\"firstname\":\"string\",\"lastname\":\"string\",\"group_name\":\"string\"},\"advanced_settings\":
{\"relay_state\":\"string\",\"idp_single_logout_url\":\"string\",\"service_provider_public_cert\":\"string\",\"service_provide
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/authentication-settings/create"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"name\":\"string\",\"default_role\":\"string\",\"is_account_role\":false,\"domain\":\"string\",\"mappings\":
{\"email\":\"string\",\"firstname\":\"string\",\"lastname\":\"string\",\"group_name\":\"string\"},\"advanced_settings\":
{\"relay_state\":\"string\",\"idp_single_logout_url\":\"string\",\"service_provider_public_cert\":\"string\",\"service_provide
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/authentication-settings/create"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":
{\"name\":\"string\",\"default_role\":\"string\",\"is_account_role\":false,\"domain\":\"string\",\"mappings\":
{\"email\":\"string\",\"firstname\":\"string\",\"lastname\":\"string\",\"group_name\":\"string\"},\"advanced_settings\":
{\"relay_state\":\"string\",\"idp_single_logout_url\":\"string\",\"service_provider_public_cert\":\"string\",\"service_provide
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

▶ namestring

The name of the SSO integration.

▶ default_rolestring

The default role automatically assigned to every user who authenticates to Cortex using SAML. This is an inherited role and is not the same as a direct role assigned to the user.

If a role with the same name exists on both Cortex Gateway and the tenant, the role will mapped to the role from the tenant. If you want to use specifically the role from Cortex Gateway, use the `is_account_role` parameter set to `true`.

▶ is_account_roleboolean

Whether the role was created in Cortex Gateway or in the tenant. When the value is `true`, the role was created in Cortex Gateway.

▶ domainstring

When configuring the first SSO, this parameter should be included as empty because it is the default SSO and has a fixed, read-only value. For additional SSOs, specify this IdP with an email domain (user@). When logging in, users are redirected to the IdP associated with their email domain or to the default IdP if no association exists.

▶ mappingsobjectrequired

These IdP attribute mappings are dependent on your organization's IdP.

▶ advanced_settingsobject

The advanced settings are optional to configure and some are specific for a particular IdP.

▶ idp_sso_urlstring

The login URL of your IdP and should be copied from your SAML integration configuration on the IdP. For example:

- Okta: https://cortex-test.okta.com/app/cortex-test/eacbt6b2jj08CasdUQ7sdf15d7/sso/SAML
- Microsoft Azure: https://login.microsoftonline.com/6a5a9780-96a4-41ef-bf45-0535d8a70025/saml2

▶ idp_certificatestring

The Idp's public X.509 digital certificate in PEM format for verification, which is copied from your organization's IdP.

▶ idp_issuerstring

The unique identifier of the IdP issuing SAML assertions, which is copied from your organization's IdP.

▶ metadata_urlstring

The metadata URL provides information about hte IdP's capabilities, endpoints, keys, and more. For example:

- Okta: https://cortex-test.okta.com/app/exkbuuzw77Bh04V6M6b8/sso/saml/metadata
- Microsoft Azure: https://login.microsoftonline.com/6a5a9780-96a4-41ef-bf45-0535d8a70025/saml2/metadata

REQUEST Example 1

```
{ "request_data": { "name": "IdP configuration", "default_role": "Analyst", "domain": "my-test-domain.com",
"mappings": { "email": "user@company.com", "firstname": "John", "lastname": "Smith", "group_name": "analysts" },
"idp_sso_url": "https://cortex-test.okta.com/app/cortex-test/xxxxxxx/sso/SAML", "idp_certificate":
"MY_CERTIFICATE_FROM_OKTA", "idp_issuer": "https://cortex-test.okta.com/idp", "advanced_settings": {},
"is_account_role": true } }
```

Responses

OK

Body
application/json
▼ replyboolean
RESPONSE Example 1

```
{ "reply": true }
```


Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.14.2 | Update authentication settings

post /public_api/v1/authentication-settings/update

Update existing authentication settings. To update the default domain, include empty value for both `current_domain_value` and `new_domain_value`.

You must have **Instance Administrator** permissions to run this endpoint.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/authentication-settings/update'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"name\":\"string\",\"default_role\":\"string\",\"is_account_role\":false,\"current_domain_value\":\"string\",\"new_domain_va
{\"email\":\"string\",\"firstname\":\"string\",\"lastname\":\"string\",\"group_name\":\"string\"},\"advanced_settings\":
{\"relay_state\":\"string\",\"idp_single_logout_url\":\"string\",\"service_provider_public_cert\":\"string\",\"service_provide
headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type':
"application/json" } conn.request("POST", "/public_api/v1/authentication-settings/update", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/authentication-
settings/update") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE'
request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "
{\"request_data\":
{\"name\":\"string\",\"default_role\":\"string\",\"is_account_role\":false,\"current_domain_value\":\"string\",\"new_domain_va
{\"email\":\"string\",\"firstname\":\"string\",\"lastname\":\"string\",\"group_name\":\"string\"},\"advanced_settings\":
{\"relay_state\":\"string\",\"idp_single_logout_url\":\"string\",\"service_provider_public_cert\":\"string\",\"service_provide
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "name": "string", "default_role": "string", "is_account_role": false,
"current_domain_value": "string", "new_domain_value": "string", "mappings": { "email": "string", "firstname":
"string", "lastname": "string", "group_name": "string" }, "advanced_settings": { "relay_state": "string",
"idp_single_logout_url": "string", "service_provider_public_cert": "string", "service_provider_private_key":
"string", "authn_context_enabled": false, "force_authn": false }, "idp_sso_url": "string", "idp_certificate":
"string", "idp_issuer": "string", "metadata_url": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/authentication-
settings/update"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/authentication-settings/update")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":
{\"name\":\"string\",\"default_role\":\"string\",\"is_account_role\":false,\"current_domain_value\":\"string\",\"new_domain_va
{\"email\":\"string\",\"firstname\":\"string\",\"lastname\":\"string\",\"group_name\":\"string\"},\"advanced_settings\":
{\"relay_state\":\"string\",\"idp_single_logout_url\":\"string\",\"service_provider_public_cert\":\"string\",\"service_provide
.asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "name": "string", "default_role": "string",
"is_account_role": false, "current_domain_value": "string", "new_domain_value": "string", "mappings": [ "email":
"string", "firstname": "string", "lastname": "string", "group_name": "string" ], "advanced_settings": [
"relay_state": "string", "idp_single_logout_url": "string", "service_provider_public_cert": "string",
"service_provider_private_key": "string", "authn_context_enabled": false, "force_authn": false ], "idp_sso_url":
"string", "idp_certificate": "string", "idp_issuer": "string", "metadata_url": "string" ]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/authentication-settings/update")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/authentication-settings/update", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"name\":\"string\",\"default_role\":\"string\",\"is_account_role\":false,\"current_domain_value\":\"string\",\"new_domain_va
{\"email\":\"string\",\"firstname\":\"string\",\"lastname\":\"string\",\"group_name\":\"string\"},\"advanced_settings\":
{\"relay_state\":\"string\",\"idp_single_logout_url\":\"string\",\"service_provider_public_cert\":\"string\",\"service_provide
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/authentication-settings/update"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"name\":\"string\",\"default_role\":\"string\",\"is_account_role\":false,\"current_domain_value\":\"string\",\"new_domain_va
{\"email\":\"string\",\"firstname\":\"string\",\"lastname\":\"string\",\"group_name\":\"string\"},\"advanced_settings\":
{\"relay_state\":\"string\",\"idp_single_logout_url\":\"string\",\"service_provider_public_cert\":\"string\",\"service_provide
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/authentication-settings/update"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":
{\"name\":\"string\",\"default_role\":\"string\",\"is_account_role\":false,\"current_domain_value\":\"string\",\"new_domain_va
{\"email\":\"string\",\"firstname\":\"string\",\"lastname\":\"string\",\"group_name\":\"string\"},\"advanced_settings\":
{\"relay_state\":\"string\",\"idp_single_logout_url\":\"string\",\"service_provider_public_cert\":\"string\",\"service_provide
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

▶ namestring

The name of the SSO integration.

▶ default_rolestring

The default role automatically assigned to every user who authenticates to Cortex using SAML. This is an inherited role and is not the same as a direct role assigned to the user.

If a role with the same name exists on both Cortex Gateway and the tenant, the role will mapped to the role from the tenant. If you want to use specifically the role from Cortex Gateway, use the `is_account_role` parameter set to `true`.

▶ is_account_roleboolean

Whether the role was created in Cortex Gateway or in the tenant. When the value is `true`, the role was created in Cortex Gateway.

▶ current_domain_valuestring

The domain whose authentication settings you want to update.

▶ new_domain_valuestring

If you want to update the domain value, include a new unique domain.

▶ mappingsobjectrequired

These IdP attribute mappings are dependent on your organization's IdP.

▶ advanced_settingsobject

The advanced settings are optional to configure and some are specific for a particular IdP.

▶ idp_sso_urlstring

The URL of your IdP's SSO, which is a fixed, read-only value based on your tenant's URL. If you are using this parameter, you must also specify: `idp_certificate` and `idp_issuer`.

▶ idp_certificatestring

The Idp's public X.509 digital certificate in PEM format for verification, which is copied from your organization's IdP.

▶ idp_issuerstring

The unique identifier of the IdP issuing SAML assertions, which is copied from your organization's IdP.

▶ metadata_urlstring

Specify your IdP SSO URL, which is a fixed, read-only value based on your tenant's URL.

REQUEST Example 1

```
{ "request_data": { "name": "IDP configuration", "default_role": "Analyst", "current_domain_value": "my-test-domain.com", "new_domain_value": "my-test-domain.org", "mappings": { "email": "user@company.com", "firstname": "John", "lastname": "Smith", "group_name": "analysts" }, "idp_sso_url": "https://cortex-test.okta.com/app/cortex-test/xxxxxxx/sso/SAML", "idp_certificate": "========MY_UPDATED_TEST_CERTIFICATE_FROM_OKTA======", "idp_issuer": "https://cortex-test.okta.com/idp", "advanced_settings": {}, "is_account_role": true } }
```

Responses

OK

Body
application/json
▼ replyboolean
RESPONSE

```
{ "reply": false }
```


Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.14.3 | Delete authentication settings by domain

post /public_api/v1/authentication-settings/delete

Delete all authentication settings for the specified domain.

**Note: ** The first configuration on the tenant is the default configuration and cannot be deleted.

You must have **Instance Administrator** permissions to run this endpoint.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/authentication-settings/delete'
-d ''
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"domain\":\"string\"}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE",
'content-type': "application/json" } conn.request("POST", "/public_api/v1/authentication-settings/delete", payload,
headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
```

```
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/authentication-
settings/delete") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE'
request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "
{\"request_data\":{\"domain\":\"string\"}}" response = http.request(request) puts response.read_body
```

```
const data = JSON.stringify({ "request_data": { "domain": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/authentication-
settings/delete"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/authentication-settings/delete")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"domain\":\"string\"}}") .asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["domain": "string"]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/authentication-settings/delete")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/authentication-settings/delete", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"domain\":\"string\"}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/authentication-settings/delete"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"domain\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/authentication-settings/delete"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"domain\":\"string\"}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ domainstring

The domain whose authentication settings you want to delete.

REQUEST Example 1

```
{ "request_data": { "domain": "my-test-domain.org" } }
```

Responses

OK

Body

application/json
▼ replyboolean
RESPONSE Example 1
{ "reply": true }

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.14.4 | Get authentication settings for all configured domains

post /public_api/v1/authentication-settings/get/settings

Get all the authentication settings for every configured domain in the tenant.

You must have **Instance Administrator** permissions to run this endpoint.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/authentication-settings/get/settings'
-d ''
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/authentication-settings/get/settings", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/authentication-
settings/get/settings") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE'
request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "
{\"request_data\":{}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": {} }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/authentication-
settings/get/settings"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-
auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/authentication-
settings/get/settings") .header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE")
.header("content-type", "application/json") .body("{\"request_data\":{}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": []] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/authentication-settings/get/settings")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/authentication-settings/get/settings", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE",
"content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/authentication-settings/get/settings"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers,
"x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/authentication-settings/get/settings"); var request =
new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-
id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{}}", ParameterType.RequestBody); IRestResponse response
= client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

REQUEST

```
{ "request_data": {} }
```

Responses

OK

Body

application/json

▼ replyarray

[

▶ tenant_idstring

▶ namestring

▶ domainstring

▶ idp_enabledboolean

▶ default_roleobject

▶ is_account_roleobject

▶ idp_certificatestring

▶ idp_issuerstring

▶ idp_sso_urlstring

▶ metadata_urlstring

▶ mappingsobject

▶ advanced_settingsobject

▶ sp_entity_idstring

▶ sp_logout_urlstring

▶ sp_urlstring
]
RESPONSE Example 1

```
{ "reply": [ { "tenant_id": "9949042437653", "name": "SSO Integration", "domain": "", "idp_enabled": true,
"default_role": null, "is_account_role": null, "idp_certificate": "certificate", "idp_issuer": "http://test.com",
"idp_sso_url": "http://test.com/", "metadata_url": "", "mappings": { "email": "user@company.com", "firstname":
"John", "group_name": "Users", "lastname": "Smith" }, "advanced_settings": { "authn_context_enabled": false,
"force_authn": null, "idp_single_logout_url": "", "relay_state": "", "service_provider_private_key": "",
"service_provider_public_cert": "" }, "sp_entity_id": "https://tenant.cortex.us.paloaltonetworks.com",
"sp_logout_url": "https://tenant.cortex.us.paloaltonetworks.com/idp/logout", "sp_url":
"https://tenant.cortex.us.paloaltonetworks.com/idp/saml" } ] }
```


Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.14.5 | Get IdP metadata

post /public_api/v1/authentication-settings/get/metadata

Get the metadata for all IdPs.

You must have **Instance Administrator** permissions to run this endpoint.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
```

```
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/authentication-settings/get/metadata'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/authentication-settings/get/metadata", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/authentication-
settings/get/metadata") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE'
request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "
{\"request_data\":{}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": {} }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/authentication-
settings/get/metadata"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-
auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/authentication-
settings/get/metadata") .header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE")
.header("content-type", "application/json") .body("{\"request_data\":{}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": []] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/authentication-settings/get/metadata")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/authentication-settings/get/metadata", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE",
"content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/authentication-settings/get/metadata"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers,
"x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/authentication-settings/get/metadata"); var request =
new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-
id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{}}", ParameterType.RequestBody); IRestResponse response
= client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

REQUEST

```
{ "request_data": {} }
```

Responses

OK

Body

application/json

▼ replyobject

▶ sp_entity_idstring

▶ sp_logout_urlstring

▶ sp_urlstring

▶ tenant_idstring

RESPONSE Example 1

```
{ "reply": { "sp_entity_id": "https://tenant.cortex.us.paloaltonetworks.com", "sp_logout_url":
"https://tenant.cortex.us.paloaltonetworks.com/idp/logout", "sp_url":
"https://tenant.cortex.us.paloaltonetworks.com/idp/saml", "tenant_id": "9949042437653" } }
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.
▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.15 | Syslog servers

APIs for managing syslog servers

2.1.15.1 | Create a syslog integration

post /public_api/v1/integrations/syslog/create

Create a new syslog integration.

You must have **View/Edit Alert Notification** permissions to run this endpoint.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/integrations/syslog/create'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"name\":\"string\",\"address\":\"string\",\"port\":0,\"protocol\":\"TCP\",\"facility\":\"string\",\"security_info\":
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
```

```
conn.request("POST", "/public_api/v1/integrations/syslog/create", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/integrations/syslog/create") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":
{\"name\":\"string\",\"address\":\"string\",\"port\":0,\"protocol\":\"TCP\",\"facility\":\"string\",\"security_info\":
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "name": "string", "address": "string", "port": 0, "protocol": "TCP",
"facility": "string", "security_info": { "certificate_name": "string", "ignore_cert_errors": true,
"certificate_content": "string" } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/integrations/syslog/create"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/integrations/syslog/create")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":
{\"name\":\"string\",\"address\":\"string\",\"port\":0,\"protocol\":\"TCP\",\"facility\":\"string\",\"security_info\":
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "name": "string", "address": "string",
"port": 0, "protocol": "TCP", "facility": "string", "security_info": [ "certificate_name": "string",
"ignore_cert_errors": true, "certificate_content": "string" ] ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/integrations/syslog/create")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/integrations/syslog/create", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"name\":\"string\",\"address\":\"string\",\"port\":0,\"protocol\":\"TCP\",\"facility\":\"string\",\"security_info\":
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/integrations/syslog/create"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"name\":\"string\",\"address\":\"string\",\"port\":0,\"protocol\":\"TCP\",\"facility\":\"string\",\"security_info\":
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/integrations/syslog/create"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":
{\"name\":\"string\",\"address\":\"string\",\"port\":0,\"protocol\":\"TCP\",\"facility\":\"string\",\"security_info\":
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

A dictionary containing the API request fields.

▶ namestring

Unique name for the syslog server integration.

▶ addressstring

IP address or fully qualified domain name (FQDN) of the syslog server.

▶ portinteger

The port number on which the syslog server listens for messages.

▶ protocolstring (Enum)

Select a method of communication:

- TCP: No validation is made on the connection with the syslog server. However, if an error occurred with the domain used to make the connection, the Test connection will fail.
- UDP: No error checking, error correction, or acknowledgment. No validation is done for the connection or when sending data.
- TLS: Cortex validates the syslog server certificate and uses the certificate signature and public key to encrypt the data sent over the connection.

▶ facilitystring

Choose one of the syslog standard values. The value maps to how your syslog server uses the facility field to manage messages. For details on the facility field, see RFC 5424.

▶ security_infoobject

The `security_info` parameters are necessary only when `protocol` is `TLS`.

REQUEST Example 1
```
{ "request_data": { "address": "xdr-splunk-qa.traps.company.com", "facility": "FAC_USER", "name":
"Syslog_PAPI_Test_7H55R76T", "port": 5006, "protocol": "TCP", "security_info": "None" } }
{ "request_data": { "address": "xdr-splunk-qa.traps.paloaltonetworks.com", "facility": "FAC_USER", "name":
"Syslog_PAPI_Test_2QYH3VGS", "port": 5002, "protocol": "TLS", "security_info": { "ignore_cert_errors": false } } }
```
Responses

OK

Body
application/json
▼ syslog_integration_idinteger
▼ namestring
RESPONSE Example 1
```
{ "syslog_integration_id": 630, "name": "Test PAPI" }
```

Bad Request

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Payment Required

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal Server Error

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"

▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.15.2 | Get all or filtered syslog servers

post /public_api/v1/integrations/syslog/get

Get a complete or filtered list of syslog servers.

You must have **View Alert Notification** permissions to run this endpoint.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/integrations/syslog/get'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}]}}" headers = { 'Authorization': "SOME_STRING_VALUE",
'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/integrations/syslog/get", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/integrations/syslog/get") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}]}}"
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "name", "operator": "eq", "value": "string" }
] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange",
function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST",
"https://api-yourfqdn/public_api/v1/integrations/syslog/get"); xhr.setRequestHeader("Authorization",
"SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-
type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/integrations/syslog/get")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["filters": [ [ "field": "name", "operator":
"eq", "value": "string" ] ]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/integrations/syslog/get")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/integrations/syslog/get", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}]}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/integrations/syslog/get"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
```

```
{\"filters\":[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}]}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/integrations/syslog/get"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
application/json
▼ request_dataobject

A dictionary containing the API request fields. An empty dictionary returns all results.

▶ filtersarray

Array of filter fields. Each JSON object must contain a field, operator, and value.

REQUEST Example 1

```
{ "request_data": { "filters": [ { "field": "id", "operator": "eq", "value": 632 } ] } }
{ "request_data": { "filters": [ { "field": "status", "operator": "eq", "value": "ACTIVE" } ] } }
```

Responses

OK

Body
application/json
▼ objects_countinteger

The number of syslog servers that are returned.

▼ objectsarray

The syslog server details.

[
▶ SYSLOG_INTEGRATION_IDinteger
▶ SYSLOG_INTEGRATION_NAMEstring

Syslog server integration name.

▶ SYSLOG_INTEGRATION_ADDRESSstring

IP address or fully qualified domain name (FQDN) of the syslog server.

▶ SYSLOG_INTEGRATION_PORTinteger

The port number on which the syslog server listens for messages.

▶ SYSLOG_INTEGRATION_PROTOCOLstring

The communication protocol:

- TCP: No validation is made on the connection with the syslog server. However, if an error occurred with the domain used to make the connection, the Test connection will fail.
- UDP: No error checking, error correction, or acknowledgment. No validation is done for the connection or when sending data.
- TLS: Cortex validates the syslog server certificate and uses the certificate signature and public key to encrypt the data sent over the connection.

▶ FACILITYstring
▶ SYSLOG_INTEGRATION_STATUSstring
▶ SYSLOG_INTEGRATION_ERRORobject
▶ SYSLOG_INTEGRATION_CERTIFICATE_NAMEobject
▶ SYSLOG_INTEGRATION_IGNORE_CERTIFICATE_ERRORstring
]
RESPONSE

```
{ "objects_count": 0, "objects": [ { "SYSLOG_INTEGRATION_ID": 0, "SYSLOG_INTEGRATION_NAME": "example",
"SYSLOG_INTEGRATION_ADDRESS": "example", "SYSLOG_INTEGRATION_PORT": 0, "SYSLOG_INTEGRATION_PROTOCOL": "example",
"FACILITY": "example", "SYSLOG_INTEGRATION_STATUS": "example", "SYSLOG_INTEGRATION_ERROR": {},
"SYSLOG_INTEGRATION_CERTIFICATE_NAME": {}, "SYSLOG_INTEGRATION_IGNORE_CERTIFICATE_ERROR": "example" } ] }
```

Bad Request

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Payment Required

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Internal Server Error

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

2.1.15.3 | Update a syslog integration

post /public_api/v1/integrations/syslog/update

Update the details of the specified syslog integration.

You must have **View/Edit Alert Notification** permissions to run this endpoint.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/integrations/syslog/update'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"syslog_id\":\"string\",\"name\":\"string\",\"address\":\"string\",\"port\":\"string\",\"protocol\":\"TCP\",\"facility\":\"s
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/integrations/syslog/update", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/integrations/syslog/update") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":
{\"syslog_id\":\"string\",\"name\":\"string\",\"address\":\"string\",\"port\":\"string\",\"protocol\":\"TCP\",\"facility\":\"s
```

```
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "syslog_id": "string", "name": "string", "address": "string", "port":
"string", "protocol": "TCP", "facility": "string", "security_info": { "certificate_name": "string",
"ignore_cert_errors": true, "certificate_content": "string" } } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/integrations/syslog/update"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/integrations/syslog/update")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":
{\"syslog_id\":\"string\",\"name\":\"string\",\"address\":\"string\",\"port\":\"string\",\"protocol\":\"TCP\",\"facility\":\"st
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "syslog_id": "string", "name": "string",
"address": "string", "port": "string", "protocol": "TCP", "facility": "string", "security_info": [
"certificate_name": "string", "ignore_cert_errors": true, "certificate_content": "string" ] ]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/integrations/syslog/update")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/integrations/syslog/update", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"syslog_id\":\"string\",\"name\":\"string\",\"address\":\"string\",\"port\":\"string\",\"protocol\":\"TCP\",\"facility\":\"st
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/integrations/syslog/update"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"syslog_id\":\"string\",\"name\":\"string\",\"address\":\"string\",\"port\":\"string\",\"protocol\":\"TCP\",\"facility\":\"st
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/integrations/syslog/update"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":
{\"syslog_id\":\"string\",\"name\":\"string\",\"address\":\"string\",\"port\":\"string\",\"protocol\":\"TCP\",\"facility\":\"st
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

A dictionary containing the API request fields.

▶ syslog_idstring

ID of the syslog server

▶ namestring

Unique name for the syslog server integration.

▶ addressstring

IP address or fully qualified domain name (FQDN) of the syslog server.

▶ portstring

The port number on which the syslog server listens for messages.

▶ protocolstring (Enum)

Select a method of communication:

- `TCP`: No validation is made on the connection with the syslog server. However, if an error occurred with the domain used to make the connection, the Test connection will fail.
- `UDP`: No error checking, error correction, or acknowledgment. No validation is done for the connection or when sending data.
- `TLS`: Cortex validates the syslog server certificate and uses the certificate signature and public key to encrypt the data sent over the connection.

▶ facilitystring

Choose one of the syslog standard values. The value maps to how your syslog server uses the facility field to manage messages. For details on the facility field, see RFC 5424.

▶ security_infoobject

The `security_info` parameters are relevant only when `protocol` is `TLS`.

REQUEST Example 1
```
{ "request_data": { "syslog_id": "123456789", "name": "Test PAPI", "address": "34.24.11.16", "port": "1234",
"protocol": "TLS", "facility": "FAC_USER", "security_info": { "certificate_name": "wf-verdict-service.key.pem",
"ignore_cert_errors": false, "certificate_content": "<binary string of the cert>" } } }
```
Responses

OK

Body
application/json
▼ replyboolean

Whether the update was successful.

RESPONSE
```
{ "reply": false }
```

Bad Request

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Payment Required

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal Server Error

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.15.4 | Delete all or filtered syslog integrations

post /public_api/v1/integrations/syslog/delete

Delete all the syslog integrations or the ones who match the filter criteria.

You must have View/Edit Alert Notification permissions to run this endpoint.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/integrations/syslog/delete'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}]}}" headers = { 'Authorization': "SOME_STRING_VALUE",
'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/integrations/syslog/delete", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/integrations/syslog/delete") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}]}}"
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "name", "operator": "eq", "value": "string" }
] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange",
function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST",
"https://api-yourfqdn/public_api/v1/integrations/syslog/delete"); xhr.setRequestHeader("Authorization",
"SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-
type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/integrations/syslog/delete")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["filters": [ [ "field": "name", "operator":
"eq", "value": "string" ] ]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/integrations/syslog/delete")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/integrations/syslog/delete", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}]}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/integrations/syslog/delete"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
```

```
{\"filters\":[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}]}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/integrations/syslog/delete"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
application/json
▼ request_dataobject

A dictionary containing the API request fields. An empty dictionary deletes all syslog servers.

▶ filtersarray

Array of filter fields. Each JSON object must contain a field, operator, and value.

REQUEST Example 1
```
{ "request_data": { "filters": [ { "field": "name", "operator": "EQ", "value": "Default syslog server" } ] } }
```

Responses

OK

Bad Request

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Payment Required

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal Server Error

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.15.5 | Test syslog integration

post /public_api/v1/integrations/syslog/test

Tests a syslog integration's validity.

You must have **View Alert Notification** permissions to run this endpoint.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}
Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/integrations/syslog/test'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"syslog_id\":\"string\",\"name\":\"string\",\"address\":\"string\",\"port\":\"string\",\"protocol\":\"TCP\",\"facility\":\"st
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/integrations/syslog/test", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/integrations/syslog/test") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":
{\"syslog_id\":\"string\",\"name\":\"string\",\"address\":\"string\",\"port\":\"string\",\"protocol\":\"TCP\",\"facility\":\"st
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "syslog_id": "string", "name": "string", "address": "string", "port":
"string", "protocol": "TCP", "facility": "string", "security_info": { "certificate_name": "string",
"ignore_cert_errors": true, "certificate_content": "string" } } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/integrations/syslog/test"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/integrations/syslog/test")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":
{\"syslog_id\":\"string\",\"name\":\"string\",\"address\":\"string\",\"port\":\"string\",\"protocol\":\"TCP\",\"facility\":\"st
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "syslog_id": "string", "name": "string",
"address": "string", "port": "string", "protocol": "TCP", "facility": "string", "security_info": [
"certificate_name": "string", "ignore_cert_errors": true, "certificate_content": "string" ] ]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/integrations/syslog/test")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/integrations/syslog/test", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"syslog_id\":\"string\",\"name\":\"string\",\"address\":\"string\",\"port\":\"string\",\"protocol\":\"TCP\",\"facility\":\"st
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/integrations/syslog/test"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"syslog_id\":\"string\",\"name\":\"string\",\"address\":\"string\",\"port\":\"string\",\"protocol\":\"TCP\",\"facility\":\"st
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}"); CURLcode ret =
curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/integrations/syslog/test"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":
{\"syslog_id\":\"string\",\"name\":\"string\",\"address\":\"string\",\"port\":\"string\",\"protocol\":\"TCP\",\"facility\":\"s
{\"certificate_name\":\"string\",\"ignore_cert_errors\":true,\"certificate_content\":\"string\"}}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
application/json
▼ request_dataobject

A dictionary containing the API request fields.

▶ syslog_idstring

If you include the `syslog_id` of an existing syslog integration, it will try to load the certificate data from the database to test the existing syslog integration.

▶ namestring

Unique name for the syslog server integration.

▶ addressstring

IP address or fully qualified domain name (FQDN) of the syslog server.

▶ portstring

The port number on which the syslog server listens for messages.

▶ protocolstring (Enum)

Select a method of communication:

- `TCP`: No validation is made on the connection with the syslog server. However, if an error occurred with the domain used to make the connection, the Test connection will fail.
- `UDP`: No error checking, error correction, or acknowledgment. No validation is done for the connection or when sending data.
- `TLS`: Cortex validates the syslog server certificate and uses the certificate signature and public key to encrypt the data sent over the connection.

▶ facilitystring

Choose one of the syslog standard values. The value maps to how your syslog server uses the facility field to manage messages. For details on the facility field, see RFC 5424.

▶ security_infoobject

The `security_info` parameters are relevant only when `protocol` is `TLS`.

REQUEST Example 1
```
{ "request_data": { "name": "Test PAPI", "address": "34.24.11.16", "port": "1234", "protocol": "TLS", "facility":
"FAC_USER" } }
```

Responses

OK

Bad Request

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Payment Required

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal Server Error

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

2.1.16 | Scheduled Queries

APIs for managing scheduled queries

2.1.16.1 | Get scheduled queries

post /public_api/v1/scheduled_queries/list

Return a list of scheduled queries. You can return all scheduled queries or filter results. You can also return extended results with all details included.

You must have **Instance Administrator** permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json,Retrieve all scheduled queries,Retrieve all scheduled queries with extended view'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/scheduled_queries/list'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"query_def_id\",\"operator\":\"EQ\",\"value\":\"string\"}],\"extended_view\":false,\"list_ids\":
[\"string\"]}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-
type': "application/json" } conn.request("POST", "/public_api/v1/scheduled_queries/list", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/scheduled_queries/list") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"query_def_id\",\"operator\":\"EQ\",\"value\":\"string\"}],\"extended_view\":false,\"list_ids\":
[\"string\"]}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "query_def_id", "operator": "EQ", "value":
"string" } ], "extended_view": false, "list_ids": [ "string" ] } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/scheduled_queries/list"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/scheduled_queries/list")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"query_def_id\",\"operator\":\"EQ\",\"value\":\"string\"}],\"extended_view\":false,\"list_ids\":
[\"string\"]}}") .asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE", "content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "query_def_id", "operator": "EQ", "value": "string" ] ], "extended_view": false, "list_ids": ["string"] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/scheduled_queries/list")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/scheduled_queries/list", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":[{\"field\":\"query_def_id\",\"operator\":\"EQ\",\"value\":\"string\"}],\"extended_view\":false,\"list_ids\":[\"string\"]}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd, CURLOPT_URL, "https://api-yourfqdn/public_api/v1/scheduled_queries/list"); struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":[{\"field\":\"query_def_id\",\"operator\":\"EQ\",\"value\":\"string\"}],\"extended_view\":false,\"list_ids\":[\"string\"]}}"); CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/scheduled_queries/list"); var request = new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{\"request_data\":{\"filters\":[{\"field\":\"query_def_id\",\"operator\":\"EQ\",\"value\":\"string\"}],\"extended_view\":false,\"list_ids\":[\"string\"]}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ filtersarray

An array of filter fields.

▶ extended_viewboolean

Display the extended view of the queries, which includes additional fields. If this is `false`, the response does not include `total_count` or `result_count`. Extended fields may change in future versions.

▶ list_idsarray[string]

List of scheduled query IDs to retrieve.

REQUEST Get all scheduled queries in extended view

```
{ "request_data": { "extended_view": "True" } }
{ "request_data": {} }
{ "request_data": { "list_ids": [ "qc_1683029855_18777", "qc_1678276807_6621" ] } }
{ "request_data": { "filters": [ { "field": "query_def_id", "operator": "EQ", "value": "qc_1683029855_18777" } ] } }
```

Responses

OK

Body

application/json

▼ replyobject

▶ dataarray

▶ filter_countinteger

▶ total_countinteger

RESPONSE

```
{ "reply": { "data": [ { "query_def_id": "example", "query_definition_name": "example", "xql": "example", "timeframe": { "relativeTime": "example" }, "schedule": { "run_date": 0, "trigger_type": "example" }, "tenants": {}, "enable": false } ], "filter_count": 0, "total_count": 0 } }
```

RESPONSE

```
[Retrieve all scheduled queries content]
```

RESPONSE

```
[Retrieve all scheduled queries with extended view content]
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.16.2 | Insert or update scheduled queries

post /public_api/v1/scheduled_queries/insert

Insert new scheduled queries or update existing scheduled queries.

You must have Instance Administrator permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/scheduled_queries/insert'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\": [{\"query_definition_name\":\"string\",\"xql\":\"string\",\"timeframe\":{\"relativeTime\":0},\"schedule\": {\"trigger_type\":\"string\",\"run_date\":0}}]}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/scheduled_queries/insert", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/scheduled_queries/insert") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
```

```
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":[{\"query_definition_name\":\"string\",\"xql\":\"string\",\"timeframe\":
{\"relativeTime\":0},\"schedule\":{\"trigger_type\":\"string\",\"run_date\":0}}]}" response = http.request(request)
puts response.read_body
const data = JSON.stringify({ "request_data": [ { "query_definition_name": "string", "xql": "string", "timeframe": {
"relativeTime": 0 }, "schedule": { "trigger_type": "string", "run_date": 0 } } ] }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/scheduled_queries/insert"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/scheduled_queries/insert")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":[{\"query_definition_name\":\"string\",\"xql\":\"string\",\"timeframe\":
{\"relativeTime\":0},\"schedule\":{\"trigger_type\":\"string\",\"run_date\":0}}]}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ [ "query_definition_name": "string", "xql":
"string", "timeframe": ["relativeTime": 0], "schedule": [ "trigger_type": "string", "run_date": 0 ] ] ]] as [String :
Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/scheduled_queries/insert")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/scheduled_queries/insert", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
[{\"query_definition_name\":\"string\",\"xql\":\"string\",\"timeframe\":{\"relativeTime\":0},\"schedule\":
{\"trigger_type\":\"string\",\"run_date\":0}}]}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE",
"content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/scheduled_queries/insert"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
[{\"query_definition_name\":\"string\",\"xql\":\"string\",\"timeframe\":{\"relativeTime\":0},\"schedule\":
{\"trigger_type\":\"string\",\"run_date\":0}}]}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/scheduled_queries/insert"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":[{\"query_definition_name\":\"string\",\"xql\":\"string\",\"timeframe\":
{\"relativeTime\":0},\"schedule\":{\"trigger_type\":\"string\",\"run_date\":0}}]}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataarray

[

▶ query_definition_namestring

Note: You should use a unique `query_definition_name` for each scheduled query on each tenant.

▶ xqlstring

▶ timeframeobject

▶ scheduleobjectrequired

]

REQUEST Example 1

```
{ "request_data": [ { "query_definition_name": "debug_john_api", "xql": "dataset = xdr_data | limit 10", "timeframe":
{ "relativeTime": 46400000 }, "schedule": { "trigger_type": "date", "run_date": 4677621540000 } } ] }
```

Responses

OK

Body

application/json

▼ replyobject

Upon successful insert or update, the reply returns each individual query_id with the query definitions. If the insert or update failed, the reply will include the scheduled query name and the error message.

▶ <query_id>object

The query_id of the scheduled query that was inserted or updated.

RESPONSE Example 1


```
{ "reply": { "qc_0123456789_01": { "query_definition_name": "test_1", "xql": "dataset = xdr_data | limit 1",
"timeframe": { "relativeTime": 86400000 }, "schedule": { "trigger_type": "date", "run_date": 1824072062000 } },
"qc_0123456789_02": { "query_definition_name": "test_2", "xql": "dataset = xdr_data | limit 1", "timeframe": {
"relativeTime": 86400000 }, "schedule": { "trigger_type": "date", "run_date": 1824072062000 } } } }
```


Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE


```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE


```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.16.3 | Delete a scheduled query

post /public_api/v1/scheduled_queries/delete

Delete scheduled queries.

You must have **Instance Administrator** permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json,Example 1'
-H 'Content-Type: application/json'
```

```
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/scheduled_queries/delete'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":[\"string\"]}"
headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type':
"application/json" } conn.request("POST", "/public_api/v1/scheduled_queries/delete", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/scheduled_queries/delete") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":[\"string\"]}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": [ "string" ] }); const xhr = new XMLHttpRequest(); xhr.withCredentials
= true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/scheduled_queries/delete"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/scheduled_queries/delete")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":[\"string\"]}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["string"]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/scheduled_queries/delete")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/scheduled_queries/delete", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":[\"string\"]}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/scheduled_queries/delete"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
[\"string\"]}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/scheduled_queries/delete"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":[\"string\"]}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataarray[string]

REQUEST Example 1

```
{ "request_data": [ "qc_1683461522_18780", "qc_1677754986_6539", "avram" ] }
```

Responses

OK

Body

application/json

▼ replyarray

[

▶ scheduled_query_idstring

Each JSON pair in the response is the scheduled query ID and the results of the deletion request.

]

RESPONSE

```
{ "reply": [ { "scheduled_query_id": "example" } ] }
```

RESPONSE

```
[Example 1 content]
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.
▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.17 | Query Library

APIs for managing XQL query libraries

2.1.17.1 | Get XQL Queries

post /public_api/xql_library/get

Retrieve a detailed list of XQL query libraries. You can filter by list of query names or by list of query tags.

You must have **Instance Administrator** permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json,Example 1,With extended view'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/xql_library/get'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\": {\"extended_view\":false,\"xql_query_names\":[\"string\"],\"xql_query_tags\":[\"string\"]}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/xql_library/get", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/xql_library/get") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
```

```
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"extended_view\":false,\"xql_query_names\":[\"string\"],\"xql_query_tags\":[\"string\"]}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "extended_view": false, "xql_query_names": [ "string" ],
"xql_query_tags": [ "string" ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/xql_library/get");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/xql_library/get")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"extended_view\":false,\"xql_query_names\":
[\"string\"],\"xql_query_tags\":[\"string\"]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "extended_view": false, "xql_query_names":
["string"], "xql_query_tags": ["string"] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject:
parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/xql_library/get")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/xql_library/get", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"extended_view\":false,\"xql_query_names\":
[\"string\"],\"xql_query_tags\":[\"string\"]}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE",
"content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/xql_library/get"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"extended_view\":false,\"xql_query_names\":[\"string\"],\"xql_query_tags\":[\"string\"]}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/xql_library/get"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"extended_view\":false,\"xql_query_names\":[\"string\"],\"xql_query_tags\":[\"string\"]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ extended_viewboolean

Whether to retrieve the detailed information on each XQL query.

▶ xql_query_namesarray[string]

An array of XQL query names to search for. Note: If searching by `xql_query_names`, you cannot search by `xql_query_tags` in the same call.

▶ xql_query_tagsarray[string]

An array of XQL tag names to search for. Note: If searching by `xql_query_tags`, you cannot search by `xql_query_names` in the same call.

REQUEST Get all XQL libraries in extended view

```
{ "request_data": { "extended_view": "True" } }
{ "request_data": {} }
{ "request_data": { "xql_query_names": [ "KILLER", "Rare Executions of PSEXEC", "Top 10 Users failing to log in" ] }
}
{ "request_data": { "xql_query_tags": [ "perf", "perf2" ] } }
```

Responses

OK

Body

application/json

▼ replyobject

▶ queries_countinteger

▶ xql_queriesarray

RESPONSE


```
{ "reply": { "queries_count": 0, "xql_queries": [ { "id": 0, "name": "example", "description": "example",
"query_text": "example", "created_at": 0, "created_by": "example", "created_by_pretty": "example", "query_metadata":
{ "is_valid": false, "query_calls": [ {} ], "is_datamodel": false, "query_tables": [ "example" ], "query_presets": [
{} ] }, "is_private": false, "labels": [ "example" ] } ] } }
```


RESPONSE

```
[Example 1 content]
```

RESPONSE

```
[With extended view content]
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE


```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE


```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.17.2 | Insert or update XQL queries

post /public_api/xql_library/insert

Insert new XQL queries or update existing XQL queries.

Note: You should use unique `xql_query_name` for each XQL query on a given tenant.

You must have **Instance Administrator** permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json,Example 1'
```

```
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/xql_library/insert'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"xql_queries_override\":false,\"xql_queries\":
[{\"xql_query\":\"string\",\"xql_query_name\":\"string\"}],\"xql_query_tags\":[\"string\"]}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/xql_library/insert", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/xql_library/insert")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"xql_queries_override\":false,\"xql_queries\":
[{\"xql_query\":\"string\",\"xql_query_name\":\"string\"}],\"xql_query_tags\":[\"string\"]}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "xql_queries_override": false, "xql_queries": [ { "xql_query":
"string", "xql_query_name": "string" } ], "xql_query_tags": [ "string" ] } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/xql_library/insert");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/xql_library/insert")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"xql_queries_override\":false,\"xql_queries\":
[{\"xql_query\":\"string\",\"xql_query_name\":\"string\"}],\"xql_query_tags\":[\"string\"]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "xql_queries_override": false,
"xql_queries": [ [ "xql_query": "string", "xql_query_name": "string" ] ], "xql_query_tags": ["string"] ]] as [String
: Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/xql_library/insert")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/xql_library/insert", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"xql_queries_override\":false,\"xql_queries\":
[{\"xql_query\":\"string\",\"xql_query_name\":\"string\"}],\"xql_query_tags\":[\"string\"]}}", CURLOPT_HTTPHEADER =>
[ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/xql_library/insert"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"xql_queries_override\":false,\"xql_queries\":
[{\"xql_query\":\"string\",\"xql_query_name\":\"string\"}],\"xql_query_tags\":[\"string\"]}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/xql_library/insert"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"xql_queries_override\":false,\"xql_queries\":
[{\"xql_query\":\"string\",\"xql_query_name\":\"string\"}],\"xql_query_tags\":[\"string\"]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobjectrequired

▶ xql_queries_overrideboolean

When the `xql_query_name` already exists on the tenant, this field defines whether or not to overwrite the existing XQL query with the new content. When `true`, the query will be overwritten. When `false`, the query will not be updated and an error will be returned.

▶ xql_queriesarray

▶ xql_query_tagsarray[string]

XQL query tags are optional

REQUEST Example 1

```
{ "request_data": { "xql_queries_override": true, "xql_queries": [ { "xql_query": "dataset = xdr_data |limit 1", "xql_query_name": "test_ql1" }, { "xql_query": "dataset = xdr_data |limit 2", "xql_query_name": "test_ql11" }, { "xql_query": "dataset = xdr_data |limit 2", "xql_query_name": "test_ql11" } ], "xql_query_tags": [ "tag2", "tag202" ] } }
```

Responses

OK

Body

application/json

▼ replyobject

▶ xql_queries_addedarray[string]

List of XQL queries that were added.

▶ xql_queries_updatedarray[string]

List of XQL queries that were updated.

▶ errorsarray[string]

List of errors received from additions/updates.

RESPONSE

```
{ "reply": { "xql_queries_added": [ "example" ], "xql_queries_updated": [ "example" ], "errors": [ "example" ] } }
```

RESPONSE

```
[Example 1 content]
```

Bad Request. Got an invalid JSON.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.17.3 | Delete XQL Queries

post /public_api/xql_library/delete

Delete XQL queries. You can filter by list of query names or by list of query tags.

You must have Instance Administrator permissions to run this endpoint.
Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json,Example 1'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/xql_library/delete'
-d ''
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"xql_query_names\":[\"string\"],\"xql_query_tags\":[\"string\"]}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/xql_library/delete", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
```

```
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/xql_library/delete")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"xql_query_names\":[\"string\"],\"xql_query_tags\":[\"string\"]}}" response = http.request(request) puts
response.read_body
```

```
const data = JSON.stringify({ "request_data": { "xql_query_names": [ "string" ], "xql_query_tags": [ "string" ] } });
const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () {
if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/xql_library/delete"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
```

```
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/xql_library/delete")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"xql_query_names\":[\"string\"],\"xql_query_tags\":[\"string\"]}}")
.asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "xql_query_names": ["string"],
"xql_query_tags": ["string"] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/xql_library/delete")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/xql_library/delete", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"xql_query_names\":[\"string\"],\"xql_query_tags\":[\"string\"]}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/xql_library/delete"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"xql_query_names\":
[\"string\"],\"xql_query_tags\":[\"string\"]}}"); CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/xql_library/delete"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"xql_query_names\":[\"string\"],\"xql_query_tags\":[\"string\"]}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters

application/json
▼ request_dataobject
▶ xql_query_namesarray[string]

List of XQL query names to delete. Note: If searching by `xql_query_names`, you cannot search by `xql_query_tags` in the same call.

▶ xql_query_tagsarray[string]

List of XQL query tags. Note: If searching by `xql_query_tags`, you cannot search by `xql_query_names` in the same call.

REQUEST Delete XQL libraries by query names
```
{ "request_data": { "xql_query_names": [ "KILLER", "Rare Executions of PSEXEC", "Top 10 Users failing to log in" ] }
}
{ "request_data": { "xql_query_tags": [ "perf", "perf2" ] } }
```
Responses

OK

Body
application/json
▼ replyobject
▶ queries_countinteger
▶ xql_query_namesarray[string]
▶ errorsarray
RESPONSE
```
{ "reply": { "queries_count": 0, "xql_query_names": [ "example" ], "errors": [ {} ] } }
```
RESPONSE
```
[Example 1 content]
```

Bad Request. Got an invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.18 | IOCs

APIs for managing IOCs

2.1.18.1 | Get Indicators (IOCs)

post /public_api/v1/indicators/get

Get a list of IOCs. You can return all IOCs or filter results. You can also return extended results with all details included.

- The response is concatenated using AND condition (OR is not supported).
- The maximum result set size is >100.
- Offset is the zero-based number of incidents from the start of the result set.

You must have Instance Administrator permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/indicators/get'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"extended_view\":true,\"filters\":[{\"field\":\"indicator\",\"operator\":\"EQ\",\"value\":
[null]}],\"search_from\":0,\"search_to\":0}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/indicators/get",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/indicators/get")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"extended_view\":true,\"filters\":[{\"field\":\"indicator\",\"operator\":\"EQ\",\"value\":
[null]}],\"search_from\":0,\"search_to\":0}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "extended_view": true, "filters": [ { "field": "indicator",
"operator": "EQ", "value": [ null ] } ], "search_from": 0, "search_to": 0 } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/indicators/get");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/indicators/get")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"extended_view\":true,\"filters\":
[{\"field\":\"indicator\",\"operator\":\"EQ\",\"value\":[null]}],\"search_from\":0,\"search_to\":0}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "extended_view": true, "filters": [ [
"field": "indicator", "operator": "EQ", "value": [] ] ], "search_from": 0, "search_to": 0 ]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/indicators/get")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/indicators/get", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"extended_view\":true,\"filters\":
[{\"field\":\"indicator\",\"operator\":\"EQ\",\"value\":[null]}],\"search_from\":0,\"search_to\":0}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/indicators/get"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
```

```
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\": {\"extended_view\":true,\"filters\":[{\"field\":\"indicator\",\"operator\":\"EQ\",\"value\": [null]}],\"search_from\":0,\"search_to\":0}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/indicators/get"); var request = new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{\"request_data\":{\"extended_view\":true,\"filters\":[{\"field\":\"indicator\",\"operator\":\"EQ\",\"value\": [null]}],\"search_from\":0,\"search_to\":0}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ extended_viewboolean

▶ filtersarray

An array of filter fields.

▶ search_frominteger

Integer representing the starting offset within the query result set from which you want indicators returned. Indicators are returned as a zero-based list. Any indicator indexed less than this value is not returned in the final result set and defaults to zero.

▶ search_tointeger

Integer representing the end offset within the result set after which you do not want indicators returned. Indicators in the indicator list that are indexed higher than this value are not returned in the final results set. Defaults to >100, which returns all indicators to the end of the list.

REQUEST Example 1

```
{ "request_data": { "extended_view": false, "filters": [ { "field": "rule_id", "operator": "IN", "value": [ 57 ] } ] } }
```

Responses

OK

Body

application/json

▼ objects_countinteger

▼ objectsarray

[

▶ rule_idinteger

▶ indicatorstring

▶ typestring

▶ severitystring

▶ expiration_dateinteger

▶ commentstring

]

▼ objects_typestring

RESPONSE Example 1

```
{ "objects_count": 1, "objects": [ { "rule_id": 57, "indicator": "virus1.exe", "type": "FILENAME", "severity": "SEV_040_HIGH", "expiration_date": -1, "comment": "test", "reputation": null, "reliability": null } ], "objects_type": "indicator" }
```

Bad Request. Got invalid JSON.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

2.1.18.2 | Insert or update IOCs

post /public_api/v1/indicators/insert

Insert new IOCs or update existing IOCs.

Note: The IOC `rule_id` is tenant specific and can't be used across tenants. Inserting IOCs with the same `rule_id` as an existing IOC on that tenant will overwrite the existing IOC.

You must have Instance Administrator permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/indicators/insert'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
[{\"rule_id\":0,\"indicator\":\"string\",\"type\":\"HASH\",\"severity\":\"SEV_010_INFO\",\"expiration_date\":0,\"default_expira
headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type':
"application/json" } conn.request("POST", "/public_api/v1/indicators/insert", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/indicators/insert")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
[{\"rule_id\":0,\"indicator\":\"string\",\"type\":\"HASH\",\"severity\":\"SEV_010_INFO\",\"expiration_date\":0,\"default_expira
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": [ { "rule_id": 0, "indicator": "string", "type": "HASH", "severity":
"SEV_010_INFO", "expiration_date": 0, "default_expiration_enabled": true, "comment": "string", "reputation": "GOOD",
"reliability": "A" } ] }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/indicators/insert");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/indicators/insert")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":
[{\"rule_id\":0,\"indicator\":\"string\",\"type\":\"HASH\",\"severity\":\"SEV_010_INFO\",\"expiration_date\":0,\"default_expira
.asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ [ "rule_id": 0, "indicator": "string",
"type": "HASH", "severity": "SEV_010_INFO", "expiration_date": 0, "default_expiration_enabled": true, "comment":
"string", "reputation": "GOOD", "reliability": "A" ] ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/indicators/insert")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/indicators/insert", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":
[{\"rule_id\":0,\"indicator\":\"string\",\"type\":\"HASH\",\"severity\":\"SEV_010_INFO\",\"expiration_date\":0,\"default_expira
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/indicators/insert"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
[{\"rule_id\":0,\"indicator\":\"string\",\"type\":\"HASH\",\"severity\":\"SEV_010_INFO\",\"expiration_date\":0,\"default_expira
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/indicators/insert"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":
[{\"rule_id\":0,\"indicator\":\"string\",\"type\":\"HASH\",\"severity\":\"SEV_010_INFO\",\"expiration_date\":0,\"default_expira
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataarray

[

▶ rule_idnumber

▶ indicatorstring

▶ typeobject (Enum)

▶ severityobject (Enum)

▶ expiration_dateinteger

▶ default_expiration_enabledboolean

▶ commentstring

▶ reputationobject (Enum)

▶ reliabilityobject (Enum)

]

REQUEST Example 1

```
{ "request_data": [ { "indicator": "virus1.exe", "type": "FILENAME", "severity": "SEV_040_HIGH", "expiration_date":
1234567891234, "default_expiration_enabled": false, "comment": "teststeststsetesteste", "reputation": null,
"reliability": null }, { "indicator": "test.exe", "type": "FILENAME", "severity": "SEV_030_MEDIUM",
"expiration_date": 1234567891234, "default_expiration_enabled": true, "reputation": null, "reliability": null } ] }
```

Responses

OK

Body

application/json

▼ added_objectsarray

List of IOC objects added.

[

▶ idinteger

▶ statusstring

]

▼ updated_objectsarray

List of IOC objects updated.

[

▶ idinteger

▶ statusstring
]
▼ errorsarray[string]

List of errors, if any.
RESPONSE Example 1

```
{ "added_objects": [ { "id": 69, "status": "Created a new IOC rule with the ID: 69 successfully" } ],
"updated_objects": [ { "id": 57, "status": "Updated the IOC rule with the indicator: 'virus1.exe' successfully" } ],
"errors": [] }
```


Bad Request. Got invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.18.3 | Delete Indicators (IOCs)

post /public_api/v1/indicators/delete

Delete IOCs selected by filter.

You must have **Instance Administrator** permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
```

```
'https://api-yourfqdn/public_api/v1/indicators/delete'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"indicator\",\"operator\":\"EQ\",\"value\":0}]}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-
xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/indicators/delete", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/indicators/delete")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"indicator\",\"operator\":\"EQ\",\"value\":0}]}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "indicator", "operator": "EQ", "value": 0 } ]
} }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function
() { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/indicators/delete"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/indicators/delete")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"indicator\",\"operator\":\"EQ\",\"value\":0}]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["filters": [ [ "field": "indicator",
"operator": "EQ", "value": 0 ] ]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject:
parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/indicators/delete")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/indicators/delete", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":[{\"field\":\"indicator\",\"operator\":\"EQ\",\"value\":0}]}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/indicators/delete"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"indicator\",\"operator\":\"EQ\",\"value\":0}]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/indicators/delete"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"indicator\",\"operator\":\"EQ\",\"value\":0}]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ filtersarray

An array of filter fields.

REQUEST Example 1

```
{ "request_data": { "extended_view": false, "filters": [ { "field": "rule_id", "operator": "EQ", "value": 57 } ] } }
```

Responses

OK

Body

application/json

▼ objects_countinteger

▼ objectsarray[integer]

RESPONSE Example 1

```
{ "objects_count": 1, "objects": [ 57 ] }
```

Bad Request. Got invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

2.1.19 | BIOCs

APIs for managing BIOCs

2.1.19.1 | Get BIOCs

post /public_api/v1/bioc/get

Return a list of BIOCs. You can return all BIOCs or filter results. You can also return extended results with all details included.- The response is concatenated using AND condition (OR is not supported).

- The maximum result set size is >100.
- Offset is the zero-based number of incidents from the start of the result set.

You must have **Instance Administrator** permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/bioc/get'
-d ''
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"extended_view\":true,\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/bioc/get", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/bioc/get") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"extended_view\":true,\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "extended_view": true, "filters": [ { "field": "name", "operator":
"EQ", "value": "string" } ], "search_from": 0, "search_to": 0 } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/bioc/get");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/bioc/get") .header("Authorization",
"SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type", "application/json")
.body("{\"request_data\":{\"extended_view\":true,\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "extended_view": true, "filters": [ [
"field": "name", "operator": "EQ", "value": "string" ] ], "search_from": 0, "search_to": 0 ]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/bioc/get")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/bioc/get",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "
{\"request_data\":{\"extended_view\":true,\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/bioc/get"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"extended_view\":true,\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/bioc/get"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"extended_view\":true,\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ extended_viewboolean

▶ filtersarray

An array of filter fields.

▶ search_frominteger

Integer representing the starting offset within the query result set from which you want BIOCs returned. BIOCs are returned as a zero-based list. Any BIOC indexed less than this value is not returned in the final result set and defaults to zero.

▶ search_tointeger

Integer representing the end offset within the result set after which you do not want BIOCs returned. BIOCs in the BIOC list that are indexed higher than this value are not returned in the final results set. Defaults to >100, which returns all BIOCs to the end of the list.

REQUEST Example 1

```
{ "request_data": { "extended_view": false } }
```

Responses

OK

Body
application/json

▼ objects_countinteger
▼ objectsarray
[
▶ rule_idinteger
▶ namestring
▶ typestring
▶ severitystring
▶ commentstring
▶ statusstring
▶ is_xqlboolean
▶ indicatorobject
▶ mitre_tactic_id_and_namearray[string]
▶ mitre_technique_id_and_namearray[string]
]
▼ objects_typestring

RESPONSE Example 1

```
{ "objects_count": 2, "objects": [ { "rule_id": 376, "name": "TestDataSourceTags", "type": "OTHER", "severity":
"SEV_030_MEDIUM", "comment": "", "status": "DISABLED", "is_xql": false, "indicator": { "runOnCGO": true,
"investigationType": "PROCESS_EXECUTION_EVENT", "investigation": { "PROCESS_EXECUTION_EVENT": { "filter": { "AND": [
{ "SEARCH_FIELD": "action_process_username", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "guyk", "EXTRA_FIELDS": [],
"isExtended": false } ] } } } }, "mitre_tactic_id_and_name": [], "mitre_technique_id_and_name": [] }, { "rule_id":
421, "name": "new_bioc_test", "type": "EXECUTION", "severity": "SEV_020_LOW", "comment": "", "status": "ENABLED",
"is_xql": true, "indicator": "dataset = xdr_data | filter event_type = 1 and actor_process_image_name =
\"SDFDSGFHFN\"", "mitre_tactic_id_and_name": [ "12 - Tactic", "45 - Another Tactic" ], "mitre_technique_id_and_name":
[ "123 - Test", "12 - Another Test" ] } ], "objects_type": "bioc" }
```

Bad Request. Got invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.19.2 | Insert or update BIOCs

post /public_api/v1/bioc/insert

Insert new BIOCs or update existing BIOCs.

Note: The BIOC `rule_id` is tenant specific and can't be used across tenants. Inserting BIOCs with the same `rule_id` as an existing BIOC on that tenant will overwrite the existing BIOC.

You must have Instance Administrator permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/bioc/insert'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
[{\"rule_id\":0,\"name\":\"string\",\"type\":\"OTHER\",\"severity\":\"SEV_010_INFO\",\"comment\":\"string\",\"status\":\"enable
{\"property1\":null,\"property2\":null},\"mitre_tactic_id_and_name\":[\"string\"],\"mitre_technique_id_and_name\":
[\"string\"]}]}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-
type': "application/json" } conn.request("POST", "/public_api/v1/bioc/insert", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/bioc/insert") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
[{\"rule_id\":0,\"name\":\"string\",\"type\":\"OTHER\",\"severity\":\"SEV_010_INFO\",\"comment\":\"string\",\"status\":\"enable
{\"property1\":null,\"property2\":null},\"mitre_tactic_id_and_name\":[\"string\"],\"mitre_technique_id_and_name\":
[\"string\"]}]}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": [ { "rule_id": 0, "name": "string", "type": "OTHER", "severity":
"SEV_010_INFO", "comment": "string", "status": "enabled", "is_xql": true, "indicator": { "property1": null,
"property2": null }, "mitre_tactic_id_and_name": [ "string" ], "mitre_technique_id_and_name": [ "string" ] } ] });
const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () {
if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/bioc/insert"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/bioc/insert")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":
[{\"rule_id\":0,\"name\":\"string\",\"type\":\"OTHER\",\"severity\":\"SEV_010_INFO\",\"comment\":\"string\",\"status\":\"enable
{\"property1\":null,\"property2\":null},\"mitre_tactic_id_and_name\":[\"string\"],\"mitre_technique_id_and_name\":
[\"string\"]}]}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ [ "rule_id": 0, "name": "string", "type":
"OTHER", "severity": "SEV_010_INFO", "comment": "string", "status": "enabled", "is_xql": true, "indicator": [
"property1": , "property2": ], "mitre_tactic_id_and_name": ["string"], "mitre_technique_id_and_name": ["string"] ] ]]
as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/bioc/insert")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/bioc/insert", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
```

```
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":
[{\"rule_id\":0,\"name\":\"string\",\"type\":\"OTHER\",\"severity\":\"SEV_010_INFO\",\"comment\":\"string\",\"status\":\"enable
{\"property1\":null,\"property2\":null},\"mitre_tactic_id_and_name\":[\"string\"],\"mitre_technique_id_and_name\":
[\"string\"]}]}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-
xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if
($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/bioc/insert"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
[{\"rule_id\":0,\"name\":\"string\",\"type\":\"OTHER\",\"severity\":\"SEV_010_INFO\",\"comment\":\"string\",\"status\":\"enable
{\"property1\":null,\"property2\":null},\"mitre_tactic_id_and_name\":[\"string\"],\"mitre_technique_id_and_name\":
[\"string\"]}]}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/bioc/insert"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":
[{\"rule_id\":0,\"name\":\"string\",\"type\":\"OTHER\",\"severity\":\"SEV_010_INFO\",\"comment\":\"string\",\"status\":\"enable
{\"property1\":null,\"property2\":null},\"mitre_tactic_id_and_name\":[\"string\"],\"mitre_technique_id_and_name\":
[\"string\"]}]}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataarray

[

▶ rule_idinteger

▶ namestring

BIOC name.

▶ typeobject (Enum)

BIOC type.

▶ severityobject (Enum)

BIOC severity.

▶ commentstring

▶ statusobject (Enum)

BIOC status.

▶ is_xqlboolean

▶ indicatorobject

▶ mitre_tactic_id_and_namearray[string]

▶ mitre_technique_id_and_namearray[string]

]

REQUEST Example 1

```
{ "request_data": [ { "name": "TestBIOC", "type": "EXECUTION", "severity": "SEV_020_LOW", "comment": "", "status":
"ENABLED", "is_xql": false, "indicator": { "runOnCGO": true, "investigationType": "FILE_EVENT", "investigation": {
"FILE_EVENT": { "filter": { "AND": [ { "OR": [ { "SEARCH_FIELD": "event_sub_type", "SEARCH_TYPE": "EQ",
"SEARCH_VALUE": "1", "isExtended": false }, { "SEARCH_FIELD": "event_sub_type", "SEARCH_TYPE": "EQ", "SEARCH_VALUE":
"2", "isExtended": false }, { "SEARCH_FIELD": "event_sub_type", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "3",
"isExtended": false }, { "SEARCH_FIELD": "event_sub_type", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "5", "isExtended":
false }, { "SEARCH_FIELD": "event_sub_type", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "6", "isExtended": false } ] }, {
"SEARCH_FIELD": "action_file_name", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "aaaaaa", "EXTRA_FIELDS": [], "isExtended":
false } ] } } } }, "mitre_tactic_id_and_name": [ "" ], "mitre_technique_id_and_name": [ "" ] } ] }
```

Responses

OK

Body

application/json

▼ added_objectsarray

List of BIOC objects added.

[

▶ idinteger

▶ statusstring

]
▼ updated_objectsarray

List of BIOC objects updated.

[
▶ idinteger
▶ statusstring
]
▼ errorsarray[string]

A list of errors.
RESPONSE Example 1
{ "added_objects": [{ "id": 34, "status": "Created a new BIOC rule with the ID: 34 successfully" }], "updated_objects": [], "errors": [] }

Bad Request. Got invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.19.3 | Delete BIOCs

post /public_api/v1/bioc/delete

Delete BIOCs selected by filter.

You must have **Instance Administrator** permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/bioc/delete'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":true}]}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-
xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/bioc/delete", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-
8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/bioc/delete") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":true}]}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "name", "operator": "EQ", "value": true } ] }
}); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function
() { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/bioc/delete"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/bioc/delete")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":true}]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["filters": [ [ "field": "name", "operator":
"EQ", "value": true ] ]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/bioc/delete")!
as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST"
request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/bioc/delete", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":true}]}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/bioc/delete"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":true}]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/bioc/delete"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":true}]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ filtersarray

An array of filter fields.

REQUEST Example 1

```
{ "request_data": { "extended_view": false, "filters": [ { "field": "severity", "operator": "EQ", "value":
"SEV_020_LOW" } ] } }
```

Responses

OK

Body

application/json

▼ objects_countinteger

Number of BIOC objects deleted.
▼ objectsarray[integer]
RESPONSE Example 1
{ "objects_count": 3, "objects": [1, 3, 7] }

Bad Request. Got invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.20 | Correlation Rules

APIs for managing Correlation Rules

2.1.20.1 | Get Correlation Rules

post /public_api/v1/correlations/get

Return a list of correlation rules. You can return all correlation rules or filter results. You can also return extended results with all details included.

- The response is concatenated using AND condition (OR is not supported).
- The maximum result set size is >100.
- Offset is the zero-based number of incidents from the start of the result set.

You must have **Instance Administrator** permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
```

```
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/correlations/get'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"extended_view\":true,\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0}}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/correlations/get", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/correlations/get")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"extended_view\":true,\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "extended_view": true, "filters": [ { "field": "name", "operator":
"EQ", "value": "string" } ], "search_from": 0, "search_to": 0 } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/correlations/get");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/correlations/get")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"extended_view\":true,\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "extended_view": true, "filters": [ [
"field": "name", "operator": "EQ", "value": "string" ] ], "search_from": 0, "search_to": 0 ]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/correlations/get")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/correlations/get", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"extended_view\":true,\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/correlations/get"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"extended_view\":true,\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/correlations/get"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"extended_view\":true,\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}],\"search_from\":0,\"search_to\":0}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ extended_viewboolean

▶ filtersarray

An array of filter fields.

▶ search_frominteger

Integer representing the starting offset within the query result set from which you want correlation rules returned. Correlation rules are returned as a zero-based list. Any correlation rule indexed less than this value is not returned in the final result set and defaults to zero.

▶ search_tointeger

Integer representing the end offset within the result set after which you do not want BIOCs returned. BIOCs in the BIOC list that are indexed higher than this value are not returned in the final results set. Defaults to >100, which returns all BIOCs to the end of the list.

REQUEST Example 1

```
{ "request_data": { "extended_view": false } }
```

Responses

OK

Body

application/json

▼ objects_countinteger

▼ objectsarray

[

▶ idinteger

▶ namestring

▶ severitystring

▶ xql_querystring

▶ is_enabledstring

▶ descriptionstring

▶ alert_namestring

▶ alert_categorystring

▶ alert_fieldsobject

▶ execution_modestring

▶ search_windowstring

▶ simple_schedulestring

▶ timezonestring

▶ crontabstring

▶ suppression_enabledboolean

▶ suppression_fieldsarray[string]

▶ datasetstring

▶ mitre_defsobject

▶ drilldown_query_timeframestring

▶ mapping_strategystring

]

▼ objects_typestring

RESPONSE Example 1

```
{ "objects_count": 2, "objects": [ { "id": 1, "name": "Test", "severity": "SEV_040_HIGH", "xql_query": "dataset =
xdr_data | filter event_type = 1", "is_enabled": "DISABLED", "description": "", "alert_name": "Test",
"alert_category": "User Defined", "alert_description": null, "alert_fields": { "agent_hostname": null,
"action_local_ip": null, "action_remote_ip": null, "action_remote_port": null, "agent_device_domain": null,
"actor_effective_username": null, "actor_process_image_name": null, "actor_process_image_path": null,
"actor_process_command_line": null, "actor_process_image_sha256": null }, "execution_mode": "SCHEDULED",
"search_window": "10 minutes", "simple_schedule": "10 minutes", "timezone": "Asia/Jerusalem", "crontab": "*/10 * * *
*", "suppression_enabled": false, "suppression_duration": null, "suppression_fields": null, "dataset": "alerts",
"user_defined_severity": null, "user_defined_category": "event_type", "mitre_defs": {}, "investigation_query_link":
null, "drilldown_query_timeframe": "ALERT", "mapping_strategy": "AUTO" }, { "id": 28, "name": "AnotherTest",
"severity": "SEV_030_MEDIUM", "xql_query": "dataset = xdr_data | fields event_type, action_process_username, uuid,
action_boot_time | comp values(*) as * by action_process_username\n", "is_enabled": "ENABLED", "description": "Some
description", "alert_name": "Test Alert", "alert_category": "DISCOVERY", "alert_description": "Test", "alert_fields":
{}, "execution_mode": "SCHEDULED", "search_window": "1 hours", "simple_schedule": "10 minutes", "timezone":
"Asia/Jerusalem", "crontab": "*/10 * * * *", "suppression_enabled": true, "suppression_duration": "1 hours",
"suppression_fields": [ "event_type" ], "dataset": "alerts", "user_defined_severity": null, "user_defined_category":
null, "mitre_defs": { "TA0005 - Defense Evasion": [ "T1014 - Rootkit" ] }, "investigation_query_link": "dataset =
xdr_data | fields event_type, action_process_username, uuid, action_boot_time | comp values(*) as * by
action_process_username", "drilldown_query_timeframe": "ALERT", "mapping_strategy": "CUSTOM" } ], "objects_type":
"correlations" }
```

Bad Request. Got invalid JSON.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.
▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

2.1.20.2 | Insert or update Correlation Rules

post /public_api/v1/correlations/insert

Insert new Correlation Rules or update existing Correlation Rules.

Note: The Correlation Rule `id` is tenant specific and can't be used across tenants. Inserting Correlation Rules with the same `id` as an existing Correlation Rule on that tenant will overwrite the existing Correlation Rule.

You must have Instance Administrator permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/correlations/insert'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
[{\"rule_id\":0,\"name\":\"string\",\"severity\":\"SEV_010_INFO\",\"xql_query\":\"string\",\"is_enabled\":true,\"description\"
{\"property1\":null,\"property2\":null},\"execution_mode\":\"SCHEDULED\",\"search_window\":\"\\\"2
hours\\\"\",\"simple_schedule\":\"\\\"5 minutes\\\"\",\"timezone\":\"\\\"Asia/Jerusalem\\\"\",\"crontab\":\"\\\"*/10
* * * *\\\"\",\"suppression_enabled\":true,\"suppression_duration\":\"\\\"1 hours\\\"\",\"suppression_fields\":
[\"\\\"event_type\\\"\"],\"dataset\":\"string\",\"user_defined_severity\":\"string\",\"user_defined_category\":\"string\",\"mi
{\"property1\":null,\"property2\":null},\"investigation_query_link\":\"string\",\"drilldown_query_timeframe\":\"QUERY\",\"mapp
headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type':
"application/json" } conn.request("POST", "/public_api/v1/correlations/insert", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/correlations/insert") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
```

```
request.body = "{\"request_data\":
[{\"rule_id\":0,\"name\":\"string\",\"severity\":\"SEV_010_INFO\",\"xql_query\":\"string\",\"is_enabled\":true,\"description\"
{\"property1\":null,\"property2\":null},\"execution_mode\":\"SCHEDULED\",\"search_window\":\"\\\"2
hours\\\"\",\"simple_schedule\":\"\\\"5 minutes\\\"\",\"timezone\":\"\\\"Asia/Jerusalem\\\"\",\"crontab\":\"\\\"*/10
* * * *\\\"\",\"suppression_enabled\":true,\"suppression_duration\":\"\\\"1 hours\\\"\",\"suppression_fields\":
[\"\\\"event_type\\\"\"],\"dataset\":\"string\",\"user_defined_severity\":\"string\",\"user_defined_category\":\"string\",\"mit
{\"property1\":null,\"property2\":null},\"investigation_query_link\":\"string\",\"drilldown_query_timeframe\":\"QUERY\",\"mappi
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": [ { "rule_id": 0, "name": "string", "severity": "SEV_010_INFO",
"xql_query": "string", "is_enabled": true, "description": "string", "alert_name": "string", "alert_category":
"OTHER", "alert_description": "string", "alert_fields": { "property1": null, "property2": null }, "execution_mode":
"SCHEDULED", "search_window": "\"2 hours\"", "simple_schedule": "\"5 minutes\"", "timezone": "\"Asia/Jerusalem\"",
"crontab": "\"*/10 * * * *\"", "suppression_enabled": true, "suppression_duration": "\"1 hours\"",
"suppression_fields": [ "\"event_type\"" ], "dataset": "string", "user_defined_severity": "string",
"user_defined_category": "string", "mitre_defs": { "property1": null, "property2": null },
"investigation_query_link": "string", "drilldown_query_timeframe": "QUERY", "mapping_strategy": "AUTO" } ] }); const
xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/correlations/insert"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/correlations/insert")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":
[{\"rule_id\":0,\"name\":\"string\",\"severity\":\"SEV_010_INFO\",\"xql_query\":\"string\",\"is_enabled\":true,\"description\"
{\"property1\":null,\"property2\":null},\"execution_mode\":\"SCHEDULED\",\"search_window\":\"\\\"2
hours\\\"\",\"simple_schedule\":\"\\\"5 minutes\\\"\",\"timezone\":\"\\\"Asia/Jerusalem\\\"\",\"crontab\":\"\\\"*/10
* * * *\\\"\",\"suppression_enabled\":true,\"suppression_duration\":\"\\\"1 hours\\\"\",\"suppression_fields\":
[\"\\\"event_type\\\"\"],\"dataset\":\"string\",\"user_defined_severity\":\"string\",\"user_defined_category\":\"string\",\"mit
{\"property1\":null,\"property2\":null},\"investigation_query_link\":\"string\",\"drilldown_query_timeframe\":\"QUERY\",\"mappi
.asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ [ "rule_id": 0, "name": "string",
"severity": "SEV_010_INFO", "xql_query": "string", "is_enabled": true, "description": "string", "alert_name":
"string", "alert_category": "OTHER", "alert_description": "string", "alert_fields": [ "property1": , "property2": ],
"execution_mode": "SCHEDULED", "search_window": "\"2 hours\"", "simple_schedule": "\"5 minutes\"", "timezone":
"\"Asia/Jerusalem\"", "crontab": "\"*/10 * * * *\"", "suppression_enabled": true, "suppression_duration": "\"1
hours\"", "suppression_fields": ["\"event_type\""], "dataset": "string", "user_defined_severity": "string",
"user_defined_category": "string", "mitre_defs": [ "property1": , "property2": ], "investigation_query_link":
"string", "drilldown_query_timeframe": "QUERY", "mapping_strategy": "AUTO" ] ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/correlations/insert")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/correlations/insert", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
[{\"rule_id\":0,\"name\":\"string\",\"severity\":\"SEV_010_INFO\",\"xql_query\":\"string\",\"is_enabled\":true,\"description\"
{\"property1\":null,\"property2\":null},\"execution_mode\":\"SCHEDULED\",\"search_window\":\"\\\"2
hours\\\"\",\"simple_schedule\":\"\\\"5 minutes\\\"\",\"timezone\":\"\\\"Asia/Jerusalem\\\"\",\"crontab\":\"\\\"*/10
* * * *\\\"\",\"suppression_enabled\":true,\"suppression_duration\":\"\\\"1 hours\\\"\",\"suppression_fields\":
[\"\\\"event_type\\\"\"],\"dataset\":\"string\",\"user_defined_severity\":\"string\",\"user_defined_category\":\"string\",\"mit
{\"property1\":null,\"property2\":null},\"investigation_query_link\":\"string\",\"drilldown_query_timeframe\":\"QUERY\",\"mappi
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/correlations/insert"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
[{\"rule_id\":0,\"name\":\"string\",\"severity\":\"SEV_010_INFO\",\"xql_query\":\"string\",\"is_enabled\":true,\"description\"
{\"property1\":null,\"property2\":null},\"execution_mode\":\"SCHEDULED\",\"search_window\":\"\\\"2
hours\\\"\",\"simple_schedule\":\"\\\"5 minutes\\\"\",\"timezone\":\"\\\"Asia/Jerusalem\\\"\",\"crontab\":\"\\\"*/10
* * * *\\\"\",\"suppression_enabled\":true,\"suppression_duration\":\"\\\"1 hours\\\"\",\"suppression_fields\":
```

[\"\\\"event_type\\\"\"],\"dataset\":\"string\",\"user_defined_severity\":\"string\",\"user_defined_category\":\"string\",\"mi
{\"property1\":null,\"property2\":null},\"investigation_query_link\":\"string\",\"drilldown_query_timeframe\":\"QUERY\",\"mapp
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/correlations/insert"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":
[{\"rule_id\":0,\"name\":\"string\",\"severity\":\"SEV_010_INFO\",\"xql_query\":\"string\",\"is_enabled\":true,\"description\"
{\"property1\":null,\"property2\":null},\"execution_mode\":\"SCHEDULED\",\"search_window\":\"\\\"2
hours\\\"\",\"simple_schedule\":\"\\\"5 minutes\\\"\",\"timezone\":\"\\\"Asia/Jerusalem\\\"\",\"crontab\":\"\\\"*/10
* * * *\\\"\",\"suppression_enabled\":true,\"suppression_duration\":\"\\\"1 hours\\\"\",\"suppression_fields\":
[\"\\\"event_type\\\"\"],\"dataset\":\"string\",\"user_defined_severity\":\"string\",\"user_defined_category\":\"string\",\"mi
{\"property1\":null,\"property2\":null},\"investigation_query_link\":\"string\",\"drilldown_query_timeframe\":\"QUERY\",\"mapp
ParameterType.RequestBody); IRestResponse response = client.Execute(request);

Body parameters

application/json

▼ request_dataarray

[

▶ rule_idinteger

▶ namestring

Correlation rule name.

▶ severityobject (Enum)

Correlation rule severity.

▶ xql_querystring

Correlation rule XQL query.

▶ is_enabledboolean

Whether the correlation rule is enabled or disabled.

▶ descriptionstring

Correlation rule description.

▶ alert_namestring

Alert name.

▶ alert_categoryobject (Enum)

Alert category.

▶ alert_descriptionstring

Alert description.

▶ alert_fieldsobject

Alert fields.

▶ execution_modeobject (Enum)

Correlation rule execution mode.

▶ search_windowstring

Search window.

▶ simple_schedulestring

Correlation rule simple schedule.

▶ timezonestring

Correlation rule timezone.

▶ crontabstring

Linux scheduling for correlation rule.

▶ suppression_enabledboolean

▶ suppression_durationstring

▶ suppression_fieldsarray[string]

▶ datasetstring
▶ user_defined_severitystring
▶ user_defined_categorystring
▶ mitre_defsobject
▶ investigation_query_linkstring
▶ drilldown_query_timeframeobject (Enum)
▶ mapping_strategyobject (Enum)
]

REQUEST Example 1


```
{ "request_data": [ { "rule_id": 28, "name": "Test", "severity": "SEV_030_MEDIUM", "xql_query": "dataset = xdr_data | fields event_type, action_process_username, uuid, action_boot_time | comp values(*) as * by action_process_username\n", "is_enabled": "true", "description": "RTESRTESTestret", "alert_name": "Test", "alert_category": "DISCOVERY", "alert_description": "Test", "alert_fields": {}, "execution_mode": "REAL_TIME", "search_window": "1 hours", "simple_schedule": "10 minutes", "timezone": "Asia/Jerusalem", "crontab": "*/10 * * * *", "suppression_enabled": true, "suppression_duration": "1 hours", "suppression_fields": [ "event_type" ], "dataset": "alerts", "user_defined_severity": null, "user_defined_category": null, "mitre_defs": { "TA0005 - Defense Evasion": [ "T1014 - Rootkit" ] }, "investigation_query_link": "dataset = xdr_data | fields event_type, action_process_username, uuid, action_boot_time | comp values(*) as * by action_process_username", "drilldown_query_timeframe": "ALERT", "mapping_strategy": "AUTO" } ] }
```


Responses

OK

Body
application/json

▼ added_objectsarray

List of Correlation Rule objects added.

[
▶ idinteger
▶ statusstring
]

▼ updated_objectsarray

List of Correlation Rule objects updated.

[
▶ idinteger
▶ statusstring
]

▼ errorsarray[string]

List of error messages, if there are any.

RESPONSE Example 1


```
{ "added_objects": [], "updated_objects": [ { "id": 28 }, { "status": "Updated the correlation rule with the ID: 28 successfully" } ], "errors": [] }
```


Bad Request. Got invalid JSON.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE


```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.
▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.20.3 | Delete Correlation Rules

post /public_api/v1/correlations/delete

Delete correlation rules selected by filter.

You must have Instance Administrator permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/correlations/delete'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}]}}" headers = { 'Authorization': "SOME_STRING_VALUE",
'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/correlations/delete", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/correlations/delete") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}]}}"
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "name", "operator": "EQ", "value": "string" }
] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange",
function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST",
"https://api-yourfqdn/public_api/v1/correlations/delete"); xhr.setRequestHeader("Authorization",
"SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-
type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/correlations/delete")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["filters": [ [ "field": "name", "operator":
"EQ", "value": "string" ] ]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/correlations/delete")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/correlations/delete", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}]}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/correlations/delete"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/correlations/delete"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"name\",\"operator\":\"EQ\",\"value\":\"string\"}]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ filtersarray

An array of filter fields.

REQUEST Example 1

```
{ "request_data": { "extended_view": false, "filters": [ { "field": "rule_id", "operator": "EQ", "value": 28 } ] } }
```

Responses

OK

Body

application/json

▼ objects_countinteger

▼ objectsarray[integer]

RESPONSE Example 1

```
{ "objects_count": 1, "objects": [ 28 ] }
```

Bad Request. Got invalid JSON.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`

▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body

application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.
Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized access. User does not have the required license type to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal server error. A unified status for API communication type errors.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE

{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.21 | Playbooks

APIs for managing playbooks

2.1.21.1 | Get a playbook

post /public_api/v1/playbooks/get

Get a playbook by filtering based on its name or ID. The playbook's YAML is returned in a ZIP file.

You must have **Instance Administrator** permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/playbooks/get'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filter\":
{\"field\":\"id\",\"value\":\"string\"}}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/playbooks/get",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/playbooks/get") http
= Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filter\":
{\"field\":\"id\",\"value\":\"string\"}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filter": { "field": "id", "value": "string" } } }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/playbooks/get"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/playbooks/get")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filter\":{\"field\":\"id\",\"value\":\"string\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["filter": [ "field": "id", "value": "string"
]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/playbooks/get")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/playbooks/get", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filter\":{\"field\":\"id\",\"value\":\"string\"}}}", CURLOPT_HTTPHEADER
=> [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/playbooks/get"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
```

```
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filter\":
{\"field\":\"id\",\"value\":\"string\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/playbooks/get"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filter\":{\"field\":\"id\",\"value\":\"string\"}}}", ParameterType.RequestBody); IRestResponse
response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ filterobject

Filter fields.

REQUEST Example 1

```
{ "request_data": { "filter": { "field": "name", "value": "Phishingplaybook" } } }
```

Responses

OK

2.1.21.2 | Insert or update playbooks

post /public_api/v1/playbooks/insert

Add or update a playbook by passing the YAML in a ZIP file.

You must have **Instance Administrator** permissions to run this endpoint.

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

Form parameters

▼ file File

format: binary

Example: `BINARY_DATA_HERE`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: multipart/form-data'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/playbooks/insert'
-F 'file=BINARY_DATA_HERE'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "----
-011000010111000001101001\r\nContent-Disposition: form-data; name=\"file\"\r\n\r\nstring\r\n----
-011000010111000001101001--\r\n" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE" } conn.request("POST", "/public_api/v1/playbooks/insert", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/playbooks/insert")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request.body = "-----011000010111000001101001\r\nContent-Disposition: form-data;
name=\"file\"\r\n\r\nstring\r\n-----011000010111000001101001--\r\n" response = http.request(request) puts
response.read_body
const data = new FormData(); data.append("file", "string"); const xhr = new XMLHttpRequest(); xhr.withCredentials =
true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/playbooks/insert");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/playbooks/insert")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
```

```
"multipart/form-data; boundary=---011000010111000001101001") .body("-----011000010111000001101001\r\nContent-Disposition: form-data; name=\"file\"\r\n\r\nstring\r\n-----011000010111000001101001--\r\n") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE", "content-type": "multipart/form-data" ] let parameters = [ [ "name": "file", "value": "string" ] ] let boundary = "---011000010111000001101001" var body = "" var error: NSError? = nil for param in parameters { let paramName = param["name"]! body += "--\(boundary)\r\n" body += "Content-Disposition:form-data; name=\"\(paramName)\"" if let filename = param["fileName"] { let contentType = param["content-type"]! let fileContent = String(contentsOfFile: filename, encoding: String.Encoding.utf8) if (error != nil) { print(error) } body += "; filename=\"\(filename)\"\r\n" body += "Content-Type: \(contentType)\r\n\r\n" body += fileContent } else if let paramValue = param["value"] { body += "\r\n\r\n\(paramValue)" } } let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/playbooks/insert")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/playbooks/insert", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "-----011000010111000001101001\r\nContent-Disposition: form-data; name=\"file\"\r\n\r\nstring\r\n-----011000010111000001101001--\r\n", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: multipart/form-data; boundary=---011000010111000001101001", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd, CURLOPT_URL, "https://api-yourfqdn/public_api/v1/playbooks/insert"); struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: multipart/form-data; boundary=---011000010111000001101001"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "-----011000010111000001101001\r\nContent-Disposition: form-data; name=\"file\"\r\n\r\nstring\r\n-----011000010111000001101001--\r\n"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/playbooks/insert"); var request = new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "multipart/form-data; boundary=---011000010111000001101001"); request.AddParameter("multipart/form-data; boundary=---011000010111000001101001", "----011000010111000001101001\r\nContent-Disposition: form-data; name=\"file\"\r\n\r\nstring\r\n----011000010111000001101001--\r\n", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```


Responses

OK

Body
application/json
▼ objects_countinteger

The number of playbooks identified and attempted to be added.

▼ objectsobject
▶ failures_itemsarray

This field in the response indicates that the playbook IDs listed failed to be added.

RESPONSE


```
{ "objects_count": 0, "objects": { "failures_items": [ { "error": "example", "id": "example" } ] } }
```

2.1.21.3 | Delete a playbook

post /public_api/v1/playbooks/delete

Delete a playbook by filtering based on its name or ID.

You must have **Instance Administrator** permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}
Example: xXdrAuthId_example
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/playbooks/delete'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filter\":
{\"field\":\"id\",\"value\":\"string\"}}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/playbooks/delete",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/playbooks/delete")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filter\":
{\"field\":\"id\",\"value\":\"string\"}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filter": { "field": "id", "value": "string" } } }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/playbooks/delete"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/playbooks/delete")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filter\":{\"field\":\"id\",\"value\":\"string\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["filter": [ "field": "id", "value": "string"
]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/playbooks/delete")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/playbooks/delete", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filter\":{\"field\":\"id\",\"value\":\"string\"}}}", CURLOPT_HTTPHEADER
=> [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/playbooks/delete"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filter\":
{\"field\":\"id\",\"value\":\"string\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/playbooks/delete"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filter\":{\"field\":\"id\",\"value\":\"string\"}}}", ParameterType.RequestBody); IRestResponse
response = client.Execute(request);
```
Body parameters
application/json
▼ request_dataobject
▶ filterobject

Filter fields.

REQUEST Example 1
```
{ "request_data": { "filter": { "field": "name", "value": "Phishingplaybook" } } }
```
Responses

OK

Body
application/json
▼ objects_countinteger

The number of playbooks identified and attempted to be deleted.
▼ objectsobject
▶ failures_itemsarray

This field in the response indicates that the playbook IDs listed failed to be deleted.

▶ succeeded_itemsarray

This field in the response indicates that the playbook IDs listed were successfully deleted.

RESPONSE
```
{ "objects_count": 0, "objects": { "failures_items": [ { "error": "example", "id": "example" } ], "succeeded_items": [ { "id": "example" } ] } }
```

2.1.22 | Scripts

APIs for managing scripts

2.1.22.1 | Get a script

post /public_api/v1/scripts/get

Get a script by filtering based on its name or ID. The script's YAML is returned in a ZIP file.

You must have **Instance Administrator** permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/zip'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/scripts/get'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filter\": {\"field\":\"id\",\"value\":\"string\"}}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/scripts/get", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/scripts/get") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filter\": {\"field\":\"id\",\"value\":\"string\"}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filter": { "field": "id", "value": "string" } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/scripts/get"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/scripts/get") .header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type", "application/json") .body("{\"request_data\":{\"filter\":{\"field\":\"id\",\"value\":\"string\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE", "content-type": "application/json" ] let parameters = ["request_data": ["filter": [ "field": "id", "value": "string" ]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/scripts/get")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/scripts/get", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filter\":{\"field\":\"id\",\"value\":\"string\"}}}", CURLOPT_HTTPHEADER
=> [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/scripts/get"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filter\":
{\"field\":\"id\",\"value\":\"string\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/scripts/get"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filter\":{\"field\":\"id\",\"value\":\"string\"}}}", ParameterType.RequestBody); IRestResponse
response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ filterobject

Filter fields.

REQUEST Example 1

```
{ "request_data": { "filter": { "field": "name", "value": "PhishingScript" } } }
```

Responses

OK

Body

application/zip

RESPONSE

```
[application/zip content]
```

2.1.22.2 | Insert or update a script

post /public_api/v1/scripts/insert

Update or add a script by passing the YAML in a ZIP file.

You must have Instance Administrator permissions to run this endpoint.

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

Form parameters

▼ file File

The script in YAML in a zipped file.

format: binary

Example: `BINARY_DATA_HERE`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: multipart/form-data'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/scripts/insert'
-F 'file=BINARY_DATA_HERE'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "----
-011000010111000001101001\r\nContent-Disposition: form-data; name=\"file\"\r\n\r\nstring\r\n----
```

```
-0110000101110000011010001--\r\n" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE" } conn.request("POST", "/public_api/v1/scripts/insert", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/scripts/insert")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request.body = "-----011000010111000001101001\r\nContent-Disposition: form-data;
name=\"file\"\r\n\r\nstring\r\n-----011000010111000001101001--\r\n" response = http.request(request) puts
response.read_body
const data = new FormData(); data.append("file", "string"); const xhr = new XMLHttpRequest(); xhr.withCredentials =
true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/scripts/insert");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/scripts/insert")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"multipart/form-data; boundary=---011000010111000001101001") .body("-----011000010111000001101001\r\nContent-
Disposition: form-data; name=\"file\"\r\n\r\nstring\r\n-----011000010111000001101001--\r\n") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "multipart/form-data" ] let parameters = [ [ "name": "file", "value": "string" ] ] let boundary = "--
-011000010111000001101001" var body = "" var error: NSError? = nil for param in parameters { let paramName =
param["name"]! body += "--\(boundary)\r\n" body += "Content-Disposition:form-data; name=\"\(paramName)\"" if let
filename = param["fileName"] { let contentType = param["content-type"]! let fileContent = String(contentsOfFile:
filename, encoding: String.Encoding.utf8) if (error != nil) { print(error) } body += "; filename=\"\(filename)\"\r\n"
body += "Content-Type: \(contentType)\r\n\r\n" body += fileContent } else if let paramValue = param["value"] { body
+= "\r\n\r\n\(paramValue)" } } let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/scripts/insert")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/scripts/insert", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "-----011000010111000001101001\r\nContent-Disposition: form-data;
name=\"file\"\r\n\r\nstring\r\n-----011000010111000001101001--\r\n", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: multipart/form-data; boundary=---011000010111000001101001", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/scripts/insert"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: multipart/form-data; boundary=--
-011000010111000001101001"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "-----011000010111000001101001\r\nContent-Disposition: form-data;
name=\"file\"\r\n\r\nstring\r\n-----011000010111000001101001--\r\n"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/scripts/insert"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "multipart/form-data; boundary=---011000010111000001101001");
request.AddParameter("multipart/form-data; boundary=---011000010111000001101001", "----
-011000010111000001101001\r\nContent-Disposition: form-data; name=\"file\"\r\n\r\nstring\r\n----
-011000010111000001101001--\r\n", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Responses

OK

Body
application/json
▼ objects_countinteger

The number of scripts identified and attempted to be added.

▼ objectsobject
▶ failures_itemsarray

This field in the response indicates that the script IDs listed failed to be added.

RESPONSE

```
{ "objects_count": 0, "objects": { "failures_items": [ { "error": "example", "id": "example" } ] } }
```

2.1.22.3 | Delete a script

post /public_api/v1/scripts/delete

Delete a script by filtering based on its name or ID.

You must have **Instance Administrator** permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/scripts/delete'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filter\":
{\"field\":\"id\",\"value\":\"string\"}}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/scripts/delete",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/scripts/delete")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filter\":
{\"field\":\"id\",\"value\":\"string\"}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filter": { "field": "id", "value": "string" } } }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/scripts/delete"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/scripts/delete")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filter\":{\"field\":\"id\",\"value\":\"string\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["filter": [ "field": "id", "value": "string"
]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/scripts/delete")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/scripts/delete", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filter\":{\"field\":\"id\",\"value\":\"string\"}}}", CURLOPT_HTTPHEADER
=> [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/scripts/delete"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filter\":
{\"field\":\"id\",\"value\":\"string\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/scripts/delete"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filter\":{\"field\":\"id\",\"value\":\"string\"}}}", ParameterType.RequestBody); IRestResponse
response = client.Execute(request);
```

Body parameters

application/json
▼ request_dataobject
▶ filterobject

Filter fields.

REQUEST Example 1
```
{ "request_data": { "filters": { "field": "name", "value": "Phishingplaybook" } } }
```
Responses

OK

Body
application/json
▼ objects_countinteger

The number of scripts identified and attempted to be deleted.

▼ objectsobject
▶ failures_itemsarray

This field in the response indicates that the script IDs listed failed to be deleted.

▶ succeeded_itemsarray

This field in the response indicates that the script IDs listed were successfully deleted.

RESPONSE
```
{ "objects_count": 0, "objects": { "failures_items": [ { "error": "example", "id": "example" } ], "succeeded_items":
[ { "id": "example" } ] } }
```

2.1.23 | Dashboards

APIs for managing Dashboards

2.1.23.1 | Get dashboards

post /public_api/v1/dashboards/get

Get dashboard details by filtering based on the dashboard name, dashboard ID, time the dashboard was generated, or dashboard source.

You must have **Instance Administrator** permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/dashboards/get'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"dashboard_id\",\"operator\":\"EQ\",\"value\":\"string\"}]}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/dashboards/get", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-
8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/dashboards/get")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"dashboard_id\",\"operator\":\"EQ\",\"value\":\"string\"}]}}" response = http.request(request) puts
response.read_body
```

```
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "dashboard_id", "operator": "EQ", "value":
"string" } ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/dashboards/get");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/dashboards/get")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"dashboard_id\",\"operator\":\"EQ\",\"value\":\"string\"}]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["filters": [ [ "field": "dashboard_id",
"operator": "EQ", "value": "string" ] ]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject:
parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/dashboards/get")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/dashboards/get", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"dashboard_id\",\"operator\":\"EQ\",\"value\":\"string\"}]}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/dashboards/get"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"dashboard_id\",\"operator\":\"EQ\",\"value\":\"string\"}]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/dashboards/get"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"dashboard_id\",\"operator\":\"EQ\",\"value\":\"string\"}]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ filtersarray

An array of filter fields.

REQUEST Example 1

```
{ "request_data": { "filters": [ { "field": "name", "value": "Incident by Severity", "operator": "EQ" }, { "field":
"source", "value": "John Doe", "operator": "EQ" } ] } }
```

Responses

OK

Body

application/json

▼ objects_countinteger

▼ objectsarray

[

▶ dashboards_dataarray

▶ widgets_dataarray

]

▼ objects_typestring

RESPONSE Example 1

```
{ "objects_count": 2, "objects": [ { "dashboards_data": [ { "name": "John Doe", "description": null, "status":
"ENABLED", "layout": [ { "id": "row-9847", "data": [ { "key": "FAKE_KEY", "data": { "type": "Pie", "width": 5435435,
"height": 451 } }, { "key": "incidentsByAssignment", "data": { "type": "Bar", "width": 33.333333333333336, "height":
451 } }, { "key": "incidentsHistory", "data": { "type": "Timeline", "width": 33.333333333333336, "height": 451,
"time_frame": { "relativeTime": 86400000 }, "alerts_or_incidents": "incident" } } ] }, { "id": "row-2618", "data": [
{ "key": "xql_1683289526210", "data": { "type": "Custom XQL", "width": 33.333333333333336, "height": 11111, "params":
[ { "name": "name", "value": "orik" } ], "phrase": "dataset = drilldown | filter name = $name\n| view graph type =
gauge subtype = radial header = \"User Score\" yaxis = score maxscalerange = 100 headerfontsize = 25 ", "time_frame":
```

{ "relativeTime": 86400000 }, "viewOptions": { "type": "gauge", "commands": [{ "command": { "op": "=", "name": "subtype", "value": "radial" } }, { "command": { "op": "=", "name": "header", "value": "\"User Score\"" } }, { "command": { "op": "=", "name": "yaxis", "value": "score" } }, { "command": { "op": "=", "name": "maxscalerange", "value": "100" } }, { "command": { "op": "=", "name": "headerfontsize", "value": "25" } }] } } }, { "key": "topIncidentsByAlertSeverity", "data": { "type": "Table", "width": 33.333333333333336, "height": 408 } }, { "key": "hostsBySeverity", "data": { "type": "Table", "width": 33.333333333333336, "height": 408 } }] }], "default_dashboard_id": 132131, "global_id": "47052a2b17db4d799c4c5da38d203600", "metadata": { "params": [] } }], "widgets_data": [{ "widget_key": "xql_1683289526210", "title": "Drilldown | User Score", "creation_time": 1687877496472, "description": "", "data": { "params": [{ "name": "name", "value": "orik" }], "phrase": "dataset = drilldown | filter name = $name\n| view graph type = gauge subtype = radial header = \"User Score\" yaxis = score maxscalerange = 100 headerfontsize = 25 ", "time_frame": { "relativeTime": 86400000 }, "viewOptions": { "type": "gauge", "commands": [{ "command": { "op": "=", "name": "subtype", "value": "radial" } }, { "command": { "op": "=", "name": "header", "value": "\"User Score\"" } }, { "command": { "op": "=", "name": "yaxis", "value": "score" } }, { "command": { "op": "=", "name": "maxscalerange", "value": "100" } }, { "command": { "op": "=", "name": "headerfontsize", "value": "25" } }] } }, "support_time_range": true, "additional_info": { "query_tables": ["drilldown"], "query_uses_library": false }, "creator_mail": "user@company.com" }] }, { "dashboards_data": [{ "name": "John Doe", "description": null, "status": "ENABLED", "layout": [{ "id": "row-9847", "data": [{ "key": "FAKE_KEY", "data": { "type": "Pie", "width": 5435435, "height": 451 } }, { "key": "incidentsByAssignment", "data": { "type": "Bar", "width": 33.333333333333336, "height": 451 } }, { "key": "incidentsHistory", "data": { "type": "Timeline", "width": 33.333333333333336, "height": 451, "time_frame": { "relativeTime": 86400000 }, "alerts_or_incidents": "incident" } }] }, { "id": "row-2618", "data": [{ "key": "xql_1683289526210", "data": { "type": "Custom XQL", "width": 33.333333333333336, "height": 11111, "params": [{ "name": "name", "value": "orik" }], "phrase": "dataset = drilldown | filter name = $name\n| view graph type = gauge subtype = radial header = \"User Score\" yaxis = score maxscalerange = 100 headerfontsize = 25 ", "time_frame": { "relativeTime": 86400000 }, "viewOptions": { "type": "gauge", "commands": [{ "command": { "op": "=", "name": "subtype", "value": "radial" } }, { "command": { "op": "=", "name": "header", "value": "\"User Score\"" } }, { "command": { "op": "=", "name": "yaxis", "value": "score" } }, { "command": { "op": "=", "name": "maxscalerange", "value": "100" } }, { "command": { "op": "=", "name": "headerfontsize", "value": "25" } }] } } }, { "key": "topIncidentsByAlertSeverity", "data": { "type": "Table", "width": 33.333333333333336, "height": 408 } }, { "key": "hostsBySeverity", "data": { "type": "Table", "width": 33.333333333333336, "height": 408 } }] }], "default_dashboard_id": 132131, "global_id": "27f78777b427452eb54974d3c14bf67a", "metadata": { "params": [] } }], "widgets_data": [{ "widget_key": "xql_1683289526210", "title": "Drilldown | User Score", "creation_time": 1687877496472, "description": "", "data": { "params": [{ "name": "name", "value": "orik" }], "phrase": "dataset = drilldown | filter name = $name\n| view graph type = gauge subtype = radial header = \"User Score\" yaxis = score maxscalerange = 100 headerfontsize = 25 ", "time_frame": { "relativeTime": 86400000 }, "viewOptions": { "type": "gauge", "commands": [{ "command": { "op": "=", "name": "subtype", "value": "radial" } }, { "command": { "op": "=", "name": "header", "value": "\"User Score\"" } }, { "command": { "op": "=", "name": "yaxis", "value": "score" } }, { "command": { "op": "=", "name": "maxscalerange", "value": "100" } }, { "command": { "op": "=", "name": "headerfontsize", "value": "25" } }] } }, "support_time_range": true, "additional_info": { "query_tables": ["drilldown"], "query_uses_library": false }, "creator_mail": "user@company.com" }] }], "objects_type": "dashboards" }

Invalid field or operator.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.23.2 | Insert or update dashboards

post /public_api/v1/dashboards/insert

Add or update the dashboards retrieved by the Get dashboards API.

You must have Instance Administrator permissions to run this endpoint.

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/dashboards/insert'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"dashboards_data\":[{\"name\":\"string\",\"description\":\"string\",\"status\":\"string\",\"layout\":
[{\"id\":\"string\",\"data\":[{\"key\":\"string\",\"data\":
{\"type\":\"string\",\"width\":0,\"height\":0,\"time_frame\":
{\"relativeTime\":0},\"alerts_or_incidents\":\"string\"}}]}],\"default_dashboard_id\":0,\"global_id\":\"string\",\"metadata\":
{\"params\":[{}]}}],\"widgets_data\":
[{\"widget_key\":\"string\",\"title\":\"string\",\"creation_time\":0,\"description\":null,\"data\":
{\"phrase\":\"string\",\"time_frame\":{\"relativeTime\":0},\"viewOptions\":{\"type\":\"string\",\"commands\":
[{}]}},\"support_time_range\":true,\"additional_info\":{\"query_tables\":
[\"string\"],\"query_uses_library\":true},\"creator_mail\":\"string\"}]}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/dashboards/insert", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/dashboards/insert")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"dashboards_data\":[{\"name\":\"string\",\"description\":\"string\",\"status\":\"string\",\"layout\":
[{\"id\":\"string\",\"data\":[{\"key\":\"string\",\"data\":
{\"type\":\"string\",\"width\":0,\"height\":0,\"time_frame\":
{\"relativeTime\":0},\"alerts_or_incidents\":\"string\"}}]}],\"default_dashboard_id\":0,\"global_id\":\"string\",\"metadata\":
{\"params\":[{}]}}],\"widgets_data\":
[{\"widget_key\":\"string\",\"title\":\"string\",\"creation_time\":0,\"description\":null,\"data\":
{\"phrase\":\"string\",\"time_frame\":{\"relativeTime\":0},\"viewOptions\":{\"type\":\"string\",\"commands\":
[{}]}},\"support_time_range\":true,\"additional_info\":{\"query_tables\":
[\"string\"],\"query_uses_library\":true},\"creator_mail\":\"string\"}]}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "dashboards_data": [ { "name": "string", "description": "string",
"status": "string", "layout": [ { "id": "string", "data": [ { "key": "string", "data": { "type": "string", "width":
0, "height": 0, "time_frame": { "relativeTime": 0 }, "alerts_or_incidents": "string" } } ] } ],
"default_dashboard_id": 0, "global_id": "string", "metadata": { "params": [ {} ] } } ], "widgets_data": [ {
"widget_key": "string", "title": "string", "creation_time": 0, "description": null, "data": { "phrase": "string",
"time_frame": { "relativeTime": 0 }, "viewOptions": { "type": "string", "commands": [ {} ] } }, "support_time_range":
true, "additional_info": { "query_tables": [ "string" ], "query_uses_library": true }, "creator_mail": "string" } ] }
}); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function
() { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/dashboards/insert"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/dashboards/insert")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"dashboards_data\":
[{\"name\":\"string\",\"description\":\"string\",\"status\":\"string\",\"layout\":[{\"id\":\"string\",\"data\":
[{\"key\":\"string\",\"data\":{\"type\":\"string\",\"width\":0,\"height\":0,\"time_frame\":
{\"relativeTime\":0},\"alerts_or_incidents\":\"string\"}}]}],\"default_dashboard_id\":0,\"global_id\":\"string\",\"metadata\":
{\"params\":[{}]}}],\"widgets_data\":
[{\"widget_key\":\"string\",\"title\":\"string\",\"creation_time\":0,\"description\":null,\"data\":
{\"phrase\":\"string\",\"time_frame\":{\"relativeTime\":0},\"viewOptions\":{\"type\":\"string\",\"commands\":
[{}]}},\"support_time_range\":true,\"additional_info\":{\"query_tables\":
[\"string\"],\"query_uses_library\":true},\"creator_mail\":\"string\"}]}}") .asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "dashboards_data": [ [ "name": "string",
"description": "string", "status": "string", "layout": [ [ "id": "string", "data": [ [ "key": "string", "data": [
"type": "string", "width": 0, "height": 0, "time_frame": ["relativeTime": 0], "alerts_or_incidents": "string" ] ] ] ]
], "default_dashboard_id": 0, "global_id": "string", "metadata": ["params": [[]]] ] ], "widgets_data": [ [
"widget_key": "string", "title": "string", "creation_time": 0, "description": , "data": [ "phrase": "string",
"time_frame": ["relativeTime": 0], "viewOptions": [ "type": "string", "commands": [[]] ] ], "support_time_range":
true, "additional_info": [ "query_tables": ["string"], "query_uses_library": true ], "creator_mail": "string" ] ] ]]
as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/dashboards/insert")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/dashboards/insert", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"dashboards_data\":
[{\"name\":\"string\",\"description\":\"string\",\"status\":\"string\",\"layout\":[{\"id\":\"string\",\"data\":
[{\"key\":\"string\",\"data\":{\"type\":\"string\",\"width\":0,\"height\":0,\"time_frame\":
{\"relativeTime\":0},\"alerts_or_incidents\":\"string\"}}]}],\"default_dashboard_id\":0,\"global_id\":\"string\",\"metadata\":
{\"params\":[{}]}}],\"widgets_data\":
[{\"widget_key\":\"string\",\"title\":\"string\",\"creation_time\":0,\"description\":null,\"data\":
{\"phrase\":\"string\",\"time_frame\":{\"relativeTime\":0},\"viewOptions\":{\"type\":\"string\",\"commands\":
[{}]}},\"support_time_range\":true,\"additional_info\":{\"query_tables\":
[\"string\"],\"query_uses_library\":true},\"creator_mail\":\"string\"}]}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/dashboards/insert"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"dashboards_data\":
[{\"name\":\"string\",\"description\":\"string\",\"status\":\"string\",\"layout\":[{\"id\":\"string\",\"data\":
[{\"key\":\"string\",\"data\":{\"type\":\"string\",\"width\":0,\"height\":0,\"time_frame\":
{\"relativeTime\":0},\"alerts_or_incidents\":\"string\"}}]}],\"default_dashboard_id\":0,\"global_id\":\"string\",\"metadata\":
{\"params\":[{}]}}],\"widgets_data\":
[{\"widget_key\":\"string\",\"title\":\"string\",\"creation_time\":0,\"description\":null,\"data\":
{\"phrase\":\"string\",\"time_frame\":{\"relativeTime\":0},\"viewOptions\":{\"type\":\"string\",\"commands\":
[{}]}},\"support_time_range\":true,\"additional_info\":{\"query_tables\":
[\"string\"],\"query_uses_library\":true},\"creator_mail\":\"string\"}]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/dashboards/insert"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"dashboards_data\":
[{\"name\":\"string\",\"description\":\"string\",\"status\":\"string\",\"layout\":[{\"id\":\"string\",\"data\":
[{\"key\":\"string\",\"data\":{\"type\":\"string\",\"width\":0,\"height\":0,\"time_frame\":
{\"relativeTime\":0},\"alerts_or_incidents\":\"string\"}}]}],\"default_dashboard_id\":0,\"global_id\":\"string\",\"metadata\":
{\"params\":[{}]}}],\"widgets_data\":
[{\"widget_key\":\"string\",\"title\":\"string\",\"creation_time\":0,\"description\":null,\"data\":
{\"phrase\":\"string\",\"time_frame\":{\"relativeTime\":0},\"viewOptions\":{\"type\":\"string\",\"commands\":
[{}]}},\"support_time_range\":true,\"additional_info\":{\"query_tables\":
[\"string\"],\"query_uses_library\":true},\"creator_mail\":\"string\"}]}}", ParameterType.RequestBody); IRestResponse
response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ dashboards_dataarray

An array of dashboard details as retrieved by the Get dashboards API.

▶ widgets_dataarray

An array of relevant XQL widget details as retrieved by the Get dashboards API.

REQUEST Example 1


```
{ "request_data": [ { "dashboards_data": [ { "name": "John Doe", "description": null, "status": "ENABLED", "layout":
[ { "id": "row-5867", "data": [ { "key": "FAKE_KEY", "data": { "type": "Pie", "width": 5435435, "height": 451 } }, {
"key": "incidentsByAssignment", "data": { "type": "Bar", "width": 33.333333333333336, "height": 451 } }, { "key":
```

"incidentsHistory", "data": { "type": "Timeline", "width": 33.333333333333336, "height": 451, "time_frame": { "relativeTime": 86400000 }, "alerts_or_incidents": "incident" } }] }, { "id": "row-3678", "data": [{ "key": "xql_1583269521210", "data": { "type": "Custom XQL", "width": 33.333333333333336, "height": 11111, "params": [{ "name": "name", "value": "orik" }], "phrase": "dataset = drilldown | filter name = $name\n| view graph type = gauge subtype = radial header = \"User Score\" yaxis = score maxscalerange = 100 headerfontsize = 25 ", "time_frame": { "relativeTime": 86400000 }, "viewOptions": { "type": "gauge", "commands": [{ "command": { "op": "=", "name": "subtype", "value": "radial" } }, { "command": { "op": "=", "name": "header", "value": "\"User Score\"" } }, { "command": { "op": "=", "name": "yaxis", "value": "score" } }, { "command": { "op": "=", "name": "maxscalerange", "value": "100" } }, { "command": { "op": "=", "name": "headerfontsize", "value": "25" } }] } } }, { "key": "topIncidentsByAlertSeverity", "data": { "type": "Table", "width": 33.333333333333336, "height": 408 } }, { "key": "hostsBySeverity", "data": { "type": "Table", "width": 33.333333333333336, "height": 408 } }] }], "default_dashboard_id": 132131, "global_id": "47052a2b17db4d799c4c5da38d203600", "metadata": { "params": [] } }] }]
}

Responses

OK

Body

application/json

▼ added_objectsarray

Array listing the the global ID and internal ID of the objects added.

[

▶ global_idstring

▶ internal_idinteger

]

▼ updated_objectsarray

Array listing the the global ID and internal ID of the objects updated.

[

▶ global_idstring

▶ internal_idinteger

]

▼ errorsarray

List of errors including the global ID and the error message.

[

▶ global_idstring

▶ errorstring

]

RESPONSE Example 1

{ "added_objects": [{ "global_id": "47052a2b47db4d899c4c5da32d203700", "internal_id": 37 }], "updated_objects": [{ "global_id": "27f78778b427452ed54974d6c14cf67a", "internal_id": 44 }], "errors": [{ "global_id": "a62ed1dag81741fc948720e2a1be2801", "error": "This is the reason for the failure." }] }

2.1.23.3 | **Delete dashboards**

post /public_api/v1/dashboards/delete

Delete the dashboards retrieved by the Get dashboards API.

You must have **Instance Administrator** permissions to run this endpoint.

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
```

```
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/dashboards/delete'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"dashboard_id\",\"operator\":\"EQ\",\"value\":\"string\"}]}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/dashboards/delete", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/dashboards/delete")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"dashboard_id\",\"operator\":\"EQ\",\"value\":\"string\"}]}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "dashboard_id", "operator": "EQ", "value":
"string" } ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/dashboards/delete");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/dashboards/delete")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"dashboard_id\",\"operator\":\"EQ\",\"value\":\"string\"}]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["filters": [ [ "field": "dashboard_id",
"operator": "EQ", "value": "string" ] ]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject:
parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/dashboards/delete")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/dashboards/delete", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"dashboard_id\",\"operator\":\"EQ\",\"value\":\"string\"}]}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/dashboards/delete"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"dashboard_id\",\"operator\":\"EQ\",\"value\":\"string\"}]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/dashboards/delete"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"dashboard_id\",\"operator\":\"EQ\",\"value\":\"string\"}]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ filtersarray

An array of filter fields.

REQUEST Example 1


```
{ "request_data": [ { "dashboards_data": [ { "name": "John Doe", "description": null, "status": "ENABLED", "layout":
[ { "id": "row-5867", "data": [ { "key": "FAKE_KEY", "data": { "type": "Pie", "width": 5435435, "height": 451 } }, {
"key": "incidentsByAssignment", "data": { "type": "Bar", "width": 33.333333333333336, "height": 451 } }, { "key":
"incidentsHistory", "data": { "type": "Timeline", "width": 33.333333333333336, "height": 451, "time_frame": {
"relativeTime": 86400000 }, "alerts_or_incidents": "incident" } } ] }, { "id": "row-3678", "data": [ { "key":
"xql_1583269521210", "data": { "type": "Custom XQL", "width": 33.333333333333336, "height": 11111, "params": [ {
"name": "name", "value": "orik" } ], "phrase": "dataset = drilldown | filter name = $name\n| view graph type = gauge
subtype = radial header = \"User Score\" yaxis = score maxscalerange = 100 headerfontsize = 25 ", "time_frame": {
"relativeTime": 86400000 }, "viewOptions": { "type": "gauge", "commands": [ { "command": { "op": "=", "name":
```

"subtype", "value": "radial" } }, { "command": { "op": "=", "name": "header", "value": "\"User Score\"" } }, { "command": { "op": "=", "name": "yaxis", "value": "score" } }, { "command": { "op": "=", "name": "maxscalerange", "value": "100" } }, { "command": { "op": "=", "name": "headerfontsize", "value": "25" } }] } } }, { "key": "topIncidentsByAlertSeverity", "data": { "type": "Table", "width": 33.333333333333336, "height": 408 } }, { "key": "hostsBySeverity", "data": { "type": "Table", "width": 33.333333333333336, "height": 408 } }] }], "default_dashboard_id": 132131, "global_id": "47052a2b17db4d799c4c5da38d203600", "metadata": { "params": [] } }] }]
}
Responses

OK

Body
application/json
▼ objects_countinteger
▼ objectsobject
▶ succeeded_itemsarray

List of the global ID and internal ID of dashboards successfully deleted.

▶ failures_itemsarray

List of global IDs and internal IDs that were not deleted and the error messages explaining why.

RESPONSE Example 1
{ "objects_count": 3, "objects": { "succeeded_items": [{ "global_id": "47052a2b17db4e799c4c5db38d603600", "internal_id": 27 }, { "global_id": "a62ab1daf82741fc948760e2e1be2401", "internal_id": 31 }], "failures_items": [{ "global_id": "25f78778b427452ec54974d3c14bf67a", "internal_id": 55, "error": "Dashboard is not allowed to be deleted." }] } }

Invalid field or operator.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

2.1.24 | Widgets

APIs for managing widgets

2.1.24.1 | Get widgets

post /public_api/v1/widgets/get

Get widget details by filtering based on the widget title and widget creator.

Note: The endpoint only returns XQL widgets and not predefined widgets.

You must have Instance Administrator permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}
Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/widgets/get'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"title\",\"operator\":\"EQ\",\"value\":\"string\"}]}}" headers = { 'Authorization': "SOME_STRING_VALUE",
'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/widgets/get", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-
8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/widgets/get") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"title\",\"operator\":\"EQ\",\"value\":\"string\"}]}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "title", "operator": "EQ", "value": "string"
} ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange",
function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST",
"https://api-yourfqdn/public_api/v1/widgets/get"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/widgets/get")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"title\",\"operator\":\"EQ\",\"value\":\"string\"}]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["filters": [ [ "field": "title", "operator":
"EQ", "value": "string" ] ]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/widgets/get")!
as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST"
request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/widgets/get", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"title\",\"operator\":\"EQ\",\"value\":\"string\"}]}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/widgets/get"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"title\",\"operator\":\"EQ\",\"value\":\"string\"}]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/widgets/get"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"title\",\"operator\":\"EQ\",\"value\":\"string\"}]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```
Body parameters
application/json
▼ request_dataobject
▶ filtersarray

An array of filter fields.

REQUEST Example 1
```
{ "request_data": { "filters": [ { "field": "title", "value": "Widget A", "operator": "EQ" }, { "field":
"created_by", "value": "John Doe", "operator": "EQ" } ] } }
```
Responses

OK

Body
application/json
▼ objects_countinteger

Number of widgets returned.

▼ objectsarray
[
▶ widgets_dataarray
]
▼ objects_typestring
RESPONSE Example 1

```
{ "objects_count": 2, "objects": [ { "widgets_data": [ { "widget_key": "xql_1646052681403", "title": "Widget A",
"creation_time": 1653303166334, "description": null, "data": { "phrase": "cold_dataset = endpoints", "time_frame": {
"relativeTime": 86400000 }, "viewOptions": { "type": "table", "commands": [] } }, "support_time_range": true,
"additional_info": { "query_tables": [ "endpoints" ], "query_uses_library": false }, "creator_mail":
"user@company.com" } ] }, { "widgets_data": [ { "widget_key": "xql_1346826725701", "title": "Widget B",
"creation_time": 1653302483610, "description": null, "data": { "phrase": "dataset = endpoints ", "time_frame": {
"relativeTime": 86400000 }, "viewOptions": { "type": "table", "commands": [] }, "gridRawStorageInfo": { "sort": null,
"coldefs": {}, "rowHeight": "{\"rowHeight\":\"regular\",\"gridRowsHeight\":\"medium-row\"}", "columnWidth": null } },
"support_time_range": true, "additional_info": { "query_tables": [ "endpoints" ], "query_uses_library": false },
"creator_mail": "user@company.com" } ] } ], "objects_type": "widgets" }
```


Invalid field or operator.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

2.1.24.2 | Insert or update widgets

post /public_api/v1/widgets/insert

Update or add the widgets retrieved by the Get widgets API.

You must have Instance Administrator permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
```

```
'https://api-yourfqdn/public_api/v1/widgets/insert'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
[{\"widgets_data\":
[{\"widget_key\":\"string\",\"title\":\"string\",\"creation_time\":0,\"description\":\"string\",\"data\":
{\"phrase\":\"string\",\"time_frame\":{\"relativeTime\":0},\"viewOptions\":{\"type\":\"string\",\"commands\":
[{}]},\"gridRawStorageInfo\":{\"sort\":null,\"coldefs\":
{},\"rowHeight\":\"string\",\"columnWidth\":null}},\"support_time_range\":true,\"additional_info\":{\"query_tables\":
[\"string\"],\"query_uses_library\":true},\"creator_mail\":\"string\"}]}]}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/widgets/insert", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-
8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/widgets/insert")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
[{\"widgets_data\":
[{\"widget_key\":\"string\",\"title\":\"string\",\"creation_time\":0,\"description\":\"string\",\"data\":
{\"phrase\":\"string\",\"time_frame\":{\"relativeTime\":0},\"viewOptions\":{\"type\":\"string\",\"commands\":
[{}]},\"gridRawStorageInfo\":{\"sort\":null,\"coldefs\":
{},\"rowHeight\":\"string\",\"columnWidth\":null}},\"support_time_range\":true,\"additional_info\":{\"query_tables\":
[\"string\"],\"query_uses_library\":true},\"creator_mail\":\"string\"}]}]}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": [ { "widgets_data": [ { "widget_key": "string", "title": "string",
"creation_time": 0, "description": "string", "data": { "phrase": "string", "time_frame": { "relativeTime": 0 },
"viewOptions": { "type": "string", "commands": [ {} ] }, "gridRawStorageInfo": { "sort": null, "coldefs": {},
"rowHeight": "string", "columnWidth": null } }, "support_time_range": true, "additional_info": { "query_tables": [
"string" ], "query_uses_library": true }, "creator_mail": "string" } ] } ] }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/widgets/insert");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/widgets/insert")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":[{\"widgets_data\":
[{\"widget_key\":\"string\",\"title\":\"string\",\"creation_time\":0,\"description\":\"string\",\"data\":
{\"phrase\":\"string\",\"time_frame\":{\"relativeTime\":0},\"viewOptions\":{\"type\":\"string\",\"commands\":
[{}]},\"gridRawStorageInfo\":{\"sort\":null,\"coldefs\":
{},\"rowHeight\":\"string\",\"columnWidth\":null}},\"support_time_range\":true,\"additional_info\":{\"query_tables\":
[\"string\"],\"query_uses_library\":true},\"creator_mail\":\"string\"}]}]}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [["widgets_data": [ [ "widget_key": "string",
"title": "string", "creation_time": 0, "description": "string", "data": [ "phrase": "string", "time_frame":
["relativeTime": 0], "viewOptions": [ "type": "string", "commands": [[]] ], "gridRawStorageInfo": [ "sort": ,
"coldefs": [], "rowHeight": "string", "columnWidth": ] ], "support_time_range": true, "additional_info": [
"query_tables": ["string"], "query_uses_library": true ], "creator_mail": "string" ] ]]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/widgets/insert")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/widgets/insert", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":[{\"widgets_data\":
[{\"widget_key\":\"string\",\"title\":\"string\",\"creation_time\":0,\"description\":\"string\",\"data\":
{\"phrase\":\"string\",\"time_frame\":{\"relativeTime\":0},\"viewOptions\":{\"type\":\"string\",\"commands\":
[{}]},\"gridRawStorageInfo\":{\"sort\":null,\"coldefs\":
{},\"rowHeight\":\"string\",\"columnWidth\":null}},\"support_time_range\":true,\"additional_info\":{\"query_tables\":
[\"string\"],\"query_uses_library\":true},\"creator_mail\":\"string\"}]}]}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/widgets/insert"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
```

```
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":[{\"widgets_data\":
[{\"widget_key\":\"string\",\"title\":\"string\",\"creation_time\":0,\"description\":\"string\",\"data\":
{\"phrase\":\"string\",\"time_frame\":{\"relativeTime\":0},\"viewOptions\":{\"type\":\"string\",\"commands\":
[{}]},\"gridRawStorageInfo\":{\"sort\":null,\"coldefs\":
{},\"rowHeight\":\"string\",\"columnWidth\":null}},\"support_time_range\":true,\"additional_info\":{\"query_tables\":
[\"string\"],\"query_uses_library\":true},\"creator_mail\":\"string\"}]}]}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/widgets/insert"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":[{\"widgets_data\":
[{\"widget_key\":\"string\",\"title\":\"string\",\"creation_time\":0,\"description\":\"string\",\"data\":
{\"phrase\":\"string\",\"time_frame\":{\"relativeTime\":0},\"viewOptions\":{\"type\":\"string\",\"commands\":
[{}]},\"gridRawStorageInfo\":{\"sort\":null,\"coldefs\":
{},\"rowHeight\":\"string\",\"columnWidth\":null}},\"support_time_range\":true,\"additional_info\":{\"query_tables\":
[\"string\"],\"query_uses_library\":true},\"creator_mail\":\"string\"}]}]}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataarray

[

▶ widgets_dataarray

An array of widget details as retrieved by the Get widgets API.

]

REQUEST Example 1

```
{ "request_data": [ { "widgets_data": [ { "widget_key": "xql_1646052681403", "title": "Widget A", "creation_time":
1653303166334, "description": null, "data": { "phrase": "cold_dataset = endpoints", "time_frame": { "relativeTime":
86400000 }, "viewOptions": { "type": "table", "commands": [] } }, "support_time_range": true, "additional_info": {
"query_tables": [ "endpoints" ], "query_uses_library": false }, "creator_mail": "user@company.com" } ] }, {
"widgets_data": [ { "widget_key": "xql_1346826725701", "title": "Widget B", "creation_time": 1653302483610,
"description": null, "data": { "phrase": "dataset = endpoints ", "time_frame": { "relativeTime": 86400000 },
"viewOptions": { "type": "table", "commands": [] }, "gridRawStorageInfo": { "sort": null, "coldefs": {}, "rowHeight":
"{\"rowHeight\":\"regular\",\"gridRowsHeight\":\"medium-row\"}", "columnWidth": null } }, "support_time_range": true,
"additional_info": { "query_tables": [ "endpoints" ], "query_uses_library": false }, "creator_mail":
"user@company.com" } ] } ] }
```

Responses

OK

Body

application/json

▼ added_objectsarray

List of widget keys and titles for all widgets successfully added.

[

▶ keystring

▶ titlestring

]

▼ updated_objectsarray

List of widget keys and titles for all widgets successfully updated.

[

▶ keystring

▶ titlestring

]

▼ errorsarray

List of widget keys and error messages for all widgets that failed to be added or updated.

[

▶ keystring

▶ errorstring

]

RESPONSE Example 1

```
{ "added_objects": [ { "key": "xql_1626053781409", "title": "Widget 1" } ], "updated_objects": [ { "key":
"xql_1636815725608", "title": "Widget 45" } ], "errors": [ { "key": "xql_1651607948383", "error": "Failed importing
widget" } ] }
```

2.1.24.3 | Delete widgets

post /public_api/v1/widgets/delete

Delete the widgets retrieved by the Get widgets API.

You must have Instance Administrator permissions to run this endpoint.

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/widgets/delete'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"created_by\",\"value\":\"EQ\",\"operator\":\"string\"}]}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/widgets/delete", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-
8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/widgets/delete")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"created_by\",\"value\":\"EQ\",\"operator\":\"string\"}]}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "created_by", "value": "EQ", "operator":
"string" } ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/widgets/delete");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/widgets/delete")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"created_by\",\"value\":\"EQ\",\"operator\":\"string\"}]}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["filters": [ [ "field": "created_by",
"value": "EQ", "operator": "string" ] ]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject:
parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/widgets/delete")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/widgets/delete", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"created_by\",\"value\":\"EQ\",\"operator\":\"string\"}]}}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/widgets/delete"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"created_by\",\"value\":\"EQ\",\"operator\":\"string\"}]}}"); CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/widgets/delete"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"created_by\",\"value\":\"EQ\",\"operator\":\"string\"}]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
application/json
▼ request_dataobject
▶ filtersarray

An array of filter fields.

REQUEST Example 1
```
{ "request_data": { "filters": [ { "field": "title", "value": "Widget A", "operator": "EQ" }, { "field":
"created_by", "value": "John Doe", "operator": "EQ" } ] } }
```

Responses

OK

Body
application/json
▼ objects_countinteger

Number of widgets deleted.

▼ objectsarray[string]

List of widget IDs of widgets successfully deleted.

RESPONSE Example 1
```
{ "objects_count": 3, "objects": [ "xql_1656052681401", "xql_1636816727601", "xql_1653607348382" ] }
```

Invalid field or operator.

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

2.1.25 | Issues

APIs for managing issues

2.1.25.1 | Create a new issue

post /public_api/v1/issue

This endpoint allows users to create a new issue by providing the necessary details. **Users can only create one issue at a time.**

- The request must include the following required fields:
  - `name`
  - `description`
  - `observation_time`
  - `domain`
  - `category`

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/issue'
-d ''
import http.client conn = http.client.HTTPSConnection("api-") payload = "{\"request_data\":{\"issue\":
{\"owner\":\"CWP\",\"name\":\"Unauthorized Access Detected\",\"description\":\"An unauthorized login attempt was
detected from an unknown IP
address.\",\"observation_time\":1700000000000,\"domain\":\"SECURITY\",\"category\":\"CONFIGURATION\",\"asset_ids\":
[\"asset-456\"],\"mitre_tactics\":[\"TA0001\",\"TA0002\"],\"mitre_techniques\":
[\"T1003\",\"T1059\"],\"type\":\"Threat Intelligence\",\"extended_description\":\"This alert was triggered due to
multiple failed login attempts within a short time frame.\",\"impact\":\"Potential unauthorized system
access\",\"tags\":
[\"critical\",\"network\"],\"is_excluded\":false,\"is_starred\":true,\"assigned_to\":\"security_team_lead\",\"assigned_to_pret
Smith\",\"severity\":\"HIGH\",\"normalized_fields\":
{\"xdm.source.location.country\":\"US\",\"xdm.source.ipv4\":\"192.168.1.1\",\"xdm.source.host.ipv4_addresses\":
[\"192.168.1.2\",\"192.168.1.3\"],\"xdm.source.identity.username\":\"admin\",\"xdm.source.process.causality_id\":\"abc123\",\"
-arg1 -
arg2\",\"xdm.source.process.executable.filename\":\"process_executable\",\"xdm.source.process.name\":\"process_name\",\"xdm.so
-
option\",\"xdm.target.process.executable.sha256\":\"7b21d50d6270f95b5a2cf582bf94b315cd75a034dd9478c0e5b4089bbd9b59ac\",\"xdm.t
[\"string\"],\"xdm.target.process.executable.path\":[\"string\"],\"xdm.target.ipv4\":
[\"string\"],\"xdm.target.host.ipv4_addresses\":[\"10.0.0.2\",\"10.0.0.3\"],\"xdm.target.host.ipv6_addresses\":
[\"string\"],\"xdm.target.ipv6\":
[\"10.0.0.2\",\"10.0.0.3\"],\"xdm.target.port\":8080,\"xdm.target.location.country\":\"US\",\"xdm.target.host.hostname\":\"hos
east-
1\",\"xdm.cloud.function.id\":\"cloud_func_123\",\"xdm.cloud.function.name\":\"cloud_function\",\"xdm.cloud.function.version\"
1\",\"xdm.vulnerability.cve_id\":\"CVE-2021-
12345\",\"xdm.vulnerability.severity\":\"HIGH\",\"xdm.vulnerability.fix_versions\":
[\"1.0.1\",\"1.0.2\"],\"xdm.vulnerability.cve_risk_factors\":
[\"Exploitability\",\"Impact\"],\"xdm.vulnerability.cvss_score\":7.8,\"xdm.vulnerability.cvss_vector\":\"AV:N/AC:L/PR:N/UI:N/S
Key\",\"xdm.secret.unique_identifier\":\"secret_id_123\",\"xdm.secret.snippet\":\"API Key:
12345\",\"xdm.secret.layer_id\":\"secret_layer123\",\"xdm.file.filename\":\"file.txt\",\"xdm.file.path\":\"/path/to/file.txt\"
[\"read\",\"write\"],\"xdm.file.permissions.group\":[\"read\"],\"xdm.file.permissions.others\":
[\"read\"],\"xdm.file.position.start.line\":1,\"xdm.file.position.start.character\":0,\"xdm.file.position.end.line\":100,\"xdm
{}}}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type':
"application/json" } conn.request("POST", "%7B%7Bfqdn%7D%7D/public_api/v1/issue", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"issue\":
{\"owner\":\"CWP\",\"name\":\"Unauthorized Access Detected\",\"description\":\"An unauthorized login attempt was
detected from an unknown IP
address.\",\"observation_time\":1700000000000,\"domain\":\"SECURITY\",\"category\":\"CONFIGURATION\",\"asset_ids\":
[\"asset-456\"],\"mitre_tactics\":[\"TA0001\",\"TA0002\"],\"mitre_techniques\":
[\"T1003\",\"T1059\"],\"type\":\"Threat Intelligence\",\"extended_description\":\"This alert was triggered due to
multiple failed login attempts within a short time frame.\",\"impact\":\"Potential unauthorized system
access\",\"tags\":
[\"critical\",\"network\"],\"is_excluded\":false,\"is_starred\":true,\"assigned_to\":\"security_team_lead\",\"assigned_to_pret
Smith\",\"severity\":\"HIGH\",\"normalized_fields\":
{\"xdm.source.location.country\":\"US\",\"xdm.source.ipv4\":\"192.168.1.1\",\"xdm.source.host.ipv4_addresses\":
[\"192.168.1.2\",\"192.168.1.3\"],\"xdm.source.identity.username\":\"admin\",\"xdm.source.process.causality_id\":\"abc123\",\"
-arg1 -
arg2\",\"xdm.source.process.executable.filename\":\"process_executable\",\"xdm.source.process.name\":\"process_name\",\"xdm.so
-
option\",\"xdm.target.process.executable.sha256\":\"7b21d50d6270f95b5a2cf582bf94b315cd75a034dd9478c0e5b4089bbd9b59ac\",\"xdm.t
```

```
[\"string\"],\"xdm.target.process.executable.path\":[\"string\"],\"xdm.target.ipv4\":
[\"string\"],\"xdm.target.host.ipv4_addresses\":[\"10.0.0.2\",\"10.0.0.3\"],\"xdm.target.host.ipv6_addresses\":
[\"string\"],\"xdm.target.ipv6\":
[\"10.0.0.2\",\"10.0.0.3\"],\"xdm.target.port\":8080,\"xdm.target.location.country\":\"US\",\"xdm.target.host.hostname\":\"hos
east-
1\",\"xdm.cloud.function.id\":\"cloud_func_123\",\"xdm.cloud.function.name\":\"cloud_function\",\"xdm.cloud.function.version\"
1\",\"xdm.vulnerability.cve_id\":\"CVE-2021-
12345\",\"xdm.vulnerability.severity\":\"HIGH\",\"xdm.vulnerability.fix_versions\":
[\"1.0.1\",\"1.0.2\"],\"xdm.vulnerability.cve_risk_factors\":
[\"Exploitability\",\"Impact\"],\"xdm.vulnerability.cvss_score\":7.8,\"xdm.vulnerability.cvss_vector\":\"AV:N/AC:L/PR:N/UI:N/S
Key\",\"xdm.secret.unique_identifier\":\"secret_id_123\",\"xdm.secret.snippet\":\"API Key:
12345\",\"xdm.secret.layer_id\":\"secret_layer123\",\"xdm.file.filename\":\"file.txt\",\"xdm.file.path\":\"/path/to/file.txt\"
[\"read\",\"write\"],\"xdm.file.permissions.group\":[\"read\"],\"xdm.file.permissions.others\":
[\"read\"],\"xdm.file.position.start.line\":1,\"xdm.file.position.start.character\":0,\"xdm.file.position.end.line\":100,\"xdm
{}}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "issue": { "owner": "CWP", "name": "Unauthorized Access Detected",
"description": "An unauthorized login attempt was detected from an unknown IP address.", "observation_time":
1700000000000, "domain": "SECURITY", "category": "CONFIGURATION", "asset_ids": [ "asset-456" ], "mitre_tactics": [
"TA0001", "TA0002" ], "mitre_techniques": [ "T1003", "T1059" ], "type": "Threat Intelligence",
"extended_description": "This alert was triggered due to multiple failed login attempts within a short time frame.",
"impact": "Potential unauthorized system access", "tags": [ "critical", "network" ], "is_excluded": false,
"is_starred": true, "assigned_to": "security_team_lead", "assigned_to_pretty": "Alice Smith", "severity": "HIGH",
"normalized_fields": { "xdm.source.location.country": "US", "xdm.source.ipv4": "192.168.1.1",
"xdm.source.host.ipv4_addresses": [ "192.168.1.2", "192.168.1.3" ], "xdm.source.identity.username": "admin",
"xdm.source.process.causality_id": "abc123", "xdm.source.process.command_line": "/usr/bin/process -arg1 -arg2",
"xdm.source.process.executable.filename": "process_executable", "xdm.source.process.name": "process_name",
"xdm.source.process.executable.path": "/usr/bin/process_executable", "xdm.source.process.executable.sha256":
"f9c7b6e24f7e93d8d3e5c76f8b1b88cd8f17b34a7a4a2e3d5b2dbf09f5b8fdc2", "xdm.source.host.hostname": "hostname1",
"xdm.source.host.os_family": "Linux", "xdm.source.agent.identifier": "agent123", "xdm.source.agent.installation_id":
"installation123", "xdm.source.host.fqdn": "hostname1.domain.com", "xdm.source.process.executable.signature_status":
"Valid", "xdm.target.file.filename": "target_file.txt", "xdm.target.module.filename": "target_module.so",
"xdm.target.file.sha256": "d4bfc6fabe8d6d1b76e5b441dc8d01758276281f56c929b282ac5c3ee704c431",
"xdm.target.module.sha256": "7f4eafdad74bfedabf370a3725a5077c", "xdm.target.process.command_line":
"/usr/bin/target_process -option", "xdm.target.process.executable.sha256":
"7b21d50d6270f95b5a2cf582bf94b315cd75a034dd9478c0e5b4089bbd9b59ac", "xdm.target.process.executable.signature_status":
"Signed", "xdm.target.process.executable.signer": [ "string" ], "xdm.target.process.executable.path": [ "string" ],
"xdm.target.ipv4": [ "string" ], "xdm.target.host.ipv4_addresses": [ "10.0.0.2", "10.0.0.3" ],
"xdm.target.host.ipv6_addresses": [ "string" ], "xdm.target.ipv6": [ "10.0.0.2", "10.0.0.3" ], "xdm.target.port":
8080, "xdm.target.location.country": "US", "xdm.target.host.hostname": "hostname", "xdm.target.identity.username":
"user1", "xdm.target.url": "https://example.com", "xdm.target.process.executable.filename": "target_process",
"xdm.target.process.name": "target_process", "xdm.target.agent.identifier": "target_agent",
"xdm.target.registry.value": "registry_value", "xdm.target.registry.data": "registry_data",
"xdm.target.registry.key": "registry_key", "xdm.email.attachment.sha256": "a1b2c3d4e5f6789abcde1234567890f2",
"xdm.email.attachment.filename": "attachment.pdf", "xdm.email.sender": "sender@example.com", "xdm.event.type":
"Intrusion", "xdm.cloud.provider": "AWS", "xdm.cloud.project": "CloudProject1", "xdm.cloud.project_id":
"cloud_project_id_123", "xdm.cloud.region": "us-east-1", "xdm.cloud.function.id": "cloud_func_123",
"xdm.cloud.function.name": "cloud_function", "xdm.cloud.function.version": "v1.0.0", "xdm.cloud.function.request_id":
"req_123", "xdm.cloud.function.runtime": "nodejs", "xdm.observer.unique_identifier": "observer123",
"xdm.observer.type": "Server", "xdm.observer.sub_type": "Linux", "xdm.observer.name": "Observer 1",
"xdm.vulnerability.cve_id": "CVE-2021-12345", "xdm.vulnerability.severity": "HIGH", "xdm.vulnerability.fix_versions":
[ "1.0.1", "1.0.2" ], "xdm.vulnerability.cve_risk_factors": [ "Exploitability", "Impact" ],
"xdm.vulnerability.cvss_score": 7.8, "xdm.vulnerability.cvss_vector": "AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H",
"xdm.software_package.version": "1.0.0", "xdm.software_package.purl": "pkg:maven/com.example/software@1.0.0",
"xdm.software_package.layer_id": "layer123", "xdm.software_package.type": "Library",
"xdm.software_package.installation_type": "Automatic", "xdm.software_package.package_manager": "npm",
"xdm.software_package.dependency_type": "Direct", "xdm.software_package.language": "JavaScript",
"xdm.malware.verdict": "Malicious", "xdm.malware.virus_total_link": "https://www.virustotal.com/gui/file/abcd1234",
"xdm.malware.layer_id": "malware_layer123", "xdm.secret.secret_type": "API Key", "xdm.secret.unique_identifier":
"secret_id_123", "xdm.secret.snippet": "API Key: 12345", "xdm.secret.layer_id": "secret_layer123",
"xdm.file.filename": "file.txt", "xdm.file.path": "/path/to/file.txt", "xdm.file.sha256":
"abc1234567890def0987654321", "xdm.file.size": 1024, "xdm.file.last_modified": 1615465123,
"xdm.file.metadata_change_time": 1615465000, "xdm.file.owner_id": "user1", "xdm.file.owner_name": "fileowner",
"xdm.file.group_id": "group1", "xdm.file.group_name": "groupname", "xdm.file.permissions.owner": [ "read", "write" ],
"xdm.file.permissions.group": [ "read" ], "xdm.file.permissions.others": [ "read" ], "xdm.file.position.start.line":
1, "xdm.file.position.start.character": 0, "xdm.file.position.end.line": 100, "xdm.file.position.end.character": 80,
"xdm.url": "https://example.com", "xdm.domain": "example.com", "xdm.application_protocol": "HTTPS" },
"custom_fields": {} } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
```

```
console.log(this.responseText); } }); xhr.open("POST", "https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"issue\":{\"owner\":\"CWP\",\"name\":\"Unauthorized Access
Detected\",\"description\":\"An unauthorized login attempt was detected from an unknown IP
address.\",\"observation_time\":1700000000000,\"domain\":\"SECURITY\",\"category\":\"CONFIGURATION\",\"asset_ids\":
[\"asset-456\"],\"mitre_tactics\":[\"TA0001\",\"TA0002\"],\"mitre_techniques\":
[\"T1003\",\"T1059\"],\"type\":\"Threat Intelligence\",\"extended_description\":\"This alert was triggered due to
multiple failed login attempts within a short time frame.\",\"impact\":\"Potential unauthorized system
access\",\"tags\":
[\"critical\",\"network\"],\"is_excluded\":false,\"is_starred\":true,\"assigned_to\":\"security_team_lead\",\"assigned_to_pret
Smith\",\"severity\":\"HIGH\",\"normalized_fields\":
{\"xdm.source.location.country\":\"US\",\"xdm.source.ipv4\":\"192.168.1.1\",\"xdm.source.host.ipv4_addresses\":
[\"192.168.1.2\",\"192.168.1.3\"],\"xdm.source.identity.username\":\"admin\",\"xdm.source.process.causality_id\":\"abc123\",\"
-arg1 -
arg2\",\"xdm.source.process.executable.filename\":\"process_executable\",\"xdm.source.process.name\":\"process_name\",\"xdm.so
-
option\",\"xdm.target.process.executable.sha256\":\"7b21d50d6270f95b5a2cf582bf94b315cd75a034dd9478c0e5b4089bbd9b59ac\",\"xdm.t
[\"string\"],\"xdm.target.process.executable.path\":[\"string\"],\"xdm.target.ipv4\":
[\"string\"],\"xdm.target.host.ipv4_addresses\":[\"10.0.0.2\",\"10.0.0.3\"],\"xdm.target.host.ipv6_addresses\":
[\"string\"],\"xdm.target.ipv6\":
[\"10.0.0.2\",\"10.0.0.3\"],\"xdm.target.port\":8080,\"xdm.target.location.country\":\"US\",\"xdm.target.host.hostname\":\"hos
east-
1\",\"xdm.cloud.function.id\":\"cloud_func_123\",\"xdm.cloud.function.name\":\"cloud_function\",\"xdm.cloud.function.version\"
1\",\"xdm.vulnerability.cve_id\":\"CVE-2021-
12345\",\"xdm.vulnerability.severity\":\"HIGH\",\"xdm.vulnerability.fix_versions\":
[\"1.0.1\",\"1.0.2\"],\"xdm.vulnerability.cve_risk_factors\":
[\"Exploitability\",\"Impact\"],\"xdm.vulnerability.cvss_score\":7.8,\"xdm.vulnerability.cvss_vector\":\"AV:N/AC:L/PR:N/UI:N/S
Key\",\"xdm.secret.unique_identifier\":\"secret_id_123\",\"xdm.secret.snippet\":\"API Key:
12345\",\"xdm.secret.layer_id\":\"secret_layer123\",\"xdm.file.filename\":\"file.txt\",\"xdm.file.path\":\"/path/to/file.txt\"
[\"read\",\"write\"],\"xdm.file.permissions.group\":[\"read\"],\"xdm.file.permissions.others\":
[\"read\"],\"xdm.file.position.start.line\":1,\"xdm.file.position.start.character\":0,\"xdm.file.position.end.line\":100,\"xdm
{}}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["issue": [ "owner": "CWP", "name":
"Unauthorized Access Detected", "description": "An unauthorized login attempt was detected from an unknown IP
address.", "observation_time": 1700000000000, "domain": "SECURITY", "category": "CONFIGURATION", "asset_ids":
["asset-456"], "mitre_tactics": ["TA0001", "TA0002"], "mitre_techniques": ["T1003", "T1059"], "type": "Threat
Intelligence", "extended_description": "This alert was triggered due to multiple failed login attempts within a short
time frame.", "impact": "Potential unauthorized system access", "tags": ["critical", "network"], "is_excluded":
false, "is_starred": true, "assigned_to": "security_team_lead", "assigned_to_pretty": "Alice Smith", "severity":
"HIGH", "normalized_fields": [ "xdm.source.location.country": "US", "xdm.source.ipv4": "192.168.1.1",
"xdm.source.host.ipv4_addresses": ["192.168.1.2", "192.168.1.3"], "xdm.source.identity.username": "admin",
"xdm.source.process.causality_id": "abc123", "xdm.source.process.command_line": "/usr/bin/process -arg1 -arg2",
"xdm.source.process.executable.filename": "process_executable", "xdm.source.process.name": "process_name",
"xdm.source.process.executable.path": "/usr/bin/process_executable", "xdm.source.process.executable.sha256":
"f9c7b6e24f7e93d8d3e5c76f8b1b88cd8f17b34a7a4a2e3d5b2dbf09f5b8fdc2", "xdm.source.host.hostname": "hostname1",
"xdm.source.host.os_family": "Linux", "xdm.source.agent.identifier": "agent123", "xdm.source.agent.installation_id":
"installation123", "xdm.source.host.fqdn": "hostname1.domain.com", "xdm.source.process.executable.signature_status":
"Valid", "xdm.target.file.filename": "target_file.txt", "xdm.target.module.filename": "target_module.so",
"xdm.target.file.sha256": "d4bfc6fabe8d6d1b76e5b441dc8d01758276281f56c929b282ac5c3ee704c431",
"xdm.target.module.sha256": "7f4eafdad74bfedabf370a3725a5077c", "xdm.target.process.command_line":
"/usr/bin/target_process -option", "xdm.target.process.executable.sha256":
"7b21d50d6270f95b5a2cf582bf94b315cd75a034dd9478c0e5b4089bbd9b59ac", "xdm.target.process.executable.signature_status":
"Signed", "xdm.target.process.executable.signer": ["string"], "xdm.target.process.executable.path": ["string"],
"xdm.target.ipv4": ["string"], "xdm.target.host.ipv4_addresses": ["10.0.0.2", "10.0.0.3"],
"xdm.target.host.ipv6_addresses": ["string"], "xdm.target.ipv6": ["10.0.0.2", "10.0.0.3"], "xdm.target.port": 8080,
"xdm.target.location.country": "US", "xdm.target.host.hostname": "hostname", "xdm.target.identity.username": "user1",
"xdm.target.url": "https://example.com", "xdm.target.process.executable.filename": "target_process",
"xdm.target.process.name": "target_process", "xdm.target.agent.identifier": "target_agent",
"xdm.target.registry.value": "registry_value", "xdm.target.registry.data": "registry_data",
"xdm.target.registry.key": "registry_key", "xdm.email.attachment.sha256": "a1b2c3d4e5f6789abcde1234567890f2",
"xdm.email.attachment.filename": "attachment.pdf", "xdm.email.sender": "sender@example.com", "xdm.event.type":
"Intrusion", "xdm.cloud.provider": "AWS", "xdm.cloud.project": "CloudProject1", "xdm.cloud.project_id":
"cloud_project_id_123", "xdm.cloud.region": "us-east-1", "xdm.cloud.function.id": "cloud_func_123",
"xdm.cloud.function.name": "cloud_function", "xdm.cloud.function.version": "v1.0.0", "xdm.cloud.function.request_id":
```

```
"req_123", "xdm.cloud.function.runtime": "nodejs", "xdm.observer.unique_identifier": "observer123",
"xdm.observer.type": "Server", "xdm.observer.sub_type": "Linux", "xdm.observer.name": "Observer 1",
"xdm.vulnerability.cve_id": "CVE-2021-12345", "xdm.vulnerability.severity": "HIGH", "xdm.vulnerability.fix_versions":
["1.0.1", "1.0.2"], "xdm.vulnerability.cve_risk_factors": ["Exploitability", "Impact"],
"xdm.vulnerability.cvss_score": 7.8, "xdm.vulnerability.cvss_vector": "AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H",
"xdm.software_package.version": "1.0.0", "xdm.software_package.purl": "pkg:maven/com.example/software@1.0.0",
"xdm.software_package.layer_id": "layer123", "xdm.software_package.type": "Library",
"xdm.software_package.installation_type": "Automatic", "xdm.software_package.package_manager": "npm",
"xdm.software_package.dependency_type": "Direct", "xdm.software_package.language": "JavaScript",
"xdm.malware.verdict": "Malicious", "xdm.malware.virus_total_link": "https://www.virustotal.com/gui/file/abcd1234",
"xdm.malware.layer_id": "malware_layer123", "xdm.secret.secret_type": "API Key", "xdm.secret.unique_identifier":
"secret_id_123", "xdm.secret.snippet": "API Key: 12345", "xdm.secret.layer_id": "secret_layer123",
"xdm.file.filename": "file.txt", "xdm.file.path": "/path/to/file.txt", "xdm.file.sha256":
"abc1234567890def0987654321", "xdm.file.size": 1024, "xdm.file.last_modified": 1615465123,
"xdm.file.metadata_change_time": 1615465000, "xdm.file.owner_id": "user1", "xdm.file.owner_name": "fileowner",
"xdm.file.group_id": "group1", "xdm.file.group_name": "groupname", "xdm.file.permissions.owner": ["read", "write"],
"xdm.file.permissions.group": ["read"], "xdm.file.permissions.others": ["read"], "xdm.file.position.start.line": 1,
"xdm.file.position.start.character": 0, "xdm.file.position.end.line": 100, "xdm.file.position.end.character": 80,
"xdm.url": "https://example.com", "xdm.domain": "example.com", "xdm.application_protocol": "HTTPS" ],
"custom_fields": [] ]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options:
[]) let request = NSMutableURLRequest(url: NSURL(string: "https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue")! as
URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST"
request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"issue\":{\"owner\":\"CWP\",\"name\":\"Unauthorized Access
Detected\",\"description\":\"An unauthorized login attempt was detected from an unknown IP
address.\",\"observation_time\":1700000000000,\"domain\":\"SECURITY\",\"category\":\"CONFIGURATION\",\"asset_ids\":
[\"asset-456\"],\"mitre_tactics\":[\"TA0001\",\"TA0002\"],\"mitre_techniques\":
[\"T1003\",\"T1059\"],\"type\":\"Threat Intelligence\",\"extended_description\":\"This alert was triggered due to
multiple failed login attempts within a short time frame.\",\"impact\":\"Potential unauthorized system
access\",\"tags\":
[\"critical\",\"network\"],\"is_excluded\":false,\"is_starred\":true,\"assigned_to\":\"security_team_lead\",\"assigned_to_pret
Smith\",\"severity\":\"HIGH\",\"normalized_fields\":
{\"xdm.source.location.country\":\"US\",\"xdm.source.ipv4\":\"192.168.1.1\",\"xdm.source.host.ipv4_addresses\":
[\"192.168.1.2\",\"192.168.1.3\"],\"xdm.source.identity.username\":\"admin\",\"xdm.source.process.causality_id\":\"abc123\",\"x
-arg1 -
arg2\",\"xdm.source.process.executable.filename\":\"process_executable\",\"xdm.source.process.name\":\"process_name\",\"xdm.sou
-
option\",\"xdm.target.process.executable.sha256\":\"7b21d50d6270f95b5a2cf582bf94b315cd75a034dd9478c0e5b4089bbd9b59ac\",\"xdm.ta
[\"string\"],\"xdm.target.process.executable.path\":[\"string\"],\"xdm.target.ipv4\":
[\"string\"],\"xdm.target.host.ipv4_addresses\":[\"10.0.0.2\",\"10.0.0.3\"],\"xdm.target.host.ipv6_addresses\":
[\"string\"],\"xdm.target.ipv6\":
[\"10.0.0.2\",\"10.0.0.3\"],\"xdm.target.port\":8080,\"xdm.target.location.country\":\"US\",\"xdm.target.host.hostname\":\"hos
east-
1\",\"xdm.cloud.function.id\":\"cloud_func_123\",\"xdm.cloud.function.name\":\"cloud_function\",\"xdm.cloud.function.version\"
1\",\"xdm.vulnerability.cve_id\":\"CVE-2021-
12345\",\"xdm.vulnerability.severity\":\"HIGH\",\"xdm.vulnerability.fix_versions\":
[\"1.0.1\",\"1.0.2\"],\"xdm.vulnerability.cve_risk_factors\":
[\"Exploitability\",\"Impact\"],\"xdm.vulnerability.cvss_score\":7.8,\"xdm.vulnerability.cvss_vector\":\"AV:N/AC:L/PR:N/UI:N/S
Key\",\"xdm.secret.unique_identifier\":\"secret_id_123\",\"xdm.secret.snippet\":\"API Key:
12345\",\"xdm.secret.layer_id\":\"secret_layer123\",\"xdm.file.filename\":\"file.txt\",\"xdm.file.path\":\"/path/to/file.txt\"
[\"read\",\"write\"],\"xdm.file.permissions.group\":[\"read\"],\"xdm.file.permissions.others\":
[\"read\"],\"xdm.file.position.start.line\":1,\"xdm.file.position.start.character\":0,\"xdm.file.position.end.line\":100,\"xdm
{}}}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"issue\":
{\"owner\":\"CWP\",\"name\":\"Unauthorized Access Detected\",\"description\":\"An unauthorized login attempt was
detected from an unknown IP
```

```
address.\",\"observation_time\":1700000000000,\"domain\":\"SECURITY\",\"category\":\"CONFIGURATION\",\"asset_ids\":
[\"asset-456\"],\"mitre_tactics\":[\"TA0001\",\"TA0002\"],\"mitre_techniques\":
[\"T1003\",\"T1059\"],\"type\":\"Threat Intelligence\",\"extended_description\":\"This alert was triggered due to
multiple failed login attempts within a short time frame.\",\"impact\":\"Potential unauthorized system
access\",\"tags\":
[\"critical\",\"network\"],\"is_excluded\":false,\"is_starred\":true,\"assigned_to\":\"security_team_lead\",\"assigned_to_pret
Smith\",\"severity\":\"HIGH\",\"normalized_fields\":
{\"xdm.source.location.country\":\"US\",\"xdm.source.ipv4\":\"192.168.1.1\",\"xdm.source.host.ipv4_addresses\":
[\"192.168.1.2\",\"192.168.1.3\"],\"xdm.source.identity.username\":\"admin\",\"xdm.source.process.causality_id\":\"abc123\",\"x
-arg1 -
arg2\",\"xdm.source.process.executable.filename\":\"process_executable\",\"xdm.source.process.name\":\"process_name\",\"xdm.so
-
option\",\"xdm.target.process.executable.sha256\":\"7b21d50d6270f95b5a2cf582bf94b315cd75a034dd9478c0e5b4089bbd9b59ac\",\"xdm.ta
[\"string\"],\"xdm.target.process.executable.path\":[\"string\"],\"xdm.target.ipv4\":
[\"string\"],\"xdm.target.host.ipv4_addresses\":[\"10.0.0.2\",\"10.0.0.3\"],\"xdm.target.host.ipv6_addresses\":
[\"string\"],\"xdm.target.ipv6\":
[\"10.0.0.2\",\"10.0.0.3\"],\"xdm.target.port\":8080,\"xdm.target.location.country\":\"US\",\"xdm.target.host.hostname\":\"hos
east-
1\",\"xdm.cloud.function.id\":\"cloud_func_123\",\"xdm.cloud.function.name\":\"cloud_function\",\"xdm.cloud.function.version\"
1\",\"xdm.vulnerability.cve_id\":\"CVE-2021-
12345\",\"xdm.vulnerability.severity\":\"HIGH\",\"xdm.vulnerability.fix_versions\":
[\"1.0.1\",\"1.0.2\"],\"xdm.vulnerability.cve_risk_factors\":
[\"Exploitability\",\"Impact\"],\"xdm.vulnerability.cvss_score\":7.8,\"xdm.vulnerability.cvss_vector\":\"AV:N/AC:L/PR:N/UI:N/S
Key\",\"xdm.secret.unique_identifier\":\"secret_id_123\",\"xdm.secret.snippet\":\"API Key:
12345\",\"xdm.secret.layer_id\":\"secret_layer123\",\"xdm.file.filename\":\"file.txt\",\"xdm.file.path\":\"/path/to/file.txt\"
[\"read\",\"write\"],\"xdm.file.permissions.group\":[\"read\"],\"xdm.file.permissions.others\":
[\"read\"],\"xdm.file.position.start.line\":1,\"xdm.file.position.start.character\":0,\"xdm.file.position.end.line\":100,\"xdm
{}}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"issue\":{\"owner\":\"CWP\",\"name\":\"Unauthorized Access Detected\",\"description\":\"An
unauthorized login attempt was detected from an unknown IP
address.\",\"observation_time\":1700000000000,\"domain\":\"SECURITY\",\"category\":\"CONFIGURATION\",\"asset_ids\":
[\"asset-456\"],\"mitre_tactics\":[\"TA0001\",\"TA0002\"],\"mitre_techniques\":
[\"T1003\",\"T1059\"],\"type\":\"Threat Intelligence\",\"extended_description\":\"This alert was triggered due to
multiple failed login attempts within a short time frame.\",\"impact\":\"Potential unauthorized system
access\",\"tags\":
[\"critical\",\"network\"],\"is_excluded\":false,\"is_starred\":true,\"assigned_to\":\"security_team_lead\",\"assigned_to_pret
Smith\",\"severity\":\"HIGH\",\"normalized_fields\":
{\"xdm.source.location.country\":\"US\",\"xdm.source.ipv4\":\"192.168.1.1\",\"xdm.source.host.ipv4_addresses\":
[\"192.168.1.2\",\"192.168.1.3\"],\"xdm.source.identity.username\":\"admin\",\"xdm.source.process.causality_id\":\"abc123\",\"x
-arg1 -
arg2\",\"xdm.source.process.executable.filename\":\"process_executable\",\"xdm.source.process.name\":\"process_name\",\"xdm.so
-
option\",\"xdm.target.process.executable.sha256\":\"7b21d50d6270f95b5a2cf582bf94b315cd75a034dd9478c0e5b4089bbd9b59ac\",\"xdm.ta
[\"string\"],\"xdm.target.process.executable.path\":[\"string\"],\"xdm.target.ipv4\":
[\"string\"],\"xdm.target.host.ipv4_addresses\":[\"10.0.0.2\",\"10.0.0.3\"],\"xdm.target.host.ipv6_addresses\":
[\"string\"],\"xdm.target.ipv6\":
[\"10.0.0.2\",\"10.0.0.3\"],\"xdm.target.port\":8080,\"xdm.target.location.country\":\"US\",\"xdm.target.host.hostname\":\"hos
east-
1\",\"xdm.cloud.function.id\":\"cloud_func_123\",\"xdm.cloud.function.name\":\"cloud_function\",\"xdm.cloud.function.version\"
1\",\"xdm.vulnerability.cve_id\":\"CVE-2021-
12345\",\"xdm.vulnerability.severity\":\"HIGH\",\"xdm.vulnerability.fix_versions\":
[\"1.0.1\",\"1.0.2\"],\"xdm.vulnerability.cve_risk_factors\":
[\"Exploitability\",\"Impact\"],\"xdm.vulnerability.cvss_score\":7.8,\"xdm.vulnerability.cvss_vector\":\"AV:N/AC:L/PR:N/UI:N/S
Key\",\"xdm.secret.unique_identifier\":\"secret_id_123\",\"xdm.secret.snippet\":\"API Key:
12345\",\"xdm.secret.layer_id\":\"secret_layer123\",\"xdm.file.filename\":\"file.txt\",\"xdm.file.path\":\"/path/to/file.txt\"
[\"read\",\"write\"],\"xdm.file.permissions.group\":[\"read\"],\"xdm.file.permissions.others\":
[\"read\"],\"xdm.file.position.start.line\":1,\"xdm.file.position.start.character\":0,\"xdm.file.position.end.line\":100,\"xdm
{}}}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```


Body parameters

required

application/json

▼ request_dataobject

▶ issueobjectrequired

REQUEST

{ "request_data": { "issue": { "owner": "CWP", "name": "Unauthorized Access Detected", "description": "An unauthorized login attempt was detected from an unknown IP address.", "observation_time": 1700000000000, "domain": "SECURITY", "category": "CONFIGURATION", "asset_ids": ["asset-456"], "mitre_tactics": ["TA0001", "TA0002"], "mitre_techniques": ["T1003", "T1059"], "type": "Threat Intelligence", "extended_description": "This alert was triggered due to multiple failed login attempts within a short time frame.", "impact": "Potential unauthorized system access", "tags": ["critical", "network"], "is_excluded": false, "is_starred": true, "assigned_to": "security_team_lead", "assigned_to_pretty": "Alice Smith", "severity": "HIGH", "normalized_fields": { "xdm.source.location.country": "US", "xdm.source.ipv4": "192.168.1.1", "xdm.source.host.ipv4_addresses": ["192.168.1.2", "192.168.1.3"], "xdm.source.identity.username": "admin", "xdm.source.process.causality_id": "abc123", "xdm.source.process.command_line": "/usr/bin/process -arg1 -arg2", "xdm.source.process.executable.filename": "process_executable", "xdm.source.process.name": "process_name", "xdm.source.process.executable.path": "/usr/bin/process_executable", "xdm.source.process.executable.sha256": "f9c7b6e24f7e93d8d3e5c76f8b1b88cd8f17b34a7a4a2e3d5b2dbf09f5b8fdc2", "xdm.source.host.hostname": "hostname1", "xdm.source.host.os_family": "Linux", "xdm.source.agent.identifier": "agent123", "xdm.source.agent.installation_id": "installation123", "xdm.source.host.fqdn": "hostname1.domain.com", "xdm.source.process.executable.signature_status": "Valid", "xdm.target.file.filename": "target_file.txt", "xdm.target.module.filename": "target_module.so", "xdm.target.file.sha256": "d4bfc6fabe8d6d1b76e5b441dc8d01758276281f56c929b282ac5c3ee704c431", "xdm.target.module.sha256": "7f4eafdad74bfedabf370a3725a5077c", "xdm.target.process.command_line": "/usr/bin/target_process -option", "xdm.target.process.executable.sha256": "7b21d50d6270f95b5a2cf582bf94b315cd75a034dd9478c0e5b4089bbd9b59ac", "xdm.target.process.executable.signature_status": "Signed", "xdm.target.process.executable.signer": ["example"], "xdm.target.process.executable.path": ["example"], "xdm.target.ipv4": ["example"], "xdm.target.host.ipv4_addresses": ["10.0.0.2", "10.0.0.3"], "xdm.target.host.ipv6_addresses": ["example"], "xdm.target.ipv6": ["10.0.0.2", "10.0.0.3"], "xdm.target.port": 8080, "xdm.target.location.country": "US", "xdm.target.host.hostname": "hostname", "xdm.target.identity.username": "user1", "xdm.target.url": "https://example.com", "xdm.target.process.executable.filename": "target_process", "xdm.target.process.name": "target_process", "xdm.target.agent.identifier": "target_agent", "xdm.target.registry.value": "registry_value", "xdm.target.registry.data": "registry_data", "xdm.target.registry.key": "registry_key", "xdm.email.attachment.sha256": "a1b2c3d4e5f6789abcde1234567890f2", "xdm.email.attachment.filename": "attachment.pdf", "xdm.email.sender": "sender@example.com", "xdm.event.type": "Intrusion", "xdm.cloud.provider": "AWS", "xdm.cloud.project": "CloudProject1", "xdm.cloud.project_id": "cloud_project_id_123", "xdm.cloud.region": "us-east-1", "xdm.cloud.function.id": "cloud_func_123", "xdm.cloud.function.name": "cloud_function", "xdm.cloud.function.version": "v1.0.0", "xdm.cloud.function.request_id": "req_123", "xdm.cloud.function.runtime": "nodejs", "xdm.observer.unique_identifier": "observer123", "xdm.observer.type": "Server", "xdm.observer.sub_type": "Linux", "xdm.observer.name": "Observer 1", "xdm.vulnerability.cve_id": "CVE-2021-12345", "xdm.vulnerability.severity": "HIGH", "xdm.vulnerability.fix_versions": ["1.0.1", "1.0.2"], "xdm.vulnerability.cve_risk_factors": ["Exploitability", "Impact"], "xdm.vulnerability.cvss_score": 7.8, "xdm.vulnerability.cvss_vector": "AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H", "xdm.software_package.version": "1.0.0", "xdm.software_package.purl": "pkg:maven/com.example/software@1.0.0", "xdm.software_package.layer_id": "layer123", "xdm.software_package.type": "Library", "xdm.software_package.installation_type": "Automatic", "xdm.software_package.package_manager": "npm", "xdm.software_package.dependency_type": "Direct", "xdm.software_package.language": "JavaScript", "xdm.malware.verdict": "Malicious", "xdm.malware.virus_total_link": "https://www.virustotal.com/gui/file/abcd1234", "xdm.malware.layer_id": "malware_layer123", "xdm.secret.secret_type": "API Key", "xdm.secret.unique_identifier": "secret_id_123", "xdm.secret.snippet": "API Key: 12345", "xdm.secret.layer_id": "secret_layer123", "xdm.file.filename": "file.txt", "xdm.file.path": "/path/to/file.txt", "xdm.file.sha256": "abc1234567890def0987654321", "xdm.file.size": 1024, "xdm.file.last_modified": 1615465123, "xdm.file.metadata_change_time": 1615465000, "xdm.file.owner_id": "user1", "xdm.file.owner_name": "fileowner", "xdm.file.group_id": "group1", "xdm.file.group_name": "groupname", "xdm.file.permissions.owner": ["read", "write"], "xdm.file.permissions.group": ["read"], "xdm.file.permissions.others": ["read"], "xdm.file.position.start.line": 1, "xdm.file.position.start.character": 0, "xdm.file.position.end.line": 100, "xdm.file.position.end.character": 80, "xdm.url": "https://example.com", "xdm.domain": "example.com", "xdm.application_protocol": "HTTPS" }, "custom_fields": {} } } }

Responses

Issue created successfully

Body
application/json
▼ external_idstring
▼ detection_methodstring
RESPONSE default

{ "external_id": "7c96737d50f74c7b9487450426e9eafb", "detection_method": "CREATE_ALERT_PUBLIC_API" }

Bad request

Body
application/json
▼ errorstring
Example: "Invalid request data"
RESPONSE

```
{ "error": "Invalid request data" }
```

Unauthorized access

Body
application/json
▼ errorstring
Example:`"Unauthorized request"`
RESPONSE
```
{ "error": "Unauthorized request" }
```

Internal server error

Body
application/json
▼ errorstring
Example:`"Internal server error"`
RESPONSE
```
{ "error": "Internal server error" }
```

2.1.25.2 | Retrieve issues based on filters

post /public_api/v1/issue/search

This endpoint retrieves a list of issues that match the specified filter criteria. It supports filtering by `issue_id`, `external_id`, `detection_method`, `domain`, `source`, `severity`, and `_insert_time`, along with sorting and pagination.

Request Body:

- `request_data`: Object containing filter criteria
  - `filters`: Array of filter objects
    - `field`: String (enum: 'issue_id', 'external_id', 'detection_method', 'domain', 'severity', '_insert_time', 'status')
    - `operator`: String (enum: 'in', 'gte', 'lte')
    - `value`: Array of integers/strings or single integer
- `search_from`: Integer (default: 0) - Starting index for pagination
- `search_to`: Integer (default: 100) - Ending index for pagination
- `sort`: Object for sorting results
  - `field`: String (enum: '_insert_time', 'severity', 'issue_id')
  - `keyword`: String (enum: 'asc', 'desc') - Sort order
- `include_fields`: Array of strings (enum: 'normalized_fields', 'custom_fields', default: []) - Fields to include in response

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/issue/search'
-d ''
import http.client conn = http.client.HTTPSConnection("api-") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"issue_id\",\"operator\":\"in\",\"value\":[0]}],\"search_from\":0,\"search_to\":0,\"sort\":
{\"field\":\"id\",\"keyword\":\"asc\"},\"include_fields\":[\"normalized_fields\"]}}" headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"%7B%7Bfqdn%7D%7D/public_api/v1/issue/search", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url =
URI("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue/search") http = Net::HTTP.new(url.host, url.port) http.use_ssl
= true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
```

```
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"issue_id\",\"operator\":\"in\",\"value\":
[0]}],\"search_from\":0,\"search_to\":0,\"sort\":{\"field\":\"id\",\"keyword\":\"asc\"},\"include_fields\":
[\"normalized_fields\"]}}" response = http.request(request) puts response.read_body
```

```
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "issue_id", "operator": "in", "value": [ 0 ]
} ], "search_from": 0, "search_to": 0, "sort": { "field": "id", "keyword": "asc" }, "include_fields": [
"normalized_fields" ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue/search");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.post("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue/search")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":[{\"field\":\"issue_id\",\"operator\":\"in\",\"value\":
[0]}],\"search_from\":0,\"search_to\":0,\"sort\":{\"field\":\"id\",\"keyword\":\"asc\"},\"include_fields\":
[\"normalized_fields\"]}}") .asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "issue_id",
"operator": "in", "value": [0] ] ], "search_from": 0, "search_to": 0, "sort": [ "field": "id", "keyword": "asc" ],
"include_fields": ["normalized_fields"] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject:
parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue/search")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue/search", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"issue_id\",\"operator\":\"in\",\"value\":[0]}],\"search_from\":0,\"search_to\":0,\"sort\":
{\"field\":\"id\",\"keyword\":\"asc\"},\"include_fields\":[\"normalized_fields\"]}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue/search"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-
id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"filters\":[{\"field\":\"issue_id\",\"operator\":\"in\",\"value\":[0]}],\"search_from\":0,\"search_to\":0,\"sort\":
{\"field\":\"id\",\"keyword\":\"asc\"},\"include_fields\":[\"normalized_fields\"]}}"); CURLcode ret =
curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue/search"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"issue_id\",\"operator\":\"in\",\"value\":
[0]}],\"search_from\":0,\"search_to\":0,\"sort\":{\"field\":\"id\",\"keyword\":\"asc\"},\"include_fields\":
[\"normalized_fields\"]}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

▶ filtersarray

▶ search_frominteger

Starting index for pagination.

▶ search_tointeger

Ending index for pagination.

▶ sortobject

▶ include_fieldsarray

A list of fields to include in the response.

- `normalized_fields`: Includes normalized fields in the response.
- `custom_fields`: Includes custom user-defined fields in the response.
- By default these fields will not be part of response payload.

Responses

Successful response with issues

Body
application/json
▼ replyobject
▶ total_countinteger
▶ result_countinteger
▶ issuesarray
RESPONSE
{ "reply": { "total_count": 0, "result_count": 0, "issues": [{ "_insert_time": "2023-05-15T10:30:00Z", "owner": "CWP", "external_id": "EXT-12345", "name": "Suspicious Network Activity", "description": "Unusual outbound traffic detected from internal server", "observation_time": 1621234567890, "domain": "SECURITY", "detection_method": "CSPM_SCANNER", "detection_rule_id": "RULE-9876", "category": "CONFIGURATION", "finding_ids": ["013543ea00ea9893bcde59cbfdc5992f"], "asset_ids": ["123"], "mitre_tactics": ["COLLECTION"], "mitre_techniques": ["ABUSE_ELEVATION_CONTROL_MECHANISM"], "type": "Identity Security", "remediation": "Isolate affected system and investigate traffic patterns", "extended_description": "Detailed analysis of the network traffic patterns and potential impact", "impact": "Potential data exfiltration or command and control activity", "issue_id": 123, "last_modified": 1621235678901, "tags": ["critical", "investigation_required"], "is_excluded": false, "is_starred": true, "assigned_to": "alice.smith@example.com", "assigned_to_pretty": "Alice Smith", "status": "Resolved", "status_resolution_reason": "RESOLVED_OTHER", "status_resolution_comment": "Investigating the source of suspicious traffic", "severity": "HIGH", "resolution_time": 1621240000000, "normalized_fields": { "xdm.source.location.country": "US", "xdm.source.ipv4": "192.168.1.1", "xdm.source.host.ipv4_addresses": ["192.168.1.2", "192.168.1.3"], "xdm.source.identity.username": "admin", "xdm.source.process.causality_id": "abc123", "xdm.source.process.command_line": "/usr/bin/process -arg1 -arg2", "xdm.source.process.executable.filename": "process_executable", "xdm.source.process.name": "process_name", "xdm.source.process.executable.path": "/usr/bin/process_executable", "xdm.source.process.executable.sha256": "f9c7b6e24f7e93d8d3e5c76f8b1b88cd8f17b34a7a4a2e3d5b2dbf09f5b8fdc2", "xdm.source.host.hostname": "hostname1", "xdm.source.host.os_family": "Linux", "xdm.source.agent.identifier": "agent123", "xdm.source.agent.installation_id": "installation123", "xdm.source.host.fqdn": "hostname1.domain.com", "xdm.source.process.executable.signature_status": "Valid", "xdm.target.file.filename": "target_file.txt", "xdm.target.module.filename": "target_module.so", "xdm.target.file.sha256": "d4bfc6fabe8d6d1b76e5b441dc8d01758276281f56c929b282ac5c3ee704c431", "xdm.target.module.sha256": "7f4eafdad74bfedabf370a3725a5077c", "xdm.target.process.command_line": "/usr/bin/target_process -option", "xdm.target.process.executable.sha256": "7b21d50d6270f95b5a2cf582bf94b315cd75a034dd9478c0e5b4089bbd9b59ac", "xdm.target.process.executable.signature_status": "Signed", "xdm.target.process.executable.signer": ["example"], "xdm.target.process.executable.path": ["example"], "xdm.target.ipv4": ["example"], "xdm.target.host.ipv4_addresses": ["10.0.0.2", "10.0.0.3"], "xdm.target.host.ipv6_addresses": ["example"], "xdm.target.ipv6": ["10.0.0.2", "10.0.0.3"], "xdm.target.port": 8080, "xdm.target.location.country": "US", "xdm.target.host.hostname": "hostname", "xdm.target.identity.username": "user1", "xdm.target.url": "https://example.com", "xdm.target.process.executable.filename": "target_process", "xdm.target.process.name": "target_process", "xdm.target.agent.identifier": "target_agent", "xdm.target.registry.value": "registry_value", "xdm.target.registry.data": "registry_data", "xdm.target.registry.key": "registry_key", "xdm.email.attachment.sha256": "a1b2c3d4e5f6789abcde1234567890f2", "xdm.email.attachment.filename": "attachment.pdf", "xdm.email.sender": "sender@example.com", "xdm.event.type": "Intrusion", "xdm.cloud.provider": "AWS", "xdm.cloud.project": "CloudProject1", "xdm.cloud.project_id": "cloud_project_id_123", "xdm.cloud.region": "us-east-1", "xdm.cloud.function.id": "cloud_func_123", "xdm.cloud.function.name": "cloud_function", "xdm.cloud.function.version": "v1.0.0", "xdm.cloud.function.request_id": "req_123", "xdm.cloud.function.runtime": "nodejs", "xdm.observer.unique_identifier": "observer123", "xdm.observer.type": "Server", "xdm.observer.sub_type": "Linux", "xdm.observer.name": "Observer 1", "xdm.vulnerability.cve_id": "CVE-2021-12345", "xdm.vulnerability.severity": "HIGH", "xdm.vulnerability.fix_versions": ["1.0.1", "1.0.2"], "xdm.vulnerability.cve_risk_factors": ["Exploitability", "Impact"], "xdm.vulnerability.cvss_score": 7.8, "xdm.vulnerability.cvss_vector": "AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H", "xdm.software_package.version": "1.0.0", "xdm.software_package.purl": "pkg:maven/com.example/software@1.0.0", "xdm.software_package.layer_id": "layer123", "xdm.software_package.type": "Library", "xdm.software_package.installation_type": "Automatic", "xdm.software_package.package_manager": "npm", "xdm.software_package.dependency_type": "Direct", "xdm.software_package.language": "JavaScript", "xdm.malware.verdict": "Malicious", "xdm.malware.virus_total_link": "https://www.virustotal.com/gui/file/abcd1234", "xdm.malware.layer_id": "malware_layer123", "xdm.secret.secret_type": "API Key", "xdm.secret.unique_identifier": "secret_id_123", "xdm.secret.snippet": "API Key: 12345", "xdm.secret.layer_id": "secret_layer123", "xdm.file.filename": "file.txt", "xdm.file.path": "/path/to/file.txt", "xdm.file.sha256": "abc1234567890def0987654321", "xdm.file.size": 1024, "xdm.file.last_modified": 1615465123, "xdm.file.metadata_change_time": 1615465000, "xdm.file.owner_id": "user1", "xdm.file.owner_name": "fileowner", "xdm.file.group_id": "group1", "xdm.file.group_name": "groupname", "xdm.file.permissions.owner": ["read", "write"], "xdm.file.permissions.group": ["read"], "xdm.file.permissions.others": ["read"], "xdm.file.position.start.line": 1, "xdm.file.position.start.character": 0, "xdm.file.position.end.line": 100, "xdm.file.position.end.character": 80, "xdm.url": "https://example.com", "xdm.domain": "example.com", "xdm.application_protocol": "HTTPS" }, "custom_fields": {} }] } }

Bad request

Body
application/json
▼ errorstring
Example:"Invalid request data"
RESPONSE
```
{ "error": "Invalid request data" }
```

Unauthorized access

Body
application/json
▼ errorstring
Example:"Unauthorized request"
RESPONSE
```
{ "error": "Unauthorized request" }
```

Internal server error

Body
application/json
▼ errorstring
Example:"Internal server error"
RESPONSE
```
{ "error": "Internal server error" }
```

2.1.25.3 | Update existing issue

post /public_api/v1/issue/{issue-id}

Update an existing issue in the system. Users can only update one issue at a time.

- At least one of the following fields is mandatory:
  - severity
  - status

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ issue-id Integer required

Numeric ID of the user to get

Example: 56
Request headers
▼ Authorization String required

{api_key}

Example: authorization_example
▼ x-xdr-auth-id String required

{api_key_id}

Example: xXdrAuthId_example
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/issue/{issue-id}'
-d ''
import http.client conn = http.client.HTTPSConnection("api-") payload = "{\"request_data\":{\"update_data\":
{\"severity\":\"HIGH\",\"status\":\"Resolved\",\"status_resolution_reason\":\"RESOLVED_OTHER\",\"status_resolution_comment\":\"
has been marked as a false positive.\"}}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"%7B%7Bfqdn%7D%7D/public_api/v1/issue/%7Bissue-id%7D", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url =
URI("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue/%7Bissue-id%7D") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url)
request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"]
```

```
= 'application/json' request.body = "{\"request_data\":{\"update_data\":
{\"severity\":\"HIGH\",\"status\":\"Resolved\",\"status_resolution_reason\":\"RESOLVED_OTHER\",\"status_resolution_comment\":\"
has been marked as a false positive.\"}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "update_data": { "severity": "HIGH", "status": "Resolved",
"status_resolution_reason": "RESOLVED_OTHER", "status_resolution_comment": "Issue has been marked as a false
positive." } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue/%7Bissue-
id%7D"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue/%7Bissue-id%7D")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"update_data\":
{\"severity\":\"HIGH\",\"status\":\"Resolved\",\"status_resolution_reason\":\"RESOLVED_OTHER\",\"status_resolution_comment\":\"
has been marked as a false positive.\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["update_data": [ "severity": "HIGH",
"status": "Resolved", "status_resolution_reason": "RESOLVED_OTHER", "status_resolution_comment": "Issue has been
marked as a false positive." ]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue/%7Bissue-id%7D")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue/%7Bissue-id%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING
=> "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"update_data\":
{\"severity\":\"HIGH\",\"status\":\"Resolved\",\"status_resolution_reason\":\"RESOLVED_OTHER\",\"status_resolution_comment\":\"
has been marked as a false positive.\"}}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-
type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue/%7Bissue-id%7D"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"update_data\":
{\"severity\":\"HIGH\",\"status\":\"Resolved\",\"status_resolution_reason\":\"RESOLVED_OTHER\",\"status_resolution_comment\":\"
has been marked as a false positive.\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/issue/%7Bissue-id%7D"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"update_data\":
{\"severity\":\"HIGH\",\"status\":\"Resolved\",\"status_resolution_reason\":\"RESOLVED_OTHER\",\"status_resolution_comment\":\"
has been marked as a false positive.\"}}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

▶ update_dataobjectrequired

REQUEST

```
{ "request_data": { "update_data": { "severity": "HIGH", "status": "Resolved", "status_resolution_reason":
"RESOLVED_OTHER", "status_resolution_comment": "Issue has been marked as a false positive." } } }
```

Responses

Issues updated successfully

Bad request

Body

application/json

▼ errorstring

Example:`"Invalid request data"`

RESPONSE

```
{ "error": "Invalid request data" }
```

Unauthorized access

Body
application/json
▼ errorstring
Example:`"Unauthorized request"`
RESPONSE
```
{ "error": "Unauthorized request" }
```

Internal server error

Body
application/json
▼ errorstring
Example:`"Internal server error"`
RESPONSE
```
{ "error": "Internal server error" }
```

2.1.26 | Cases

APIs for managing cases

2.1.26.1 | Retrieve cases based on filters

post /public_api/v1/case/search

This endpoint retrieves a list of cases that match the specified filter criteria. It supports filtering by `case_id`, `case_domain`, `status_progress`, `severity`, and `creation_time`, along with sorting and pagination.

Request Body:

- `request_data`: Object containing filter criteria
  - `filters`: Array of filter objects
    - `field`: String (enum: 'case_id', 'case_domain', 'severity', 'creation_time', 'status_progress')
    - `operator`: String (enum: 'in', 'gte', 'lte')
    - `value`: Array of integers/strings or single integer
- `search_from`: Integer (default: 0) - Starting index for pagination
- `search_to`: Integer (default: 100) - Ending index for pagination
- `sort`: Object for sorting results
  - `field`: String (enum: 'creation_time', 'severity', 'case_id')
  - `keyword`: String (enum: 'asc', 'desc') - Sort order

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/case/search'
-d ''
import http.client conn = http.client.HTTPSConnection("api-") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"case_id\",\"operator\":\"in\",\"value\":[0]}],\"search_from\":0,\"search_to\":100,\"sort\":
{\"field\":\"case_id\",\"keyword\":\"asc\"}}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"%7B%7Bfqdn%7D%7D/public_api/v1/case/search", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url =
URI("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/search") http = Net::HTTP.new(url.host, url.port) http.use_ssl
= true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":[{\"field\":\"case_id\",\"operator\":\"in\",\"value\":
```

```
[0]}],\"search_from\":0,\"search_to\":100,\"sort\":{\"field\":\"case_id\",\"keyword\":\"asc\"}}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "case_id", "operator": "in", "value": [ 0 ] }
], "search_from": 0, "search_to": 100, "sort": { "field": "case_id", "keyword": "asc" } } }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST",
"https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/search"); xhr.setRequestHeader("Authorization",
"SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-
type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/search")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":[{\"field\":\"case_id\",\"operator\":\"in\",\"value\":
[0]}],\"search_from\":0,\"search_to\":100,\"sort\":{\"field\":\"case_id\",\"keyword\":\"asc\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": [ "filters": [ [ "field": "case_id",
"operator": "in", "value": [0] ] ], "search_from": 0, "search_to": 100, "sort": [ "field": "case_id", "keyword":
"asc" ] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let
request = NSMutableURLRequest(url: NSURL(string: "https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/search")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/search", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"case_id\",\"operator\":\"in\",\"value\":[0]}],\"search_from\":0,\"search_to\":100,\"sort\":
{\"field\":\"case_id\",\"keyword\":\"asc\"}}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-
type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/search"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"case_id\",\"operator\":\"in\",\"value\":[0]}],\"search_from\":0,\"search_to\":100,\"sort\":
{\"field\":\"case_id\",\"keyword\":\"asc\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/search"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"case_id\",\"operator\":\"in\",\"value\":
[0]}],\"search_from\":0,\"search_to\":100,\"sort\":{\"field\":\"case_id\",\"keyword\":\"asc\"}}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

▶ filtersarray

▶ search_frominteger

Starting index for pagination.

▶ search_tointeger

Ending index for pagination.

▶ sortobject

REQUEST

```
{ "request_data": { "filters": [ { "field": "case_id", "operator": "in", "value": [ 0 ] } ], "search_from": 0,
"search_to": 0, "sort": { "field": "case_id", "keyword": "asc" } } }
```

Responses

Successful response with cases

Body

application/json

▼ replyobject

▶ TOTAL_COUNTinteger

▶ FILTER_COUNTinteger

▶ DATAarray

RESPONSE


```
{ "reply": { "TOTAL_COUNT": 0, "FILTER_COUNT": 0, "DATA": [ { "case_id": "123", "creation_time": "2024-02-
26T12:34:56Z", "owner": "CWP", "case_domain": "SECURITY", "auto_resolve_case": false, "custom_fields": { "priority":
"High", "incident_id": "INC98765" }, "status_progress": "UNDER_INVESTIGATION", "resolve_reason": "RESOLVED_OTHER",
"resolve_comment": "Case has been marked as a false positive.", "last_modified": 1708950896000, "score": 85,
"severity": "HIGH", "description": "Unauthorized access detected", "scoring_type": "Risk-Based", "assigned_to":
"U56789", "assigned_to_name": "JaneSmith", "is_cloud": true, "is_automated": false, "issues": [ "123" ], "assets": [
"6f1d9b9361f8a6964bf3e79e6667e5fe" ], "artifacts": [ "12345" ] } ] } }
```


Bad request

Body
application/json
▼ errorstring
Example:`"Invalid request data"`
RESPONSE


```
{ "error": "Invalid request data" }
```


Unauthorized access

Body
application/json
▼ errorstring
Example:`"Unauthorized request"`
RESPONSE


```
{ "error": "Unauthorized request" }
```


Internal server error

Body
application/json
▼ errorstring
Example:`"Internal server error"`
RESPONSE


```
{ "error": "Internal server error" }
```

2.1.26.2 | Update existing case

post /public_api/v1/case/update/{case-id}

Update an existing case in the system. Users can only update one case at a time.

- At least one of the following fields is mandatory:
  - case_id

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ case-id Integer required

Numeric ID of the case to update

Example: `56`
Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/case/update/{case-id}'
-d ''
```

```
import http.client conn = http.client.HTTPSConnection("api-") payload = "{\"request_data\":{\"update_data\":
{\"status_progress\":\"RESOLVED\",\"resolve_reason\":\"RESOLVED_OTHER\",\"resolve_comment\":\"Case has been marked as
a false positive.\"}}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE",
'content-type': "application/json" } conn.request("POST", "%7B%7Bfqdn%7D%7D/public_api/v1/case/update/%7Bcase-id%7D",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url =
URI("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/update/%7Bcase-id%7D") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url)
request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"]
= 'application/json' request.body = "{\"request_data\":{\"update_data\":
{\"status_progress\":\"RESOLVED\",\"resolve_reason\":\"RESOLVED_OTHER\",\"resolve_comment\":\"Case has been marked as
a false positive.\"}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "update_data": { "status_progress": "RESOLVED", "resolve_reason":
"RESOLVED_OTHER", "resolve_comment": "Case has been marked as a false positive." } } }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST",
"https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/update/%7Bcase-id%7D"); xhr.setRequestHeader("Authorization",
"SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-
type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/update/%7Bcase-id%7D")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"update_data\":
{\"status_progress\":\"RESOLVED\",\"resolve_reason\":\"RESOLVED_OTHER\",\"resolve_comment\":\"Case has been marked as
a false positive.\"}}}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["request_data": ["update_data": [ "status_progress":
"RESOLVED", "resolve_reason": "RESOLVED_OTHER", "resolve_comment": "Case has been marked as a false positive." ]]] as
[String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/update/%7Bcase-id%7D")! as
URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST"
request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/update/%7Bcase-id%7D", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"update_data\":
{\"status_progress\":\"RESOLVED\",\"resolve_reason\":\"RESOLVED_OTHER\",\"resolve_comment\":\"Case has been marked as
a false positive.\"}}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type:
application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/update/%7Bcase-id%7D"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers,
"x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"update_data\":{\"status_progress\":\"RESOLVED\",\"resolve_reason\":\"RESOLVED_OTHER\",\"resolve_comment\":\"Case
has been marked as a false positive.\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/update/%7Bcase-id%7D"); var request =
new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-
id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"update_data\":
{\"status_progress\":\"RESOLVED\",\"resolve_reason\":\"RESOLVED_OTHER\",\"resolve_comment\":\"Case has been marked as
a false positive.\"}}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

▶ update_dataobjectrequired

REQUEST

```
{ "request_data": { "update_data": { "status_progress": "RESOLVED", "resolve_reason": "RESOLVED_OTHER",
"resolve_comment": "Case has been marked as a false positive." } } }
```

Responses

Case updated successfully

Bad request

Body
application/json
▼ errorstring
Example:"Invalid request data"
RESPONSE
```
{ "error": "Invalid request data" }
```

Unauthorized access

Body
application/json
▼ errorstring
Example:"Unauthorized request"
RESPONSE
```
{ "error": "Unauthorized request" }
```

Internal server error

Body
application/json
▼ errorstring
Example:"Internal server error"
RESPONSE
```
{ "error": "Internal server error" }
```

2.1.26.3 | Retrieve Case Artifacts by Case Id

get /public_api/v1/case/artifacts/{case-id}

This endpoint retrieves the artifacts for a given case_id

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ case-id Integer required

Numeric ID of the case

Example: 56
Request headers
▼ Authorization String required

{api_key}

Example: authorization_example
▼ x-xdr-auth-id String required

{api_key_id}

Example: xXdrAuthId_example
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/case/artifacts/{case-id}'
import http.client conn = http.client.HTTPSConnection("api-") headers = { 'Authorization': "SOME_STRING_VALUE", 'x-
xdr-auth-id': "SOME_STRING_VALUE" } conn.request("GET", "%7B%7Bfqdn%7D%7D/public_api/v1/case/artifacts/%7Bcase-
id%7D", headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url =
URI("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/artifacts/%7Bcase-id%7D") http = Net::HTTP.new(url.host,
url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url)
request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' response =
http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET",
"https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/artifacts/%7Bcase-id%7D"); xhr.setRequestHeader("Authorization",
"SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/artifacts/%7Bcase-
id%7D") .header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/artifacts/%7Bcase-id%7D")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/artifacts/%7Bcase-id%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL, "https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/artifacts/%7Bcase-id%7D"); struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id: SOME_STRING_VALUE"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/public_api/v1/case/artifacts/%7Bcase-id%7D"); var request = new RestRequest(Method.GET); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); IRestResponse response = client.Execute(request);
```

Responses

Successful response with cases

Body
application/json
▼ replyobject
▶ TOTAL_COUNTinteger
▶ FILTER_COUNTinteger
▶ DATAarray
RESPONSE

```
{ "reply": { "TOTAL_COUNT": 0, "FILTER_COUNT": 0, "DATA": [ { "case_id": 1234, "network_artifacts": { "DATA": [ { "artifact_id": "A12345", "file_name": "test", "file_sha256": "1111", "file_signature_status": "SIGNATURE_UNAVAILABLE", "file_wildfire_verdict": "BENIGN", "is_malicious": true, "is_manual": false, "is_process": false, "low_confidence": false, "type": "HASH" } ], "TOTAL_COUNT": 10 }, "file_artifacts": { "DATA": [ { "artifact_id": "A12345", "file_name": "test", "file_sha256": "1111", "file_signature_status": "SIGNATURE_UNAVAILABLE", "file_wildfire_verdict": "BENIGN", "is_malicious": true, "is_manual": false, "is_process": false, "low_confidence": false, "type": "HASH" } ], "TOTAL_COUNT": 10 } } ] } }
```

Bad request

Body
application/json
▼ errorstring
Example:"Invalid request data"
RESPONSE

```
{ "error": "Invalid request data" }
```

Unauthorized access

Body
application/json
▼ errorstring
Example:"Unauthorized request"
RESPONSE

```
{ "error": "Unauthorized request" }
```

Internal server error

Body
application/json
▼ errorstring
Example:"Internal server error"
RESPONSE

```
{ "error": "Internal server error" }
```

2.1.26.4 | Get War Room entries

post /public_api/v1/entries/get

Get the War Room entries for a specific case or alert. You can filter by timestamp, ID, and tags. You can also choose which type of entries you want to retrieve (notes, chat, attachments...). The response depends on what type of entry you choose to retrieve.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management
Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/entries/get'
-d ''
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"id\":\"string\",\"filter\":
{\"firstID\":\"string\",\"lastID\":\"string\",\"pagesize\":0,\"fromTime\":\"string\",\"categories\":
[\"tags\"],\"tags\":[\"string\"]}}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/entries/get", payload,
headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
```

```
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/entries/get") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"id\":\"string\",\"filter\":
{\"firstID\":\"string\",\"lastID\":\"string\",\"pagesize\":0,\"fromTime\":\"string\",\"categories\":
[\"tags\"],\"tags\":[\"string\"]}}" response = http.request(request) puts response.read_body
```

```
const data = JSON.stringify({ "id": "string", "filter": { "firstID": "string", "lastID": "string", "pagesize": 0,
"fromTime": "string", "categories": [ "tags" ], "tags": [ "string" ] } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/entries/get");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/entries/get")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"id\":\"string\",\"filter\":
{\"firstID\":\"string\",\"lastID\":\"string\",\"pagesize\":0,\"fromTime\":\"string\",\"categories\":
[\"tags\"],\"tags\":[\"string\"]}}") .asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = [ "id": "string", "filter": [ "firstID": "string", "lastID":
"string", "pagesize": 0, "fromTime": "string", "categories": ["tags"], "tags": ["string"] ] ] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/entries/get")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/entries/get", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"id\":\"string\",\"filter\":
{\"firstID\":\"string\",\"lastID\":\"string\",\"pagesize\":0,\"fromTime\":\"string\",\"categories\":
[\"tags\"],\"tags\":[\"string\"]}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type:
application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/entries/get"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"id\":\"string\",\"filter\":
{\"firstID\":\"string\",\"lastID\":\"string\",\"pagesize\":0,\"fromTime\":\"string\",\"categories\":
[\"tags\"],\"tags\":[\"string\"]}}"); CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/entries/get"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
```

```
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"id\":\"string\",\"filter\":
{\"firstID\":\"string\",\"lastID\":\"string\",\"pagesize\":0,\"fromTime\":\"string\",\"categories\":
[\"tags\"],\"tags\":[\"string\"]}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ idstring

The ID of the case or alert you want to get the War Room entries of. For a case ID, prepend "CASE-" to the case ID. For example, if the case ID is 3, the value of `id` should be `CASE-3`. For alert IDs, just put the ID. For example, if the alert ID is 3, the value of `id` should be `3`.

▼ filterobject

▶ firstIDstring

Return results starting from the investigation ID in `firstID` until the last investigation ID.

▶ lastIDstring

Return results starting from the first investigation ID until the investigation ID in `lastID`.

▶ pagesizeinteger

Filter the results by the number of entries you want returned. `"pagesize": 0` returns all the results.

▶ fromTimestring

RFC3339 timestamp. Filter results from the time the entry is created until the latest entry.

▶ categoriesarray[string]

The categories you want to filter and the results you want to receive:

- `tags`: Tags added to the investigation.
- `chats`: Communication between team members in the form of chat messages.
- `notes`: Any entries marked as notes.
- `attachments`: Any files uploaded to the War Room in a playbook, script, or by the analyst.
- `incidentInfo`: The case history.
- `commandAndResults`: Command and return the result of the command.
- `playbookTaskResult`: Return the task result.
- `playbookTaskStartAndDone`: Task and return the task result.
- `playbookErrors`: When there are no playbook errors, the response returned is `0`.

▶ tagsarray[string]

If using the filter category of `tags`, include the tags you want to filter by.

REQUEST

```
{ "id": "example", "filter": { "firstID": "example", "lastID": "example", "pagesize": 0, "fromTime": "example",
"categories": [ "tags" ], "tags": [ "example" ] } }
```

Responses

OK

Body

application/json

▼ totalinteger

▼ dataarray

[

▶ idstring

▶ modifiedstring

▶ createdstring

▶ userstring

▶ parentContentstring

▶ contentsstring

▶ formatstring

▶ investigationIdstring

▶ categorystring

▶ isTodoboolean

▶ tagsarray[string]

]

RESPONSE Filter by entry tags

```
{ "total": 2, "data": [ { "id": "f1db36f0-d00f-4470-818f-c6911d392f76@INCIDENT-1", "modified": "2025-01-
02T10:17:57.738577545Z", "created": "2025-01-01T13:15:48.302901Z", "user": "", "parentContent": "!markAsNote
entryIDs=\"1\"", "contents": "done", "format": "text", "investigationId": "INCIDENT-1", "category": "artifact",
```

"isTodo": false, "tags": ["john100"] }, { "id": "0d6a01a4-58b5-4121-8db7-c371e38284db@INCIDENT-1", "modified": "2025-01-02T10:22:26.6396269Z", "created": "2025-01-02T09:11:51.747644Z", "user": "", "contents": "'aaaaaa'", "format": "text", "investigationId": "INCIDENT-1", "category": "artifact", "isTodo": false, "tags": ["john200"] }] }
{ "total": 2, "data": [{ "id": "39101423-126c-4438-80d0-6c19dcd20f2e@INCIDENT-1", "modified": "2024-12-31T10:08:54.656397851Z", "created": "2024-12-31T10:08:54.656257Z", "user": "", "contents": "testing", "format": "markdown", "investigationId": "INCIDENT-1", "category": "chat", "isTodo": false }, { "id": "dbef4f98-a115-46c1-8e7c-b8e17ff98683@INCIDENT-1", "modified": "2025-01-02T09:52:06.239744361Z", "created": "2025-01-02T09:52:06.239717Z", "user": "", "contents": "testing 2", "format": "markdown", "investigationId": "INCIDENT-1", "category": "chat", "isTodo": false }] }
{ "total": 2, "data": [{ "id": "8040c9b3-cbd7-4ac5-8178-46fb4f9e85cb@INCIDENT-2", "modified": "2025-01-01T14:38:52.945021634Z", "created": "2025-01-01T14:38:52.945006Z", "user": "user@company.com", "contents": "very important notes", "format": "markdown", "investigationId": "INCIDENT-2", "category": "chat", "note": true, "isTodo": false }, { "id": "bb011b05-b9a4-45b1-8355-38d1673ea520@INCIDENT-2", "modified": "2025-01-01T14:39:08.577516148Z", "created": "2025-01-01T14:39:08.5775Z", "user": "user@company.com", "contents": "even more important notes", "format": "markdown", "investigationId": "INCIDENT-2", "category": "chat", "note": true, "isTodo": false }] }
{ "total": 0, "data": [{ "id": "string", "modified": "string", "created": "string", "user": "string", "parentContent": "string", "contents": "string", "format": "string", "investigationId": "string", "category": "string", "isTodo": true, "tags": ["string"] }] }

Bad Request

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Unauthorized

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"
▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }

Payment Required

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.
▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Internal Server Error

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

2.1.26.5 | Add War Room entries

post /public_api/v1/entries/insert

Add an entry to the cases or alert War Room, including data.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}
Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/entries/insert'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "
{\"id\":\"string\",\"data\":\"string\"}" headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/entries/insert",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/entries/insert")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "
{\"id\":\"string\",\"data\":\"string\"}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "id": "string", "data": "string" }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/entries/insert");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/entries/insert")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"id\":\"string\",\"data\":\"string\"}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = [ "id": "string", "data": "string" ] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/entries/insert")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/entries/insert", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"id\":\"string\",\"data\":\"string\"}", CURLOPT_HTTPHEADER => [ "Authorization:
SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/entries/insert"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"id\":\"string\",\"data\":\"string\"}");
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/entries/insert"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"id\":\"string\",\"data\":\"string\"}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```
Body parameters
application/json
▼ idstring

The ID of the case or issue for which you want to add a War Room entry. For a case ID, prepend "CASE-" to the case ID. For example, if the case ID is 3, the value of `id` should be `CASE-3`. For alert IDs, just put the ID. For example, if the alert ID is 3, the value of `id` should be `3`.

▼ datastring

The data you want to add or the command you want to run in the War Room.

REQUEST Insert command to alert War Room
```
{ "id": "1", "data": "!Print value=1000" }
{ "id": "CASE-1", "data": "!Print value=1" }
```
Responses

OK

Body
application/json
▼ idstring
▼ versioninteger
▼ cacheVersninteger
▼ modifiedstring
▼ sizeInBytesinteger
▼ typeinteger
▼ createdstring
▼ incidentCreationTimestring
▼ investigationCreationTimestring
▼ retryTimestring
▼ userstring
▼ errorSourcestring
▼ contentsstring
▼ formatstring
▼ investigationIdstring
▼ filestring
▼ fileIDstring
▼ parentIdstring
▼ pinnedboolean
▼ fileMetadataobject
▼ parentContentobject
▼ parentEntryTruncatedboolean
▼ systemstring
▼ reputationsobject
▼ categorystring
▼ noteboolean
▼ isTodoboolean
▼ tagsobject
▼ tagsRawobject
▼ startDatestring
▼ timesinteger
▼ recurrentboolean
▼ endingDatestring
▼ timezoneOffsetinteger
▼ cronViewboolean
▼ scheduledboolean
▼ scheduledEntryGuidstring
▼ entryTaskobject
▼ taskIdstring
▼ playbookIdstring
▼ reputationSizeinteger
▼ contentsSizeinteger
▼ brandstring
▼ InstanceIDstring
▼ IndicatorTimelineobject
▼ Relationshipsobject
▼ mirroredboolean
RESPONSE Example 1


```
{ "id": "b30fadee-53e3-4859-8c79-fa328ba7b4fd@1", "version": 1, "cacheVersn": 0, "modified": "2025-01-
29T13:48:41.591466357Z", "sizeInBytes": 0, "type": 1, "created": "2025-01-29T13:48:41.591444333Z",
"incidentCreationTime": "2025-01-29T13:48:41.591444333Z", "investigationCreationTime": "2024-12-25T14:43:07.697Z",
"retryTime": "0001-01-01T00:00:00Z", "user": "", "errorSource": "", "contents": "!Print value=1", "format": "text",
"investigationId": "1", "file": "", "fileID": "", "parentId": "", "pinned": false, "fileMetadata": null,
"parentContent": null, "parentEntryTruncated": false, "system": "", "reputations": null, "category": "chat", "note":
false, "isTodo": false, "tags": null, "tagsRaw": null, "startDate": "0001-01-01T00:00:00Z", "times": 0, "recurrent":
false, "endingDate": "0001-01-01T00:00:00Z", "timezoneOffset": 0, "cronView": false, "scheduled": false,
"scheduledEntryGuid": "", "entryTask": null, "taskId": "", "playbookId": "", "reputationSize": 0, "contentsSize": 14,
"brand": "none", "InstanceID": "", "IndicatorTimeline": null, "Relationships": null, "mirrored": false }
```


Bad Request

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.
▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Unauthorized

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Payment Required

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}", "err_extra": "example" }
```

Forbidden

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.
RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```


Internal Server Error

Body
application/json

The query result upon error.

▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

Example:`"{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input '|alter2'\"}"`
▼ err_extrastring

Additional information describing the error.

RESPONSE

```
{ "err_code": "example", "err_msg": "{\"line\": 1, \"column\": 19, \"message\": \"no viable alternative at input
'|alter2'\"}", "err_extra": "example" }
```

2.1.27 | Asset inventory

APIs for managing assets in the asset inventory

2.1.27.1 | Get all or filtered assets

post /public_api/v1/assets

Retrieve detailed information about all assets within your environment, including enterprise, multi-cloud, code, and external surfaces.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/assets'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"filters\":{\"AND\":
[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]},\"on_demand_fields\":
[\"xdm.host.ipv4_addresses\"],\"sort\":
[{\"FIELD\":\"xdm.asset.name\",\"ORDER\":\"DESC\"}],\"search_from\":0,\"search_to\":1000}" headers = {
'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/assets", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/assets") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"filters\":{\"AND\":
[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]},\"on_demand_fields\":
[\"xdm.host.ipv4_addresses\"],\"sort\":
```

```
[{\"FIELD\":\"xdm.asset.name\",\"ORDER\":\"DESC\"}],\"search_from\":0,\"search_to\":1000}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "filters": { "AND": [ { "SEARCH_FIELD": "xdm.asset.type.class", "SEARCH_TYPE": "NEQ",
"SEARCH_VALUE": "Other" } ] }, "on_demand_fields": [ "xdm.host.ipv4_addresses" ], "sort": [ { "FIELD":
"xdm.asset.name", "ORDER": "DESC" } ], "search_from": 0, "search_to": 1000 }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/assets");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/assets") .header("Authorization",
"SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type", "application/json")
.body("{\"filters\":{\"AND\":
[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]},\"on_demand_fields\":
[\"xdm.host.ipv4_addresses\"],\"sort\":
[{\"FIELD\":\"xdm.asset.name\",\"ORDER\":\"DESC\"}],\"search_from\":0,\"search_to\":1000}") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = [ "filters": ["AND": [ [ "SEARCH_FIELD":
"xdm.asset.type.class", "SEARCH_TYPE": "NEQ", "SEARCH_VALUE": "Other" ] ]], "on_demand_fields":
["xdm.host.ipv4_addresses"], "sort": [ [ "FIELD": "xdm.asset.name", "ORDER": "DESC" ] ], "search_from": 0,
"search_to": 1000 ] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: [])
let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/assets")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/assets",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"filters\":
{\"AND\":
[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]},\"on_demand_fields\":
[\"xdm.host.ipv4_addresses\"],\"sort\":
[{\"FIELD\":\"xdm.asset.name\",\"ORDER\":\"DESC\"}],\"search_from\":0,\"search_to\":1000}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/assets"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"filters\":{\"AND\":
[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]},\"on_demand_fields\":
[\"xdm.host.ipv4_addresses\"],\"sort\":
[{\"FIELD\":\"xdm.asset.name\",\"ORDER\":\"DESC\"}],\"search_from\":0,\"search_to\":1000}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/assets"); var request = new RestRequest(Method.POST);
request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id", "SOME_STRING_VALUE");
request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{\"filters\":
{\"AND\":
[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]},\"on_demand_fields\":
[\"xdm.host.ipv4_addresses\"],\"sort\":
[{\"FIELD\":\"xdm.asset.name\",\"ORDER\":\"DESC\"}],\"search_from\":0,\"search_to\":1000}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ filtersobject

Any of AndFilter

▶ ANDarray

▶ ORarray

▼ on_demand_fieldsarray[string]

▼ sortarray

[

▶ FIELDstring

The field according to which you want the results to be sorted.

▶ ORDERstring (Enum)

Sort order. Valid values include:

- `ASC` (ascending order)
- `DESC` (descending order)

]

▼ search_frominteger

An integer representing the starting offset within the query result set from which you want assets returned.

▼ search_tointeger

An integer representing the end offset within the result set after which you do not want assets returned.
Assets in the asset inventory that are indexed higher than this value are not returned in the final results set.

Example:`1000`

REQUEST

```
{ "filters": { "AND": [ { "SEARCH_FIELD": "xdm.asset.type.class", "SEARCH_TYPE": "NEQ", "SEARCH_VALUE": "Other" } ]
}, "on_demand_fields": [ "xdm.host.ipv4_addresses" ], "sort": [ { "FIELD": "xdm.asset.name", "ORDER": "DESC" } ],
"search_from": 0, "search_to": 1000 }
```

Responses

Ok

Body

application/json

▼ replyobject

▶ dataarray

▶ metadataobject

RESPONSE Example 1

```
{ "reply": { "data": [ { "issues_breakdown": { "critical": 0, "high": 0, "low": 0, "medium": 0 },
"xdm.asset.first_observed": 1747834085000, "xdm.asset.cloud.region": null, "xdm.asset.last_observed": 1748399709000,
"issues_critical": 0, "xdm.asset.strong_id": "172.16.33.51", "xdm.asset.type.category": "Device", "xdm.asset.name":
null, "xdm.asset.type.name": "Generic Device", "cases_breakdown": { "critical": 0, "high": 0, "low": 0, "medium": 0
}, "xdm.asset.provider": "ON_PREM", "xdm.asset.type.class": "Compute", "xdm.asset.id":
"fffd007cff1c15f3a0d152ae630df9630ce00e39bc66811a9fa9b6457a788afc", "xdm.asset.type.id": "GENERIC_DEVICE",
"cases_critical": 0, "xdm.asset.group_ids": [], "xdm.asset.realm": "Other", "xdm.host.ipv4_addresses": [
"172.16.33.51" ] } ], "metadata": { "filter_count": 1, "total_count": 915 } } }
```

Unauthorized

Body

application/json

[

▶ err_codeinteger

▶ err_msgstring

▶ err_extrastring

]

RESPONSE

```
[ { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra": "example" } ]
```

Internal Server Error, Invalid Input

Body

application/json

[

▶ err_codeinteger

▶ err_msgstring

▶ err_extrastring

]

RESPONSE

```
[ { "err_code": 500, "err_msg": "An unexpected behavior occurred by Cortex Pubic API", "err_extra": "example" } ]
```

2.1.27.2 | Get asset by ID

get /public_api/v1/assets/{id}

Retrieve detailed information about the asset specified by asset ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters

▼ id String required

Asset ID
Example: `id_example`
Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/assets/{id}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE" } conn.request("GET", "/public_api/v1/assets/%7Bid%7D",
headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/assets/%7Bid%7D")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE'
response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/v1/assets/%7Bid%7D");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/v1/assets/%7Bid%7D")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE" ] let
request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/assets/%7Bid%7D")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields =
headers let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/assets/%7Bid%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/v1/assets/%7Bid%7D"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/assets/%7Bid%7D"); var request = new
RestRequest(Method.GET); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); IRestResponse response = client.Execute(request);
```
Responses

Ok

Body
application/json
▼ dataarray
[
▶ xdm.asset.last_observedinteger

Timestamp of when the asset was last observed

▶ xdm.asset.cloud.regionstring

Region of the cloud asset

▶ xdm.asset.idstring

Asset ID

▶ xdm.asset.providerstring

▶ xdm.asset.first_observedinteger

Timestamp of when the asset was first observed

▶ xdm.asset.type.namestring

Represents the provider-specific name for a particular asset within a category

▶ xdm.asset.strong_idstring
▶ xdm.asset.namestring

Asset name

▶ xdm.asset.type.classstring

The highest-level grouping of assets based on their general purpose or domain. It is a broad classification that defines the overall function of the assets.

▶ xdm.asset.type.idstring

Asset type ID

▶ xdm.asset.type.categorystring

A more detailed grouping within a class. It categorizes assets based on their normalized function or common type, regardless of the provider or implementation.

▶ issues_criticalinteger
▶ xdm.asset.realmstring
▶ xdm.asset.group_idsarray[string]

List of asset group IDs

▶ cases_breakdownobject
▶ issues_breakdownobject
▶ cases_criticalinteger
▶ xdm.host.ipv4_addressesarray[string]
]
▼ metadataobject
▶ filter_countinteger
▶ total_countinteger
RESPONSE Example 1

```
{ "reply": { "data": [ { "xdm.asset.last_observed": 1748399709000, "xdm.asset.cloud.region": null, "xdm.asset.id":
"fffd007cff1c15f3a0d132ae630df9630fe00e39bc66813a9fa9b6457a786afc", "xdm.asset.provider": "ON_PREM",
"xdm.asset.first_observed": 1747834085000, "xdm.asset.type.name": "Generic Device", "xdm.asset.strong_id":
"172.16.33.51", "xdm.asset.name": null, "xdm.asset.type.class": "Compute", "xdm.asset.type.id": "GENERIC_DEVICE",
"xdm.asset.type.category": "Device", "issues_critical": 0, "xdm.asset.realm": "Other", "xdm.asset.group_ids": [],
"cases_breakdown": { "critical": 0, "high": 0, "low": 0, "medium": 0 }, "issues_breakdown": { "critical": 0, "high":
0, "low": 0, "medium": 0 }, "cases_critical": 0, "xdm.host.ipv4_addresses": [ "172.16.33.51" ] } ], "metadata": {
"filter_count": 1, "total_count": 915 } } }
```


Unauthorized

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring
]
RESPONSE

```
[ { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra": "example" } ]
```


Asset Not Found

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring
]
RESPONSE

```
[ { "err_code": 404, "err_msg": "An unexpected behavior occurred by Cortex Pubic API", "err_extra": "example" } ]
```


Internal Server Error

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring
]
RESPONSE
```
[ { "err_code": 500, "err_msg": "An unexpected behavior occurred by Cortex Pubic API", "err_extra": "example" } ]
```

2.1.27.3 | Get raw fields of asset by ID

get /public_api/v1/assets/{id}/raw_fields

Get the raw fields of the asset specified by asset ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ id String required

Asset ID

Example: `id_example`
Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/assets/{id}/raw_fields'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE" } conn.request("GET",
"/public_api/v1/assets/%7Bid%7D/raw_fields", headers=headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/assets/%7Bid%7D/raw_fields") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' response = http.request(request) puts
response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/v1/assets/%7Bid%7D/raw_fields"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/v1/assets/%7Bid%7D/raw_fields")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE" ] let
request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/assets/%7Bid%7D/raw_fields")! as
URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET"
request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask = session.dataTask(with: request
as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/assets/%7Bid%7D/raw_fields", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL, "https://api-yourfqdn/public_api/v1/assets/%7Bid%7D/raw_fields"); struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id: SOME_STRING_VALUE"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/assets/%7Bid%7D/raw_fields"); var request = new RestRequest(Method.GET); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
▼ replyobject
▶ dataarray
▶ metadataobject
RESPONSE Example 1


```
{ "reply": { "data": [ { "xdm__asset__raw_fields": { "Platform Discovery": { "Adjustable": false, "Description": "The maximum number of times that you can call model inference in one minute for Anthropic Claude 3.5 Sonnet. The quota considers the combined sum of Converse, ConverseStream, InvokeModel and InvokeModelWithResponseStream.", "GlobalQuota": false, "QuotaAppliedAtLevel": "ACCOUNT", "QuotaArn": "arn:aws:servicequotas:ap-south-1:384346710675:bedrock/L-F457545D", "QuotaCode": "L-F457545D", "QuotaName": "Cross-region InvokeModel requests per minute for Anthropic Claude 3.5 Sonnet", "ServiceCode": "bedrock", "ServiceName": "Amazon Bedrock", "Unit": "None", "Value": 100 } } } ], "metadata": { "filter_count": 1, "total_count": 921 } } }
```


Unauthorized

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring
]
RESPONSE


```
[ { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra": "example" } ]
```


Asset Not Found

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring
]
RESPONSE


```
[ { "err_code": 404, "err_msg": "An unexpected behavior occurred by Cortex Pubic API", "err_extra": "example" } ]
```


Internal Server Error

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring
]
RESPONSE


```
[ { "err_code": 500, "err_msg": "An unexpected behavior occurred by Cortex Pubic API", "err_extra": "example" } ]
```

2.1.27.4 | Get schema of asset inventory

get /public_api/v1/assets/schema

Get the schema of the asset inventory, including the field names, pretty field names, and field type for every entry in the data model.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers

▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/assets/schema'
```
```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE" } conn.request("GET", "/public_api/v1/assets/schema",
headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
```
```
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/assets/schema") http
= Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE'
response = http.request(request) puts response.read_body
```
```
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/v1/assets/schema");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.send(data);
```
```
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/v1/assets/schema")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .asString();
```
```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE" ] let
request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/assets/schema")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields =
headers let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
```
```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/assets/schema", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
```
```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/v1/assets/schema"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); CURLcode ret = curl_easy_perform(hnd);
```
```
var client = new RestClient("https://api-yourfqdn/public_api/v1/assets/schema"); var request = new
RestRequest(Method.GET); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); IRestResponse response = client.Execute(request);
```
Responses

Ok

Body
application/json
[
▶ replyobject
]
RESPONSE Example 1
```
{ "reply": { "DATA": [ { "field_name": "xdm.asset.id", "field_pretty_name": "Asset ID", "data_type": "TEXT" }, {
"field_name": "xdm.asset.name", "field_pretty_name": "Name", "data_type": "TEXT" }, { "field_name":
"xdm.asset.provider", "field_pretty_name": "Provider", "data_type": "ENUM" }, { "field_name": "xdm.asset.type.class",
"field_pretty_name": "Class", "data_type": "ENUM" } ] } }
```

Unauthorized

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring

]
RESPONSE
```
[ { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra": "example" } ]
```

Internal Server Error

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring
]
RESPONSE
```
[ { "err_code": 500, "err_msg": "An unexpected behavior occurred by Cortex Pubic API", "err_extra": "example" } ]
```

2.1.27.5 | Get enum values of specified field

get /public_api/v1/assets/enum/{field_name}

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ field_name String required

Field name - must be of type Enum

Example: `fieldName_example`
Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/assets/enum/{field_name}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE" } conn.request("GET",
"/public_api/v1/assets/enum/%7Bfield_name%7D", headers=headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/assets/enum/%7Bfield_name%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' response = http.request(request) puts
response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/v1/assets/enum/%7Bfield_name%7D"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/v1/assets/enum/%7Bfield_name%7D")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE" ] let
request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/assets/enum/%7Bfield_name%7D")!
as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET"
request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask = session.dataTask(with: request
as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/assets/enum/%7Bfield_name%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]);
```

```
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL, "https://api-yourfqdn/public_api/v1/assets/enum/%7Bfield_name%7D"); struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id: SOME_STRING_VALUE"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/assets/enum/%7Bfield_name%7D"); var request = new RestRequest(Method.GET); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
[
▶ replyobject
]
RESPONSE Example 1

```
{ "reply": { "DATA": [ { "NAME": "ACTIVE_DIRECTORY", "PRETTY_NAME": "Active Directory" }, { "NAME": "AKAMAI", "PRETTY_NAME": "Akamai" }, { "NAME": "ALIBABA_CLOUD", "PRETTY_NAME": "Alibaba Cloud" } ] } }
```


Unauthorized

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring
]
RESPONSE

```
[ { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra": "example" } ]
```


Internal Server Error

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring
]
RESPONSE

```
[ { "err_code": 500, "err_msg": "An unexpected behavior occurred by Cortex Pubic API", "err_extra": "example" } ]
```

2.1.28 | Asset groups

APIs for managing asset groups

2.1.28.1 | Get all or filtered asset groups

post /public_api/v1/asset-groups

By grouping assets based on shared attributes, you can address them collectively. Asset groups enable more efficient bulk actions and simplifies both filtering and scoping within the inventory and across the platform.

Get all or filtered asset groups.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/asset-groups'
-d ''
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"filters\":{\"AND\":
[{\"SEARCH_FIELD\":\"XDM.ASSET_GROUP.TYPE\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"Dynamic\"}]},\"sort\":
[{\"FIELD\":\"XDM.ASSET_GROUP.LAST_UPDATE_TIME\",\"ORDER\":\"DESC\"}],\"search_from\":0,\"search_to\":1000}" headers
= { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/asset-groups", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
```

```
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/asset-groups") http
= Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE' request["x-xdr-auth-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "{\"filters\":{\"AND\":
[{\"SEARCH_FIELD\":\"XDM.ASSET_GROUP.TYPE\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"Dynamic\"}]},\"sort\":
[{\"FIELD\":\"XDM.ASSET_GROUP.LAST_UPDATE_TIME\",\"ORDER\":\"DESC\"}],\"search_from\":0,\"search_to\":1000}" response
= http.request(request) puts response.read_body
```

```
const data = JSON.stringify({ "filters": { "AND": [ { "SEARCH_FIELD": "XDM.ASSET_GROUP.TYPE", "SEARCH_TYPE": "EQ",
"SEARCH_VALUE": "Dynamic" } ] }, "sort": [ { "FIELD": "XDM.ASSET_GROUP.LAST_UPDATE_TIME", "ORDER": "DESC" } ],
"search_from": 0, "search_to": 1000 }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/asset-groups");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/asset-groups")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"filters\":{\"AND\":
[{\"SEARCH_FIELD\":\"XDM.ASSET_GROUP.TYPE\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"Dynamic\"}]},\"sort\":
[{\"FIELD\":\"XDM.ASSET_GROUP.LAST_UPDATE_TIME\",\"ORDER\":\"DESC\"}],\"search_from\":0,\"search_to\":1000}")
.asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = [ "filters": ["AND": [ [ "SEARCH_FIELD":
"XDM.ASSET_GROUP.TYPE", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "Dynamic" ] ]], "sort": [ [ "FIELD":
"XDM.ASSET_GROUP.LAST_UPDATE_TIME", "ORDER": "DESC" ] ], "search_from": 0, "search_to": 1000 ] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/asset-groups")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/asset-
groups", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"filters\":
{\"AND\":[{\"SEARCH_FIELD\":\"XDM.ASSET_GROUP.TYPE\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"Dynamic\"}]},\"sort\":
[{\"FIELD\":\"XDM.ASSET_GROUP.LAST_UPDATE_TIME\",\"ORDER\":\"DESC\"}],\"search_from\":0,\"search_to\":1000}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/asset-groups"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"filters\":{\"AND\":
[{\"SEARCH_FIELD\":\"XDM.ASSET_GROUP.TYPE\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"Dynamic\"}]},\"sort\":
[{\"FIELD\":\"XDM.ASSET_GROUP.LAST_UPDATE_TIME\",\"ORDER\":\"DESC\"}],\"search_from\":0,\"search_to\":1000}");
CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/asset-groups"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"filters\":{\"AND\":
[{\"SEARCH_FIELD\":\"XDM.ASSET_GROUP.TYPE\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"Dynamic\"}]},\"sort\":
[{\"FIELD\":\"XDM.ASSET_GROUP.LAST_UPDATE_TIME\",\"ORDER\":\"DESC\"}],\"search_from\":0,\"search_to\":1000}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ filtersobject
One of AndFilterForGroups
▶ ANDarray
▶ ANDarray
▼ sortarray
[
▶ FIELDstring
▶ ORDERstring (Enum)
]
▼ search_frominteger
▼ search_tointeger
Example:1000
REQUEST
```
{ "filters": { "AND": [ { "SEARCH_FIELD": "XDM.ASSET_GROUP.TYPE", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "Dynamic" } ]
}, "sort": [ { "FIELD": "XDM.ASSET_GROUP.LAST_UPDATE_TIME", "ORDER": "DESC" } ], "search_from": 0, "search_to": 1000
}
```
Responses

Ok

Body
application/json
[
▶ replyobject
]
RESPONSE
```
[ { "reply": { "data": [ {} ], "metadata": { "filter_count": 1, "total_count": 1000 } } } ]
```

Unauthorized

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring
]
RESPONSE
```
[ { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra": "example" } ]
```

Internal Server Error, Invalid Input

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring
]
RESPONSE
```
[ { "err_code": 500, "err_msg": "An unexpected behavior occurred by Cortex Pubic API", "err_extra": "example" } ]
```

2.1.28.2 | Create an Asset Group

post /public_api/v1/asset-groups/create

Create a dynamic Asset Group by specifying the filters, or a static group to manually include individual assets.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/asset-groups/create'
-d ''
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"asset_group\":
{\"group_name\":\"string\",\"group_type\":\"Dynamic\",\"group_description\":\"string\",\"membership_predicate\":
{\"AND\":[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]}}}" headers
= { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/asset-groups/create", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
```

```
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/asset-
groups/create") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE'
request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "
{\"asset_group\":
{\"group_name\":\"string\",\"group_type\":\"Dynamic\",\"group_description\":\"string\",\"membership_predicate\":
{\"AND\":[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]}}}"
response = http.request(request) puts response.read_body
```

```
const data = JSON.stringify({ "asset_group": { "group_name": "string", "group_type": "Dynamic", "group_description":
"string", "membership_predicate": { "AND": [ { "SEARCH_FIELD": "xdm.asset.type.class", "SEARCH_TYPE": "NEQ",
"SEARCH_VALUE": "Other" } ] } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/asset-groups/create");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/asset-groups/create")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"asset_group\":
{\"group_name\":\"string\",\"group_type\":\"Dynamic\",\"group_description\":\"string\",\"membership_predicate\":
{\"AND\":[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]}}}")
.asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["asset_group": [ "group_name": "string", "group_type":
"Dynamic", "group_description": "string", "membership_predicate": ["AND": [ [ "SEARCH_FIELD": "xdm.asset.type.class",
"SEARCH_TYPE": "NEQ", "SEARCH_VALUE": "Other" ] ]] ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/asset-groups/create")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/asset-
groups/create", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT =>
30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "
{\"asset_group\":
{\"group_name\":\"string\",\"group_type\":\"Dynamic\",\"group_description\":\"string\",\"membership_predicate\":
{\"AND\":[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]}}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/asset-groups/create"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-xdr-auth-id:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"asset_group\":
{\"group_name\":\"string\",\"group_type\":\"Dynamic\",\"group_description\":\"string\",\"membership_predicate\":
{\"AND\":[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]}}}");
CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/asset-groups/create"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"asset_group\":
{\"group_name\":\"string\",\"group_type\":\"Dynamic\",\"group_description\":\"string\",\"membership_predicate\":
{\"AND\":[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]}}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
application/json
▼ asset_groupobject
▶ group_namestring

Asset Group name

▶ group_typestring (Enum)

The type of Asset Group. Valid values include:

- `Dynamic`: Assets grouped using filters. Any asset that meets the defined criteria is included.
- `Static`: Manually add individual assets to be included in a group.

▶ group_descriptionstring

Add an optional description to clarify the purpose of the Asset Group.

▶ membership_predicateobject

Define the filter conditions for selecting which assets to be included in a dynamic Asset Group.

REQUEST
```
{ "asset_group": { "group_name": "example", "group_type": "Dynamic", "group_description": "example",
"membership_predicate": { "AND": [ { "SEARCH_FIELD": "xdm.asset.type.class", "SEARCH_TYPE": "NEQ", "SEARCH_VALUE":
"Other" } ] } } }
```
Responses

Ok

Body
application/json
[
▶ replyobject
]
RESPONSE
```
[ { "reply": { "data": { "success": false, "asset_group_id": 1 } } } ]
```

Unauthorized

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring
]
RESPONSE
```
[ { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra": "example" } ]
```

Internal Server Error, Invalid Input

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring
]
RESPONSE
```
[ { "err_code": 500, "err_msg": "An unexpected behavior occurred by Cortex Pubic API", "err_extra": "example" } ]
```

2.1.28.3 | Update an Asset Group

post /public_api/v1/asset-groups/update/{group_id}

Update the Asset Group specified by Asset Group ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ group_id String required

Asset Group ID
Example: `groupId_example`
Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/asset-groups/update/{group_id}'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"asset_group\":
{\"group_name\":\"string\",\"group_type\":\"Dynamic\",\"group_description\":\"string\",\"membership_predicate\":
{\"AND\":[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]}}}" headers
= { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/asset-groups/update/%7Bgroup_id%7D", payload, headers) res = conn.getresponse()
data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/asset-
groups/update/%7Bgroup_id%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE'
request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "
{\"asset_group\":
{\"group_name\":\"string\",\"group_type\":\"Dynamic\",\"group_description\":\"string\",\"membership_predicate\":
{\"AND\":[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]}}}"
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "asset_group": { "group_name": "string", "group_type": "Dynamic", "group_description":
"string", "membership_predicate": { "AND": [ { "SEARCH_FIELD": "xdm.asset.type.class", "SEARCH_TYPE": "NEQ",
"SEARCH_VALUE": "Other" } ] } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/asset-
groups/update/%7Bgroup_id%7D"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-
xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/asset-groups/update/%7Bgroup_id%7D")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"asset_group\":
{\"group_name\":\"string\",\"group_type\":\"Dynamic\",\"group_description\":\"string\",\"membership_predicate\":
{\"AND\":[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]}}}")
.asString();
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["asset_group": [ "group_name": "string", "group_type":
"Dynamic", "group_description": "string", "membership_predicate": ["AND": [ [ "SEARCH_FIELD": "xdm.asset.type.class",
"SEARCH_TYPE": "NEQ", "SEARCH_VALUE": "Other" ] ]] ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/asset-groups/update/%7Bgroup_id%7D")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/asset-
groups/update/%7Bgroup_id%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"asset_group\":
{\"group_name\":\"string\",\"group_type\":\"Dynamic\",\"group_description\":\"string\",\"membership_predicate\":
{\"AND\":[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]}}}",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/asset-groups/update/%7Bgroup_id%7D"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers,
"x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
```

```
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"asset_group\":
{\"group_name\":\"string\",\"group_type\":\"Dynamic\",\"group_description\":\"string\",\"membership_predicate\":
{\"AND\":[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]}}}");
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/asset-groups/update/%7Bgroup_id%7D"); var request =
new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-
id", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"asset_group\":
{\"group_name\":\"string\",\"group_type\":\"Dynamic\",\"group_description\":\"string\",\"membership_predicate\":
{\"AND\":[{\"SEARCH_FIELD\":\"xdm.asset.type.class\",\"SEARCH_TYPE\":\"NEQ\",\"SEARCH_VALUE\":\"Other\"}]}}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ asset_groupobject

▶ group_namestring

Asset Group name

▶ group_typestring (Enum)

The type of Asset Group. Valid values include:

- `Dynamic`: Assets grouped using filters. Any asset that meets the defined criteria is included.
- `Static`: Manually add individual assets to be included in a group.

▶ group_descriptionstring

Add an optional description to clarify the purpose of the Asset Group.

▶ membership_predicateobject

Define the filter conditions for selecting which assets to be included in a dynamic Asset Group.

REQUEST

```
{ "asset_group": { "group_name": "example", "group_type": "Dynamic", "group_description": "example",
"membership_predicate": { "AND": [ { "SEARCH_FIELD": "xdm.asset.type.class", "SEARCH_TYPE": "NEQ", "SEARCH_VALUE":
"Other" } ] } } }
```

Responses

Ok

Body

application/json

[

▶ replyobject

]

RESPONSE

```
[ { "reply": { "data": { "success": false } } } ]
```

Unauthorized

Body

application/json

[

▶ err_codeinteger

▶ err_msgstring

▶ err_extrastring

]

RESPONSE

```
[ { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra": "example" } ]
```

Internal Server Error, Invalid Input

Body

application/json

[

▶ err_codeinteger

▶ err_msgstring

▶ err_extrastring

]

RESPONSE

```
[ { "err_code": 500, "err_msg": "An unexpected behavior occurred by Cortex Pubic API", "err_extra": "example" } ]
```

2.1.28.4 | Delete an Asset Group

post /public_api/v1/asset-groups/delete/{group_id}

Delete the Asset Group specified by Asset Group ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ group_id String required

Asset Group ID

Example: `groupId_example`
Request headers
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/v1/asset-groups/delete/{group_id}'
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE" } conn.request("POST", "/public_api/v1/asset-
groups/delete/%7Bgroup_id%7D", headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-
8"))
```

```
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/asset-
groups/delete/%7Bgroup_id%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] = 'SOME_STRING_VALUE'
request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' response = http.request(request) puts response.read_body
```

```
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/asset-
groups/delete/%7Bgroup_id%7D"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("x-
xdr-auth-id", "SOME_STRING_VALUE"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/asset-groups/delete/%7Bgroup_id%7D")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .asString();
```

```
import Foundation let headers = [ "Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE" ] let
request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/asset-
groups/delete/%7Bgroup_id%7D")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask =
session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error !=
nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/asset-
groups/delete/%7Bgroup_id%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/asset-groups/delete/%7Bgroup_id%7D"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers,
"x-xdr-auth-id: SOME_STRING_VALUE"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); CURLcode ret =
curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/asset-groups/delete/%7Bgroup_id%7D"); var request =
new RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-
id", "SOME_STRING_VALUE"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json

[
▶ replyobject
]
RESPONSE
[{ "reply": { "data": { "success": false } } }]

Unauthorized

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring
]
RESPONSE
[{ "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra": "example" }]

Internal Server Error, Invalid Input

Body
application/json
[
▶ err_codeinteger
▶ err_msgstring
▶ err_extrastring
]
RESPONSE
[{ "err_code": 500, "err_msg": "An unexpected behavior occurred by Cortex Pubic API", "err_extra": "example" }]

2.2 | Cortex Platform IAM APIs

2.2.1 | Preface

Identity and Access Management (IAM) APIs for the Cortex platform. These APIs enable programmatic management of roles, user groups, users, API keys, and scope-based access control (SBAC) configurations. With these endpoints, you can create, read, update, and delete IAM entities, manage permissions and access controls, and configure security scopes for users, groups, and API keys. These APIs are intended to be accessed via API keys. Therefore, the key must have the RBAC permission 'access_management_view' for list & read operations and permission 'access_management_action' for create, edit, & delete operations.

Version: Cortex Cloud 1.3

2.2.1.1 | Servers

https://api-{{fqdn}}

2.2.1.2 | Authentication methods

▼ Authorization
Api Key "x-api-key"

Authorized requests to the API should use: **x-api-key** inside **a header** with the access token.

▼ x-xdr-auth-id
Api Key "x-api-key-id"

Authorized requests to the API should use: **x-api-key-id** inside **a header** with the access token.

2.2.2 | Roles

Operations for managing roles including creation, editing, deletion, and listing roles.

2.2.2.1 | List all roles

get /platform/iam/v1/role

This endpoint retrieves a list of roles and returns their attributes. This API does not list permissions for said roles. To receive that information, please leverage the existing Get Roles API (https://docs-cortex.paloaltonetworks.com/r/Cortex-Cloud-Platform-APIs/Get-Roles). Note: 'role_id' refers to the immutable role ID, while 'pretty_name' refers to the display name.

CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
'https://api-}/platform/iam/v1/role'
import http.client conn = http.client.HTTPSConnection("api-") conn.request("GET",
"%7B%7Bfqdn%7D%7D/platform/iam/v1/role") res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role");
xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string:
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "GET" let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error
#:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role"); var request = new
RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```

Responses

Successfully retrieved roles

Body
application/json
▼ metadataobject

Metadata for the response

Free-Form object
RESPONSE

```
{ "metadata": {} }
```

Unauthorized access

Body
application/json
RESPONSE

```
{ "reply": { "err_code": 401, "err_msg": "Public API request unauthorized", "err_extra": null } }
```

Unauthorized access due to lack of sufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_msgstring
▶ err_extrastring
▶ err_codeinteger
RESPONSE

```
{ "data": { "reply": { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra":
"Insufficient permissions for api key" } } }
```

Internal server error. A unified status for API communication type errors.

2.2.2.2 | Create a new role

post /platform/iam/v1/role

To identify all potential permissions that can be entered into the 'component_permissions' field, please utilize the 'GET /platform/iam/v1/role/permission-config' endpoint and utilize the 'view_name' and 'action_name' properties. Note: if you input an 'action' permission, then the relevant 'view' permission will be added for you. There will also be checks done on sub-permissions, ensuring that the main 'action' permissions are present as well. The available datasets can also be found by utilizing the 'GET /platform/iam/v1/role/permission-config' endpoint. Any dataset related permissions are not permissible in the 'component_permissions' field (an error will be thrown). The 'permissions' field in the entry for 'dataset_permissions' refers to dataset names for said dataset category.It is important to note that the 'access_all' field will only grant the role access to all datasets in said category that the API key has access to.

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-}/platform/iam/v1/role'
-d '{ "request_data" : { "component_permissions" : [ "rules_action", "wf_verdict_change" ], "dataset_permissions" : [
{ "category" : "Lookup", "access_all" : true, "permissions" : [ ] } ], "pretty_name" : "CustomRoleName",
"description" : "A custom role with specific permissions" } }'
import http.client conn = http.client.HTTPSConnection("api-") payload = "{\"request_data\":
{\"component_permissions\":[\"rules_action\",\"wf_verdict_change\"],\"dataset_permissions\":
[{\"category\":\"Lookup\",\"access_all\":true,\"permissions\":
[]}],\"pretty_name\":\"CustomRoleName\",\"description\":\"A custom role with specific permissions\"}}" headers = {
'content-type': "application/json" } conn.request("POST", "%7B%7Bfqdn%7D%7D/platform/iam/v1/role", payload, headers)
res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"component_permissions\":[\"rules_action\",\"wf_verdict_change\"],\"dataset_permissions\":
[{\"category\":\"Lookup\",\"access_all\":true,\"permissions\":
[]}],\"pretty_name\":\"CustomRoleName\",\"description\":\"A custom role with specific permissions\"}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "component_permissions": [ "rules_action", "wf_verdict_change" ],
"dataset_permissions": [ { "category": "Lookup", "access_all": true, "permissions": [] } ], "pretty_name":
"CustomRoleName", "description": "A custom role with specific permissions" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role");
xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role") .header("content-
type", "application/json") .body("{\"request_data\":{\"component_permissions\":
[\"rules_action\",\"wf_verdict_change\"],\"dataset_permissions\":
[{\"category\":\"Lookup\",\"access_all\":true,\"permissions\":
[]}],\"pretty_name\":\"CustomRoleName\",\"description\":\"A custom role with specific permissions\"}}") .asString();
import Foundation let headers = ["content-type": "application/json"] let parameters = ["request_data": [
"component_permissions": ["rules_action", "wf_verdict_change"], "dataset_permissions": [ [ "category": "Lookup",
"access_all": true, "permissions": [] ] ], "pretty_name": "CustomRoleName", "description": "A custom role with
specific permissions" ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options:
[]) let request = NSMutableURLRequest(url: NSURL(string: "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role")! as
URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST"
request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"component_permissions\":
[\"rules_action\",\"wf_verdict_change\"],\"dataset_permissions\":
[{\"category\":\"Lookup\",\"access_all\":true,\"permissions\":
[]}],\"pretty_name\":\"CustomRoleName\",\"description\":\"A custom role with specific permissions\"}}",
CURLOPT_HTTPHEADER => [ "content-type: application/json" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"component_permissions\":
[\"rules_action\",\"wf_verdict_change\"],\"dataset_permissions\":
[{\"category\":\"Lookup\",\"access_all\":true,\"permissions\":
[]}],\"pretty_name\":\"CustomRoleName\",\"description\":\"A custom role with specific permissions\"}}"); CURLcode ret
= curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role"); var request = new
RestRequest(Method.POST); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"component_permissions\":
```

```
[\"rules_action\",\"wf_verdict_change\"],\"dataset_permissions\":
[{\"category\":\"Lookup\",\"access_all\":true,\"permissions\":
[]}],\"pretty_name\":\"CustomRoleName\",\"description\":\"A custom role with specific permissions\"}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
required
application/json
▼ request_dataobjectrequired
▶ component_permissionsarray[string]

List of component permissions for the role. Possible values can be found by using the permission configs API and referring to the `rbac_permissions` field.

▶ dataset_permissionsarray

Optional list of dataset permissions. This must be a list of JSONs. The layout for this JSON can be found under the 'DatasetPermission' schema . Note: possible dataset values can be found by using the permission configs API and referring to the `datasetGroups` field.

▶ pretty_namestring

Name of the role

▶ descriptionstring

Optional description of the role

REQUEST
```
{ "request_data": { "component_permissions": [ "rules_action", "wf_verdict_change" ], "dataset_permissions": [ {
"category": "Lookup", "access_all": true, "permissions": [] } ], "pretty_name": "CustomRoleName", "description": "A
custom role with specific permissions" } }
```
Responses

Role created successfully

Body
application/json
▼ dataobjectrequired
▶ messagestring

Success message indicating the role was created

RESPONSE
```
{ "data": { "message": "example" } }
```

Bad request for role create call

Body
application/json
▼ dataobjectrequired
▶ err_msgstring

Error message describing the issue

▶ metadataobjectrequired
RESPONSE
```
{ "data": { "err_msg": "example", "metadata": { "err_extra": "example", "err_code": 0 } } }
```

Unauthorized access

Body
application/json
RESPONSE
```
{ "reply": { "err_code": 401, "err_msg": "Public API request unauthorized", "err_extra": null } }
```

Unauthorized access due to lack of sufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_msgstring
▶ err_extrastring
▶ err_codeinteger
RESPONSE
```
{ "data": { "reply": { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra":
"Insufficient permissions for api key" } } }
```

Internal server error. A unified status for API communication type errors.

2.2.2.3 | Delete an existing role

delete /platform/iam/v1/role/{role_id}

Delete an existing Role

Path parameters
▼ role_id String required

Full role id of the custom role

Example: `roleId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'DELETE'
-H 'Accept: application/json'
'https://api-}/platform/iam/v1/role/{role_id}'
import http.client conn = http.client.HTTPSConnection("api-") conn.request("DELETE",
"%7B%7Bfqdn%7D%7D/platform/iam/v1/role/%7Brole_id%7D") res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url =
URI("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role/%7Brole_id%7D") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Delete.new(url) response =
http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("DELETE",
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role/%7Brole_id%7D"); xhr.send(data);
HttpResponse<String> response = Unirest.delete("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role/%7Brole_id%7D")
.asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string:
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role/%7Brole_id%7D")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "DELETE" let session = URLSession.shared let dataTask =
session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error !=
nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role/%7Brole_id%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING
=> "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "DELETE", ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if
($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "DELETE"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role/%7Brole_id%7D"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role/%7Brole_id%7D"); var request = new
RestRequest(Method.DELETE); IRestResponse response = client.Execute(request);
```
Responses

Role deleted successfully

Body
application/json
▼ dataobjectrequired
▶ messagestring

Success message indicating the role was deleted

RESPONSE
```
{ "data": { "message": "example" } }
```

Bad request for role delete call

Body
application/json
▼ dataobjectrequired
▶ err_msgstring

Error message describing the issue

▶ metadataobjectrequired
RESPONSE
{ "data": { "err_msg": "example", "metadata": { "err_extra": "example", "err_code": 0 } } }

Unauthorized access

Body
application/json
RESPONSE
{ "reply": { "err_code": 401, "err_msg": "Public API request unauthorized", "err_extra": null } }

Unauthorized access due to lack of sufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_msgstring
▶ err_extrastring
▶ err_codeinteger
RESPONSE
{ "data": { "reply": { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra": "Insufficient permissions for api key" } } }

Internal server error. A unified status for API communication type errors.

2.2.2.4 | List all permission configs

get /platform/iam/v1/role/permission-config

Returns the current list of permissions available for the specified tenant. Each permission includes:

- name: Display name of the permission as shown in the UI.
- view_name: Permission key for the "View" option, used in role creation APIs.
- action_name: Permission key for the "View/Edit" option, used in role creation APIs.

Intended use: Use this endpoint to identify the correct permission keys corresponding to the display names visible in the UI.
Note - The response mirrors how permissions are organized and displayed in the UI (by category and subcategory).

CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
'https://api-}/platform/iam/v1/role/permission-config'
import http.client conn = http.client.HTTPSConnection("api-") conn.request("GET",
"%7B%7Bfqdn%7D%7D/platform/iam/v1/role/permission-config") res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url =
URI("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role/permission-config") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) response =
http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role/permission-
config"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role/permission-config")
.asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string:
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role/permission-config")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role/permission-config", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role/permission-config"); CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/role/permission-config"); var request =
new RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```

Responses

Successfully retrieved permission configs

Body
application/json
▼ metadataobject

Metadata for the response

Free-Form object
RESPONSE

```
{ "metadata": {} }
```

Unauthorized access

Body
application/json
RESPONSE

```
{ "reply": { "err_code": 401, "err_msg": "Public API request unauthorized", "err_extra": null } }
```

Unauthorized access due to lack of sufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_msgstring
▶ err_extrastring
▶ err_codeinteger
RESPONSE

```
{ "data": { "reply": { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra":
"Insufficient permissions for api key" } } }
```

Internal server error. A unified status for API communication type errors.

2.2.3 | User Groups

Operations for managing user groups including creation, editing, deletion, and listing user groups.

2.2.3.1 | List all user groups

get /platform/iam/v1/user-group

This endpoint retrieves a list of user groups and returns their attributes. For possible 'group_type' values, please refer to the enum documentation for 'GroupType'. Also, 'nested_groups' refers to the list of direct child groups of the given user group. Similarly, 'idp_groups' refers to the identifiers of associated identity provider groups to the given user group.

CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
'https://api-}/platform/iam/v1/user-group'
import http.client conn = http.client.HTTPSConnection("api-") conn.request("GET",
"%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group") res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-
group") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE
request = Net::HTTP::Get.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group");
xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string:
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "GET" let session = URLSession.shared let dataTask =
session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error !=
nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error
#:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group"); var request = new
RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```

Responses

Successfully retrieved user groups

Body
application/json
▼ metadataobject

Metadata for the response

Free-Form object
RESPONSE

```
{ "metadata": {} }
```

Unauthorized access

Body
application/json
RESPONSE

```
{ "reply": { "err_code": 401, "err_msg": "Public API request unauthorized", "err_extra": null } }
```

Unauthorized access due to lack of sufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_msgstring
▶ err_extrastring
▶ err_codeinteger
RESPONSE

```
{ "data": { "reply": { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra":
"Insufficient permissions for api key" } } }
```

Internal server error. A unified status for API communication type errors.

2.2.3.2 | Create a new user group

post /platform/iam/v1/user-group

This endpoint creates a new user group with the specified configuration. You can assign a role, add users, configure nested groups, and link identity provider groups. All fields except 'group_name' are optional.

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-}/platform/iam/v1/user-group'
-d '{ "request_data" : { "group_name" : "test_group_name", "role_id" : "role_name_123", "description" : "Test
Description", "users" : [ "user1@test.com" ], "nested_group_ids" : [ "7f3o5b2c-4d5e-4f6a-8b9c-1d3o5f4a5b6c_123" ],
"idp_groups" : [ "test idp group" ] } }'
import http.client conn = http.client.HTTPSConnection("api-") payload = "{\"request_data\":
{\"group_name\":\"test_group_name\",\"role_id\":\"role_name_123\",\"description\":\"Test Description\",\"users\":
[\"user1@test.com\"],\"nested_group_ids\":[\"7f3o5b2c-4d5e-4f6a-8b9c-1d3o5f4a5b6c_123\"],\"idp_groups\":[\"test idp
group\"]}}" headers = { 'content-type': "application/json" } conn.request("POST",
"%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-
group") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE
request = Net::HTTP::Post.new(url) request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"group_name\":\"test_group_name\",\"role_id\":\"role_name_123\",\"description\":\"Test Description\",\"users\":
```

```
[\"user1@test.com\"],\"nested_group_ids\":[\"7f3o5b2c-4d5e-4f6a-8b9c-1d3o5f4a5b6c_123\"],\"idp_groups\":[\"test idp group\"]}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "group_name": "test_group_name", "role_id": "role_name_123", "description": "Test Description", "users": [ "user1@test.com" ], "nested_group_ids": [ "7f3o5b2c-4d5e-4f6a-8b9c-1d3o5f4a5b6c_123" ], "idp_groups": [ "test idp group" ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group") .header("content-type", "application/json") .body("{\"request_data\": {\"group_name\":\"test_group_name\",\"role_id\":\"role_name_123\",\"description\":\"Test Description\",\"users\": [\"user1@test.com\"],\"nested_group_ids\":[\"7f3o5b2c-4d5e-4f6a-8b9c-1d3o5f4a5b6c_123\"],\"idp_groups\":[\"test idp group\"]}}") .asString();
import Foundation let headers = ["content-type": "application/json"] let parameters = ["request_data": [ "group_name": "test_group_name", "role_id": "role_name_123", "description": "Test Description", "users": ["user1@test.com"], "nested_group_ids": ["7f3o5b2c-4d5e-4f6a-8b9c-1d3o5f4a5b6c_123"], "idp_groups": ["test idp group"] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\": {\"group_name\":\"test_group_name\",\"role_id\":\"role_name_123\",\"description\":\"Test Description\",\"users\": [\"user1@test.com\"],\"nested_group_ids\":[\"7f3o5b2c-4d5e-4f6a-8b9c-1d3o5f4a5b6c_123\"],\"idp_groups\":[\"test idp group\"]}}", CURLOPT_HTTPHEADER => [ "content-type: application/json" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd, CURLOPT_URL, "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group"); struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\": {\"group_name\":\"test_group_name\",\"role_id\":\"role_name_123\",\"description\":\"Test Description\",\"users\": [\"user1@test.com\"],\"nested_group_ids\":[\"7f3o5b2c-4d5e-4f6a-8b9c-1d3o5f4a5b6c_123\"],\"idp_groups\":[\"test idp group\"]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group"); var request = new RestRequest(Method.POST); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{\"request_data\": {\"group_name\":\"test_group_name\",\"role_id\":\"role_name_123\",\"description\":\"Test Description\",\"users\": [\"user1@test.com\"],\"nested_group_ids\":[\"7f3o5b2c-4d5e-4f6a-8b9c-1d3o5f4a5b6c_123\"],\"idp_groups\":[\"test idp group\"]}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ group_namestring

Name of the user group

▶ role_idstring

Unique identifier of the role

▶ descriptionstring

Optional description of the user group

▶ usersarray[string]

List of users for the user group

▶ nested_group_idsarray[string]

List of nested groups

▶ idp_groupsarray[string]

List of idp groups

REQUEST

{ "request_data": { "group_name": "test_group_name", "role_id": "role_name_123", "description": "Test Description", "users": ["user1@test.com"], "nested_group_ids": ["7f3o5b2c-4d5e-4f6a-8b9c-1d3o5f4a5b6c_123"], "idp_groups": ["test idp group"] } }
Responses

User Group created successfully

Body
application/json
▼ dataobjectrequired
▶ messagestring

Success message indicating the user group was created

RESPONSE
{ "data": { "message": "example" } }

Bad request for user group create call

Body
application/json
▼ dataobjectrequired
▶ err_msgstring

Error message describing the issue

▶ metadataobjectrequired
RESPONSE
{ "data": { "err_msg": "example", "metadata": { "err_extra": "example", "err_code": 0 } } }

Unauthorized access

Body
application/json
RESPONSE
{ "reply": { "err_code": 401, "err_msg": "Public API request unauthorized", "err_extra": null } }

Unauthorized access due to lack of sufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_msgstring
▶ err_extrastring
▶ err_codeinteger
RESPONSE
{ "data": { "reply": { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra": "Insufficient permissions for api key" } } }

Internal server error. A unified status for API communication type errors.

2.2.3.3 | Edit an existing user group

patch /platform/iam/v1/user-group/{group_id}

This endpoint allows modification of an existing user group by providing the group_id in the path and the update details in the request body. Only specified fields will be updated. Please note that to remove attributes, you must provide either an empty string or empty list as the value of the respective field in the request body.

Path parameters
▼ group_id String required

Full user group id

Example: 7f8a9b2c-4d5e-4f6a-8b9c-1d2e3f4a5b6c_123
CLIENT REQUEST Bash+curl
```
curl -X 'PATCH'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-}/platform/iam/v1/user-group/{group_id}'
-d '{ "request_data" : { "group_name" : "test_group_name", "role_id" : "role_name_123", "description" : "Test Updated Description", "users" : [ "test user" ], "nested_group_ids" : [ "7f3o5b2c-4d5e-4f6a-8b9c-1d3o5f4a5b6c_123" ],
```

```
"idp_groups" : [ "test idp group" ] } }'
import http.client conn = http.client.HTTPSConnection("api-") payload = "{\"request_data\":
{\"group_name\":\"test_group_name\",\"role_id\":\"role_name_123\",\"description\":\"Test Updated
Description\",\"users\":[\"test user\"],\"nested_group_ids\":[\"7f3o5b2c-4d5e-4f6a-8b9c-
1d3o5f4a5b6c_123\"],\"idp_groups\":[\"test idp group\"]}}" headers = { 'content-type': "application/json" }
conn.request("PATCH", "%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group/7f8a9b2c-4d5e-4f6a-8b9c-1d2e3f4a5b6c_123",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-
group/7f8a9b2c-4d5e-4f6a-8b9c-1d2e3f4a5b6c_123") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Patch.new(url) request["content-type"] =
'application/json' request.body = "{\"request_data\":
{\"group_name\":\"test_group_name\",\"role_id\":\"role_name_123\",\"description\":\"Test Updated
Description\",\"users\":[\"test user\"],\"nested_group_ids\":[\"7f3o5b2c-4d5e-4f6a-8b9c-
1d3o5f4a5b6c_123\"],\"idp_groups\":[\"test idp group\"]}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "group_name": "test_group_name", "role_id": "role_name_123",
"description": "Test Updated Description", "users": [ "test user" ], "nested_group_ids": [ "7f3o5b2c-4d5e-4f6a-8b9c-
1d3o5f4a5b6c_123" ], "idp_groups": [ "test idp group" ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials =
true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("PATCH", "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-
group/7f8a9b2c-4d5e-4f6a-8b9c-1d2e3f4a5b6c_123"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.patch("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group/7f8a9b2c-
4d5e-4f6a-8b9c-1d2e3f4a5b6c_123") .header("content-type", "application/json") .body("{\"request_data\":
{\"group_name\":\"test_group_name\",\"role_id\":\"role_name_123\",\"description\":\"Test Updated
Description\",\"users\":[\"test user\"],\"nested_group_ids\":[\"7f3o5b2c-4d5e-4f6a-8b9c-
1d3o5f4a5b6c_123\"],\"idp_groups\":[\"test idp group\"]}}") .asString();
import Foundation let headers = ["content-type": "application/json"] let parameters = ["request_data": [
"group_name": "test_group_name", "role_id": "role_name_123", "description": "Test Updated Description", "users":
["test user"], "nested_group_ids": ["7f3o5b2c-4d5e-4f6a-8b9c-1d3o5f4a5b6c_123"], "idp_groups": ["test idp group"] ]]
as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group/7f8a9b2c-4d5e-4f6a-
8b9c-1d2e3f4a5b6c_123")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod =
"PATCH" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group/7f8a9b2c-4d5e-4f6a-8b9c-1d2e3f4a5b6c_123",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "PATCH", CURLOPT_POSTFIELDS => "
{\"request_data\":{\"group_name\":\"test_group_name\",\"role_id\":\"role_name_123\",\"description\":\"Test Updated
Description\",\"users\":[\"test user\"],\"nested_group_ids\":[\"7f3o5b2c-4d5e-4f6a-8b9c-
1d3o5f4a5b6c_123\"],\"idp_groups\":[\"test idp group\"]}}", CURLOPT_HTTPHEADER => [ "content-type: application/json"
], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" .
$err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "PATCH"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group/7f8a9b2c-4d5e-4f6a-8b9c-1d2e3f4a5b6c_123");
struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"group_name\":\"test_group_name\",\"role_id\":\"role_name_123\",\"description\":\"Test Updated
Description\",\"users\":[\"test user\"],\"nested_group_ids\":[\"7f3o5b2c-4d5e-4f6a-8b9c-
1d3o5f4a5b6c_123\"],\"idp_groups\":[\"test idp group\"]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group/7f8a9b2c-4d5e-4f6a-8b9c-
1d2e3f4a5b6c_123"); var request = new RestRequest(Method.PATCH); request.AddHeader("content-type",
"application/json"); request.AddParameter("application/json", "{\"request_data\":
{\"group_name\":\"test_group_name\",\"role_id\":\"role_name_123\",\"description\":\"Test Updated
Description\",\"users\":[\"test user\"],\"nested_group_ids\":[\"7f3o5b2c-4d5e-4f6a-8b9c-
1d3o5f4a5b6c_123\"],\"idp_groups\":[\"test idp group\"]}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

▶ group_namestring

Name of the user group

▶ role_idstring

Unique identifier of the role

▶ descriptionstring

Optional description of the user group

▶ usersarray[string]

List of users for the user group

▶ nested_group_idsarray[string]

List of nested groups

▶ idp_groupsarray[string]

List of idp groups

REQUEST


```
{ "request_data": { "group_name": "test_group_name", "role_id": "role_name_123", "description": "Test Updated Description", "users": [ "test user" ], "nested_group_ids": [ "7f3o5b2c-4d5e-4f6a-8b9c-1d3o5f4a5b6c_123" ], "idp_groups": [ "test idp group" ] } }
```


Responses

User Group edited successfully

Body

application/json

▼ dataobjectrequired

▶ messagestring

Success message indicating the user group was edited

RESPONSE


```
{ "data": { "message": "example" } }
```


Bad request for user group edit call

Body

application/json

▼ dataobjectrequired

▶ err_msgstring

Error message describing the issue

▶ metadataobjectrequired

RESPONSE


```
{ "data": { "err_msg": "example", "metadata": { "err_extra": "example", "err_code": 0 } } }
```


Unauthorized access

Body

application/json

RESPONSE


```
{ "reply": { "err_code": 401, "err_msg": "Public API request unauthorized", "err_extra": null } }
```


Unauthorized access due to lack of sufficient permissions

Body

application/json

▼ replyobjectrequired

▶ err_msgstring

▶ err_extrastring

▶ err_codeinteger

RESPONSE


```
{ "data": { "reply": { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra": "Insufficient permissions for api key" } } }
```


Internal server error. A unified status for API communication type errors.

2.2.3.4 | Delete an existing user group

delete /platform/iam/v1/user-group/{group_id}

Delete an existing user group. Note: upon deletion of a user group, its nested groups are not deleted; the relationships are simply removed.

Path parameters

▼ group_id String required

Full user group id

Example: `7f8a9b2c-4d5e-4f6a-8b9c-1d2e3f4a5b6c_123`

CLIENT REQUEST Bash+curl

```
curl -X 'DELETE'
-H 'Accept: application/json'
'https://api-}/platform/iam/v1/user-group/{group_id}'
```

```
import http.client conn = http.client.HTTPSConnection("api-") conn.request("DELETE",
"%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group/7f8a9b2c-4d5e-4f6a-8b9c-1d2e3f4a5b6c_123") res = conn.getresponse() data
= res.read() print(data.decode("utf-8"))
```

```
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-
group/7f8a9b2c-4d5e-4f6a-8b9c-1d2e3f4a5b6c_123") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Delete.new(url) response = http.request(request)
puts response.read_body
```

```
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("DELETE", "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-
group/7f8a9b2c-4d5e-4f6a-8b9c-1d2e3f4a5b6c_123"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.delete("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group/7f8a9b2c-
4d5e-4f6a-8b9c-1d2e3f4a5b6c_123") .asString();
```

```
import Foundation let request = NSMutableURLRequest(url: NSURL(string:
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group/7f8a9b2c-4d5e-4f6a-8b9c-1d2e3f4a5b6c_123")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "DELETE" let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group/7f8a9b2c-4d5e-4f6a-8b9c-1d2e3f4a5b6c_123",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "DELETE", ]); $response = curl_exec($curl);
$err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "DELETE"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group/7f8a9b2c-4d5e-4f6a-8b9c-1d2e3f4a5b6c_123");
CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user-group/7f8a9b2c-4d5e-4f6a-8b9c-
1d2e3f4a5b6c_123"); var request = new RestRequest(Method.DELETE); IRestResponse response = client.Execute(request);
```

Responses

User Group deleted successfully

Body

application/json

▼ dataobjectrequired

▶ messagestring

Success message indicating the user group was deleted

RESPONSE

```
{ "data": { "message": "example" } }
```

Bad request for user group delete call

Body

application/json

▼ dataobjectrequired

▶ err_msgstring

Error message describing the issue

▶ metadataobjectrequired

RESPONSE

```
{ "data": { "err_msg": "example", "metadata": { "err_extra": "example", "err_code": 0 } } }
```

Unauthorized access

Body

application/json

RESPONSE

```
{ "reply": { "err_code": 401, "err_msg": "Public API request unauthorized", "err_extra": null } }
```

Unauthorized access due to lack of sufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_msgstring
▶ err_extrastring
▶ err_codeinteger
RESPONSE

```
{ "data": { "reply": { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra":
"Insufficient permissions for api key" } } }
```

Internal server error. A unified status for API communication type errors.

2.2.4 | Scopes

Operations for managing Scope-Based Access Control (SBAC) configurations for users, groups, and API keys.

2.2.4.1 | Retrieve an existing scope

get /platform/iam/v1/scope/{entity_type}/{entity_id}

This API endpoint allows retrieving the scope details for a specific entity type and entity ID. For possible 'mode' and 'entity_type' values, please refer to the enum documentation for 'ModeType' and 'EntityType'.

Path parameters
▼ entity_type EntityType required

Full name of the entity's type

▼ entity_id String required

Full id of the entity

Example: `entityId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
'https://api-}/platform/iam/v1/scope/{entity_type}/{entity_id}'
import http.client conn = http.client.HTTPSConnection("api-") conn.request("GET",
"%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D") res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url =
URI("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET",
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D"); xhr.send(data);
HttpResponse<String> response =
Unirest.get("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string:
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D", CURLOPT_RETURNTRANSFER =>
true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D");
var request = new RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```

Responses

Successfully retrieved scope

Body
application/json
▼ metadataobject

Metadata for the response

Free-Form object
RESPONSE
```
{ "metadata": {} }
```

Unauthorized access

Body
application/json
RESPONSE
```
{ "reply": { "err_code": 401, "err_msg": "Public API request unauthorized", "err_extra": null } }
```

Unauthorized access due to lack of sufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_msgstring
▶ err_extrastring
▶ err_codeinteger
RESPONSE
```
{ "data": { "reply": { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra":
"Insufficient permissions for api key" } } }
```

Internal server error. A unified status for API communication type errors.

2.2.4.2 | Edit an existing scope

put /platform/iam/v1/scope/{entity_type}/{entity_id}

This API endpoint allows editing the scope details for a specific entity type and entity ID. For possible 'mode' and 'entity_type' values, please refer to the enum documentation for 'ModeType' and 'EntityType'. Please note that the 'datasets_rows' field can only be passed in to the request body if the dataset scope config is enabled.

Path parameters
▼ entity_type EntityType required

Full name of the entity's type

▼ entity_id String required

Full id of the entity

Example: `entityId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'PUT'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-}/platform/iam/v1/scope/{entity_type}/{entity_id}'
-d '{ "request_data" : { "endpoints" : { "endpoint_groups" : { "names" : [ "test-eg-1" ], "mode" : "scope" },
"endpoint_tags" : { "names" : [ ], "mode" : "any" } }, "cases_issues" : { "mode" : "scope", "names" : [ "Health",
"IT" ] }, "assets" : { "mode" : "scope", "asset_group_ids" : [ 1, 2, 3 ] }, "datasets_rows" : { "filters" : [ {
"dataset" : "test-dataset-1", "filter" : "_collector_name = bu2_collector" }, { "dataset" : "test-dataset-2",
"filter" : "_collector_name = bu2_collector" } ], "default_filter_mode" : "no_scope" } } }'
import http.client conn = http.client.HTTPSConnection("api-") payload = "{\"request_data\":{\"endpoints\":
{\"endpoint_groups\":{\"names\":[\"test-eg-1\"],\"mode\":\"scope\"},\"endpoint_tags\":{\"names\":
[],\"mode\":\"any\"}},\"cases_issues\":{\"mode\":\"scope\",\"names\":[\"Health\",\"IT\"]},\"assets\":
{\"mode\":\"scope\",\"asset_group_ids\":[1,2,3]},\"datasets_rows\":{\"filters\":[{\"dataset\":\"test-dataset-
1\",\"filter\":\"_collector_name = bu2_collector\"},{\"dataset\":\"test-dataset-2\",\"filter\":\"_collector_name =
bu2_collector\"}],\"default_filter_mode\":\"no_scope\"}}}" headers = { 'content-type': "application/json" }
conn.request("PUT", "%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D", payload, headers) res
= conn.getresponse() data = res.read() print(data.decode("utf-8"))
```

```
require 'uri' require 'net/http' require 'openssl' url =
URI("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Put.new(url) request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"endpoints\":{\"endpoint_groups\":{\"names\":[\"test-eg-1\"],\"mode\":\"scope\"},\"endpoint_tags\":{\"names\":
[],\"mode\":\"any\"}},\"cases_issues\":{\"mode\":\"scope\",\"names\":[\"Health\",\"IT\"]},\"assets\":
{\"mode\":\"scope\",\"asset_group_ids\":[1,2,3]},\"datasets_rows\":{\"filters\":[{\"dataset\":\"test-dataset-
1\",\"filter\":\"_collector_name = bu2_collector\"},{\"dataset\":\"test-dataset-2\",\"filter\":\"_collector_name =
bu2_collector\"}],\"default_filter_mode\":\"no_scope\"}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "endpoints": { "endpoint_groups": { "names": [ "test-eg-1" ], "mode":
"scope" }, "endpoint_tags": { "names": [], "mode": "any" } }, "cases_issues": { "mode": "scope", "names": [ "Health",
"IT" ] }, "assets": { "mode": "scope", "asset_group_ids": [ 1, 2, 3 ] }, "datasets_rows": { "filters": [ { "dataset":
"test-dataset-1", "filter": "_collector_name = bu2_collector" }, { "dataset": "test-dataset-2", "filter":
"_collector_name = bu2_collector" } ], "default_filter_mode": "no_scope" } } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("PUT",
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D");
xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response =
Unirest.put("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D")
.header("content-type", "application/json") .body("{\"request_data\":{\"endpoints\":{\"endpoint_groups\":{\"names\":
[\"test-eg-1\"],\"mode\":\"scope\"},\"endpoint_tags\":{\"names\":[],\"mode\":\"any\"}},\"cases_issues\":
{\"mode\":\"scope\",\"names\":[\"Health\",\"IT\"]},\"assets\":{\"mode\":\"scope\",\"asset_group_ids\":
[1,2,3]},\"datasets_rows\":{\"filters\":[{\"dataset\":\"test-dataset-1\",\"filter\":\"_collector_name =
bu2_collector\"},{\"dataset\":\"test-dataset-2\",\"filter\":\"_collector_name =
bu2_collector\"}],\"default_filter_mode\":\"no_scope\"}}}") .asString();
import Foundation let headers = ["content-type": "application/json"] let parameters = ["request_data": [ "endpoints":
[ "endpoint_groups": [ "names": ["test-eg-1"], "mode": "scope" ], "endpoint_tags": [ "names": [], "mode": "any" ] ],
"cases_issues": [ "mode": "scope", "names": ["Health", "IT"] ], "assets": [ "mode": "scope", "asset_group_ids": [1,
2, 3] ], "datasets_rows": [ "filters": [ [ "dataset": "test-dataset-1", "filter": "_collector_name = bu2_collector"
], [ "dataset": "test-dataset-2", "filter": "_collector_name = bu2_collector" ] ], "default_filter_mode": "no_scope"
] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string:
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "PUT" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D", CURLOPT_RETURNTRANSFER =>
true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "PUT", CURLOPT_POSTFIELDS => "{\"request_data\":{\"endpoints\":
{\"endpoint_groups\":{\"names\":[\"test-eg-1\"],\"mode\":\"scope\"},\"endpoint_tags\":{\"names\":
[],\"mode\":\"any\"}},\"cases_issues\":{\"mode\":\"scope\",\"names\":[\"Health\",\"IT\"]},\"assets\":
{\"mode\":\"scope\",\"asset_group_ids\":[1,2,3]},\"datasets_rows\":{\"filters\":[{\"dataset\":\"test-dataset-
1\",\"filter\":\"_collector_name = bu2_collector\"},{\"dataset\":\"test-dataset-2\",\"filter\":\"_collector_name =
bu2_collector\"}],\"default_filter_mode\":\"no_scope\"}}}", CURLOPT_HTTPHEADER => [ "content-type: application/json"
], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" .
$err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "PUT"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D"); struct curl_slist *headers
= NULL; headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"endpoints\":
{\"endpoint_groups\":{\"names\":[\"test-eg-1\"],\"mode\":\"scope\"},\"endpoint_tags\":{\"names\":
[],\"mode\":\"any\"}},\"cases_issues\":{\"mode\":\"scope\",\"names\":[\"Health\",\"IT\"]},\"assets\":
{\"mode\":\"scope\",\"asset_group_ids\":[1,2,3]},\"datasets_rows\":{\"filters\":[{\"dataset\":\"test-dataset-
1\",\"filter\":\"_collector_name = bu2_collector\"},{\"dataset\":\"test-dataset-2\",\"filter\":\"_collector_name =
bu2_collector\"}],\"default_filter_mode\":\"no_scope\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/scope/%7Bentity_type%7D/%7Bentity_id%7D");
var request = new RestRequest(Method.PUT); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"endpoints\":{\"endpoint_groups\":{\"names\":[\"test-
eg-1\"],\"mode\":\"scope\"},\"endpoint_tags\":{\"names\":[],\"mode\":\"any\"}},\"cases_issues\":
{\"mode\":\"scope\",\"names\":[\"Health\",\"IT\"]},\"assets\":{\"mode\":\"scope\",\"asset_group_ids\":
[1,2,3]},\"datasets_rows\":{\"filters\":[{\"dataset\":\"test-dataset-1\",\"filter\":\"_collector_name =
bu2_collector\"},{\"dataset\":\"test-dataset-2\",\"filter\":\"_collector_name =
bu2_collector\"}],\"default_filter_mode\":\"no_scope\"}}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

required
application/json
▼ request_dataobject
▶ tags_scopeobject

Definition of tags scope configuration

▶ tagless_scopeobject

Definition of tagless scope configuration

REQUEST

```
{ "request_data": { "endpoints": { "endpoint_groups": { "names": [ "test-eg-1" ], "mode": "scope" }, "endpoint_tags":
{ "names": [], "mode": "any" } }, "cases_issues": { "mode": "scope", "names": [ "Health", "IT" ] }, "assets": {
"mode": "scope", "asset_group_ids": [ 1, 2, 3 ] }, "datasets_rows": { "filters": [ { "dataset": "test-dataset-1",
"filter": "_collector_name = bu2_collector" }, { "dataset": "test-dataset-2", "filter": "_collector_name =
bu2_collector" } ], "default_filter_mode": "no_scope" } } }
```

Responses

Scope edited successfully

Body
application/json
▼ dataobjectrequired
▶ messagestring

Success message indicating the scope was edited

RESPONSE

```
{ "data": { "message": "example" } }
```


Bad request for scope edit call

Body
application/json
▼ dataobjectrequired
▶ err_msgstring

Error message describing the issue

▶ metadataobjectrequired
RESPONSE

```
{ "data": { "err_msg": "example", "metadata": { "err_extra": "example", "err_code": 0 } } }
```


Unauthorized access

Body
application/json
RESPONSE

```
{ "reply": { "err_code": 401, "err_msg": "Public API request unauthorized", "err_extra": null } }
```


Unauthorized access due to lack of sufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_msgstring
▶ err_extrastring
▶ err_codeinteger
RESPONSE

```
{ "data": { "reply": { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra":
"Insufficient permissions for api key" } } }
```


Internal server error. A unified status for API communication type errors.

2.2.5 | User

Operations for managing users including listing users and editing user configurations.

2.2.5.1 | List all users

get /platform/iam/v1/user

This endpoint retrieves a list of all users and their respective properties. Note: the 'role_name' field refers to the 'pretty_name' attribute from the Roles APIs and the direct role assigned to the user, not roles inherited through groups.

CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
'https://api-}/platform/iam/v1/user'
import http.client conn = http.client.HTTPSConnection("api-") conn.request("GET",
"%7B%7Bfqdn%7D%7D/platform/iam/v1/user") res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user");
xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string:
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "GET" let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error
#:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user"); var request = new
RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```

Responses

Successfully retrieved users

Body
application/json
▼ metadataobject

Metadata for the response

Free-Form object
RESPONSE

```
{ "metadata": {} }
```

Unauthorized access

Body
application/json
RESPONSE

```
{ "reply": { "err_code": 401, "err_msg": "Public API request unauthorized", "err_extra": null } }
```

Unauthorized access due to lack of sufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_msgstring
▶ err_extrastring
▶ err_codeinteger
RESPONSE

```
{ "data": { "reply": { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra":
"Insufficient permissions for api key" } } }
```

Internal server error. A unified status for API communication type errors.

2.2.5.2 | Get user

get /platform/iam/v1/user/{user_email}

This endpoint retrieves a user and its respective properties. Note: the 'role_name' field refers to the 'pretty_name' attribute from the Roles APIs, not the unique role identifier.

Path parameters

▼ user_email String required

Full email of the user

Example: `userEmail_example`

CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
'https://api-}/platform/iam/v1/user/{user_email}'
import http.client conn = http.client.HTTPSConnection("api-") conn.request("GET",
"%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D") res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url =
URI("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) response =
http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET",
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D")
.asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string:
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "GET" let session = URLSession.shared let dataTask =
session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error !=
nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D"); var request = new
RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```

Responses

Successfully retrieved user

Body

application/json

▼ metadataobject

Metadata for the response

Free-Form object

RESPONSE

```
{ "metadata": {} }
```

Unauthorized access

Body

application/json

RESPONSE

```
{ "reply": { "err_code": 401, "err_msg": "Public API request unauthorized", "err_extra": null } }
```

Unauthorized access due to lack of sufficient permissions

Body

application/json

▼ replyobjectrequired

▶ err_msgstring

▶ err_extrastring

▶ err_codeinteger

RESPONSE

```
{ "data": { "reply": { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra":
"Insufficient permissions for api key" } } }
```

Internal server error. A unified status for API communication type errors.

2.2.5.3 | Edit an existing user

patch /platform/iam/v1/user/{user_email}

This endpoint allows updating various user details. Please note that you must pass an empty string for the 'role_id' field if you are attempting to remove a role from a user. For the 'user_groups' field, please pass an empty list to remove user group associations for the provided user.

Path parameters
▼ user_email String required

Full email of the user

Example: `userEmail_example`
CLIENT REQUEST Bash+curl

```
curl -X 'PATCH'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-}/platform/iam/v1/user/{user_email}'
-d '{ "request_data" : { "phone_number" : "408-753-4000", "status" : "Active", "role_id" : "role_name_123",
"is_hidden" : true, "user_groups" : [ "user_groups_123" ] } }'
import http.client conn = http.client.HTTPSConnection("api-") payload = "{\"request_data\":{\"phone_number\":\"408-
753-4000\",\"status\":\"Active\",\"role_id\":\"role_name_123\",\"is_hidden\":true,\"user_groups\":
[\"user_groups_123\"]}}" headers = { 'content-type': "application/json" } conn.request("PATCH",
"%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url =
URI("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Patch.new(url)
request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"phone_number\":\"408-753-
4000\",\"status\":\"Active\",\"role_id\":\"role_name_123\",\"is_hidden\":true,\"user_groups\":
[\"user_groups_123\"]}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "phone_number": "408-753-4000", "status": "Active", "role_id":
"role_name_123", "is_hidden": true, "user_groups": [ "user_groups_123" ] } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("PATCH",
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D"); xhr.setRequestHeader("content-type",
"application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.patch("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D")
.header("content-type", "application/json") .body("{\"request_data\":{\"phone_number\":\"408-753-
4000\",\"status\":\"Active\",\"role_id\":\"role_name_123\",\"is_hidden\":true,\"user_groups\":
[\"user_groups_123\"]}}") .asString();
import Foundation let headers = ["content-type": "application/json"] let parameters = ["request_data": [
"phone_number": "408-753-4000", "status": "Active", "role_id": "role_name_123", "is_hidden": true, "user_groups":
["user_groups_123"] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options:
[]) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "PATCH" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "PATCH", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"phone_number\":\"408-753-
4000\",\"status\":\"Active\",\"role_id\":\"role_name_123\",\"is_hidden\":true,\"user_groups\":
[\"user_groups_123\"]}}", CURLOPT_HTTPHEADER => [ "content-type: application/json" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "PATCH"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"phone_number\":\"408-
753-4000\",\"status\":\"Active\",\"role_id\":\"role_name_123\",\"is_hidden\":true,\"user_groups\":
[\"user_groups_123\"]}}"); CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/user/%7Buser_email%7D"); var request = new RestRequest(Method.PATCH); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{\"request_data\":{\"phone_number\":\"408-753-4000\",\"status\":\"Active\",\"role_id\":\"role_name_123\",\"is_hidden\":true,\"user_groups\":[\"user_groups_123\"]}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
required
application/json
▼ request_dataobject
▶ user_first_namestring

User's first name

▶ user_last_namestring

User's last name

▶ role_idstring

User's assigned role. Note: this refers to the 'role_id' attribute from the Role APIs (the unique role identifier)

▶ phone_numberstring

User's phone number

▶ statusstring

User's status

▶ is_hiddenboolean

User's hidden status

▶ user_groupsarray[string]

User's assigned groups

REQUEST

```
{ "request_data": { "phone_number": "408-753-4000", "status": "Active", "role_id": "role_name_123", "is_hidden": true, "user_groups": [ "user_groups_123" ] } }
```

Responses

User edited successfully

Body
application/json
▼ dataobjectrequired
▶ messagestring

Success message indicating the user was edited

RESPONSE

```
{ "data": { "message": "example" } }
```

Bad request for user edit call

Body
application/json
▼ dataobjectrequired
▶ err_msgstring

Error message describing the issue

▶ metadataobjectrequired

RESPONSE

```
{ "data": { "err_msg": "example", "metadata": { "err_extra": "example", "err_code": 0 } } }
```

Unauthorized access

Body
application/json

RESPONSE

```
{ "reply": { "err_code": 401, "err_msg": "Public API request unauthorized", "err_extra": null } }
```

Unauthorized access due to lack of sufficient permissions

Body

application/json
▼ replyobjectrequired
▶ err_msgstring
▶ err_extrastring
▶ err_codeinteger
RESPONSE
{ "data": { "reply": { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra": "Insufficient permissions for api key" } } }

Internal server error. A unified status for API communication type errors.

2.2.6 | API Keys

Operations for managing API keys including retrieving and editing API key configurations.

2.2.6.1 | Get API Key

get /platform/iam/v1/api-key/{api_key_id}

This endpoint retrieves an API key and its respective properties. Note: the 'roles' field refers to the 'pretty_name' attributes from the Roles APIs, not the unique role identifiers. For possible 'security_level' values, please refer to the enum documentation for 'SecurityLevel'.

Path parameters
▼ api_key_id Integer required

Full API key ID

Example: 56
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
'https://api-}/platform/iam/v1/api-key/{api_key_id}'
import http.client conn = http.client.HTTPSConnection("api-") conn.request("GET",
"%7B%7Bfqdn%7D%7D/platform/iam/v1/api-key/%7Bapi_key_id%7D") res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/api-
key/%7Bapi_key_id%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/api-
key/%7Bapi_key_id%7D"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/api-key/%7Bapi_key_id%7D")
.asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string:
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/api-key/%7Bapi_key_id%7D")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/api-key/%7Bapi_key_id%7D", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/api-key/%7Bapi_key_id%7D"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/api-key/%7Bapi_key_id%7D"); var request =
new RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```
Responses

Successfully retrieved API key

Body
application/json
▼ metadataobject

Metadata for the response

Free-Form object

RESPONSE
```
{ "metadata": {} }
```

Bad request for api key get call

Body
application/json
▼ dataobjectrequired
▶ err_msgstring

Error message describing the issue

▶ metadataobjectrequired
RESPONSE
```
{ "data": { "err_msg": "example", "metadata": { "err_extra": "example", "err_code": 0 } } }
```

Unauthorized access

Body
application/json
RESPONSE
```
{ "reply": { "err_code": 401, "err_msg": "Public API request unauthorized", "err_extra": null } }
```

Unauthorized access due to lack of sufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_msgstring
▶ err_extrastring
▶ err_codeinteger
RESPONSE
```
{ "data": { "reply": { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra":
"Insufficient permissions for api key" } } }
```

Internal server error. A unified status for API communication type errors.

2.2.6.2 | Edit an API key

put /platform/iam/v1/api-key/{api_key_id}

This endpoint allows for the editing of an existing API key. For possible 'security_level' values, please refer to the enum documentation for 'SecurityLevel'.

Path parameters
▼ api_key_id Integer required

Full API key ID

Example: 56
CLIENT REQUEST Bash+curl
```
curl -X 'PUT'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-}/platform/iam/v1/api-key/{api_key_id}'
-d '{ "request_data" : { "roles" : [ "role_name_123" ], "security_level" : "standard", "comment" : "useful comment" }
}'
import http.client conn = http.client.HTTPSConnection("api-") payload = "{\"request_data\":{\"roles\":
[\"role_name_123\"],\"security_level\":\"standard\",\"comment\":\"useful comment\"}}" headers = { 'content-type':
"application/json" } conn.request("PUT", "%7B%7Bfqdn%7D%7D/platform/iam/v1/api-key/%7Bapi_key_id%7D", payload,
headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/api-
key/%7Bapi_key_id%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Put.new(url) request["content-type"] = 'application/json' request.body
= "{\"request_data\":{\"roles\":[\"role_name_123\"],\"security_level\":\"standard\",\"comment\":\"useful comment\"}}"
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "roles": [ "role_name_123" ], "security_level": "standard",
"comment": "useful comment" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("PUT", "https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/api-
key/%7Bapi_key_id%7D"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.put("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/api-key/%7Bapi_key_id%7D")
.header("content-type", "application/json") .body("{\"request_data\":{\"roles\":
[\"role_name_123\"],\"security_level\":\"standard\",\"comment\":\"useful comment\"}}") .asString();
import Foundation let headers = ["content-type": "application/json"] let parameters = ["request_data": [ "roles":
["role_name_123"], "security_level": "standard", "comment": "useful comment" ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/api-key/%7Bapi_key_id%7D")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "PUT" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL =>
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/api-key/%7Bapi_key_id%7D", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "PUT", CURLOPT_POSTFIELDS => "{\"request_data\":{\"roles\":
[\"role_name_123\"],\"security_level\":\"standard\",\"comment\":\"useful comment\"}}", CURLOPT_HTTPHEADER => [
"content-type: application/json" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if
($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "PUT"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/api-key/%7Bapi_key_id%7D"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"roles\":
[\"role_name_123\"],\"security_level\":\"standard\",\"comment\":\"useful comment\"}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-/%7B%7Bfqdn%7D%7D/platform/iam/v1/api-key/%7Bapi_key_id%7D"); var request =
new RestRequest(Method.PUT); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"roles\":
[\"role_name_123\"],\"security_level\":\"standard\",\"comment\":\"useful comment\"}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ rolesarray[string]

API Key's assigned roles. Note: this refers to the 'role_id' attributes from the Role APIs (the unique role identifiers)

▶ security_levelstring

API Key's security level

▶ commentstring

API Key's comment

REQUEST

```
{ "request_data": { "roles": [ "role_name_123" ], "security_level": "standard", "comment": "useful comment" } }
```

Responses

API key edited successfully

Body

application/json

▼ dataobjectrequired

▶ messagestring

Success message indicating the API key was edited

RESPONSE

```
{ "data": { "message": "example" } }
```

Bad request for api key edit call

Body

application/json

▼ dataobjectrequired

▶ err_msgstring

Error message describing the issue

▶ metadataobjectrequired

RESPONSE

```
{ "data": { "err_msg": "example", "metadata": { "err_extra": "example", "err_code": 0 } } }
```

Unauthorized access

Body
application/json
RESPONSE
{ "reply": { "err_code": 401, "err_msg": "Public API request unauthorized", "err_extra": null } }

Unauthorized access due to lack of sufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_msgstring
▶ err_extrastring
▶ err_codeinteger
RESPONSE
{ "data": { "reply": { "err_code": 403, "err_msg": "Forbidden. Access was denied to this resource.", "err_extra": "Insufficient permissions for api key" } } }

Internal server error. A unified status for API communication type errors.

2.3 | Cloud Onboarding APIs

2.3.1 | Preface

You can export the OpenAPI specs in the Cortex Stoplight instance.

Public APIs for cloud onboarding and instance management.

The following API categories are included:

- Cloud integration instance management: These APIs enable you to create an integration instance in *pending* state and provide a link to download the authentication template to be executed in your CSP. You can get details on the integration instances currently in Cortex Cloud and you can enable, disable, or delete them. You can also edit the integration instance configuration.
- Cloud account management: The cloud account APIs allow you to manage accounts within a specific integration instance. You can have multiple cloud accounts within one integration instance. Use these APIs to get the cloud accounts in the specified integration instance and to enable or disable cloud accounts.
- Outpost management: Use these APIs to create or edit templates for onboarding outposts.
- General: These APIs are for general management.

Required license: Cortex Cloud Posture Management or Cortex Cloud Runtime Security.

Version: Cortex Cloud 1.3

2.3.1.1 | Servers

https://api-yourfqdn

2.3.1.2 | Authentication methods

▼ Authorization
Api Key "x-api-key"

Authorized requests to the API should use: x-api-key inside a header with the access token.

▼ x-xdr-auth-id
Api Key "x-api-key-id"

Authorized requests to the API should use: x-api-key-id inside a header with the access token.

2.3.2 | Cloud integration instance management

APIs for managing cloud integration instances

2.3.2.1 | Create a cloud onboarding integration template

post /public_api/v1/cloud_onboarding/create_instance_template

Create a template to facilitate the seamless setup of CSP data in Cortex. Define the scope and specify the scan mode. For full control of the setup, you can use the advanced settings. Based on the onboarding settings, Cortex generates an authentication template to establish trust to the CSP and grant permissions to Cortex. Cortex Cloud creates an instance in *pending* state. The API returns a link to download the template from the Cortex tenant.

Note: If the template execution in the CSP fails, there is currently no manual execution of the template by API. You must perform manual execution in the UI.

For scope modifications, the parameters are CSP-specific. For AWS, use `accounts`, for GCP, use `projects`, and for Azure, use `subscriptions`.

Required license: Cortex Cloud Posture Management
Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cloud_onboarding/create_instance_template'
-d '{ "request_data" : { "scope" : "ORGANIZATION", "scan_mode" : "MANAGED", "cloud_provider" : "AWS", "instance_name"
: "My AWS Instance", "custom_resources_tags" : [ { "key" : "environment", "value" : "production" } ],
"collection_configuration" : { "audit_logs" : { "enabled" : true } }, "scope_modifications" : { "accounts" : {
"enabled" : true, "type" : "INCLUDE", "account_ids" : [ "123456789012" ] }, "regions" : { "enabled" : true, "type" :
"INCLUDE", "regions" : [ "us-east-1", "us-west-2" ] } }, "additional_capabilities" : { "xsiam_analytics" : true,
"data_security_posture_management" : true, "registry_scanning" : true, "registry_scanning_options" : { "type" : "ECR"
}, "serverless_scanning" : true, "agentless_disk_scanning" : true } } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"scope\":\"ORGANIZATION\",\"scan_mode\":\"MANAGED\",\"cloud_provider\":\"AWS\",\"instance_name\":\"My AWS
Instance\",\"custom_resources_tags\":
[{\"key\":\"environment\",\"value\":\"production\"}],\"collection_configuration\":{\"audit_logs\":
{\"enabled\":true}},\"scope_modifications\":{\"accounts\":{\"enabled\":true,\"type\":\"INCLUDE\",\"account_ids\":
[\"123456789012\"]},\"regions\":{\"enabled\":true,\"type\":\"INCLUDE\",\"regions\":[\"us-east-1\",\"us-west-
2\"]}},\"additional_capabilities\":
{\"xsiam_analytics\":true,\"data_security_posture_management\":true,\"registry_scanning\":true,\"registry_scanning_options\":
{\"type\":\"ECR\"},\"serverless_scanning\":true,\"agentless_disk_scanning\":true}}}" headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/cloud_onboarding/create_instance_template", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cloud_onboarding/create_instance_template") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url)
request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"scope\":\"ORGANIZATION\",\"scan_mode\":\"MANAGED\",\"cloud_provider\":\"AWS\",\"instance_name\":\"My AWS
Instance\",\"custom_resources_tags\":
[{\"key\":\"environment\",\"value\":\"production\"}],\"collection_configuration\":{\"audit_logs\":
{\"enabled\":true}},\"scope_modifications\":{\"accounts\":{\"enabled\":true,\"type\":\"INCLUDE\",\"account_ids\":
[\"123456789012\"]},\"regions\":{\"enabled\":true,\"type\":\"INCLUDE\",\"regions\":[\"us-east-1\",\"us-west-
2\"]}},\"additional_capabilities\":
{\"xsiam_analytics\":true,\"data_security_posture_management\":true,\"registry_scanning\":true,\"registry_scanning_options\":
{\"type\":\"ECR\"},\"serverless_scanning\":true,\"agentless_disk_scanning\":true}}}" response = http.request(request)
puts response.read_body
const data = JSON.stringify({ "request_data": { "scope": "ORGANIZATION", "scan_mode": "MANAGED", "cloud_provider":
"AWS", "instance_name": "My AWS Instance", "custom_resources_tags": [ { "key": "environment", "value": "production" }
], "collection_configuration": { "audit_logs": { "enabled": true } }, "scope_modifications": { "accounts": {
"enabled": true, "type": "INCLUDE", "account_ids": [ "123456789012" ] }, "regions": { "enabled": true, "type":
"INCLUDE", "regions": [ "us-east-1", "us-west-2" ] } }, "additional_capabilities": { "xsiam_analytics": true,
"data_security_posture_management": true, "registry_scanning": true, "registry_scanning_options": { "type": "ECR" },
"serverless_scanning": true, "agentless_disk_scanning": true } } }); const xhr = new XMLHttpRequest();
```

```
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/cloud_onboarding/create_instance_template"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-
yourfqdn/public_api/v1/cloud_onboarding/create_instance_template") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"request_data\":
{\"scope\":\"ORGANIZATION\",\"scan_mode\":\"MANAGED\",\"cloud_provider\":\"AWS\",\"instance_name\":\"My AWS
Instance\",\"custom_resources_tags\":
[{\"key\":\"environment\",\"value\":\"production\"}],\"collection_configuration\":{\"audit_logs\":
{\"enabled\":true}},\"scope_modifications\":{\"accounts\":{\"enabled\":true,\"type\":\"INCLUDE\",\"account_ids\":
[\"123456789012\"]},\"regions\":{\"enabled\":true,\"type\":\"INCLUDE\",\"regions\":[\"us-east-1\",\"us-west-
2\"]}},\"additional_capabilities\":
{\"xsiam_analytics\":true,\"data_security_posture_management\":true,\"registry_scanning\":true,\"registry_scanning_options\":
{\"type\":\"ECR\"},\"serverless_scanning\":true,\"agentless_disk_scanning\":true}}}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = ["request_data": [ "scope":
"ORGANIZATION", "scan_mode": "MANAGED", "cloud_provider": "AWS", "instance_name": "My AWS Instance",
"custom_resources_tags": [ [ "key": "environment", "value": "production" ] ], "collection_configuration":
["audit_logs": ["enabled": true]], "scope_modifications": [ "accounts": [ "enabled": true, "type": "INCLUDE",
"account_ids": ["123456789012"] ], "regions": [ "enabled": true, "type": "INCLUDE", "regions": ["us-east-1", "us-
west-2"] ] ], "additional_capabilities": [ "xsiam_analytics": true, "data_security_posture_management": true,
"registry_scanning": true, "registry_scanning_options": ["type": "ECR"], "serverless_scanning": true,
"agentless_disk_scanning": true ] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject:
parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/cloud_onboarding/create_instance_template")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cloud_onboarding/create_instance_template", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING
=> "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"scope\":\"ORGANIZATION\",\"scan_mode\":\"MANAGED\",\"cloud_provider\":\"AWS\",\"instance_name\":\"My AWS
Instance\",\"custom_resources_tags\":
[{\"key\":\"environment\",\"value\":\"production\"}],\"collection_configuration\":{\"audit_logs\":
{\"enabled\":true}},\"scope_modifications\":{\"accounts\":{\"enabled\":true,\"type\":\"INCLUDE\",\"account_ids\":
[\"123456789012\"]},\"regions\":{\"enabled\":true,\"type\":\"INCLUDE\",\"regions\":[\"us-east-1\",\"us-west-
2\"]}},\"additional_capabilities\":
{\"xsiam_analytics\":true,\"data_security_posture_management\":true,\"registry_scanning\":true,\"registry_scanning_options\":
{\"type\":\"ECR\"},\"serverless_scanning\":true,\"agentless_disk_scanning\":true}}}", CURLOPT_HTTPHEADER => [
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cloud_onboarding/create_instance_template"); struct curl_slist
*headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":
{\"scope\":\"ORGANIZATION\",\"scan_mode\":\"MANAGED\",\"cloud_provider\":\"AWS\",\"instance_name\":\"My AWS
Instance\",\"custom_resources_tags\":
[{\"key\":\"environment\",\"value\":\"production\"}],\"collection_configuration\":{\"audit_logs\":
{\"enabled\":true}},\"scope_modifications\":{\"accounts\":{\"enabled\":true,\"type\":\"INCLUDE\",\"account_ids\":
[\"123456789012\"]},\"regions\":{\"enabled\":true,\"type\":\"INCLUDE\",\"regions\":[\"us-east-1\",\"us-west-
2\"]}},\"additional_capabilities\":
{\"xsiam_analytics\":true,\"data_security_posture_management\":true,\"registry_scanning\":true,\"registry_scanning_options\":
{\"type\":\"ECR\"},\"serverless_scanning\":true,\"agentless_disk_scanning\":true}}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cloud_onboarding/create_instance_template"); var
request = new RestRequest(Method.POST); request.AddHeader("Authorization",
```

```
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":
{\"scope\":\"ORGANIZATION\",\"scan_mode\":\"MANAGED\",\"cloud_provider\":\"AWS\",\"instance_name\":\"My AWS
Instance\",\"custom_resources_tags\":
[{\"key\":\"environment\",\"value\":\"production\"}],\"collection_configuration\":{\"audit_logs\":
{\"enabled\":true}},\"scope_modifications\":{\"accounts\":{\"enabled\":true,\"type\":\"INCLUDE\",\"account_ids\":
[\"123456789012\"]},\"regions\":{\"enabled\":true,\"type\":\"INCLUDE\",\"regions\":[\"us-east-1\",\"us-west-
2\"]}},\"additional_capabilities\":
{\"xsiam_analytics\":true,\"data_security_posture_management\":true,\"registry_scanning\":true,\"registry_scanning_options\":
{\"type\":\"ECR\"},\"serverless_scanning\":true,\"agentless_disk_scanning\":true}}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters
required
application/json

▼ request_dataobjectrequired

▶ scopestring (Enum)

Define the scope for this integration instance. Valid values include:

- `ORGANIZATION`: Use this to indicate an AWS organization, GCP organization, or Azure tenant.
- `ACCOUNT_GROUP`: Use this to indicate an AWS account group, GCP folder, or Azure management group.
- `ACCOUNT`: Use this to indicate an AWS account, GCP project, or Azure subscription.

▶ scan_modestring (Enum)

Define the scan mode. Valid values include:

- `MANAGED`: (Recommended) Security scanning is performed in the Cortex cloud environment.
- `OUTPOST`: Security scanning is performed on infrastructure deployed to a cloud account owned by you.

**Note: ** Scanning with an outpost may require additional CSP permissions and may incur additional CSP costs.

▶ scan_env_idstring

When the scan mode is `OUTPOST`, you must include the `scan_env_id` field with the outpost ID as the value. To obtain the outpost ID, call `/public_api/v1/cloud_onboarding/get_outposts`. The value of `outpost_id` in the response is the outpost ID.

▶ account_detailsobject

`account_details` is required for onboarding Azure and is not used for onboarding AWS or GCP.

▶ instance_namestring

▶ cloud_providerstring (Enum)

The cloud service provider. Valid values include:

- `AWS`
- `AZURE`
- `GCP`
- `OCI`

▶ custom_resources_tagsarray

▶ collection_configurationobject

▶ scope_modificationsobject

▶ additional_capabilitiesobjectrequired

Define which security capabilities you want to benefit from.

Note: Adding security capability typically requires additional cloud provider permissions. For detailed information on the permissions required, see Cloud service provider permissions.

REQUEST AWS, Managed scan, region excluded, audit logs enabled with custom collection

```
{ "request_data": { "scope": "ACCOUNT", "scan_mode": "MANAGED", "instance_name": "AWS 1 Instance",
"scope_modifications": { "regions": { "enabled": true, "type": "EXCLUDE", "regions": [ "us-east-1" ] } },
"additional_capabilities": { "xsiam_analytics": false, "data_security_posture_management": true, "registry_scanning":
false, "serverless_scanning": true, "agentless_disk_scanning": true }, "custom_resources_tags": [ { "key":
"managed_by", "value": "paloaltonetworks" } ], "collection_configuration": { "audit_logs": { "enabled": true,
"collection_method": "AUTOMATED" } }, "cloud_provider": "AWS" } }
{ "request_data": { "instance_name": "test-oci-instance-01", "id": "2867753s3ac43ee094f320baa4751efe", "scan_mode":
"OUTPOST", "scan_env_id": "acafb912391c44f59a01b6512317abf9", "cloud_provider": "OCI", "collection_configuration": {
"audit_logs": { "enabled": true, "collection_method": "CUSTOM", "custom_collectors": [ { "region": "us-phoenix-1",
"bucket_name": "prod-audit-logs-bucket", "compartment_ocid": "ocid1.compartment.oc1..xxxxxxxxxxxxxx" }, { "region":
```

"us-ashburn-1", "bucket_name": "dr-audit-logs-bucket", "compartment_ocid": "ocid1.compartment.oc1..yyyyyyyyyyyyyy" }
] } } } }
Responses

Successful Response

Body
application/json
▼ replyobject

The main payload of the response, containing instance information data. It includes a link to download the template file from your tenant. If you use automated upload (only for AWS), the link will take you directly to AWS CloudFormation to create the stack. Otherwise, download the template and upload it in your CSP and follow the instructions to manually upload the template:

- AWS
- GCP
- Azure ARM
- Azure CLI

RESPONSE
{ "reply": { "automated": { "link": "string", "tracking_guid": "string" }, "manual": { "TF/ARM": "string" } } }

Invalid Client Request Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }

Internal Server Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }

2.3.2.2 | Get integration instance details

post /public_api/v1/cloud_onboarding/get_instance_details

Get the configuration details of the specified integration instance.

Required license: Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cloud_onboarding/get_instance_details'
-d '{ "request_data" : { "instance_id" : "instance-1" } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"instance_id\":\"instance-1\"}}" headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/cloud_onboarding/get_instance_details", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_instance_details") http = Net::HTTP.new(url.host, url.port) http.use_ssl
= true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"instance_id\":\"instance-1\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "instance_id": "instance-1" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_instance_details"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_instance_details") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"request_data\":
{\"instance_id\":\"instance-1\"}}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = ["request_data": ["instance_id":
"instance-1"]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let
request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_instance_details")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_instance_details", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING =>
"", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"instance_id\":\"instance-1\"}}",
CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cloud_onboarding/get_instance_details"); struct curl_slist *headers
= NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":{\"instance_id\":\"instance-1\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cloud_onboarding/get_instance_details"); var request
= new RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
```

```
request.AddParameter("application/json", "{\"request_data\":{\"instance_id\":\"instance-1\"}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

▶ instance_idstring

The ID of the integration instance of which you want to get the details.

REQUEST

```
{ "request_data": { "instance_id": "instance-1" } }
```

Responses

Successful Response

Body

application/json

▼ replyobject

The main payload of the response, containing integration instance details.

RESPONSE

```
{ "reply": { "id": "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx", "collector": "AZURE", "instance_name": "instance-1", "scope":
"ORGANIZATION", "tags": [ { "key": "managed_by", "value": "paloaltonetworks" } ], "scan": { "scan_method": "MANAGED"
}, "security_capabilities": [ { "name": "PERMISSIONS", "description": "Permissions", "status": 0 }, { "name":
"DISCOVERY", "description": "Discovery Engine", "status": 4 }, { "name": "ADS", "description": "Agentless Disk
Scanning", "status": 4 }, { "name": "DSPM", "description": "Data Security Scanning", "status": 4 }, { "name":
"AUDIT_LOGS", "description": "Audit Logs", "status": 1 }, { "name": "REGISTRY", "description": "Registry Scanning",
"status": 2 }, { "name": "SERVERLESS", "description": "Serverless Scanning", "status": 2 }, { "name":
"IDENTITY_SECURITY", "description": "Identity Security", "status": 4 } ], "status": "ENABLED", "cloud_provider":
"AZURE", "collection_configuration": "{\"audit_logs\": {\"enabled\": true}}", "additional_capabilities": "
{\"xsiam_analytics\": true, \"registry_scanning\": true, \"serverless_scanning\":\ntrue,
\"registry_scanning_options\": {\"type\": \"ALL\"}, \"data_security_posture_management\":\ntrue
\"agentless_disk_scanning\":\ntrue,}" } }
```

Invalid Client Request Error

Body

application/json

▼ replyobject

▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE

```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

Internal Server Error

Body

application/json

▼ replyobject

▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE

```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

2.3.2.3 | Get all or filtered integration instances

post /public_api/v1/cloud_onboarding/get_instances

Get the configuration details of all or filtered integration instances. This does not include instances in *pending* state.

Required license: Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cloud_onboarding/get_instances'
-d '{ "request_data" : { "filter_data" : { "sort" : [ { "FIELD" : "STATUS", "ORDER" : "DESC" } ], "paging" : { "from"
: 0, "to" : 50 }, "filter" : { "AND" : [ { "SEARCH_FIELD" : "CLOUD_PROVIDER", "SEARCH_TYPE" : "EQ", "SEARCH_VALUE" :
"AWS" } ] } } } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filter_data\":
{\"sort\":[{\"FIELD\":\"STATUS\",\"ORDER\":\"DESC\"}],\"paging\":{\"from\":0,\"to\":50},\"filter\":{\"AND\":
[{\"SEARCH_FIELD\":\"CLOUD_PROVIDER\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"AWS\"}]}}}}" headers = {
'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/cloud_onboarding/get_instances", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_instances") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"filter_data\":{\"sort\":[{\"FIELD\":\"STATUS\",\"ORDER\":\"DESC\"}],\"paging\":{\"from\":0,\"to\":50},\"filter\":
{\"AND\":[{\"SEARCH_FIELD\":\"CLOUD_PROVIDER\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"AWS\"}]}}}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filter_data": { "sort": [ { "FIELD": "STATUS", "ORDER": "DESC" } ],
"paging": { "from": 0, "to": 50 }, "filter": { "AND": [ { "SEARCH_FIELD": "CLOUD_PROVIDER", "SEARCH_TYPE": "EQ",
"SEARCH_VALUE": "AWS" } ] } } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_instances"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/cloud_onboarding/get_instances")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"request_data\":
{\"filter_data\":{\"sort\":[{\"FIELD\":\"STATUS\",\"ORDER\":\"DESC\"}],\"paging\":{\"from\":0,\"to\":50},\"filter\":
{\"AND\":[{\"SEARCH_FIELD\":\"CLOUD_PROVIDER\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"AWS\"}]}}}}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = ["request_data": ["filter_data": [
"sort": [ [ "FIELD": "STATUS", "ORDER": "DESC" ] ], "paging": [ "from": 0, "to": 50 ], "filter": ["AND": [ [
"SEARCH_FIELD": "CLOUD_PROVIDER", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "AWS" ] ] ]]] as [String : Any] let postData
= JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/cloud_onboarding/get_instances")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
```

```
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_instances", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filter_data\":{\"sort\":
[{\"FIELD\":\"STATUS\",\"ORDER\":\"DESC\"}],\"paging\":{\"from\":0,\"to\":50},\"filter\":{\"AND\":
[{\"SEARCH_FIELD\":\"CLOUD_PROVIDER\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"AWS\"}]}}}}", CURLOPT_HTTPHEADER => [
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cloud_onboarding/get_instances"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":{\"filter_data\":{\"sort\":[{\"FIELD\":\"STATUS\",\"ORDER\":\"DESC\"}],\"paging\":
{\"from\":0,\"to\":50},\"filter\":{\"AND\":
[{\"SEARCH_FIELD\":\"CLOUD_PROVIDER\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"AWS\"}]}}}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cloud_onboarding/get_instances"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"filter_data\":{\"sort\":
[{\"FIELD\":\"STATUS\",\"ORDER\":\"DESC\"}],\"paging\":{\"from\":0,\"to\":50},\"filter\":{\"AND\":
[{\"SEARCH_FIELD\":\"CLOUD_PROVIDER\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"AWS\"}]}}}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

Valid values for the sort field `FIELD` the filter field `SEARCH_FIELD` include:

- `STATUS`
- `PROVIDER`
- `INSTANCE_NAME`
- `SCOPE`
- `SCAN_MODE`
- `CREATION_TIME`
- `OUTPOST_ID`
- `AUTHENTICATION_METHOD`
- `ID`

▶ filter_dataobject

REQUEST

```
{ "request_data": { "filter_data": { "sort": [ { "FIELD": "STATUS", "ORDER": "DESC" } ], "paging": { "from": 0, "to":
50 }, "filter": { "AND": [ { "SEARCH_FIELD": "CLOUD_PROVIDER", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "AWS" } ] } } } }
```

Responses

Successful Response

Body

application/json

▼ replyobject

The main payload of the response, containing integration instance details.

RESPONSE

```
{ "reply": { "DATA": [ { "instance_name": "xyz-instance", "cloud_provider": "AZURE", "scope": "ORGANIZATION",
"scan_mode": "MANAGED", "custom_resources_tags": [ { "key": "managed_by", "value": "paloaltonetworks" } ],
"provisioning_method": "ARM", "account_details": { "account_id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
"csp_org_id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx", "account_name": "account-1", "account_group": "xxxxxxxx-xxxx-
xxxx-xxxx-xxxxxxxxxxxx", "organization_id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx" }, "scope_modifications": {
"regions": { "enabled": false }, "subscriptions": { "enabled": false } }, "additional_capabilities": {
"xsiam_analytics": true, "data_security_posture_management": true, "registry_scanning": true,
"registry_scanning_options": { "type": "ECR" }, "serverless_scanning": true, "agentless_disk_scanning": true },
"collection_configuration": { "audit_logs": { "enabled": true } }, "instance_id": "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
```

"status": "DELETED", "cloud_partition": "COMMERCIAL", "created_at": 1741268418, "modified_at": 1741626494,
"deleted_at": 1741626494, "default_scanning_scope": { "registry_scanning_scope": { "enabled": false },
"agentless_disk_scanning_scope": { "enabled": false }, "data_assets_classification_options": {} }, "outpost_id":
"xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx" }], "FILTER_COUNT": 1, "TOTAL_COUNT": 33 } }

Invalid Client Request Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }

Internal Server Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }

2.3.2.4 | Edit an integration instance

post /public_api/v1/cloud_onboarding/edit_instance

Edit the configuration settings of the specified integration instance. After editing, there is a notice in the UI that there are pending changes. The changes are applied after you execute the template in the CSP after editing.

Required license: Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cloud_onboarding/edit_instance'
```

```
-d '{ "request_data" : { "instance_id" : "instance-1", "scan_env_id" : "env-1", "instance_name" : "Updated AWS
Instance", "cloud_provider" : "AWS", "custom_resources_tags" : [ { "key" : "environment", "value" : "staging" } ],
"collection_configuration" : { "audit_logs" : { "enabled" : true } }, "additional_capabilities" : { "xsiam_analytics"
: true, "data_security_posture_management" : true, "registry_scanning" : true, "registry_scanning_options" : { "type"
: "ECR" }, "serverless_scanning" : true, "agentless_disk_scanning" : true }, "scope_modifications" : { "accounts" : {
"enabled" : true, "type" : "INCLUDE", "account_ids" : [ "123456789012", "210987654321" ] }, "regions" : { "enabled" :
true, "type" : "EXCLUDE", "regions" : [ "us-east-1" ] } } } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"instance_id\":\"instance-1\",\"scan_env_id\":\"env-1\",\"instance_name\":\"Updated AWS
Instance\",\"cloud_provider\":\"AWS\",\"custom_resources_tags\":
[{\"key\":\"environment\",\"value\":\"staging\"}],\"collection_configuration\":{\"audit_logs\":
{\"enabled\":true}},\"additional_capabilities\":
{\"xsiam_analytics\":true,\"data_security_posture_management\":true,\"registry_scanning\":true,\"registry_scanning_options\":
{\"type\":\"ECR\"},\"serverless_scanning\":true,\"agentless_disk_scanning\":true},\"scope_modifications\":
{\"accounts\":{\"enabled\":true,\"type\":\"INCLUDE\",\"account_ids\":
[\"123456789012\",\"210987654321\"]},\"regions\":{\"enabled\":true,\"type\":\"EXCLUDE\",\"regions\":[\"us-east-
1\"]}}}}" headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/cloud_onboarding/edit_instance", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cloud_onboarding/edit_instance") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"instance_id\":\"instance-1\",\"scan_env_id\":\"env-1\",\"instance_name\":\"Updated AWS
Instance\",\"cloud_provider\":\"AWS\",\"custom_resources_tags\":
[{\"key\":\"environment\",\"value\":\"staging\"}],\"collection_configuration\":{\"audit_logs\":
{\"enabled\":true}},\"additional_capabilities\":
{\"xsiam_analytics\":true,\"data_security_posture_management\":true,\"registry_scanning\":true,\"registry_scanning_options\":
{\"type\":\"ECR\"},\"serverless_scanning\":true,\"agentless_disk_scanning\":true},\"scope_modifications\":
{\"accounts\":{\"enabled\":true,\"type\":\"INCLUDE\",\"account_ids\":
[\"123456789012\",\"210987654321\"]},\"regions\":{\"enabled\":true,\"type\":\"EXCLUDE\",\"regions\":[\"us-east-
1\"]}}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "instance_id": "instance-1", "scan_env_id": "env-1", "instance_name":
"Updated AWS Instance", "cloud_provider": "AWS", "custom_resources_tags": [ { "key": "environment", "value":
"staging" } ], "collection_configuration": { "audit_logs": { "enabled": true } }, "additional_capabilities": {
"xsiam_analytics": true, "data_security_posture_management": true, "registry_scanning": true,
"registry_scanning_options": { "type": "ECR" }, "serverless_scanning": true, "agentless_disk_scanning": true },
"scope_modifications": { "accounts": { "enabled": true, "type": "INCLUDE", "account_ids": [ "123456789012",
"210987654321" ] }, "regions": { "enabled": true, "type": "EXCLUDE", "regions": [ "us-east-1" ] } } } }); const xhr =
new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/cloud_onboarding/edit_instance"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/cloud_onboarding/edit_instance")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"request_data\":
{\"instance_id\":\"instance-1\",\"scan_env_id\":\"env-1\",\"instance_name\":\"Updated AWS
Instance\",\"cloud_provider\":\"AWS\",\"custom_resources_tags\":
[{\"key\":\"environment\",\"value\":\"staging\"}],\"collection_configuration\":{\"audit_logs\":
{\"enabled\":true}},\"additional_capabilities\":
{\"xsiam_analytics\":true,\"data_security_posture_management\":true,\"registry_scanning\":true,\"registry_scanning_options\":
{\"type\":\"ECR\"},\"serverless_scanning\":true,\"agentless_disk_scanning\":true},\"scope_modifications\":
{\"accounts\":{\"enabled\":true,\"type\":\"INCLUDE\",\"account_ids\":
[\"123456789012\",\"210987654321\"]},\"regions\":{\"enabled\":true,\"type\":\"EXCLUDE\",\"regions\":[\"us-east-
1\"]}}}}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = ["request_data": [ "instance_id":
"instance-1", "scan_env_id": "env-1", "instance_name": "Updated AWS Instance", "cloud_provider": "AWS",
"custom_resources_tags": [ [ "key": "environment", "value": "staging" ] ], "collection_configuration": ["audit_logs":
["enabled": true]], "additional_capabilities": [ "xsiam_analytics": true, "data_security_posture_management": true,
"registry_scanning": true, "registry_scanning_options": ["type": "ECR"], "serverless_scanning": true,
```

```
"agentless_disk_scanning": true ], "scope_modifications": [ "accounts": [ "enabled": true, "type": "INCLUDE",
"account_ids": ["123456789012", "210987654321"] ], "regions": [ "enabled": true, "type": "EXCLUDE", "regions": ["us-
east-1"] ] ] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let
request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/cloud_onboarding/edit_instance")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cloud_onboarding/edit_instance", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"instance_id\":\"instance-1\",\"scan_env_id\":\"env-
1\",\"instance_name\":\"Updated AWS Instance\",\"cloud_provider\":\"AWS\",\"custom_resources_tags\":
[{\"key\":\"environment\",\"value\":\"staging\"}],\"collection_configuration\":{\"audit_logs\":
{\"enabled\":true}},\"additional_capabilities\":
{\"xsiam_analytics\":true,\"data_security_posture_management\":true,\"registry_scanning\":true,\"registry_scanning_options\":
{\"type\":\"ECR\"},\"serverless_scanning\":true,\"agentless_disk_scanning\":true},\"scope_modifications\":
{\"accounts\":{\"enabled\":true,\"type\":\"INCLUDE\",\"account_ids\":
[\"123456789012\",\"210987654321\"]},\"regions\":{\"enabled\":true,\"type\":\"EXCLUDE\",\"regions\":[\"us-east-
1\"]}}}}", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cloud_onboarding/edit_instance"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":{\"instance_id\":\"instance-1\",\"scan_env_id\":\"env-1\",\"instance_name\":\"Updated AWS
Instance\",\"cloud_provider\":\"AWS\",\"custom_resources_tags\":
[{\"key\":\"environment\",\"value\":\"staging\"}],\"collection_configuration\":{\"audit_logs\":
{\"enabled\":true}},\"additional_capabilities\":
{\"xsiam_analytics\":true,\"data_security_posture_management\":true,\"registry_scanning\":true,\"registry_scanning_options\":
{\"type\":\"ECR\"},\"serverless_scanning\":true,\"agentless_disk_scanning\":true},\"scope_modifications\":
{\"accounts\":{\"enabled\":true,\"type\":\"INCLUDE\",\"account_ids\":
[\"123456789012\",\"210987654321\"]},\"regions\":{\"enabled\":true,\"type\":\"EXCLUDE\",\"regions\":[\"us-east-
1\"]}}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cloud_onboarding/edit_instance"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"instance_id\":\"instance-1\",\"scan_env_id\":\"env-
1\",\"instance_name\":\"Updated AWS Instance\",\"cloud_provider\":\"AWS\",\"custom_resources_tags\":
[{\"key\":\"environment\",\"value\":\"staging\"}],\"collection_configuration\":{\"audit_logs\":
{\"enabled\":true}},\"additional_capabilities\":
{\"xsiam_analytics\":true,\"data_security_posture_management\":true,\"registry_scanning\":true,\"registry_scanning_options\":
{\"type\":\"ECR\"},\"serverless_scanning\":true,\"agentless_disk_scanning\":true},\"scope_modifications\":
{\"accounts\":{\"enabled\":true,\"type\":\"INCLUDE\",\"account_ids\":
[\"123456789012\",\"210987654321\"]},\"regions\":{\"enabled\":true,\"type\":\"EXCLUDE\",\"regions\":[\"us-east-
1\"]}}}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

▶ instance_idstring

Integration instance ID.

▶ scan_env_idstring

When the scan mode is `OUTPOST`, you must include the `scan_env_id` field with the outpost ID as the value. To obtain the outpost ID, call `/public_api/v1/cloud_onboarding/get_outposts`. The value of `outpost_id` in the response is the outpost ID.

▶ instance_namestring

Integration instance name.

▶ additional_capabilitiesobjectrequired

Define which security capabilities you want to benefit from.

Note: Adding security capability typically requires additional cloud provider permissions. For detailed information on the permissions required, see Cloud service provider permissions.

▶ cloud_providerstring (Enum)

The cloud service provider. Valid values include:

- AWS
- AZURE
- GCP
- OCI

▶ custom_resources_tagsarray

▶ collection_configurationobject

▶ scope_modificationsobject

REQUEST

```
{ "request_data": { "instance_id": "instance-1", "scan_env_id": "env-1", "instance_name": "Updated AWS Instance", "cloud_provider": "AWS", "custom_resources_tags": [ { "key": "environment", "value": "staging" } ], "collection_configuration": { "audit_logs": { "enabled": true } }, "additional_capabilities": { "xsiam_analytics": true, "data_security_posture_management": true, "registry_scanning": true, "registry_scanning_options": { "type": "ECR" }, "serverless_scanning": true, "agentless_disk_scanning": true }, "scope_modifications": { "accounts": { "enabled": true, "type": "INCLUDE", "account_ids": [ "123456789012", "210987654321" ] }, "regions": { "enabled": true, "type": "EXCLUDE", "regions": [ "us-east-1" ] } } } }
```

Responses

Successful Response

Body
application/json
▼ replyobject

The main payload of the response, containing instance detail after edit. It includes a link to download the template file from your tenant.

RESPONSE

```
{ "reply": { "automated": { "link": "string", "tracking_guid": "string" }, "manual": { "TF/ARM": "string" } } }
```

Invalid Client Request Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE

```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

Internal Server Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE

```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

2.3.2.5 | Enable or disable cloud integration instances

post /public_api/v1/cloud_onboarding/enable_disable_instance

Enable or disable the cloud integration instances specified by their instance IDs. To obtain the instance IDs, use `/public_api/v1/cloud_onboarding/get_instances/`.

Required license: Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cloud_onboarding/enable_disable_instance'
-d '{ "request_data" : { "ids" : [ "e22b3a29260c404183d1a0ea6cbd4418", "d32d3b19730d414387d2a3ea7eac4218" ], "enable"
: true } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"ids\":
[\"e22b3a29260c404183d1a0ea6cbd4418\",\"d32d3b19730d414387d2a3ea7eac4218\"],\"enable\":true}}" headers = {
'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/cloud_onboarding/enable_disable_instance", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cloud_onboarding/enable_disable_instance") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url)
request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"ids\":[\"e22b3a29260c404183d1a0ea6cbd4418\",\"d32d3b19730d414387d2a3ea7eac4218\"],\"enable\":true}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "ids": [ "e22b3a29260c404183d1a0ea6cbd4418",
"d32d3b19730d414387d2a3ea7eac4218" ], "enable": true } }); const xhr = new XMLHttpRequest(); xhr.withCredentials =
true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/cloud_onboarding/enable_disable_instance"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-
yourfqdn/public_api/v1/cloud_onboarding/enable_disable_instance") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"request_data\":{\"ids\":
[\"e22b3a29260c404183d1a0ea6cbd4418\",\"d32d3b19730d414387d2a3ea7eac4218\"],\"enable\":true}}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = ["request_data": [ "ids":
["e22b3a29260c404183d1a0ea6cbd4418", "d32d3b19730d414387d2a3ea7eac4218"], "enable": true ]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/cloud_onboarding/enable_disable_instance")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cloud_onboarding/enable_disable_instance", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING =>
```

```
"", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"ids\":
[\"e22b3a29260c404183d1a0ea6cbd4418\",\"d32d3b19730d414387d2a3ea7eac4218\"],\"enable\":true}}", CURLOPT_HTTPHEADER =>
[ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cloud_onboarding/enable_disable_instance"); struct curl_slist
*headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":{\"ids\":
[\"e22b3a29260c404183d1a0ea6cbd4418\",\"d32d3b19730d414387d2a3ea7eac4218\"],\"enable\":true}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cloud_onboarding/enable_disable_instance"); var
request = new RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"ids\":
[\"e22b3a29260c404183d1a0ea6cbd4418\",\"d32d3b19730d414387d2a3ea7eac4218\"],\"enable\":true}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

▶ idsarray[string]

List of integration instance IDs to enable/disable/

▶ enableboolean

Whether to enable the integration instances (`true`) or disable (`false`).

REQUEST

```
{ "request_data": { "ids": [ "e22b3a29260c404183d1a0ea6cbd4418", "d32d3b19730d414387d2a3ea7eac4218" ], "enable": true
} }
```

Responses

Successful Response

Body

application/json

▼ replyobject

The main payload of the response, containing operation-specific data.

RESPONSE

```
{ "reply": {} }
```

Invalid Client Request Error

Body

application/json

▼ replyobject

▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE

```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

Internal Server Error

Body

application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE
```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

2.3.2.6 | Delete the specified integration instances

post /public_api/v1/cloud_onboarding/delete_instance

Delete the specified integration instances.

Required license: Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
```
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
```
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cloud_onboarding/delete_instance'
-d '{ "request_data" : { "ids" : [ "instance-1", "instance-2" ] } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"ids\":
[\"instance-1\",\"instance-2\"]}}" headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/cloud_onboarding/delete_instance", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cloud_onboarding/delete_instance") http = Net::HTTP.new(url.host, url.port) http.use_ssl =
true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"ids\":[\"instance-1\",\"instance-2\"]}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "ids": [ "instance-1", "instance-2" ] } }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/cloud_onboarding/delete_instance"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/cloud_onboarding/delete_instance")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"request_data\":{\"ids\":
[\"instance-1\",\"instance-2\"]}}") .asString();
```

```
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = ["request_data": ["ids": ["instance-
1", "instance-2"]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: [])
let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/cloud_onboarding/delete_instance")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cloud_onboarding/delete_instance", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"ids\":[\"instance-1\",\"instance-2\"]}}", CURLOPT_HTTPHEADER
=> [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cloud_onboarding/delete_instance"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":{\"ids\":[\"instance-1\",\"instance-2\"]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cloud_onboarding/delete_instance"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"ids\":[\"instance-1\",\"instance-2\"]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
required
application/json
▼ request_dataobject
▶ idsarray[string]

List of integration instance IDs to delete.

REQUEST
```
{ "request_data": { "ids": [ "instance-1", "instance-2" ] } }
```
Responses

Successful Response

Body
application/json
▼ replyobject

The main payload of the response, containing operation-specific data.

RESPONSE
```
{ "reply": {} }
```

Invalid Client Request Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE
```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

Internal Server Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE
```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

2.3.2.7 | Get identifier roles for integration instance

post /public_api/v1/cloud_onboarding/get_identifiers

List all of the identifiers roles for the cloud instance. In the UI, these details are listed under Authorization details of the cloud instance.

Required license: In Cortex XSIAM Premium, Cortex XSIAM Enterprise, or Cortex XSIAM NG SIEM, requires the Cortex Cloud Posture Management add-on. In Cortex XSIAM Enterprise Plus, requires the Data Collection add-on.

CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/cloud_onboarding/get_identifiers'
-d '{ "instance_id" : "instance_id", "cloud_provider" : "GCP" }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "
{\"instance_id\":\"string\",\"cloud_provider\":\"GCP\"}" headers = { 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/cloud_onboarding/get_identifiers", payload, headers) res = conn.getresponse()
data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_identifiers") http = Net::HTTP.new(url.host, url.port) http.use_ssl =
true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["content-type"] =
'application/json' request.body = "{\"instance_id\":\"string\",\"cloud_provider\":\"GCP\"}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "instance_id": "string", "cloud_provider": "GCP" }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_identifiers"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/cloud_onboarding/get_identifiers")
.header("content-type", "application/json") .body("{\"instance_id\":\"string\",\"cloud_provider\":\"GCP\"}")
.asString();
import Foundation let headers = ["content-type": "application/json"] let parameters = [ "instance_id": "string",
"cloud_provider": "GCP" ] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_identifiers")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_identifiers", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"instance_id\":\"string\",\"cloud_provider\":\"GCP\"}", CURLOPT_HTTPHEADER => [
"content-type: application/json" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if
($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cloud_onboarding/get_identifiers"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
```

```
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"instance_id\":\"string\",\"cloud_provider\":\"GCP\"}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cloud_onboarding/get_identifiers"); var request = new
RestRequest(Method.POST); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"instance_id\":\"string\",\"cloud_provider\":\"GCP\"}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
required
application/json
▼ instance_idstring

The cloud instance ID for which you want to list the authorization details.

▼ cloud_providerstring (Enum)

The cloud service provider for which you want to list the authorization details.

Allowed values:"GCP"
REQUEST

```
{ "instance_id": "example", "cloud_provider": "GCP" }
```

Responses

Successful Response

Body
application/json
▼ replyobject
▶ cortex_discovery_rolestring

The Cortex discovery role identifier

▶ cortex_scan_platform_rolestring

The Cortex scan platform role identifier

▶ cortex_log_collection_rolestring

The Cortex log collection role identifier

▶ cortex_outpost_rolestring

The Cortex outpost role identifier

RESPONSE

```
{ "reply": { "cortex_discovery_role": "example", "cortex_scan_platform_role": "example",
"cortex_log_collection_role": "example", "cortex_outpost_role": "example" } }
```

Bad Request

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE

```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

Internal Server Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE

```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

2.3.3 | Cloud account management

APIs for managing cloud accounts per instance

2.3.3.1 | Get accounts in the specified integration instances

post /public_api/v1/cloud_onboarding/get_accounts

List accounts and their details for the specified integration instance.

Required license: Cortex Cloud Posture Management

Request headers

▼ Authorization String required

{api_key}

Example:

```
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
```

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cloud_onboarding/get_accounts'
-d '{ "request_data" : { "instance_id" : "instance_id", "filter_data" : { "sort" : [ { "FIELD" : "STATUS", "ORDER" :
"DESC" } ], "paging" : { "from" : 0, "to" : 50 }, "filter" : { "AND" : [ { "SEARCH_FIELD" : "STATUS", "SEARCH_TYPE" :
"EQ", "SEARCH_VALUE" : "ACTIVE" } ] } } } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"instance_id\":\"string\",\"filter_data\":{\"sort\":[{\"FIELD\":\"STATUS\",\"ORDER\":\"DESC\"}],\"paging\":
{\"from\":0,\"to\":50},\"filter\":{\"AND\":
[{\"SEARCH_FIELD\":\"STATUS\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"ACTIVE\"}]}}}}" headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/cloud_onboarding/get_accounts", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_accounts") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"instance_id\":\"string\",\"filter_data\":{\"sort\":[{\"FIELD\":\"STATUS\",\"ORDER\":\"DESC\"}],\"paging\":
{\"from\":0,\"to\":50},\"filter\":{\"AND\":
[{\"SEARCH_FIELD\":\"STATUS\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"ACTIVE\"}]}}}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "instance_id": "string", "filter_data": { "sort": [ { "FIELD":
"STATUS", "ORDER": "DESC" } ], "paging": { "from": 0, "to": 50 }, "filter": { "AND": [ { "SEARCH_FIELD": "STATUS",
"SEARCH_TYPE": "EQ", "SEARCH_VALUE": "ACTIVE" } ] } } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials =
true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_accounts"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
```

```
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/cloud_onboarding/get_accounts")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"request_data\":
{\"instance_id\":\"string\",\"filter_data\":{\"sort\":[{\"FIELD\":\"STATUS\",\"ORDER\":\"DESC\"}],\"paging\":
{\"from\":0,\"to\":50},\"filter\":{\"AND\":
[{\"SEARCH_FIELD\":\"STATUS\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"ACTIVE\"}]}}}}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = ["request_data": [ "instance_id":
"string", "filter_data": [ "sort": [ [ "FIELD": "STATUS", "ORDER": "DESC" ] ], "paging": [ "from": 0, "to": 50 ],
"filter": ["AND": [ [ "SEARCH_FIELD": "STATUS", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "ACTIVE" ] ]] ] ]] as [String :
Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/cloud_onboarding/get_accounts")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_accounts", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"instance_id\":\"string\",\"filter_data\":{\"sort\":
[{\"FIELD\":\"STATUS\",\"ORDER\":\"DESC\"}],\"paging\":{\"from\":0,\"to\":50},\"filter\":{\"AND\":
[{\"SEARCH_FIELD\":\"STATUS\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"ACTIVE\"}]}}}}", CURLOPT_HTTPHEADER => [
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cloud_onboarding/get_accounts"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":{\"instance_id\":\"string\",\"filter_data\":{\"sort\":
[{\"FIELD\":\"STATUS\",\"ORDER\":\"DESC\"}],\"paging\":{\"from\":0,\"to\":50},\"filter\":{\"AND\":
[{\"SEARCH_FIELD\":\"STATUS\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"ACTIVE\"}]}}}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cloud_onboarding/get_accounts"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"instance_id\":\"string\",\"filter_data\":{\"sort\":
[{\"FIELD\":\"STATUS\",\"ORDER\":\"DESC\"}],\"paging\":{\"from\":0,\"to\":50},\"filter\":{\"AND\":
[{\"SEARCH_FIELD\":\"STATUS\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"ACTIVE\"}]}}}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

Valid values for the filter field `SEARCH_FIELD` include:

- `STATUS`
- `ACCOUNT_NAME`
- `CLOUD_ACCOUNT_ID`
- `ENVIRONMENT`
- `TYPE`
- `CREATED_AT`

▶ instance_idstring

▶ filter_dataobject

REQUEST

```
{ "request_data": { "instance_id": "example", "filter_data": { "sort": [ { "FIELD": "STATUS", "ORDER": "DESC" } ],
"paging": { "from": 0, "to": 50 }, "filter": { "AND": [ { "SEARCH_FIELD": "STATUS", "SEARCH_TYPE": "EQ",
"SEARCH_VALUE": "ACTIVE" } ] } } } }
```

Responses

Successful Response

Body
application/json
▼ replyobject

The main payload of the response, containing a list of account details.

RESPONSE
{ "reply": { "DATA": [{ "status": "string", "account_name": "string", "account_id": "string", "environment": "string", "type": "string", "created_at": "string" }], "FILTER_COUNT": "integer", "TOTAL_COUNT": "integer" } }

Invalid Client Request Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }

Internal Server Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }

2.3.3.2 | Enable or disable cloud accounts

post /public_api/v1/cloud_onboarding/enable_disable_account

Enable or disable the specified cloud accounts within a specific integration instance.

Required license: Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
```

```
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cloud_onboarding/enable_disable_account'
-d '{ "request_data" : { "ids" : [ "account-1", "account-2" ], "instance_id" : "instance-1", "enable" : true } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"ids\":
[\"account-1\",\"account-2\"],\"instance_id\":\"instance-1\",\"enable\":true}}" headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/cloud_onboarding/enable_disable_account", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cloud_onboarding/enable_disable_account") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url)
request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"ids\":[\"account-1\",\"account-2\"],\"instance_id\":\"instance-1\",\"enable\":true}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "ids": [ "account-1", "account-2" ], "instance_id": "instance-1",
"enable": true } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/cloud_onboarding/enable_disable_account"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-
yourfqdn/public_api/v1/cloud_onboarding/enable_disable_account") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"request_data\":{\"ids\":
[\"account-1\",\"account-2\"],\"instance_id\":\"instance-1\",\"enable\":true}}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = ["request_data": [ "ids": ["account-
1", "account-2"], "instance_id": "instance-1", "enable": true ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/cloud_onboarding/enable_disable_account")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cloud_onboarding/enable_disable_account", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING =>
"", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"ids\":[\"account-1\",\"account-
2\"],\"instance_id\":\"instance-1\",\"enable\":true}}", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cloud_onboarding/enable_disable_account"); struct curl_slist
*headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":{\"ids\":[\"account-1\",\"account-2\"],\"instance_id\":\"instance-1\",\"enable\":true}}"); CURLcode
ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cloud_onboarding/enable_disable_account"); var
request = new RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"ids\":[\"account-1\",\"account-
2\"],\"instance_id\":\"instance-1\",\"enable\":true}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject
▶ idsarray[string]

List of account IDs to enable or disable.

▶ instance_idstring

The integration instance ID where the cloud accounts should be disabled.

▶ enableboolean

Whether to enable (`true`) or disable (`false`) to specified accounts.

REQUEST
```
{ "request_data": { "ids": [ "account-1", "account-2" ], "instance_id": "instance-1", "enable": true } }
```
Responses

Successful Response

Body
application/json
▼ replyobject
RESPONSE
```
{ "reply": {} }
```

Invalid Client Request Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE
```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

Internal Server Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE
```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

2.3.4 | Outpost management

APIs for managing outposts

2.3.4.1 | Create an outpost template

post /public_api/v1/cloud_onboarding/create_outpost_template

Create a template to onboard an outpost. Define the custom resource tags you want associated with the outpost and Cortex generates an authentication template to establish trust to the CSP and grant permissions to Cortex.

Required license: Cortex Cloud Posture Management

Request headers

▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cloud_onboarding/create_outpost_template'
-d '{ "request_data" : { "cloud_provider" : "AWS", "custom_resources_tags" : [ { "key" : "purpose", "value" :
"outpost" } ] } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"cloud_provider\":\"AWS\",\"custom_resources_tags\":[{\"key\":\"purpose\",\"value\":\"outpost\"}]}}" headers = {
'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/cloud_onboarding/create_outpost_template", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cloud_onboarding/create_outpost_template") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url)
request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"cloud_provider\":\"AWS\",\"custom_resources_tags\":[{\"key\":\"purpose\",\"value\":\"outpost\"}]}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "cloud_provider": "AWS", "custom_resources_tags": [ { "key":
"purpose", "value": "outpost" } ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/cloud_onboarding/create_outpost_template"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-
yourfqdn/public_api/v1/cloud_onboarding/create_outpost_template") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"request_data\":
{\"cloud_provider\":\"AWS\",\"custom_resources_tags\":[{\"key\":\"purpose\",\"value\":\"outpost\"}]}}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = ["request_data": [ "cloud_provider":
"AWS", "custom_resources_tags": [ [ "key": "purpose", "value": "outpost" ] ] ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/cloud_onboarding/create_outpost_template")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cloud_onboarding/create_outpost_template", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING =>
"", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"cloud_provider\":\"AWS\",\"custom_resources_tags\":[{\"key\":\"purpose\",\"value\":\"outpost\"}]}}",
CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cloud_onboarding/create_outpost_template"); struct curl_slist
*headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":{\"cloud_provider\":\"AWS\",\"custom_resources_tags\":
[{\"key\":\"purpose\",\"value\":\"outpost\"}]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cloud_onboarding/create_outpost_template"); var
request = new RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"cloud_provider\":\"AWS\",\"custom_resources_tags\":
[{\"key\":\"purpose\",\"value\":\"outpost\"}]}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

▶ cloud_providerstring (Enum)

The cloud service provider. Valid values include:

- AWS
- AZURE
- GCP
- OCI

▶ custom_resources_tagsarray

REQUEST

```
{ "request_data": { "cloud_provider": "AWS", "custom_resources_tags": [ { "key": "purpose", "value": "outpost" } ] }
}
```

Responses

Successful Response

Body

application/json

▼ replyobject

The main payload of the response, containing a link to download the outpost template

RESPONSE

```
{ "reply": { "automated": {}, "manual": { "TF": "https://qa2-test-9999219916297-cloud-onboarding-
templates.storage.googleapis.com/outposts/aws/tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz?
Expires=1744052440&GoogleAccessId=api-pod%40qa2-test-9999219916297.iam.gserviceaccount.com&Signature=&response-
content-disposition=attachment%3Bfilename%3Doutposts_aws_tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz" } } }
```

Invalid Client Request Error

Body

application/json

▼ replyobject

▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE

```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

Internal Server Error

Body

application/json

▼ replyobject

▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE

```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

2.3.4.2 | Edit an outpost

post /public_api/v1/cloud_onboarding/edit_outpost

Edit the custom resource tags of the specified outpost. The template must be applied in the CSP after editing.

Required license: Cortex Cloud Posture Management

Request headers

▼ Authorization String required

{api_key}

Example:

UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cloud_onboarding/edit_outpost'
-d '{ "reply" : { "automated" : { }, "manual" : { "TF" : "https://qa2-test-9999219916297-cloud-onboarding-
templates.storage.googleapis.com/outposts/aws/tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz?
Expires=1744052440&GoogleAccessId=api-pod%40qa2-test-9999219916297.iam.gserviceaccount.com&Signature=&response-
content-disposition=attachment%3Bfilename%3Doutposts_aws_tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz" } }
}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"reply\":{\"automated\":
{},\"manual\":{\"TF\":\"https://qa2-test-9999219916297-cloud-onboarding-
templates.storage.googleapis.com/outposts/aws/tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz?
Expires=1744052440&GoogleAccessId=api-pod%40qa2-test-9999219916297.iam.gserviceaccount.com&Signature=&response-
content-disposition=attachment%3Bfilename%3Doutposts_aws_tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz\"}}}"
headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/cloud_onboarding/edit_outpost", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cloud_onboarding/edit_outpost") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"reply\":
{\"automated\":{},\"manual\":{\"TF\":\"https://qa2-test-9999219916297-cloud-onboarding-
templates.storage.googleapis.com/outposts/aws/tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz?
Expires=1744052440&GoogleAccessId=api-pod%40qa2-test-9999219916297.iam.gserviceaccount.com&Signature=&response-
content-disposition=attachment%3Bfilename%3Doutposts_aws_tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz\"}}}"
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "reply": { "automated": {}, "manual": { "TF": "https://qa2-test-9999219916297-cloud-
onboarding-templates.storage.googleapis.com/outposts/aws/tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz?
Expires=1744052440&GoogleAccessId=api-pod%40qa2-test-9999219916297.iam.gserviceaccount.com&Signature=&response-
content-disposition=attachment%3Bfilename%3Doutposts_aws_tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz" } }
}); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function
```

```
() { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/cloud_onboarding/edit_outpost"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/cloud_onboarding/edit_outpost")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"reply\":{\"automated\":
{},\"manual\":{\"TF\":\"https://qa2-test-9999219916297-cloud-onboarding-
templates.storage.googleapis.com/outposts/aws/tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz?
Expires=1744052440&GoogleAccessId=api-pod%40qa2-test-9999219916297.iam.gserviceaccount.com&Signature=&response-
content-disposition=attachment%3Bfilename%3Doutposts_aws_tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz\"}}}")
.asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = ["reply": [ "automated": [],
"manual": ["TF": "https://qa2-test-9999219916297-cloud-onboarding-templates.storage.googleapis.com/outposts/aws/tf-
12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz?Expires=1744052440&GoogleAccessId=api-pod%40qa2-test-
9999219916297.iam.gserviceaccount.com&Signature=&response-content-
disposition=attachment%3Bfilename%3Doutposts_aws_tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz"] ]] as
[String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/cloud_onboarding/edit_outpost")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cloud_onboarding/edit_outpost", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"reply\":{\"automated\":{},\"manual\":{\"TF\":\"https://qa2-test-9999219916297-
cloud-onboarding-templates.storage.googleapis.com/outposts/aws/tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz?
Expires=1744052440&GoogleAccessId=api-pod%40qa2-test-9999219916297.iam.gserviceaccount.com&Signature=&response-
content-disposition=attachment%3Bfilename%3Doutposts_aws_tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz\"}}}",
CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cloud_onboarding/edit_outpost"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"reply\":{\"automated\":{},\"manual\":{\"TF\":\"https://qa2-test-9999219916297-cloud-onboarding-
templates.storage.googleapis.com/outposts/aws/tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz?
Expires=1744052440&GoogleAccessId=api-pod%40qa2-test-9999219916297.iam.gserviceaccount.com&Signature=&response-
content-disposition=attachment%3Bfilename%3Doutposts_aws_tf-12711a531d51412d930a647ed2fdacdf-
1743447640.tar.gz\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cloud_onboarding/edit_outpost"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"reply\":{\"automated\":{},\"manual\":{\"TF\":\"https://qa2-test-
9999219916297-cloud-onboarding-templates.storage.googleapis.com/outposts/aws/tf-12711a531d51412d930a647ed2fdacdf-
1743447640.tar.gz?Expires=1744052440&GoogleAccessId=api-pod%40qa2-test-
9999219916297.iam.gserviceaccount.com&Signature=&response-content-
disposition=attachment%3Bfilename%3Doutposts_aws_tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz\"}}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ replyobject

The main payload of the response, containing a link to download the edited outpost template

REQUEST

```
{ "reply": { "automated": {}, "manual": { "TF": "https://qa2-test-9999219916297-cloud-onboarding-
templates.storage.googleapis.com/outposts/aws/tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz?
```

Expires=1744052440&GoogleAccessId=api-pod%40qa2-test-9999219916297.iam.gserviceaccount.com&Signature=&response-content-disposition=attachment%3Bfilename%3Doutposts_aws_tf-12711a531d51412d930a647ed2fdacdf-1743447640.tar.gz" } } }

Responses

Successful Response

Body
application/json
▼ replyobject

The main payload of the response, containing operation-specific data.

RESPONSE
{ "reply": {} }

Invalid Client Request Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }

Internal Server Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }

2.3.4.3 | Get all or filtered outposts

post /public_api/v1/cloud_onboarding/get_outposts

Get a list of all or filtered outposts.

Required license: Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cloud_onboarding/get_outposts'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filter_data\":
{\"sort\":[{\"FIELD\":\"string\",\"ORDER\":\"string\"}],\"filter\":{\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"string\",\"SEARCH_VALUE\":\"string\"}]},\"paging\":
{\"from\":0,\"to\":0}}}}" headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/cloud_onboarding/get_outposts", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_outposts") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"filter_data\":{\"sort\":[{\"FIELD\":\"string\",\"ORDER\":\"string\"}],\"filter\":{\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"string\",\"SEARCH_VALUE\":\"string\"}]},\"paging\":
{\"from\":0,\"to\":0}}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filter_data": { "sort": [ { "FIELD": "string", "ORDER": "string" }
], "filter": { "OR": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE": "string", "SEARCH_VALUE": "string" } ] }, "paging":
{ "from": 0, "to": 0 } } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_outposts"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/cloud_onboarding/get_outposts")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"request_data\":
{\"filter_data\":{\"sort\":[{\"FIELD\":\"string\",\"ORDER\":\"string\"}],\"filter\":{\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"string\",\"SEARCH_VALUE\":\"string\"}]},\"paging\":
{\"from\":0,\"to\":0}}}}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = ["request_data": ["filter_data": [
"sort": [ [ "FIELD": "string", "ORDER": "string" ] ], "filter": ["OR": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE":
"string", "SEARCH_VALUE": "string" ] ]], "paging": [ "from": 0, "to": 0 ] ]]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/cloud_onboarding/get_outposts")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_outposts", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filter_data\":{\"sort\":
[{\"FIELD\":\"string\",\"ORDER\":\"string\"}],\"filter\":{\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"string\",\"SEARCH_VALUE\":\"string\"}]},\"paging\":
{\"from\":0,\"to\":0}}}}", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cloud_onboarding/get_outposts"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
```

```
{\"request_data\":{\"filter_data\":{\"sort\":[{\"FIELD\":\"string\",\"ORDER\":\"string\"}],\"filter\":{\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"string\",\"SEARCH_VALUE\":\"string\"}]},\"paging\":
{\"from\":0,\"to\":0}}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cloud_onboarding/get_outposts"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"filter_data\":{\"sort\":
[{\"FIELD\":\"string\",\"ORDER\":\"string\"}],\"filter\":{\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"string\",\"SEARCH_VALUE\":\"string\"}]},\"paging\":
{\"from\":0,\"to\":0}}}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ filter_dataobject

REQUEST Example 1

```
{ "request_data": { "filter_data": { "sort": [ { "FIELD": "CREATED_AT", "ORDER": "ASC" } ], "filter": { "OR": [ {
"SEARCH_FIELD": "CLOUD_PROVIDER", "SEARCH_TYPE": "CONTAINS", "SEARCH_VALUE": "AWS" }, { "SEARCH_FIELD":
"CLOUD_PROVIDER", "SEARCH_TYPE": "CONTAINS", "SEARCH_VALUE": "GCP" } ] }, "paging": { "from": 0, "to": 20 } } } }
```

Responses

OK

Body

application/json

▼ replyobject

▶ DATAarray

▶ FILTER_COUNTinteger

▶ TOTAL_COUNTinteger

RESPONSE Example 1

```
{ "reply": { "DATA": [ { "cloud_provider": "AWS", "outpost_id": "094935393dcf46128e5a2138a51bd54a", "created_at":
"1748309918599", "type": "MANAGED" } ], "FILTER_COUNT": 1, "TOTAL_COUNT": 1 } }
```

2.3.5 | General

APIs for general tasks

2.3.5.1 | Get available regions for a CSP

post /public_api/v1/cloud_onboarding/list_regions

Get a list of all available regions for the specified cloud service provider.

Required license: Cortex Cloud Posture Management

Request headers

▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cloud_onboarding/list_regions'
-d '{ "request_data" : { "cloud_provider" : "AWS" } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"cloud_provider\":\"AWS\"}}" headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
```

```
"/public_api/v1/cloud_onboarding/list_regions", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cloud_onboarding/list_regions") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"cloud_provider\":\"AWS\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "cloud_provider": "AWS" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/cloud_onboarding/list_regions"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/cloud_onboarding/list_regions")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"request_data\":
{\"cloud_provider\":\"AWS\"}}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = ["request_data": ["cloud_provider":
"AWS"]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request
= NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/cloud_onboarding/list_regions")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cloud_onboarding/list_regions", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"cloud_provider\":\"AWS\"}}", CURLOPT_HTTPHEADER => [
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cloud_onboarding/list_regions"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":{\"cloud_provider\":\"AWS\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cloud_onboarding/list_regions"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"cloud_provider\":\"AWS\"}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

▶ cloud_providerstring (Enum)

The cloud service provider for which you want to list the regions.

REQUEST

```
{ "request_data": { "cloud_provider": "AWS" } }
```

Responses

Successful Response

Body

application/json

▼ replyobject

The main payload of the response, containing list of available regions

▶ cloud_providerstring (Enum)

Cloud provider name

RESPONSE
{ "reply": { "name": "US East (Ohio)", "value": "us-east-2" } }

Invalid Client Request Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }

Internal Server Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error

RESPONSE
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }

2.3.5.2 | Get approved Azure tenants

post /public_api/v1/cloud_onboarding/get_azure_approved_tenants

In order to onboard an Azure tenant to Cortex, Cortex must first be added as an enterprise-approved application on the tenant. Get a list of all the Azure tenants that are already enterprise-approved.

Required license: Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cloud_onboarding/get_azure_approved_tenants'
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("POST", "/public_api/v1/cloud_onboarding/get_azure_approved_tenants",
headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_azure_approved_tenants") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url)
request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_azure_approved_tenants"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_azure_approved_tenants") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_azure_approved_tenants")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cloud_onboarding/get_azure_approved_tenants", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING
=> "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cloud_onboarding/get_azure_approved_tenants"); struct curl_slist
*headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cloud_onboarding/get_azure_approved_tenants"); var
request = new RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Successful Response

Body
application/json
▼ replyobject

The main payload of the response, containing the Azure tenant ID and whether the tenant is approved.

RESPONSE

```
{ "reply": { "tenants": [ { "tenant_id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx", "is_still_approved": true } ] } }
```

Internal Server Error

Body
application/json
▼ replyobject
▶ err_codestring

Error code

▶ err_msgstring

Error message

▶ err_extrastring

Extra information about the error
RESPONSE
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }

2.4 | ASPM, CICD and Application Security APIs

2.4.1 | Preface

You can export the OpenAPI specs in the Cortex Stoplight instance.

Enhance your security operations and automation capabilities using the ASPM, CICD and Application Security APIs to optimize the management of applications, repositories, data sources, rules, policies, and scans.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Version: Cortex Cloud 1.3

2.4.1.1 | Servers

https://api-yourfqdn

2.4.1.2 | Authentication methods

▼ XDRAuth
Api Key "x-xdr-auth-id"

API Key ID for authentication

Authorized requests to the API should use: **x-xdr-auth-id** inside **a header** with the access token.

▼ XDRAuthToken
Api Key "Authorization"

API Key for authentication

Authorized requests to the API should use: **Authorization** inside **a header** with the access token.

2.4.1.2.1 | Default authentication

Please note that this documentation assumes a minimum level of authentication is required, unless explicitly stated otherwise.
It is essential to ensure that any of the following authentication mechanisms are in place to access the API endpoints:

- XDRAuth=[], XDRAuthToken

2.4.2 | Applications

APIs for managing applications

2.4.2.1 | Get an application configuration

get /public_api/appsec/v1/application/configuration

Retrieves the application configuration settings, including flags for *Auto-group apps by cloud tags* (runtime) and *Auto-refresh related app assets*.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
▼ x-xdr-auth-id String required

{api_key_id}

Example: `241`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
```

```
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/application/configuration'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/application/configuration", headers=headers)
res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/application/configuration") http = Net::HTTP.new(url.host, url.port) http.use_ssl =
true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/appsec/v1/application/configuration"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/application/configuration")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/application/configuration")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/application/configuration", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/application/configuration"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/application/configuration"); var request = new
RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
▼ areSbomIssuesConsideredNewboolean
▼ isAutoGroupAppsboolean
▼ keepApplicationRefreshboolean
RESPONSE

```
{ "areSbomIssuesConsideredNew": false, "isAutoGroupApps": false, "keepApplicationRefresh": false }
```

2.4.2.2 | Create an application

post /public_api/appsec/v1/application

Creates a new application asset based on the provided details, such as name, criticality, and asset selection.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers

▼ x-cas-trace-id String

Example: `xCasTraceId_example`

▼ Authorization String required

{api_key}

Example:
`UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `241`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'x-cas-trace-id: xCasTraceId_example' -H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/application'
-d '{ "productManager" : [ "productManager", "productManager" ], "assetSelection" : { "filter" : { "values" : [
"values", "values" ] }, "assetIds" : [ "assetIds", "assetIds" ] }, "compliance" : [ null, null ], "name" : "name",
"description" : "description", "devOwner" : [ "devOwner", "devOwner" ], "devOpsOwner" : [ "devOpsOwner",
"devOpsOwner" ], "businessOwner" : [ "businessOwner", "businessOwner" ] }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "
{\"name\":\"string\",\"category\":\"Business
Application\",\"businessCriticality\":\"Critical\",\"creationType\":\"Auto\",\"description\":\"string\",\"compliance\":
[\"PCI-DSS\"],\"businessOwner\":[\"string\"],\"devOwner\":[\"string\"],\"devOpsOwner\":
[\"string\"],\"productManager\":[\"string\"],\"assetSelection\":
{\"selectionType\":\"filter\",\"section\":\"code\",\"filter\":{\"values\":
[\"string\"],\"provider\":\"CIRCLE_CI\",\"filterType\":\"REPOSITORY\"},\"assetIds\":[\"string\"]}}" headers = { 'x-
cas-trace-id': "SOME_STRING_VALUE", 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
"/public_api/appsec/v1/application", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/appsec/v1/application")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["x-cas-trace-id"] = 'SOME_STRING_VALUE' request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "
{\"name\":\"string\",\"category\":\"Business
Application\",\"businessCriticality\":\"Critical\",\"creationType\":\"Auto\",\"description\":\"string\",\"compliance\":
[\"PCI-DSS\"],\"businessOwner\":[\"string\"],\"devOwner\":[\"string\"],\"devOpsOwner\":
[\"string\"],\"productManager\":[\"string\"],\"assetSelection\":
{\"selectionType\":\"filter\",\"section\":\"code\",\"filter\":{\"values\":
[\"string\"],\"provider\":\"CIRCLE_CI\",\"filterType\":\"REPOSITORY\"},\"assetIds\":[\"string\"]}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "name": "string", "category": "Business Application", "businessCriticality":
"Critical", "creationType": "Auto", "description": "string", "compliance": [ "PCI-DSS" ], "businessOwner": [ "string"
], "devOwner": [ "string" ], "devOpsOwner": [ "string" ], "productManager": [ "string" ], "assetSelection": {
"selectionType": "filter", "section": "code", "filter": { "values": [ "string" ], "provider": "CIRCLE_CI",
"filterType": "REPOSITORY" }, "assetIds": [ "string" ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials =
true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/appsec/v1/application");
xhr.setRequestHeader("x-cas-trace-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/appsec/v1/application") .header("x-cas-
trace-id", "SOME_STRING_VALUE") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("
{\"name\":\"string\",\"category\":\"Business
Application\",\"businessCriticality\":\"Critical\",\"creationType\":\"Auto\",\"description\":\"string\",\"compliance\":
[\"PCI-DSS\"],\"businessOwner\":[\"string\"],\"devOwner\":[\"string\"],\"devOpsOwner\":
[\"string\"],\"productManager\":[\"string\"],\"assetSelection\":
```

```
{\"selectionType\":\"filter\",\"section\":\"code\",\"filter\":{\"values\":
[\"string\"],\"provider\":\"CIRCLE_CI\",\"filterType\":\"REPOSITORY\"},\"assetIds\":[\"string\"]}}") .asString();
import Foundation let headers = [ "x-cas-trace-id": "SOME_STRING_VALUE", "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = [ "name": "string", "category":
"Business Application", "businessCriticality": "Critical", "creationType": "Auto", "description": "string",
"compliance": ["PCI-DSS"], "businessOwner": ["string"], "devOwner": ["string"], "devOpsOwner": ["string"],
"productManager": ["string"], "assetSelection": [ "selectionType": "filter", "section": "code", "filter": [ "values":
["string"], "provider": "CIRCLE_CI", "filterType": "REPOSITORY" ], "assetIds": ["string"] ] ] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/appsec/v1/application")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/application", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"name\":\"string\",\"category\":\"Business
Application\",\"businessCriticality\":\"Critical\",\"creationType\":\"Auto\",\"description\":\"string\",\"compliance\":
[\"PCI-DSS\"],\"businessOwner\":[\"string\"],\"devOwner\":[\"string\"],\"devOpsOwner\":
[\"string\"],\"productManager\":[\"string\"],\"assetSelection\":
{\"selectionType\":\"filter\",\"section\":\"code\",\"filter\":{\"values\":
[\"string\"],\"provider\":\"CIRCLE_CI\",\"filterType\":\"REPOSITORY\"},\"assetIds\":[\"string\"]}}",
CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-cas-trace-id: SOME_STRING_VALUE", "x-xdr-auth-id: 241" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/appsec/v1/application"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "x-cas-trace-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"name\":\"string\",\"category\":\"Business
Application\",\"businessCriticality\":\"Critical\",\"creationType\":\"Auto\",\"description\":\"string\",\"compliance\":
[\"PCI-DSS\"],\"businessOwner\":[\"string\"],\"devOwner\":[\"string\"],\"devOpsOwner\":
[\"string\"],\"productManager\":[\"string\"],\"assetSelection\":
{\"selectionType\":\"filter\",\"section\":\"code\",\"filter\":{\"values\":
[\"string\"],\"provider\":\"CIRCLE_CI\",\"filterType\":\"REPOSITORY\"},\"assetIds\":[\"string\"]}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/application"); var request = new
RestRequest(Method.POST); request.AddHeader("x-cas-trace-id", "SOME_STRING_VALUE");
request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"name\":\"string\",\"category\":\"Business
Application\",\"businessCriticality\":\"Critical\",\"creationType\":\"Auto\",\"description\":\"string\",\"compliance\":
[\"PCI-DSS\"],\"businessOwner\":[\"string\"],\"devOwner\":[\"string\"],\"devOpsOwner\":
[\"string\"],\"productManager\":[\"string\"],\"assetSelection\":
{\"selectionType\":\"filter\",\"section\":\"code\",\"filter\":{\"values\":
[\"string\"],\"provider\":\"CIRCLE_CI\",\"filterType\":\"REPOSITORY\"},\"assetIds\":[\"string\"]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ namestring

▼ categorystring (Enum)

Allowed values:"Business Application"

▼ businessCriticalitystring (Enum)

Allowed values:"Critical""High""Medium""Low"

▼ creationTypestring (Enum)

Allowed values:"Auto""Manual"

▼ descriptionstring

▼ compliancearray[string]

▼ businessOwnerarray[string]

▼ devOwnerarray[string]

▼ devOpsOwnerarray[string]
▼ productManagerarray[string]
▼ assetSelectionobjectrequired
▶ selectionTypestring (Enum)
▶ sectionstring (Enum)
▶ filterobjectrequired
▶ assetIdsarray[string]
Responses

Ok

Body
application/json
▼ statusboolean
▼ applicationIdstring
▼ errorMessagestring
RESPONSE
```
{ "status": false, "applicationId": "example", "errorMessage": "example" }
```

2.4.2.3 | Get applications

get /public_api/appsec/v1/application

Retrieves a list of all application assets using pagination.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
Query parameters
▼ page Double
double
Example: 1.2
▼ pageSize Double
double
Example: 1.2
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/application?page=1.2&pageSize=1.2'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/application?
page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE", headers=headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/appsec/v1/application?
page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/appsec/v1/application?
page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/application?
page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/application?page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/application?page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE", CURLOPT_RETURNTRANSFER
=> true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/application?page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE"); struct
curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/application?
page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE"); var request = new RestRequest(Method.GET);
request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body

application/json

▼ dataarray

[

▶ criteriaIdstring

▶ risknumberdouble

▶ idstring

▶ strongIdstring

▶ namestring

▶ descriptionstring

▶ categorystring (Enum)

▶ exposuresarray[string]

▶ coveragenumberdouble

▶ businessCriticalitystring (Enum)

▶ businessUnitstring

▶ creationTypestring (Enum)

▶ businessOwnerarray[string]

▶ devOwnerarray[string]

▶ devOpsOwnerarray[string]

▶ productManagerarray[string]

▶ dataGovernancestring (Enum)

▶ compliancearray[string]

▶ environmentsstring

▶ assetsCountnumberdouble

▶ configobjectrequired

▶ updatedAtstringdate-time

▶ deletedAtstringdate-time

▶ enrichStatusstring (Enum)

]

▼ hasNextboolean

RESPONSE


```
{ "data": [ { "criteriaId": "example", "risk": 0.1, "id": "example", "strongId": "example", "name": "example",
"description": "example", "category": "Business Application", "exposures": [ "DeployedToProduction" ], "coverage":
0.1, "businessCriticality": "Critical", "businessUnit": "example", "creationType": "Auto", "businessOwner": [
"example" ], "devOwner": [ "example" ], "devOpsOwner": [ "example" ], "productManager": [ "example" ],
```

"dataGovernance": "PII", "compliance": ["PCI-DSS"], "environments": "example", "assetsCount": 0.1, "config": { "lastRefreshDate": "2020-01-01T12:00:00Z", "assetSelection": { "selectionType": "filter", "section": "code", "filter": { "values": ["example"], "provider": "CIRCLE_CI", "filterType": "REPOSITORY" }, "assetIds": ["example"] } }, "updatedAt": "2020-01-01T12:00:00Z", "deletedAt": "2020-01-01T12:00:00Z", "enrichStatus": "IN_PROGRESS" }], "hasNext": false }

2.4.2.4 | Get an application

get /public_api/appsec/v1/application/{applicationId}

Retrieves detailed information for a specific application asset using its unique ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ applicationId String required
Example: `applicationId_example`
Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/application/{applicationId}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/application/%7BapplicationId%7D",
headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl
= true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-
yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING =>
"", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
```

```
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D"); var request
= new RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
▼ criteriaIdstring
▼ risknumberdouble
▼ idstring
▼ strongIdstring
▼ namestring
▼ descriptionstring
▼ categorystring (Enum)
Allowed values:"Business Application"
▼ exposuresarray[string]
▼ coveragenumberdouble
▼ businessCriticalitystring (Enum)
Allowed values:"Critical""High""Medium""Low"
▼ businessUnitstring
▼ creationTypestring (Enum)
Allowed values:"Auto""Manual"
▼ businessOwnerarray[string]
▼ devOwnerarray[string]
▼ devOpsOwnerarray[string]
▼ productManagerarray[string]
▼ dataGovernancestring (Enum)
Allowed values:"PII""PCI""Financial""PHI""Sensitive"
▼ compliancearray[string]
▼ environmentsstring
▼ assetsCountnumberdouble
▼ configobjectrequired
▶ lastRefreshDatestringdate-time
▶ assetSelectionobjectrequired
▼ updatedAtstringdate-time
▼ deletedAtstringdate-time
▼ enrichStatusstring (Enum)
Allowed values:"IN_PROGRESS""NOT_IN_PROGRESS"
RESPONSE


```
{ "criteriaId": "example", "risk": 0.1, "id": "example", "strongId": "example", "name": "example", "description":
"example", "category": "Business Application", "exposures": [ "DeployedToProduction" ], "coverage": 0.1,
"businessCriticality": "Critical", "businessUnit": "example", "creationType": "Auto", "businessOwner": [ "example" ],
"devOwner": [ "example" ], "devOpsOwner": [ "example" ], "productManager": [ "example" ], "dataGovernance": "PII",
"compliance": [ "PCI-DSS" ], "environments": "example", "assetsCount": 0.1, "config": { "lastRefreshDate": "2020-01-
01T12:00:00Z", "assetSelection": { "selectionType": "filter", "section": "code", "filter": { "values": [ "example" ],
"provider": "CIRCLE_CI", "filterType": "REPOSITORY" }, "assetIds": [ "example" ] } }, "updatedAt": "2020-01-
01T12:00:00Z", "deletedAt": "2020-01-01T12:00:00Z", "enrichStatus": "IN_PROGRESS" }
```

2.4.2.5 | Update an application

put /public_api/appsec/v1/application/{applicationId}

Updates the attributes of an existing application asset identified by its unique ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters

▼ applicationId String required
Example: `applicationId_example`
Request headers
▼ x-cas-trace-id String
Example: `xCasTraceId_example`
▼ Authorization String required

{api_key}

Example:
`UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `241`
CLIENT REQUEST Bash+curl

```
curl -X 'PUT'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'x-cas-trace-id: xCasTraceId_example' -H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/application/{applicationId}'
-d '{ "productManager" : [ "productManager", "productManager" ], "assetSelection" : { "filter" : { "values" : [
"values", "values" ] }, "assetIds" : [ "assetIds", "assetIds" ] }, "compliance" : [ null, null ], "name" : "name",
"description" : "description", "devOwner" : [ "devOwner", "devOwner" ], "devOpsOwner" : [ "devOpsOwner",
"devOpsOwner" ], "businessOwner" : [ "businessOwner", "businessOwner" ] }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "
{\"name\":\"string\",\"businessCriticality\":\"Critical\",\"description\":\"string\",\"compliance\":[\"PCI-
DSS\"],\"businessOwner\":[\"string\"],\"devOwner\":[\"string\"],\"devOpsOwner\":[\"string\"],\"productManager\":
[\"string\"],\"assetSelection\":{\"selectionType\":\"filter\",\"section\":\"code\",\"filter\":{\"values\":
[\"string\"],\"provider\":\"CIRCLE_CI\",\"filterType\":\"REPOSITORY\"},\"assetIds\":[\"string\"]}}" headers = { 'x-
cas-trace-id': "SOME_STRING_VALUE", 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("PUT",
"/public_api/appsec/v1/application/%7BapplicationId%7D", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl
= true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Put.new(url) request["x-cas-trace-id"] =
'SOME_STRING_VALUE' request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "
{\"name\":\"string\",\"businessCriticality\":\"Critical\",\"description\":\"string\",\"compliance\":[\"PCI-
DSS\"],\"businessOwner\":[\"string\"],\"devOwner\":[\"string\"],\"devOpsOwner\":[\"string\"],\"productManager\":
[\"string\"],\"assetSelection\":{\"selectionType\":\"filter\",\"section\":\"code\",\"filter\":{\"values\":
[\"string\"],\"provider\":\"CIRCLE_CI\",\"filterType\":\"REPOSITORY\"},\"assetIds\":[\"string\"]}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "name": "string", "businessCriticality": "Critical", "description": "string",
"compliance": [ "PCI-DSS" ], "businessOwner": [ "string" ], "devOwner": [ "string" ], "devOpsOwner": [ "string" ],
"productManager": [ "string" ], "assetSelection": { "selectionType": "filter", "section": "code", "filter": {
"values": [ "string" ], "provider": "CIRCLE_CI", "filterType": "REPOSITORY" }, "assetIds": [ "string" ] } }); const
xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("PUT", "https://api-
yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D"); xhr.setRequestHeader("x-cas-trace-id",
"SOME_STRING_VALUE"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.put("https://api-
yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D") .header("x-cas-trace-id", "SOME_STRING_VALUE")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("
{\"name\":\"string\",\"businessCriticality\":\"Critical\",\"description\":\"string\",\"compliance\":[\"PCI-
DSS\"],\"businessOwner\":[\"string\"],\"devOwner\":[\"string\"],\"devOpsOwner\":[\"string\"],\"productManager\":
[\"string\"],\"assetSelection\":{\"selectionType\":\"filter\",\"section\":\"code\",\"filter\":{\"values\":
[\"string\"],\"provider\":\"CIRCLE_CI\",\"filterType\":\"REPOSITORY\"},\"assetIds\":[\"string\"]}}") .asString();
```

```
import Foundation let headers = [ "x-cas-trace-id": "SOME_STRING_VALUE", "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = [ "name": "string",
"businessCriticality": "Critical", "description": "string", "compliance": ["PCI-DSS"], "businessOwner": ["string"],
"devOwner": ["string"], "devOpsOwner": ["string"], "productManager": ["string"], "assetSelection": [ "selectionType":
"filter", "section": "code", "filter": [ "values": ["string"], "provider": "CIRCLE_CI", "filterType": "REPOSITORY" ],
"assetIds": ["string"] ] ] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "PUT" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING =>
"", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "PUT", CURLOPT_POSTFIELDS => "
{\"name\":\"string\",\"businessCriticality\":\"Critical\",\"description\":\"string\",\"compliance\":[\"PCI-
DSS\"],\"businessOwner\":[\"string\"],\"devOwner\":[\"string\"],\"devOpsOwner\":[\"string\"],\"productManager\":
[\"string\"],\"assetSelection\":{\"selectionType\":\"filter\",\"section\":\"code\",\"filter\":{\"values\":
[\"string\"],\"provider\":\"CIRCLE_CI\",\"filterType\":\"REPOSITORY\"},\"assetIds\":[\"string\"]}}",
CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "content-type: application/json", "x-cas-trace-id: SOME_STRING_VALUE", "x-xdr-auth-id: 241" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "PUT"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "x-cas-trace-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers,
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"name\":\"string\",\"businessCriticality\":\"Critical\",\"description\":\"string\",\"compliance\":[\"PCI-
DSS\"],\"businessOwner\":[\"string\"],\"devOwner\":[\"string\"],\"devOpsOwner\":[\"string\"],\"productManager\":
[\"string\"],\"assetSelection\":{\"selectionType\":\"filter\",\"section\":\"code\",\"filter\":{\"values\":
[\"string\"],\"provider\":\"CIRCLE_CI\",\"filterType\":\"REPOSITORY\"},\"assetIds\":[\"string\"]}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D"); var request
= new RestRequest(Method.PUT); request.AddHeader("x-cas-trace-id", "SOME_STRING_VALUE");
request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "
{\"name\":\"string\",\"businessCriticality\":\"Critical\",\"description\":\"string\",\"compliance\":[\"PCI-
DSS\"],\"businessOwner\":[\"string\"],\"devOwner\":[\"string\"],\"devOpsOwner\":[\"string\"],\"productManager\":
[\"string\"],\"assetSelection\":{\"selectionType\":\"filter\",\"section\":\"code\",\"filter\":{\"values\":
[\"string\"],\"provider\":\"CIRCLE_CI\",\"filterType\":\"REPOSITORY\"},\"assetIds\":[\"string\"]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ namestring

▼ businessCriticalitystring (Enum)

Allowed values:"Critical""High""Medium""Low"

▼ descriptionstring

▼ compliancearray[string]

▼ businessOwnerarray[string]

▼ devOwnerarray[string]

▼ devOpsOwnerarray[string]

▼ productManagerarray[string]

▼ assetSelectionobjectrequired

▶ selectionTypestring (Enum)

▶ sectionstring (Enum)

▶ filterobjectrequired

▶ assetIdsarray[string]

REQUEST

{ "name": "example", "businessCriticality": "Critical", "description": "example", "compliance": ["PCI-DSS"], "businessOwner": ["example"], "devOwner": ["example"], "devOpsOwner": ["example"], "productManager": ["example"], "assetSelection": { "selectionType": "filter", "section": "code", "filter": { "values": ["example"], "provider": "CIRCLE_CI", "filterType": "REPOSITORY" }, "assetIds": ["example"] } }
Responses

Ok

Body
application/json
▼ statusboolean
▼ applicationIdstring
▼ errorMessagestring
RESPONSE
{ "status": false, "applicationId": "example", "errorMessage": "example" }

2.4.2.6 | Delete an application

delete /public_api/appsec/v1/application/{applicationId}

Deletes a specific application asset identified by its unique ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ applicationId String required
Example: `applicationId_example`
Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
curl -X 'DELETE'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/application/{applicationId}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("DELETE", "/public_api/appsec/v1/application/%7BapplicationId%7D",
headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl
= true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Delete.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("DELETE", "https://api-
yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.delete("https://api-
yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = ["Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241"] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D")! as URL, cachePolicy: .useProtocolCachePolicy,

```
timeoutInterval: 10.0) request.httpMethod = "DELETE" request.allHTTPHeaderFields = headers let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING =>
"", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "DELETE", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "DELETE"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D"); struct curl_slist *headers
= NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/application/%7BapplicationId%7D"); var request
= new RestRequest(Method.DELETE); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
▼ messagestring
RESPONSE

```
{ "message": "example" }
```

2.4.3 | Policies

APIs for managing policies

2.4.3.1 | Create an AppSec policy

post /public_api/appsec/v1/policies

An Application Security policy defines how a system should respond to application security threats. It includes conditions that trigger the policy, the scope of its application, and the actions to be taken when these conditions are met. When a policy detects a threat, it generates an issue for remediation.

2.4.3.1.1 | Create an Application Security policy.

2.4.3.1.1.1 | How to access Application Security policies in the UI:

Go to **Modules**, then select **Application Security** > **AppSec policies** under **Policy management**.

> Required license:
>
> Cortex Cloud Posture Management or Cortex Cloud Runtime Security.

2.4.3.1.1.2 | Limitations

Be aware of the following limitations when creating policies:

Trigger and Action Constraints by Finding Type

1. **CI/CD Risks Finding Type** (`CAS_CI_CD_RISK_SCANNER`):

- Only the **periodic** scan trigger is supported.
- Only the **reportIssue** action is available.
- For **conditions**, the `category` parameter supports only the following values:
  - Application
  - CI/CD Instance
  - CI/CD Pipeline
  - VCS Collaborator
  - VCS Organization
- For **scope filters**, only the following parameters are supported:
  - Business Criticality
  - Business Owner
  - Provider
  - CI/CD Instance Name
  - CI/CD Instance Id
  - CI/CD Pipeline Name
  - CI/CD Pipeline Id
  - VCS Collaborator Name
  - VCS Collaborator Email
  - VCS Collaborator MFA Enabled
  - VCS Collaborator Last Observed
  - VCS Organization Name

2. **Other Finding Types** (for example, CVE, SAST, IaC, Secret, Operational Risk)
   - For **conditions**, the `category` parameter only supports `Application` and `Repository` values.
   - For **scope filters**, only the following parameters are supported:
     - Business Criticality
     - Business Owner
     - Application Provider
     - Is Public Repository
     - Has Deployed Assets
     - Has Internet-exposed deployed assets
     - Has deployed assets with Access to sensitive data
     - Has deployed assets with privileged capabilities
     - Repository Name
     - Repository Id

General Policy Constraints

1. **Third-Party Integration Data**
   - For ingested data from third-party integrations:
     - Only the **periodic** scan trigger is supported, along with the `ingestedData` action that must be explicitly configured.
2. **Multiple Finding Types in Conditions**
   - When selecting multiple `Finding Type` values within a single condition, only common filters across those types are available.
   - To use filters specific to a `Finding Type`, add another condition group that is separated by the **OR** operator.
3. **Condition and Scope Depth**
   - The UI supports up to **2 levels** of depth for conditions and scope.
   - The API allows up to **10 levels** of depth for more complex policy configurations.
4. **Scope vs. Asset Groups**
   - You must define either a **scope** (detailed asset filters) or specify **assetGroupIds**, but **not** both in the same policy.
   - If both fields are defined (contain values), then only the **assetGroupIds** will be applied and the **scope** will be ignored.
5. **Empty Results during Configuration**
   - `Conditions` and `scope` filters may return **empty results** during policy configuration.
   - Policies can still be saved for future use.
   - Once the policies are active, they will evaluate the conditions and trigger defined actions if matching results are found.

Request headers

▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/policies'
-d '{ "scope" : { "OR" : [ { "OR" : [ null, null ], "SEARCH_FIELD" : "SEARCH_FIELD", "AND" : [ null, null ],
"SEARCH_VALUE" : true }, { "OR" : [ null, null ], "SEARCH_FIELD" : "SEARCH_FIELD", "AND" : [ null, null ],
"SEARCH_VALUE" : true } ], "SEARCH_FIELD" : "SEARCH_FIELD", "AND" : [ { "OR" : [ null, null ], "SEARCH_FIELD" :
"SEARCH_FIELD", "AND" : [ null, null ], "SEARCH_VALUE" : true }, { "OR" : [ null, null ], "SEARCH_FIELD" :
"SEARCH_FIELD", "AND" : [ null, null ], "SEARCH_VALUE" : true } ] }, "name" : "name", "description" : "description",
"assetGroupIds" : [ 0.8008281904610115, 0.8008281904610115 ], "conditions" : { "OR" : [ { "OR" : [ null, null ],
"SEARCH_FIELD" : "SEARCH_FIELD", "AND" : [ null, null ], "SEARCH_VALUE" : true }, { "OR" : [ null, null ],
"SEARCH_FIELD" : "SEARCH_FIELD", "AND" : [ null, null ], "SEARCH_VALUE" : true } ], "SEARCH_FIELD" : "SEARCH_FIELD",
"AND" : [ { "OR" : [ null, null ], "SEARCH_FIELD" : "SEARCH_FIELD", "AND" : [ null, null ], "SEARCH_VALUE" : true },
{ "OR" : [ null, null ], "SEARCH_FIELD" : "SEARCH_FIELD", "AND" : [ null, null ], "SEARCH_VALUE" : true } ],
"SEARCH_VALUE" : true }, "triggers" : { "pr" : { "isEnabled" : true, "actions" : { "reportIssue" : true, "blockPr" :
true, "reportPrComment" : true }, "overrideIssueSeverity" : "Critical" }, "periodic" : { "isEnabled" : true,
"actions" : { "reportIssue" : true }, "overrideIssueSeverity" : "Critical" }, "cicd" : { "isEnabled" : true,
"actions" : { "reportIssue" : true, "blockCicd" : true, "reportCicd" : true }, "overrideIssueSeverity" : "Critical" }
} }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "
{\"name\":\"string\",\"description\":\"string\",\"conditions\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"scope\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"assetGroupIds\":[0.1],\"triggers\":{\"periodic\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true},\"overrideIssueSeverity\":\"Critical\"},\"pr\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockPr\":true,\"reportPrComment\":true},\"overrideIssueSeverity\":\"Critical\"},\"cicd\":
{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockCicd\":true,\"reportCicd\":true},\"overrideIssueSeverity\":\"Critical\"}}}" headers = {
'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
"/public_api/appsec/v1/policies", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/appsec/v1/policies")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "
{\"name\":\"string\",\"description\":\"string\",\"conditions\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
```

```
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"scope\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"assetGroupIds\":[0.1],\"triggers\":{\"periodic\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true},\"overrideIssueSeverity\":\"Critical\"},\"pr\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockPr\":true,\"reportPrComment\":true},\"overrideIssueSeverity\":\"Critical\"},\"cicd\":
{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockCicd\":true,\"reportCicd\":true},\"overrideIssueSeverity\":\"Critical\"}}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "name": "string", "description": "string", "conditions": { "SEARCH_FIELD": "string",
"SEARCH_TYPE": "EQ", "SEARCH_VALUE": true, "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS",
"SEARCH_VALUE": true, "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true,
"AND": [ null ], "OR": [ null ] } ], "OR": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS",
"SEARCH_VALUE": true, "AND": [ null ], "OR": [ null ] } ] } ], "OR": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS",
"SEARCH_VALUE": true, "AND": [ null ], "OR": [ null ] } ], "OR": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ null ], "OR": [ null ] } ] } ] }, "scope": { "SEARCH_FIELD":
"string", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": true, "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS",
"SEARCH_VALUE": true, "AND": [ null ], "OR": [ null ] } ], "OR": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ null ], "OR": [ null ] } ] } ] }, "OR": [ { "SEARCH_FIELD": "string",
"SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ null ], "OR": [ null ] } ], "OR": [ { "SEARCH_FIELD": "string",
"SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ null ], "OR": [ null ] } ] } ] }, "assetGroupIds": [
0.1 ], "triggers": { "periodic": { "isEnabled": true, "actions": { "reportIssue": true }, "overrideIssueSeverity":
"Critical" }, "pr": { "isEnabled": true, "actions": { "reportIssue": true, "blockPr": true, "reportPrComment": true
}, "overrideIssueSeverity": "Critical" }, "cicd": { "isEnabled": true, "actions": { "reportIssue": true, "blockCicd":
true, "reportCicd": true }, "overrideIssueSeverity": "Critical" } } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/appsec/v1/policies");
xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/appsec/v1/policies")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("
{\"name\":\"string\",\"description\":\"string\",\"conditions\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"scope\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
```

```
[null],\"OR\":[null]}]}]},\"assetGroupIds\":[0.1],\"triggers\":{\"periodic\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true},\"overrideIssueSeverity\":\"Critical\"},\"pr\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockPr\":true,\"reportPrComment\":true},\"overrideIssueSeverity\":\"Critical\"},\"cicd\":
{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockCicd\":true,\"reportCicd\":true},\"overrideIssueSeverity\":\"Critical\"}}}")
.asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = [ "name": "string", "description":
"string", "conditions": [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": true, "AND": [ [
"SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ [ "SEARCH_FIELD": "string",
"SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [], "OR": [] ] ], "OR": [ [ "SEARCH_FIELD": "string",
"SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [], "OR": [] ] ] ] ], "OR": [ [ "SEARCH_FIELD":
"string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [], "OR": [] ] ], "OR": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [], "OR": [] ] ] ] ], "scope": [ "SEARCH_FIELD": "string",
"SEARCH_TYPE": "EQ", "SEARCH_VALUE": true, "AND": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS",
"SEARCH_VALUE": true, "AND": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true,
"AND": [], "OR": [] ] ], "OR": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true,
"AND": [], "OR": [] ] ] ] ], "OR": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE":
true, "AND": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [], "OR": []
] ], "OR": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [], "OR": [] ]
] ] ] ], "assetGroupIds": [0.1], "triggers": [ "periodic": [ "isEnabled": true, "actions": ["reportIssue": true],
"overrideIssueSeverity": "Critical" ], "pr": [ "isEnabled": true, "actions": [ "reportIssue": true, "blockPr": true,
"reportPrComment": true ], "overrideIssueSeverity": "Critical" ], "cicd": [ "isEnabled": true, "actions": [
"reportIssue": true, "blockCicd": true, "reportCicd": true ], "overrideIssueSeverity": "Critical" ] ] ] as [String :
Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/appsec/v1/policies")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/policies", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"name\":\"string\",\"description\":\"string\",\"conditions\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"scope\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"assetGroupIds\":[0.1],\"triggers\":{\"periodic\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true},\"overrideIssueSeverity\":\"Critical\"},\"pr\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockPr\":true,\"reportPrComment\":true},\"overrideIssueSeverity\":\"Critical\"},\"cicd\":
{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockCicd\":true,\"reportCicd\":true},\"overrideIssueSeverity\":\"Critical\"}}}",
CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/appsec/v1/policies"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
```

```
{\"name\":\"string\",\"description\":\"string\",\"conditions\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"scope\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"assetGroupIds\":[0.1],\"triggers\":{\"periodic\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true},\"overrideIssueSeverity\":\"Critical\"},\"pr\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockPr\":true,\"reportPrComment\":true},\"overrideIssueSeverity\":\"Critical\"},\"cicd\":
{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockCicd\":true,\"reportCicd\":true},\"overrideIssueSeverity\":\"Critical\"}}}"); CURLcode
ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/policies"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"name\":\"string\",\"description\":\"string\",\"conditions\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"scope\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"assetGroupIds\":[0.1],\"triggers\":{\"periodic\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true},\"overrideIssueSeverity\":\"Critical\"},\"pr\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockPr\":true,\"reportPrComment\":true},\"overrideIssueSeverity\":\"Critical\"},\"cicd\":
{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockCicd\":true,\"reportCicd\":true},\"overrideIssueSeverity\":\"Critical\"}}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```


Body parameters
required
application/json

Define the Application Security policy

▼ namestring

A unique name for the AppSec policy.

▼ descriptionstring

A brief description of the AppSec policy's purpose.

▼ conditionsobjectrequired

Defines the specific criteria (conditions) that will trigger the policy. You can combine multiple conditions to create complex rules for when the policy should be applied. If you combine multiple `Finding Type` values using `OR`, only the common fields across those types will be valid.

Note:

- When used in **AND**, all conditions within this array must be met.
- When used in **OR**, at least one condition within this array must be met.
- Selecting **Finding Type** determines which condition fields you can configure. Each **Finding Type** supports a specific set of condition fields. To see which fields are allowed for each type, see the Supported Condition Fields table.

2.4.3.2 | List AppSec policies

get /public_api/appsec/v1/policies

2.4.3.2.1 | List all or filtered Application Security policies.

2.4.3.2.1.1 | How to access Application Security policies in the UI:

Go to **Modules**, then select **Application Security** > **Policy Management** > **AppSec Policies**.

Required license:

Cortex Cloud Posture Management or Cortex Cloud Runtime Security.

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
Query parameters
▼ findingTypes array[FindingType]

Types of findings the policy applies to

Allowed values: CAS_CI_CD_RISK_SCANNER CAS_CVE_SCANNER CAS_IAC_SCANNER CAS_LICENSE_SCANNER CAS_OPERATIONAL_RISK_SCANNER CAS_SAST_SCANNER CAS_SECRET_SCANNER
▼ actions array[Action]
▼ status PolicyStatus
▼ triggers array[PolicyTrigger]
▼ isCustom Boolean

Indicates if the policy is a custom policy or a system-provided policy

Example: true
CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/policies?findingTypes=&actions=&status=&triggers=&isCustom=true'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/policies?
findingTypes=SOME_ARRAY_VALUE&actions=SOME_ARRAY_VALUE&status=SOME_STRING_VALUE&triggers=SOME_ARRAY_VALUE&isCustom=SOME_BOOLEAN
headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/appsec/v1/policies?
findingTypes=SOME_ARRAY_VALUE&actions=SOME_ARRAY_VALUE&status=SOME_STRING_VALUE&triggers=SOME_ARRAY_VALUE&isCustom=SOME_BOOLEAN
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/appsec/v1/policies?
findingTypes=SOME_ARRAY_VALUE&actions=SOME_ARRAY_VALUE&status=SOME_STRING_VALUE&triggers=SOME_ARRAY_VALUE&isCustom=SOME_BOOLEAN
xhr.setRequestHeader("Authorization",
```

```
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/policies?
findingTypes=SOME_ARRAY_VALUE&actions=SOME_ARRAY_VALUE&status=SOME_STRING_VALUE&triggers=SOME_ARRAY_VALUE&isCustom=SOME_BOOLEAN
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/policies?
findingTypes=SOME_ARRAY_VALUE&actions=SOME_ARRAY_VALUE&status=SOME_STRING_VALUE&triggers=SOME_ARRAY_VALUE&isCustom=SOME_BOOLEAN
as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET"
request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask = session.dataTask(with: request
as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/policies?
findingTypes=SOME_ARRAY_VALUE&actions=SOME_ARRAY_VALUE&status=SOME_STRING_VALUE&triggers=SOME_ARRAY_VALUE&isCustom=SOME_BOOLEAN
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/policies?
findingTypes=SOME_ARRAY_VALUE&actions=SOME_ARRAY_VALUE&status=SOME_STRING_VALUE&triggers=SOME_ARRAY_VALUE&isCustom=SOME_BOOLEAN
struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/policies?
findingTypes=SOME_ARRAY_VALUE&actions=SOME_ARRAY_VALUE&status=SOME_STRING_VALUE&triggers=SOME_ARRAY_VALUE&isCustom=SOME_BOOLEAN
var request = new RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```


Responses

Ok

Body
application/json
[
▶ idstring

Unique identifier for the policy

▶ namestring

Name of the AppSec policy

▶ descriptionstring

Description of the policy

▶ statusstring (Enum)

Indicates whether the policy is currently enabled or disabled.

▶ isCustomboolean

Indicates if the policy is a custom policy or a system-provided policy

▶ triggersobject

Defines when the AppSec policy should be evaluated. At least one of the options must be `true`.

▶ isClonableboolean

Indicates if the policy is cloneable

▶ isEditableboolean

Indicates if the policy is editable

▶ policyTypestring

Indicates if the policy is a **Code Scanner** policy or a **CI/CD Configuration Scanner** policy.

▶ conditionsobjectrequired

Defines the specific criteria that trigger the policy and defines the type of assets to be evaluated by the policy (the scope). You can combine multiple conditions to create complex rules for when the policy should be applied.

Note:

- When used in **AND**, all conditions within this array must be met.
- When used in **OR**, at least one condition within this array must be met.

▶ relatedDetectionRulesarray[string]

List of related detection rules

▶ scopeobjectrequired

Defines the specific criteria that trigger the policy and defines the type of assets to be evaluated by the policy (the scope). You can combine multiple conditions to create complex rules for when the policy should be applied.

Note:

- When used in **AND**, all conditions within this array must be met.
- When used in **OR**, at least one condition within this array must be met.

▶ dateCreatedstring

The date and time when the policy was created

▶ createdBystring

The user or system that created the policy

▶ dateModifiedstring

The date and time when the policy was last modified

▶ modifiedBystring

The user or system that last modified the policy

▶ deletedBystring

The user or system that deleted the policy

▶ dateDeletedstring

The date and time when the policy was deleted

▶ lastTriggeredstring

The date and time when the policy was last triggered

▶ versionnumberdouble

The version of the policy - goes up by one every policy update

▶ assetGroupIdsarray[number]

List of asset groups to which the policy applies. If the array is empty, the policy applies to all asset groups.

]

RESPONSE


```
[ { "id": "example", "name": "example", "description": "example", "status": "disabled", "isCustom": false,
"triggers": { "periodic": { "isEnabled": false, "actions": { "reportIssue": false }, "overrideIssueSeverity":
"Critical" }, "pr": { "isEnabled": false, "actions": { "reportIssue": false, "blockPr": false, "reportPrComment":
false }, "overrideIssueSeverity": "Critical" }, "cicd": { "isEnabled": false, "actions": { "reportIssue": false,
"blockCicd": false, "reportCicd": false }, "overrideIssueSeverity": "Critical" } }, "isClonable": false,
"isEditable": false, "policyType": "example", "conditions": { "SEARCH_FIELD": "example", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": false, "AND": [ { "SEARCH_FIELD": "example", "SEARCH_TYPE": "ARRAY_CONTAINS",
"SEARCH_VALUE": false, "AND": [ {} ], "OR": [ {} ] } ], "OR": [ { "SEARCH_FIELD": "example", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": false, "AND": [ {} ], "OR": [ {} ] } ] }, "relatedDetectionRules": [ "example" ],
"scope": { "SEARCH_FIELD": "example", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": false, "AND": [ {
"SEARCH_FIELD": "example", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": false, "AND": [ {} ], "OR": [ {} ] } ],
```

"OR": [{ "SEARCH_FIELD": "example", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": false, "AND": [{}], "OR": [{}] }] }, "dateCreated": "example", "createdBy": "example", "dateModified": "example", "modifiedBy": "example", "deletedBy": "example", "dateDeleted": "example", "lastTriggered": "example", "version": 0.1, "assetGroupIds": [0.1] }]

2.4.3.3 | Get an AppSec policy

get /public_api/appsec/v1/policies/{policyId}

A Cortex Cloud Application Security policy defines how a system should respond to application security threats.

2.4.3.3.1 | Get the details of the Application Security policy specified by its policy ID.

2.4.3.3.1.1 | How to access Application Security policies in the UI:

Go to **Modules**, then select **Application Security** > **Policy Management** > **AppSec Policies**.

Required license:

Cortex Cloud Posture Management or Cortex Cloud Runtime Security.

Path parameters
▼ policyId String required
Example: `policyId_example`
Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/policies/{policyId}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/policies/%7BpolicyId%7D", headers=headers) res
= conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D"); var request = new
RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
▼ idstring

Unique identifier for the policy

▼ namestring

Name of the AppSec policy

▼ descriptionstring

Description of the policy

▼ statusstring (Enum)

Indicates whether the policy is currently enabled or disabled.

Default:`"enabled"`
Allowed values:"disabled""enabled"
▼ isCustomboolean

Indicates if the policy is a custom policy or a system-provided policy

▼ triggersobject

Defines when the AppSec policy should be evaluated. At least one of the options must be `true`.

▶ periodicobjectrequired

If `true`, the AppSec policy is evaluated periodically (for example, daily or weekly).

▶ probjectrequired

If `true`, the AppSec policy is evaluated on Pull Request (PR) events

▶ cicdobjectrequired

If `true`, the AppSec policy is evaluated on CI/CD pipeline events.

▼ isClonableboolean

Indicates if the policy is cloneable

▼ isEditableboolean

Indicates if the policy is editable

▼ policyTypestring

Indicates if the policy is a Code Scanner policy or a CI/CD Configuration Scanner policy.

▼ conditionsobjectrequired

Defines the specific criteria that trigger the policy and defines the type of assets to be evaluated by the policy (the scope). You can combine multiple conditions to create complex rules for when the policy should be applied.

Note:

- When used in **AND**, all conditions within this array must be met.
- When used in **OR**, at least one condition within this array must be met.

▶ SEARCH_FIELDstring

Defines the field the condition filter should match.

▶ SEARCH_TYPEstring (Enum)

Defines the comparison operator you want to use for this filter.

▶ SEARCH_VALUEobject

Value that the condition filter must match. The type of this field will differ depending on the `SEARCH_FIELD` that you specified.

▶ ANDarray
▶ ORarray
▼ relatedDetectionRulesarray[string]

List of related detection rules

▼ scopeobjectrequired

Defines the specific criteria that trigger the policy and defines the type of assets to be evaluated by the policy (the scope). You can combine multiple conditions to create complex rules for when the policy should be applied.

Note:

- When used in **AND**, all conditions within this array must be met.
- When used in **OR**, at least one condition within this array must be met.

▶ SEARCH_FIELDstring

Defines the field the condition filter should match.

▶ SEARCH_TYPEstring (Enum)

Defines the comparison operator you want to use for this filter.

▶ SEARCH_VALUEobject

Value that the condition filter must match. The type of this field will differ depending on the `SEARCH_FIELD` that you specified.

▶ ANDarray
▶ ORarray
▼ dateCreatedstring

The date and time when the policy was created

▼ createdBystring

The user or system that created the policy

▼ dateModifiedstring

The date and time when the policy was last modified

▼ modifiedBystring

The user or system that last modified the policy

▼ deletedBystring

The user or system that deleted the policy

▼ dateDeletedstring

The date and time when the policy was deleted

▼ lastTriggeredstring

The date and time when the policy was last triggered

▼ versionnumberdouble

The version of the policy - goes up by one every policy update

▼ assetGroupIdsarray[number]

List of asset groups to which the policy applies. If the array is empty, the policy applies to all asset groups.
RESPONSE

```
{ "id": "example", "name": "example", "description": "example", "status": "disabled", "isCustom": false, "triggers":
{ "periodic": { "isEnabled": false, "actions": { "reportIssue": false }, "overrideIssueSeverity": "Critical" }, "pr":
{ "isEnabled": false, "actions": { "reportIssue": false, "blockPr": false, "reportPrComment": false },
"overrideIssueSeverity": "Critical" }, "cicd": { "isEnabled": false, "actions": { "reportIssue": false, "blockCicd":
false, "reportCicd": false }, "overrideIssueSeverity": "Critical" } }, "isClonable": false, "isEditable": false,
"policyType": "example", "conditions": { "SEARCH_FIELD": "example", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE":
false, "AND": [ { "SEARCH_FIELD": "example", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": false, "AND": [ {} ],
"OR": [ {} ] } ], "OR": [ { "SEARCH_FIELD": "example", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": false, "AND":
[ {} ], "OR": [ {} ] } ] }, "relatedDetectionRules": [ "example" ], "scope": { "SEARCH_FIELD": "example",
"SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": false, "AND": [ { "SEARCH_FIELD": "example", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": false, "AND": [ {} ], "OR": [ {} ] } ], "OR": [ { "SEARCH_FIELD": "example",
"SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": false, "AND": [ {} ], "OR": [ {} ] } ] }, "dateCreated": "example",
"createdBy": "example", "dateModified": "example", "modifiedBy": "example", "deletedBy": "example", "dateDeleted":
"example", "lastTriggered": "example", "version": 0.1, "assetGroupIds": [ 0.1 ] }
```

2.4.3.4 | Delete an AppSec policy

delete /public_api/appsec/v1/policies/{policyId}

A Cortex Cloud Application Security policy defines how a system should respond to application security threats.

2.4.3.4.1 | Delete an Application Security policy by specifying its policy ID.

2.4.3.4.1.1 | How to access Application Security policies in the UI:

Go to Modules > Application Security > Policy Management > AppSec Policies.

Required license:

Cortex Cloud Posture Management or Cortex Cloud Runtime Security.

Path parameters
▼ policyId String required
Example: `policyId_example`
Request headers
▼ Authorization String required

{api_key}

Example:

`UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W`

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl

```
curl -X 'DELETE'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/policies/{policyId}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("DELETE", "/public_api/appsec/v1/policies/%7BpolicyId%7D", headers=headers)
res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Delete.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("DELETE", "https://api-
yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.delete("https://api-yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "DELETE" request.allHTTPHeaderFields = headers let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "DELETE", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "DELETE"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D"); var request = new
RestRequest(Method.DELETE); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
▼ messagestring
RESPONSE

```
{ "message": "example" }
```

2.4.3.5 | Update an AppSec policy

put /public_api/appsec/v1/policies/{policyId}

A Cortex Cloud Application Security policy defines how a system should respond to application security threats.

2.4.3.5.1 | Update the details of the Application Security policy specified by its policy ID.

2.4.3.5.1.1 | How to access Application Security policies in the UI:

Go to **Modules**, then select **Application Security** > **Policy Management** > **AppSec Policies**.

Required license:

Cortex Cloud Posture Management or Cortex Cloud Runtime Security.

Path parameters
▼ policyId String required
Example: `policyId_example`
Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: **241**

CLIENT REQUEST Bash+curl

```
curl -X 'PUT'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/policies/{policyId}'
-d '{ "scope" : { "OR" : [ { "OR" : [ null, null ], "SEARCH_FIELD" : "SEARCH_FIELD", "AND" : [ null, null ],
"SEARCH_VALUE" : true }, { "OR" : [ null, null ], "SEARCH_FIELD" : "SEARCH_FIELD", "AND" : [ null, null ],
"SEARCH_VALUE" : true } ], "SEARCH_FIELD" : "SEARCH_FIELD", "AND" : [ { "OR" : [ null, null ], "SEARCH_FIELD" :
"SEARCH_FIELD", "AND" : [ null, null ], "SEARCH_VALUE" : true }, { "OR" : [ null, null ], "SEARCH_FIELD" :
"SEARCH_FIELD", "AND" : [ null, null ], "SEARCH_VALUE" : true } ] }, "name" : "name", "description" : "description",
"assetGroupIds" : [ 0.8008281904610115, 0.8008281904610115 ], "conditions" : { "OR" : [ { "OR" : [ null, null ],
"SEARCH_FIELD" : "SEARCH_FIELD", "AND" : [ null, null ], "SEARCH_VALUE" : true }, { "OR" : [ null, null ],
"SEARCH_FIELD" : "SEARCH_FIELD", "AND" : [ null, null ], "SEARCH_VALUE" : true } ], "SEARCH_FIELD" : "SEARCH_FIELD",
"AND" : [ { "OR" : [ null, null ], "SEARCH_FIELD" : "SEARCH_FIELD", "AND" : [ null, null ], "SEARCH_VALUE" : true },
{ "OR" : [ null, null ], "SEARCH_FIELD" : "SEARCH_FIELD", "AND" : [ null, null ], "SEARCH_VALUE" : true } ],
"SEARCH_VALUE" : true }, "triggers" : { "pr" : { "isEnabled" : true, "actions" : { "reportIssue" : true, "blockPr" :
true, "reportPrComment" : true }, "overrideIssueSeverity" : "Critical" }, "periodic" : { "isEnabled" : true,
"actions" : { "reportIssue" : true }, "overrideIssueSeverity" : "Critical" }, "cicd" : { "isEnabled" : true,
"actions" : { "reportIssue" : true, "blockCicd" : true, "reportCicd" : true }, "overrideIssueSeverity" : "Critical" }
} }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "
{\"name\":\"string\",\"description\":\"string\",\"conditions\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"scope\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"assetGroupIds\":[0.1],\"triggers\":{\"periodic\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true},\"overrideIssueSeverity\":\"Critical\"},\"pr\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockPr\":true,\"reportPrComment\":true},\"overrideIssueSeverity\":\"Critical\"},\"cicd\":
{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockCicd\":true,\"reportCicd\":true},\"overrideIssueSeverity\":\"Critical\"}}}" headers = {
'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("PUT",
"/public_api/appsec/v1/policies/%7BpolicyId%7D", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Put.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "
{\"name\":\"string\",\"description\":\"string\",\"conditions\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"scope\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
```

```
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"assetGroupIds\":[0.1],\"triggers\":{\"periodic\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true},\"overrideIssueSeverity\":\"Critical\"},\"pr\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockPr\":true,\"reportPrComment\":true},\"overrideIssueSeverity\":\"Critical\"},\"cicd\":
{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockCicd\":true,\"reportCicd\":true},\"overrideIssueSeverity\":\"Critical\"}}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "name": "string", "description": "string", "conditions": { "SEARCH_FIELD": "string",
"SEARCH_TYPE": "EQ", "SEARCH_VALUE": true, "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS",
"SEARCH_VALUE": true, "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true,
"AND": [ null ], "OR": [ null ] } ], "OR": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS",
"SEARCH_VALUE": true, "AND": [ null ], "OR": [ null ] } ] } ], "OR": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS",
"SEARCH_VALUE": true, "AND": [ null ], "OR": [ null ] } ], "OR": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ null ], "OR": [ null ] } ] } ] }, "scope": { "SEARCH_FIELD":
"string", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": true, "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS",
"SEARCH_VALUE": true, "AND": [ null ], "OR": [ null ] } ], "OR": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ null ], "OR": [ null ] } ] } ] }, "OR": [ { "SEARCH_FIELD":
"string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ null ], "OR": [ null ] } ], "OR": [ { "SEARCH_FIELD": "string",
"SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ null ], "OR": [ null ] } ] } ] }, "assetGroupIds": [
0.1 ], "triggers": { "periodic": { "isEnabled": true, "actions": { "reportIssue": true }, "overrideIssueSeverity":
"Critical" }, "pr": { "isEnabled": true, "actions": { "reportIssue": true, "blockPr": true, "reportPrComment": true
}, "overrideIssueSeverity": "Critical" }, "cicd": { "isEnabled": true, "actions": { "reportIssue": true, "blockCicd":
true, "reportCicd": true }, "overrideIssueSeverity": "Critical" } } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("PUT", "https://api-
yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.put("https://api-yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("
{\"name\":\"string\",\"description\":\"string\",\"conditions\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"scope\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"assetGroupIds\":[0.1],\"triggers\":{\"periodic\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true},\"overrideIssueSeverity\":\"Critical\"},\"pr\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockPr\":true,\"reportPrComment\":true},\"overrideIssueSeverity\":\"Critical\"},\"cicd\":
{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockCicd\":true,\"reportCicd\":true},\"overrideIssueSeverity\":\"Critical\"}}}")
.asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = [ "name": "string", "description":
```

```
"string", "conditions": [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": true, "AND": [ [
"SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ [ "SEARCH_FIELD": "string",
"SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [], "OR": [] ] ], "OR": [ [ "SEARCH_FIELD": "string",
"SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [], "OR": [] ] ] ] ], "OR": [ [ "SEARCH_FIELD":
"string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [], "OR": [] ] ], "OR": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE":
"ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [], "OR": [] ] ] ] ], "scope": [ "SEARCH_FIELD": "string",
"SEARCH_TYPE": "EQ", "SEARCH_VALUE": true, "AND": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS",
"SEARCH_VALUE": true, "AND": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true,
"AND": [], "OR": [] ] ], "OR": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true,
"AND": [], "OR": [] ] ] ] ], "OR": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE":
true, "AND": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [], "OR": []
] ], "OR": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "ARRAY_CONTAINS", "SEARCH_VALUE": true, "AND": [], "OR": [] ]
] ] ] ], "assetGroupIds": [0.1], "triggers": [ "periodic": [ "isEnabled": true, "actions": ["reportIssue": true],
"overrideIssueSeverity": "Critical" ], "pr": [ "isEnabled": true, "actions": [ "reportIssue": true, "blockPr": true,
"reportPrComment": true ], "overrideIssueSeverity": "Critical" ], "cicd": [ "isEnabled": true, "actions": [
"reportIssue": true, "blockCicd": true, "reportCicd": true ], "overrideIssueSeverity": "Critical" ] ] ] as [String :
Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "PUT" request.allHTTPHeaderFields =
headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "PUT", CURLOPT_POSTFIELDS => "{\"name\":\"string\",\"description\":\"string\",\"conditions\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"scope\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"assetGroupIds\":[0.1],\"triggers\":{\"periodic\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true},\"overrideIssueSeverity\":\"Critical\"},\"pr\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockPr\":true,\"reportPrComment\":true},\"overrideIssueSeverity\":\"Critical\"},\"cicd\":
{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockCicd\":true,\"reportCicd\":true},\"overrideIssueSeverity\":\"Critical\"}}}",
CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "PUT"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"name\":\"string\",\"description\":\"string\",\"conditions\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
```

```
[null],\"OR\":[null]}]}]},\"scope\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"assetGroupIds\":[0.1],\"triggers\":{\"periodic\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true},\"overrideIssueSeverity\":\"Critical\"},\"pr\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockPr\":true,\"reportPrComment\":true},\"overrideIssueSeverity\":\"Critical\"},\"cicd\":
{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockCicd\":true,\"reportCicd\":true},\"overrideIssueSeverity\":\"Critical\"}}}"); CURLcode
ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/policies/%7BpolicyId%7D"); var request = new
RestRequest(Method.PUT); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"name\":\"string\",\"description\":\"string\",\"conditions\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"scope\":
{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}],\"OR\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":[null],\"OR\":
[null]}],\"OR\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"ARRAY_CONTAINS\",\"SEARCH_VALUE\":true,\"AND\":
[null],\"OR\":[null]}]}]},\"assetGroupIds\":[0.1],\"triggers\":{\"periodic\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true},\"overrideIssueSeverity\":\"Critical\"},\"pr\":{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockPr\":true,\"reportPrComment\":true},\"overrideIssueSeverity\":\"Critical\"},\"cicd\":
{\"isEnabled\":true,\"actions\":
{\"reportIssue\":true,\"blockCicd\":true,\"reportCicd\":true},\"overrideIssueSeverity\":\"Critical\"}}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
Partial_BasePolicyRequest
required
application/json

Make all properties in T optional

▼ namestring

A unique name for the AppSec policy.

▼ descriptionstring

A brief description of the AppSec policy's purpose.

▼ conditionsobjectrequired

Defines the specific criteria (conditions) that will trigger the policy. You can combine multiple conditions to create complex rules for when the policy should be applied. If you combine multiple `Finding Type` values using `OR`, only the common fields across those types will be valid.

Note:

- When used in `AND`, all conditions within this array must be met.
- When used in `OR`, at least one condition within this array must be met.
- Selecting `Finding Type` determines which condition fields you can configure. Each `Finding Type` supports a specific set of condition fields. To see which fields are allowed for each type, see the Supported Condition Fields table.

2.4.3.6 | Supported Condition Fields

▼

This table defines the available fields for building policy conditions when you select `Finding Type` option.

Field Name (SEARCH_FIELD)	Pretty Name (FIELD_PRETTY_NAME)	Data Type (DATA_TYPE)	Allowed SEARCH_TYPE Values	Possible Values (ENUM
Severity	Severity	ENUM	EQ, NEQ	CRITICAL, HIGH, MEDIU
Finding Type	Finding Type	ENUM	EQ	CAS_CI_CD_RISK_SCAN CAS_CVE_SCANNER, CAS_IAC_SCANNER, CAS_LICENSE_SCANNE CAS_OPERATIONAL_RIS CAS_SECRET_SCANNE CAS_SAST_SCANNER
Backlog Status	Backlog Status	ENUM	EQ, NEQ	BACKLOG, NEW
Respect Developer Suppression	Respect Dev Suppression	BOOLEAN	EQ	true, false
Category	Category	ENUM	EQ, NEQ	(See Scope Categories: APPLICATION, REPOSIT
Provider	Provider	ENUM	EQ, NEQ	GITLAB_CI, AZURE_PIP GITHUB_ACTIONS, CIR JENKINS
Subcategory	Subcategory	ENUM	EQ, NEQ	THIRD_PARTY_SERVICE CREDENTIAL_HYGIENE DATA_PROTECTION, PP
AppSec Rule	AppSec Rule	ENUM	EQ, NEQ	(Varies, specific rule IDs/
AppSec Rule Label	AppSec Rule Label	ENUM	EQ, NEQ	(Varies, specific label val
RiskFactors	Risk Factors	ENUM	EQ, NEQ	Critical severity, Has fix, execution, Exploit exists
CVE ID	CVE ID	STRING	EQ, NEQ, CONTAINS	(No predefined list)
CVSS	CVSS Score	INT	EQ, NEQ, LT, GT, LTE, GTE	(No predefined list)

Field Name (SEARCH_FIELD)	Pretty Name (FIELD_PRETTY_NAME)	Data Type (DATA_TYPE)	Allowed SEARCH_TYPE Values	Possible Values (ENUM
EPSS	EPSS Score	INT	EQ, NEQ, LT, GT, LTE, GTE	(No predefined list)
CVE Publish Date	CVE Publish Date	DATE	EQ, NEQ, LT, GT, LTE, GTE	(No predefined list)
CVE Fix Available Date	CVE Fix Available Date	DATE	EQ, NEQ, LT, GT, LTE, GTE	(No predefined list)
Package Name	Package Name	STRING	EQ, NEQ, CONTAINS	(No predefined list)
Package Version	Package Version	STRING	EQ, NEQ, LT, GT	(No predefined list)
Package Popularity	Package Popularity	ENUM	EQ, NEQ	LOW, MEDIUM, HIGH
Package Maintained	Package Maintained	ENUM	EQ, NEQ	LOW, MEDIUM, HIGH
Package Deprecated	Package Deprecated	BOOLEAN	EQ	true, false
Package Operational Risk	Package Operational Risk	ENUM	EQ, NEQ	LOW, MEDIUM, HIGH
HasAFix	Has A Fix	BOOLEAN	EQ	true, false
IsKev	Is KEV	BOOLEAN	EQ	true, false
HasAnAutomatedFix	Has Automated Fix	BOOLEAN	EQ	true, false

Field Name (SEARCH_FIELD)	Pretty Name (FIELD_PRETTY_NAME)	Data Type (DATA_TYPE)	Allowed SEARCH_TYPE Values	Possible Values (ENUM
IacTag	IaC Tag	STRING	EQ, NEQ, CONTAINS	(No predefined list)
LicenseType	License Type	ENUM	EQ, NEQ	(Specific license short na MIT, GPL-3.0)
Popularity	Popularity	ENUM	EQ, NEQ	LOW, MEDIUM, HIGH
Maintained	Maintained	ENUM	EQ, NEQ	LOW, MEDIUM, HIGH
SoftwarePackageName	Software Package Name	STRING	EQ, NEQ, CONTAINS	(No predefined list)
SoftwarePackageVersion	Software Package Version	STRING	EQ, NEQ, CONTAINS	(No predefined list)
SecretValidity	Secret Validity	ENUM	EQ, NEQ	NO_VALIDATION, PRIVIL VALID, INVALID, UNAVA
CWE	CWE ID	STRING	EQ, NEQ, CONTAINS	(No predefined list)
OwaspCategory	OWASP Category	STRING	EQ, NEQ, CONTAINS	(No predefined list)
Source	Source	STRING	EQ, NEQ, CONTAINS	(No predefined list)
Language	Language	STRING	EQ, NEQ, CONTAINS	(No predefined list)

▶ SEARCH_FIELDstring

Defines the field the condition filter should match. To see which fields are allowed for each type, see the Supported Condition Fields table.

▶ SEARCH_TYPEstring (Enum)

Defines the comparison operator you want to use for this filter.

▶ SEARCH_VALUEobject

Value that the condition filter must match. The type of this field will differ depending on the `SEARCH_FIELD` that you specified.

▶ ANDarray

▶ ORarray
▼ scopeobjectrequired

Defines the type of assets to be evaluated by the policy (the scope). You can combine multiple conditions to create complex rules for when the policy should be applied.

Note:

- When used in **AND**, all conditions within this array must be met.
- When used in **OR**, at least one condition within this array must be met.
- Cannot be used if `assetGroupIds` are set.

2.4.3.7 | Supported Scope Fields

▼

This table defines the available fields for building policy scopes that target specific assets.

Field Name (SEARCH_FIELD)	Pretty Name (FIELD_PRETTY_NAME)	Data Type (DATA_TYPE)	Allowed SEARCH_TYPE Values	Possible Values (ENUM
Category	Category	ENUM	EQ, NEQ	APPLICATION, REPOSIT CICD_INSTANCE, CICD_ VCS_COLLABORATOR, VCS_ORGANIZATION
Provider	Provider	ENUM	EQ, NEQ	GITHUB, GITLAB, AZUR BITBUCKET, AWS_CODE_COMMIT, e
Application Business Criticality	App Business Criticality	ENUM	EQ, NEQ	CRITICAL, HIGH, MEDIU
Application Business Owner	App Business Owner	STRING	EQ, NEQ, CONTAINS	(No predefined list)
Business Application Names	Business App Names	STRING	EQ, NEQ, CONTAINS	(No predefined list)
Repository Name	Repository Name	STRING	EQ, NEQ, CONTAINS	(No predefined list)
Repository Id	Repository ID	STRING	EQ, NEQ, CONTAINS	(No predefined list)
Is Public Repository	Is Public Repository	BOOLEAN	EQ	true, false
Has Deployed Assets	Has Deployed Assets	BOOLEAN	EQ	true, false
Has Internet-exposed deployed assets	Has Internet Exposed Assets	BOOLEAN	EQ	true, false

Field Name (SEARCH_FIELD)	Pretty Name (FIELD_PRETTY_NAME)	Data Type (DATA_TYPE)	Allowed SEARCH_TYPE Values	Possible Values (ENUM
Has deployed assets with Access to sensitive data	Has Assets with Sensitive Data Access	BOOLEAN	EQ	true, false
Has deployed assets with privileged capabilities	Has Assets with Privileged Capabilities	BOOLEAN	EQ	true, false

▶ SEARCH_FIELDstring

Defines the field the matching criteria filter should match. To see which fields are allowed for each type, see the Supported Scope Fields table.

▶ SEARCH_TYPEstring (Enum)

Defines the comparison operator you want to use for this filter.

▶ SEARCH_VALUEobject

Value that the condition filter must match. The type of this field will differ depending on the `SEARCH_FIELD` that you specified.

▶ ANDarray
▶ ORarray
▼ assetGroupIdsarray[number]

A list of asset group IDs that the policy should apply to. If the list is empty, the policy applies globally, unless other scope conditions are defined. **Note** : Cannot be used with the `scope` parameter.

▼ triggersobject

Defines when the AppSec policy should be evaluated. At least one of the options must be `true`.

▶ periodicobjectrequired

If `true`, the AppSec policy is evaluated periodically (for example, daily or weekly).

▶ probjectrequired

If `true`, the AppSec policy is evaluated on Pull Request (PR) events

▶ cicdobjectrequired

If `true`, the AppSec policy is evaluated on CI/CD pipeline events.

REQUEST Example 1


```
{ "name": "Test code-scanners policy type", "description": "Critical issues found on deployed assets policy.",
"conditions": { "AND": [ { "OR": [ { "SEARCH_FIELD": "Finding Type", "SEARCH_TYPE": "EQ", "SEARCH_VALUE":
"CAS_CVE_SCANNER" }, { "SEARCH_FIELD": "Finding Type", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "CAS_IAC_SCANNER" }, {
"SEARCH_FIELD": "Finding Type", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "CAS_LICENSE_SCANNER" }, { "SEARCH_FIELD":
"Finding Type", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "CAS_OPERATIONAL_RISK_SCANNER" }, { "SEARCH_FIELD": "Finding
Type", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "CAS_SECRET_SCANNER" }, { "SEARCH_FIELD": "Finding Type", "SEARCH_TYPE":
"EQ", "SEARCH_VALUE": "CAS_SAST_SCANNER" } ] }, { "SEARCH_FIELD": "Severity", "SEARCH_TYPE": "EQ", "SEARCH_VALUE":
"CRITICAL" } ] }, "scope": { "AND": [ { "SEARCH_FIELD": "has_deployed_assets", "SEARCH_TYPE": "EQ", "SEARCH_VALUE":
true } ] }, "assetGroupIds": [], "triggers": { "periodic": { "isEnabled": true, "actions": { "reportIssue": true },
"overrideIssueSeverity": "Medium" }, "pr": { "isEnabled": true, "actions": { "reportIssue": true, "blockPr": true,
"reportPrComment": true }, "overrideIssueSeverity": "Critical" }, "cicd": { "isEnabled": true, "actions": {
"reportIssue": true, "blockCicd": true, "reportCicd": true }, "overrideIssueSeverity": "High" } } }
{ "name": "test cicdscanners policy type", "description": "Critical issues found on deployed assets policy.",
"conditions": { "AND": [ { "SEARCH_FIELD": "Finding Type", "SEARCH_TYPE": "EQ", "SEARCH_VALUE":
"CAS_CI_CD_RISK_SCANNER" } ] }, "scope": { "OR": [ { "AND": [ { "OR": [ { "SEARCH_FIELD": "category", "SEARCH_TYPE":
"EQ", "SEARCH_VALUE": "APPLICATION" }, { "SEARCH_FIELD": "category", "SEARCH_TYPE": "EQ", "SEARCH_VALUE":
"CICD_INSTANCE" }, { "SEARCH_FIELD": "category", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "CICD_PIPELINE" }, {
"SEARCH_FIELD": "category", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "VCS_COLLABORATOR" }, { "SEARCH_FIELD": "category",
"SEARCH_TYPE": "EQ", "SEARCH_VALUE": "VCS_ORGANIZATION" } ] } ] }, { "AND": [ { "SEARCH_FIELD":
"application_business_criticality", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "CRITICAL" }, { "OR": [ { "SEARCH_FIELD":
"repository_provider", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "AWS_CODE_COMMIT" }, { "SEARCH_FIELD":
"repository_provider", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "AWS_CODE_BUILD" } ] } ] } ] }, "assetGroupIds": [],
```

"triggers": { "periodic": { "isEnabled": true, "actions": { "reportIssue": true }, "overrideIssueSeverity": "High" }, "pr": { "isEnabled": false, "actions": { "reportIssue": false, "blockPr": false, "reportPrComment": false }, "overrideIssueSeverity": null }, "cicd": { "isEnabled": false, "actions": { "reportIssue": false, "blockCicd": false, "reportCicd": false }, "overrideIssueSeverity": null } } }

Responses

No content

You will receive **Bad Request** error when the server cannot process the request due to client-side errors in the request body. The error is returned when:

- Required fields are missing
- Field values are invalid (for example, incorrect `SEARCH_TYPE` for the given `SEARCH_FIELD`)
- The JSON request body is not structured properly (invalid payload structure)

Example response:

```
{
  "error": "Bad Request",
  "message": "Invalid value for 'name': must not be empty."
}
```

Resolution

Ensure that:

- All required fields are included
- Field values follow the expected format and constraints
- The JSON request body is properly structured

You will receive **Conflict** error if a policy with the specified name already exists.

Example response:

```
{
  "error": "Conflict",
  "message": "A policy with the name 'test_cicdscanners' already exists."
}
```

Resolution

Provide a unique name for the new policy.

You will receive **Internal Server Error** when the server encounters an unexpected condition that prevents it from fulfilling the request.

Example response:

```
{
  "error": "Internal Server Error",
  "message": "An unexpected error occurred. Please try again later."
}
```

Resolution

This is a server-side issue. No action is required from you. If the problem persists, try again later.

2.4.3.8 | Supported Condition Fields

▼

This table defines the available fields for building policy conditions when you select `Finding Type` option.

Field Name (SEARCH_FIELD)	Pretty Name (FIELD_PRETTY_NAME)	Data Type (DATA_TYPE)	Allowed SEARCH_TYPE Values	Possible Values (ENUM
Severity	Severity	ENUM	EQ, NEQ	CRITICAL, HIGH, MEDIU
Finding Type	Finding Type	ENUM	EQ	CAS_CI_CD_RISK_SCAN CAS_CVE_SCANNER, CAS_IAC_SCANNER, CAS_LICENSE_SCANNE CAS_OPERATIONAL_RIS CAS_SECRET_SCANNE CAS_SAST_SCANNER

Field Name (SEARCH_FIELD)	Pretty Name (FIELD_PRETTY_NAME)	Data Type (DATA_TYPE)	Allowed SEARCH_TYPE Values	Possible Values (ENUM
Backlog Status	Backlog Status	ENUM	EQ, NEQ	BACKLOG, NEW
Respect Developer Suppression	Respect Dev Suppression	BOOLEAN	EQ	true, false
Category	Category	ENUM	EQ, NEQ	(See Scope Categories: APPLICATION, REPOSIT
Provider	Provider	ENUM	EQ, NEQ	GITLAB_CI, AZURE_PIP GITHUB_ACTIONS, CIR JENKINS
Subcategory	Subcategory	ENUM	EQ, NEQ	THIRD_PARTY_SERVICE CREDENTIAL_HYGIENE DATA_PROTECTION, PP
AppSec Rule	AppSec Rule	ENUM	EQ, NEQ	(Varies, specific rule IDs/
AppSec Rule Label	AppSec Rule Label	ENUM	EQ, NEQ	(Varies, specific label va
RiskFactors	Risk Factors	ENUM	EQ, NEQ	Critical severity, Has fix, execution, Exploit exists
CVE ID	CVE ID	STRING	EQ, NEQ, CONTAINS	(No predefined list)
CVSS	CVSS Score	INT	EQ, NEQ, LT, GT, LTE, GTE	(No predefined list)
EPSS	EPSS Score	INT	EQ, NEQ, LT, GT, LTE, GTE	(No predefined list)
CVE Publish Date	CVE Publish Date	DATE	EQ, NEQ, LT, GT, LTE, GTE	(No predefined list)
CVE Fix Available Date	CVE Fix Available Date	DATE	EQ, NEQ, LT, GT, LTE, GTE	(No predefined list)

Field Name (SEARCH_FIELD)	Pretty Name (FIELD_PRETTY_NAME)	Data Type (DATA_TYPE)	Allowed SEARCH_TYPE Values	Possible Values (ENUM
Package Name	Package Name	STRING	EQ, NEQ, CONTAINS	(No predefined list)
Package Version	Package Version	STRING	EQ, NEQ, LT, GT	(No predefined list)
Package Popularity	Package Popularity	ENUM	EQ, NEQ	LOW, MEDIUM, HIGH
Package Maintained	Package Maintained	ENUM	EQ, NEQ	LOW, MEDIUM, HIGH
Package Deprecated	Package Deprecated	BOOLEAN	EQ	true, false
Package Operational Risk	Package Operational Risk	ENUM	EQ, NEQ	LOW, MEDIUM, HIGH
HasAFix	Has A Fix	BOOLEAN	EQ	true, false
IsKev	Is KEV	BOOLEAN	EQ	true, false
HasAnAutomatedFix	Has Automated Fix	BOOLEAN	EQ	true, false
IacTag	IaC Tag	STRING	EQ, NEQ, CONTAINS	(No predefined list)
LicenseType	License Type	ENUM	EQ, NEQ	(Specific license short na MIT, GPL-3.0)
Popularity	Popularity	ENUM	EQ, NEQ	LOW, MEDIUM, HIGH

Field Name (SEARCH_FIELD)	Pretty Name (FIELD_PRETTY_NAME)	Data Type (DATA_TYPE)	Allowed SEARCH_TYPE Values	Possible Values (ENUM
Maintained	Maintained	ENUM	EQ, NEQ	LOW, MEDIUM, HIGH
SoftwarePackageName	Software Package Name	STRING	EQ, NEQ, CONTAINS	(No predefined list)
SoftwarePackageVersion	Software Package Version	STRING	EQ, NEQ, CONTAINS	(No predefined list)
SecretValidity	Secret Validity	ENUM	EQ, NEQ	NO_VALIDATION, PRIVIL VALID, INVALID, UNAVA
CWE	CWE ID	STRING	EQ, NEQ, CONTAINS	(No predefined list)
OwaspCategory	OWASP Category	STRING	EQ, NEQ, CONTAINS	(No predefined list)
Source	Source	STRING	EQ, NEQ, CONTAINS	(No predefined list)
Language	Language	STRING	EQ, NEQ, CONTAINS	(No predefined list)

▶ SEARCH_FIELDstring

Defines the field the condition filter should match. To see which fields are allowed for each type, see the Supported Condition Fields table.

▶ SEARCH_TYPEstring (Enum)

Defines the comparison operator you want to use for this filter.

▶ SEARCH_VALUEobject

Value that the condition filter must match. The type of this field will differ depending on the `SEARCH_FIELD` that you specified.

▶ ANDarray
▶ ORarray
▼ scopeobjectrequired

Defines the type of assets to be evaluated by the policy (the scope). You can combine multiple conditions to create complex rules for when the policy should be applied.

Note:

- When used in **`AND`**, all conditions within this array must be met.
- When used in `OR`, at least one condition within this array must be met.
- Cannot be used if `assetGroupIds` are set.

2.4.3.9 | Supported Scope Fields

▼

This table defines the available fields for building policy scopes that target specific assets.

Field Name (SEARCH_FIELD)	Pretty Name (FIELD_PRETTY_NAME)	Data Type (DATA_TYPE)	Allowed SEARCH_TYPE Values	Possible Values (ENUM
Category	Category	ENUM	EQ, NEQ	APPLICATION, REPOSIT CICD_INSTANCE, CICD_ VCS_COLLABORATOR, VCS_ORGANIZATION
Provider	Provider	ENUM	EQ, NEQ	GITHUB, GITLAB, AZUR BITBUCKET, AWS_CODE_COMMIT, e
Application Business Criticality	App Business Criticality	ENUM	EQ, NEQ	CRITICAL, HIGH, MEDIU
Application Business Owner	App Business Owner	STRING	EQ, NEQ, CONTAINS	(No predefined list)
Business Application Names	Business App Names	STRING	EQ, NEQ, CONTAINS	(No predefined list)
Repository Name	Repository Name	STRING	EQ, NEQ, CONTAINS	(No predefined list)
Repository Id	Repository ID	STRING	EQ, NEQ, CONTAINS	(No predefined list)
Is Public Repository	Is Public Repository	BOOLEAN	EQ	true, false
Has Deployed Assets	Has Deployed Assets	BOOLEAN	EQ	true, false
Has Internet-exposed deployed assets	Has Internet Exposed Assets	BOOLEAN	EQ	true, false
Has deployed assets with Access to sensitive data	Has Assets with Sensitive Data Access	BOOLEAN	EQ	true, false
Has deployed assets with privileged capabilities	Has Assets with Privileged Capabilities	BOOLEAN	EQ	true, false

▶ SEARCH_FIELDstring

Defines the field the matching criteria filter should match. To see which fields are allowed for each type, see the Supported Scope Fields table.

▶ SEARCH_TYPEstring (Enum)

Defines the comparison operator you want to use for this filter.

▶ SEARCH_VALUEobject

Value that the condition filter must match. The type of this field will differ depending on the `SEARCH_FIELD` that you specified.

▶ ANDarray
▶ ORarray
▼ assetGroupIdsarray[number]

List of asset groups to which the policy applies. If the array is empty, the policy applies to all asset groups.

▼ triggersobject

Defines when the AppSec policy should be evaluated. At least one of the options must be `true`.

▶ periodicobjectrequired

If `true`, the AppSec policy is evaluated periodically (for example, daily or weekly).

▶ probjectrequired

If `true`, the AppSec policy is evaluated on Pull Request (PR) events

▶ cicdobjectrequired

If `true`, the AppSec policy is evaluated on CI/CD pipeline events.

REQUEST Example 1

```
{ "name": "string", "description": "string", "conditions": { "SEARCH_FIELD": "string", "SEARCH_TYPE": "EQ",
"SEARCH_VALUE": true }, "scope": { "SEARCH_FIELD": "string", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": true },
"assetGroupIds": [ 0 ], "triggers": { "periodic": { "isEnabled": true, "actions": { "reportIssue": true },
"overrideIssueSeverity": "Critical" }, "pr": { "isEnabled": true, "actions": { "reportIssue": true, "blockPr": true,
"reportPrComment": true }, "overrideIssueSeverity": "Critical" }, "cicd": { "isEnabled": true, "actions": {
"reportIssue": true, "blockCicd": true, "reportCicd": true }, "overrideIssueSeverity": "Critical" } } }
```

Responses

No content

2.4.4 | Rules

APIs for managing rules

2.4.4.1 | Get AppSec rules

get /public_api/appsec/v1/rules

Application Security rules are designed to detect security threats within your application security environment. Application Security rules identify and flag issues based on predefined criteria.

Get a list of all the Application Security rules. We recommend you use the parameters to filter the rules since there are many of them.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:

`UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `241`
▼ get-scanner-rule-id Boolean
Example: `true`
Query parameters

▼ enabled Boolean

Whether the rule is enabled

Example: `true`
▼ isCustom Boolean
Example: `true`
▼ scanners array[Scanner]

Type of security scanner used to detect findings of this rule

▼ severities array[Severity]

The priority level assigned to findings identified by the rule

▼ frameworks array[FrameworkName]

The framework or language that the Application Security rule applies to (for example, GitHub, Terraform, JavaScript)

▼ labels array[String]

Labels assigned to the rule

▼ categories array[String]
▼ subCategories array[String]
▼ cloudProviders array[String]
Allowed values: ALIBABA_CLOUD AWS AZURE GCP IBM ORACLE OTHER
▼ offset Double
double
Example: `1.2`
Default: `0`
▼ limit Double
double
Example: `1.2`
Default: `100`
▼ sortBy String
Allowed values: created_at name labels
Example: `sortBy_example`
Default: `name`
▼ sortOrder SortDirection

Sort direction
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241' -H 'get-scanner-rule-id: true'
'https://api-yourfqdn/public_api/appsec/v1/rules?
enabled=true&isCustom=true&scanners=&severities=&frameworks=&labels=&categories=&subCategories=&cloudProviders=&offset=1.2&lim
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'get-scanner-rule-id': "SOME_BOOLEAN_VALUE" } conn.request("GET",
"/public_api/appsec/v1/rules?
enabled=SOME_BOOLEAN_VALUE&isCustom=SOME_BOOLEAN_VALUE&scanners=SOME_ARRAY_VALUE&severities=SOME_ARRAY_VALUE&frameworks=SOME_AR
headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/appsec/v1/rules?
enabled=SOME_BOOLEAN_VALUE&isCustom=SOME_BOOLEAN_VALUE&scanners=SOME_ARRAY_VALUE&severities=SOME_ARRAY_VALUE&frameworks=SOME_AR
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["get-scanner-rule-id"] = 'SOME_BOOLEAN_VALUE' response =
http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/appsec/v1/rules?
enabled=SOME_BOOLEAN_VALUE&isCustom=SOME_BOOLEAN_VALUE&scanners=SOME_ARRAY_VALUE&severities=SOME_ARRAY_VALUE&frameworks=SOME_AR
xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("get-scanner-rule-id", "SOME_BOOLEAN_VALUE");
xhr.send(data);
```

```
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/rules?
enabled=SOME_BOOLEAN_VALUE&isCustom=SOME_BOOLEAN_VALUE&scanners=SOME_ARRAY_VALUE&severities=SOME_ARRAY_VALUE&frameworks=SOME_AR
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("get-scanner-rule-id", "SOME_BOOLEAN_VALUE") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "get-scanner-rule-id": "SOME_BOOLEAN_VALUE" ] let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/appsec/v1/rules?
enabled=SOME_BOOLEAN_VALUE&isCustom=SOME_BOOLEAN_VALUE&scanners=SOME_ARRAY_VALUE&severities=SOME_ARRAY_VALUE&frameworks=SOME_AR
as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET"
request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask = session.dataTask(with: request
as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/rules?
enabled=SOME_BOOLEAN_VALUE&isCustom=SOME_BOOLEAN_VALUE&scanners=SOME_ARRAY_VALUE&severities=SOME_ARRAY_VALUE&frameworks=SOME_AR
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "get-scanner-rule-id: SOME_BOOLEAN_VALUE", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/rules?
enabled=SOME_BOOLEAN_VALUE&isCustom=SOME_BOOLEAN_VALUE&scanners=SOME_ARRAY_VALUE&severities=SOME_ARRAY_VALUE&frameworks=SOME_AR
struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "get-scanner-
rule-id: SOME_BOOLEAN_VALUE"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/rules?
enabled=SOME_BOOLEAN_VALUE&isCustom=SOME_BOOLEAN_VALUE&scanners=SOME_ARRAY_VALUE&severities=SOME_ARRAY_VALUE&frameworks=SOME_AR
var request = new RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("get-scanner-rule-id", "SOME_BOOLEAN_VALUE");
IRestResponse response = client.Execute(request);
```

Responses

Ok

Body

application/json

▼ offsetnumberdouble

▼ rulesarray

[

▶ categorystring

▶ cloudProviderstring (Enum)

▶ createdAtstringdate-time

▶ descriptionstring

Description of the rule

▶ docLinkstring

▶ domainstring

▶ findingCategorystring (Enum)

▶ findingDocsstring

▶ findingTypeIdnumberdouble

▶ frameworksobject

▶ idstring

▶ isCustomboolean

▶ isEnabledboolean

▶ labelsarray[string]

▶ mitreTacticsarray[string]

▶ mitreTechniquesarray[string]

▶ namestring

▶ ownerstring

▶ scannerstring (Enum)

▶ scannerRuleIdstring

▶ severitystring (Enum)

Severity
▶ sourcestring
▶ sourceVersionstring
▶ subCategorystring
▶ updatedAtstringdate-time
]
RESPONSE

```
{ "offset": 0.1, "rules": [ { "category": "example", "cloudProvider": "ALIBABA_CLOUD", "createdAt": "2020-01-
01T12:00:00Z", "description": "example", "docLink": "example", "domain": "example", "findingCategory": "Code",
"findingDocs": "example", "findingTypeId": 0.1, "frameworks": { "frameworkDetails": { "definition": "example",
"definition_link": "example", "remediation_description": "example", "remediation_ids": [ "example" ],
"resource_types": [ "example" ] }, "frameworkName": "ANSIBLE" }, "id": "example", "isCustom": false, "isEnabled":
false, "labels": [ "example" ], "mitreTactics": [ "example" ], "mitreTechniques": [ "example" ], "name": "example",
"owner": "example", "scanner": "CICD", "scannerRuleId": "example", "severity": "CRITICAL", "source": "example",
"sourceVersion": "example", "subCategory": "example", "updatedAt": "2020-01-01T12:00:00Z" } ] }
```

2.4.4.2 | Create or clone an AppSec rule

post /public_api/appsec/v1/rules

Application Security rules are designed to detect security threats within your application security environment. Application Security rules identify and flag issues based on predefined criteria.

Create an Application Security rule or clone an existing out-of-the-box Application Security rule.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:

```
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
```

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/rules'
-d '{ "subCategory" : "", "frameworks" : [ { "definitionLink" : "definitionLink", "definition" : "definition",
"remediationDescription" : "remediationDescription" }, { "definitionLink" : "definitionLink", "definition" :
"definition", "remediationDescription" : "remediationDescription" } ], "name" : "name", "description" :
"description", "labels" : [ "labels", "labels" ] }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "
{\"name\":\"string\",\"severity\":\"CRITICAL\",\"scanner\":\"IAC\",\"frameworks\":
[{\"name\":\"ANSIBLE\",\"definition\":\"string\",\"definitionLink\":\"string\",\"remediationDescription\":\"string\"}],\"catego
[\"string\"]}" headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST", "/public_api/appsec/v1/rules",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/appsec/v1/rules") http
= Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "
{\"name\":\"string\",\"severity\":\"CRITICAL\",\"scanner\":\"IAC\",\"frameworks\":
[{\"name\":\"ANSIBLE\",\"definition\":\"string\",\"definitionLink\":\"string\",\"remediationDescription\":\"string\"}],\"catego
[\"string\"]}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "name": "string", "severity": "CRITICAL", "scanner": "IAC", "frameworks": [ { "name":
"ANSIBLE", "definition": "string", "definitionLink": "string", "remediationDescription": "string" } ], "category":
"AI_ML", "subCategory": "GUARDRAILS", "description": "string", "labels": [ "string" ] }); const xhr = new
```

```
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/appsec/v1/rules"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/appsec/v1/rules")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("
{\"name\":\"string\",\"severity\":\"CRITICAL\",\"scanner\":\"IAC\",\"frameworks\":
[{\"name\":\"ANSIBLE\",\"definition\":\"string\",\"definitionLink\":\"string\",\"remediationDescription\":\"string\"}],\"categ
[\"string\"]}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = [ "name": "string", "severity":
"CRITICAL", "scanner": "IAC", "frameworks": [ [ "name": "ANSIBLE", "definition": "string", "definitionLink":
"string", "remediationDescription": "string" ] ], "category": "AI_ML", "subCategory": "GUARDRAILS", "description":
"string", "labels": ["string"] ] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/appsec/v1/rules")!
as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST"
request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/rules", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"name\":\"string\",\"severity\":\"CRITICAL\",\"scanner\":\"IAC\",\"frameworks\":
[{\"name\":\"ANSIBLE\",\"definition\":\"string\",\"definitionLink\":\"string\",\"remediationDescription\":\"string\"}],\"categ
[\"string\"]}", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/appsec/v1/rules"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"name\":\"string\",\"severity\":\"CRITICAL\",\"scanner\":\"IAC\",\"frameworks\":
[{\"name\":\"ANSIBLE\",\"definition\":\"string\",\"definitionLink\":\"string\",\"remediationDescription\":\"string\"}],\"categ
[\"string\"]}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/rules"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "
{\"name\":\"string\",\"severity\":\"CRITICAL\",\"scanner\":\"IAC\",\"frameworks\":
[{\"name\":\"ANSIBLE\",\"definition\":\"string\",\"definitionLink\":\"string\",\"remediationDescription\":\"string\"}],\"categ
[\"string\"]}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
required
application/json

Define the Application Security custom rule. The `category` option should match your selection for `scanner`.

▼ namestring
▼ severitystring (Enum)

Severity

Allowed values:"CRITICAL""HIGH""INFO""LOW""MEDIUM"
▼ scannerstring (Enum)

The type of security scanner used to detect findings of this rule

Allowed values:"IAC""SECRETS"
▼ frameworksarray
[

▶ namestring (Enum)

The framework or language name

▶ definitionstring
▶ definitionLinkstring
▶ remediationDescriptionstring
]
▼ categoryobject
Any of CustomRuleIacCategory
▶ string (Enum)

Custom rule IaC category

▶ string (Enum)

Custom rule secret category

▼ subCategoryobject
▶ string (Enum)

Custom rule subcategory

▼ descriptionstring

Description of the rule

▼ labelsarray[string]

Labels to be assigned to the rule
REQUEST
```
{ "name": "example", "severity": "CRITICAL", "scanner": "IAC", "frameworks": [ { "name": "ANSIBLE", "definition": "example", "definitionLink": "example", "remediationDescription": "example" } ], "category": "AI_ML", "subCategory": "GUARDRAILS", "description": "example", "labels": [ "example" ] }
```
Responses

Ok

Body
application/json
Any of FrameworksErrorsResponse
▶ frameworksErrorsarray
▶ categorystring
▶ cloudProviderstring (Enum)
▶ createdAtstringdate-time
▶ descriptionstring

Description of the rule

▶ docLinkstring
▶ domainstring
▶ findingCategorystring (Enum)
▶ findingDocsstring
▶ findingTypeIdnumberdouble
▶ frameworksobject
▶ idstring
▶ isCustomboolean
▶ isEnabledboolean
▶ labelsarray[string]
▶ mitreTacticsarray[string]
▶ mitreTechniquesarray[string]
▶ namestring
▶ ownerstring
▶ scannerstring (Enum)
▶ scannerRuleIdstring
▶ severitystring (Enum)

Severity

▶ sourcestring
▶ sourceVersionstring
▶ subCategorystring
▶ updatedAtstringdate-time
RESPONSE

{ "frameworksErrors": [{ "framework": "ANSIBLE", "errors": ["example"] }] }

2.4.4.3 | Get an AppSec rule

get /public_api/appsec/v1/rules/{ruleId}

Get the details of the specified Application Security rule.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ ruleId String required
Example: `ruleId_example`
Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/rules/{ruleId}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/rules/%7BruleId%7D", headers=headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask =
session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error !=
nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization:
```

```
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D"); var request = new
RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
Any of 0

- ▶ categorystring
- ▶ cloudProviderstring (Enum)
- ▶ createdAtstringdate-time
- ▶ descriptionstring

Description of the rule

- ▶ docLinkstring
- ▶ domainstring
- ▶ findingCategorystring (Enum)
- ▶ findingDocsstring
- ▶ findingTypeIdnumberdouble
- ▶ frameworksobject
- ▶ idstring
- ▶ isCustomboolean
- ▶ isEnabledboolean
- ▶ labelsarray[string]
- ▶ mitreTacticsarray[string]
- ▶ mitreTechniquesarray[string]
- ▶ namestring
- ▶ ownerstring
- ▶ scannerstring (Enum)
- ▶ scannerRuleIdstring
- ▶ severitystring (Enum)

Severity

- ▶ sourcestring
- ▶ sourceVersionstring
- ▶ subCategorystring
- ▶ updatedAtstringdate-time

RESPONSE


```
{ "category": "example", "cloudProvider": "ALIBABA_CLOUD", "createdAt": "2020-01-01T12:00:00Z", "description":
"example", "docLink": "example", "domain": "example", "findingCategory": "Code", "findingDocs": "example",
"findingTypeId": 0.1, "frameworks": { "frameworkDetails": { "definition": "example", "definition_link": "example",
"remediation_description": "example", "remediation_ids": [ "example" ], "resource_types": [ "example" ] },
"frameworkName": "ANSIBLE" }, "id": "example", "isCustom": false, "isEnabled": false, "labels": [ "example" ],
"mitreTactics": [ "example" ], "mitreTechniques": [ "example" ], "name": "example", "owner": "example", "scanner":
"CICD", "scannerRuleId": "example", "severity": "CRITICAL", "source": "example", "sourceVersion": "example",
"subCategory": "example", "updatedAt": "2020-01-01T12:00:00Z" }
```

2.4.4.4 | Modify an AppSec rule

patch /public_api/appsec/v1/rules/{ruleId}

Modify an existing Application Security rule. If it's an out-of-the-box rule, the only modification you can make is to add labels. For custom rules, you can modify all of the fields.

Note: To customize an out-of-the-box rule, you can create a custom rule by cloning the existing one. This allows you to make changes to the original rule according to your requirements.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ ruleId String required
Example: `ruleId_example`

Request headers

▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241

CLIENT REQUEST Bash+curl

```
curl -X 'PATCH'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/rules/{ruleId}'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"labels\":[\"string\"]}" headers =
{ 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("PATCH",
"/public_api/appsec/v1/rules/%7BruleId%7D", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Patch.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"labels\":
[\"string\"]}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "labels": [ "string" ] }); const xhr = new XMLHttpRequest(); xhr.withCredentials =
true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("PATCH", "https://api-
yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.patch("https://api-yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"labels\":[\"string\"]}")
.asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = ["labels": ["string"]] as [String :
Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "PATCH" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "PATCH", CURLOPT_POSTFIELDS => "{\"labels\":[\"string\"]}", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "PATCH"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"labels\":[\"string\"]}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D"); var request = new
RestRequest(Method.PATCH); request.AddHeader("Authorization",
```

```
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"labels\":[\"string\"]}", ParameterType.RequestBody); IRestResponse
response = client.Execute(request);
```

Body parameters

required

application/json

Any of ModifyRuleParams

▶ labelsarray[string]

List of rule labels

▶ namestring

▶ severitystring (Enum)

Severity

▶ scannerstring (Enum)

The type of security scanner used to detect findings of this rule

▶ frameworksarray

▶ categoryobject

▶ subCategoryobject

▶ descriptionstring

Description of the rule

▶ labelsarray[string]

Labels to be assigned to the rule

REQUEST

```
{ "labels": [ "example" ] }
```

Responses

Ok

Body

application/json

Updated details of the modified Application Security rule

▼ ruleobject

Details of the Application Security rule

▶ categorystring

▶ cloudProviderstring (Enum)

▶ createdAtstringdate-time

▶ descriptionstring

Description of the rule

▶ docLinkstring

▶ domainstring

▶ findingCategorystring (Enum)

▶ findingDocsstring

▶ findingTypeIdnumberdouble

▶ frameworksobject

▶ idstring

▶ isCustomboolean

▶ isEnabledboolean

▶ labelsarray[string]

▶ mitreTacticsarray[string]

▶ mitreTechniquesarray[string]

▶ namestring

▶ ownerstring

▶ scannerstring (Enum)

▶ scannerRuleIdstring

▶ severitystring (Enum)

Severity

▶ sourcestring

▶ sourceVersionstring
▶ subCategorystring
▶ updatedAtstringdate-time
RESPONSE

```
{ "rule": { "category": "example", "cloudProvider": "ALIBABA_CLOUD", "createdAt": "2020-01-01T12:00:00Z",
"description": "example", "docLink": "example", "domain": "example", "findingCategory": "Code", "findingDocs":
"example", "findingTypeId": 0.1, "frameworks": { "frameworkDetails": { "definition": "example", "definition_link":
"example", "remediation_description": "example", "remediation_ids": [ "example" ], "resource_types": [ "example" ] },
"frameworkName": "ANSIBLE" }, "id": "example", "isCustom": false, "isEnabled": false, "labels": [ "example" ],
"mitreTactics": [ "example" ], "mitreTechniques": [ "example" ], "name": "example", "owner": "example", "scanner":
"CICD", "scannerRuleId": "example", "severity": "CRITICAL", "source": "example", "sourceVersion": "example",
"subCategory": "example", "updatedAt": "2020-01-01T12:00:00Z" } }
```

2.4.4.5 | Delete an AppSec rule

delete /public_api/appsec/v1/rules/{ruleId}

Delete the specified Application Security rule.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ ruleId String required
Example: `ruleId_example`
Request headers
▼ Authorization String required

{api_key}

Example:

UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl

```
curl -X 'DELETE'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/rules/{ruleId}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("DELETE", "/public_api/appsec/v1/rules/%7BruleId%7D", headers=headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Delete.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("DELETE", "https://api-
yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.delete("https://api-yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "DELETE" request.allHTTPHeaderFields = headers let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
```

```
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "DELETE", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "DELETE"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/rules/%7BruleId%7D"); var request = new
RestRequest(Method.DELETE); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
▼ messagestring
RESPONSE

```
{ "message": "example" }
```

2.4.4.6 | Get AppSec rule labels

get /public_api/appsec/v1/rules/rule-labels

Get a list of all of the Application Security rule labels.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/rules/rule-labels'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/rules/rule-labels", headers=headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/appsec/v1/rules/rule-
labels") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE
request = Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/appsec/v1/rules/rule-labels");
xhr.setRequestHeader("Authorization",
```

```
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/rules/rule-labels")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/rules/rule-labels")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask =
session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error !=
nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/rules/rule-labels", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/rules/rule-labels"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/rules/rule-labels"); var request = new
RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json

A list of labels

▼ labelsarray[string]
RESPONSE

```
{ "labels": [ "example" ] }
```

2.4.4.7 | Create an AppSec rule validation

post /public_api/appsec/v1/rules/validate

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/rules/validate'
```

```
-d '{ "definition" : "definition" }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "
[{\"framework\":\"ANSIBLE\",\"definition\":\"string\"}]" headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
"/public_api/appsec/v1/rules/validate", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/rules/validate") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "
[{\"framework\":\"ANSIBLE\",\"definition\":\"string\"}]" response = http.request(request) puts response.read_body
const data = JSON.stringify([ { "framework": "ANSIBLE", "definition": "string" } ]); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/appsec/v1/rules/validate"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/appsec/v1/rules/validate")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("
[{\"framework\":\"ANSIBLE\",\"definition\":\"string\"}]") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = [ [ "framework": "ANSIBLE",
"definition": "string" ] ] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/rules/validate")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/rules/validate", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "[{\"framework\":\"ANSIBLE\",\"definition\":\"string\"}]", CURLOPT_HTTPHEADER => [
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/appsec/v1/rules/validate"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
[{\"framework\":\"ANSIBLE\",\"definition\":\"string\"}]"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/rules/validate"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "[{\"framework\":\"ANSIBLE\",\"definition\":\"string\"}]",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

array[ValidateCustomRuleRequestParams]

required

application/json

[

▶ frameworkstring (Enum)

The framework or language name

▶ definitionstring

]

REQUEST

```
[ { "framework": "ANSIBLE", "definition": "example" } ]
```

Responses

Ok

Body
application/json
Any of 0
▶ frameworksErrorsarray
▶ isValidboolean
RESPONSE
{ "frameworksErrors": [{ "framework": "ANSIBLE", "errors": ["example"] }] }

2.4.5 | Repositories

APIs for managing repository assets

2.4.5.1 | Get repositories

get /public_api/appsec/v1/repositories

Get details on all repositories integrated with Cortex Cloud Application Security, providing detailed information and insights into repository artifacts, configurations, and dependencies.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: `241`
Query parameters
▼ source array[IntegrationType]
▼ search String
Example: `search_example`
▼ offset BigDecimal
Example: `8.14`
▼ limit BigDecimal
Example: `8.14`
CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/repositories?source=&search=search_example&offset=8.14&limit=8.14'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/repositories?
source=SOME_ARRAY_VALUE&search=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE", headers=headers)
res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/appsec/v1/repositories?
source=SOME_ARRAY_VALUE&search=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/appsec/v1/repositories?
source=SOME_ARRAY_VALUE&search=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE");
xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/repositories?
source=SOME_ARRAY_VALUE&search=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/repositories?
source=SOME_ARRAY_VALUE&search=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields =
headers let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/repositories?
source=SOME_ARRAY_VALUE&search=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/repositories?
source=SOME_ARRAY_VALUE&search=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE"); struct
curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/repositories?
source=SOME_ARRAY_VALUE&search=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE"); var request =
new RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
[
▶ namestring

The name of the repository in the version control system (VCS)

▶ descriptionstring

Description describing the repository

▶ creationDatestring
▶ defaultBranchstring
▶ diskSizenumberdouble
▶ externalIdstring
▶ idstring

Repository ID

▶ integrationIdstring
▶ isArchivedboolean

Whether the repository is archived

▶ isBillableboolean

Whether the repository is billable

▶ isForkedboolean

Whether the repository is forked

▶ isPublicboolean

Whether the repository is public
▶ isSelectedboolean

Whether the repository is selected

▶ lastScanDatestring
▶ observationTimenumberdouble

The date the repository was last updated

▶ organizationIdstring
▶ ownerstring

Repository owner

▶ projectIdstring
▶ scanConfigurationobject

The configuration of an Application Security code scanner

▶ scannedBranchstring
▶ scannedBranchesarray
▶ sourcestring (Enum)
▶ statusobjectrequired
▶ urlstring

URL of the repository

]
RESPONSE

```
[ { "name": "example", "description": "example", "creationDate": "example", "defaultBranch": "example", "diskSize":
0.1, "externalId": "example", "id": "example", "integrationId": "example", "isArchived": false, "isBillable": false,
"isForked": false, "isPublic": false, "isSelected": false, "lastScanDate": "example", "observationTime": 0.1,
"organizationId": "example", "owner": "example", "projectId": "example", "scanConfiguration": { "excludedPaths": [
"example" ], "prScanning": { "blockOnError": false, "isEnabled": false }, "scanners": { "GIT_HISTORY": { "isEnabled":
false, "scanOptions": { "additionalProperties": false } }, "IAC": { "isEnabled": false, "scanOptions": {
"additionalProperties": false } }, "SCA": { "isEnabled": false, "scanOptions": { "additionalProperties": false } },
"SECRETS": { "isEnabled": false, "scanOptions": { "additionalProperties": false } } }, "taggingBot": {
"tagModuleBlocks": false, "tagResourceBlocks": false } }, "scannedBranch": "example", "scannedBranches": [ {
"diskSize": 0.1, "firstScanDate": "example", "id": "example", "isPrimary": false, "lastApiCallDate": "example",
"lastCloneDate": "example", "lastCloneSha": "example", "lastScanDate": "example", "name": "example", "repositoryId":
"example" } ], "source": "AWS_CODE_BUILD", "status": { "apiRead": { "reason": "example", "status": "INVALID" },
"apiWrite": { "reason": "example", "status": "INVALID" }, "clone": { "reason": "example", "status": "INVALID" },
"connection": { "reason": "example", "status": "INVALID" }, "webhookSubscription": { "reason": "example", "status":
"INVALID" }, "token": { "reason": "example", "status": "INVALID" } }, "url": "example" } ]
```

2.4.5.2 | Get a repository

get /public_api/appsec/v1/repositories/{assetId}

Get details on the Application Security repository specified by its asset ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ assetId String required

Asset ID

Example: `assetId_example`
Request headers
▼ Authorization String required

{api_key}

Example:

`UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `241`
CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/repositories/{assetId}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/repositories/%7BassetId%7D", headers=headers)
res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl =
true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D"); var request =
new RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
▼ namestring

The name of the repository in the version control system (VCS)

▼ descriptionstring

Description describing the repository

▼ creationDatestring
▼ defaultBranchstring
▼ diskSizenumberdouble
▼ externalIdstring
▼ idstring

Repository ID

▼ integrationIdstring
▼ isArchivedboolean

Whether the repository is archived

▼ isBillableboolean

Whether the repository is billable

▼ isForkedboolean

Whether the repository is forked

▼ isPublicboolean

Whether the repository is public

▼ isSelectedboolean

Whether the repository is selected

▼ lastScanDatestring
▼ observationTimenumberdouble

The date the repository was last updated

▼ organizationIdstring
▼ ownerstring

Repository owner

▼ projectIdstring
▼ scanConfigurationobject

The configuration of an Application Security code scanner

▶ excludedPathsarray[string]
▶ prScanningobjectrequired

Pull Request (PR) scans are initiated by events triggered by version control systems such as GitHub, GitLab, Bitbucket and Azure Repos, or via webhooks. The scan results are based on default enforcement thresholds.

▶ scannersobjectrequired

Details on each type of scanner: Branch periodic scans, CI scans, IaC scans, Secrets scans

▶ taggingBotobjectrequired
▼ scannedBranchstring
▼ scannedBranchesarray
[
▶ diskSizenumberdouble
▶ firstScanDatestring

Timestamp of the first scan performed

▶ idstring

Branch ID

▶ isPrimaryboolean

Indicates whether or not the branch is the primary branch

▶ lastApiCallDatestring
▶ lastCloneDatestring
▶ lastCloneShastring
▶ lastScanDatestring

Timestamp of the last scan execution

▶ namestring

Branch name

▶ repositoryIdstring

Repository ID

]

▼ sourcestring (Enum)
Allowed
values:"AWS_CODE_BUILD""AWS_CODE_COMMIT""AZURE_REPOS""BITBUCKET""BITBUCKET_DATACENTER""CIRCLE_CI""CIRCLE_CI_CD""CORTEX_CLI""GITHUB""GITH
▼ statusobjectrequired
▶ apiReadobjectrequired
▶ apiWriteobjectrequired
▶ cloneobjectrequired
▶ connectionobjectrequired
▶ webhookSubscriptionobjectrequired
▶ tokenobjectrequired
▶ Additional propertiesobjectrequired
▼ urlstring

URL of the repository
RESPONSE

```
{ "name": "example", "description": "example", "creationDate": "example", "defaultBranch": "example", "diskSize":
0.1, "externalId": "example", "id": "example", "integrationId": "example", "isArchived": false, "isBillable": false,
"isForked": false, "isPublic": false, "isSelected": false, "lastScanDate": "example", "observationTime": 0.1,
"organizationId": "example", "owner": "example", "projectId": "example", "scanConfiguration": { "excludedPaths": [
"example" ], "prScanning": { "blockOnError": false, "isEnabled": false }, "scanners": { "GIT_HISTORY": { "isEnabled":
false, "scanOptions": { "additionalProperties": false } }, "IAC": { "isEnabled": false, "scanOptions": {
"additionalProperties": false } }, "SCA": { "isEnabled": false, "scanOptions": { "additionalProperties": false } },
"SECRETS": { "isEnabled": false, "scanOptions": { "additionalProperties": false } } }, "taggingBot": {
"tagModuleBlocks": false, "tagResourceBlocks": false } }, "scannedBranch": "example", "scannedBranches": [ {
"diskSize": 0.1, "firstScanDate": "example", "id": "example", "isPrimary": false, "lastApiCallDate": "example",
"lastCloneDate": "example", "lastCloneSha": "example", "lastScanDate": "example", "name": "example", "repositoryId":
"example" } ], "source": "AWS_CODE_BUILD", "status": { "apiRead": { "reason": "example", "status": "INVALID" },
"apiWrite": { "reason": "example", "status": "INVALID" }, "clone": { "reason": "example", "status": "INVALID" },
"connection": { "reason": "example", "status": "INVALID" }, "webhookSubscription": { "reason": "example", "status":
"INVALID" }, "token": { "reason": "example", "status": "INVALID" } }, "url": "example" }
```

2.4.5.3 | Get a repository scan configuration

get /public_api/appsec/v1/repositories/{assetId}/scan-configuration

Get the details of an Application Security scan configuration.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ assetId String required
Example: `assetId_example`
Request headers
▼ Authorization String required

{api_key}

Example:

UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/repositories/{assetId}/scan-configuration'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/repositories/%7BassetId%7D/scan-
configuration", headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/scan-configuration") http = Net::HTTP.new(url.host,
url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url)
request["Authorization"] =
```

```
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/scan-configuration"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/scan-configuration") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/scan-configuration")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/scan-configuration", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/scan-configuration"); struct curl_slist
*headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/scan-
configuration"); var request = new RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json

The configuration of an Application Security code scanner

▼ excludedPathsarray[string]
▼ prScanningobjectrequired

Pull Request (PR) scans are initiated by events triggered by version control systems such as GitHub, GitLab, Bitbucket and Azure Repos, or via webhooks. The scan results are based on default enforcement thresholds.

▶ blockOnErrorboolean
▶ isEnabledboolean

Indicates whether the PR scan is enabled

▼ scannersobjectrequired

Details on each type of scanner: Branch periodic scans, CI scans, IaC scans, Secrets scans

▶ GIT_HISTORYobjectrequired

Scanner configuration

▶ IACobjectrequired

Scanner configuration

▶ SCAobjectrequired

Scanner configuration

▶ SECRETSobjectrequired

Scanner configuration
▼ taggingBotobjectrequired
▶ tagModuleBlocksboolean
▶ tagResourceBlocksboolean
RESPONSE

```
{ "excludedPaths": [ "example" ], "prScanning": { "blockOnError": false, "isEnabled": false }, "scanners": {
"GIT_HISTORY": { "isEnabled": false, "scanOptions": { "additionalProperties": false } }, "IAC": { "isEnabled": false,
"scanOptions": { "additionalProperties": false } }, "SCA": { "isEnabled": false, "scanOptions": {
"additionalProperties": false } }, "SECRETS": { "isEnabled": false, "scanOptions": { "additionalProperties": false }
} }, "taggingBot": { "tagModuleBlocks": false, "tagResourceBlocks": false } }
```

2.4.5.4 | Update a repository scan configuration

put /public_api/appsec/v1/repositories/{assetId}/scan-configuration

Update the configuration of an Application Security scan.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ assetId String required
Example: `assetId_example`
Request headers
▼ Authorization String required

{api_key}

Example:

```
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
```

▼ x-xdr-auth-id String required

{api_key_id}

Example: `241`
CLIENT REQUEST Bash+curl

```
curl -X 'PUT'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/repositories/{assetId}/scan-configuration'
-d '{ "scanners" : { "SCA" : { "owner" : "owner", "scannedBranches" : [ { "diskSize" : 1.4658129805029452,
"lastApiCallDate" : "lastApiCallDate", "firstScanDate" : "firstScanDate", "lastCloneSha" : "lastCloneSha",
"isPrimary" : true, "name" : "name", "repositoryId" : "repositoryId", "lastScanDate" : "lastScanDate", "id" : "id",
"lastCloneDate" : "lastCloneDate" }, { "diskSize" : 1.4658129805029452, "lastApiCallDate" : "lastApiCallDate",
"firstScanDate" : "firstScanDate", "lastCloneSha" : "lastCloneSha", "isPrimary" : true, "name" : "name",
"repositoryId" : "repositoryId", "lastScanDate" : "lastScanDate", "id" : "id", "lastCloneDate" : "lastCloneDate" } ],
"scanConfiguration" : { "scanners" : { "SCA" : { "scanOptions" : { "key" : true }, "isEnabled" : true },
"GIT_HISTORY" : { "scanOptions" : { "key" : true }, "isEnabled" : true }, "SECRETS" : { "scanOptions" : { "key" :
true }, "isEnabled" : true }, "IAC" : { "scanOptions" : { "key" : true }, "isEnabled" : true } }, "taggingBot" : {
"tagModuleBlocks" : true, "tagResourceBlocks" : true }, "excludedPaths" : [ "excludedPaths", "excludedPaths" ],
"prScanning" : { "blockOnError" : true, "isEnabled" : true } }, "isArchived" : true, "defaultBranch" :
"defaultBranch", "isForked" : true, "description" : "description", "externalId" : "externalId", "integrationId" :
"integrationId", "observationTime" : 6.027456183070403, "creationDate" : "creationDate", "url" : "url", "isBillable"
: true, "organizationId" : "organizationId", "diskSize" : 0.8008281904610115, "scannedBranch" : "scannedBranch",
"name" : "name", "isSelected" : true, "isPublic" : true, "lastScanDate" : "lastScanDate", "id" : "id", "projectId" :
"projectId", "status" : { "clone" : { "reason" : "reason" }, "apiRead" : { "reason" : "reason" },
"webhookSubscription" : { "reason" : "reason" }, "connection" : { "reason" : "reason" }, "apiWrite" : { "reason" :
"reason" }, "token" : { "reason" : "reason" } } }, "SECRETS" : { "scanOptions" : { "gitHistory" : true,
"secretValidation" : true }, "isEnabled" : true }, "IAC" : { "owner" : "owner", "scannedBranches" : [ { "diskSize" :
1.4658129805029452, "lastApiCallDate" : "lastApiCallDate", "firstScanDate" : "firstScanDate", "lastCloneSha" :
"lastCloneSha", "isPrimary" : true, "name" : "name", "repositoryId" : "repositoryId", "lastScanDate" :
"lastScanDate", "id" : "id", "lastCloneDate" : "lastCloneDate" }, { "diskSize" : 1.4658129805029452,
"lastApiCallDate" : "lastApiCallDate", "firstScanDate" : "firstScanDate", "lastCloneSha" : "lastCloneSha",
"isPrimary" : true, "name" : "name", "repositoryId" : "repositoryId", "lastScanDate" : "lastScanDate", "id" : "id",
"lastCloneDate" : "lastCloneDate" } ], "scanConfiguration" : { "scanners" : { "SCA" : { "scanOptions" : { "key" :
true }, "isEnabled" : true }, "GIT_HISTORY" : { "scanOptions" : { "key" : true }, "isEnabled" : true }, "SECRETS" : {
"scanOptions" : { "key" : true }, "isEnabled" : true }, "IAC" : { "scanOptions" : { "key" : true }, "isEnabled" :
true } }, "taggingBot" : { "tagModuleBlocks" : true, "tagResourceBlocks" : true }, "excludedPaths" : [
```

```
"excludedPaths", "excludedPaths" ], "prScanning" : { "blockOnError" : true, "isEnabled" : true } }, "isArchived" :
true, "defaultBranch" : "defaultBranch", "isForked" : true, "description" : "description", "externalId" :
"externalId", "integrationId" : "integrationId", "observationTime" : 6.027456183070403, "creationDate" :
"creationDate", "url" : "url", "isBillable" : true, "organizationId" : "organizationId", "diskSize" :
0.8008281904610115, "scannedBranch" : "scannedBranch", "name" : "name", "isSelected" : true, "isPublic" : true,
"lastScanDate" : "lastScanDate", "id" : "id", "projectId" : "projectId", "status" : { "clone" : { "reason" : "reason"
}, "apiRead" : { "reason" : "reason" }, "webhookSubscription" : { "reason" : "reason" }, "connection" : { "reason" :
"reason" }, "apiWrite" : { "reason" : "reason" }, "token" : { "reason" : "reason" } } } }, "taggingBot" : {
"tagModuleBlocks" : true, "tagResourceBlocks" : true }, "excludedPaths" : [ "excludedPaths", "excludedPaths" ],
"prScanning" : { "blockOnError" : true, "isEnabled" : true } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"excludedPaths\":
[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true},\"scanners\":{\"SECRETS\":{\"isEnabled\":true,\"scanOptions\":
{\"gitHistory\":true,\"secretValidation\":true}},\"SCA\":{\"name\":\"string\",\"description\":\"string\",\"status\":
{\"apiRead\":{\"reason\":\"string\",\"status\":\"INVALID\"},\"apiWrite\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"clone\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"connection\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"webhookSubscription\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"token\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property1\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property2\":
{\"reason\":\"string\",\"status\":\"INVALID\"}},\"creationDate\":\"string\",\"defaultBranch\":\"string\",\"diskSize\":0.1,\"ex
{\"excludedPaths\":[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"scanners\":
{\"GIT_HISTORY\":{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"IAC\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SCA\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SECRETS\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}}},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true}},\"scannedBranch\":\"string\",\"scannedBranches\":
[{\"diskSize\":0.1,\"firstScanDate\":\"string\",\"id\":\"string\",\"isPrimary\":true,\"lastApiCallDate\":\"string\",\"lastClone
{\"name\":\"string\",\"description\":\"string\",\"status\":{\"apiRead\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"apiWrite\":{\"reason\":\"string\",\"status\":\"INVALID\"},\"clone\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"connection\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"webhookSubscription\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"token\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property1\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property2\":
{\"reason\":\"string\",\"status\":\"INVALID\"}},\"creationDate\":\"string\",\"defaultBranch\":\"string\",\"diskSize\":0.1,\"ex
{\"excludedPaths\":[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"scanners\":
{\"GIT_HISTORY\":{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"IAC\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SCA\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SECRETS\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}}},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true}},\"scannedBranch\":\"string\",\"scannedBranches\":
[{\"diskSize\":0.1,\"firstScanDate\":\"string\",\"id\":\"string\",\"isPrimary\":true,\"lastApiCallDate\":\"string\",\"lastClone
headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("PUT",
"/public_api/appsec/v1/repositories/%7BassetId%7D/scan-configuration", payload, headers) res = conn.getresponse()
data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/scan-configuration") http = Net::HTTP.new(url.host,
url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Put.new(url)
request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"excludedPaths\":
[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true},\"scanners\":{\"SECRETS\":{\"isEnabled\":true,\"scanOptions\":
{\"gitHistory\":true,\"secretValidation\":true}},\"SCA\":{\"name\":\"string\",\"description\":\"string\",\"status\":
{\"apiRead\":{\"reason\":\"string\",\"status\":\"INVALID\"},\"apiWrite\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"clone\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"connection\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"webhookSubscription\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"token\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property1\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property2\":
{\"reason\":\"string\",\"status\":\"INVALID\"}},\"creationDate\":\"string\",\"defaultBranch\":\"string\",\"diskSize\":0.1,\"ex
{\"excludedPaths\":[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"scanners\":
{\"GIT_HISTORY\":{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"IAC\":
```

{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SCA\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SECRETS\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}}},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true}},\"scannedBranch\":\"string\",\"scannedBranches\":
[{\"diskSize\":0.1,\"firstScanDate\":\"string\",\"id\":\"string\",\"isPrimary\":true,\"lastApiCallDate\":\"string\",\"lastClone
{\"name\":\"string\",\"description\":\"string\",\"status\":{\"apiRead\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"apiWrite\":{\"reason\":\"string\",\"status\":\"INVALID\"},\"clone\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"connection\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"webhookSubscription\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"token\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property1\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property2\":
{\"reason\":\"string\",\"status\":\"INVALID\"}},\"creationDate\":\"string\",\"defaultBranch\":\"string\",\"diskSize\":0.1,\"ex
{\"excludedPaths\":[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"scanners\":
{\"GIT_HISTORY\":{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"IAC\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SCA\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SECRETS\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}}},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true}},\"scannedBranch\":\"string\",\"scannedBranches\":
[{\"diskSize\":0.1,\"firstScanDate\":\"string\",\"id\":\"string\",\"isPrimary\":true,\"lastApiCallDate\":\"string\",\"lastClone
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "excludedPaths": ["string"], "prScanning": { "blockOnError": true, "isEnabled": true
}, "taggingBot": { "tagModuleBlocks": true, "tagResourceBlocks": true }, "scanners": { "SECRETS": { "isEnabled":
true, "scanOptions": { "gitHistory": true, "secretValidation": true } }, "SCA": { "name": "string", "description":
"string", "status": { "apiRead": { "reason": "string", "status": "INVALID" }, "apiWrite": { "reason": "string",
"status": "INVALID" }, "clone": { "reason": "string", "status": "INVALID" }, "connection": { "reason": "string",
"status": "INVALID" }, "webhookSubscription": { "reason": "string", "status": "INVALID" }, "token": { "reason":
"string", "status": "INVALID" }, "property1": { "reason": "string", "status": "INVALID" }, "property2": { "reason":
"string", "status": "INVALID" } }, "creationDate": "string", "defaultBranch": "string", "diskSize": 0.1,
"externalId": "string", "id": "string", "integrationId": "string", "isArchived": true, "isBillable": true,
"isForked": true, "isPublic": true, "isSelected": true, "lastScanDate": "string", "observationTime": 0.1,
"organizationId": "string", "owner": "string", "projectId": "string", "scanConfiguration": { "excludedPaths": [
"string"], "prScanning": { "blockOnError": true, "isEnabled": true }, "scanners": { "GIT_HISTORY": { "isEnabled":
true, "scanOptions": { "property1": true, "property2": true } }, "IAC": { "isEnabled": true, "scanOptions": {
"property1": true, "property2": true } }, "SCA": { "isEnabled": true, "scanOptions": { "property1": true,
"property2": true } }, "SECRETS": { "isEnabled": true, "scanOptions": { "property1": true, "property2": true } } },
"taggingBot": { "tagModuleBlocks": true, "tagResourceBlocks": true } }, "scannedBranch": "string", "scannedBranches":
[{ "diskSize": 0.1, "firstScanDate": "string", "id": "string", "isPrimary": true, "lastApiCallDate": "string",
"lastCloneDate": "string", "lastCloneSha": "string", "lastScanDate": "string", "name": "string", "repositoryId":
"string" }], "source": "AWS_CODE_BUILD", "url": "string" }, "IAC": { "name": "string", "description": "string",
"status": { "apiRead": { "reason": "string", "status": "INVALID" }, "apiWrite": { "reason": "string", "status":
"INVALID" }, "clone": { "reason": "string", "status": "INVALID" }, "connection": { "reason": "string", "status":
"INVALID" }, "webhookSubscription": { "reason": "string", "status": "INVALID" }, "token": { "reason": "string",
"status": "INVALID" }, "property1": { "reason": "string", "status": "INVALID" }, "property2": { "reason": "string",
"status": "INVALID" } }, "creationDate": "string", "defaultBranch": "string", "diskSize": 0.1, "externalId":
"string", "id": "string", "integrationId": "string", "isArchived": true, "isBillable": true, "isForked": true,
"isPublic": true, "isSelected": true, "lastScanDate": "string", "observationTime": 0.1, "organizationId": "string",
"owner": "string", "projectId": "string", "scanConfiguration": { "excludedPaths": ["string"], "prScanning": {
"blockOnError": true, "isEnabled": true }, "scanners": { "GIT_HISTORY": { "isEnabled": true, "scanOptions": {
"property1": true, "property2": true } }, "IAC": { "isEnabled": true, "scanOptions": { "property1": true,
"property2": true } }, "SCA": { "isEnabled": true, "scanOptions": { "property1": true, "property2": true } },
"SECRETS": { "isEnabled": true, "scanOptions": { "property1": true, "property2": true } } }, "taggingBot": {
"tagModuleBlocks": true, "tagResourceBlocks": true } }, "scannedBranch": "string", "scannedBranches": [{ "diskSize":
0.1, "firstScanDate": "string", "id": "string", "isPrimary": true, "lastApiCallDate": "string", "lastCloneDate":
"string", "lastCloneSha": "string", "lastScanDate": "string", "name": "string", "repositoryId": "string" }],
"source": "AWS_CODE_BUILD", "url": "string" } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("PUT", "https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/scan-configuration"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.put("https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/scan-configuration") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"excludedPaths\":
[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true},\"scanners\":{\"SECRETS\":{\"isEnabled\":true,\"scanOptions\":

{\"gitHistory\":true,\"secretValidation\":true}},\"SCA\":{\"name\":\"string\",\"description\":\"string\",\"status\":
{\"apiRead\":{\"reason\":\"string\",\"status\":\"INVALID\"},\"apiWrite\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"clone\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"connection\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"webhookSubscription\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"token\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property1\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property2\":
{\"reason\":\"string\",\"status\":\"INVALID\"}},\"creationDate\":\"string\",\"defaultBranch\":\"string\",\"diskSize\":0.1,\"ex
{\"excludedPaths\":[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"scanners\":
{\"GIT_HISTORY\":{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"IAC\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SCA\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SECRETS\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}}},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true}},\"scannedBranch\":\"string\",\"scannedBranches\":
[{\"diskSize\":0.1,\"firstScanDate\":\"string\",\"id\":\"string\",\"isPrimary\":true,\"lastApiCallDate\":\"string\",\"lastClon
{\"name\":\"string\",\"description\":\"string\",\"status\":{\"apiRead\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"apiWrite\":{\"reason\":\"string\",\"status\":\"INVALID\"},\"clone\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"connection\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"webhookSubscription\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"token\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property1\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property2\":
{\"reason\":\"string\",\"status\":\"INVALID\"}},\"creationDate\":\"string\",\"defaultBranch\":\"string\",\"diskSize\":0.1,\"ex
{\"excludedPaths\":[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"scanners\":
{\"GIT_HISTORY\":{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"IAC\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SCA\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SECRETS\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}}},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true}},\"scannedBranch\":\"string\",\"scannedBranches\":
[{\"diskSize\":0.1,\"firstScanDate\":\"string\",\"id\":\"string\",\"isPrimary\":true,\"lastApiCallDate\":\"string\",\"lastClon
.asString();
import Foundation let headers = ["Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json"] let parameters = ["excludedPaths": ["string"],
"prScanning": ["blockOnError": true, "isEnabled": true], "taggingBot": ["tagModuleBlocks": true,
"tagResourceBlocks": true], "scanners": ["SECRETS": ["isEnabled": true, "scanOptions": ["gitHistory": true,
"secretValidation": true]], "SCA": ["name": "string", "description": "string", "status": ["apiRead": ["reason":
"string", "status": "INVALID"], "apiWrite": ["reason": "string", "status": "INVALID"], "clone": ["reason":
"string", "status": "INVALID"], "connection": ["reason": "string", "status": "INVALID"], "webhookSubscription": [
"reason": "string", "status": "INVALID"], "token": ["reason": "string", "status": "INVALID"], "property1": [
"reason": "string", "status": "INVALID"], "property2": ["reason": "string", "status": "INVALID"]],
"creationDate": "string", "defaultBranch": "string", "diskSize": 0.1, "externalId": "string", "id": "string",
"integrationId": "string", "isArchived": true, "isBillable": true, "isForked": true, "isPublic": true, "isSelected":
true, "lastScanDate": "string", "observationTime": 0.1, "organizationId": "string", "owner": "string", "projectId":
"string", "scanConfiguration": ["excludedPaths": ["string"], "prScanning": ["blockOnError": true, "isEnabled": true
], "scanners": ["GIT_HISTORY": ["isEnabled": true, "scanOptions": ["property1": true, "property2": true]],
"IAC": ["isEnabled": true, "scanOptions": ["property1": true, "property2": true]], "SCA": ["isEnabled": true,
"scanOptions": ["property1": true, "property2": true]], "SECRETS": ["isEnabled": true, "scanOptions": [
"property1": true, "property2": true]]], "taggingBot": ["tagModuleBlocks": true, "tagResourceBlocks": true]],
"scannedBranch": "string", "scannedBranches": [["diskSize": 0.1, "firstScanDate": "string", "id": "string",
"isPrimary": true, "lastApiCallDate": "string", "lastCloneDate": "string", "lastCloneSha": "string", "lastScanDate":
"string", "name": "string", "repositoryId": "string"]], "source": "AWS_CODE_BUILD", "url": "string"], "IAC": [
"name": "string", "description": "string", "status": ["apiRead": ["reason": "string", "status": "INVALID"],
"apiWrite": ["reason": "string", "status": "INVALID"], "clone": ["reason": "string", "status": "INVALID"],
"connection": ["reason": "string", "status": "INVALID"], "webhookSubscription": ["reason": "string", "status":
"INVALID"], "token": ["reason": "string", "status": "INVALID"], "property1": ["reason": "string", "status":
"INVALID"], "property2": ["reason": "string", "status": "INVALID"]], "creationDate": "string", "defaultBranch":
"string", "diskSize": 0.1, "externalId": "string", "id": "string", "integrationId": "string", "isArchived": true,
"isBillable": true, "isForked": true, "isPublic": true, "isSelected": true, "lastScanDate": "string",
"observationTime": 0.1, "organizationId": "string", "owner": "string", "projectId": "string", "scanConfiguration": [
"excludedPaths": ["string"], "prScanning": ["blockOnError": true, "isEnabled": true], "scanners": ["GIT_HISTORY":
["isEnabled": true, "scanOptions": ["property1": true, "property2": true]], "IAC": ["isEnabled": true,
"scanOptions": ["property1": true, "property2": true]], "SCA": ["isEnabled": true, "scanOptions": ["property1":
true, "property2": true]], "SECRETS": ["isEnabled": true, "scanOptions": ["property1": true, "property2": true]
]], "taggingBot": ["tagModuleBlocks": true, "tagResourceBlocks": true]], "scannedBranch": "string",
"scannedBranches": [["diskSize": 0.1, "firstScanDate": "string", "id": "string", "isPrimary": true,
"lastApiCallDate": "string", "lastCloneDate": "string", "lastCloneSha": "string", "lastScanDate": "string", "name":

```
"string", "repositoryId": "string" ] ], "source": "AWS_CODE_BUILD", "url": "string" ] ] ] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/scan-configuration")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "PUT" request.allHTTPHeaderFields =
headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/scan-configuration", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "PUT", CURLOPT_POSTFIELDS => "{\"excludedPaths\":
[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true},\"scanners\":{\"SECRETS\":{\"isEnabled\":true,\"scanOptions\":
{\"gitHistory\":true,\"secretValidation\":true}},\"SCA\":{\"name\":\"string\",\"description\":\"string\",\"status\":
{\"apiRead\":{\"reason\":\"string\",\"status\":\"INVALID\"},\"apiWrite\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"clone\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"connection\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"webhookSubscription\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"token\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property1\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property2\":
{\"reason\":\"string\",\"status\":\"INVALID\"}},\"creationDate\":\"string\",\"defaultBranch\":\"string\",\"diskSize\":0.1,\"ex
{\"excludedPaths\":[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"scanners\":
{\"GIT_HISTORY\":{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"IAC\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SCA\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SECRETS\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}}},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true}},\"scannedBranch\":\"string\",\"scannedBranches\":
[{\"diskSize\":0.1,\"firstScanDate\":\"string\",\"id\":\"string\",\"isPrimary\":true,\"lastApiCallDate\":\"string\",\"lastClone
{\"name\":\"string\",\"description\":\"string\",\"status\":{\"apiRead\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"apiWrite\":{\"reason\":\"string\",\"status\":\"INVALID\"},\"clone\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"connection\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"webhookSubscription\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"token\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property1\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property2\":
{\"reason\":\"string\",\"status\":\"INVALID\"}},\"creationDate\":\"string\",\"defaultBranch\":\"string\",\"diskSize\":0.1,\"ex
{\"excludedPaths\":[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"scanners\":
{\"GIT_HISTORY\":{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"IAC\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SCA\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SECRETS\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}}},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true}},\"scannedBranch\":\"string\",\"scannedBranches\":
[{\"diskSize\":0.1,\"firstScanDate\":\"string\",\"id\":\"string\",\"isPrimary\":true,\"lastApiCallDate\":\"string\",\"lastClone
CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "PUT"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/scan-configuration"); struct curl_slist
*headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"excludedPaths\":[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true},\"scanners\":{\"SECRETS\":{\"isEnabled\":true,\"scanOptions\":
{\"gitHistory\":true,\"secretValidation\":true}},\"SCA\":{\"name\":\"string\",\"description\":\"string\",\"status\":
{\"apiRead\":{\"reason\":\"string\",\"status\":\"INVALID\"},\"apiWrite\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"clone\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"connection\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"webhookSubscription\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"token\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property1\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property2\":
{\"reason\":\"string\",\"status\":\"INVALID\"}},\"creationDate\":\"string\",\"defaultBranch\":\"string\",\"diskSize\":0.1,\"ex
{\"excludedPaths\":[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"scanners\":
{\"GIT_HISTORY\":{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"IAC\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SCA\":
```

{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SECRETS\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}}},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true}},\"scannedBranch\":\"string\",\"scannedBranches\":
[{\"diskSize\":0.1,\"firstScanDate\":\"string\",\"id\":\"string\",\"isPrimary\":true,\"lastApiCallDate\":\"string\",\"lastClone
{\"name\":\"string\",\"description\":\"string\",\"status\":{\"apiRead\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"apiWrite\":{\"reason\":\"string\",\"status\":\"INVALID\"},\"clone\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"connection\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"webhookSubscription\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"token\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property1\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property2\":
{\"reason\":\"string\",\"status\":\"INVALID\"}},\"creationDate\":\"string\",\"defaultBranch\":\"string\",\"diskSize\":0.1,\"ex
{\"excludedPaths\":[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"scanners\":
{\"GIT_HISTORY\":{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"IAC\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SCA\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SECRETS\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}}},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true}},\"scannedBranch\":\"string\",\"scannedBranches\":
[{\"diskSize\":0.1,\"firstScanDate\":\"string\",\"id\":\"string\",\"isPrimary\":true,\"lastApiCallDate\":\"string\",\"lastClone
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/scan-
configuration"); var request = new RestRequest(Method.PUT); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"excludedPaths\":[\"string\"],\"prScanning\":
{\"blockOnError\":true,\"isEnabled\":true},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true},\"scanners\":{\"SECRETS\":{\"isEnabled\":true,\"scanOptions\":
{\"gitHistory\":true,\"secretValidation\":true}},\"SCA\":{\"name\":\"string\",\"description\":\"string\",\"status\":
{\"apiRead\":{\"reason\":\"string\",\"status\":\"INVALID\"},\"apiWrite\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"clone\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"connection\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"webhookSubscription\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"token\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property1\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property2\":
{\"reason\":\"string\",\"status\":\"INVALID\"}},\"creationDate\":\"string\",\"defaultBranch\":\"string\",\"diskSize\":0.1,\"ex
{\"excludedPaths\":[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"scanners\":
{\"GIT_HISTORY\":{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"IAC\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SCA\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SECRETS\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}}},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true}},\"scannedBranch\":\"string\",\"scannedBranches\":
[{\"diskSize\":0.1,\"firstScanDate\":\"string\",\"id\":\"string\",\"isPrimary\":true,\"lastApiCallDate\":\"string\",\"lastClone
{\"name\":\"string\",\"description\":\"string\",\"status\":{\"apiRead\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"apiWrite\":{\"reason\":\"string\",\"status\":\"INVALID\"},\"clone\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"connection\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"webhookSubscription\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"token\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property1\":
{\"reason\":\"string\",\"status\":\"INVALID\"},\"property2\":
{\"reason\":\"string\",\"status\":\"INVALID\"}},\"creationDate\":\"string\",\"defaultBranch\":\"string\",\"diskSize\":0.1,\"ex
{\"excludedPaths\":[\"string\"],\"prScanning\":{\"blockOnError\":true,\"isEnabled\":true},\"scanners\":
{\"GIT_HISTORY\":{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"IAC\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SCA\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}},\"SECRETS\":
{\"isEnabled\":true,\"scanOptions\":{\"property1\":true,\"property2\":true}}},\"taggingBot\":
{\"tagModuleBlocks\":true,\"tagResourceBlocks\":true}},\"scannedBranch\":\"string\",\"scannedBranches\":
[{\"diskSize\":0.1,\"firstScanDate\":\"string\",\"id\":\"string\",\"isPrimary\":true,\"lastApiCallDate\":\"string\",\"lastClone
ParameterType.RequestBody); IRestResponse response = client.Execute(request);

Body parameters

required

application/json

▼ excludedPathsarray[string]

▼ prScanningobjectrequired

Pull Request (PR) scans are initiated by events triggered by version control systems such as GitHub, GitLab, Bitbucket and Azure Repos, or via webhooks. The scan results are based on default enforcement thresholds.

▶ blockOnErrorboolean

▶ isEnabledboolean

Indicates whether the PR scan is enabled

▼ taggingBotobjectrequired
▶ tagModuleBlocksboolean
▶ tagResourceBlocksboolean
▼ scannersobject
▶ SECRETSobject
▶ SCAobjectrequired

From T, pick a set of properties whose keys are in the union K

▶ IACobjectrequired

From T, pick a set of properties whose keys are in the union K

REQUEST


```
{ "excludedPaths": [ "example" ], "prScanning": { "blockOnError": false, "isEnabled": false }, "taggingBot": {
"tagModuleBlocks": false, "tagResourceBlocks": false }, "scanners": { "SECRETS": { "isEnabled": false, "scanOptions":
{ "gitHistory": false, "secretValidation": false } }, "SCA": { "name": "example", "description": "example", "status":
{ "apiRead": { "reason": "example", "status": "INVALID" }, "apiWrite": { "reason": "example", "status": "INVALID" },
"clone": { "reason": "example", "status": "INVALID" }, "connection": { "reason": "example", "status": "INVALID" },
"webhookSubscription": { "reason": "example", "status": "INVALID" }, "token": { "reason": "example", "status":
"INVALID" } }, "creationDate": "example", "defaultBranch": "example", "diskSize": 0.1, "externalId": "example", "id":
"example", "integrationId": "example", "isArchived": false, "isBillable": false, "isForked": false, "isPublic":
false, "isSelected": false, "lastScanDate": "example", "observationTime": 0.1, "organizationId": "example", "owner":
"example", "projectId": "example", "scanConfiguration": { "excludedPaths": [ "example" ], "prScanning": {
"blockOnError": false, "isEnabled": false }, "scanners": { "GIT_HISTORY": { "isEnabled": false, "scanOptions": {
"additionalProperties": false } }, "IAC": { "isEnabled": false, "scanOptions": { "additionalProperties": false } },
"SCA": { "isEnabled": false, "scanOptions": { "additionalProperties": false } }, "SECRETS": { "isEnabled": false,
"scanOptions": { "additionalProperties": false } } }, "taggingBot": { "tagModuleBlocks": false, "tagResourceBlocks":
false } }, "scannedBranch": "example", "scannedBranches": [ { "diskSize": 0.1, "firstScanDate": "example", "id":
"example", "isPrimary": false, "lastApiCallDate": "example", "lastCloneDate": "example", "lastCloneSha": "example",
"lastScanDate": "example", "name": "example", "repositoryId": "example" } ], "source": "AWS_CODE_BUILD", "url":
"example" }, "IAC": { "name": "example", "description": "example", "status": { "apiRead": { "reason": "example",
"status": "INVALID" }, "apiWrite": { "reason": "example", "status": "INVALID" }, "clone": { "reason": "example",
"status": "INVALID" }, "connection": { "reason": "example", "status": "INVALID" }, "webhookSubscription": { "reason":
"example", "status": "INVALID" }, "token": { "reason": "example", "status": "INVALID" } }, "creationDate": "example",
"defaultBranch": "example", "diskSize": 0.1, "externalId": "example", "id": "example", "integrationId": "example",
"isArchived": false, "isBillable": false, "isForked": false, "isPublic": false, "isSelected": false, "lastScanDate":
"example", "observationTime": 0.1, "organizationId": "example", "owner": "example", "projectId": "example",
"scanConfiguration": { "excludedPaths": [ "example" ], "prScanning": { "blockOnError": false, "isEnabled": false },
"scanners": { "GIT_HISTORY": { "isEnabled": false, "scanOptions": { "additionalProperties": false } }, "IAC": {
"isEnabled": false, "scanOptions": { "additionalProperties": false } }, "SCA": { "isEnabled": false, "scanOptions": {
"additionalProperties": false } }, "SECRETS": { "isEnabled": false, "scanOptions": { "additionalProperties": false }
} }, "taggingBot": { "tagModuleBlocks": false, "tagResourceBlocks": false } }, "scannedBranch": "example",
"scannedBranches": [ { "diskSize": 0.1, "firstScanDate": "example", "id": "example", "isPrimary": false,
"lastApiCallDate": "example", "lastCloneDate": "example", "lastCloneSha": "example", "lastScanDate": "example",
"name": "example", "repositoryId": "example" } ], "source": "AWS_CODE_BUILD", "url": "example" } } }
```


Responses

No content

2.4.5.5 | Get AppSec repository branches

get /public_api/appsec/v1/repositories/{assetId}/branches

Get the details of the Application Security repository branches.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ assetId String required
Example: `assetId_example`
Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/repositories/{assetId}/branches'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/repositories/%7BassetId%7D/branches",
headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/branches") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url)
request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/branches"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/branches") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/branches")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/branches", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING
=> "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/branches"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/branches"); var
request = new RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```
Responses

Ok

Body
application/json
[
▶ diskSizenumberdouble
▶ firstScanDatestring

Timestamp of the first scan performed

▶ idstring

Branch ID

▶ isPrimaryboolean

Indicates whether or not the branch is the primary branch

▶ lastApiCallDatestring
▶ lastCloneDatestring
▶ lastCloneShastring
▶ lastScanDatestring

Timestamp of the last scan execution

▶ namestring

Branch name

▶ repositoryIdstring

Repository ID

]
RESPONSE

```
[ { "diskSize": 0.1, "firstScanDate": "example", "id": "example", "isPrimary": false, "lastApiCallDate": "example",
"lastCloneDate": "example", "lastCloneSha": "example", "lastScanDate": "example", "name": "example", "repositoryId":
"example" } ]
```

2.4.5.6 | Update an AppSec repository branch

put /public_api/appsec/v1/repositories/{assetId}/branches

Update the Application Security repository branch by indicating the primary branch.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ assetId String required
Example: `assetId_example`
Request headers
▼ Authorization String required

{api_key}

Example:

`UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `241`
CLIENT REQUEST Bash+curl

```
curl -X 'PUT'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/repositories/{assetId}/branches'
-d '{ "selectedBranches" : [ "selectedBranches", "selectedBranches" ], "primaryBranch" : "primaryBranch" }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"selectedBranches\":
[\"string\"],\"primaryBranch\":\"string\"}" headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("PUT",
"/public_api/appsec/v1/repositories/%7BassetId%7D/branches", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/branches") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Put.new(url)
request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "
```

```
{\"selectedBranches\":[\"string\"],\"primaryBranch\":\"string\"}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "selectedBranches": [ "string" ], "primaryBranch": "string" }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("PUT", "https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/branches"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.put("https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/branches") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"selectedBranches\":
[\"string\"],\"primaryBranch\":\"string\"}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = [ "selectedBranches": ["string"],
"primaryBranch": "string" ] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/branches")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "PUT" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/branches", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING
=> "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "PUT", CURLOPT_POSTFIELDS => "{\"selectedBranches\":
[\"string\"],\"primaryBranch\":\"string\"}", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "PUT"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/branches"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"selectedBranches\":[\"string\"],\"primaryBranch\":\"string\"}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/repositories/%7BassetId%7D/branches"); var
request = new RestRequest(Method.PUT); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"selectedBranches\":[\"string\"],\"primaryBranch\":\"string\"}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ selectedBranchesarray[string]

▼ primaryBranchstring

REQUEST

```
{ "selectedBranches": [ "example" ], "primaryBranch": "example" }
```

Responses

No content

2.4.6 | Data Sources

APIs for managing integrations

2.4.6.1 | Get Data Sources

get /public_api/appsec/v1/integrations

Application Security capabilities support a wide range of Cloud DevSecOps systems integrations.

Get details on the Application Security integrations in your Cortex environment. You can filter the type and category of integrations you want to list.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers

▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241

Query parameters

▼ type UrlIntegrationType

Integration type

▼ type_category UrlIntegrationTypeCategory

Integration category

CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/integrations?type=&type_category='
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/integrations?
type=SOME_STRING_VALUE&type_category=SOME_STRING_VALUE", headers=headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/appsec/v1/integrations?
type=SOME_STRING_VALUE&type_category=SOME_STRING_VALUE") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/appsec/v1/integrations?
type=SOME_STRING_VALUE&type_category=SOME_STRING_VALUE"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/integrations?
type=SOME_STRING_VALUE&type_category=SOME_STRING_VALUE") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/integrations?type=SOME_STRING_VALUE&type_category=SOME_STRING_VALUE")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields =
headers let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/integrations?type=SOME_STRING_VALUE&type_category=SOME_STRING_VALUE",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/integrations?type=SOME_STRING_VALUE&type_category=SOME_STRING_VALUE");
struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/integrations?
type=SOME_STRING_VALUE&type_category=SOME_STRING_VALUE"); var request = new RestRequest(Method.GET);
request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
[
▶ creationDatestring

Date the integration was created

▶ domainobject

Integration hostname and protocol

▶ idstring

Integration ID

▶ instanceVersionstring

Integration instance version

▶ lastUpdateDatestring

Date the integration was last updated

▶ selectionTypestring (Enum)
▶ selfSignedCertificatestring

Self-signed certificate

▶ statearray[string]

Integration state

▶ statusstring (Enum)

The status of the integration

▶ typestring (Enum)
▶ typeCategorystring (Enum)
▶ uniqueIdentifierstring

Unique identifier of the integration

]
RESPONSE

```
[ { "creationDate": "example", "domain": { "hostname": "example", "protocol": "http" }, "id": "example",
"instanceVersion": "example", "lastUpdateDate": "example", "selectionType": "CURRENT_STATE", "selfSignedCertificate":
"example", "state": [ "example" ], "status": "COMPLETED", "type": "AWS_CODE_BUILD", "typeCategory": "DEFAULT",
"uniqueIdentifier": "example" } ]
```

2.4.6.2 | Get an AppSec Data Source

get /public_api/appsec/v1/integrations/{id}

Get the details of the Application Security integration specified by its ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ id String required
Example: `id_example`
Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241

CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/integrations/{id}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/integrations/%7Bid%7D", headers=headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask =
session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error !=
nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D"); var request = new
RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json

Integration details

▼ creationDatestring

Date the integration was created

▼ domainobject

Integration hostname and protocol

▶ hostnamestring

Integration hostname

▶ protocolstring (Enum)

Integration protocol

▼ idstring

Integration ID

▼ instanceVersionstring

Integration instance version

▼ lastUpdateDatestring

Date the integration was last updated

▼ selectionTypestring (Enum)
Allowed values:"CURRENT_STATE""CURRENT_STATE_AND_FUTURE""CURRENT_STATE_PENDING""MANUAL_SELECTION"
▼ selfSignedCertificatestring

Self-signed certificate

▼ statearray[string]

Integration state

▼ statusstring (Enum)

The status of the integration

Allowed values:"COMPLETED""CONNECTED""DESTROYING""ERROR""PENDING""TEMPORARY""WARNING"
▼ typestring (Enum)
Allowed
values:"AWS_CODE_BUILD""AWS_CODE_COMMIT""AZURE_REPOS""BITBUCKET""BITBUCKET_DATACENTER""CIRCLE_CI""CIRCLE_CI_CD""CORTEX_CLI""GITHUB""GITH
▼ typeCategorystring (Enum)
Allowed values:"DEFAULT""EXTERNAL_VENDOR_INTEGRATIONS"
▼ uniqueIdentifierstring

Unique identifier of the integration
RESPONSE

```
{ "creationDate": "example", "domain": { "hostname": "example", "protocol": "http" }, "id": "example",
"instanceVersion": "example", "lastUpdateDate": "example", "selectionType": "CURRENT_STATE", "selfSignedCertificate":
"example", "state": [ "example" ], "status": "COMPLETED", "type": "AWS_CODE_BUILD", "typeCategory": "DEFAULT",
"uniqueIdentifier": "example" }
```

2.4.6.3 | Update an AppSec Data Source

put /public_api/appsec/v1/integrations/{id}

Update some of the Application Security integration details.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ id String required
Example: `id_example`
Request headers
▼ Authorization String required

{api_key}

Example:
`UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9`
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241

CLIENT REQUEST Bash+curl

```
curl -X 'PUT'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/integrations/{id}'
-d '{ "externalProjects" : [ { "externalBranchName" : "externalBranchName", "repoId" : "repoId", "externalProjectId"
: "externalProjectId", "branchName" : "branchName", "externalId" : "externalId" }, { "externalBranchName" :
"externalBranchName", "repoId" : "repoId", "externalProjectId" : "externalProjectId", "branchName" : "branchName",
"externalId" : "externalId" } ], "state" : [ "state", "state" ] }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "
{\"selectionType\":\"CURRENT_STATE\",\"state\":[\"string\"],\"externalProjects\":
[{\"branchName\":\"string\",\"externalBranchName\":\"string\",\"externalId\":\"string\",\"externalProjectId\":\"string\",\"repo
headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("PUT",
"/public_api/appsec/v1/integrations/%7Bid%7D", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Put.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "
{\"selectionType\":\"CURRENT_STATE\",\"state\":[\"string\"],\"externalProjects\":
[{\"branchName\":\"string\",\"externalBranchName\":\"string\",\"externalId\":\"string\",\"externalProjectId\":\"string\",\"repo
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "selectionType": "CURRENT_STATE", "state": [ "string" ], "externalProjects": [ {
"branchName": "string", "externalBranchName": "string", "externalId": "string", "externalProjectId": "string",
"repoId": "string" } ] }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("PUT", "https://api-
yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.put("https://api-yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("
{\"selectionType\":\"CURRENT_STATE\",\"state\":[\"string\"],\"externalProjects\":
[{\"branchName\":\"string\",\"externalBranchName\":\"string\",\"externalId\":\"string\",\"externalProjectId\":\"string\",\"repo
.asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = [ "selectionType": "CURRENT_STATE",
"state": ["string"], "externalProjects": [ [ "branchName": "string", "externalBranchName": "string", "externalId":
"string", "externalProjectId": "string", "repoId": "string" ] ] ] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "PUT" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "PUT", CURLOPT_POSTFIELDS => "{\"selectionType\":\"CURRENT_STATE\",\"state\":[\"string\"],\"externalProjects\":
[{\"branchName\":\"string\",\"externalBranchName\":\"string\",\"externalId\":\"string\",\"externalProjectId\":\"string\",\"repo
CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "PUT"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization:
```

```
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"selectionType\":\"CURRENT_STATE\",\"state\":[\"string\"],\"externalProjects\":
[{\"branchName\":\"string\",\"externalBranchName\":\"string\",\"externalId\":\"string\",\"externalProjectId\":\"string\",\"repo
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D"); var request = new
RestRequest(Method.PUT); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"selectionType\":\"CURRENT_STATE\",\"state\":
[\"string\"],\"externalProjects\":
[{\"branchName\":\"string\",\"externalBranchName\":\"string\",\"externalId\":\"string\",\"externalProjectId\":\"string\",\"repo
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters
required
application/json

Define the integration details

▼ selectionTypestring (Enum)
Allowed values:"CURRENT_STATE""CURRENT_STATE_AND_FUTURE""CURRENT_STATE_PENDING""MANUAL_SELECTION"
▼ statearray[string]

To update the integration for specific repositories, specify them here in the following format:

```
"state":["org1"/"repo_name1", "org2"/"repo_name2"]
```

▼ externalProjectsarray
[
▶ branchNamestring

Name of the branch

▶ externalBranchNamestring

Name of the external branch

▶ externalIdstring

External ID

▶ externalProjectIdstring

External project ID

▶ repoIdstring

Repository ID

]
REQUEST

```
{ "selectionType": "CURRENT_STATE", "state": [ "example" ], "externalProjects": [ { "branchName": "example",
"externalBranchName": "example", "externalId": "example", "externalProjectId": "example", "repoId": "example" } ] }
```

Responses

Ok

Body
application/json

Integration details

▼ creationDatestring

Date the integration was created

▼ domainobject

Integration hostname and protocol

▶ hostnamestring

Integration hostname

▶ protocolstring (Enum)

Integration protocol
▼ idstring

Integration ID

▼ instanceVersionstring

Integration instance version

▼ lastUpdateDatestring

Date the integration was last updated

▼ selectionTypestring (Enum)
Allowed values:"CURRENT_STATE""CURRENT_STATE_AND_FUTURE""CURRENT_STATE_PENDING""MANUAL_SELECTION"
▼ selfSignedCertificatestring

Self-signed certificate

▼ statearray[string]

Integration state

▼ statusstring (Enum)

The status of the integration

Allowed values:"COMPLETED""CONNECTED""DESTROYING""ERROR""PENDING""TEMPORARY""WARNING"
▼ typestring (Enum)
Allowed
values:"AWS_CODE_BUILD""AWS_CODE_COMMIT""AZURE_REPOS""BITBUCKET""BITBUCKET_DATACENTER""CIRCLE_CI""CIRCLE_CI_CD""CORTEX_CLI""GITHUB""GITH
▼ typeCategorystring (Enum)
Allowed values:"DEFAULT""EXTERNAL_VENDOR_INTEGRATIONS"
▼ uniqueIdentifierstring

Unique identifier of the integration
RESPONSE

```
{ "creationDate": "example", "domain": { "hostname": "example", "protocol": "http" }, "id": "example",
"instanceVersion": "example", "lastUpdateDate": "example", "selectionType": "CURRENT_STATE", "selfSignedCertificate":
"example", "state": [ "example" ], "status": "COMPLETED", "type": "AWS_CODE_BUILD", "typeCategory": "DEFAULT",
"uniqueIdentifier": "example" }
```

2.4.6.4 | Delete an AppSec Data Source

delete /public_api/appsec/v1/integrations/{id}

Delete the specified Application Security integration.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ id String required
Example: `id_example`
Request headers
▼ Authorization String required

{api_key}

Example:

`UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `241`
CLIENT REQUEST Bash+curl
```
curl -X 'DELETE'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/integrations/{id}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p
```

```
", 'x-xdr-auth-id': "241" } conn.request("DELETE", "/public_api/appsec/v1/integrations/%7Bid%7D", headers=headers)
res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Delete.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("DELETE", "https://api-
yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.delete("https://api-yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "DELETE" request.allHTTPHeaderFields = headers let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "DELETE", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "DELETE"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/integrations/%7Bid%7D"); var request = new
RestRequest(Method.DELETE); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

No content

2.4.7 | Scan management

APIs for managing scans

2.4.7.1 | Get unscanned AppSec scan management repositories

get /public_api/appsec/v1/scans/unscanned-repositories

Get a list of Application Security repositories that have not been scanned. You can optionally filter by specifying the number of days past during which the repositories were not scanned.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers

▼ Authorization String required

{api_key}

Example:

```
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
```

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
Query parameters
▼ days Double

Define the time period in days previous to now

double
Example: 1.2
Default: 7
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/scans/unscanned-repositories?days=1.2'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/scans/unscanned-repositories?
days=SOME_NUMBER_VALUE", headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/scans/unscanned-repositories?days=SOME_NUMBER_VALUE") http = Net::HTTP.new(url.host,
url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url)
request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/appsec/v1/scans/unscanned-
repositories?days=SOME_NUMBER_VALUE"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/scans/unscanned-repositories?
days=SOME_NUMBER_VALUE") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/scans/unscanned-repositories?days=SOME_NUMBER_VALUE")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/scans/unscanned-repositories?days=SOME_NUMBER_VALUE", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/scans/unscanned-repositories?days=SOME_NUMBER_VALUE"); struct curl_slist
*headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/scans/unscanned-repositories?
days=SOME_NUMBER_VALUE"); var request = new RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```
Responses

Ok

Body
application/json
[
▶ organizationIdstring

Organization ID
▶ repoNamestring

Name of the repository

▶ repoIdstring

ID of the repository

]
RESPONSE
```
[ { "organizationId": "example", "repoName": "example", "repoId": "example" } ]
```

2.4.7.2 | Get AppSec branch periodic scans

get /public_api/appsec/v1/scans/periodic

Branch periodic scans are automated checks that assess the security posture of applications and infrastructure.

Get the details of branch periodic scans.

How to access branch periodic scans in the UI: Under Modules select Application Security > Branch Periodic Scanning (under Scans).

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: `241`
Query parameters
▼ organizationName String
Example: `organizationName_example`
▼ repositories array[String]
▼ branchName String
Example: `branchName_example`
▼ scanHealth ScanHealthEnum
▼ days Double
double
Example: `1.2`
Default: `7`
▼ offset Double
double
Example: `1.2`
Default: `0`
▼ limit Double
double
Example: `1.2`
Default: `100`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/scans/periodic?
organizationName=organizationName_example&repositories=&branchName=branchName_example&scanHealth=&days=1.2&offset=1.2&limit=1.2
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/scans/periodic?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days
headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/scans/periodic?
```

```
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/appsec/v1/scans/periodic?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days
xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/scans/periodic?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/scans/periodic?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days
as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET"
request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask = session.dataTask(with: request
as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/scans/periodic?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/scans/periodic?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days
struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/scans/periodic?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days
var request = new RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body

application/json

[

- ▶ issuesFoundobject
- ▶ findingsSeverityBreakdownobject
- ▶ findingsTypesBreakdownobject
- ▶ issuesTypesBreakdownobject
- ▶ repositoryIdstring

ID of the scanned repository

- ▶ integrationIdstring
- ▶ providerstring
- ▶ scanIdstring

Scan ID

- ▶ scanHealthstring (Enum)

The health of the scan. Valid values include:

- `ERROR`: Indicates an error with the scan
- `PARTIALLY_COMPLETED`: Indicates that the scan executed partially with some scan modules succeeded and others failing
- `IN_PROGRESS`: The scan is in progress
- `COMPLETED`: Indicates that the scan is complete

▶ scanDatestring

Timestamp of the last scan execution

▶ branchNamestring

The branch analyzed during the scan

▶ organizationNamestring

The organization owning the repository

▶ repoNamestring

Name of the scanned repository

]
RESPONSE

```
[ { "issuesFound": {}, "findingsSeverityBreakdown": {}, "findingsTypesBreakdown": {}, "issuesTypesBreakdown": {},
"repositoryId": "example", "integrationId": "example", "provider": "example", "scanId": "example", "scanHealth":
"COMPLETED", "scanDate": "example", "branchName": "example", "organizationName": "example", "repoName": "example" } ]
```

2.4.7.3 | Get AppSec Pull Request scans

get /public_api/appsec/v1/scans/pr

Pull Request (PR) scans are initiated by events triggered by version control systems such as GitHub, GitLab, Bitbucket and Azure Repos, or via webhooks. These scans are run on default or non-default branches containing open PRs or Merge Requests (MR) from your integrated repositories.

Get the details of PR scans.

How to access Pull Request scans in the UI: Under Modules select Application Security Pull Request Scans (under Scans).

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:

`UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `241`
Query parameters
▼ organizationName String
Example: `organizationName_example`
▼ repositories array[String]
▼ branchName String
Example: `branchName_example`
▼ prId String
Example: `prId_example`
▼ prTitle String
Example: `prTitle_example`
▼ prStatus ScanStatus
▼ scanHealth ScanHealthEnum
▼ days Double
double
Example: `1.2`
Default: `7`
▼ offset Double
double
Example: `1.2`
Default: `0`
▼ limit Double

double
Example: `1.2`
Default: `100`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/scans/pr?
organizationName=organizationName_example&repositories=&branchName=branchName_example&prId=prId_example&prTitle=prTitle_example
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/scans/pr?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&prId=SOME_STRING_VALUE&prTitle=SO
headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/appsec/v1/scans/pr?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&prId=SOME_STRING_VALUE&prTitle=SO
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/appsec/v1/scans/pr?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&prId=SOME_STRING_VALUE&prTitle=SO
xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/scans/pr?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&prId=SOME_STRING_VALUE&prTitle=SO
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/scans/pr?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&prId=SOME_STRING_VALUE&prTitle=SO
as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET"
request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask = session.dataTask(with: request
as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/scans/pr?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&prId=SOME_STRING_VALUE&prTitle=SO
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/scans/pr?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&prId=SOME_STRING_VALUE&prTitle=SO
struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/scans/pr?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&branchName=SOME_STRING_VALUE&prId=SOME_STRING_VALUE&prTitle=SO
var request = new RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```
Responses

Ok

Body

application/json
[
▶ issuesFoundobject
▶ findingsSeverityBreakdownobject
▶ findingsTypesBreakdownobject
▶ issuesTypesBreakdownobject
▶ pullRequestTitlestring
▶ integrationIdstring
▶ repositoryIdstring

ID of the scanned repository

▶ scanIdstring
▶ providerstring
▶ userEmailstring
▶ blockingPolicyNamestring
▶ blockingPolicyIdstring
▶ scanHealthstring (Enum)

The health of the scan. Valid values include:

- `ERROR`: Indicates an error with the scan
- `PARTIALLY_COMPLETED`: Indicates that the scan executed partially with some scan modules succeeded and others failing
- `IN_PROGRESS`: The scan is in progress
- `COMPLETED`: Indicates that the scan is complete

▶ scanDatestring

The timestamp of the last scan execution

▶ commitIdstring

The commit included in the PR

▶ pullRequestStatusstring (Enum)

Status of the PR scan. Valid values include:

- `PASSED`
- `PASSED_WITH_ISSUES`
- `BLOCKED`
- `IN_PROGRESS`
- `ERROR`
- `FAILED`

▶ pullRequestIdstring

The ID of the PR

▶ branchNamestring

Name of the branch analyzed during the scan

▶ organizationNamestring

Specifies the organization owning the repository

▶ repoNamestring
]
RESPONSE


```
[ { "issuesFound": {}, "findingsSeverityBreakdown": {}, "findingsTypesBreakdown": {}, "issuesTypesBreakdown": {},
"pullRequestTitle": "example", "integrationId": "example", "repositoryId": "example", "scanId": "example",
"provider": "example", "userEmail": "example", "blockingPolicyName": "example", "blockingPolicyId": "example",
"scanHealth": "COMPLETED", "scanDate": "example", "commitId": "example", "pullRequestStatus": "ERROR",
"pullRequestId": "example", "branchName": "example", "organizationName": "example", "repoName": "example" } ]
```

2.4.7.4 | Get AppSec CI scans

get /public_api/appsec/v1/scans/ci

CI scans detect exposed secrets, misconfigurations in your infrastructure-as-code (IaC) files, vulnerabilities in your software composition analysis (SCA) packages, and license non-compliance in your CI pipelines.

Get the details of CI scans.

How to access CI scans in the UI: Under Modules select Application Security > CI Scans (under Scans).

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management
Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
Query parameters
▼ organizationName String
Example: organizationName_example
▼ repositories array[String]
▼ ciStatus ScanStatus
▼ scanHealth ScanHealthEnum
▼ days Double
double
Example: 1.2
Default: 7
▼ offset Double
double
Example: 1.2
Default: 0
▼ limit Double
double
Example: 1.2
Default: 100
CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/scans/ci?
organizationName=organizationName_example&repositories=&ciStatus=&scanHealth=&days=1.2&offset=1.2&limit=1.2'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/scans/ci?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&ciStatus=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days=S
headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/appsec/v1/scans/ci?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&ciStatus=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days=S
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/appsec/v1/scans/ci?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&ciStatus=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days=S
xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/scans/ci?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&ciStatus=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days=S
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/scans/ci?
```

```
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&ciStatus=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days=S
as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET"
request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask = session.dataTask(with: request
as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/scans/ci?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&ciStatus=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days=S
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/scans/ci?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&ciStatus=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days=S
struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/scans/ci?
organizationName=SOME_STRING_VALUE&repositories=SOME_ARRAY_VALUE&ciStatus=SOME_STRING_VALUE&scanHealth=SOME_STRING_VALUE&days=S
var request = new RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
[
▶ issuesFoundobject
▶ findingsSeverityBreakdownobject
▶ findingsTypesBreakdownobject
▶ issuesTypesBreakdownobject
▶ repositoryIdstring

ID of the scanned repository

▶ integrationIdstring
▶ scanIdstring
▶ providerstring
▶ blockingPolicyNamestring
▶ blockingPolicyIdstring
▶ ciStatusstring (Enum)

Status of the PR scan. Valid values include:

- `PASSED`
- `PASSED_WITH_ISSUES`
- `BLOCKED`
- `IN_PROGRESS`
- `ERROR`
- `FAILED`

▶ scanHealthstring (Enum)

The health of the scan. Valid values include:

- `ERROR`: Indicates an error with the scan
- `PARTIALLY_COMPLETED`: Indicates that the scan executed partially with some scan modules succeeded and others failing
- `IN_PROGRESS`: The scan is in progress
- `COMPLETED`: Indicates that the scan is complete

▶ scanDatestring

The timestamp of the last scan execution

▶ branchNamestring

The name of the branch analyzed during the scan

▶ organizationNamestring

Name of the organization owning the repository

▶ repoNamestring

Name of the scanned repository

]
RESPONSE

```
[ { "issuesFound": {}, "findingsSeverityBreakdown": {}, "findingsTypesBreakdown": {}, "issuesTypesBreakdown": {},
"repositoryId": "example", "integrationId": "example", "scanId": "example", "provider": "example",
"blockingPolicyName": "example", "blockingPolicyId": "example", "ciStatus": "ERROR", "scanHealth": "COMPLETED",
"scanDate": "example", "branchName": "example", "organizationName": "example", "repoName": "example" } ]
```

2.4.7.5 | List AppSec scan issues

get /public_api/appsec/v1/scans/{scanId}/issues

Get a list of the issues discovered in the scan specified by scan ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ scanId String required
Example: `scanId_example`
Request headers
▼ Authorization String required

{api_key}

Example:

`UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W`

▼ x-xdr-auth-id String required

{api_key_id}

Example: `241`
Query parameters
▼ severity SeverityEnum
▼ offset Double
double
Example: `1.2`
Default: `0`
▼ limit Double
double
Example: `1.2`
Default: `100`
CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/scans/{scanId}/issues?severity=&offset=1.2&limit=1.2'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/scans/%7BscanId%7D/issues?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE", headers=headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/scans/%7BscanId%7D/issues?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE") http = Net::HTTP.new(url.host,
url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url)
request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
```

```
yourfqdn/public_api/appsec/v1/scans/%7BscanId%7D/issues?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/scans/%7BscanId%7D/issues?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/scans/%7BscanId%7D/issues?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/scans/%7BscanId%7D/issues?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/scans/%7BscanId%7D/issues?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/scans/%7BscanId%7D/issues?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE"); var request = new
RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body

application/json

[

- ▶ resolution_commentstring
- ▶ resolution_reasonstring (Enum)
- ▶ resolution_statusstring (Enum)
- ▶ assigneestring
- ▶ branchstring
- ▶ run_idstring
- ▶ pull_request_idstring
- ▶ repo_idstring
- ▶ createdstring
- ▶ file_pathstring
- ▶ asset_namestring
- ▶ detection_methodstring (Enum)
- ▶ alert_namestring
- ▶ severitystring (Enum)
- ▶ scan_idstring
- ▶ issue_idstring

]

RESPONSE


```
[ { "resolution_comment": "example", "resolution_reason": "Resolved - Known Issue", "resolution_status": "New",
"assignee": "example", "branch": "example", "run_id": "example", "pull_request_id": "example", "repo_id": "example",
"created": "example", "file_path": "example", "asset_name": "example", "detection_method": "Secrets Security",
"alert_name": "example", "severity": "Critical", "scan_id": "example", "issue_id": "example" } ]
```

2.4.7.6 | List AppSec scan findings

get /public_api/appsec/v1/scans/{scanId}/findings

Get a list of the findings discovered in the scan specified by scan ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ scanId String required
Example: `scanId_example`
Request headers
▼ Authorization String required

{api_key}

Example:
`UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `241`
Query parameters
▼ severity SeverityEnum
▼ offset Double
double
Example: `1.2`
Default: `0`
▼ limit Double
double
Example: `1.2`
Default: `100`
CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/scans/{scanId}/findings?severity=&offset=1.2&limit=1.2'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/scans/%7BscanId%7D/findings?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE", headers=headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/scans/%7BscanId%7D/findings?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE") http = Net::HTTP.new(url.host,
url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url)
request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/appsec/v1/scans/%7BscanId%7D/findings?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/scans/%7BscanId%7D/findings?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/scans/%7BscanId%7D/findings?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
```

```
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/scans/%7BscanId%7D/findings?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/scans/%7BscanId%7D/findings?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/scans/%7BscanId%7D/findings?
severity=SOME_STRING_VALUE&offset=SOME_NUMBER_VALUE&limit=SOME_NUMBER_VALUE"); var request = new
RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
[
▶ scan_sourcestring (Enum)
▶ git_userstring
▶ first_hash_timestring
▶ first_hashstring
▶ platform_idstring
▶ branchstring
▶ detection_methodstring (Enum)
▶ file_pathstring
▶ descriptionstring
▶ last_observedstring
▶ first_observedstring
▶ suppression_typestring (Enum)
▶ data_sourcestring (Enum)
▶ namestring
▶ categorystring
▶ asset_idstring
▶ asset_namestring
▶ severitystring (Enum)
▶ scan_idstring
▶ issue_namestring
]
RESPONSE

```
[ { "scan_source": "pr", "git_user": "example", "first_hash_time": "example", "first_hash": "example", "platform_id":
"example", "branch": "example", "detection_method": "Secrets Security", "file_path": "example", "description":
"example", "last_observed": "example", "first_observed": "example", "suppression_type": "Inline", "data_source":
"GitHub", "name": "example", "category": "example", "asset_id": "example", "asset_name": "example", "severity":
"Critical", "scan_id": "example", "issue_name": "example" } ]
```

2.4.7.7 | Rerun a repository scan

post /public_api/appsec/v1/scan/repository/{repositoryId}

Create a repository scan by specifying the branch name.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ repositoryId String required
Example: `repositoryId_example`

Request headers

▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9U

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241

Query parameters

▼ scanFullGitHistory Boolean

Example: true

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9U
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/scan/repository/{repositoryId}?scanFullGitHistory=true'
-d '{ "branchName" : "branchName" }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"branchName\":\"string\"}" headers
= { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST",
"/public_api/appsec/v1/scan/repository/%7BrepositoryId%7D?scanFullGitHistory=SOME_BOOLEAN_VALUE", payload, headers)
res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/scan/repository/%7BrepositoryId%7D?scanFullGitHistory=SOME_BOOLEAN_VALUE") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "
{\"branchName\":\"string\"}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "branchName": "string" }); const xhr = new XMLHttpRequest(); xhr.withCredentials =
true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/appsec/v1/scan/repository/%7BrepositoryId%7D?scanFullGitHistory=SOME_BOOLEAN_VALUE");
xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-
yourfqdn/public_api/appsec/v1/scan/repository/%7BrepositoryId%7D?scanFullGitHistory=SOME_BOOLEAN_VALUE")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"branchName\":\"string\"}")
.asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = ["branchName": "string"] as [String
: Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/appsec/v1/scan/repository/%7BrepositoryId%7D?
scanFullGitHistory=SOME_BOOLEAN_VALUE")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/scan/repository/%7BrepositoryId%7D?scanFullGitHistory=SOME_BOOLEAN_VALUE",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "
{\"branchName\":\"string\"}", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9U
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/appsec/v1/scan/repository/%7BrepositoryId%7D?
scanFullGitHistory=SOME_BOOLEAN_VALUE"); struct curl_slist *headers = NULL; headers = curl_slist_append(headers,
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"branchName\":\"string\"}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/scan/repository/%7BrepositoryId%7D?
scanFullGitHistory=SOME_BOOLEAN_VALUE"); var request = new RestRequest(Method.POST);
request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"branchName\":\"string\"}", ParameterType.RequestBody); IRestResponse
response = client.Execute(request);
```

Body parameters
required
application/json
▼ branchNamestring
REQUEST

```
{ "branchName": "example" }
```

Responses

Ok

Body
application/json
Any of ScanRepositorySuccessResponse
▶ metadataobjectrequired
▶ err_msgstring
▶ metadataobject
RESPONSE

```
{ "metadata": { "scan_id": "example" } }
```

2.4.8 | SBOM management

APIs for managing SBOMs

2.4.8.1 | Get an SBOM for the specified repository

get /public_api/appsec/v1/sbom/repository/{repoId}

Get an SBOM (Software Bill of Materials) report for the specified repository.

Only the following combinations are supported:

`format`: `cyclonedx` `fileType`: `json` or `xml` `version:` 1.4`or`1.5`or`1.6`

`format`: `spdx` `fileType`: `json` or `txt` `version:` 2.3`

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ repoId String required
Example: `repoId_example`
Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
Query parameters
▼ branchName String required

Branch name for which you want to export the SBOM.

Example: `branchName_example`

▼ fileType String required

Define the file type you want to export. For cyclonedx, either json or xml. For spdx, either json or txt.

Allowed values: json xml txt
Example: `fileType_example`
Default: `xml`

▼ version String required

For cyclonedx, version can be 1.4, 1.5, or 1.6. For spdx, version is 2.3.

Allowed values: 1.4 1.5 1.6 2.3
Example: `version_example`
Default: `1.6`

▼ format String required

Define the SBOM format.

Allowed values: cyclonedx spdx
Example: `format_example`
Default: `cyclonedx`

CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/sbom/repository/{repoId}?
branchName=branchName_example&fileType=fileType_example&version=version_example&format=format_example'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/sbom/repository/%7BrepoId%7D?
branchName=SOME_STRING_VALUE&fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE",
headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/sbom/repository/%7BrepoId%7D?
branchName=SOME_STRING_VALUE&fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/appsec/v1/sbom/repository/%7BrepoId%7D?
branchName=SOME_STRING_VALUE&fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE");
xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/sbom/repository/%7BrepoId%7D?
branchName=SOME_STRING_VALUE&fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/sbom/repository/%7BrepoId%7D?
branchName=SOME_STRING_VALUE&fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields =
headers let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/sbom/repository/%7BrepoId%7D?
branchName=SOME_STRING_VALUE&fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [
"Authorization:
```

```
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/sbom/repository/%7BrepoId%7D?
branchName=SOME_STRING_VALUE&fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE"); struct
curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/sbom/repository/%7BrepoId%7D?
branchName=SOME_STRING_VALUE&fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE"); var
request = new RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
▼ stringbyte
RESPONSE
`"Ym9uam91cg=="`

2.4.8.2 | Get all SBOMs for the specified organization

get /public_api/appsec/v1/sbom/organization/{orgName}

Get SBOM (Software Bill of Materials) reports for all repositories in the specified organization.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ orgName String required
Example: `orgName_example`
Request headers
▼ Authorization String required

{api_key}

Example:
`UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W`
▼ x-xdr-auth-id String required

{api_key_id}

Example: `241`
Query parameters
▼ fileType String required

Define the file type you want to export. For cyclonedx, either json or xml. For spdx, either json or txt.

Allowed values: json xml txt
Example: `fileType_example`
Default: `xml`
▼ version String required

For cyclonedx, version can be 1.4, 1.5, or 1.6. For spdx, version is 2.3.

Allowed values: 1.4 1.5 1.6 2.3
Example: `version_example`
Default: `1.6`
▼ format String required

Define the SBOM format.

Allowed values: cyclonedx spdx
Example: `format_example`
Default: `cyclonedx`
CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/appsec/v1/sbom/organization/{orgName}?
fileType=fileType_example&version=version_example&format=format_example'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/appsec/v1/sbom/organization/%7BorgName%7D?
fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE", headers=headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/sbom/organization/%7BorgName%7D?
fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE") http = Net::HTTP.new(url.host,
url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url)
request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/appsec/v1/sbom/organization/%7BorgName%7D?
fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE");
xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-
yourfqdn/public_api/appsec/v1/sbom/organization/%7BorgName%7D?
fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/sbom/organization/%7BorgName%7D?
fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/sbom/organization/%7BorgName%7D?
fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/sbom/organization/%7BorgName%7D?
fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/sbom/organization/%7BorgName%7D?
fileType=SOME_STRING_VALUE&version=SOME_STRING_VALUE&format=SOME_STRING_VALUE"); var request = new
RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body
application/json
▼ stringbyte
RESPONSE

```
"Ym9uam91cg=="
```

2.4.9 | Remediations

APIs for managing Remediation

2.4.9.1 | Get Fix Suggestion

get /public_api/appsec/v1/issues/fix/{issueId}/fix_suggestion

Retrieves a comprehensive fix suggestion for a specified application security issue. Use optional query parameters to include data such as the original code block, remediation instructions, or the suggested code fix.

Path parameters
▼ issueId String required

The unique identifier for the specific application security issue.

Example: `1234567`
Query parameters
▼ showCodeBlock Boolean

Specifies whether to include the original code block in the response.

Example: `true`
Default: `true`
▼ showRemediationInstruction Boolean

Specifies whether to include manual remediation instructions in the response.

Example: `true`
Default: `false`
▼ showSuggestedCodeBlock Boolean

Specifies whether to include the automated code fix in the response.

Example: `true`
Default: `true`
CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
'https://api-yourfqdn/public_api/appsec/v1/issues/fix/{issueId}/fix_suggestion?
showCodeBlock=true&showRemediationInstruction=true&showSuggestedCodeBlock=true'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("GET",
"/public_api/appsec/v1/issues/fix/1234567/fix_suggestion?
showCodeBlock=SOME_BOOLEAN_VALUE&showRemediationInstruction=SOME_BOOLEAN_VALUE&showSuggestedCodeBlock=SOME_BOOLEAN_VALUE")
res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/issues/fix/1234567/fix_suggestion?
showCodeBlock=SOME_BOOLEAN_VALUE&showRemediationInstruction=SOME_BOOLEAN_VALUE&showSuggestedCodeBlock=SOME_BOOLEAN_VALUE")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/appsec/v1/issues/fix/1234567/fix_suggestion?
showCodeBlock=SOME_BOOLEAN_VALUE&showRemediationInstruction=SOME_BOOLEAN_VALUE&showSuggestedCodeBlock=SOME_BOOLEAN_VALUE");
xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-
yourfqdn/public_api/appsec/v1/issues/fix/1234567/fix_suggestion?
showCodeBlock=SOME_BOOLEAN_VALUE&showRemediationInstruction=SOME_BOOLEAN_VALUE&showSuggestedCodeBlock=SOME_BOOLEAN_VALUE")
.asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/issues/fix/1234567/fix_suggestion?
showCodeBlock=SOME_BOOLEAN_VALUE&showRemediationInstruction=SOME_BOOLEAN_VALUE&showSuggestedCodeBlock=SOME_BOOLEAN_VALUE")!
as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/issues/fix/1234567/fix_suggestion?
showCodeBlock=SOME_BOOLEAN_VALUE&showRemediationInstruction=SOME_BOOLEAN_VALUE&showSuggestedCodeBlock=SOME_BOOLEAN_VALUE",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", ]); $response = curl_exec($curl); $err
= curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/issues/fix/1234567/fix_suggestion?
showCodeBlock=SOME_BOOLEAN_VALUE&showRemediationInstruction=SOME_BOOLEAN_VALUE&showSuggestedCodeBlock=SOME_BOOLEAN_VALUE");
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/issues/fix/1234567/fix_suggestion?
showCodeBlock=SOME_BOOLEAN_VALUE&showRemediationInstruction=SOME_BOOLEAN_VALUE&showSuggestedCodeBlock=SOME_BOOLEAN_VALUE");
var request = new RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```

Responses

Fix Suggestion Response

Body
application/json
▼ issueIdstring

Unique identifier for the issue.

▼ issueNamestring

Name of the security issue.

▼ existingCodeBlockstring

Original vulnerable code.

▼ remediationInstructionstring

Manual remediation instructions.

▼ suggestedCodeBlockstring

Automated fix suggestions.

RESPONSE

```
{ "issueId": "example", "issueName": "example", "existingCodeBlock": "example", "remediationInstruction": "example",
"suggestedCodeBlock": "example" }
```

2.4.9.2 | Trigger Fix Pull Request

post /public_api/appsec/v1/issues/fix/trigger_fix_pull_request

Create automated pull requests to fix multiple security issues in a single bulk operation.

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/appsec/v1/issues/fix/trigger_fix_pull_request'
-d '{ "fixBranchName" : "fixBranchName", "issueIds" : [ "1234567", "9876543" ], "title" : "title" }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "
{\"fixBranchName\":\"string\",\"title\":\"string\",\"issueIds\":[\"1234567\",\"9876543\"]}" headers = { 'content-
type': "application/json" } conn.request("POST", "/public_api/appsec/v1/issues/fix/trigger_fix_pull_request",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/issues/fix/trigger_fix_pull_request") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["content-
type"] = 'application/json' request.body = "{\"fixBranchName\":\"string\",\"title\":\"string\",\"issueIds\":
[\"1234567\",\"9876543\"]}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "fixBranchName": "string", "title": "string", "issueIds": [ "1234567", "9876543" ] });
const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () {
if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/appsec/v1/issues/fix/trigger_fix_pull_request"); xhr.setRequestHeader("content-type",
"application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-
yourfqdn/public_api/appsec/v1/issues/fix/trigger_fix_pull_request") .header("content-type", "application/json")
```

```
.body("{\"fixBranchName\":\"string\",\"title\":\"string\",\"issueIds\":[\"1234567\",\"9876543\"]}") .asString();
import Foundation let headers = ["content-type": "application/json"] let parameters = [ "fixBranchName": "string",
"title": "string", "issueIds": ["1234567", "9876543"] ] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/appsec/v1/issues/fix/trigger_fix_pull_request")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/issues/fix/trigger_fix_pull_request", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING
=> "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "
{\"fixBranchName\":\"string\",\"title\":\"string\",\"issueIds\":[\"1234567\",\"9876543\"]}", CURLOPT_HTTPHEADER => [
"content-type: application/json" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if
($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/appsec/v1/issues/fix/trigger_fix_pull_request"); struct curl_slist
*headers = NULL; headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"fixBranchName\":\"string\",\"title\":\"string\",\"issueIds\":[\"1234567\",\"9876543\"]}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/issues/fix/trigger_fix_pull_request"); var
request = new RestRequest(Method.POST); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"fixBranchName\":\"string\",\"title\":\"string\",\"issueIds\":
[\"1234567\",\"9876543\"]}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ fixBranchNamestring

The branch name for the fix pull request. If not specified, a custom branch will be used.

▼ titlestring

The title for the fix pull request. If not specified, a default message will be used.

▼ issueIdsarray[string]

A list of unique identifiers for the issues to be fixed. Maximum 10 IDs per request.

Example: `["1234567","9876543"]`

REQUEST

```
{ "fixBranchName": "example", "title": "example", "issueIds": [ "1234567", "9876543" ] }
```

Responses

Ok

Body

application/json

▼ messagestring

Error message if the process fails.

▼ triggeredPrsarray

Status of each issue.

[

▶ issueIdstring

A unique identifier for issue.

▶ statusstring (Enum)

PR status. Possible values: -`triggered`: If the PR is in triggered state. -`automated_fix_not_available`: If the PR is in automated_fix_not_available state.

]

▼ statusstring

Overall process status.

▼ remediationIdstring

Unique identifier to track the remediation process

▼ errorstring

Error details if the process fails.
RESPONSE

```
{ "message": "example", "triggeredPrs": [ { "issueId": "example", "status": "triggered" } ], "status": "example",
"remediationId": "example", "error": "example" }
```

2.4.9.3 I Get Fix Status

get /public_api/appsec/v1/issues/fix/{remediationId}

Retrieves the fix status of remediation processes using the remediation ID.

Path parameters
▼ remediationId String required

Remediation Id

Example: `remediationId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
'https://api-yourfqdn/public_api/appsec/v1/issues/fix/{remediationId}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("GET",
"/public_api/appsec/v1/issues/fix/%7BremediationId%7D") res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/issues/fix/%7BremediationId%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl
= true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) response =
http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/appsec/v1/issues/fix/%7BremediationId%7D"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-
yourfqdn/public_api/appsec/v1/issues/fix/%7BremediationId%7D") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/issues/fix/%7BremediationId%7D")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "GET" let session = URLSession.shared let dataTask =
session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error !=
nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/issues/fix/%7BremediationId%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING =>
"", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "GET", ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if
($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/issues/fix/%7BremediationId%7D"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/issues/fix/%7BremediationId%7D"); var request
= new RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```
Responses

Ok

Body
application/json
▼ isProcessDoneboolean

Indicates whether all remediation processes are complete.

▼ resultsarray

Remediation results grouped by repository.

[
▶ prUrlstring

URL of the created pull request.

▶ errorMessagestring

Error details if the process failed.
▶ remediationRepositoryIdstring

Internal repository identifier.

▶ failureIssueIdsarray[string]
▶ successfulIssueIdsarray[string]
]
RESPONSE
{ "isProcessDone": false, "results": [{ "prUrl": "example", "errorMessage": "example", "remediationRepositoryId": "example", "failureIssueIds": ["example"], "successfulIssueIds": ["example"] }] }

2.4.10 | Criteria

APIs for managing Criteria

2.4.10.1 | Get all criteria

get /public_api/appsec/v1/application/criteria/all

Retrieves a paginated list of all criteria.

Query parameters
▼ page Double

Current page number in the paginated response.

double
Example: `1.2`
▼ pageSize Double

Total number of items in the current page.

double
Example: `1.2`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
'https://api-yourfqdn/public_api/appsec/v1/application/criteria/all?page=1.2&pageSize=1.2'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("GET",
"/public_api/appsec/v1/application/criteria/all?page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE") res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/application/criteria/all?page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/appsec/v1/application/criteria/all?page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE");
xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/appsec/v1/application/criteria/all?
page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/application/criteria/all?page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/application/criteria/all?page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", ]); $response = curl_exec($curl); $err
= curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/application/criteria/all?
page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/application/criteria/all?
page=SOME_NUMBER_VALUE&pageSize=SOME_NUMBER_VALUE"); var request = new RestRequest(Method.GET); IRestResponse
response = client.Execute(request);
```

Responses

Ok

Body
application/json
▼ itemsarray
[
▶ idstring

Unique Id of the criteria.

▶ namestring

Unique name of the criteria.

▶ descriptionstring

Criteria description.

▶ creationMethodstring (Enum)

Creation method for the criteria. Possible values: -`manual`: For the criteria created manually. -`automatic`: Criteria that are automatically created.

▶ createdBystring

Owner of the criteria.

▶ createdAtstringdate-time

Timestamp when the criteria was created.

▶ lastUpdatedstringdate-time

Timestamp when the criteria was updated.

▶ deletedAtstringdate-time

Timestamp when the criteria was deleted.

▶ metadataobject
▶ typestring (Enum)

Criteria Type. Allowed values - `cloud`.

▶ configobjectrequired
]
▼ currentPagenumberdouble

Current page number in the paginated response.

▼ totalPagesnumberdouble

Total number of pages for the paginated response.

▼ pageSizenumberdouble

Number of entries in each page.

▼ totalItemsnumberdouble

Total number of items in the response.

RESPONSE

```
{ "items": [ { "id": "example", "name": "example", "description": "example", "creationMethod": "manual", "createdBy": "example", "createdAt": "2020-01-01T12:00:00Z", "lastUpdated": "2020-01-01T12:00:00Z", "deletedAt": "2020-01-01T12:00:00Z", "metadata": { "applicationCount": 0.1, "assetCount": 0.1, "refreshStatus": "in_progress", "deletedFromUai": false }, "type": "cloud", "config": { "unifyWithinProvider": false, "unifyAcrossProviders": false, "criteriaDefinitions": [ { "type": "tags", "provider": "GITLAB", "tagKeys": [ "example" ] } ], "applicationMetaData": { "applicationNameSourceTags": "example", "businessCriticalitySourceTags": "example", "businessOwnerSourceTags": [ "example" ], "businessUnitSourceTags": "example" } } } ], "currentPage": 0.1, "totalPages": 0.1, "pageSize": 0.1, "totalItems": 0.1 }
```

2.4.10.2 | Get a Criteria by ID

get /public_api/appsec/v1/application/criteria/{criteriaId}

Retrieves detailed information for a specific criteria using its unique Id.

Path parameters

▼ criteriaId String required

Unique identifier for the criteria

Example: `criteriaId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
'https://api-yourfqdn/public_api/appsec/v1/application/criteria/{criteriaId}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("GET",
"/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D") res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) response =
http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-
yourfqdn/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "GET" let session = URLSession.shared let dataTask =
session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error !=
nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D"); var
request = new RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```

Responses

Ok

Body

application/json

▼ idstring

Unique Id of the criteria.

▼ namestring

Unique name of the criteria.

▼ descriptionstring

Criteria description.

▼ creationMethodstring (Enum)

Creation method for the criteria. Possible values: -`manual`: For the criteria created manually. -`automatic`: Criteria that are automatically created.

Allowed values:"manual""automatic"

▼ createdBystring

Owner of the criteria.

▼ createdAtstringdate-time

Timestamp when the criteria was created.

▼ lastUpdatedstringdate-time

Timestamp when the criteria was updated.

▼ deletedAtstringdate-time

Timestamp when the criteria was deleted.

▼ metadataobject
▶ applicationCountnumberdouble
▶ assetCountnumberdouble
▶ refreshStatusstring (Enum)
▶ deletedFromUaiboolean
▼ typestring (Enum)

Criteria Type. Allowed values - `cloud`.

Allowed values:"cloud"
▼ configobjectrequired
▶ unifyWithinProviderboolean
▶ unifyAcrossProvidersboolean
▶ criteriaDefinitionsarray
▶ applicationMetaDataobject
RESPONSE

```
{ "id": "example", "name": "example", "description": "example", "creationMethod": "manual", "createdBy": "example",
"createdAt": "2020-01-01T12:00:00Z", "lastUpdated": "2020-01-01T12:00:00Z", "deletedAt": "2020-01-01T12:00:00Z",
"metadata": { "applicationCount": 0.1, "assetCount": 0.1, "refreshStatus": "in_progress", "deletedFromUai": false },
"type": "cloud", "config": { "unifyWithinProvider": false, "unifyAcrossProviders": false, "criteriaDefinitions": [ {
"type": "tags", "provider": "GITLAB", "tagKeys": [ "example" ] } ], "applicationMetaData": {
"applicationNameSourceTags": "example", "businessCriticalitySourceTags": "example", "businessOwnerSourceTags": [
"example" ], "businessUnitSourceTags": "example" } } }
```

2.4.10.3 | Delete a Criteria

delete /public_api/appsec/v1/application/criteria/{criteriaId}

Deletes a specific criteria identified by its unique Id.

Path parameters
▼ criteriaId String required

Unique identifier for the criteria

Example: `criteriaId_example`
Request headers
▼ x-cas-trace-id String
Example: `xCasTraceId_example`
CLIENT REQUEST Bash+curl
```
curl -X 'DELETE'
-H 'x-cas-trace-id: xCasTraceId_example'
'https://api-yourfqdn/public_api/appsec/v1/application/criteria/{criteriaId}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'x-cas-trace-id':
"SOME_STRING_VALUE" } conn.request("DELETE", "/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D",
headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Delete.new(url) request["x-cas-
trace-id"] = 'SOME_STRING_VALUE' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("DELETE", "https://api-
yourfqdn/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D"); xhr.setRequestHeader("x-cas-trace-id",
"SOME_STRING_VALUE"); xhr.send(data);
HttpResponse<String> response = Unirest.delete("https://api-
yourfqdn/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D") .header("x-cas-trace-id", "SOME_STRING_VALUE")
.asString();
import Foundation let headers = ["x-cas-trace-id": "SOME_STRING_VALUE"] let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "DELETE"
request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask = session.dataTask(with: request
as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "DELETE", CURLOPT_HTTPHEADER => [ "x-cas-trace-id: SOME_STRING_VALUE"
], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" .
$err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "DELETE"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D"); struct curl_slist
*headers = NULL; headers = curl_slist_append(headers, "x-cas-trace-id: SOME_STRING_VALUE"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/application/criteria/%7BcriteriaId%7D"); var
request = new RestRequest(Method.DELETE); request.AddHeader("x-cas-trace-id", "SOME_STRING_VALUE"); IRestResponse
response = client.Execute(request);
```

Responses

No content

2.4.10.4 | Create a Criteria

post /public_api/appsec/v1/application/criteria

Create a Criteria

Request headers

▼ x-cas-trace-id String

Example: `xCasTraceId_example`

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'x-cas-trace-id: xCasTraceId_example'
'https://api-yourfqdn/public_api/appsec/v1/application/criteria'
-d '{ "name" : "name", "description" : "description", "config" : { "applicationMetaData" : {
"businessOwnerSourceTags" : [ "businessOwnerSourceTags", "businessOwnerSourceTags" ], "businessUnitSourceTags" :
"businessUnitSourceTags", "businessCriticalitySourceTags" : "businessCriticalitySourceTags",
"applicationNameSourceTags" : "applicationNameSourceTags" }, "unifyAcrossProviders" : true, "unifyWithinProvider" :
true, "criteriaDefinitions" : [ { "tagKeys" : [ "tagKeys", "tagKeys" ] }, { "tagKeys" : [ "tagKeys", "tagKeys" ] } ]
} }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "
{\"name\":\"string\",\"description\":\"string\",\"type\":\"cloud\",\"config\":
{\"unifyWithinProvider\":true,\"unifyAcrossProviders\":true,\"criteriaDefinitions\":
[{\"type\":\"tags\",\"provider\":\"GITLAB\",\"tagKeys\":[\"string\"]}],\"applicationMetaData\":
{\"applicationNameSourceTags\":\"string\",\"businessCriticalitySourceTags\":\"string\",\"businessOwnerSourceTags\":
[\"string\"],\"businessUnitSourceTags\":\"string\"}}}" headers = { 'x-cas-trace-id': "SOME_STRING_VALUE", 'content-
type': "application/json" } conn.request("POST", "/public_api/appsec/v1/application/criteria", payload, headers) res
= conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/appsec/v1/application/criteria") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-cas-trace-id"] =
'SOME_STRING_VALUE' request["content-type"] = 'application/json' request.body = "
{\"name\":\"string\",\"description\":\"string\",\"type\":\"cloud\",\"config\":
{\"unifyWithinProvider\":true,\"unifyAcrossProviders\":true,\"criteriaDefinitions\":
[{\"type\":\"tags\",\"provider\":\"GITLAB\",\"tagKeys\":[\"string\"]}],\"applicationMetaData\":
{\"applicationNameSourceTags\":\"string\",\"businessCriticalitySourceTags\":\"string\",\"businessOwnerSourceTags\":
[\"string\"],\"businessUnitSourceTags\":\"string\"}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "name": "string", "description": "string", "type": "cloud", "config": {
"unifyWithinProvider": true, "unifyAcrossProviders": true, "criteriaDefinitions": [ { "type": "tags", "provider":
"GITLAB", "tagKeys": [ "string" ] } ], "applicationMetaData": { "applicationNameSourceTags": "string",
"businessCriticalitySourceTags": "string", "businessOwnerSourceTags": [ "string" ], "businessUnitSourceTags":
"string" } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/appsec/v1/application/criteria"); xhr.setRequestHeader("x-cas-trace-id", "SOME_STRING_VALUE");
xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/appsec/v1/application/criteria")
.header("x-cas-trace-id", "SOME_STRING_VALUE") .header("content-type", "application/json") .body("
{\"name\":\"string\",\"description\":\"string\",\"type\":\"cloud\",\"config\":
```

{\"unifyWithinProvider\":true,\"unifyAcrossProviders\":true,\"criteriaDefinitions\":
[{\"type\":\"tags\",\"provider\":\"GITLAB\",\"tagKeys\":[\"string\"]}],\"applicationMetaData\":
{\"applicationNameSourceTags\":\"string\",\"businessCriticalitySourceTags\":\"string\",\"businessOwnerSourceTags\":
[\"string\"],\"businessUnitSourceTags\":\"string\"}}}") .asString();
import Foundation let headers = ["x-cas-trace-id": "SOME_STRING_VALUE", "content-type": "application/json"] let
parameters = ["name": "string", "description": "string", "type": "cloud", "config": ["unifyWithinProvider": true,
"unifyAcrossProviders": true, "criteriaDefinitions": [["type": "tags", "provider": "GITLAB", "tagKeys": ["string"]
]], "applicationMetaData": ["applicationNameSourceTags": "string", "businessCriticalitySourceTags": "string",
"businessOwnerSourceTags": ["string"], "businessUnitSourceTags": "string"]]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/appsec/v1/application/criteria")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [CURLOPT_URL => "https://api-
yourfqdn/public_api/appsec/v1/application/criteria", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"name\":\"string\",\"description\":\"string\",\"type\":\"cloud\",\"config\":
{\"unifyWithinProvider\":true,\"unifyAcrossProviders\":true,\"criteriaDefinitions\":
[{\"type\":\"tags\",\"provider\":\"GITLAB\",\"tagKeys\":[\"string\"]}],\"applicationMetaData\":
{\"applicationNameSourceTags\":\"string\",\"businessCriticalitySourceTags\":\"string\",\"businessOwnerSourceTags\":
[\"string\"],\"businessUnitSourceTags\":\"string\"}}}", CURLOPT_HTTPHEADER => ["content-type: application/json", "x-
cas-trace-id: SOME_STRING_VALUE"],]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if
($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/appsec/v1/application/criteria"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "x-cas-trace-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"name\":\"string\",\"description\":\"string\",\"type\":\"cloud\",\"config\":
{\"unifyWithinProvider\":true,\"unifyAcrossProviders\":true,\"criteriaDefinitions\":
[{\"type\":\"tags\",\"provider\":\"GITLAB\",\"tagKeys\":[\"string\"]}],\"applicationMetaData\":
{\"applicationNameSourceTags\":\"string\",\"businessCriticalitySourceTags\":\"string\",\"businessOwnerSourceTags\":
[\"string\"],\"businessUnitSourceTags\":\"string\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/appsec/v1/application/criteria"); var request = new
RestRequest(Method.POST); request.AddHeader("x-cas-trace-id", "SOME_STRING_VALUE"); request.AddHeader("content-type",
"application/json"); request.AddParameter("application/json", "
{\"name\":\"string\",\"description\":\"string\",\"type\":\"cloud\",\"config\":
{\"unifyWithinProvider\":true,\"unifyAcrossProviders\":true,\"criteriaDefinitions\":
[{\"type\":\"tags\",\"provider\":\"GITLAB\",\"tagKeys\":[\"string\"]}],\"applicationMetaData\":
{\"applicationNameSourceTags\":\"string\",\"businessCriticalitySourceTags\":\"string\",\"businessOwnerSourceTags\":
[\"string\"],\"businessUnitSourceTags\":\"string\"}}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);

Body parameters

required

application/json

▼ namestring

Unique name for the criteria.

▼ descriptionstring

Optional description for the criteria.

▼ typestring (Enum)

Criteria Type. Allowed values - `cloud`.

Allowed values:"cloud"

▼ configobjectrequired

▶ unifyWithinProviderboolean

▶ unifyAcrossProvidersboolean

▶ criteriaDefinitionsarray

▶ applicationMetaDataobject

REQUEST

{ "name": "example", "description": "example", "type": "cloud", "config": { "unifyWithinProvider": false,
"unifyAcrossProviders": false, "criteriaDefinitions": [{ "type": "tags", "provider": "GITLAB", "tagKeys": [
"example"] }], "applicationMetaData": { "applicationNameSourceTags": "example", "businessCriticalitySourceTags":
"example", "businessOwnerSourceTags": ["example"], "businessUnitSourceTags": "example" } } }

Responses

Ok

Body
application/json
▼ idstring

Unique Id of the criteria.

▼ namestring

Unique name of the criteria.

RESPONSE
```
{ "id": "example", "name": "example" }
```

2.5 | Vulnerability Intelligence public APIs

2.5.1 | Preface

You can export the OpenAPI specs in the Cortex Stoplight instance.

Vulnerability Intelligence is a real-time feed that contains vulnerability data and threat intelligence from a variety of certified upstream sources. This feed continuously pulls data from known vulnerability databases, official vendor feeds and commercial providers to provide the most accurate vulnerability detection results.

In addition to the information collected from official feeds, Vulnerability Intelligence is enriched with data curated by a dedicated research team. Our security researchers monitor cloud and open-source projects to identify security issues through automated and manual means. As a result, we can detect new vulnerabilities that were only recently disclosed, and even vulnerabilities that were quietly patched.

Use these public APIs to obtain details on vulnerabilities and affected software.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Version: Cloud 1.3

2.5.1.1 | Servers

https://api-yourfqdn

2.5.1.2 | Authentication methods

▼ Authorization
Api Key "x-api-key"

Authorized requests to the API should use: **x-api-key** inside **a header** with the access token.

▼ x-xdr-auth-id
Api Key "x-api-key-id"

Authorized requests to the API should use: **x-api-key-id** inside **a header** with the access token.

2.5.2 | Vulnerabilities

APIs for managing vulnerabilities

2.5.2.1 | Get list of vulnerabilities

post /public_api/uvem/v1/get_vulnerabilities

Get a list of vulnerabilities that match the filter fields. The list includes key information about each vulnerability.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/uvem/v1/get_vulnerabilities'
-d ''
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"attack_vector\",\"operator\":\"string\",\"value\":\"string\"}],\"sort\":
{\"field\":\"string\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":500,\"use_page_token\":true,\"next_page_token\":\"st
headers = { 'content-type': "application/json" } conn.request("POST", "/public_api/uvem/v1/get_vulnerabilities",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/uvem/v1/get_vulnerabilities") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["content-type"] =
'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"attack_vector\",\"operator\":\"string\",\"value\":\"string\"}],\"sort\":
{\"field\":\"string\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":500,\"use_page_token\":true,\"next_page_token\":\"st
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "attack_vector", "operator": "string",
"value": "string" } ], "sort": { "field": "string", "keyword": "asc" }, "search_from": 0, "search_to": 500,
"use_page_token": true, "next_page_token": "string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials =
true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/uvem/v1/get_vulnerabilities"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/uvem/v1/get_vulnerabilities")
.header("content-type", "application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"attack_vector\",\"operator\":\"string\",\"value\":\"string\"}],\"sort\":
{\"field\":\"string\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":500,\"use_page_token\":true,\"next_page_token\":\"st
.asString();
import Foundation let headers = ["content-type": "application/json"] let parameters = ["request_data": [ "filters": [
[ "field": "attack_vector", "operator": "string", "value": "string" ] ], "sort": [ "field": "string", "keyword":
"asc" ], "search_from": 0, "search_to": 500, "use_page_token": true, "next_page_token": "string" ]] as [String : Any]
let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/uvem/v1/get_vulnerabilities")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/uvem/v1/get_vulnerabilities", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"attack_vector\",\"operator\":\"string\",\"value\":\"string\"}],\"sort\":
{\"field\":\"string\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":500,\"use_page_token\":true,\"next_page_token\":\"st
CURLOPT_HTTPHEADER => [ "content-type: application/json" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/uvem/v1/get_vulnerabilities"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER,
headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"attack_vector\",\"operator\":\"string\",\"value\":\"string\"}],\"sort\":
{\"field\":\"string\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":500,\"use_page_token\":true,\"next_page_token\":\"st
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/uvem/v1/get_vulnerabilities"); var request = new
RestRequest(Method.POST); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"request_data\":{\"filters\":
[{\"field\":\"attack_vector\",\"operator\":\"string\",\"value\":\"string\"}],\"sort\":
{\"field\":\"string\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":500,\"use_page_token\":true,\"next_page_token\":\"st
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ filtersarray

An array of filter fields

▶ sortobject

▶ search_frominteger

An integer representing the start offset index of results Default value - 0

▶ search_tointeger

An integer representing the start offset index of results. Use this field to specify the number of results on a page when using page token pagination. Default value - 500

▶ use_page_tokenboolean

Use `"use_page_token":true` in the initial request to paginate the response data.

▶ next_page_tokenstring

If `"use_page_token":true` was included in the initial request, the response for that request will include a page token. Use `"next_page_token":"string"` to pass that page token into the next request to paginate the next set of data.

REQUEST

```
{ "request_data": { "filters": [ { "field": "attack_vector", "operator": "example", "value": "example" } ], "sort": { "field": "example", "keyword": "asc" }, "search_from": 0, "search_to": 0, "use_page_token": false, "next_page_token": "example" } }
```

Responses

OK

Body
application/json
▼ replyobject
▶ result_countinteger
▶ total_countinteger
▶ vulnerabilitiesarray
▶ next_page_tokenstring
RESPONSE

```
{ "reply": { "result_count": 0, "total_count": 0, "vulnerabilities": [ { "vulnerability_id": "example", "epss_score": 0, "cvss_score": 0, "cvss_severity": "example", "package_names": [ "example" ], "vendors": [ "example" ], "distribution_and_releases": [ "example" ], "cisa_kev": {}, "cvss_severity_source": "example", "cvss_score_source": "example", "first_published": 0, "last_modified": 0, "reported_exploited_by": [ {} ], "attack_vector": "example", "cvss_version": "example" } ], "next_page_token": "example" } }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body
application/json
▼ replyobject
▶ err_codestring
▶ err_msgstring
▶ err_extrastring
RESPONSE

```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```


Internal server error. A unified status for API communication type errors.

Body
application/json
▼ replyobject
▶ err_codestring
▶ err_msgstring
▶ err_extrastring
RESPONSE

```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

2.5.2.2 | Get vulnerability details

get /public_api/uvem/v1/vulnerabilities

Get detailed information about a specific vulnerability.

For example, the following request retrieves detailed information about CVE-2021-28799: https://api-{{fqdn}}/public_api/uvem/v1/vulnerabilities?vulnerabilityId=CVE-2021-28799

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ authorization String required

api-key

Example: `{{api-key}}`
▼ x-xdr-auth-id String required

api-key-id
Example: {{api_key_id}}
Query parameters
▼ vulnerabilityId String required

Vulnerability ID

Example: CVE-2023-1234
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json'
-H 'authorization: {{api-key}}' -H 'x-xdr-auth-id: {{api_key_id}}'
'https://api-yourfqdn/public_api/uvem/v1/vulnerabilities?vulnerabilityId=CVE-2023-1234'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'authorization': "{{api-key}}", 'x-xdr-auth-id': "{{api_key_id}}" } conn.request("GET", "/public_api/uvem/v1/vulnerabilities?vulnerabilityId=CVE-2023-1234", headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/uvem/v1/vulnerabilities?vulnerabilityId=CVE-2023-1234") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["authorization"] = '{{api-key}}' request["x-xdr-auth-id"] = '{{api_key_id}}' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/uvem/v1/vulnerabilities?vulnerabilityId=CVE-2023-1234"); xhr.setRequestHeader("authorization", "{{api-key}}"); xhr.setRequestHeader("x-xdr-auth-id", "{{api_key_id}}"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/uvem/v1/vulnerabilities?vulnerabilityId=CVE-2023-1234") .header("authorization", "{{api-key}}") .header("x-xdr-auth-id", "{{api_key_id}}") .asString();
import Foundation let headers = [ "authorization": "{{api-key}}", "x-xdr-auth-id": "{{api_key_id}}" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/uvem/v1/vulnerabilities?vulnerabilityId=CVE-2023-1234")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/uvem/v1/vulnerabilities?vulnerabilityId=CVE-2023-1234", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [ "authorization: {{api-key}}", "x-xdr-auth-id: {{api_key_id}}" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL, "https://api-yourfqdn/public_api/uvem/v1/vulnerabilities?vulnerabilityId=CVE-2023-1234"); struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "authorization: {{api-key}}"); headers = curl_slist_append(headers, "x-xdr-auth-id: {{api_key_id}}"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/uvem/v1/vulnerabilities?vulnerabilityId=CVE-2023-1234"); var request = new RestRequest(Method.GET); request.AddHeader("authorization", "{{api-key}}"); request.AddHeader("x-xdr-auth-id", "{{api_key_id}}"); IRestResponse response = client.Execute(request);
```
Responses

OK

Body
application/json
▼ cvssobject
▶ baseScoreMetricsobject
▶ scorenumber
▶ scoreSourcestring
▶ severitystring
▶ severitySourcestring
▶ vectorStringstring
▶ versionstring
▼ descriptionstring
▼ epss_scorenumber
▼ exploitDetailsobject
▶ commercialExploitFoundboolean
▶ exploitMaturitystring
▶ firstReportedThreatActorinteger
▶ mostRecentReportedThreatActorinteger

▶ publicExploitFoundboolean
▶ reportedExploitedboolean
▶ reportedExploitedByBotnetsboolean
▶ reportedExploitedByRansomwareboolean
▶ reportedExploitedByThreatActorsboolean
▶ weaponizedExploitFoundboolean
▼ fixVersionsstring
▼ isKevboolean
▼ temporalCvssobject
▶ confidencestring
▶ scorenumber
▶ vectorStringstring
▶ versionstring
▼ vulnerabilityIDstring
RESPONSE

```
{ "cvss": { "baseScoreMetrics": { "attackComplexity": "example", "attackVector": "example", "availabilityImpact":
"example", "confidentialityImpact": "example", "integrityImpact": "example", "privilegesRequired": "example",
"scope": "example", "userInteraction": "example" }, "score": 0.1, "scoreSource": "example", "severity": "example",
"severitySource": "example", "vectorString": "example", "version": "example" }, "description": "example",
"epss_score": 0.1, "exploitDetails": { "commercialExploitFound": false, "exploitMaturity": "example",
"firstReportedThreatActor": 0, "mostRecentReportedThreatActor": 0, "publicExploitFound": false, "reportedExploited":
false, "reportedExploitedByBotnets": false, "reportedExploitedByRansomware": false,
"reportedExploitedByThreatActors": false, "weaponizedExploitFound": false }, "fixVersions": "example", "isKev":
false, "temporalCvss": { "confidence": "example", "score": 0.1, "vectorString": "example", "version": "example" },
"vulnerabilityID": "example" }
```

2.5.3 | Affected software

APIs for managing affected software

2.5.3.1 | Get affected software

post /public_api/uvem/v1/get_affected_software

Get a filtered list of the software affected by one or more vulnerabilities.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api-key}

Example: `authorization_example`
▼ x-xdr-auth-id String required

{api-key-id}

Example: `xXdrAuthId_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'Authorization: authorization_example' -H 'x-xdr-auth-id: xXdrAuthId_example'
'https://api-yourfqdn/public_api/uvem/v1/get_affected_software'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"affected_cpu_archs\",\"operator\":\"contains\",\"value\":[\"string\"]}],\"sort\":
{\"field\":\"string\",\"keyword\":\"string\"},\"search_from\":0,\"search_to\":500,\"use_page_token\":true,\"next_page_token\":"
headers = { 'Authorization': "SOME_STRING_VALUE", 'x-xdr-auth-id': "SOME_STRING_VALUE", 'content-type':
"application/json" } conn.request("POST", "/public_api/uvem/v1/get_affected_software", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/uvem/v1/get_affected_software") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'SOME_STRING_VALUE' request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"affected_cpu_archs\",\"operator\":\"contains\",\"value\":[\"string\"]}],\"sort\":
```

{\"field\":\"string\",\"keyword\":\"string\"},\"search_from\":0,\"search_to\":500,\"use_page_token\":true,\"next_page_token\":\
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [{ "field": "affected_cpu_archs", "operator": "contains",
"value": ["string"] }], "sort": { "field": "string", "keyword": "string" }, "search_from": 0, "search_to": 500,
"use_page_token": true, "next_page_token": "string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials =
true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/uvem/v1/get_affected_software"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/uvem/v1/get_affected_software")
.header("Authorization", "SOME_STRING_VALUE") .header("x-xdr-auth-id", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"affected_cpu_archs\",\"operator\":\"contains\",\"value\":[\"string\"]}],\"sort\":
{\"field\":\"string\",\"keyword\":\"string\"},\"search_from\":0,\"search_to\":500,\"use_page_token\":true,\"next_page_token\":\
.asString();
import Foundation let headers = ["Authorization": "SOME_STRING_VALUE", "x-xdr-auth-id": "SOME_STRING_VALUE",
"content-type": "application/json"] let parameters = ["request_data": ["filters": [["field":
"affected_cpu_archs", "operator": "contains", "value": ["string"]]], "sort": ["field": "string", "keyword":
"string"], "search_from": 0, "search_to": 500, "use_page_token": true, "next_page_token": "string"]] as [String :
Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/uvem/v1/get_affected_software")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [CURLOPT_URL => "https://api-
yourfqdn/public_api/uvem/v1/get_affected_software", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"affected_cpu_archs\",\"operator\":\"contains\",\"value\":[\"string\"]}],\"sort\":
{\"field\":\"string\",\"keyword\":\"string\"},\"search_from\":0,\"search_to\":500,\"use_page_token\":true,\"next_page_token\":\
CURLOPT_HTTPHEADER => ["Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id:
SOME_STRING_VALUE"],]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo
"cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/uvem/v1/get_affected_software"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "x-
xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"filters\":[{\"field\":\"affected_cpu_archs\",\"operator\":\"contains\",\"value\":[\"string\"]}],\"sort\":
{\"field\":\"string\",\"keyword\":\"string\"},\"search_from\":0,\"search_to\":500,\"use_page_token\":true,\"next_page_token\":\
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/uvem/v1/get_affected_software"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("x-xdr-auth-id",
"SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json",
"{\"request_data\":{\"filters\":[{\"field\":\"affected_cpu_archs\",\"operator\":\"contains\",\"value\":
[\"string\"]}],\"sort\":
{\"field\":\"string\",\"keyword\":\"string\"},\"search_from\":0,\"search_to\":500,\"use_page_token\":true,\"next_page_token\":\
ParameterType.RequestBody); IRestResponse response = client.Execute(request);

Body parameters

application/json

▼ request_dataobject

▶ filtersarray

▶ sortobject

▶ search_frominteger

An integer representing the start offset index of results Default value - 0

▶ search_tointeger

An integer representing the start offset index of results. Use this field to specify the number of results on a page when using page token pagination. Default value - 500

▶ use_page_tokenboolean

Use `"use_page_token":true` in the initial request to paginate the response data.

▶ next_page_tokenstring

If `"use_page_token":true` was included in the initial request, the response for that request will include a page token. Use `"next_page_token":"string"` to pass that page token into the next request to paginate the next set of data.

REQUEST


```
{ "request_data": { "filters": [ { "field": "affected_cpu_archs", "operator": "contains", "value": [ "example" ] } ], "sort": { "field": "example", "keyword": "example" }, "search_from": 0, "search_to": 0, "use_page_token": false, "next_page_token": "example" } }
```


Responses

OK

Body

application/json

▼ replyobject

▶ result_countinteger

▶ total_countinteger

▶ vulnerabilitiesarray

▶ next_page_tokenstring

RESPONSE


```
{ "reply": { "result_count": 0, "total_count": 0, "vulnerabilities": [ { "vulnerability_id": "example", "cvss_score": 0.1, "cvss_severity": "example", "package_name": "example", "distro": "example", "release": "example", "affected_cpu_archs": [ {} ], "last_modified": 0, "affected_versions": [ "example" ] } ], "next_page_token": "example" } }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body

application/json

▼ replyobject

▶ err_codestring

▶ err_msgstring

▶ err_extrastring

RESPONSE


```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```


Internal server error. A unified status for API communication type errors.

Body

application/json

▼ replyobject

▶ err_codestring

▶ err_msgstring

▶ err_extrastring

RESPONSE


```
{ "reply": { "err_code": "example", "err_msg": "example", "err_extra": "example" } }
```

2.6 | Vulnerability Management APIs

2.6.1 | Preface

You can export the OpenAPI specs in the Cortex Stoplight instance.

Managing vulnerabilities effectively is crucial to proactively maintaining the security, integrity, and availability of IT infrastructure. Cortex Cloud provides a comprehensive vulnerability management platform, helping you identify, assess, prioritize, and remediate security vulnerabilities across your entire IT infrastructure including endpoints, code, and cloud.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Version: Cortex Cloud 1.3

2.6.1.1 | Servers

https://api-yourfqdn

2.6.2 | Vulnerability Management

Public APIs for vulnerability policy management

2.6.2.1 | Get Policies List

post /public_api/uvm_public/v1/list_policies

Retrieve a paginated list of vulnerability management policies with optional filtering and sorting capabilities. This endpoint allows you to search through policies based on various criteria such as name, status, priority, and other attributes.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ x-xdr-auth-id required
▼ Authorization String required
Example: `authorization_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'x-xdr-auth-id: ' -H 'Authorization: authorization_example'
'https://api-yourfqdn/public_api/uvm_public/v1/list_policies'
-d '{ "filter_data" : { "filter" : { "AND" : [ { "SEARCH_TYPE" : "SEARCH_TYPE", "SEARCH_FIELD" : "SEARCH_FIELD",
"SEARCH_VALUE" : "Search_Value_1" }, { "SEARCH_TYPE" : "SEARCH_TYPE", "SEARCH_FIELD" : "SEARCH_FIELD", "SEARCH_VALUE"
: "Search_Value_1" } ] }, "paging" : { "from" : 0, "to" : 1 }, "sort" : [ { "ORDER" : "ASC", "FIELD" : "FIELD" }, {
"ORDER" : "ASC", "FIELD" : "FIELD" } ], "locked" : { "AND" : [ { "SEARCH_TYPE" : "SEARCH_TYPE", "SEARCH_FIELD" :
"SEARCH_FIELD", "SEARCH_VALUE" : "Search_Value_1" }, { "SEARCH_TYPE" : "SEARCH_TYPE", "SEARCH_FIELD" :
"SEARCH_FIELD", "SEARCH_VALUE" : "Search_Value_1" } ] } } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"filter_data\":{\"filter\":
{\"AND\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"sort\":
[{\"FIELD\":\"string\",\"ORDER\":\"ASC\"}],\"paging\":{\"from\":0,\"to\":1},\"locked\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]}}}" headers = { 'x-xdr-auth-id':
"SOME_INTEGER_VALUE", 'Authorization': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/uvm_public/v1/list_policies", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/uvm_public/v1/list_policies") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'SOME_INTEGER_VALUE' request["Authorization"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"filter_data\":{\"filter\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"sort\":
[{\"FIELD\":\"string\",\"ORDER\":\"ASC\"}],\"paging\":{\"from\":0,\"to\":1},\"locked\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]}}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "filter_data": { "filter": { "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE": "EQ",
"SEARCH_VALUE": "string" } ] }, "sort": [ { "FIELD": "string", "ORDER": "ASC" } ], "paging": { "from": 0, "to": 1 },
"locked": { "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "string" } ] } } }); const xhr
= new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/uvm_public/v1/list_policies"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_INTEGER_VALUE");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/uvm_public/v1/list_policies")
.header("x-xdr-auth-id", "SOME_INTEGER_VALUE") .header("Authorization", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("{\"filter_data\":{\"filter\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"sort\":
[{\"FIELD\":\"string\",\"ORDER\":\"ASC\"}],\"paging\":{\"from\":0,\"to\":1},\"locked\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]}}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "SOME_INTEGER_VALUE", "Authorization": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = ["filter_data": [ "filter": ["AND": [ [ "SEARCH_FIELD":
"string", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "string" ] ]], "sort": [ [ "FIELD": "string", "ORDER": "ASC" ] ],
"paging": [ "from": 0, "to": 1 ], "locked": ["AND": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "EQ",
"SEARCH_VALUE": "string" ] ]] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/uvm_public/v1/list_policies")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/uvm_public/v1/list_policies", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
```

```
=> "POST", CURLOPT_POSTFIELDS => "{\"filter_data\":{\"filter\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"sort\":
[{\"FIELD\":\"string\",\"ORDER\":\"ASC\"}],\"paging\":{\"from\":0,\"to\":1},\"locked\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]}}}", CURLOPT_HTTPHEADER => [
"Authorization: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_INTEGER_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/uvm_public/v1/list_policies"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "x-xdr-auth-id: SOME_INTEGER_VALUE"); headers = curl_slist_append(headers,
"Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"filter_data\":
{\"filter\":{\"AND\":[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"sort\":
[{\"FIELD\":\"string\",\"ORDER\":\"ASC\"}],\"paging\":{\"from\":0,\"to\":1},\"locked\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]}}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/uvm_public/v1/list_policies"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "SOME_INTEGER_VALUE");
request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"filter_data\":{\"filter\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"sort\":
[{\"FIELD\":\"string\",\"ORDER\":\"ASC\"}],\"paging\":{\"from\":0,\"to\":1},\"locked\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]}}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ filter_dataobject

▶ filterobject

▶ sortarray

▶ pagingobjectrequired

▶ lockedobject

Filter passed in from the application that is not intended to be modified by the user. Be aware that this field is not guaranteed to not be modified by the user.

REQUEST

```
{ "filter_data": { "filter": { "AND": [ { "SEARCH_FIELD": "example", "SEARCH_TYPE": "example", "SEARCH_VALUE":
"example" } ] }, "sort": [ { "FIELD": "example", "ORDER": "ASC" } ], "paging": { "from": 0, "to": 0 }, "locked": {
"AND": [ { "SEARCH_FIELD": "example", "SEARCH_TYPE": "example", "SEARCH_VALUE": "example" } ] } } }
```

Responses

Successful Response

Body

application/json

▼ DATAarray

[

]

▼ FILTER_COUNTinteger

▼ TOTAL_COUNTinteger

RESPONSE

```
{ "DATA": [ {} ], "FILTER_COUNT": 0, "TOTAL_COUNT": 0 }
```

Validation Error

Body

application/json

▼ detailarray

[

▶ locarray

▶ msgstring

▶ typestring

]

RESPONSE

```
{ "detail": [ { "loc": [ "example" ], "msg": "example", "type": "example" } ] }
```

2.6.2.2 | Create Policy Public

post /public_api/uvm_public/v1/create_policy

Create a new vulnerability management policy for issue creation, kubernetes runtime protection, or prevention actions.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ x-xdr-auth-id required
▼ Authorization String required
Example: `authorization_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'x-xdr-auth-id: ' -H 'Authorization: authorization_example'
'https://api-yourfqdn/public_api/uvm_public/v1/create_policy'
-d '{ "match_criteria" : { "AND" : [ { "SEARCH_TYPE" : "SEARCH_TYPE", "SEARCH_FIELD" : "SEARCH_FIELD", "SEARCH_VALUE"
: "Search_Value_1" }, { "SEARCH_TYPE" : "SEARCH_TYPE", "SEARCH_FIELD" : "SEARCH_FIELD", "SEARCH_VALUE" :
"Search_Value_1" } ] }, "asset_group_scope" : [ 6, 6 ], "exclusion_criteria" : { }, "name" : "name", "issue_type" :
"issue_type", "description" : "description", "action" : [ { "action_type" : "BLOCK_BUILD", "take_action" : true,
"category" : "BLOCK" }, { "action_type" : "BLOCK_BUILD", "take_action" : true, "category" : "BLOCK" } ], "priority" :
0 }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "
{\"name\":\"string\",\"description\":\"string\",\"priority\":0,\"status\":\"ENABLED\",\"match_criteria\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"exclusion_criteria\":
{},\"action\":
[{\"action_type\":\"BLOCK_BUILD\",\"take_action\":true,\"category\":\"BLOCK\"}],\"action_category\":\"BLOCK\",\"issue_type\":\"
[0],\"policy_type\":\"GLOBAL_EXCLUSION\"}" headers = { 'x-xdr-auth-id': "SOME_INTEGER_VALUE", 'Authorization':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/uvm_public/v1/create_policy", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/uvm_public/v1/create_policy") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'SOME_INTEGER_VALUE' request["Authorization"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "
{\"name\":\"string\",\"description\":\"string\",\"priority\":0,\"status\":\"ENABLED\",\"match_criteria\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"exclusion_criteria\":
{},\"action\":
[{\"action_type\":\"BLOCK_BUILD\",\"take_action\":true,\"category\":\"BLOCK\"}],\"action_category\":\"BLOCK\",\"issue_type\":\"
[0],\"policy_type\":\"GLOBAL_EXCLUSION\"}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "name": "string", "description": "string", "priority": 0, "status": "ENABLED",
"match_criteria": { "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "string" } ] },
"exclusion_criteria": {}, "action": [ { "action_type": "BLOCK_BUILD", "take_action": true, "category": "BLOCK" } ],
"action_category": "BLOCK", "issue_type": "string", "severity": "CRITICAL", "asset_group_scope": [ 0 ],
"policy_type": "GLOBAL_EXCLUSION" }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/uvm_public/v1/create_policy"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_INTEGER_VALUE");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/uvm_public/v1/create_policy")
.header("x-xdr-auth-id", "SOME_INTEGER_VALUE") .header("Authorization", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("
{\"name\":\"string\",\"description\":\"string\",\"priority\":0,\"status\":\"ENABLED\",\"match_criteria\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"exclusion_criteria\":
{},\"action\":
[{\"action_type\":\"BLOCK_BUILD\",\"take_action\":true,\"category\":\"BLOCK\"}],\"action_category\":\"BLOCK\",\"issue_type\":\"
[0],\"policy_type\":\"GLOBAL_EXCLUSION\"}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "SOME_INTEGER_VALUE", "Authorization": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = [ "name": "string", "description": "string", "priority": 0,
"status": "ENABLED", "match_criteria": ["AND": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "EQ", "SEARCH_VALUE":
"string" ] ]], "exclusion_criteria": [], "action": [ [ "action_type": "BLOCK_BUILD", "take_action": true, "category":
"BLOCK" ] ], "action_category": "BLOCK", "issue_type": "string", "severity": "CRITICAL", "asset_group_scope": [0],
"policy_type": "GLOBAL_EXCLUSION" ] as [String : Any] let postData = JSONSerialization.data(withJSONObject:
parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/uvm_public/v1/create_policy")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
```

```
10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/uvm_public/v1/create_policy", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "
{\"name\":\"string\",\"description\":\"string\",\"priority\":0,\"status\":\"ENABLED\",\"match_criteria\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"exclusion_criteria\":
{},\"action\":
[{\"action_type\":\"BLOCK_BUILD\",\"take_action\":true,\"category\":\"BLOCK\"}],\"action_category\":\"BLOCK\",\"issue_type\":\"
[0],\"policy_type\":\"GLOBAL_EXCLUSION\"}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-
type: application/json", "x-xdr-auth-id: SOME_INTEGER_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/uvm_public/v1/create_policy"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "x-xdr-auth-id: SOME_INTEGER_VALUE"); headers = curl_slist_append(headers,
"Authorization: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json");
curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"name\":\"string\",\"description\":\"string\",\"priority\":0,\"status\":\"ENABLED\",\"match_criteria\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"exclusion_criteria\":
{},\"action\":
[{\"action_type\":\"BLOCK_BUILD\",\"take_action\":true,\"category\":\"BLOCK\"}],\"action_category\":\"BLOCK\",\"issue_type\":\"
[0],\"policy_type\":\"GLOBAL_EXCLUSION\"}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/uvm_public/v1/create_policy"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "SOME_INTEGER_VALUE");
request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "
{\"name\":\"string\",\"description\":\"string\",\"priority\":0,\"status\":\"ENABLED\",\"match_criteria\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"exclusion_criteria\":
{},\"action\":
[{\"action_type\":\"BLOCK_BUILD\",\"take_action\":true,\"category\":\"BLOCK\"}],\"action_category\":\"BLOCK\",\"issue_type\":\"
[0],\"policy_type\":\"GLOBAL_EXCLUSION\"}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

required

application/json

▼ namestring

▼ descriptionstring

▼ priorityinteger

▼ statusstring (Enum)

Allowed values:"ENABLED""DISABLED"

▼ match_criteriaobject

Any of AND-Input

▶ ANDarray

▶ ORarray

▼ exclusion_criteriaobject

Any of EMPTY

▶ ANDarray

▶ ORarray

▼ actionarray

[

Any of BlockBuild

▶ action_typestring

▶ take_actionboolean

▶ categorystring

▶ action_typestring

▶ take_actionboolean

▶ categorystring

▶ action_typestring

▶ namestring

▶ take_actionboolean

▶ categorystring

]

▼ action_categorystring (Enum)

Allowed values:"BLOCK""CREATE_ISSUE"

▼ issue_typestring

▼ severitystring (Enum)
Allowed values:"CRITICAL""HIGH""MEDIUM""LOW""INFO""UNKNOWN""USE_CVE_SEVERITY"
▼ asset_group_scopearray[integer]
▼ policy_typestring (Enum)
Allowed values:"GLOBAL_EXCLUSION""STANDARD_POLICY"
REQUEST
```
{ "name": "example", "description": "example", "priority": 0, "status": "ENABLED", "match_criteria": { "AND": [ {
"SEARCH_FIELD": "example", "SEARCH_TYPE": "example", "SEARCH_VALUE": "example" } ] }, "exclusion_criteria": {},
"action": [ { "action_type": "example", "take_action": false, "category": "example" } ], "action_category": "BLOCK",
"issue_type": "example", "severity": "CRITICAL", "asset_group_scope": [ 0 ], "policy_type": "GLOBAL_EXCLUSION" }
```
Responses

Successful Response

Body
application/json
▼ idstringuuid
RESPONSE
```
{ "id": "uuid string" }
```

Validation Error

Body
application/json
▼ detailarray
[
▶ locarray
▶ msgstring
▶ typestring
]
RESPONSE
```
{ "detail": [ { "loc": [ "example" ], "msg": "example", "type": "example" } ] }
```

2.6.2.3 | Update Policy

put /public_api/uvm_public/v1/update_policy/{id}

Update an existing vulnerability management policy by providing its unique identifier.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ id UUID required
Example: `38400000-8cf0-11bd-b23e-10b96e4ef00d`
Request headers
▼ x-xdr-auth-id required
▼ Authorization String required
Example: `authorization_example`
CLIENT REQUEST Bash+curl
```
curl -X 'PUT'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'x-xdr-auth-id: ' -H 'Authorization: authorization_example'
'https://api-yourfqdn/public_api/uvm_public/v1/update_policy/{id}'
-d '{ "match_criteria" : { "AND" : [ { "SEARCH_TYPE" : "SEARCH_TYPE", "SEARCH_FIELD" : "SEARCH_FIELD", "SEARCH_VALUE"
: "Search_Value_1" }, { "SEARCH_TYPE" : "SEARCH_TYPE", "SEARCH_FIELD" : "SEARCH_FIELD", "SEARCH_VALUE" :
"Search_Value_1" } ] }, "asset_group_scope" : [ 6, 6 ], "exclusion_criteria" : { }, "name" : "name", "issue_type" :
"issue_type", "description" : "description", "action" : [ { "action_type" : "BLOCK_BUILD", "take_action" : true,
"category" : "BLOCK" }, { "action_type" : "BLOCK_BUILD", "take_action" : true, "category" : "BLOCK" } ], "priority" :
0 }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "
{\"name\":\"string\",\"description\":\"string\",\"priority\":0,\"status\":\"ENABLED\",\"match_criteria\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"exclusion_criteria\":
{},\"action\":
[{\"action_type\":\"BLOCK_BUILD\",\"take_action\":true,\"category\":\"BLOCK\"}],\"action_category\":\"BLOCK\",\"issue_type\":\"
[0],\"policy_type\":\"GLOBAL_EXCLUSION\"}" headers = { 'x-xdr-auth-id': "SOME_INTEGER_VALUE", 'Authorization':
"SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("PUT",
"/public_api/uvm_public/v1/update_policy/%7Bid%7D", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
```

```
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/uvm_public/v1/update_policy/%7Bid%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl =
true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Put.new(url) request["x-xdr-auth-id"] =
'SOME_INTEGER_VALUE' request["Authorization"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "
{\"name\":\"string\",\"description\":\"string\",\"priority\":0,\"status\":\"ENABLED\",\"match_criteria\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"exclusion_criteria\":
{},\"action\":
[{\"action_type\":\"BLOCK_BUILD\",\"take_action\":true,\"category\":\"BLOCK\"}],\"action_category\":\"BLOCK\",\"issue_type\":\"
[0],\"policy_type\":\"GLOBAL_EXCLUSION\"}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "name": "string", "description": "string", "priority": 0, "status": "ENABLED",
"match_criteria": { "AND": [ { "SEARCH_FIELD": "string", "SEARCH_TYPE": "EQ", "SEARCH_VALUE": "string" } ] },
"exclusion_criteria": {}, "action": [ { "action_type": "BLOCK_BUILD", "take_action": true, "category": "BLOCK" } ],
"action_category": "BLOCK", "issue_type": "string", "severity": "CRITICAL", "asset_group_scope": [ 0 ],
"policy_type": "GLOBAL_EXCLUSION" }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("PUT", "https://api-
yourfqdn/public_api/uvm_public/v1/update_policy/%7Bid%7D"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_INTEGER_VALUE"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-
type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.put("https://api-yourfqdn/public_api/uvm_public/v1/update_policy/%7Bid%7D")
.header("x-xdr-auth-id", "SOME_INTEGER_VALUE") .header("Authorization", "SOME_STRING_VALUE") .header("content-type",
"application/json") .body("
{\"name\":\"string\",\"description\":\"string\",\"priority\":0,\"status\":\"ENABLED\",\"match_criteria\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"exclusion_criteria\":
{},\"action\":
[{\"action_type\":\"BLOCK_BUILD\",\"take_action\":true,\"category\":\"BLOCK\"}],\"action_category\":\"BLOCK\",\"issue_type\":\"
[0],\"policy_type\":\"GLOBAL_EXCLUSION\"}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "SOME_INTEGER_VALUE", "Authorization": "SOME_STRING_VALUE",
"content-type": "application/json" ] let parameters = [ "name": "string", "description": "string", "priority": 0,
"status": "ENABLED", "match_criteria": ["AND": [ [ "SEARCH_FIELD": "string", "SEARCH_TYPE": "EQ", "SEARCH_VALUE":
"string" ] ]], "exclusion_criteria": [], "action": [ [ "action_type": "BLOCK_BUILD", "take_action": true, "category":
"BLOCK" ] ], "action_category": "BLOCK", "issue_type": "string", "severity": "CRITICAL", "asset_group_scope": [0],
"policy_type": "GLOBAL_EXCLUSION" ] as [String : Any] let postData = JSONSerialization.data(withJSONObject:
parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/uvm_public/v1/update_policy/%7Bid%7D")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "PUT" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/uvm_public/v1/update_policy/%7Bid%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "PUT", CURLOPT_POSTFIELDS => "
{\"name\":\"string\",\"description\":\"string\",\"priority\":0,\"status\":\"ENABLED\",\"match_criteria\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"exclusion_criteria\":
{},\"action\":
[{\"action_type\":\"BLOCK_BUILD\",\"take_action\":true,\"category\":\"BLOCK\"}],\"action_category\":\"BLOCK\",\"issue_type\":\"
[0],\"policy_type\":\"GLOBAL_EXCLUSION\"}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "content-
type: application/json", "x-xdr-auth-id: SOME_INTEGER_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "PUT"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/uvm_public/v1/update_policy/%7Bid%7D"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "x-xdr-auth-id: SOME_INTEGER_VALUE"); headers = curl_slist_append(headers, "Authorization:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd,
CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"name\":\"string\",\"description\":\"string\",\"priority\":0,\"status\":\"ENABLED\",\"match_criteria\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"exclusion_criteria\":
{},\"action\":
[{\"action_type\":\"BLOCK_BUILD\",\"take_action\":true,\"category\":\"BLOCK\"}],\"action_category\":\"BLOCK\",\"issue_type\":\"
[0],\"policy_type\":\"GLOBAL_EXCLUSION\"}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/uvm_public/v1/update_policy/%7Bid%7D"); var request =
new RestRequest(Method.PUT); request.AddHeader("x-xdr-auth-id", "SOME_INTEGER_VALUE");
request.AddHeader("Authorization", "SOME_STRING_VALUE"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "
{\"name\":\"string\",\"description\":\"string\",\"priority\":0,\"status\":\"ENABLED\",\"match_criteria\":{\"AND\":
[{\"SEARCH_FIELD\":\"string\",\"SEARCH_TYPE\":\"EQ\",\"SEARCH_VALUE\":\"string\"}]},\"exclusion_criteria\":
{},\"action\":
```

```
[{\"action_type\":\"BLOCK_BUILD\",\"take_action\":true,\"category\":\"BLOCK\"}],\"action_category\":\"BLOCK\",\"issue_type\":\"
[0],\"policy_type\":\"GLOBAL_EXCLUSION\"}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

required

application/json

▼ namestring

▼ descriptionstring

▼ priorityinteger

▼ statusstring (Enum)

Allowed values:"ENABLED""DISABLED"

▼ match_criteriaobject

Any of AND-Input

▶ ANDarray

▶ ORarray

▼ exclusion_criteriaobject

Any of EMPTY

▶ ANDarray

▶ ORarray

▼ actionarray

[

Any of BlockBuild

▶ action_typestring

▶ take_actionboolean

▶ categorystring

▶ action_typestring

▶ take_actionboolean

▶ categorystring

▶ action_typestring

▶ namestring

▶ take_actionboolean

▶ categorystring

]

▼ action_categorystring (Enum)

Allowed values:"BLOCK""CREATE_ISSUE"

▼ issue_typestring

▼ severitystring (Enum)

Allowed values:"CRITICAL""HIGH""MEDIUM""LOW""INFO""UNKNOWN""USE_CVE_SEVERITY"

▼ asset_group_scopearray[integer]

▼ policy_typestring (Enum)

Allowed values:"GLOBAL_EXCLUSION""STANDARD_POLICY"

REQUEST

```
{ "name": "example", "description": "example", "priority": 0, "status": "ENABLED", "match_criteria": { "AND": [ {
"SEARCH_FIELD": "example", "SEARCH_TYPE": "example", "SEARCH_VALUE": "example" } ] }, "exclusion_criteria": {},
"action": [ { "action_type": "example", "take_action": false, "category": "example" } ], "action_category": "BLOCK",
"issue_type": "example", "severity": "CRITICAL", "asset_group_scope": [ 0 ], "policy_type": "GLOBAL_EXCLUSION" }
```

Responses

Successful Response

Body

application/json

▼ idstringuuid

RESPONSE

```
{ "id": "uuid string" }
```

Validation Error

Body

application/json

▼ detailarray

[

▶ locarray

▶ msgstring

▶ typestring

]

RESPONSE

```
{ "detail": [ { "loc": [ "example" ], "msg": "example", "type": "example" } ] }
```

2.6.2.4 | Get Policy By ID

get /public_api/uvm_public/v1/get_policy/{id}

Retrieve a specific vulnerability management policy by its unique identifier.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ id UUID required
Example: `38400000-8cf0-11bd-b23e-10b96e4ef00d`
Request headers
▼ x-xdr-auth-id required
▼ Authorization String required
Example: `authorization_example`
CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json'
-H 'x-xdr-auth-id: ' -H 'Authorization: authorization_example'
'https://api-yourfqdn/public_api/uvm_public/v1/get_policy/{id}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'x-xdr-auth-id':
"SOME_INTEGER_VALUE", 'Authorization': "SOME_STRING_VALUE" } conn.request("GET",
"/public_api/uvm_public/v1/get_policy/%7Bid%7D", headers=headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/uvm_public/v1/get_policy/%7Bid%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["x-xdr-auth-id"] =
'SOME_INTEGER_VALUE' request["Authorization"] = 'SOME_STRING_VALUE' response = http.request(request) puts
response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-
yourfqdn/public_api/uvm_public/v1/get_policy/%7Bid%7D"); xhr.setRequestHeader("x-xdr-auth-id", "SOME_INTEGER_VALUE");
xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/uvm_public/v1/get_policy/%7Bid%7D")
.header("x-xdr-auth-id", "SOME_INTEGER_VALUE") .header("Authorization", "SOME_STRING_VALUE") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "SOME_INTEGER_VALUE", "Authorization": "SOME_STRING_VALUE" ] let
request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/uvm_public/v1/get_policy/%7Bid%7D")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/uvm_public/v1/get_policy/%7Bid%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "x-xdr-auth-id: SOME_INTEGER_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/uvm_public/v1/get_policy/%7Bid%7D"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "x-xdr-auth-id: SOME_INTEGER_VALUE"); headers = curl_slist_append(headers, "Authorization:
SOME_STRING_VALUE"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/uvm_public/v1/get_policy/%7Bid%7D"); var request = new
RestRequest(Method.GET); request.AddHeader("x-xdr-auth-id", "SOME_INTEGER_VALUE"); request.AddHeader("Authorization",
"SOME_STRING_VALUE"); IRestResponse response = client.Execute(request);
```

Responses

Successful Response

Body
application/json
▼ PRIORITYinteger
▼ IDstringuuid
▼ NAMEstring
▼ STATUSstring
▼ MATCH_CRITERIAobject
Any of EMPTY
▶ ANDarray
▶ ORarray

▼ EXCLUSIONSobject
Any of EMPTY
▶ ANDarray
▶ ORarray
▼ SEVERITYstring (Enum)
Allowed values:"CRITICAL""HIGH""MEDIUM""LOW""INFO""UNKNOWN""USE_CVE_SEVERITY"
▼ OPEN_ISSUESinteger
▼ ESTIMATED_MATCH_COUNTinteger
▼ DESCRIPTIONstring
▼ MODIFIED_BYstring
▼ MODIFIED_TIMESTAMPstringdate-time
▼ ACTIONarray
[
Any of BlockBuild
▶ action_typestring
▶ take_actionboolean
▶ categorystring
▶ action_typestring
▶ take_actionboolean
▶ categorystring
▶ action_typestring
▶ namestring
▶ take_actionboolean
▶ categorystring
]
▼ ACTION_CATEGORYstring (Enum)
Allowed values:"BLOCK""CREATE_ISSUE"
▼ ASSET_GROUP_SCOPEarray[integer]
▼ POLICY_TYPEstring (Enum)
Allowed values:"GLOBAL_EXCLUSION""STANDARD_POLICY"
RESPONSE


```
{ "PRIORITY": 0, "ID": "uuid string", "NAME": "example", "STATUS": "example", "MATCH_CRITERIA": {}, "EXCLUSIONS": {},
"SEVERITY": "CRITICAL", "OPEN_ISSUES": 0, "ESTIMATED_MATCH_COUNT": 0, "DESCRIPTION": "example", "MODIFIED_BY":
"example", "MODIFIED_TIMESTAMP": "2020-01-01T12:00:00Z", "ACTION": [ { "action_type": "example", "take_action":
false, "category": "example" } ], "ACTION_CATEGORY": "BLOCK", "ASSET_GROUP_SCOPE": [ 0 ], "POLICY_TYPE":
"GLOBAL_EXCLUSION" }
```


Validation Error

Body
application/json
▼ detailarray
[
▶ locarray
▶ msgstring
▶ typestring
]
RESPONSE


```
{ "detail": [ { "loc": [ "example" ], "msg": "example", "type": "example" } ] }
```

2.6.2.5 | Delete Policy

delete /public_api/uvm_public/v1/delete_policy/{id}

Delete an existing vulnerability management policy using a policy ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ id UUID required
Example: `38400000-8cf0-11bd-b23e-10b96e4ef00d`
Request headers
▼ x-xdr-auth-id required
▼ Authorization String required
Example: `authorization_example`
CLIENT REQUEST Bash+curl


```
curl -X 'DELETE'
-H 'Accept: application/json'
```

```
-H 'x-xdr-auth-id: ' -H 'Authorization: authorization_example'
'https://api-yourfqdn/public_api/uvm_public/v1/delete_policy/{id}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'x-xdr-auth-id':
"SOME_INTEGER_VALUE", 'Authorization': "SOME_STRING_VALUE" } conn.request("DELETE",
"/public_api/uvm_public/v1/delete_policy/%7Bid%7D", headers=headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/uvm_public/v1/delete_policy/%7Bid%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl =
true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Delete.new(url) request["x-xdr-auth-id"] =
'SOME_INTEGER_VALUE' request["Authorization"] = 'SOME_STRING_VALUE' response = http.request(request) puts
response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("DELETE", "https://api-
yourfqdn/public_api/uvm_public/v1/delete_policy/%7Bid%7D"); xhr.setRequestHeader("x-xdr-auth-id",
"SOME_INTEGER_VALUE"); xhr.setRequestHeader("Authorization", "SOME_STRING_VALUE"); xhr.send(data);
HttpResponse<String> response = Unirest.delete("https://api-
yourfqdn/public_api/uvm_public/v1/delete_policy/%7Bid%7D") .header("x-xdr-auth-id", "SOME_INTEGER_VALUE")
.header("Authorization", "SOME_STRING_VALUE") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "SOME_INTEGER_VALUE", "Authorization": "SOME_STRING_VALUE" ] let
request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/uvm_public/v1/delete_policy/%7Bid%7D")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "DELETE" request.allHTTPHeaderFields = headers let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/uvm_public/v1/delete_policy/%7Bid%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "DELETE", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "x-xdr-auth-id: SOME_INTEGER_VALUE" ], ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "DELETE"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/uvm_public/v1/delete_policy/%7Bid%7D"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "x-xdr-auth-id: SOME_INTEGER_VALUE"); headers = curl_slist_append(headers,
"Authorization: SOME_STRING_VALUE"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/uvm_public/v1/delete_policy/%7Bid%7D"); var request =
new RestRequest(Method.DELETE); request.AddHeader("x-xdr-auth-id", "SOME_INTEGER_VALUE");
request.AddHeader("Authorization", "SOME_STRING_VALUE"); IRestResponse response = client.Execute(request);
```

Responses

Successful Response

Body
application/json
▼ boolean
RESPONSE

```
false
```

Validation Error

Body
application/json
▼ detailarray
[
▶ locarray
▶ msgstring
▶ typestring
]
RESPONSE

```
{ "detail": [ { "loc": [ "example" ], "msg": "example", "type": "example" } ] }
```

2.7 | Cloud Infrastructure Entitlement Management APIs

2.7.1 | Preface

You can export the OpenAPI specs in the Cortex Stoplight instancehttps://cortex-panw.stoplight.io/docs/cortex-cloud/iul4tcqnj800b-cloud-infrastructure-entitlement-management-ap-is).

API for searching and retrieving access information within the Cloud Infrastructure Entitlement Management(CIEM) module.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Version: Cortex Cloud 1.3

2.7.1.1 | Servers

https://api-yourfqdn

2.7.2 | CIEM access

Search CIEM access

2.7.2.1 | Search CIEM Access

post /public_api/v1/ciem/access/search

Retrieves a list of access entries based on specified filters, sorting, and pagination.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ x-xdr-auth-id String required

{api_key_id}

Example: `xXdrAuthId_example`
▼ Authorization String required

{api_key}

Example: `authorization_example`
▼ Content-Type String required

Specifies the request body format.

Example: `contentType_example`
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json'
-H 'Content-Type: application/json'
-H 'x-xdr-auth-id: xXdrAuthId_example' -H 'Authorization: authorization_example' -H 'Content-Type:
contentType_example'
'https://api-yourfqdn/public_api/v1/ciem/access/search'
-d '{ "filter" : { "AND" : [ { "search_field" : "dest_cloud_resource_id", "search_type" : "EQ", "search_value" :
"abc" } ] }, "sort" : [ { "field" : "dest_cloud_resource_id", "order" : "DESC" } ], "metadata" : { "next_page_token"
: "some_base64_string" } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"filter\":{\"AND\":
[{\"search_field\":\"dest_cloud_resource_id\",\"search_type\":\"EQ\",\"search_value\":\"abc\"}]},\"sort\":
[{\"field\":\"dest_cloud_resource_id\",\"order\":\"DESC\"}],\"metadata\":
{\"next_page_token\":\"some_base64_string\"}}" headers = { 'x-xdr-auth-id': "SOME_STRING_VALUE", 'Authorization':
"SOME_STRING_VALUE", 'Content-Type': "SOME_STRING_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/ciem/access/search", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/ciem/access/search")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["x-xdr-auth-id"] = 'SOME_STRING_VALUE' request["Authorization"] =
'SOME_STRING_VALUE' request["Content-Type"] = 'SOME_STRING_VALUE' request["content-type"] = 'application/json'
request.body = "{\"filter\":{\"AND\":
[{\"search_field\":\"dest_cloud_resource_id\",\"search_type\":\"EQ\",\"search_value\":\"abc\"}]},\"sort\":
[{\"field\":\"dest_cloud_resource_id\",\"order\":\"DESC\"}],\"metadata\":
{\"next_page_token\":\"some_base64_string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "filter": { "AND": [ { "search_field": "dest_cloud_resource_id", "search_type": "EQ",
"search_value": "abc" } ] }, "sort": [ { "field": "dest_cloud_resource_id", "order": "DESC" } ], "metadata": {
"next_page_token": "some_base64_string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
```

```
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/ciem/access/search");
xhr.setRequestHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); xhr.setRequestHeader("Authorization",
"SOME_STRING_VALUE"); xhr.setRequestHeader("Content-Type", "SOME_STRING_VALUE"); xhr.setRequestHeader("content-type",
"application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/ciem/access/search") .header("x-xdr-
auth-id", "SOME_STRING_VALUE") .header("Authorization", "SOME_STRING_VALUE") .header("Content-Type",
"SOME_STRING_VALUE") .header("content-type", "application/json") .body("{\"filter\":{\"AND\":
[{\"search_field\":\"dest_cloud_resource_id\",\"search_type\":\"EQ\",\"search_value\":\"abc\"}]},\"sort\":
[{\"field\":\"dest_cloud_resource_id\",\"order\":\"DESC\"}],\"metadata\":
{\"next_page_token\":\"some_base64_string\"}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "SOME_STRING_VALUE", "Authorization": "SOME_STRING_VALUE",
"Content-Type": "SOME_STRING_VALUE", "content-type": "application/json" ] let parameters = [ "filter": ["AND": [ [
"search_field": "dest_cloud_resource_id", "search_type": "EQ", "search_value": "abc" ] ]], "sort": [ [ "field":
"dest_cloud_resource_id", "order": "DESC" ] ], "metadata": ["next_page_token": "some_base64_string"] ] as [String :
Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/ciem/access/search")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/ciem/access/search", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS
=> 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"filter\":{\"AND\":
[{\"search_field\":\"dest_cloud_resource_id\",\"search_type\":\"EQ\",\"search_value\":\"abc\"}]},\"sort\":
[{\"field\":\"dest_cloud_resource_id\",\"order\":\"DESC\"}],\"metadata\":
{\"next_page_token\":\"some_base64_string\"}}", CURLOPT_HTTPHEADER => [ "Authorization: SOME_STRING_VALUE", "Content-
Type: SOME_STRING_VALUE", "content-type: application/json", "x-xdr-auth-id: SOME_STRING_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/ciem/access/search"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "x-xdr-auth-id: SOME_STRING_VALUE"); headers = curl_slist_append(headers, "Authorization:
SOME_STRING_VALUE"); headers = curl_slist_append(headers, "Content-Type: SOME_STRING_VALUE"); headers =
curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"filter\":{\"AND\":
[{\"search_field\":\"dest_cloud_resource_id\",\"search_type\":\"EQ\",\"search_value\":\"abc\"}]},\"sort\":
[{\"field\":\"dest_cloud_resource_id\",\"order\":\"DESC\"}],\"metadata\":
{\"next_page_token\":\"some_base64_string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/ciem/access/search"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "SOME_STRING_VALUE"); request.AddHeader("Authorization",
"SOME_STRING_VALUE"); request.AddHeader("Content-Type", "SOME_STRING_VALUE"); request.AddHeader("content-type",
"application/json"); request.AddParameter("SOME_STRING_VALUE", "{\"filter\":{\"AND\":
[{\"search_field\":\"dest_cloud_resource_id\",\"search_type\":\"EQ\",\"search_value\":\"abc\"}]},\"sort\":
[{\"field\":\"dest_cloud_resource_id\",\"order\":\"DESC\"}],\"metadata\":
{\"next_page_token\":\"some_base64_string\"}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

required

application/json

▼ filterobject

Optional filters to apply to the search results.

▶ ANDarray

A list of filter conditions. Currently only supports AND logic.

▼ sortarray

Optional sorting criteria for the search results.

[

▶ fieldstring (Enum)

The field to sort by.

▶ orderstring (Enum)

The sort order.

]

▼ metadataobject

Optional metadata for pagination.

▶ next_page_tokenstring

A token provided by a previous response to fetch the next page of results.

REQUEST default

```
{ "filter": { "AND": [ { "search_field": "dest_cloud_resource_id", "search_type": "EQ", "search_value": "abc" } ] },
"sort": [ { "field": "dest_cloud_resource_id", "order": "DESC" } ], "metadata": { "next_page_token":
"some_base64_string" } }
```

Responses

Successful response with CIEM access data.

Body
application/json
▼ dataarray

An array of access entries.

[
▶ access_levelsarray[string]

Access levels granted

▶ account_access_transformedstring

Transformed account access details

▶ dest_cloud_account_idstring

Destination account ID

▶ dest_cloud_account_namestring

Name of the destination cloud account

▶ dest_cloud_regionstring

Destination region

▶ dest_cloud_resource_idstring

Destination resource ID

▶ dest_cloud_resource_namestring

Name of the destination resource

▶ dest_cloud_resource_typestring

Type of the destination resource

▶ dest_cloud_resource_uaistring

Destination resource UAI

▶ dest_cloud_service_namestring

Name of the destination cloud service

▶ dest_categorystring (Enum)

Category of the destination entity. Valid values include:

- `Human`: All cloud, identity provider (IdP), and platform users.
- `Non-human`: Machine identities that can assume permissions and perform cloud Identity and Access Management (IAM) actions such as VMs and functions.
- `Cloud Service Account`: A category unifying AWS roles, Microsoft Azure service accounts and managed identities, and GCP service accounts.
- `IAM Group`: IAM group, which is a collection of IAM users.
- `IAM Policy`: Permission documents, such as AWS policies, Azure roles, and GCP roles.

▶ destination_access_labelsarray[string]

Labels describing access to data

▶ excessive_policies_countinteger

Number of excessive policies

▶ grantedby_cloud_entity_idstring

Granted by cloud entity ID

▶ grantedby_cloud_entity_namestring

Name of the cloud entity that granted access

▶ grantedby_cloud_entity_typestring

Type of the cloud entity that granted access

▶ grantedby_cloud_entity_uaistring

Granted by cloud entity UAI

▶ granter_categorystring

Category of the granter entity

▶ is_last_access_supportedboolean

Indicates if last access is supported

▶ last_usedstringdate-time

Timestamp of when access was last used

▶ permission_scopestring

Scope of the permission

▶ source_cloud_account_idstring

Source account ID

▶ source_cloud_account_is_vendorboolean

Indicates if the source account is a known vendor

▶ source_cloud_account_namestring

Name of the source cloud account

▶ source_cloud_regionstring

Source region

▶ source_cloud_resource_idstring

Source resource ID

▶ source_cloud_resource_namestring

Name of the source resource

▶ source_cloud_resource_typestring

Type of the source resource

▶ source_cloud_resource_uaistring

Source resource UAI

▶ source_cloud_service_namestring

Name of the source cloud service

▶ source_categorystring (Enum)

Category of the source entity. Valid values include:

- `Human`: All cloud, identity provider (IdP), and platform users.
- `Non-human`: Machine identities that can assume permissions and perform cloud Identity and Access Management (IAM) actions such as VMs and functions.
- `Cloud Service Account`: A category unifying AWS roles, Microsoft Azure service accounts and managed identities, and GCP service accounts.
- `IAM Group`: IAM group, which is a collection of IAM users.
- `IAM Policy`: Permission documents, such as AWS policies, Azure roles, and GCP roles.

▶ source_vendor_namestring

Name of the source vendor

▶ unused_actions_countinteger

Number of unused actions

]

▼ metadataobject

Metadata for pagination.

▶ next_page_tokenstring

A token to retrieve the next page of results, if available.

▼ filter_countintegerint32

The number of results after applying filters.

▼ total_countintegerint32

The total number of available results, ignoring pagination.

RESPONSE


```
{ "data": [ { "access_levels": [ "Read", "Write" ], "account_access_transformed": "admin_access",
"dest_cloud_account_name": "prod-account", "dest_cloud_resource_name": "S3 Bucket", "dest_cloud_resource_type":
"AWS::S3::Bucket", "dest_cloud_service_name": "Amazon S3", "dest_category": "Cloud Service Account",
"destination_access_labels": [], "excessive_policies_count": 5, "grantedby_cloud_entity_name": "AdminRole",
"grantedby_cloud_entity_type": "IAM Role", "granter_category": "IAM Group", "last_used": "2023-01-15T10:30:00Z",
"permission_scope": "Global", "source_cloud_account_name": "dev-account", "source_cloud_account_is_vendor": false,
"source_cloud_resource_name": "EC2 Instance", "source_cloud_resource_type": "AWS::EC2::Instance",
"source_cloud_service_name": "Amazon EC2", "source_category": "Human", "source_vendor_name": "AWS",
"unused_actions_count": 10 } ], "metadata": { "next_page_token": "another_base64_string" }, "filter_count": 100,
"total_count": 500 }
```


Bad Request. Invalid JSON.

Body

application/json

▼ replyobject

▶ err_codeinteger

▶ err_msgstring

▶ err_extraobject

RESPONSE Example 1


```
{ "reply": { "err_code": 400, "err_msg": "Invalid Request Body", "err_extra": { "err_msg": "The request body contains
invalid JSON or malformed parameters for CIEM Access Search." } } }
```


Unauthorized access. An issue occurred during authentication. This can indicate an incorrect key, id, or other invalid authentication parameters.

Body

application/json

▼ replyobject

▶ err_codeinteger

▶ err_msgstring

▶ err_extraobject

RESPONSE Example 1


```
{ "reply": { "err_code": 401, "err_msg": "Authentication Failed", "err_extra": { "err_msg": "Invalid API key or key
ID provided. Please check your authentication credentials." } } }
```


Unauthorized access. User does not have the required license type to run this API.

Body

application/json

▼ replyobject

▶ err_codeinteger

▶ err_msgstring

▶ err_extraobject

RESPONSE Example 1


```
{ "reply": { "err_code": 402, "err_msg": "License Required", "err_extra": { "err_msg": "Your account does not have
the necessary license to access the CIEM Access Search API." } } }
```


Forbidden access. The provided API Key does not have the required RBAC permissions to run this API.

Body

application/json

▼ replyobject

▶ err_codeinteger
▶ err_msgstring
▶ err_extraobject
RESPONSE Example 1
{ "reply": { "err_code": 403, "err_msg": "Forbidden Access", "err_extra": { "err_msg": "The provided API key lacks the required RBAC permissions for the CIEM Access Search operation." } } }

Internal server error. A unified status for API communication type errors.

Body
application/json
▼ replyobject
▶ err_codeinteger
▶ err_msgstring
▶ err_extraobject
RESPONSE Example 1
{ "reply": { "err_code": 500, "err_msg": "An error occurred while processing CIEM Access Search query", "err_extra": { "err_msg": "Internal server issue encountered during CIEM Access Search." } } }

2.8 | Compliance Controls API

2.8.1 | Preface

You can export the OpenAPI specs in the Cortex Stoplight instance.

API for managing compliance controls, rules and retrieving compliance results.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Version: Cortex Cloud 1.3

2.8.1.1 | Servers

https://api-yourfqdn

2.8.1.2 | Authentication methods

▼ XDRAuth
Api Key "x-xdr-auth-id"

API Key ID for authentication

Authorized requests to the API should use: **x-xdr-auth-id** inside **a header** with the access token.

▼ XDRAuthToken
Api Key "Authorization"

API Key for authentication

Authorized requests to the API should use: **Authorization** inside **a header** with the access token.

2.8.1.2.1 | Default authentication

Please note that this documentation assumes a minimum level of authentication is required, unless explicitly stated otherwise.
It is essential to ensure that any of the following authentication mechanisms are in place to access the API endpoints:

- XDRAuth=[], XDRAuthToken

2.8.2 | Assessment profiles

Operations for managing assessment profiles

2.8.2.1 | Get assessment profiles

post /public_api/v1/compliance/get_assessment_profiles

An assessment profile uses a standard to run scans on an asset group to check whether the assets adhere to the standard.

Retrieve compliance assessment profiles with optional filtering, sorting and pagination.

- The response is concatenated using AND condition (OR is not supported)
- The maximum result set size is >100
- Offset is the zero-based number of assessment profiles from the start of the result set

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/get_assessment_profiles'
-d '{ "request_data" : { "pagination" : { "search_from" : 0, "search_to" : 1 }, "filters" : [ { "field" : "field",
"value" : "AssessmentProfileFilter_value", "operator" : "operator" }, { "field" : "field", "value" :
"AssessmentProfileFilter_value", "operator" : "operator" } ], "sort" : { "field" : "creation_time", "keyword" : "asc"
} } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"string\",\"operator\":\"string\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"pagination\":{\"search_from\":0,\"search_to\":100}}}" headers = { 'x-xdr-
auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/compliance/get_assessment_profiles", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/get_assessment_profiles") http = Net::HTTP.new(url.host, url.port) http.use_ssl =
true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"string\",\"operator\":\"string\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"pagination\":{\"search_from\":0,\"search_to\":100}}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "string", "operator": "string", "value":
"string" } ], "sort": { "field": "name", "keyword": "asc" }, "pagination": { "search_from": 0, "search_to": 100 } }
}); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function
() { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/get_assessment_profiles"); xhr.setRequestHeader("x-xdr-auth-id",
"REPLACE_KEY_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/get_assessment_profiles")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"filters\":[{\"field\":\"string\",\"operator\":\"string\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"pagination\":{\"search_from\":0,\"search_to\":100}}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": [ "filters": [ [ "field": "string", "operator": "string", "value": "string" ] ],
"sort": [ "field": "name", "keyword": "asc" ], "pagination": [ "search_from": 0, "search_to": 100 ] ]] as [String :
Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/compliance/get_assessment_profiles")! as
URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST"
request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/get_assessment_profiles", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"string\",\"operator\":\"string\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"pagination\":{\"search_from\":0,\"search_to\":100}}}", CURLOPT_HTTPHEADER
=> [ "content-type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/get_assessment_profiles"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"string\",\"operator\":\"string\",\"value\":\"string\"}],\"sort\":
```

```
{\"field\":\"name\",\"keyword\":\"asc\"},\"pagination\":{\"search_from\":0,\"search_to\":100}}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/get_assessment_profiles"); var request =
new RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-
type", "application/json"); request.AddParameter("application/json", "{\"request_data\":{\"filters\":
[{\"field\":\"string\",\"operator\":\"string\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"pagination\":{\"search_from\":0,\"search_to\":100}}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

▶ filtersarray

▶ sortobject

▶ paginationobject

REQUEST Example 1

```
{ "request_data": { "search_from": 0, "search_to": 50, "sort": { "field": "report_type", "keyword": "desc" },
"filters": [ { "field": "creation_time", "operator": "gte", "value": 0 }, { "field": "name", "operator": "eq",
"value": "John" } ] } }
```

Responses

Successfully retrieved assessment profiles

Body

application/json

▼ replyobject

▶ total_countinteger

Total number of records

▶ result_countinteger

Number of records matching the filter + paging

▶ assessment_profilesarray

RESPONSE

```
{ "reply": { "total_count": 925, "result_count": 1, "assessment_profiles": [ { "ID":
"472141782aff4a2f999c5e3c35745b3a", "NAME": "Assessment example", "STANDARD_ID": "2nnd684-4z14-5cs82-c1a0-
7c322b671844", "STANDARD_NAME": "CIS Amazon Linux 2 Benchmark v1.0.0", "ASSET_GROUP_ID": 1, "ASSET_GROUP_NAME":
"asset group name", "DESCRIPTION": "assessment description", "REPORT_FREQUENCY": null, "REPORT_TARGETS": [],
"REPORT_TYPE": "NONE", "ENABLED": true, "INSERT_TS": 1748259453615, "MODIFY_TS": 1748259453615, "CREATED_BY":
"Generic Name", "MODIFIED_BY": "Generic Name" } ] } }
```

Bad request - invalid input parameters

Body

application/json

▼ replyobject

▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field

```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory,
is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq,
contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra":
{} } }
```

Authentication required

Body

application/json

▼ replyobjectrequired

▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong

▶ err_extraobject

Additional error details

RESPONSE
```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

Access denied - insufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong

▶ err_extraobject

Additional error details

RESPONSE
```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.2.2 | Get assessment profile by ID

post /public_api/v1/compliance/get_assessment_profile

An assessment profile uses a standard to run scans on an asset group to check whether the assets adhere to the standard.

Get details of the assessment profile specified by its ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
```

```
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/get_assessment_profile'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"id\":\"string\"}}" headers = { 'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/compliance/get_assessment_profile", payload, headers) res = conn.getresponse()
data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/get_assessment_profile") http = Net::HTTP.new(url.host, url.port) http.use_ssl =
true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"id\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "id": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/get_assessment_profile"); xhr.setRequestHeader("x-xdr-auth-id",
"REPLACE_KEY_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/get_assessment_profile")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"id\":\"string\"}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": ["id": "string"]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/compliance/get_assessment_profile")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/get_assessment_profile", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"id\":\"string\"}}", CURLOPT_HTTPHEADER => [ "content-type:
application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/get_assessment_profile"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":{\"id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/get_assessment_profile"); var request =
new RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-
type", "application/json"); request.AddParameter("application/json", "{\"request_data\":{\"id\":\"string\"}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ idstring

Assessment profile ID

REQUEST

```
{ "request_data": { "id": "example" } }
```

Responses

Successfully retrieved assessment profile

Body

application/json

▼ idstring

Assessment profile ID

▼ namestring

Assessment profile name

▼ descriptionstring

Description of the assessment profile

▼ enabledboolean

Whether the assessment profile is enabled

▼ standard_idstring

Standard ID

▼ standard_namestring

Standard name

▼ asset_group_idinteger

Asset Group ID

▼ asset_group_namestring

Asset Group name

▼ report_typestring

Report type

▼ report_targetsarray[string]

▼ evaluation_frequencystring

▼ creation_timestringdate-time

Assessment profile creation time

▼ modification_timestringdate-time

Assessment profile modification time

▼ created_bystring

Assessment profile creator

▼ modified_bystring

Assessment profile modified by

RESPONSE


```
{ "id": "example", "name": "example", "description": "example", "enabled": false, "standard_id": "example",
"standard_name": "example", "asset_group_id": 0, "asset_group_name": "example", "report_type": "example",
"report_targets": [ "email string" ], "evaluation_frequency": "example", "creation_time": "2020-01-01T12:00:00Z",
"modification_time": "2020-01-01T12:00:00Z", "created_by": "example", "modified_by": "example" }
```


Bad request - invalid input parameters

Body

application/json

▼ replyobject

▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field


```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory,
is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq,
contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra":
{} } }
```


Authentication required

Body

application/json

▼ replyobjectrequired

▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Access denied - insufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Resource not found

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

2.8.2.3 | Add assessment profile

post /public_api/v1/compliance/add_assessment_profile

Create a custom compliance assessment profile that is tailored to your own business needs and organizational policies.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/add_assessment_profile'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"profile_name\":\"string\",\"asset_group_id\":\"string\",\"standard_id\":\"string\",\"description\":\"string\",\"report_targe
[\"user@example.com\"],\"report_type\":\"ALL\",\"evaluation_frequency\":\"0 12 * * *\"}}" headers = { 'x-xdr-auth-
id': "REPLACE_KEY_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/compliance/add_assessment_profile", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/add_assessment_profile") http = Net::HTTP.new(url.host, url.port) http.use_ssl =
true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"profile_name\":\"string\",\"asset_group_id\":\"string\",\"standard_id\":\"string\",\"description\":\"string\",\"report_targe
[\"user@example.com\"],\"report_type\":\"ALL\",\"evaluation_frequency\":\"0 12 * * *\"}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "profile_name": "string", "asset_group_id": "string", "standard_id":
"string", "description": "string", "report_targets": [ "user@example.com" ], "report_type": "ALL",
"evaluation_frequency": "0 12 * * *" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/add_assessment_profile"); xhr.setRequestHeader("x-xdr-auth-id",
"REPLACE_KEY_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/add_assessment_profile")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"profile_name\":\"string\",\"asset_group_id\":\"string\",\"standard_id\":\"string\",\"description\":\"string\",\"report_targe
[\"user@example.com\"],\"report_type\":\"ALL\",\"evaluation_frequency\":\"0 12 * * *\"}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": [ "profile_name": "string", "asset_group_id": "string", "standard_id": "string",
"description": "string", "report_targets": ["user@example.com"], "report_type": "ALL", "evaluation_frequency": "0 12
* * *" ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let
request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/compliance/add_assessment_profile")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/add_assessment_profile", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"profile_name\":\"string\",\"asset_group_id\":\"string\",\"standard_id\":\"string\",\"description\":\"string\",\"report_targe
[\"user@example.com\"],\"report_type\":\"ALL\",\"evaluation_frequency\":\"0 12 * * *\"}}", CURLOPT_HTTPHEADER => [
"content-type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/add_assessment_profile"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":
{\"profile_name\":\"string\",\"asset_group_id\":\"string\",\"standard_id\":\"string\",\"description\":\"string\",\"report_targe
[\"user@example.com\"],\"report_type\":\"ALL\",\"evaluation_frequency\":\"0 12 * * *\"}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/add_assessment_profile"); var request =
new RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-
type", "application/json"); request.AddParameter("application/json", "{\"request_data\":
{\"profile_name\":\"string\",\"asset_group_id\":\"string\",\"standard_id\":\"string\",\"description\":\"string\",\"report_targe
```

```
[\"user@example.com\"],\"report_type\":\"ALL\",\"evaluation_frequency\":\"0 12 * * *\"}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ profile_namestring

Name of the assessment profile

▶ asset_group_idstring

Asset Group ID

▶ standard_idstring

Standard ID

▶ descriptionstring

Description of the assessment profile

▶ report_targetsarray[string]

Assessment profile targets

▶ report_typestring (Enum)

Assessment profile report type. Valid values include:

- `ALL`
- `PDF`
- `CSV`
- `NONE`

▶ evaluation_frequencystringcron

Assessment report evaluation frequency

REQUEST

```
{ "request_data": { "profile_name": "example", "asset_group_id": "example", "standard_id": "example", "description":
"example", "report_targets": [ "email string" ], "report_type": "ALL", "evaluation_frequency": "0 12 * * *" } }
```

Responses

Successfully created assessment profile

Body

application/json

▼ successboolean

Whether the assessment profile creation succeeded

RESPONSE

```
{ "success": false }
```

Bad request - invalid input parameters

Body

application/json

▼ replyobject

▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field

```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory,
is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq,
contains, not_contains", "err_extra": {} } }
```

{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra": {} } }

Authentication required

Body
application/json
▼ replyobjectrequired
▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Access denied - insufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Conflict - duplicate resource

Body
application/json
▼ replyobjectrequired
▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details
RESPONSE
```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.2.4 | Edit assessment profile

post /public_api/v1/compliance/edit_assessment_profile

Edit an existing compliance custom assessment profile.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/edit_assessment_profile'
-d '{ "request_data" : { "standard_id" : "standard_id", "profile_name" : "profile_name", "asset_group_id" : 0,
"evaluation_frequency" : "0 12 * * *", "description" : "description", "report_targets" : [ "report_targets",
"report_targets" ], "id" : "id", "report_type" : "ALL", "enabled" : "yes" } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"\":{\"request_data\":
{\"id\":\"string\",\"enabled\":\"yes\",\"profile_name\":\"string\",\"description\":\"string\",\"report_targets\":
[\"user@example.com\"],\"report_type\":\"ALL\",\"evaluation_frequency\":\"0 12 * *
*\",\"asset_group_id\":0,\"standard_id\":\"string\"}}}" headers = { 'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-
type': "application/json" } conn.request("POST", "/public_api/v1/compliance/edit_assessment_profile", payload,
headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/edit_assessment_profile") http = Net::HTTP.new(url.host, url.port) http.use_ssl =
true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"\":{\"request_data\":
{\"id\":\"string\",\"enabled\":\"yes\",\"profile_name\":\"string\",\"description\":\"string\",\"report_targets\":
[\"user@example.com\"],\"report_type\":\"ALL\",\"evaluation_frequency\":\"0 12 * *
*\",\"asset_group_id\":0,\"standard_id\":\"string\"}}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "": { "request_data": { "id": "string", "enabled": "yes", "profile_name": "string",
"description": "string", "report_targets": [ "user@example.com" ], "report_type": "ALL", "evaluation_frequency": "0
12 * * *", "asset_group_id": 0, "standard_id": "string" } } }); const xhr = new XMLHttpRequest(); xhr.withCredentials
= true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/edit_assessment_profile"); xhr.setRequestHeader("x-xdr-auth-id",
"REPLACE_KEY_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/edit_assessment_profile")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"\":
{\"request_data\":
{\"id\":\"string\",\"enabled\":\"yes\",\"profile_name\":\"string\",\"description\":\"string\",\"report_targets\":
[\"user@example.com\"],\"report_type\":\"ALL\",\"evaluation_frequency\":\"0 12 * *
*\",\"asset_group_id\":0,\"standard_id\":\"string\"}}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["": ["request_data": [ "id": "string", "enabled": "yes", "profile_name": "string", "description":
"string", "report_targets": ["user@example.com"], "report_type": "ALL", "evaluation_frequency": "0 12 * * *",
"asset_group_id": 0, "standard_id": "string" ]]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/compliance/edit_assessment_profile")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/edit_assessment_profile", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"\":{\"request_data\":
```

{\"id\":\"string\",\"enabled\":\"yes\",\"profile_name\":\"string\",\"description\":\"string\",\"report_targets\":
[\"user@example.com\"],\"report_type\":\"ALL\",\"evaluation_frequency\":\"0 12 * *
*\",\"asset_group_id\":0,\"standard_id\":\"string\"}}}", CURLOPT_HTTPHEADER => ["content-type: application/json",
"x-xdr-auth-id: REPLACE_KEY_VALUE"],]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl);
if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/edit_assessment_profile"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"\":{\"request_data\":
{\"id\":\"string\",\"enabled\":\"yes\",\"profile_name\":\"string\",\"description\":\"string\",\"report_targets\":
[\"user@example.com\"],\"report_type\":\"ALL\",\"evaluation_frequency\":\"0 12 * *
*\",\"asset_group_id\":0,\"standard_id\":\"string\"}}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/edit_assessment_profile"); var request =
new RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-
type", "application/json"); request.AddParameter("application/json", "{\"\":{\"request_data\":
{\"id\":\"string\",\"enabled\":\"yes\",\"profile_name\":\"string\",\"description\":\"string\",\"report_targets\":
[\"user@example.com\"],\"report_type\":\"ALL\",\"evaluation_frequency\":\"0 12 * *
*\",\"asset_group_id\":0,\"standard_id\":\"string\"}}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);

Body parameters

required

application/json

▼ objectrequired

▶ request_dataobjectrequired

REQUEST Example 1

{ "request_data": { "id": "48e2f6a9fdc049479e9c6a8eda0bd163", "control_name": "Control 1" } }

Responses

Successfully updated assessment profile

Body

application/json

▼ successboolean

RESPONSE

{ "success": false }

Bad request - invalid input parameters

Body

application/json

▼ replyobject

▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field

{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory,
is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq,
contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra":
{} } }

Authentication required

Body

application/json

▼ replyobjectrequired

▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong
▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Access denied - insufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Resource not found

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Conflict - duplicate resource

Body
application/json
▼ replyobjectrequired
▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE

```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.2.5 | Delete assessment profile

post /public_api/v1/compliance/delete_assessment_profile

Delete an assessment profile specified by its ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/delete_assessment_profile'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"id\":\"string\"}}" headers = { 'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/compliance/delete_assessment_profile", payload, headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/delete_assessment_profile") http = Net::HTTP.new(url.host, url.port) http.use_ssl =
true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"id\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "id": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/delete_assessment_profile"); xhr.setRequestHeader("x-xdr-auth-id",
"REPLACE_KEY_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-
yourfqdn/public_api/v1/compliance/delete_assessment_profile") .header("x-xdr-auth-id", "REPLACE_KEY_VALUE")
.header("content-type", "application/json") .body("{\"request_data\":{\"id\":\"string\"}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": ["id": "string"]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/compliance/delete_assessment_profile")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/delete_assessment_profile", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"id\":\"string\"}}", CURLOPT_HTTPHEADER => [ "content-type:
application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/delete_assessment_profile"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":{\"id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/delete_assessment_profile"); var request =
new RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-
```

```
type", "application/json"); request.AddParameter("application/json", "{\"request_data\":{\"id\":\"string\"}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ idstring

Assessment profile ID

REQUEST

```
{ "request_data": { "id": "example" } }
```

Responses

Successfully deleted assessment profile

Body

application/json

▼ successboolean

Whether the assessment profile deletion succeeded

RESPONSE

```
{ "success": false }
```

Bad request - invalid input parameters

Body

application/json

▼ replyobject

▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field

```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory,
is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq,
contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra":
{} } }
```

Authentication required

Body

application/json

▼ replyobjectrequired

▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong

▶ err_extraobject

Additional error details

RESPONSE

```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

Access denied - insufficient permissions

Body

application/json

▼ replyobjectrequired

▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Resource not found

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

2.8.3 | Assessment results

Operations for managing assessment results

2.8.3.1 | Get assessment profile results

post /public_api/v1/compliance/get_assessment_results

Retrieve assessment profile results with optional filtering based on the last successful evaluation.

Filtering Support:

- Field (`labels`): `contains`, `not_contains`

Note: This API does not support sorting or pagination. Sorting will not take effect even if it is specified in the payload. The results will not return the results based on sorting'

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/get_assessment_results'
-d '{ "request_data" : { "filters" : [ { "field" : "labels", "operator" : "contains", "value" : "aws" } ] } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"labels\",\"operator\":\"contains\",\"value\":\"aws\"}]}}" headers = { 'x-xdr-auth-id':
"REPLACE_KEY_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/compliance/get_assessment_results", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/get_assessment_results") http = Net::HTTP.new(url.host, url.port) http.use_ssl =
true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"labels\",\"operator\":\"contains\",\"value\":\"aws\"}]}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "labels", "operator": "contains", "value":
"aws" } ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/get_assessment_results"); xhr.setRequestHeader("x-xdr-auth-id",
"REPLACE_KEY_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/get_assessment_results")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"filters\":[{\"field\":\"labels\",\"operator\":\"contains\",\"value\":\"aws\"}]}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": ["filters": [ [ "field": "labels", "operator": "contains", "value": "aws" ] ]]] as
[String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/compliance/get_assessment_results")! as
URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST"
request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/get_assessment_results", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"labels\",\"operator\":\"contains\",\"value\":\"aws\"}]}}", CURLOPT_HTTPHEADER => [ "content-type:
application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/get_assessment_results"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"labels\",\"operator\":\"contains\",\"value\":\"aws\"}]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/get_assessment_results"); var request =
new RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-
type", "application/json"); request.AddParameter("application/json", "{\"request_data\":{\"filters\":
[{\"field\":\"labels\",\"operator\":\"contains\",\"value\":\"aws\"}]}}", ParameterType.RequestBody); IRestResponse
response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

▶ filtersarray

Array of filter objects

REQUEST Example 1

```
{ "request_data": { "filters": [ { "field": "labels", "operator": "contains", "value": "aws" } ] } }
```

Responses

Successfully retrieved assessment profile results

Body
application/json
▼ replyobject
▶ total_countinteger

Total number of assessment results available

▶ result_countinteger

Number of assessment results matching the applied filters

▶ standard_filter_countinteger

Count from standard filter processing

▶ standard_total_countinteger

Total count from standard processing

▶ dataarray

Array of assessment profile results

RESPONSE

```
{ "reply": { "total_count": 150, "result_count": 25, "standard_filter_count": 25, "standard_total_count": 150,
"data": [ { "TYPE": "standard", "SCORE": null, "ASSET_GROUP_NAME": null, "ASSET_GROUP_ID": null, "ADDITIONAL_DATA":
null, "LAST_EVALUATION_TIME": 1751533328000, "STANDARD_NAME": "CIS Amazon Elastic Kubernetes Service (EKS) Benchmark
v1.4", "PROFILES_COUNT": 3, "STANDARD_ID": 9101546381587280000, "LABELS": [ "eks" ], "PATH": [ "CIS Amazon Elastic
Kubernetes Service (EKS) Benchmark v1.4" ], "FAILED_CONTROLS_SEVERITY": [ { "value": "CRITICAL", "count": 0,
"pretty_name": "Critical" }, { "value": "HIGH", "count": 0, "pretty_name": "High" }, { "value": "MEDIUM", "count": 1,
"pretty_name": "Medium" }, { "value": "LOW", "count": 2, "pretty_name": "Low" }, { "value": "INFORMATIONAL", "count":
0, "pretty_name": "Informational" } ], "CONTROLS_STATUS": [ { "value": "PASSED", "count": 0, "pretty_name": "Passed"
}, { "value": "NOT_ASSESSED", "count": 153, "pretty_name": "Not Assessed" }, { "value": "FAILED", "count": 0,
"pretty_name": "Failed" } ], "ASSESSMENT_PROFILE_REVISION": null } ] } }
```


Bad request - invalid input parameters

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field

```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory,
is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq,
contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra":
{} } }
```


Authentication required

Body
application/json
▼ replyobjectrequired
▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong

▶ err_extraobject

Additional error details
RESPONSE
```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

Access denied - insufficient permissions

Body
application/json
▼ replyobjectrequired
▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong

▶ err_extraobject

Additional error details

RESPONSE
```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.4 | Controls

Operations for managing compliance controls

2.8.4.1 | Get compliance controls

post /public_api/v1/compliance/get_controls

Compliance controls are measures related to the standard that ensure compliance and mitigate risks. Controls are built from one or more rules, the specific checks that run on an asset. Controls can be grouped into categories, for example RBAC and Pod security.

Retrieve compliance control details with optional filtering, sorting and pagination.

- The response is concatenated using AND condition (OR is not supported)
- The maximum result set size is >100
- Offset is the zero-based number of assessment profiles from the start of the result set

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
```

```
'https://api-yourfqdn/public_api/v1/compliance/get_controls'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":100}}" headers = { 'x-xdr-auth-id':
"REPLACE_KEY_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/compliance/get_controls", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/get_controls") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":100}}" response = http.request(request) puts
response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "name", "operator": "eq", "value": "string" }
], "sort": { "field": "name", "keyword": "asc" }, "search_from": 0, "search_to": 100 } }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/get_controls"); xhr.setRequestHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE");
xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/get_controls")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"filters\":[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":100}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": [ "filters": [ [ "field": "name", "operator": "eq", "value": "string" ] ], "sort": [
"field": "name", "keyword": "asc" ], "search_from": 0, "search_to": 100 ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/compliance/get_controls")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/get_controls", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":100}}", CURLOPT_HTTPHEADER => [ "content-
type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/get_controls"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":100}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/get_controls"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-type",
"application/json"); request.AddParameter("application/json", "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":100}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters

application/json

▼ request_dataobject

▶ filtersarray

▶ sortobject

▶ search_frominteger

Integer representing the starting offset within the query result set from which you want compliance controls returned. Compliance controls are returned as a zero-based list. Any compliance control indexed less than this value is not returned in the final result set and defaults to zero.

▶ search_tointeger

Integer representing the end offset within the result set after which you do not want compliance controls returned. Compliance controls in the compliance control list that are indexed higher than this value are not returned in the final results set. Defaults to >100, which returns all compliance controls to the end of the list.

REQUEST basic_filter

```
{ "request_data": { "filters": [ { "field": "name", "operator": "contains", "value": "Access Enforcement" } ] } }
{ "request_data": { "filters": [ { "field": "is_custom", "operator": "in", "value": [ "yes" ] } ] } }
{ "request_data": { "filters": [ { "field": "creation_time", "operator": "gte", "value": 1640995200000 }, { "field": "creation_time", "operator": "lte", "value": 1672531200000 } ] } }
```

Responses

Successfully retrieved controls

Body
application/json
▼ replyobject
▶ total_countinteger

Total number of controls

▶ result_countinteger

Number of controls matching filters + paging

▶ controlsarray

RESPONSE

```
{ "reply": { "total_count": 0, "result_count": 0, "controls": [ { "id": "example", "name": "example", "description": "example", "category": "example", "subcategory": "example", "creation_time": 0, "modification_time": 0, "created_by": "example", "modified_by": "example", "is_custom": false } ] } }
```

Bad request - invalid input parameters

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field

```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory, is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq, contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra": {} } }
```

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE

```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.4.2 | Get compliance control by ID

post /public_api/v1/compliance/get_control

Get details of the compliance control specified by its ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/get_control'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"id\":\"string\"}}" headers = { 'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/compliance/get_control", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/get_control") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"id\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "id": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/get_control"); xhr.setRequestHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE");
xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/get_control") .header("x-
xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"id\":\"string\"}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": ["id": "string"]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/compliance/get_control")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/get_control", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"id\":\"string\"}}", CURLOPT_HTTPHEADER => [ "content-type:
application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/get_control"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":{\"id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/get_control"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-type",
"application/json"); request.AddParameter("application/json", "{\"request_data\":{\"id\":\"string\"}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ idstring

Compliance control ID

REQUEST Example 1

{ "request_data": { "id": "ede175cb30ef4dd4a0e3031b1ee77868" } }

Responses

Successfully retrieved control

Body
application/json
▼ controlobject
▶ CATEGORYstring
▶ CATEGORY_DESCRIPTIONstring
▶ SUBCATEGORYstring
▶ SUBCATEGORY_DESCRIPTIONstring
▶ CONTROL_IDstring
▶ STANDARDSarray[string]
▶ CONTROL_NAMEstring
▶ SEVERITYstring
▶ SUPPORTEDboolean
▶ INSERTION_TIMEinteger
▶ MODIFICATION_TIMEinteger
▶ MODIFIED_BYnull
▶ CREATED_BYstring
▶ DESCRIPTIONstring
▶ MITIGATIONnull
▶ ADDITIONAL_DATAarray
▶ COMPLIANCE_RULESarray
▶ RULESinteger
▶ REVISIONstring
▶ IMPACTnull
▶ AUTOMATION_STATUSstring
▶ AUDIT_PROCEDUREnull
▶ ENABLEDboolean
▶ IS_CUSTOMboolean
▶ STATUSstring
RESPONSE
{ "control": { "CATEGORY": "example", "CATEGORY_DESCRIPTION": "example", "SUBCATEGORY": "example", "SUBCATEGORY_DESCRIPTION": "example", "CONTROL_ID": "example", "STANDARDS": ["example"], "CONTROL_NAME": "example", "SEVERITY": "example", "SUPPORTED": false, "INSERTION_TIME": 0, "MODIFICATION_TIME": 0, "MODIFIED_BY": null, "CREATED_BY": "example", "DESCRIPTION": "example", "MITIGATION": null, "ADDITIONAL_DATA": [{}], "COMPLIANCE_RULES": [{}], "RULES": 0, "REVISION": "example", "IMPACT": null, "AUTOMATION_STATUS": "example", "AUDIT_PROCEDURE": null, "ENABLED": false, "IS_CUSTOM": false, "STATUS": "example" } }

Bad request - invalid input parameters

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory, is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq, contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra": {} } }

Resource not found

Body
application/json
▼ replyobject

▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

2.8.4.3 | Add new control

post /public_api/v1/compliance/add_control

Create a custom compliance control.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/add_control'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"control_name\":\"string\",\"description\":\"string\",\"category\":\"string\",\"subcategory\":\"string\"}}" headers
= { 'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/compliance/add_control", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/add_control") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"control_name\":\"string\",\"description\":\"string\",\"category\":\"string\",\"subcategory\":\"string\"}}"
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "control_name": "string", "description": "string", "category":
"string", "subcategory": "string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/compliance/add_control");
```

```
xhr.setRequestHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/add_control") .header("x-
xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"control_name\":\"string\",\"description\":\"string\",\"category\":\"string\",\"subcategory\":\"string\"}}")
.asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": [ "control_name": "string", "description": "string", "category": "string",
"subcategory": "string" ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/compliance/add_control")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/add_control", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"control_name\":\"string\",\"description\":\"string\",\"category\":\"string\",\"subcategory\":\"string\"}}",
CURLOPT_HTTPHEADER => [ "content-type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/add_control"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":
{\"control_name\":\"string\",\"description\":\"string\",\"category\":\"string\",\"subcategory\":\"string\"}}");
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/add_control"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-type",
"application/json"); request.AddParameter("application/json", "{\"request_data\":
{\"control_name\":\"string\",\"description\":\"string\",\"category\":\"string\",\"subcategory\":\"string\"}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ control_namestring

Name of the compliance control

▶ descriptionstring

Description of the compliance control

▶ categorystring

Compliance control category

▶ subcategorystring

Compliance control subcategory

REQUEST Example 1

```
{ "request_data": { "control_name": "control name", "category": "Network Configuration", "subcategory": "4.2" } }
```

Responses

Control added successfully

Body

application/json

▼ successboolean

Whether the compliance control creation was successful

RESPONSE

```
{ "success": false }
```

Bad request - invalid input parameters

Body

application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field

```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory,
is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq,
contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra":
{} } }
```


Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE

```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.4.4 | Edit existing control

post /public_api/v1/compliance/edit_control

Edit an existing compliance custom compliance control.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/edit_control'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"id\":\"string\",\"control_name\":\"string\",\"description\":\"string\",\"category\":\"string\",\"subcategory\":\"string\"}}"
headers = { 'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/compliance/edit_control", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/edit_control") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
```

```
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"id\":\"string\",\"control_name\":\"string\",\"description\":\"string\",\"category\":\"string\",\"subcategory\":\"string\"}}"
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "id": "string", "control_name": "string", "description": "string",
"category": "string", "subcategory": "string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/compliance/edit_control");
xhr.setRequestHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/edit_control")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"id\":\"string\",\"control_name\":\"string\",\"description\":\"string\",\"category\":\"string\",\"subcategory\":\"string\"}}"
.asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": [ "id": "string", "control_name": "string", "description": "string", "category":
"string", "subcategory": "string" ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject:
parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/compliance/edit_control")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/edit_control", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"id\":\"string\",\"control_name\":\"string\",\"description\":\"string\",\"category\":\"string\",\"subcategory\":\"string\"}}"
CURLOPT_HTTPHEADER => [ "content-type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/edit_control"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":
{\"id\":\"string\",\"control_name\":\"string\",\"description\":\"string\",\"category\":\"string\",\"subcategory\":\"string\"}}"
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/edit_control"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-type",
"application/json"); request.AddParameter("application/json", "{\"request_data\":
{\"id\":\"string\",\"control_name\":\"string\",\"description\":\"string\",\"category\":\"string\",\"subcategory\":\"string\"}}"
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ idstring

Compliance control ID

▶ control_namestring

Name of the compliance control

▶ descriptionstring

Description of the compliance control

▶ categorystring

Compliance control category

▶ subcategorystring

Compliance control subcategory

REQUEST Example 1

```
{ "request_data": { "id": "48e2f6a9fcc049579e9c6b8eda0bd123", "control_name": "Great new name" } }
```

Responses

Control updated successfully

Body
application/json
▼ successboolean

Whether the custom compliance control was edited successfully

RESPONSE
```
{ "success": false }
```

Bad request - invalid input parameters

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field
```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory,
is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq,
contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra":
{} } }
```

Resource not found

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.4.5 | Delete control

post /public_api/v1/compliance/delete_control

Delete a compliance control specified by its ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/delete_control'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"id\":\"string\"}}" headers = { 'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/compliance/delete_control", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/delete_control") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"id\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "id": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/delete_control"); xhr.setRequestHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE");
xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/delete_control")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"id\":\"string\"}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": ["id": "string"]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/compliance/delete_control")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/delete_control", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"id\":\"string\"}}", CURLOPT_HTTPHEADER => [ "content-type:
application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/delete_control"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":{\"id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/delete_control"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-type",
"application/json"); request.AddParameter("application/json", "{\"request_data\":{\"id\":\"string\"}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ idstring

Compliance control ID

REQUEST Example 1

{ "request_data": { "id": "f9f744717b284e9383f9c1210ad3143d" } }

Responses

Control deleted successfully

Body
application/json
▼ successboolean

Whether the compliance control was successfully deleted

RESPONSE
{ "success": false }

Bad request - invalid input parameters

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory, is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq, contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra": {} } }

Resource not found

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE

```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.4.6 | Get control by revision

post /public_api/v1/compliance/get_control_by_revision

Get details of the compliance control specified by its revision ID, including inactive controls.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/get_control_by_revision'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"control_revision\":\"string\"}}" headers = { 'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type':
"application/json" } conn.request("POST", "/public_api/v1/compliance/get_control_by_revision", payload, headers) res
= conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/get_control_by_revision") http = Net::HTTP.new(url.host, url.port) http.use_ssl =
true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"control_revision\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "control_revision": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/get_control_by_revision"); xhr.setRequestHeader("x-xdr-auth-id",
"REPLACE_KEY_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/get_control_by_revision")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"control_revision\":\"string\"}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": ["control_revision": "string"]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/compliance/get_control_by_revision")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/get_control_by_revision", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"control_revision\":\"string\"}}", CURLOPT_HTTPHEADER => [
"content-type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/get_control_by_revision"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":{\"control_revision\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/get_control_by_revision"); var request =
new RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-
type", "application/json"); request.AddParameter("application/json", "{\"request_data\":
{\"control_revision\":\"string\"}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ control_revisionstring

Control revision ID

REQUEST Example 1
{ "request_data": { "control_revision": "4983640851729713422" } }
Responses

Successfully retrieved control

Body
application/json
▼ replyobject
▶ controlobject
RESPONSE
{ "reply": { "control": { "id": "example", "name": "example", "description": "example", "category": "example", "subcategory": "example", "creation_time": 0, "modification_time": 0, "created_by": "example", "modified_by": "example", "is_custom": false } } }

Bad request - invalid input parameters

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory, is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq, contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra": {} } }

Resource not found

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.5 | Rules

Operations for managing compliance rules

2.8.5.1 | Add compliance rules to a compliance control

post /public_api/v1/compliance/add_rules_to_control

Add compliance rules to the specific compliance control.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"

CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/add_rules_to_control'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"control_id\":\"string\",\"rules\":
[{\"name\":\"string\",\"description\":\"string\",\"type\":\"Identity\",\"logical_id\":\"string\",\"severity\":\"low\",\"scannal
[\"KUBERNETES_NODE\"],\"remediation_steps\":\"string\",\"generate_findings\":true,\"generate_issues\":true,\"generate_scan_logs
headers = { 'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/compliance/add_rules_to_control", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/add_rules_to_control") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"control_id\":\"string\",\"rules\":
[{\"name\":\"string\",\"description\":\"string\",\"type\":\"Identity\",\"logical_id\":\"string\",\"severity\":\"low\",\"scannal
[\"KUBERNETES_NODE\"],\"remediation_steps\":\"string\",\"generate_findings\":true,\"generate_issues\":true,\"generate_scan_logs
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "control_id": "string", "rules": [ { "name": "string", "description":
"string", "type": "Identity", "logical_id": "string", "severity": "low", "scannable_assets": [ "KUBERNETES_NODE" ],
"remediation_steps": "string", "generate_findings": true, "generate_issues": true, "generate_scan_logs": true } ] }
}); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function
() { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/add_rules_to_control"); xhr.setRequestHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE");
xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/add_rules_to_control")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"control_id\":\"string\",\"rules\":
[{\"name\":\"string\",\"description\":\"string\",\"type\":\"Identity\",\"logical_id\":\"string\",\"severity\":\"low\",\"scannal
[\"KUBERNETES_NODE\"],\"remediation_steps\":\"string\",\"generate_findings\":true,\"generate_issues\":true,\"generate_scan_logs
.asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": [ "control_id": "string", "rules": [ [ "name": "string", "description": "string",
"type": "Identity", "logical_id": "string", "severity": "low", "scannable_assets": ["KUBERNETES_NODE"],
"remediation_steps": "string", "generate_findings": true, "generate_issues": true, "generate_scan_logs": true ] ] ]]
as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/compliance/add_rules_to_control")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields
= headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with:
```

```
request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) }
else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/add_rules_to_control", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"control_id\":\"string\",\"rules\":
[{\"name\":\"string\",\"description\":\"string\",\"type\":\"Identity\",\"logical_id\":\"string\",\"severity\":\"low\",\"scannal
[\"KUBERNETES_NODE\"],\"remediation_steps\":\"string\",\"generate_findings\":true,\"generate_issues\":true,\"generate_scan_logs
CURLOPT_HTTPHEADER => [ "content-type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/add_rules_to_control"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":{\"control_id\":\"string\",\"rules\":
[{\"name\":\"string\",\"description\":\"string\",\"type\":\"Identity\",\"logical_id\":\"string\",\"severity\":\"low\",\"scannal
[\"KUBERNETES_NODE\"],\"remediation_steps\":\"string\",\"generate_findings\":true,\"generate_issues\":true,\"generate_scan_logs
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/add_rules_to_control"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-type",
"application/json"); request.AddParameter("application/json", "{\"request_data\":
{\"control_id\":\"string\",\"rules\":
[{\"name\":\"string\",\"description\":\"string\",\"type\":\"Identity\",\"logical_id\":\"string\",\"severity\":\"low\",\"scannal
[\"KUBERNETES_NODE\"],\"remediation_steps\":\"string\",\"generate_findings\":true,\"generate_issues\":true,\"generate_scan_logs
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ control_idstring

Compliance control ID

▶ rulesarray

REQUEST Example 1

```
{ "request_data": { "control_id": "0b0b5354d06d43ffc3d7465858378135", "rules": [ { "name": "Rule 1", "description":
"Description 1", "type": "Identity", "logical_id": "2b16a9f3-260x2-4xsf0cc-a9091", "severity": "high",
"scannable_assets": [ "KUBERNETES_NODE" ], "remediation_steps": "Run the following command...", "generate_findings":
true, "generate_issues": true, "generate_scan_logs": true }, { "name": "Rule 2", "description": "Description 2",
"type": "Identity", "logical_id": "2b16a9f3-cc2x602-4vvxXccXfn0c1", "scannable_assets": [ "KUBERNETES_NODE" ],
"generate_findings": false, "generate_issues": true, "generate_scan_logs": true } ] } }
```

Responses

Rules added successfully

Body

application/json

▼ successboolean

Whether the rules were added successfully

RESPONSE

```
{ "success": false }
```

Bad request - invalid input parameters

Body

application/json

▼ replyobject

▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field

{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory, is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq, contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra": {} } }

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

2.8.5.2 | Delete rules from control

post /public_api/v1/compliance/delete_rules_from_control

Remove specific rules from the specified compliance control.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"
CLIENT REQUEST Bash+curl
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/delete_rules_from_control'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\": {\"control_id\":\"string\",\"rules_ids\":[\"string\"]}}" headers = { 'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/compliance/delete_rules_from_control", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/compliance/delete_rules_from_control") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] = 'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\": {\"control_id\":\"string\",\"rules_ids\":[\"string\"]}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "control_id": "string", "rules_ids": ["string"] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/compliance/delete_rules_from_control"); xhr.setRequestHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/delete_rules_from_control") .header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":{\"control_id\":\"string\",\"rules_ids\":[\"string\"]}}") .asString();
import Foundation let headers = ["x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json"] let parameters = ["request_data": ["control_id": "string", "rules_ids": ["string"]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:

```
"https://api-yourfqdn/public_api/v1/compliance/delete_rules_from_control")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/delete_rules_from_control", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"control_id\":\"string\",\"rules_ids\":[\"string\"]}}",
CURLOPT_HTTPHEADER => [ "content-type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/delete_rules_from_control"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":{\"control_id\":\"string\",\"rules_ids\":[\"string\"]}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/delete_rules_from_control"); var request =
new RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-
type", "application/json"); request.AddParameter("application/json", "{\"request_data\":
{\"control_id\":\"string\",\"rules_ids\":[\"string\"]}}", ParameterType.RequestBody); IRestResponse response =
client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ control_idstring

Compliance control ID

▶ rules_idsarray[string]

List of rule IDs to delete

REQUEST Example 1

```
{ "request_data": { "control_id": "48e2f6a9cdc049479e3c6a8eda3bd161", "rules_ids": [ "2b26a9f3-ca2x602-4vvxXccXf3c1"
] } }
```

Responses

Rules deleted successfully

Body

application/json

▼ successboolean

Whether the rules were successfully deleted from the compliance control

RESPONSE

```
{ "success": false }
```

Bad request - invalid input parameters

Body

application/json

▼ replyobject

▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field

```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory,
is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq,
contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
```

{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra": {} } }

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

2.8.6 | Categories

Operations for retrieving categories and subcategories

2.8.6.1 | Get categories and subcategories

post /public_api/v1/compliance/get_control_categories_and_subcategories

Retrieve available compliance control categories and subcategories.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/get_control_categories_and_subcategories'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{}}" headers = {
'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/compliance/get_control_categories_and_subcategories", payload, headers) res = conn.getresponse() data
= res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/get_control_categories_and_subcategories") http = Net::HTTP.new(url.host, url.port)
http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-
auth-id"] = 'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{}}"
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": {} }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/get_control_categories_and_subcategories"); xhr.setRequestHeader("x-xdr-auth-id",
"REPLACE_KEY_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-
yourfqdn/public_api/v1/compliance/get_control_categories_and_subcategories") .header("x-xdr-auth-id",
"REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":{}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": []] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/compliance/get_control_categories_and_subcategories")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
```

```
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/compliance/get_control_categories_and_subcategories", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{}}", CURLOPT_HTTPHEADER => [ "content-type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd, CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/get_control_categories_and_subcategories"); struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":{}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/get_control_categories_and_subcategories"); var request = new RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{\"request_data\":{}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

REQUEST Example 1

```
{ "request_data": {} }
```

Responses

Successfully retrieved categories and subcategories

Body

application/json

▼ replyobject

▶ dataobject

RESPONSE

```
{ "reply": { "data": { "categories": [ "example" ], "subcategories": [ "example" ] } } }
```

Internal server error

Body

application/json

▼ replyobject

▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE

```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.7 | Results

Operations for retrieving compliance results and findings

2.8.7.1 | Get control failed results

post /public_api/v1/compliance/get_control_failed_results

Get the failed findings and issues for a specific compliance control.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/get_control_failed_results'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"control_revision\":\"string\",\"assessment_profile_revision\":\"string\",\"last_evaluation_time\":0}}" headers = {
'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/compliance/get_control_failed_results", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/get_control_failed_results") http = Net::HTTP.new(url.host, url.port) http.use_ssl
= true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"control_revision\":\"string\",\"assessment_profile_revision\":\"string\",\"last_evaluation_time\":0}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "control_revision": "string", "assessment_profile_revision":
"string", "last_evaluation_time": 0 } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/get_control_failed_results"); xhr.setRequestHeader("x-xdr-auth-id",
"REPLACE_KEY_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-
yourfqdn/public_api/v1/compliance/get_control_failed_results") .header("x-xdr-auth-id", "REPLACE_KEY_VALUE")
.header("content-type", "application/json") .body("{\"request_data\":
{\"control_revision\":\"string\",\"assessment_profile_revision\":\"string\",\"last_evaluation_time\":0}}")
.asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": [ "control_revision": "string", "assessment_profile_revision": "string",
"last_evaluation_time": 0 ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/compliance/get_control_failed_results")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/get_control_failed_results", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING =>
"", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1,
CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"control_revision\":\"string\",\"assessment_profile_revision\":\"string\",\"last_evaluation_time\":0}}",
CURLOPT_HTTPHEADER => [ "content-type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/get_control_failed_results"); struct curl_slist *headers
= NULL; headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers =
curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"request_data\":
{\"control_revision\":\"string\",\"assessment_profile_revision\":\"string\",\"last_evaluation_time\":0}}"); CURLcode
ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/get_control_failed_results"); var request
= new RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-
type", "application/json"); request.AddParameter("application/json", "{\"request_data\":
{\"control_revision\":\"string\",\"assessment_profile_revision\":\"string\",\"last_evaluation_time\":0}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ control_revisionstring

Compliance control revision ID

▶ assessment_profile_revisionstring

Assessment profile revision ID

▶ last_evaluation_timeintegerint64

Last evaluation timestamp (epoch time in milliseconds)

REQUEST

```
{ "request_data": { "control_revision": "example", "assessment_profile_revision": "example", "last_evaluation_time": 0 } }
```

Responses

Successfully retrieved failed results

Body
application/json
▼ replyobject
▶ dataobject

RESPONSE Example 1

```
{ "reply": { "data": { "findings": { "ids": [ "45667890n85678908765456789bkk" ], "value": 1 }, "issues": [ { "name": "SEV_020_LOW", "pretty_name": "Low", "value": 0, "ids": [] }, { "name": "SEV_030_MEDIUM", "pretty_name": "Medium", "value": 0, "ids": [] }, { "name": "SEV_040_HIGH", "pretty_name": "High", "value": 1, "ids": [ "4537893ijnmskiu2y6t5rxt7y789ok" ] }, { "name": "SEV_050_CRITICAL", "pretty_name": "Critical", "value": 0, "ids": [] } ] } } }
```

Bad request - invalid input parameters

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field

```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory, is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq, contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra": {} } }
```

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE

```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.7.2 | Get rule failed results

post /public_api/v1/compliance/get_rule_failed_results

Get the failed findings and issues for a specific rule.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/get_rule_failed_results'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"control_revision\":\"string\",\"assessment_profile_revision\":\"string\",\"last_evaluation_time\":0,\"rule_id\":\"string\"}}
headers = { 'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/compliance/get_rule_failed_results", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/get_rule_failed_results") http = Net::HTTP.new(url.host, url.port) http.use_ssl =
true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"control_revision\":\"string\",\"assessment_profile_revision\":\"string\",\"last_evaluation_time\":0,\"rule_id\":\"string\"}}
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "control_revision": "string", "assessment_profile_revision":
"string", "last_evaluation_time": 0, "rule_id": "string" } }); const xhr = new XMLHttpRequest(); xhr.withCredentials
= true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/get_rule_failed_results"); xhr.setRequestHeader("x-xdr-auth-id",
"REPLACE_KEY_VALUE"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/get_rule_failed_results")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"control_revision\":\"string\",\"assessment_profile_revision\":\"string\",\"last_evaluation_time\":0,\"rule_id\":\"string\"}}
.asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": [ "control_revision": "string", "assessment_profile_revision": "string",
"last_evaluation_time": 0, "rule_id": "string" ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/compliance/get_rule_failed_results")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/get_rule_failed_results", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"control_revision\":\"string\",\"assessment_profile_revision\":\"string\",\"last_evaluation_time\":0,\"rule_id\":\"string\"}}
CURLOPT_HTTPHEADER => [ "content-type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/get_rule_failed_results"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":
{\"control_revision\":\"string\",\"assessment_profile_revision\":\"string\",\"last_evaluation_time\":0,\"rule_id\":\"string\"}}
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/get_rule_failed_results"); var request =
new RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-
type", "application/json"); request.AddParameter("application/json", "{\"request_data\":
```

{\"control_revision\":\"string\",\"assessment_profile_revision\":\"string\",\"last_evaluation_time\":0,\"rule_id\":\"string\"}
```
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ control_revisionstring

Compliance control revision ID

▶ assessment_profile_revisionstring

Assessment profile revision ID

▶ last_evaluation_timeintegerint64

Last evaluation timestamp (epoch time in milliseconds)

▶ rule_idstring

Rule ID

REQUEST

```
{ "request_data": { "control_revision": "example", "assessment_profile_revision": "example", "last_evaluation_time":
0, "rule_id": "example" } }
```

Responses

Successfully retrieved rule failed results

Body

application/json

▼ replyobject

▶ dataobject

RESPONSE Example 1

```
{ "reply": { "data": { "findings": { "ids": [ "45667890n85678908765456789bkk" ], "value": 1 }, "issues": [ { "name":
"SEV_020_LOW", "pretty_name": "Low", "value": 0, "ids": [] }, { "name": "SEV_030_MEDIUM", "pretty_name": "Medium",
"value": 0, "ids": [] }, { "name": "SEV_040_HIGH", "pretty_name": "High", "value": 1, "ids": [
"4567893ijnmskiu7y6t5rxt6y789ok" ] }, { "name": "SEV_050_CRITICAL", "pretty_name": "Critical", "value": 0, "ids": []
} ] } } }
```

Bad request - invalid input parameters

Body

application/json

▼ replyobject

▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field

```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory,
is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq,
contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra":
{} } }
```

Internal server error

Body

application/json

▼ replyobject

▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE

```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.8 | Reports

Operations for managing reports

2.8.8.1 | Get compliance reports

post /public_api/v1/compliance/get_reports

Retrieve archived compliance assessment reports with optional filtering, sorting and pagination.

- The response is concatenated using AND condition (OR is not supported)
- The maximum result set size is >100
- Offset is the zero-based number of assessment profiles from the start of the result set

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/get_reports'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"standard_name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"standard_name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":1}}" headers = { 'x-xdr-auth-id':
"REPLACE_KEY_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/compliance/get_reports", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/get_reports") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"standard_name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"standard_name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":1}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "standard_name", "operator": "eq", "value":
"string" } ], "sort": { "field": "standard_name", "keyword": "asc" }, "search_from": 0, "search_to": 1 } }); const
xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/get_reports"); xhr.setRequestHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE");
xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/get_reports") .header("x-
xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":{\"filters\":
[{\"field\":\"standard_name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"standard_name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":1}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": [ "filters": [ [ "field": "standard_name", "operator": "eq", "value": "string" ] ],
"sort": [ "field": "standard_name", "keyword": "asc" ], "search_from": 0, "search_to": 1 ]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/compliance/get_reports")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/get_reports", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"standard_name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"standard_name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":1}}", CURLOPT_HTTPHEADER => [
"content-type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/get_reports"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":{\"filters\":[{\"field\":\"standard_name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"standard_name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":1}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/get_reports"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-type",
"application/json"); request.AddParameter("application/json", "{\"request_data\":{\"filters\":
[{\"field\":\"standard_name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"standard_name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":1}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobject

▶ filtersarray

Optional filters to apply to the reports

▶ sortobject

Sorting configuration

▶ search_frominteger

Starting index for pagination

▶ search_tointeger

Ending index for pagination

REQUEST basic_request

```
{ "request_data": { "search_from": 0, "search_to": 1, "filters": [ { "field": "standard_name", "operator": "neq",
"value": "PCI DSS v4.0.1" } ] } }
{ "request_data": { "search_from": 0, "search_to": 25, "filters": [ { "field": "score", "operator": "gte", "value":
80 } ], "sort": { "field": "score", "keyword": "desc" } } }
{ "request_data": { "search_from": 0, "search_to": 50, "filters": [ { "field": "standard_name", "operator": "eq",
"value": "SOC2" }, { "field": "evaluation_time", "operator": "gte", "value": 1640995200000 } ], "sort": { "field":
"evaluation_time", "keyword": "desc" } } }
```

Responses

Successfully retrieved compliance reports

Body

application/json

▼ replyobject

▶ total_countinteger

Total number of reports in the archive

▶ result_countinteger

Number of reports returned in this response (filter + paging)

▶ filter_countinteger

Number of reports matching the filter criteria

▶ reportsarray

List of compliance reports

RESPONSE

{ "reply": { "total_count": 0, "result_count": 0, "filter_count": 0, "reports": [{ "report_id": 2, "standard_name": "Secure Controls Framework (SCF) v2022.2.1", "assessment_profile": "test-p1", "asset_group": "Asset group for AWS, Azure & GCP", "score": 63, "controls_status": [{ "count": 11, "value": "PASSED", "pretty_name": "Passed" }, { "count": 1027, "value": "NOT_ASSESSED", "pretty_name": "Not assessed" }, { "count": 19, "value": "FAILED", "pretty_name": "Failed" }], "failed_controls_severity": [{ "count": 1, "value": "CRITICAL", "pretty_name": "Critical" }, { "count": 11, "value": "HIGH", "pretty_name": "High" }, { "count": 5, "value": "MEDIUM", "pretty_name": "Medium" }, { "count": 2, "value": "LOW", "pretty_name": "Low" }, { "count": 0, "value": "INFORMATIONAL", "pretty_name": "Informational" }], "evaluation_time": 1759305778000, "assessment_profile_id": "ddc31d5742ba4480b6d1c940c51651c2", "cvs_file_deleted": false }] } }

Bad request - invalid input parameters for reports

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_score_value
{ "reply": { "err_code": 400, "err_msg": "Invalid score value. Score must be between 0 and 100", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'contains' for field 'score'. Allowed operators: gte, lte, eq, neq", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid sort field. Allowed fields: standard_name, assessment_profile_name, asset_group_name, score, evaluation_time", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field 'invalid_field'. Allowed fields: standard_name, assessment_profile_name, asset_group_name, score, evaluation_time", "err_extra": {} } }

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

2.8.9 | Standards

Operations for managing standards

2.8.9.1 | Get compliance standards

post /public_api/v1/compliance/get_standards

Retrieve compliance standards with optional filtering, sorting and pagination.

- The response is concatenated using AND condition (OR is not supported)
- The maximum result set size is >100
- Offset is the zero-based number of assessment profiles from the start of the result set

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/get_standards'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"pagination\":{\"search_from\":0,\"search_to\":100}}}" headers = { 'x-xdr-
auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/compliance/get_standards", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/get_standards") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"pagination\":{\"search_from\":0,\"search_to\":100}}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "filters": [ { "field": "name", "operator": "eq", "value": "string" }
], "sort": { "field": "name", "keyword": "asc" }, "pagination": { "search_from": 0, "search_to": 100 } } }); const
xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/get_standards"); xhr.setRequestHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE");
xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/get_standards")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"filters\":[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"pagination\":{\"search_from\":0,\"search_to\":100}}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": [ "filters": [ [ "field": "name", "operator": "eq", "value": "string" ] ], "sort": [
"field": "name", "keyword": "asc" ], "pagination": [ "search_from": 0, "search_to": 100 ] ]] as [String : Any] let
postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url:
NSURL(string: "https://api-yourfqdn/public_api/v1/compliance/get_standards")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/get_standards", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"pagination\":{\"search_from\":0,\"search_to\":100}}}", CURLOPT_HTTPHEADER
=> [ "content-type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/get_standards"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"pagination\":{\"search_from\":0,\"search_to\":100}}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/get_standards"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-type",
"application/json"); request.AddParameter("application/json", "{\"request_data\":{\"filters\":
[{\"field\":\"name\",\"operator\":\"eq\",\"value\":\"string\"}],\"sort\":
{\"field\":\"name\",\"keyword\":\"asc\"},\"pagination\":{\"search_from\":0,\"search_to\":100}}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

application/json
▼ request_dataobject
▶ filtersarray

Array of filter conditions

▶ sortobject
▶ paginationobject
REQUEST basic_request

```
{ "request_data": { "pagination": { "search_from": 0, "search_to": 49 } } }
{ "request_data": { "filters": [ { "field": "is_custom", "operator": "in", "value": [ "yes" ] }, { "field": "labels",
"operator": "contains", "value": "aws" } ], "sort": { "field": "insertion_time", "keyword": "desc" }, "pagination": {
"search_from": 0, "search_to": 24 } } }
{ "request_data": { "filters": [ { "field": "labels", "operator": "contains", "value": "security" }, { "field":
"is_custom", "operator": "in", "value": [ "yes" ] } ] } }
{ "request_data": { "filters": [ { "field": "insertion_time", "operator": "gte", "value": 1640995200000 }, { "field":
"insertion_time", "operator": "lte", "value": 1672531200000 } ], "sort": { "field": "insertion_time", "keyword":
"desc" } } }
```

Responses

Successfully retrieved standards

Body
application/json
▼ total_countinteger

Total number of standards in the system

▼ result_countinteger

Number of standards matching the filter criteria + paging

▼ standardsarray
[
▶ idstring

Standard ID

▶ namestring

Standard name

▶ descriptionstring

Description of the standard

▶ publisherstring

Standard publisher/organization

▶ labelsarray[string]

List of labels for categorization

▶ insertion_timeintegerint64

Creation timestamp (epoch)

▶ created_bystring

User who created the standard

▶ is_customboolean

Whether this is a custom standard

▶ controls_countinteger

Number of associated controls

▶ controls_idsarray[string]

List of associated control IDs

]
RESPONSE success_response

```
{ "reply": { "total_count": 17, "result_count": 2, "standards": [ { "id": "394fc8fc210645f6af8bb4267321482a", "name":
"standard name", "description": "convert ...", "version": "", "assessments_profiles_count": 0, "controls_ids": [
```

"0b0b5304d06d44ffb3d7465855378185", "98497e7bf21f413b9df0d84ccb8857d0"], "labels": ["aws", "azure", "gcp"], "revision": -6043636965775741000, "publisher": null, "release_date": "2025-06-18", "created_date": "2025-06-18", "created_by": "Palo Alto Networks", "insert_ts": 1750247438000, "modify_ts": 1750247439000, "is_custom": false }, { "id": "36ed307155e446938f157e3ed214fd72", "name": "standard name ...", "description": "", "version": "", "assessments_profiles_count": 0, "controls_ids": ["0b0b5304d06d44ffb3d7465855378185", "98497e7bf21f413b9df0d84ccb8857d0"], "labels": ["aws"], "revision": 8279863739578999000, "publisher": null, "release_date": "2025-05-18", "created_date": "2025-05-18", "created_by": "Palo Alto Networks", "insert_ts": 1750247311000, "modify_ts": 1750247313000, "is_custom": false }] } }

Bad request - invalid input parameters

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory, is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq, contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra": {} } }

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

2.8.9.2 | Get single standard by ID

post /public_api/v1/compliance/get_standard

Retrieve details of the compliance standard specified by its ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
```

```
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/get_standard'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"id\":\"string\"}}" headers = { 'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/compliance/get_standard", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/get_standard") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"id\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "id": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/get_standard"); xhr.setRequestHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE");
xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/get_standard")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"id\":\"string\"}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": ["id": "string"]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/compliance/get_standard")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/get_standard", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"id\":\"string\"}}", CURLOPT_HTTPHEADER => [ "content-type:
application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/get_standard"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":{\"id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/get_standard"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-type",
"application/json"); request.AddParameter("application/json", "{\"request_data\":{\"id\":\"string\"}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ idstring

Compliance standard ID

REQUEST get_single_standard

```
{ "request_data": { "id": "eee175cb04da4a0e3091b1ee77368" } }
```

Responses

Successfully retrieved standard

Body

application/json

▼ standardobject

▶ idstring

Standard ID

▶ namestring

Standard name

▶ descriptionstring

Description of the standard

▶ publisherstring

Standard publisher/organization

▶ labelsarray[string]

List of labels for categorization

▶ insertion_timeintegerint64

Creation timestamp (epoch)

▶ created_bystring

User who created the standard

▶ is_customboolean

Whether this is a custom standard

▶ controls_countinteger

Number of associated controls

▶ controls_idsarray[string]

List of associated control IDs

RESPONSE single_standard


```
{ "standard": { "id": "36ed307155e446938f157e3ed214fd72", "name": "standard name ...", "description": "", "version":
"", "assessments_profiles_count": 0, "controls_ids": [ "0b0b5304d06d44ffb3d7465855378185",
"98497e7bf21f413b9df0d84ccb8857d0" ], "labels": [ "aws", "azure", "gcp" ], "revision": 8279863739578999000,
"publisher": null, "release_date": "2025-06-18", "created_date": "2025-06-18", "created_by": "Palo Alto Networks",
"insert_ts": 1750247411000, "modify_ts": 1750247413000, "is_custom": true } }
```


Bad request - invalid input parameters

Body

application/json

▼ replyobject

▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field


```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory,
is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq,
contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra":
{} } }
```


Resource not found

Body

application/json

▼ replyobject

▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.9.3 | Add new standard

post /public_api/v1/compliance/add_standard

Create a new compliance standard.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/add_standard'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"standard_name\":\"string\",\"description\":\"string\",\"labels\":[\"string\"],\"controls_ids\":[\"string\"]}}"
headers = { 'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/compliance/add_standard", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/add_standard") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"standard_name\":\"string\",\"description\":\"string\",\"labels\":[\"string\"],\"controls_ids\":[\"string\"]}}"
response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "standard_name": "string", "description": "string", "labels": [
"string" ], "controls_ids": [ "string" ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/compliance/add_standard");
xhr.setRequestHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/add_standard")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"standard_name\":\"string\",\"description\":\"string\",\"labels\":[\"string\"],\"controls_ids\":[\"string\"]}}")
.asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": [ "standard_name": "string", "description": "string", "labels": ["string"],
"controls_ids": ["string"] ]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/compliance/add_standard")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
```

```
10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/add_standard", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"standard_name\":\"string\",\"description\":\"string\",\"labels\":[\"string\"],\"controls_ids\":[\"string\"]}}",
CURLOPT_HTTPHEADER => [ "content-type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/add_standard"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":{\"standard_name\":\"string\",\"description\":\"string\",\"labels\":
[\"string\"],\"controls_ids\":[\"string\"]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/add_standard"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-type",
"application/json"); request.AddParameter("application/json", "{\"request_data\":
{\"standard_name\":\"string\",\"description\":\"string\",\"labels\":[\"string\"],\"controls_ids\":[\"string\"]}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ standard_namestring

Name of the compliance standard

▶ descriptionstring

Description of the standard

▶ labelsarray[string]

List of labels for categorization

▶ controls_idsarray[string]

List of control IDs to associate with this standard. Only custom controls can be included.

REQUEST Add new standard

```
{ "standard_name": "Standard name", "description": "Standard description....", "labels": [ "aws", "azure", "gcp",
"oci" ], "controls_ids": [ "48f2f6a9fde049479e9c8c8eda0be163" ] }
```

Responses

Standard added successfully

Body

application/json

▼ successboolean

Operation success status

RESPONSE success

```
{ "reply": { "success": true } }
```

Bad request - invalid input parameters

Body

application/json

▼ replyobject

▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field

```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory,
is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq,
contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra":
{} } }
```


Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE

```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.9.4 | Edit existing standard

post /public_api/v1/compliance/edit_standard

Edit an existing compliance standard specified by its ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/edit_standard'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"id\":\"string\",\"standard_name\":\"string\",\"description\":\"string\",\"labels\":[\"string\"],\"controls_ids\":
[\"string\"]}}" headers = { 'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/compliance/edit_standard", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/edit_standard") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"id\":\"string\",\"standard_name\":\"string\",\"description\":\"string\",\"labels\":[\"string\"],\"controls_ids\":
[\"string\"]}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "id": "string", "standard_name": "string", "description": "string",
"labels": [ "string" ], "controls_ids": [ "string" ] } }); const xhr = new XMLHttpRequest(); xhr.withCredentials =
true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/edit_standard"); xhr.setRequestHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE");
xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/edit_standard")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
```

{\"id\":\"string\",\"standard_name\":\"string\",\"description\":\"string\",\"labels\":[\"string\"],\"controls_ids\":
[\"string\"]}}") .asString();
import Foundation let headers = ["x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json"] let
parameters = ["request_data": ["id": "string", "standard_name": "string", "description": "string", "labels":
["string"], "controls_ids": ["string"]]] as [String : Any] let postData = JSONSerialization.data(withJSONObject:
parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/compliance/edit_standard")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval:
10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/edit_standard", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"id\":\"string\",\"standard_name\":\"string\",\"description\":\"string\",\"labels\":[\"string\"],\"controls_ids\":
[\"string\"]}}", CURLOPT_HTTPHEADER => ["content-type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE"],]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/edit_standard"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":
{\"id\":\"string\",\"standard_name\":\"string\",\"description\":\"string\",\"labels\":[\"string\"],\"controls_ids\":
[\"string\"]}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/edit_standard"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-type",
"application/json"); request.AddParameter("application/json", "{\"request_data\":
{\"id\":\"string\",\"standard_name\":\"string\",\"description\":\"string\",\"labels\":[\"string\"],\"controls_ids\":
[\"string\"]}}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ idstring

Compliance standard ID

▶ standard_namestring

Name of the standard

▶ descriptionstring

Description of the standard

▶ labelsarray[string]

List of labels for categorization

▶ controls_idsarray[string]

List of control IDs to associate with this standard. Only custom controls can be included.

REQUEST edit_existing_standard

{ "request_data": { "id": "48e2f6a9fdc049479e9c6a8eda0bd163", "standard_name": "test new name" } }

Responses

Standard updated successfully

Body

application/json

▼ successboolean

Operation success status

RESPONSE success

{ "reply": { "success": true } }

Bad request - invalid input parameters

Body

application/json

▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field


```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory,
is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq,
contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra":
{} } }
```


Resource not found

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE


```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```


Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE


```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.9.5 | Delete standard

post /public_api/v1/compliance/delete_standard

Delete a the compliance standard specified by its ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/delete_standard'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"id\":\"string\"}}" headers = { 'x-xdr-auth-id': "REPLACE_KEY_VALUE", 'content-type': "application/json" }
conn.request("POST", "/public_api/v1/compliance/delete_standard", payload, headers) res = conn.getresponse() data =
res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/delete_standard") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"id\":\"string\"}}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "id": "string" } }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/delete_standard"); xhr.setRequestHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE");
xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/delete_standard")
.header("x-xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"id\":\"string\"}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": ["id": "string"]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/compliance/delete_standard")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/delete_standard", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":{\"id\":\"string\"}}", CURLOPT_HTTPHEADER => [ "content-type:
application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/delete_standard"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers,
"content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd,
CURLOPT_POSTFIELDS, "{\"request_data\":{\"id\":\"string\"}}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/delete_standard"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-type",
"application/json"); request.AddParameter("application/json", "{\"request_data\":{\"id\":\"string\"}}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ idstring

Compliance standard ID

REQUEST delete_standard

```
{ "request_data": { "id": "f9f764717b284e9483f9c1210ed3149d" } }
```

Responses

Standard deleted successfully

Body

application/json

▼ successboolean

Operation success status

RESPONSE success


```
{ "reply": { "success": true } }
```


Bad request - invalid input parameters

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field


```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory,
is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq,
contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra":
{} } }
```


Resource not found

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE


```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```


Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE


```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```

2.8.10 | Compliance Assets

Operations for retrieving compliance assets

2.8.10.1 | Get compliance assets

post /public_api/v1/compliance/get_assets

Retrieve compliance assets with required assessment profile revision and last evaluation time, with optional filtering, sorting and pagination.

Filtering Support:

- String fields (`asset_name`, `rule_name`, `control`): `eq`, `neq`, `contains`, `not_contains`
- Status fields (`asset_type`, `status`, `severity`, `rule_type`): `eq`, `neq`
- Status valid values: [`FAILED`, `PASSED`, `NOT_ASSESSED`]
- Severity valid values: [`low`, `medium`, `high`, `critical`]

Sorting Support:

- Available fields: `asset_name`, `asset_type`, `status`, `rule_name`, `control`, `severity`
- Sort orders: `asc`, `desc`
- Default: `asset_name` in ascending order

Pagination: (max number of elements is 100)

Authentication: XDRAuthToken Api Key "Authorization"

Authentication: XDRAuth Api Key "x-xdr-auth-id"
CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H "Authorization: [[apiKey]]" \
-H "x-xdr-auth-id: [[apiKey]]" \
-H 'Accept: application/json'
-H 'Content-Type: application/json'
'https://api-yourfqdn/public_api/v1/compliance/get_assets'
-d '{ "request_data" : { "search_from" : 0, "assessment_profile_revision" : "343554605578776831",
"last_evaluation_time" : 1758816221000, "filters" : [ { "field" : "SEVERITY", "value" : "high", "operator" : "eq" },
{ "field" : "SEVERITY", "value" : "high", "operator" : "eq" } ], "sort" : { "field" : "asset_name", "keyword" : "asc"
}, "search_to" : 1 } }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"request_data\":
{\"assessment_profile_revision\":\"343554605578776831\",\"last_evaluation_time\":1758816221000,\"filters\":
[{\"field\":\"severity\",\"operator\":\"eq\",\"value\":\"low\"}],\"sort\":
{\"field\":\"asset_name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":100}}" headers = { 'x-xdr-auth-id':
"REPLACE_KEY_VALUE", 'content-type': "application/json" } conn.request("POST",
"/public_api/v1/compliance/get_assets", payload, headers) res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/compliance/get_assets") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["x-xdr-auth-id"] =
'REPLACE_KEY_VALUE' request["content-type"] = 'application/json' request.body = "{\"request_data\":
{\"assessment_profile_revision\":\"343554605578776831\",\"last_evaluation_time\":1758816221000,\"filters\":
[{\"field\":\"severity\",\"operator\":\"eq\",\"value\":\"low\"}],\"sort\":
{\"field\":\"asset_name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":100}}" response =
http.request(request) puts response.read_body
const data = JSON.stringify({ "request_data": { "assessment_profile_revision": "343554605578776831",
"last_evaluation_time": 1758816221000, "filters": [ { "field": "severity", "operator": "eq", "value": "low" } ],
"sort": { "field": "asset_name", "keyword": "asc" }, "search_from": 0, "search_to": 100 } }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/compliance/get_assets"); xhr.setRequestHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE");
xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/compliance/get_assets") .header("x-
xdr-auth-id", "REPLACE_KEY_VALUE") .header("content-type", "application/json") .body("{\"request_data\":
{\"assessment_profile_revision\":\"343554605578776831\",\"last_evaluation_time\":1758816221000,\"filters\":
[{\"field\":\"severity\",\"operator\":\"eq\",\"value\":\"low\"}],\"sort\":
{\"field\":\"asset_name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":100}}") .asString();
import Foundation let headers = [ "x-xdr-auth-id": "REPLACE_KEY_VALUE", "content-type": "application/json" ] let
parameters = ["request_data": [ "assessment_profile_revision": "343554605578776831", "last_evaluation_time":
1758816221000, "filters": [ [ "field": "severity", "operator": "eq", "value": "low" ] ], "sort": [ "field":
"asset_name", "keyword": "asc" ], "search_from": 0, "search_to": 100 ]] as [String : Any] let postData =
JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string:
"https://api-yourfqdn/public_api/v1/compliance/get_assets")! as URL, cachePolicy: .useProtocolCachePolicy,
```

```
timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData
as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/compliance/get_assets", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "POST", CURLOPT_POSTFIELDS => "{\"request_data\":
{\"assessment_profile_revision\":\"343554605578776831\",\"last_evaluation_time\":1758816221000,\"filters\":
[{\"field\":\"severity\",\"operator\":\"eq\",\"value\":\"low\"}],\"sort\":
{\"field\":\"asset_name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":100}}", CURLOPT_HTTPHEADER => [
"content-type: application/json", "x-xdr-auth-id: REPLACE_KEY_VALUE" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/compliance/get_assets"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "x-xdr-auth-id: REPLACE_KEY_VALUE"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"request_data\":
{\"assessment_profile_revision\":\"343554605578776831\",\"last_evaluation_time\":1758816221000,\"filters\":
[{\"field\":\"severity\",\"operator\":\"eq\",\"value\":\"low\"}],\"sort\":
{\"field\":\"asset_name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":100}}"); CURLcode ret =
curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/compliance/get_assets"); var request = new
RestRequest(Method.POST); request.AddHeader("x-xdr-auth-id", "REPLACE_KEY_VALUE"); request.AddHeader("content-type",
"application/json"); request.AddParameter("application/json", "{\"request_data\":
{\"assessment_profile_revision\":\"343554605578776831\",\"last_evaluation_time\":1758816221000,\"filters\":
[{\"field\":\"severity\",\"operator\":\"eq\",\"value\":\"low\"}],\"sort\":
{\"field\":\"asset_name\",\"keyword\":\"asc\"},\"search_from\":0,\"search_to\":100}}", ParameterType.RequestBody);
IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ request_dataobjectrequired

▶ assessment_profile_revisionstring

Assessment profile revision identifier

▶ last_evaluation_timeintegerint64

Last evaluation timestamp in epoch format

▶ filtersarray

Array of filter objects for filtering assets

▶ sortobject

▶ search_frominteger

Starting index for pagination

▶ search_tointeger

Ending index for pagination

REQUEST

```
{ "request_data": { "assessment_profile_revision": "343554605578776831", "last_evaluation_time": 1758816221000,
"filters": [ { "field": "SEVERITY", "operator": "eq", "value": "high" } ], "sort": { "field": "asset_name",
"keyword": "asc" }, "search_from": 0, "search_to": 0 } }
```

Responses

Successfully retrieved compliance assets

Body

application/json

▼ replyobject

▶ total_countinteger

Total number of assets available

▶ result_countinteger

Number of assets matching the applied filters + paging

▶ filter_countinteger

Number of assets matching the applied filters

▶ assetsarray

Array of compliance assets

RESPONSE


```
{ "reply": { "total_count": 134, "result_count": 29, "filter_count": 29, "assets": [ { "ASSET_ID":
"a4d9b7b4e7c88cecaf24a063d86cb0069ed659f8bc66c4c1572c90e716291e3d", "ASSET_NAME": "mock-gcloud-storage-buckets-list-
3d0a69ff21184406aa1cbd07b29f6f1f", "ASSET_TYPE": "GOOGLE_CLOUD_STORAGE_BUCKET", "STATUS": "PASSED", "RULE_NAME":
"Sensitive public object in private storage", "CONTROL": "Safeguard 3.2 : Establish and Maintain a Data Inventory",
"RULE_REVISION_ID": "DSPM_1006", "CONTROL_REVISION_ID": -7177800670050593000, "SOURCE": "No Finding or Issue",
"SEVERITY": "high", "RULE_TYPE": "Data", "PROVIDER": "GCP", "REGION": "us", "REALM": "gcp-c0844a50-dd0f-47ec-9d14-
c7af5f5325aa", "TAGS": { "Name": "mock-gcloud-storage-buckets-list-3d0a69ff21184406aa1cbd07b29f6f1f" },
"ORGANIZATION": null, "ASSET_STRONG_ID": "mock-gcloud-storage-buckets-list-3d0a69ff21184406aa1cbd07b29f6f1f" } ] } }
```


Bad request - invalid input parameters

Body

application/json

▼ replyobject

▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE invalid_field


```
{ "reply": { "err_code": 400, "err_msg": "Invalid filter field. Allowed fields: name, category, subcategory,
is_custom, creation_time, created_by, modified_by, modification_time, id, description", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid operator 'range' for field 'name'. Allowed operators: eq, neq,
contains, not_contains", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Missing required parameter: control_name", "err_extra": {} } }
{ "reply": { "err_code": 400, "err_msg": "Invalid value for is_custom field. Allowed values: yes, no", "err_extra":
{} } }
```


Authentication required

Body

application/json

▼ replyobjectrequired

▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong

▶ err_extraobject

Additional error details

RESPONSE


```
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }
```


Access denied - insufficient permissions

Body

application/json

▼ replyobjectrequired

▶ err_codeinteger

Error code

▶ err_msgstring

Error message describing what went wrong

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

Internal server error

Body
application/json
▼ replyobject
▶ err_codeinteger

Error code

▶ err_msgstring

Error message

▶ err_extraobject

Additional error details

RESPONSE
{ "reply": { "err_code": 0, "err_msg": "example", "err_extra": {} } }

2.9 | Cloud Workload Protection APIs

2.9.1 | Preface

You can export the OpenAPI specs in the Cortex Stoplight instance.

Cloud Workload Policies and Rules ensure security compliance, prevent misconfigurations, and mitigate risks across cloud environments. Cloud Workload Policies leverage actionable findings or enforce preventive measures at defined stages of the Software Development Life Cycle (SDLC).

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Version: Cortex Cloud 1.3

2.9.1.1 | Servers

https://api-yourfqdn

2.9.1.2 | Authentication methods

▼ Authorization
Api Key "x-api-key"

Authorized requests to the API should use: **x-api-key** inside **a header** with the access token.

▼ x-xdr-auth-id
Api Key "x-api-key-id"

Authorized requests to the API should use: **x-api-key-id** inside **a header** with the access token.

2.9.2 | SBOM

APIs for managing SBOMs

2.9.2.1 | Get the SBOM of the specified asset

get /public_api/v1/assets/{assetID}/sbom

Get an SBOM (Software Bill of Materials) report for the specified asset. You can specify the format and version of the SBOM, as well as which attribute you would like included.

Required license: Cortex Cloud Runtime Security. In Cortex Cloud Posture Management, requires the Cortex Cloud Runtime Security add-on.

Path parameters
▼ assetID String required

ID of the asset of which you want to export the SBOM

Example: `assetID_example`

Request headers

▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241

Query parameters

▼ format String

The format in which the SBOM is to be exported.

Allowed values: XML JSON

Example: format_example

▼ output_format String

The version of the SBOM format to be used.

Allowed values: CycloneDX 1.4 SPDX

Example: outputFormat_example

▼ attributes String

The list of attributes to be included in the SBOM. If not specified, all fields are included except Type and Name, which are always included.

Allowed values: PURL Author Version License Source Package Binaries

Example: attributes_example

CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/xml,application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/assets/{assetID}/sbom?
format=format_example&output_format=outputFormat_example&attributes=attributes_example'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/v1/assets/%7BassetID%7D/sbom?
format=SOME_STRING_VALUE&output_format=SOME_STRING_VALUE&attributes=SOME_STRING_VALUE", headers=headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/assets/%7BassetID%7D/sbom?
format=SOME_STRING_VALUE&output_format=SOME_STRING_VALUE&attributes=SOME_STRING_VALUE") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/v1/assets/%7BassetID%7D/sbom?
format=SOME_STRING_VALUE&output_format=SOME_STRING_VALUE&attributes=SOME_STRING_VALUE");
xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/v1/assets/%7BassetID%7D/sbom?
format=SOME_STRING_VALUE&output_format=SOME_STRING_VALUE&attributes=SOME_STRING_VALUE") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/assets/%7BassetID%7D/sbom?
format=SOME_STRING_VALUE&output_format=SOME_STRING_VALUE&attributes=SOME_STRING_VALUE")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let
session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data,
response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
```

```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/assets/%7BassetID%7D/sbom?
format=SOME_STRING_VALUE&output_format=SOME_STRING_VALUE&attributes=SOME_STRING_VALUE", CURLOPT_RETURNTRANSFER =>
true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/v1/assets/%7BassetID%7D/sbom?
format=SOME_STRING_VALUE&output_format=SOME_STRING_VALUE&attributes=SOME_STRING_VALUE"); struct curl_slist *headers =
NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/assets/%7BassetID%7D/sbom?
format=SOME_STRING_VALUE&output_format=SOME_STRING_VALUE&attributes=SOME_STRING_VALUE"); var request = new
RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

OK. The request was successful, and the SBOM data is returned in the specified format.

Body
application/xml
▼ stringbinary
RESPONSE
```
<root>[binary file]</root>
```

Bad Request. The request was malformed, missing the required parameters, or the Asset ID provided does not support SBOM Export.

Body
application/json
▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "example", "err_extra": "example" }
```

The API key provided is invalid or missing.

Body
application/json
▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

▼ err_extrastring

Additional information describing the error.

RESPONSE
```
{ "err_code": "example", "err_msg": "example", "err_extra": "example" }
```

Forbidden. The API key does not have the necessary permissions to access the requested resource.

Body
application/json
▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

▼ err_extrastring

Additional information describing the error.
RESPONSE
{ "err_code": "example", "err_msg": "example", "err_extra": "example" }

The specified asset was not found.

Body
application/json
▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "example", "err_extra": "example" }

An error occurred on the server while processing the request.

Body
application/json
▼ err_codestring

HTTP response code.

▼ err_msgstring

Error message.

▼ err_extrastring

Additional information describing the error.

RESPONSE
{ "err_code": "example", "err_msg": "example", "err_extra": "example" }

2.9.3 | Policies

APIs for managing Cloud Workload Policies

2.9.3.1 | Get CWP policies

get /public_api/v1/cwp/policies

Cloud Workload Policies help you prevent and manage security violations in your cloud runtime instances. They enable you to apply detection logic to specific asset groups at the desired SDLC stage, and define what action needs to be taken if the conditions are met.

Get all CWP policies of the given type. Default behavior is all.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
Query parameters

▼ types array[PolicyType]

The policy type

CLIENT REQUEST Bash+curl

```
curl -X 'GET'
-H 'Accept: application/json,application/json; charset=UTF-8'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cwp/policies?types='
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/v1/cwp/policies?types=SOME_ARRAY_VALUE",
headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/cwp/policies?
types=SOME_ARRAY_VALUE") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/v1/cwp/policies?
types=SOME_ARRAY_VALUE"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/v1/cwp/policies?types=SOME_ARRAY_VALUE")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/cwp/policies?types=SOME_ARRAY_VALUE")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "GET" request.allHTTPHeaderFields = headers let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cwp/policies?types=SOME_ARRAY_VALUE", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/v1/cwp/policies?types=SOME_ARRAY_VALUE"); struct curl_slist *headers = NULL; headers
= curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cwp/policies?types=SOME_ARRAY_VALUE"); var request =
new RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Success

Body

application/json

The response you get when calling the `get policies` path

[

▶ idstring

▶ revisioninteger

▶ createdAtstringdate-time

▶ modifiedAtstringdate-time

▶ typestring (Enum)

One of the supported policy types
▶ createdBystring
▶ disabledboolean
▶ namestring
▶ descriptionstring
▶ evaluationModesarray[string]
▶ evaluationStagestring (Enum)

One of the supported evaluation stages

▶ rulesIdsarray[string]

The UUIDs of the rules that define the condition The `rulesIds` field is only required for non-compliance policies.

▶ conditionstringbyte

The `condition` field is only required for non-compliance policies.

▶ exceptionstringbyte
▶ assetScopestringbyte
▶ assetGroupsIDsarray[integer]
▶ assetGroupsarray[string]
▶ actionstring (Enum)

One of the supported policy actions

▶ severitystring (Enum)

One of the supported policy severities

▶ remediationGuidancestring
]
RESPONSE

```
[ { "id": "1", "revision": 3, "createdAt": "2024-07-28T18:50Z", "modifiedAt": "2024-07-28T18:50Z", "type": "COMPLIANCE", "createdBy": "admin", "disabled": false, "name": "some policy", "description": "some policy description", "evaluationModes": [ "periodic" ], "evaluationStage": "CI", "rulesIds": [ "1", "2" ], "condition": "<some condition in blob form>", "exception": "<some exception in blob form>", "assetScope": "<some asset scope in blob form>", "assetGroupsIDs": [ 1, 2 ], "assetGroups": [ "group1", "group2" ], "action": "ISSUE", "severity": "CRITICAL", "remediationGuidance": "some remediation guidance" } ]
```


Client error

Body
application/json
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

A short, programmatically safe string indicating the error code reported

RESPONSE default

```
{ "err_msg": "Bad Request", "metadata": { "code": "BAD_REQUEST" } }
```


Service had unexpected internal error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

A short, programmatically safe string indicating the error code reported

RESPONSE default

```
[application/json; charset=UTF-8 content]
```

2.9.3.2 | Add CWP policies

post /public_api/v1/cwp/policies

Add CWP policy instances.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cwp/policies'
-d '{ "id" : "1", "revision" : 3, "createdAt" : "2024-07-28T18:50Z", "modifiedAt" : "2024-07-28T18:50Z", "type" :
"COMPLIANCE", "createdBy" : "admin", "disabled" : false, "name" : "some policy", "description" : "some policy
description", "evaluationModes" : [ "PERIODIC" ], "evaluationStage" : "CI", "rulesIds" : [ ], "condition" : "<some
condition in blob form>", "exception" : "<some exception in blob form>", "assetScope" : "<some asset scope in blob
form>", "assetGroupsIDs" : [ 1, 2 ], "assetGroups" : [ "group1", "group2" ], "action" : "ISSUE", "severity" :
"CRITICAL", "remediationGuidance" : "some remediation guidance" }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "
{\"id\":\"1\",\"revision\":3,\"createdAt\":\"2024-07-28T18:50Z\",\"modifiedAt\":\"2024-07-
28T18:50Z\",\"type\":\"COMPLIANCE\",\"createdBy\":\"admin\",\"disabled\":false,\"name\":\"some
policy\",\"description\":\"some policy description\",\"evaluationModes\":
[\"PERIODIC\"],\"evaluationStage\":\"CI\",\"rulesIds\":[],\"condition\":\"<some condition in blob
form>\",\"exception\":\"<some exception in blob form>\",\"assetScope\":\"<some asset scope in blob
form>\",\"assetGroupsIDs\":[1,2],\"assetGroups\":
[\"group1\",\"group2\"],\"action\":\"ISSUE\",\"severity\":\"CRITICAL\",\"remediationGuidance\":\"some remediation
guidance\"}" headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/cwp/policies",
payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/cwp/policies") http
= Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "
{\"id\":\"1\",\"revision\":3,\"createdAt\":\"2024-07-28T18:50Z\",\"modifiedAt\":\"2024-07-
28T18:50Z\",\"type\":\"COMPLIANCE\",\"createdBy\":\"admin\",\"disabled\":false,\"name\":\"some
policy\",\"description\":\"some policy description\",\"evaluationModes\":
[\"PERIODIC\"],\"evaluationStage\":\"CI\",\"rulesIds\":[],\"condition\":\"<some condition in blob
form>\",\"exception\":\"<some exception in blob form>\",\"assetScope\":\"<some asset scope in blob
form>\",\"assetGroupsIDs\":[1,2],\"assetGroups\":
[\"group1\",\"group2\"],\"action\":\"ISSUE\",\"severity\":\"CRITICAL\",\"remediationGuidance\":\"some remediation
guidance\"}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "id": "1", "revision": 3, "createdAt": "2024-07-28T18:50Z", "modifiedAt": "2024-07-
28T18:50Z", "type": "COMPLIANCE", "createdBy": "admin", "disabled": false, "name": "some policy", "description":
"some policy description", "evaluationModes": [ "PERIODIC" ], "evaluationStage": "CI", "rulesIds": [], "condition": "
<some condition in blob form>", "exception": "<some exception in blob form>", "assetScope": "<some asset scope in
blob form>", "assetGroupsIDs": [ 1, 2 ], "assetGroups": [ "group1", "group2" ], "action": "ISSUE", "severity":
"CRITICAL", "remediationGuidance": "some remediation guidance" }); const xhr = new XMLHttpRequest();
xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE)
{ console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/cwp/policies");
xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
```

```
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/cwp/policies")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("
{\"id\":\"1\",\"revision\":3,\"createdAt\":\"2024-07-28T18:50Z\",\"modifiedAt\":\"2024-07-
28T18:50Z\",\"type\":\"COMPLIANCE\",\"createdBy\":\"admin\",\"disabled\":false,\"name\":\"some
policy\",\"description\":\"some policy description\",\"evaluationModes\":
[\"PERIODIC\"],\"evaluationStage\":\"CI\",\"rulesIds\":[],\"condition\":\"<some condition in blob
form>\",\"exception\":\"<some exception in blob form>\",\"assetScope\":\"<some asset scope in blob
form>\",\"assetGroupsIDs\":[1,2],\"assetGroups\":
[\"group1\",\"group2\"],\"action\":\"ISSUE\",\"severity\":\"CRITICAL\",\"remediationGuidance\":\"some remediation
guidance\"}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = [ "id": "1", "revision": 3,
"createdAt": "2024-07-28T18:50Z", "modifiedAt": "2024-07-28T18:50Z", "type": "COMPLIANCE", "createdBy": "admin",
"disabled": false, "name": "some policy", "description": "some policy description", "evaluationModes": ["PERIODIC"],
"evaluationStage": "CI", "rulesIds": [], "condition": "<some condition in blob form>", "exception": "<some exception
in blob form>", "assetScope": "<some asset scope in blob form>", "assetGroupsIDs": [1, 2], "assetGroups": ["group1",
"group2"], "action": "ISSUE", "severity": "CRITICAL", "remediationGuidance": "some remediation guidance" ] as [String
: Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request =
NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/cwp/policies")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers
request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cwp/policies", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"id\":\"1\",\"revision\":3,\"createdAt\":\"2024-07-28T18:50Z\",\"modifiedAt\":\"2024-07-
28T18:50Z\",\"type\":\"COMPLIANCE\",\"createdBy\":\"admin\",\"disabled\":false,\"name\":\"some
policy\",\"description\":\"some policy description\",\"evaluationModes\":
[\"PERIODIC\"],\"evaluationStage\":\"CI\",\"rulesIds\":[],\"condition\":\"<some condition in blob
form>\",\"exception\":\"<some exception in blob form>\",\"assetScope\":\"<some asset scope in blob
form>\",\"assetGroupsIDs\":[1,2],\"assetGroups\":
[\"group1\",\"group2\"],\"action\":\"ISSUE\",\"severity\":\"CRITICAL\",\"remediationGuidance\":\"some remediation
guidance\"}", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cwp/policies"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"id\":\"1\",\"revision\":3,\"createdAt\":\"2024-07-28T18:50Z\",\"modifiedAt\":\"2024-07-
28T18:50Z\",\"type\":\"COMPLIANCE\",\"createdBy\":\"admin\",\"disabled\":false,\"name\":\"some
policy\",\"description\":\"some policy description\",\"evaluationModes\":
[\"PERIODIC\"],\"evaluationStage\":\"CI\",\"rulesIds\":[],\"condition\":\"<some condition in blob
form>\",\"exception\":\"<some exception in blob form>\",\"assetScope\":\"<some asset scope in blob
form>\",\"assetGroupsIDs\":[1,2],\"assetGroups\":
[\"group1\",\"group2\"],\"action\":\"ISSUE\",\"severity\":\"CRITICAL\",\"remediationGuidance\":\"some remediation
guidance\"}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cwp/policies"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"id\":\"1\",\"revision\":3,\"createdAt\":\"2024-07-
28T18:50Z\",\"modifiedAt\":\"2024-07-
28T18:50Z\",\"type\":\"COMPLIANCE\",\"createdBy\":\"admin\",\"disabled\":false,\"name\":\"some
policy\",\"description\":\"some policy description\",\"evaluationModes\":
[\"PERIODIC\"],\"evaluationStage\":\"CI\",\"rulesIds\":[],\"condition\":\"<some condition in blob
form>\",\"exception\":\"<some exception in blob form>\",\"assetScope\":\"<some asset scope in blob
form>\",\"assetGroupsIDs\":[1,2],\"assetGroups\":
[\"group1\",\"group2\"],\"action\":\"ISSUE\",\"severity\":\"CRITICAL\",\"remediationGuidance\":\"some remediation
guidance\"}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required
application/json

The schema of a policy

▼ idstring
▼ revisioninteger
▼ createdAtstringdate-time
▼ modifiedAtstringdate-time
▼ typestring (Enum)

One of the supported policy types

Example:`"MALWARE"`
Allowed values:"COMPLIANCE""MALWARE""SECRET"
▼ createdBystring
▼ disabledboolean
▼ namestring
▼ descriptionstring
▼ evaluationModesarray[string]
▼ evaluationStagestring (Enum)

One of the supported evaluation stages

Example:`"CI"`
Allowed values:"CI""RUNTIME""DEPLOY"null
▼ rulesIdsarray[string]

The UUIDs of the rules that define the condition The `rulesIds` field is only required for non-compliance policies.

▼ conditionstringbyte

The `condition` field is only required for non-compliance policies.

▼ exceptionstringbyte
▼ assetScopestringbyte
▼ assetGroupsIDsarray[integer]
▼ assetGroupsarray[string]
▼ actionstring (Enum)

One of the supported policy actions

Example:`"ISSUE"`
Allowed values:"ISSUE""PREVENT"null
▼ severitystring (Enum)

One of the supported policy severities

Example:`"CRITICAL"`
Allowed values:"LOW""MEDIUM""HIGH""CRITICAL"null
▼ remediationGuidancestring
REQUEST

```
{ "id": "1", "revision": 3, "createdAt": "2024-07-28T18:50Z", "modifiedAt": "2024-07-28T18:50Z", "type": "COMPLIANCE", "createdBy": "admin", "disabled": false, "name": "some policy", "description": "some policy description", "evaluationModes": [ "PERIODIC" ], "evaluationStage": "CI", "rulesIds": [], "condition": "<some condition in blob form>", "exception": "<some exception in blob form>", "assetScope": "<some asset scope in blob form>", "assetGroupsIDs": [ 1, 2 ], "assetGroups": [ "group1", "group2" ], "action": "ISSUE", "severity": "CRITICAL", "remediationGuidance": "some remediation guidance" }
```

Responses

Success

Body
application/json; charset=UTF-8
▼ idstring
RESPONSE

```
[application/json; charset=UTF-8 content]
```

Client error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

A short, programmatically safe string indicating the error code reported

RESPONSE default
[application/json; charset=UTF-8 content]

Service had unexpected internal error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

A short, programmatically safe string indicating the error code reported

RESPONSE default
[application/json; charset=UTF-8 content]

2.9.3.3 | Get a CWP policy by ID

get /public_api/v1/cwp/policies/{id}

Get a CWP policy by its ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ id String required
Example: id_example
Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json,application/json; charset=UTF-8'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cwp/policies/{id}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("GET", "/public_api/v1/cwp/policies/%7Bid%7D", headers=headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D");
xhr.setRequestHeader("Authorization",
```

```
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "GET" request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask =
session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error !=
nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D"); var request = new
RestRequest(Method.GET); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Success

Body
application/json

The schema of a policy

▼ idstring
▼ revisioninteger
▼ createdAtstringdate-time
▼ modifiedAtstringdate-time
▼ typestring (Enum)

One of the supported policy types

Example:`"MALWARE"`
Allowed values:"COMPLIANCE""MALWARE""SECRET"
▼ createdBystring
▼ disabledboolean
▼ namestring
▼ descriptionstring
▼ evaluationModesarray[string]
▼ evaluationStagestring (Enum)

One of the supported evaluation stages

Example:`"CI"`
Allowed values:"CI""RUNTIME""DEPLOY"null
▼ rulesIdsarray[string]

The UUIDs of the rules that define the condition The `rulesIds` field is only required for non-compliance policies.

▼ conditionstringbyte

The `condition` field is only required for non-compliance policies.

▼ exceptionstringbyte
▼ assetScopestringbyte
▼ assetGroupsIDsarray[integer]

▼ assetGroupsarray[string]
▼ actionstring (Enum)

One of the supported policy actions

Example:`"ISSUE"`
Allowed values:"ISSUE""PREVENT"null
▼ severitystring (Enum)

One of the supported policy severities

Example:`"CRITICAL"`
Allowed values:"LOW""MEDIUM""HIGH""CRITICAL"null
▼ remediationGuidancestring
RESPONSE

```
{ "id": "1", "revision": 3, "createdAt": "2024-07-28T18:50Z", "modifiedAt": "2024-07-28T18:50Z", "type": "COMPLIANCE", "createdBy": "admin", "disabled": false, "name": "some policy", "description": "some policy description", "evaluationModes": [ "PERIODIC" ], "evaluationStage": "CI", "rulesIds": [], "condition": "<some condition in blob form>", "exception": "<some exception in blob form>", "assetScope": "<some asset scope in blob form>", "assetGroupsIDs": [ 1, 2 ], "assetGroups": [ "group1", "group2" ], "action": "ISSUE", "severity": "CRITICAL", "remediationGuidance": "some remediation guidance" }
```

Policy ID not found

Body
application/json
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

A short, programmatically safe string indicating the error code reported

RESPONSE default

```
{ "err_msg": "Policy ID not found", "metadata": { "code": "NOT_FOUND" } }
```

Service had unexpected internal error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

A short, programmatically safe string indicating the error code reported

RESPONSE default

```
[application/json; charset=UTF-8 content]
```

2.9.3.4 | Delete a CWP policy by ID

delete /public_api/v1/cwp/policies/{id}

Delete a CWP policy by its ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ id String required

The policy ID to delete

Example: `id_example`
Request headers

▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
Query parameters
▼ closeIssues Boolean

If set, will trigger issue closing of issues opened by the deleted policy

Example: true
CLIENT REQUEST Bash+curl
```
curl -X 'DELETE'
-H 'Accept: application/json; charset=UTF-8'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cwp/policies/{id}?closeIssues=true'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("DELETE", "/public_api/v1/cwp/policies/%7Bid%7D?
closeIssues=SOME_BOOLEAN_VALUE", headers=headers) res = conn.getresponse() data = res.read() print(data.decode("utf-
8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D?closeIssues=SOME_BOOLEAN_VALUE") http = Net::HTTP.new(url.host,
url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Delete.new(url)
request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("DELETE", "https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D?
closeIssues=SOME_BOOLEAN_VALUE"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.delete("https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D?
closeIssues=SOME_BOOLEAN_VALUE") .header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D?closeIssues=SOME_BOOLEAN_VALUE")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "DELETE" request.allHTTPHeaderFields = headers
let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: {
(data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as?
HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D?closeIssues=SOME_BOOLEAN_VALUE", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "DELETE", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "DELETE"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D?closeIssues=SOME_BOOLEAN_VALUE"); struct
curl_slist *headers = NULL; headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D?
closeIssues=SOME_BOOLEAN_VALUE"); var request = new RestRequest(Method.DELETE); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```

Responses

Success

Client error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

A short, programmatically safe string indicating the error code reported

RESPONSE default
`[application/json; charset=UTF-8 content]`

Policy ID not found

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

A short, programmatically safe string indicating the error code reported

RESPONSE default
`[application/json; charset=UTF-8 content]`

Service had unexpected internal error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

A short, programmatically safe string indicating the error code reported

RESPONSE default
`[application/json; charset=UTF-8 content]`

2.9.3.5 | Update a CWP policy by ID

put /public_api/v1/cwp/policies/{id}

Update a CWP policy by its ID.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Path parameters
▼ id String required

The policy ID to delete

Example: `id_example`
Request headers

▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9U
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
```
curl -X 'PUT'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json; charset=UTF-8'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9U
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/cwp/policies/{id}'
-d '{ "id" : "1", "revision" : 3, "createdAt" : "2024-07-28T18:50Z", "modifiedAt" : "2024-07-28T18:50Z", "type" :
"COMPLIANCE", "createdBy" : "admin", "disabled" : false, "name" : "some policy", "description" : "some policy
description", "evaluationModes" : [ "PERIODIC" ], "evaluationStage" : "CI", "rulesIds" : [ ], "condition" : "<some
condition in blob form>", "exception" : "<some exception in blob form>", "assetScope" : "<some asset scope in blob
form>", "assetGroupsIDs" : [ 1, 2 ], "assetGroups" : [ "group1", "group2" ], "action" : "ISSUE", "severity" :
"CRITICAL", "remediationGuidance" : "some remediation guidance" }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241" } conn.request("PUT", "/public_api/v1/cwp/policies/%7Bid%7D", headers=headers) res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Put.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("PUT", "https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D");
xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.send(data);
HttpResponse<String> response = Unirest.put("https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241" ] let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "PUT" request.allHTTPHeaderFields = headers let session = URLSession.shared let dataTask =
session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error !=
nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "PUT", CURLOPT_HTTPHEADER => [ "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9U
", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) {
echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "PUT"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D"); struct curl_slist *headers = NULL; headers =
curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9U
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers);
CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D"); var request = new
RestRequest(Method.PUT); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); IRestResponse response = client.Execute(request);
```
Body parameters

required
application/json; charset=UTF-8

The schema of a policy

▼ idstring
▼ revisioninteger
▼ createdAtstringdate-time
▼ modifiedAtstringdate-time
▼ typestring (Enum)

One of the supported policy types

Example:`"MALWARE"`
Allowed values:"COMPLIANCE""MALWARE""SECRET"
▼ createdBystring
▼ disabledboolean
▼ namestring
▼ descriptionstring
▼ evaluationModesarray[string]
▼ evaluationStagestring (Enum)

One of the supported evaluation stages

Example:`"CI"`
Allowed values:"CI""RUNTIME""DEPLOY"null
▼ rulesIdsarray[string]

The UUIDs of the rules that define the condition The `rulesIds` field is only required for non-compliance policies.

▼ conditionstringbyte

The `condition` field is only required for non-compliance policies.

▼ exceptionstringbyte
▼ assetScopestringbyte
▼ assetGroupsIDsarray[integer]
▼ assetGroupsarray[string]
▼ actionstring (Enum)

One of the supported policy actions

Example:`"ISSUE"`
Allowed values:"ISSUE""PREVENT"null
▼ severitystring (Enum)

One of the supported policy severities

Example:`"CRITICAL"`
Allowed values:"LOW""MEDIUM""HIGH""CRITICAL"null
▼ remediationGuidancestring
REQUEST
`[application/json; charset=UTF-8 content]`
Responses

Success

Client error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

A short, programmatically safe string indicating the error code reported

RESPONSE default
`[application/json; charset=UTF-8 content]`

Policy ID not found

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

A short, programmatically safe string indicating the error code reported

RESPONSE default
```
[application/json; charset=UTF-8 content]
```

Service had unexpected internal error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

A short, programmatically safe string indicating the error code reported

RESPONSE default
```
[application/json; charset=UTF-8 content]
```

2.10 | Data Security Posture Management APIs

2.10.1 | Preface

You can export the OpenAPI specs in the Cortex Stoplight instance.

The Cortex Cloud Data Security solution is an agentless multi-cloud data security platform that discovers, classifies, protects, and governs sensitive data.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Version: Cortex Cloud 1.3

2.10.1.1 | Servers

https://api-yourfqdn

2.10.2 | Inventory

Inventory Management

2.10.2.1 | Get data pattern inventory

post /public_api/v1/data-security/data-patterns

Get all data patterns found in the environment, including high-level aggregations for each of them.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
```
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
```

▼ x-xdr-auth-id String required

{api_key_id}

Example: 241

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: */*'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/data-security/data-patterns'
-d '{ "filter" : "{}", "next_page_token" : "next_page_token", "sort" : [ "sort", "sort" ] }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"filter\":{},\"sort\":
[],\"next_page_token\":\"string\"}" headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/data-
security/data-patterns", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/data-security/data-
patterns") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE
request = Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"filter\":
{},\"sort\":[],\"next_page_token\":\"string\"}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "filter": {}, "sort": [], "next_page_token": "string" }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/data-security/data-patterns"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/data-security/data-patterns")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"filter\":{},\"sort\":
[],\"next_page_token\":\"string\"}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = [ "filter": [], "sort": [],
"next_page_token": "string" ] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/data-
security/data-patterns")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod =
"POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/data-
security/data-patterns", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"filter\":{},\"sort\":[],\"next_page_token\":\"string\"}", CURLOPT_HTTPHEADER => [
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/data-security/data-patterns"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"filter\":{},\"sort\":[],\"next_page_token\":\"string\"}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/data-security/data-patterns"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"filter\":{},\"sort\":[],\"next_page_token\":\"string\"}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required
application/json
▼ filterobject
▼ sortarray
▼ next_page_tokenstring
REQUEST
```
{ "filter": {}, "sort": null, "next_page_token": "example" }
```
Responses

OK

Body
/
▼ dataarray
▼ metadataobject
▶ total_countintegerint32
▶ filter_countintegerint32
▶ next_page_tokenstring
RESPONSE
```
[*/* content]
```

Bad Request

Body
/
▼ err_msgstring
▼ err_extraarray
RESPONSE Example#1
```
[*/* content]
```

Internal Server Error

Body
/
▼ err_msgstring
▼ err_extraarray
RESPONSE Example#1
```
[*/* content]
```

2.10.2.2 | Get field inventory details

post /public_api/v1/data-security/objects/fields

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
```
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
```
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: */*'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/data-security/objects/fields'
-d '{ "filter" : "{}", "next_page_token" : "next_page_token", "sort" : [ "sort", "sort" ] }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"filter\":{},\"sort\":
[],\"next_page_token\":\"string\"}" headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
```

```
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/data-
security/objects/fields", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/data-
security/objects/fields") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"filter\":
{},\"sort\":[],\"next_page_token\":\"string\"}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "filter": {}, "sort": [], "next_page_token": "string" }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/data-security/objects/fields"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/data-security/objects/fields")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"filter\":{},\"sort\":
[],\"next_page_token\":\"string\"}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = [ "filter": [], "sort": [],
"next_page_token": "string" ] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/data-
security/objects/fields")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod =
"POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/data-
security/objects/fields", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"filter\":{},\"sort\":[],\"next_page_token\":\"string\"}", CURLOPT_HTTPHEADER => [
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/data-security/objects/fields"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"filter\":{},\"sort\":[],\"next_page_token\":\"string\"}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/data-security/objects/fields"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"filter\":{},\"sort\":[],\"next_page_token\":\"string\"}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ filterobject

▼ sortarray

▼ next_page_tokenstring

REQUEST

```
{ "filter": {}, "sort": null, "next_page_token": "example" }
```

Responses

OK

Body

/

▼ dataarray

▼ metadataobject

▶ total_countintegerint32

▶ filter_countintegerint32

▶ next_page_tokenstring
RESPONSE
[*/* content]

Bad Request

Body
/
▼ err_msgstring
▼ err_extraarray
RESPONSE Example#1
[*/* content]

Internal Server Error

Body
/
▼ err_msgstring
▼ err_extraarray
RESPONSE Example#1
[*/* content]

2.10.2.3 | Get file inventory details

post /public_api/v1/data-security/objects/files

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Request headers
▼ Authorization String required

{api_key}

Example:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
▼ x-xdr-auth-id String required

{api_key_id}

Example: 241
CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: */*'
-H 'Content-Type: application/json'
-H 'Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
' -H 'x-xdr-auth-id: 241'
'https://api-yourfqdn/public_api/v1/data-security/objects/files'
-d '{ "filter" : "{}", "next_page_token" : "next_page_token", "sort" : [ "sort", "sort" ] }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"filter\":{},\"sort\":
[],\"next_page_token\":\"string\"}" headers = { 'Authorization':
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", 'x-xdr-auth-id': "241", 'content-type': "application/json" } conn.request("POST", "/public_api/v1/data-
security/objects/files", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/data-
security/objects/files") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["Authorization"] =
'UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
' request["x-xdr-auth-id"] = '241' request["content-type"] = 'application/json' request.body = "{\"filter\":
{},\"sort\":[],\"next_page_token\":\"string\"}" response = http.request(request) puts response.read_body
const data = JSON.stringify({ "filter": {}, "sort": [], "next_page_token": "string" }); const xhr = new
XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if
(this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-
yourfqdn/public_api/v1/data-security/objects/files"); xhr.setRequestHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); xhr.setRequestHeader("x-xdr-auth-id", "241"); xhr.setRequestHeader("content-type", "application/json");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/data-security/objects/files")
.header("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
```

```
") .header("x-xdr-auth-id", "241") .header("content-type", "application/json") .body("{\"filter\":{},\"sort\":
[],\"next_page_token\":\"string\"}") .asString();
import Foundation let headers = [ "Authorization":
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
", "x-xdr-auth-id": "241", "content-type": "application/json" ] let parameters = [ "filter": [], "sort": [],
"next_page_token": "string" ] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters,
options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/public_api/v1/data-
security/objects/files")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod =
"POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/public_api/v1/data-
security/objects/files", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST",
CURLOPT_POSTFIELDS => "{\"filter\":{},\"sort\":[],\"next_page_token\":\"string\"}", CURLOPT_HTTPHEADER => [
"Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
", "content-type: application/json", "x-xdr-auth-id: 241" ], ]); $response = curl_exec($curl); $err =
curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/data-security/objects/files"); struct curl_slist *headers = NULL;
headers = curl_slist_append(headers, "Authorization:
UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9W
"); headers = curl_slist_append(headers, "x-xdr-auth-id: 241"); headers = curl_slist_append(headers, "content-type:
application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "
{\"filter\":{},\"sort\":[],\"next_page_token\":\"string\"}"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/data-security/objects/files"); var request = new
RestRequest(Method.POST); request.AddHeader("Authorization",
"UCoWpG4rkNzgCp2dsh8m02iVpZsskwKHz7N1tErPcUV3Wmf59Gc9kytmgOv0pDWoem3PBlORyRIPiir4OcYdWUOWAM3JyTgoCxQf4nQoTlKmFRKz9Bj5vIjluw66p9
"); request.AddHeader("x-xdr-auth-id", "241"); request.AddHeader("content-type", "application/json");
request.AddParameter("application/json", "{\"filter\":{},\"sort\":[],\"next_page_token\":\"string\"}",
ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```

Body parameters

required

application/json

▼ filterobject

▼ sortarray

▼ next_page_tokenstring

REQUEST

```
{ "filter": {}, "sort": null, "next_page_token": "example" }
```

Responses

OK

Body

/

▼ dataarray

▼ metadataobject

▶ total_countintegerint32

▶ filter_countintegerint32

▶ next_page_tokenstring

RESPONSE

```
[*/* content]
```

Bad Request

Body

/

▼ err_msgstring

▼ err_extraarray

RESPONSE Example#1

```
[*/* content]
```

Internal Server Error

Body

/

▼ err_msgstring

▼ err_extraarray

RESPONSE Example#1
```
[*/* content]
```

2.11 | Trusted Images APIs

2.11.1 | Preface

You can export the OpenAPI specs in the Cortex Stoplight instance.

Trusted images is a security control that lets you declare, by policy, which registries, repositories, and images you trust. These policies also help you define the actions you wish to take when untrusted images are initiated in your environment.This microservice provides a CRUD model for managing rules in the Cloud Workload Protection module.

Required license: Cortex Cloud Runtime Security or Cortex Cloud Posture Management

Version: Cortex Cloud 1.3

2.11.1.1 | Servers

https://api-yourfqdn

2.11.2 | Rules Management Service

Manages Cortex Cloud security detection rules

2.11.2.1 | Delete Policies

delete /api/v1/policies

Delete policies/rule instances from the rules management service.

Query parameters
▼ closeIssues Boolean

If set, will trigger issue closing of issues opened by the deleted policies

Example: `true`
CLIENT REQUEST Bash+curl
```
curl -X 'DELETE'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v1/policies?closeIssues=true'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("DELETE", "/api/v1/policies?
closeIssues=null") res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v1/policies?closeIssues=null")
http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Delete.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("DELETE", "https://api-yourfqdn/api/v1/policies?closeIssues=null");
xhr.send(data);
HttpResponse<String> response = Unirest.delete("https://api-yourfqdn/api/v1/policies?closeIssues=null") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/api/v1/policies?
closeIssues=null")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod =
"DELETE" let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest,
completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse =
response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v1/policies?
closeIssues=null", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT
=> 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "DELETE", ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "DELETE"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/api/v1/policies?closeIssues=null"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/api/v1/policies?closeIssues=null"); var request = new
RestRequest(Method.DELETE); IRestResponse response = client.Execute(request);
```
Body parameters

array[String]
required
application/json; charset=UTF-8
Array
REQUEST
[application/json; charset=UTF-8 content]
Responses

Success

Client error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Forbidden error

Body
application/json; charset=UTF-8

Indicates a forbidden access response

▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Service had unexpected internal error

2.11.2.2 | Get Policies

get /api/v1/policies

Get all latest policies/rule instances from a list of given policy types. Default behavior is all.

Query parameters
▼ types array[PolicyType]

Lists policy types to fetch.

▼ disableVerbose Boolean

This flag bypasses calling the platform for asset groups data when fetching policies. Note that this means asset groups will not be updated when fetching policies.

Example: true
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v1/policies?types=&disableVerbose=true'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("GET", "/api/v1/policies?
types=null&disableVerbose=false") res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v1/policies?
types=null&disableVerbose=false") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/api/v1/policies?
types=null&disableVerbose=false"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/api/v1/policies?types=null&disableVerbose=false")
.asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/api/v1/policies?
types=null&disableVerbose=false")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "GET" let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v1/policies?
types=null&disableVerbose=false", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", ]); $response =
```

```
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL, "https://api-yourfqdn/api/v1/policies?types=null&disableVerbose=false"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/api/v1/policies?types=null&disableVerbose=false"); var request = new RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```

Responses

Success

Body
application/json; charset=UTF-8

The response you get when calling the `get policies` path

[
▶ actionstring (Enum)

One of the supported policy actions

▶ assetGroupsarray[string]
▶ assetGroupsIDsarray[integer]
▶ assetScopestringbyte
▶ conditionstringbyte
▶ createdAtstringdate-time
▶ createdBystring
▶ descriptionstring
▶ disabledboolean
▶ evaluationModesarray[string]
▶ evaluationStagestring (Enum)

Lists supported evaluation stages

▶ exceptionstringbyte
▶ idstring
▶ missingInformationActionstring (Enum)

One of the supported policy actions

▶ modifiedAtstringdate-time
▶ namestring
▶ remediationGuidancestring
▶ revisioninteger
▶ rulesIdsarray[string]

The UUIDs of the rules that define the condition

▶ severitystring (Enum)

One of the supported policy severities

▶ typestring (Enum)

One of the supported policy types

]
RESPONSE
`[application/json; charset=UTF-8 content]`

Client error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
`[application/json; charset=UTF-8 content]`

Service had unexpected internal error

2.11.2.3 | Add Policies

post /api/v1/policies

Adds policies and rule instances to the rules management service.

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v1/policies'
-d '{ "action" : "ISSUE", "assetGroups" : [ "group1", "group2" ], "assetGroupsIDs" : [ 1, 2 ], "assetScope" : "<some
asset scope in blob form>", "condition" : "<some condition in blob form>", "createdAt" : "2024-07-28T18:50Z",
"createdBy" : "admin", "description" : "some policy description", "disabled" : false, "evaluationModes" : [
"PERIODIC" ], "evaluationStage" : "CI", "exception" : "<some exception in blob form>", "id" : "1",
"missingInformationAction" : "ISSUE", "modifiedAt" : "2024-07-28T18:50Z", "name" : "some policy",
"remediationGuidance" : "some remediation guidance", "revision" : "3", "ruleId" : "", "severity" : "CRITICAL", "type"
: "COMPLIANCE" }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("POST", "/api/v1/policies") res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v1/policies") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/api/v1/policies"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/api/v1/policies") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/api/v1/policies")! as
URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v1/policies",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", ]); $response = curl_exec($curl);
$err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/api/v1/policies"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/api/v1/policies"); var request = new RestRequest(Method.POST);
IRestResponse response = client.Execute(request);
```

Body parameters

array[PolicyData]

required

application/json; charset=UTF-8

[

▶ actionstring (Enum)

One of the supported policy actions

▶ assetGroupsarray[string]
▶ assetGroupsIDsarray[integer]
▶ assetScopestringbyte
▶ conditionstringbyte
▶ createdAtstringdate-time
▶ createdBystring
▶ descriptionstring
▶ disabledboolean
▶ evaluationModesarray[string]
▶ evaluationStagestring (Enum)

Lists supported evaluation stages

▶ exceptionstringbyte
▶ idstring
▶ missingInformationActionstring (Enum)

One of the supported policy actions

▶ modifiedAtstringdate-time
▶ namestring
▶ remediationGuidancestring
▶ revisioninteger
▶ rulesIdsarray[string]

The UUIDs of the rules that define the condition

▶ severitystring (Enum)

One of the supported policy severities

▶ typestring (Enum)

One of the supported policy types

]
REQUEST
[application/json; charset=UTF-8 content]
Responses

Success

Client error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Forbidden error

Body
application/json; charset=UTF-8

Indicates a forbidden access response

▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Service encountered unexpected internal error

2.11.2.4 | Update Policies

put /api/v1/policies

Updates policies and rule instances in the rules management service.

CLIENT REQUEST Bash+curl
```
curl -X 'PUT'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v1/policies'
-d '{ "action" : "ISSUE", "assetGroups" : [ "group1", "group2" ], "assetGroupsIDs" : [ 1, 2 ], "assetScope" : "<some
asset scope in blob form>", "condition" : "<some condition in blob form>", "createdAt" : "2024-07-28T18:50Z",
"createdBy" : "admin", "description" : "some policy description", "disabled" : false, "evaluationModes" : [
"PERIODIC" ], "evaluationStage" : "CI", "exception" : "<some exception in blob form>", "id" : "1",
"missingInformationAction" : "ISSUE", "modifiedAt" : "2024-07-28T18:50Z", "name" : "some policy",
"remediationGuidance" : "some remediation guidance", "revision" : "3", "ruleId" : "", "severity" : "CRITICAL", "type"
: "COMPLIANCE" }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("PUT", "/api/v1/policies") res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v1/policies") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Put.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("PUT", "https://api-yourfqdn/api/v1/policies"); xhr.send(data);
HttpResponse<String> response = Unirest.put("https://api-yourfqdn/api/v1/policies") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/api/v1/policies")! as
URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "PUT" let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v1/policies",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
```

```
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "PUT", ]); $response = curl_exec($curl); $err
= curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "PUT"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/api/v1/policies"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/api/v1/policies"); var request = new RestRequest(Method.PUT);
IRestResponse response = client.Execute(request);
```

Body parameters
array[PolicyData]
required
application/json; charset=UTF-8
[
▶ actionstring (Enum)

One of the supported policy actions

▶ assetGroupsarray[string]
▶ assetGroupsIDsarray[integer]
▶ assetScopestringbyte
▶ conditionstringbyte
▶ createdAtstringdate-time
▶ createdBystring
▶ descriptionstring
▶ disabledboolean
▶ evaluationModesarray[string]
▶ evaluationStagestring (Enum)

Lists supported evaluation stages

▶ exceptionstringbyte
▶ idstring
▶ missingInformationActionstring (Enum)

One of the supported policy actions

▶ modifiedAtstringdate-time
▶ namestring
▶ remediationGuidancestring
▶ revisioninteger
▶ rulesIdsarray[string]

The UUIDs of the rules that define the condition

▶ severitystring (Enum)

One of the supported policy severities

▶ typestring (Enum)

One of the supported policy types

]
REQUEST
`[application/json; charset=UTF-8 content]`
Responses

Success

Client error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
`[application/json; charset=UTF-8 content]`

Forbidden error

Body
application/json; charset=UTF-8

Indicates a forbidden access response

▼ errorstring
RESPONSE
`[application/json; charset=UTF-8 content]`

2.11.2.5 | Disable Policies

post /api/v1/policies/disable

Disable policies/rule instances.

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v1/policies/disable'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("POST",
"/api/v1/policies/disable") res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v1/policies/disable") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/api/v1/policies/disable");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/api/v1/policies/disable") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/api/v1/policies/disable")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v1/policies/disable",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", ]); $response = curl_exec($curl);
$err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/api/v1/policies/disable"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/api/v1/policies/disable"); var request = new
RestRequest(Method.POST); IRestResponse response = client.Execute(request);
```

Body parameters
array[String]
required
application/json; charset=UTF-8
Array
REQUEST

```
[application/json; charset=UTF-8 content]
```

Responses

Success

Client error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE

```
[application/json; charset=UTF-8 content]
```

Forbidden error

Body
application/json; charset=UTF-8

Indicates a forbidden access response

▼ errorstring
RESPONSE

```
[application/json; charset=UTF-8 content]
```

2.11.2.6 | Enable Policies

post /api/v1/policies/enable

Enable policies/rule instances.

CLIENT REQUEST Bash+curl

```
curl -X 'POST'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v1/policies/enable'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("POST", "/api/v1/policies/enable")
res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v1/policies/enable") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/api/v1/policies/enable");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/api/v1/policies/enable") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/api/v1/policies/enable")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v1/policies/enable",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", ]); $response = curl_exec($curl);
$err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/api/v1/policies/enable"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/api/v1/policies/enable"); var request = new
RestRequest(Method.POST); IRestResponse response = client.Execute(request);
```

Body parameters
array[String]
required
application/json; charset=UTF-8
Array
REQUEST

```
[application/json; charset=UTF-8 content]
```

Responses

Success

Client error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE

```
[application/json; charset=UTF-8 content]
```

Forbidden error

Body
application/json; charset=UTF-8

Indicates a forbidden access response

▼ errorstring
RESPONSE

```
[application/json; charset=UTF-8 content]
```

Service encountered unexpected internal error

2.11.2.7 | Get a Policy by ID

get /api/v1/policies/{id}

Get a policy instance by its ID.
Path parameters
▼ id String required
Example: `id_example`
Query parameters
▼ disableVerbose Boolean

Flag to bypass calling platform for asset groups data when fetching policy. Note that this means asset groups will not be updated upon fetching policy.

Example: `true`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v1/policies/{id}?disableVerbose=true'
```
```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("GET", "/api/v1/policies/%7Bid%7D?
disableVerbose=false") res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
```
```
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v1/policies/%7Bid%7D?
disableVerbose=false") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) response = http.request(request) puts response.read_body
```
```
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/api/v1/policies/%7Bid%7D?
disableVerbose=false"); xhr.send(data);
```
```
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/api/v1/policies/%7Bid%7D?disableVerbose=false")
.asString();
```
```
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/api/v1/policies/%7Bid%7D?disableVerbose=false")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "GET" let session = URLSession.shared let dataTask =
session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error !=
nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
```
```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v1/policies/%7Bid%7D?
disableVerbose=false", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
```
```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/api/v1/policies/%7Bid%7D?disableVerbose=false"); CURLcode ret = curl_easy_perform(hnd);
```
```
var client = new RestClient("https://api-yourfqdn/api/v1/policies/%7Bid%7D?disableVerbose=false"); var request = new
RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```
Responses

Success

Body
application/json; charset=UTF-8

The schema of a returned policy

▼ actionstring (Enum)

One of the supported policy actions

Example:`"ISSUE"`
Allowed values:"ISSUE""PREVENT"null
▼ assetGroupsarray[string]
▼ assetGroupsIDsarray[integer]
▼ assetScopestringbyte
▼ conditionstringbyte
▼ createdAtstringdate-time
▼ createdBystring
▼ descriptionstring
▼ disabledboolean
▼ evaluationModesarray[string]
▼ evaluationStagestring (Enum)

Lists supported evaluation stages

Example:`"CI"`
Allowed values:"CI""RUNTIME""DEPLOY"
▼ exceptionstringbyte
▼ idstring

▼ missingInformationActionstring (Enum)

One of the supported policy actions

Example:"ISSUE"
Allowed values:"ISSUE""PREVENT"null
▼ modifiedAtstringdate-time
▼ namestring
▼ remediationGuidancestring
▼ revisioninteger
▼ rulesIdsarray[string]

The UUIDs of the rules that define the condition

▼ severitystring (Enum)

One of the supported policy severities

Example:"CRITICAL"
Allowed values:"INFO""LOW""MEDIUM""HIGH""CRITICAL"null
▼ typestring (Enum)

One of the supported policy types

Example:"MALWARE"
Allowed values:"COMPLIANCE""MALWARE""SECRET""TRUSTED_IMAGES"null
RESPONSE
[application/json; charset=UTF-8 content]

Bad request error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Policy not found

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Service had unexpected internal error

2.11.2.8 | Add (custom) Compliance rules

post /api/v1/rules/add

Adds a custom compliance rule to the rules management service.

CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v1/rules/add'
-d '{ "changeLog" : "format the rego script", "controlsIDs" : [ "2a16a9f3-2602-4f0c-a909-7c184ddfeda2", "2a16a9f3-2602-4f0c-a909-7c184ddfe11" ], "createdAt" : "2024-06-12T09:55:22", "createdBy" : "eli", "description" : "Some description for the rule", "engine" : "rego", "evaluationScript" : " package pan.k8s_policy default allow = false allow { not has_wildcard_resources(input.resource) not has_wildcard_verbs(input.resource) } has_wildcard_resources(role) { some i role.resources[i] == \"*\" } has_wildcard_verbs(role) { some i role.verbs[i] == \"*\" } ", "extraData" : { "compliance" : [ { "control" : "5.4.1", "name" : "CIS AKS", "version" : "1.5" } ], "finding" : { "category" : "IDENTITY", "description" : "Cool k8s finding", "name" : "Kubernetes role with excessive rights found", "typeId" : 80 }, "scannableAssets" : [ { "group" : "rbac.authorization.k8s.io", "resource" : "role", "version" : "v1" }, { "group" : "rbac.authorization.k8s.io", "resourfce" : "clusterroles", "version" : "v1" } ], "supportedActions" : [ "Issue" ] }, "isSystemRule" : true, "modifiedAt" : "2024-11-23T19:35:48", "modifiedBy" : "gil", "name" : "Minimize wildcard use in Roles and ClusterRoles", "platforms" : [ "Linux" ], "providers" : [ "AWS", "GCP" ], "remediation" : "Enable ssl mode", "revision" : 1, "ruleId" : "2808bbd9-51a4-47cc-a2fa-60d54903fe9c", "severity" : "LOW", "source" : [ "K8sApi" ], "specVer" : "1", "type" : "COMPLIANCE" }'
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("POST", "/api/v1/rules/add") res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v1/rules/add") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/api/v1/rules/add"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/api/v1/rules/add") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/api/v1/rules/add")! as
URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v1/rules/add",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", ]); $response = curl_exec($curl);
$err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/api/v1/rules/add"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/api/v1/rules/add"); var request = new RestRequest(Method.POST);
IRestResponse response = client.Execute(request);
```

Body parameters

array[RuleDef]

required

application/json; charset=UTF-8

[

▶ availableActionsarray[string]

A list of the available actions

▶ changeLogstring
▶ controlsIDsarray[string]
▶ createdAtstringdate-time
▶ createdBystring
▶ descriptionstring
▶ enginestring (Enum)
▶ evaluationScriptstring
▶ extraDataobject
▶ isSystemRuleboolean
▶ modifiedAtstringdate-time
▶ modifiedBystring
▶ namestring
▶ platformsarray[string]
▶ providersarray[string]
▶ remediationstring
▶ revisioninteger
▶ ruleIdstring
▶ ruleStandardsarray

Lists rule standards

▶ ruleUAIAssetsarray

Lists UAI assets relevant to the rule

▶ severitystring (Enum)

Supported rule severities

▶ sourcearray[string]
▶ specVerstring
▶ typestring

]

REQUEST

```
[application/json; charset=UTF-8 content]
```

Responses

Success

Client error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
```
[application/json; charset=UTF-8 content]
```

Forbidden error

Body
application/json; charset=UTF-8

Indicates a forbidden access response

▼ errorstring
RESPONSE
```
[application/json; charset=UTF-8 content]
```

Service encountered an unexpected internal error

2.11.2.9 | Delete (custom) CWP Rules

post /api/v1/rules/delete

Delete custom CWP Rules instances from the rules management service.

CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v1/rules/delete'
-d ''
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("POST", "/api/v1/rules/delete")
res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v1/rules/delete") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/api/v1/rules/delete"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/api/v1/rules/delete") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/api/v1/rules/delete")!
as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v1/rules/delete",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", ]); $response = curl_exec($curl);
$err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/api/v1/rules/delete"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/api/v1/rules/delete"); var request = new RestRequest(Method.POST);
IRestResponse response = client.Execute(request);
```
Body parameters
array[String]
required
application/json; charset=UTF-8
Array
REQUEST
```
[application/json; charset=UTF-8 content]
```
Responses

Success

Client error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE

[application/json; charset=UTF-8 content]

Forbidden error

Body
application/json; charset=UTF-8

Indicates a forbidden access response

▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Service encountered an unexpected internal error

2.11.2.10 | Get rules by IDs

post /api/v1/rules/fetch

Get rules by a list of IDs. If a rule is not found for a specific ID, an error is returned.

CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json'
'https://api-yourfqdn/api/v1/rules/fetch'
-d ''
```
```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") payload = "{\"ids\":[\"rule-id-1\",\"rule-id-2\",\"rule-id-3\"]}" headers = { 'content-type': "application/json" } conn.request("POST", "/api/v1/rules/fetch", payload, headers) res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
```
```
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v1/rules/fetch") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Post.new(url) request["content-type"] = 'application/json' request.body = "{\"ids\":[\"rule-id-1\",\"rule-id-2\",\"rule-id-3\"]}" response = http.request(request) puts response.read_body
```
```
const data = JSON.stringify({ "ids": [ "rule-id-1", "rule-id-2", "rule-id-3" ] }); const xhr = new XMLHttpRequest(); xhr.withCredentials = true; xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) { console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/api/v1/rules/fetch"); xhr.setRequestHeader("content-type", "application/json"); xhr.send(data);
```
```
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/api/v1/rules/fetch") .header("content-type", "application/json") .body("{\"ids\":[\"rule-id-1\",\"rule-id-2\",\"rule-id-3\"]}") .asString();
```
```
import Foundation let headers = ["content-type": "application/json"] let parameters = ["ids": ["rule-id-1", "rule-id-2", "rule-id-3"]] as [String : Any] let postData = JSONSerialization.data(withJSONObject: parameters, options: []) let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/api/v1/rules/fetch")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" request.allHTTPHeaderFields = headers request.httpBody = postData as Data let session = URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
```
```
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v1/rules/fetch", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", CURLOPT_POSTFIELDS => "{\"ids\":[\"rule-id-1\",\"rule-id-2\",\"rule-id-3\"]}", CURLOPT_HTTPHEADER => [ "content-type: application/json" ], ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
```
```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd, CURLOPT_URL, "https://api-yourfqdn/api/v1/rules/fetch"); struct curl_slist *headers = NULL; headers = curl_slist_append(headers, "content-type: application/json"); curl_easy_setopt(hnd, CURLOPT_HTTPHEADER, headers); curl_easy_setopt(hnd, CURLOPT_POSTFIELDS, "{\"ids\":[\"rule-id-1\",\"rule-id-2\",\"rule-id-3\"]}"); CURLcode ret = curl_easy_perform(hnd);
```
```
var client = new RestClient("https://api-yourfqdn/api/v1/rules/fetch"); var request = new RestRequest(Method.POST); request.AddHeader("content-type", "application/json"); request.AddParameter("application/json", "{\"ids\":[\"rule-id-1\",\"rule-id-2\",\"rule-id-3\"]}", ParameterType.RequestBody); IRestResponse response = client.Execute(request);
```
Body parameters
required
application/json
▼ idsarray[string]
REQUEST
```
{ "ids": [ "rule-id-1", "rule-id-2", "rule-id-3" ] }
```
Responses

Success. Returns a list of rule objects.

Body
application/json; charset=UTF-8
[
▶ availableActionsarray[string]

A list of the available actions

▶ changeLogstring
▶ controlsIDsarray[string]
▶ createdAtstringdate-time
▶ createdBystring
▶ descriptionstring
▶ enginestring (Enum)
▶ evaluationScriptstring
▶ extraDataobject
▶ isSystemRuleboolean
▶ modifiedAtstringdate-time
▶ modifiedBystring
▶ namestring
▶ platformsarray[string]
▶ providersarray[string]
▶ remediationstring
▶ revisioninteger
▶ ruleIdstring
▶ ruleStandardsarray

Lists rule standards

▶ ruleUAIAssetsarray

Lists UAI assets relevant to the rule

▶ severitystring (Enum)

Supported rule severities

▶ sourcearray[string]
▶ specVerstring
▶ typestring
]
RESPONSE
`[application/json; charset=UTF-8 content]`

Bad request error.

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
`[application/json; charset=UTF-8 content]`

Rules not found

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
`[application/json; charset=UTF-8 content]`

Service encountered an unexpected internal error.

2.11.2.11 | Get the Compliance Rules

get /api/v1/rules/get

Get all Compliance rules (system and custom).This API supports pagination.

Query parameters
▼ offset Integer

Fetches rules from offset location, default is 0.
Example: `56`
▼ limit Integer

The maximum number of rules to fetch in a single page request, if not specified. default is set to fetch all.

Example: `56`
▼ types array[PolicyType]

The policy rule definitions types to fetch.

▼ platform String

The platform to which the rules should be applied.

Example: `platform_example`
▼ source String

The scanner source type for the rule.

Example: `K8sApi`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v1/rules/get?offset=56&limit=56&types=&platform=platform_example&source=K8sApi'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("GET", "/api/v1/rules/get?
offset=SOME_INTEGER_VALUE&limit=SOME_INTEGER_VALUE&types=null&platform=SOME_STRING_VALUE&source=K8sApi") res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v1/rules/get?
offset=SOME_INTEGER_VALUE&limit=SOME_INTEGER_VALUE&types=null&platform=SOME_STRING_VALUE&source=K8sApi") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/api/v1/rules/get?
offset=SOME_INTEGER_VALUE&limit=SOME_INTEGER_VALUE&types=null&platform=SOME_STRING_VALUE&source=K8sApi");
xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/api/v1/rules/get?
offset=SOME_INTEGER_VALUE&limit=SOME_INTEGER_VALUE&types=null&platform=SOME_STRING_VALUE&source=K8sApi") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/api/v1/rules/get?
offset=SOME_INTEGER_VALUE&limit=SOME_INTEGER_VALUE&types=null&platform=SOME_STRING_VALUE&source=K8sApi")! as URL,
cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "GET" let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v1/rules/get?
offset=SOME_INTEGER_VALUE&limit=SOME_INTEGER_VALUE&types=null&platform=SOME_STRING_VALUE&source=K8sApi",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", ]); $response = curl_exec($curl); $err
= curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/api/v1/rules/get?
offset=SOME_INTEGER_VALUE&limit=SOME_INTEGER_VALUE&types=null&platform=SOME_STRING_VALUE&source=K8sApi"); CURLcode
ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/api/v1/rules/get?
offset=SOME_INTEGER_VALUE&limit=SOME_INTEGER_VALUE&types=null&platform=SOME_STRING_VALUE&source=K8sApi"); var request
= new RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```
Responses

Success

Body
application/json; charset=UTF-8

Returns a compliance rules page

▼ limitinteger

The maximum rules to fetch in a single page request, if not provided default is to fetch all.

Example:`50`
▼ offsetinteger

The offset from where the rules fetched
Example:`150`
▼ rulesarray
[
▶ availableActionsarray[string]

Lists available actions

▶ changeLogstring
▶ controlsIDsarray[string]
▶ createdAtstringdate-time
▶ createdBystring
▶ descriptionstring
▶ enginestring (Enum)
▶ evaluationScriptstring
▶ extraDataobject
▶ isSystemRuleboolean
▶ modifiedAtstringdate-time
▶ modifiedBystring
▶ namestring
▶ platformsarray[string]
▶ providersarray[string]
▶ remediationstring
▶ revisioninteger
▶ ruleIdstring
▶ ruleStandardsarray

Lists rule standards

▶ ruleUAIAssetsarray

Lists UAI assets relevant to the rule

▶ severitystring (Enum)

Supported rule severities

▶ sourcearray[string]
▶ specVerstring
▶ typestring
]
▼ totalinteger

Total filtered count of rules

Example:`347`
RESPONSE
`[application/json; charset=UTF-8 content]`

Client error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
`[application/json; charset=UTF-8 content]`

Service encountered an unexpected internal error

2.11.2.12 | Get a rule by ID

get /api/v1/rules/get/{id}

Get a rule instance by its ID.

Path parameters
▼ id String required
Example: `id_example`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json; charset=UTF-8'
```

```
'https://api-yourfqdn/api/v1/rules/get/{id}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("GET",
"/api/v1/rules/get/%7Bid%7D") res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v1/rules/get/%7Bid%7D") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Get.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/api/v1/rules/get/%7Bid%7D");
xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/api/v1/rules/get/%7Bid%7D") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/api/v1/rules/get/%7Bid%7D")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "GET" let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/api/v1/rules/get/%7Bid%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/api/v1/rules/get/%7Bid%7D"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/api/v1/rules/get/%7Bid%7D"); var request = new
RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```

Responses

Success

Body
application/json; charset=UTF-8

The schema of a rule definition

▼ availableActionsarray[string]

A list of the available actions

▼ changeLogstring
▼ controlsIDsarray[string]
▼ createdAtstringdate-time
▼ createdBystring
▼ descriptionstring
▼ enginestring (Enum)
Example:"rego"
Allowed values:"rego""bash""embedded""python"
▼ evaluationScriptstring
▼ extraDataobject
▼ isSystemRuleboolean
▼ modifiedAtstringdate-time
▼ modifiedBystring
▼ namestring
▼ platformsarray[string]
▼ providersarray[string]
▼ remediationstring
▼ revisioninteger
▼ ruleIdstring
▼ ruleStandardsarray

Lists rule standards

[
]
▼ ruleUAIAssetsarray

Lists UAI assets relevant to the rule

[
]
▼ severitystring (Enum)

Supported rule severities

Example:"CRITICAL"
Allowed values:"INFO""LOW""MEDIUM""HIGH""CRITICAL"
▼ sourcearray[string]
▼ specVerstring
▼ typestring
RESPONSE
[application/json; charset=UTF-8 content]

Bad request error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Rule not found

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Service encountered an unexpected internal error

2.11.2.13 | rego script linter

post /api/v1/rules/lint/rego

lint the content of the rego script

CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v1/rules/lint/rego'
-d '{ "script" :
"cGFja2FnZSBwYW53LmNvbXBsaWFuY2UKCmltcG9ydCByZWdvLnYxCgptYXRjaCBjb250YWlucyB7Im1zZyI6IG1zZ30gaWYgewojIEFkZCB5b3VyIGxvZ2ljIGhlcm
}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("POST", "/api/v1/rules/lint/rego")
res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v1/rules/lint/rego") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/api/v1/rules/lint/rego");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/api/v1/rules/lint/rego") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/api/v1/rules/lint/rego")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v1/rules/lint/rego",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", ]); $response = curl_exec($curl);
$err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/api/v1/rules/lint/rego"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/api/v1/rules/lint/rego"); var request = new
RestRequest(Method.POST); IRestResponse response = client.Execute(request);
```
Body parameters
required
application/json; charset=UTF-8
▼ scriptstringbyte
Example:"package panw.compliance\n\nimport rego.v1\n\nmatch contains {\"msg\": msg} if {\n# Add your logic here\nmsg
:= \"<custom violation message>\"\n}"

REQUEST
[application/json; charset=UTF-8 content]
Responses

Success

Client error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Invalid script syntax

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Service encountered an unexpected internal error

2.11.2.14 | Update (custom) Compliance rules

post /api/v1/rules/update

Update (custom) Compliance rules in the rules management service.

CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v1/rules/update'
-d '{ "changeLog" : "format the rego script", "controlsIDs" : [ "2a16a9f3-2602-4f0c-a909-7c184ddfeda2", "2a16a9f3-
2602-4f0c-a909-7c184ddfe11" ], "createdAt" : "2024-06-12T09:55:22", "createdBy" : "eli", "description" : "Some
description for the rule", "engine" : "rego", "evaluationScript" : " package pan.k8s_policy default allow = false
allow { not has_wildcard_resources(input.resource) not has_wildcard_verbs(input.resource) }
has_wildcard_resources(role) { some i role.resources[i] == \"*\" } has_wildcard_verbs(role) { some i role.verbs[i] ==
\"*\" } ", "extraData" : { "compliance" : [ { "control" : "5.4.1", "name" : "CIS AKS", "version" : "1.5" } ],
"finding" : { "category" : "IDENTITY", "description" : "Cool k8s finding", "name" : "Kubernetes role with excessive
rights found", "typeId" : 80 }, "scannableAssets" : [ { "group" : "rbac.authorization.k8s.io", "resource" : "role",
"version" : "v1" }, { "group" : "rbac.authorization.k8s.io", "resourfce" : "clusterroles", "version" : "v1" } ],
"supportedActions" : [ "Issue" ] }, "isSystemRule" : true, "modifiedAt" : "2024-11-23T19:35:48", "modifiedBy" :
"gil", "name" : "Minimize wildcard use in Roles and ClusterRoles", "platforms" : [ "Linux" ], "providers" : [ "AWS",
"GCP" ], "remediation" : "Enable ssl mode", "revision" : 1, "ruleId" : "2808bbd9-51a4-47cc-a2fa-60d54903fe9c",
"severity" : "LOW", "source" : [ "K8sApi" ], "specVer" : "1", "type" : "COMPLIANCE" }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("POST", "/api/v1/rules/update")
res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v1/rules/update") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/api/v1/rules/update"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/api/v1/rules/update") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/api/v1/rules/update")!
as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v1/rules/update",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", ]); $response = curl_exec($curl);
$err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/api/v1/rules/update"); CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/api/v1/rules/update"); var request = new RestRequest(Method.POST);
IRestResponse response = client.Execute(request);
```

Body parameters
array[RuleDef]
required
application/json; charset=UTF-8
[
▶ availableActionsarray[string]

A list of the available actions

▶ changeLogstring
▶ controlsIDsarray[string]
▶ createdAtstringdate-time
▶ createdBystring
▶ descriptionstring
▶ enginestring (Enum)
▶ evaluationScriptstring
▶ extraDataobject
▶ isSystemRuleboolean
▶ modifiedAtstringdate-time
▶ modifiedBystring
▶ namestring
▶ platformsarray[string]
▶ providersarray[string]
▶ remediationstring
▶ revisioninteger
▶ ruleIdstring
▶ ruleStandardsarray

Lists rule standards

▶ ruleUAIAssetsarray

Lists UAI assets relevant to the rule

▶ severitystring (Enum)

Supported rule severities

▶ sourcearray[string]
▶ specVerstring
▶ typestring
]
REQUEST
```
[application/json; charset=UTF-8 content]
```
Responses

Success

Client error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
```
[application/json; charset=UTF-8 content]
```

Forbidden error

Body
application/json; charset=UTF-8

Indicates a forbidden access response

▼ errorstring
RESPONSE
```
[application/json; charset=UTF-8 content]
```

Service encountered an unexpected internal error

2.11.2.15 | Get Policies

get /api/v2/policies

Gets all policy instances from a list of given policy types. Default behavior is all.

Query parameters
▼ types array[PolicyType]

Policy types to fetch

▼ disableVerbose Boolean

Flag to bypass calling platform for asset groups data when fetching policies. Note that this means asset groups will not be updated when fetching policies.

Example: `true`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v2/policies?types=&disableVerbose=true'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("GET", "/api/v2/policies?
types=null&disableVerbose=false") res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v2/policies?
types=null&disableVerbose=false") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/api/v2/policies?
types=null&disableVerbose=false"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/api/v2/policies?types=null&disableVerbose=false")
.asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/api/v2/policies?
types=null&disableVerbose=false")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "GET" let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v2/policies?
types=null&disableVerbose=false", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/api/v2/policies?types=null&disableVerbose=false"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/api/v2/policies?types=null&disableVerbose=false"); var request =
new RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```
Responses

Success

Body
application/json; charset=UTF-8

The response you get when calling the `get policies V2` path

[
▶ actionstring (Enum)

One of the supported policy actions

▶ assetGroupsarray[string]
▶ assetGroupsIDsarray[integer]
▶ assetScopestringbyte
▶ conditionstringbyte
▶ createdAtstringdate-time
▶ createdBystring
▶ descriptionstring
▶ disabledboolean
▶ evaluationModesarray[string]
▶ evaluationStagestring (Enum)

Lists supported evaluation stages

▶ exceptionstringbyte

▶ idstring
▶ missingInformationActionstring (Enum)

One of the supported policy actions

▶ modifiedAtstringdate-time
▶ namestring
▶ policyRulesarray

Array of policy_rule objects related to the policy

▶ remediationGuidancestring
▶ revisioninteger
▶ severitystring (Enum)

One of the supported policy severities

▶ typestring (Enum)

One of the supported policy types

▶ usingSystemAssetGroupsboolean
]
RESPONSE
[application/json; charset=UTF-8 content]

Client error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Service encountered an unexpected internal error

2.11.2.16 | Add Policies

post /api/v2/policies

Adds policies and rule instances to the rules management service.

CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v2/policies'
-d '{ "action" : "ISSUE", "assetGroups" : [ "group1", "group2" ], "assetGroupsIDs" : [ 1, 2 ], "assetScope" : "<some
asset scope in blob form>", "condition" : "<some condition in blob form>", "createdBy" : "user", "description" :
"some policy description", "disabled" : false, "evaluationModes" : [ "PERIODIC" ], "evaluationStage" : "CI",
"exception" : "<some exception in blob form>", "missingInformationAction" : "ISSUE", "name" : "some policy",
"policyRules" : [ { "action" : "ISSUE", "rule_id" : "3", "severity" : "CRITICAL" }, { "action" : "PREVENT", "rule_id"
: "4", "severity" : "HIGH" } ], "remediationGuidance" : "some remediation guidance", "severity" : "CRITICAL", "type"
: "COMPLIANCE", "usingSystemAssetGroups" : false }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("POST", "/api/v2/policies") res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v2/policies") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/api/v2/policies"); xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/api/v2/policies") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/api/v2/policies")! as
URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "POST" let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v2/policies",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
```

```
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", ]); $response = curl_exec($curl);
$err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/api/v2/policies"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/api/v2/policies"); var request = new RestRequest(Method.POST);
IRestResponse response = client.Execute(request);
```

Body parameters

array[PolicyPostRequestDataV2]

required

application/json; charset=UTF-8

[

▶ actionstring (Enum)

One of the supported policy actions

▶ assetGroupsarray[string]

▶ assetGroupsIDsarray[integer]

▶ assetScopestringbyte

▶ conditionstringbyte

▶ createdBystring

▶ descriptionstring

▶ disabledboolean

▶ evaluationModesarray[string]

▶ evaluationStagestring (Enum)

Lists supported evaluation stages

▶ exceptionstringbyte

▶ missingInformationActionstring (Enum)

One of the supported policy actions

▶ namestring

▶ policyRulesarray

Array of policy_rule objects related to the policy

▶ remediationGuidancestring

▶ severitystring (Enum)

One of the supported policy severities

▶ typestring (Enum)

One of the supported policy types

▶ usingSystemAssetGroupsboolean

]

REQUEST

```
[application/json; charset=UTF-8 content]
```

Responses

Success

Body

application/json; charset=UTF-8

▼ idstring

RESPONSE

```
[application/json; charset=UTF-8 content]
```

Client error

Body

application/json; charset=UTF-8

▼ errorstring

RESPONSE

```
[application/json; charset=UTF-8 content]
```

Forbidden error

Body

application/json; charset=UTF-8

Indicates a forbidden access response

▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Service encountered an unexpected internal error

2.11.2.17 | Update Policies

put /api/v2/policies

Update policies/rule instances in the rules management service.

CLIENT REQUEST Bash+curl
```
curl -X 'PUT'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v2/policies'
-d '{ "action" : "ISSUE", "assetGroups" : [ "group1", "group2" ], "assetGroupsIDs" : [ 1, 2 ], "assetScope" : "<some
asset scope in blob form>", "condition" : "<some condition in blob form>", "createdBy" : "user", "description" :
"some policy description", "disabled" : false, "evaluationModes" : [ "PERIODIC" ], "evaluationStage" : "CI",
"exception" : "<some exception in blob form>", "id" : "policyID", "missingInformationAction" : "ISSUE", "name" :
"some policy", "policyRules" : [ { "action" : "ISSUE", "rule_id" : "3", "severity" : "CRITICAL" }, { "action" :
"PREVENT", "rule_id" : "4", "severity" : "HIGH" } ], "remediationGuidance" : "some remediation guidance", "severity"
: "CRITICAL", "type" : "COMPLIANCE", "usingSystemAssetGroups" : false }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("PUT", "/api/v2/policies") res =
conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v2/policies") http =
Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Put.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("PUT", "https://api-yourfqdn/api/v2/policies"); xhr.send(data);
HttpResponse<String> response = Unirest.put("https://api-yourfqdn/api/v2/policies") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-yourfqdn/api/v2/policies")! as
URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "PUT" let session =
URLSession.shared let dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response,
error) -> Void in if (error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse
print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v2/policies",
CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30,
CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "PUT", ]); $response = curl_exec($curl); $err
= curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "PUT"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/api/v2/policies"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/api/v2/policies"); var request = new RestRequest(Method.PUT);
IRestResponse response = client.Execute(request);
```
Body parameters
array[PolicyPutRequestDataV2]
required
application/json; charset=UTF-8
[
▶ actionstring (Enum)

One of the supported policy actions

▶ assetGroupsarray[string]
▶ assetGroupsIDsarray[integer]
▶ assetScopestringbyte
▶ conditionstringbyte
▶ createdBystring
▶ descriptionstring
▶ disabledboolean
▶ evaluationModesarray[string]
▶ evaluationStagestring (Enum)

Lists supported evaluation stages

▶ exceptionstringbyte
▶ idstring

▶ missingInformationActionstring (Enum)

One of the supported policy actions

▶ namestring
▶ policyRulesarray

Array of policy_rule objects related to the policy

▶ remediationGuidancestring
▶ severitystring (Enum)

One of the supported policy severities

▶ typestring (Enum)

One of the supported policy types

▶ usingSystemAssetGroupsboolean
]
REQUEST
[application/json; charset=UTF-8 content]
Responses

Success

Client error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Forbidden error

Body
application/json; charset=UTF-8

Indicates a forbidden access response

▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Service had unexpected internal error

2.11.2.18 | Get a Policy by ID

get /api/v2/policies/{id}

Get a policy instance by its ID.

Path parameters
▼ id String required
Example: id_example
Query parameters
▼ disableVerbose Boolean

Flag to bypass calling platform for asset groups data when fetching policy. Note that this means asset groups will not be updated when fetching policy.

Example: true
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json; charset=UTF-8'
'https://api-yourfqdn/api/v2/policies/{id}?disableVerbose=true'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("GET", "/api/v2/policies/%7Bid%7D?
disableVerbose=false") res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/api/v2/policies/%7Bid%7D?
disableVerbose=false") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
```

```
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/api/v2/policies/%7Bid%7D?
disableVerbose=false"); xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/api/v2/policies/%7Bid%7D?disableVerbose=false")
.asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/api/v2/policies/%7Bid%7D?disableVerbose=false")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "GET" let session = URLSession.shared let dataTask =
session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error !=
nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-yourfqdn/api/v2/policies/%7Bid%7D?
disableVerbose=false", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10,
CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "GET", ]); $response =
curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo
$response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/api/v2/policies/%7Bid%7D?disableVerbose=false"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/api/v2/policies/%7Bid%7D?disableVerbose=false"); var request = new
RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```

Responses

Success

Body
application/json; charset=UTF-8

The schema of a returned policy

▼ actionstring (Enum)

One of the supported policy actions

Example:`"ISSUE"`
Allowed values:"ISSUE""PREVENT"null
▼ assetGroupsarray[string]
▼ assetGroupsIDsarray[integer]
▼ assetScopestringbyte
▼ conditionstringbyte
▼ createdAtstringdate-time
▼ createdBystring
▼ descriptionstring
▼ disabledboolean
▼ evaluationModesarray[string]
▼ evaluationStagestring (Enum)

Lists supported evaluation stages

Example:`"CI"`
Allowed values:"CI""RUNTIME""DEPLOY"
▼ exceptionstringbyte
▼ idstring
▼ missingInformationActionstring (Enum)

One of the supported policy actions

Example:`"ISSUE"`
Allowed values:"ISSUE""PREVENT"null
▼ modifiedAtstringdate-time
▼ namestring
▼ policyRulesarray

Array of policy_rule objects related to the policy

[
▶ actionstring (Enum)

One of the supported policy actions

▶ idstring
▶ policy_idstring
▶ policy_revisioninteger
▶ remediation_guidancestring
▶ rule_idstring

▶ rule_namestring
▶ severitystring (Enum)

One of the supported policy severities

▶ user_remediation_guidancestring

Additional remediation guidance that can be added to the rule's predefined guidance.

]
▼ remediationGuidancestring
▼ revisioninteger
▼ severitystring (Enum)

One of the supported policy severities

Example:"CRITICAL"
Allowed values:"INFO""LOW""MEDIUM""HIGH""CRITICAL"null
▼ typestring (Enum)

One of the supported policy types

Example:"MALWARE"
Allowed values:"COMPLIANCE""MALWARE""SECRET""TRUSTED_IMAGES"null
▼ usingSystemAssetGroupsboolean
RESPONSE
[application/json; charset=UTF-8 content]

Bad request error

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Policy not found

Body
application/json; charset=UTF-8
▼ errorstring
RESPONSE
[application/json; charset=UTF-8 content]

Service encountered an unexpected internal error

2.11.2.19 | Get Policies.

get /public_api/v1/cwp/policies

Get all policy instances of a given type. Default behavior is all.

Query parameters
▼ types array[PolicyType]

The policies type

CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json; charset=UTF-8'
'https://api-yourfqdn/public_api/v1/cwp/policies?types='
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("GET",
"/public_api/v1/cwp/policies?types=SOME_ARRAY_VALUE") res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/cwp/policies?
types=SOME_ARRAY_VALUE") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode =
OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/v1/cwp/policies?
types=SOME_ARRAY_VALUE"); xhr.send(data);
```

```
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/v1/cwp/policies?types=SOME_ARRAY_VALUE")
.asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/cwp/policies?types=SOME_ARRAY_VALUE")! as URL, cachePolicy: .useProtocolCachePolicy,
timeoutInterval: 10.0) request.httpMethod = "GET" let session = URLSession.shared let dataTask =
session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if (error !=
nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cwp/policies?types=SOME_ARRAY_VALUE", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error
#:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/v1/cwp/policies?types=SOME_ARRAY_VALUE"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cwp/policies?types=SOME_ARRAY_VALUE"); var request =
new RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```

Responses

Success

Body
application/json; charset=UTF-8

The response you get when calling the `get policies` path

[
▶ actionstring (Enum)

One of the supported policy actions

▶ assetGroupsarray[string]
▶ assetGroupsIDsarray[integer]
▶ assetScopestringbyte
▶ conditionstringbyte
▶ createdAtstringdate-time
▶ createdBystring
▶ descriptionstring
▶ disabledboolean
▶ evaluationModesarray[string]
▶ evaluationStagestring (Enum)

Lists supported evaluation stages

▶ exceptionstringbyte
▶ idstring
▶ missingInformationActionstring (Enum)

One of the supported policy actions

▶ modifiedAtstringdate-time
▶ namestring
▶ remediationGuidancestring
▶ revisioninteger
▶ rulesIdsarray[string]

The UUIDs of the rules that define the condition

▶ severitystring (Enum)

One of the supported policy severities

▶ typestring (Enum)

One of the supported policy types

]
RESPONSE
```
[application/json; charset=UTF-8 content]
```

Client error

Body
application/json; charset=UTF-8

▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

Reported error code

RESPONSE
[application/json; charset=UTF-8 content]

Service encountered an unexpected internal error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

Reported error code

RESPONSE
[application/json; charset=UTF-8 content]

2.11.2.20 | Add Policy.

post /public_api/v1/cwp/policies

Add policy instances.

CLIENT REQUEST Bash+curl
```
curl -X 'POST'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json; charset=UTF-8'
'https://api-yourfqdn/public_api/v1/cwp/policies'
-d '{ "action" : "ISSUE", "assetGroups" : [ "group1", "group2" ], "assetGroupsIDs" : [ 1, 2 ], "assetScope" : "<some
asset scope in blob form>", "condition" : "<some condition in blob form>", "createdAt" : "2024-07-28T18:50Z",
"createdBy" : "admin", "description" : "some policy description", "disabled" : false, "evaluationModes" : [
"PERIODIC" ], "evaluationStage" : "CI", "exception" : "<some exception in blob form>", "id" : "1",
"missingInformationAction" : "ISSUE", "modifiedAt" : "2024-07-28T18:50Z", "name" : "some policy",
"remediationGuidance" : "some remediation guidance", "revision" : "3", "ruleId" : "", "severity" : "CRITICAL", "type"
: "COMPLIANCE" }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("POST",
"/public_api/v1/cwp/policies") res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-yourfqdn/public_api/v1/cwp/policies") http
= Net::HTTP.new(url.host, url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request =
Net::HTTP::Post.new(url) response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("POST", "https://api-yourfqdn/public_api/v1/cwp/policies");
xhr.send(data);
HttpResponse<String> response = Unirest.post("https://api-yourfqdn/public_api/v1/cwp/policies") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/cwp/policies")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "POST" let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cwp/policies", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS =>
10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "POST", ]);
$response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error #:" . $err; }
else { echo $response; }
```

```
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "POST"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cwp/policies"); CURLcode ret = curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cwp/policies"); var request = new
RestRequest(Method.POST); IRestResponse response = client.Execute(request);
```

Body parameters
required
application/json; charset=UTF-8

The schema of a returned policy

▼ actionstring (Enum)

One of the supported policy actions

Example:`"ISSUE"`
Allowed values:"ISSUE""PREVENT"null
▼ assetGroupsarray[string]
▼ assetGroupsIDsarray[integer]
▼ assetScopestringbyte
▼ conditionstringbyte
▼ createdAtstringdate-time
▼ createdBystring
▼ descriptionstring
▼ disabledboolean
▼ evaluationModesarray[string]
▼ evaluationStagestring (Enum)

Lists supported evaluation stages

Example:`"CI"`
Allowed values:"CI""RUNTIME""DEPLOY"
▼ exceptionstringbyte
▼ idstring
▼ missingInformationActionstring (Enum)

One of the supported policy actions

Example:`"ISSUE"`
Allowed values:"ISSUE""PREVENT"null
▼ modifiedAtstringdate-time
▼ namestring
▼ remediationGuidancestring
▼ revisioninteger
▼ rulesIdsarray[string]

The UUIDs of the rules that define the condition

▼ severitystring (Enum)

One of the supported policy severities

Example:`"CRITICAL"`
Allowed values:"INFO""LOW""MEDIUM""HIGH""CRITICAL"null
▼ typestring (Enum)

One of the supported policy types

Example:`"MALWARE"`
Allowed values:"COMPLIANCE""MALWARE""SECRET""TRUSTED_IMAGES"null
REQUEST
`[application/json; charset=UTF-8 content]`
Responses

Success

Body
application/json; charset=UTF-8
▼ idstring
RESPONSE
`[application/json; charset=UTF-8 content]`

Client error

Body

application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

Reported error code

RESPONSE
`[application/json; charset=UTF-8 content]`

Forbidden error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

Reported error code

RESPONSE
`[application/json; charset=UTF-8 content]`

Service encountered an unexpected internal error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

Reported error code

RESPONSE
`[application/json; charset=UTF-8 content]`

2.11.2.21 | Delete a Policy by ID

delete /public_api/v1/cwp/policies/{id}

Delete a policy instance by its ID.

Path parameters
▼ id String required

Policy ID to delete

Example: `id_example`
Query parameters
▼ closeIssues Boolean

If set, will trigger closing of issues opened by the deleted policy.

Example: `true`
CLIENT REQUEST Bash+curl
```
curl -X 'DELETE'
-H 'Accept: application/json; charset=UTF-8'
'https://api-yourfqdn/public_api/v1/cwp/policies/{id}?closeIssues=true'
```

```
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("DELETE",
"/public_api/v1/cwp/policies/%7Bid%7D?closeIssues=SOME_BOOLEAN_VALUE") res = conn.getresponse() data = res.read()
print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D?closeIssues=SOME_BOOLEAN_VALUE") http = Net::HTTP.new(url.host,
url.port) http.use_ssl = true http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Delete.new(url)
response = http.request(request) puts response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("DELETE", "https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D?
closeIssues=SOME_BOOLEAN_VALUE"); xhr.send(data);
HttpResponse<String> response = Unirest.delete("https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D?
closeIssues=SOME_BOOLEAN_VALUE") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D?closeIssues=SOME_BOOLEAN_VALUE")! as URL, cachePolicy:
.useProtocolCachePolicy, timeoutInterval: 10.0) request.httpMethod = "DELETE" let session = URLSession.shared let
dataTask = session.dataTask(with: request as URLRequest, completionHandler: { (data, response, error) -> Void in if
(error != nil) { print(error) } else { let httpResponse = response as? HTTPURLResponse print(httpResponse) } })
dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D?closeIssues=SOME_BOOLEAN_VALUE", CURLOPT_RETURNTRANSFER => true,
CURLOPT_ENCODING => "", CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION =>
CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST => "DELETE", ]); $response = curl_exec($curl); $err = curl_error($curl);
curl_close($curl); if ($err) { echo "cURL Error #:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "DELETE"); curl_easy_setopt(hnd,
CURLOPT_URL, "https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D?closeIssues=SOME_BOOLEAN_VALUE"); CURLcode ret
= curl_easy_perform(hnd);
var client = new RestClient("https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D?
closeIssues=SOME_BOOLEAN_VALUE"); var request = new RestRequest(Method.DELETE); IRestResponse response =
client.Execute(request);
```

Responses

Success

Client error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

Reported error code

RESPONSE
`[application/json; charset=UTF-8 content]`

Forbidden error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

Reported error code

RESPONSE
`[application/json; charset=UTF-8 content]`

Policy ID not found

Body

application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

Reported error code

RESPONSE
```
[application/json; charset=UTF-8 content]
```

Service had unexpected internal error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

Reported error code

RESPONSE
```
[application/json; charset=UTF-8 content]
```

2.11.2.22 | Get a Policy by ID.

get /public_api/v1/cwp/policies/{id}

Get a policy instance by its ID.

Path parameters
▼ id String required
Example: `id_example`
CLIENT REQUEST Bash+curl
```
curl -X 'GET'
-H 'Accept: application/json; charset=UTF-8'
'https://api-yourfqdn/public_api/v1/cwp/policies/{id}'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("GET",
"/public_api/v1/cwp/policies/%7Bid%7D") res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Get.new(url) response = http.request(request) puts
response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("GET", "https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D");
xhr.send(data);
HttpResponse<String> response = Unirest.get("https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "GET" let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "GET", ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error
#:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "GET"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D"); CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D"); var request = new
RestRequest(Method.GET); IRestResponse response = client.Execute(request);
```

Responses

Success

Body
application/json; charset=UTF-8

The schema of a returned policy

▼ actionstring (Enum)

One of the supported policy actions

Example:`"ISSUE"`
Allowed values:"ISSUE""PREVENT"null
▼ assetGroupsarray[string]
▼ assetGroupsIDsarray[integer]
▼ assetScopestringbyte
▼ conditionstringbyte
▼ createdAtstringdate-time
▼ createdBystring
▼ descriptionstring
▼ disabledboolean
▼ evaluationModesarray[string]
▼ evaluationStagestring (Enum)

Lists supported evaluation stages

Example:`"CI"`
Allowed values:"CI""RUNTIME""DEPLOY"
▼ exceptionstringbyte
▼ idstring
▼ missingInformationActionstring (Enum)

One of the supported policy actions

Example:`"ISSUE"`
Allowed values:"ISSUE""PREVENT"null
▼ modifiedAtstringdate-time
▼ namestring
▼ remediationGuidancestring
▼ revisioninteger
▼ rulesIdsarray[string]

The UUIDs of the rules that define the condition

▼ severitystring (Enum)

One of the supported policy severities

Example:`"CRITICAL"`
Allowed values:"INFO""LOW""MEDIUM""HIGH""CRITICAL"null
▼ typestring (Enum)

One of the supported policy types

Example:`"MALWARE"`
Allowed values:"COMPLIANCE""MALWARE""SECRET""TRUSTED_IMAGES"null
RESPONSE
```
[application/json; charset=UTF-8 content]
```

Policy ID not found

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

Reported error code
RESPONSE
[application/json; charset=UTF-8 content]

Service had unexpected internal error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

Reported error code

RESPONSE
[application/json; charset=UTF-8 content]

2.11.2.23 | Update a Policy by ID.

put /public_api/v1/cwp/policies/{id}

Update a policy instance by its ID

Path parameters
▼ id String required

Policy ID to delete

Example: id_example
CLIENT REQUEST Bash+curl
```
curl -X 'PUT'
-H 'Accept: application/json; charset=UTF-8'
-H 'Content-Type: application/json; charset=UTF-8'
'https://api-yourfqdn/public_api/v1/cwp/policies/{id}'
-d '{ "action" : "ISSUE", "assetGroups" : [ "group1", "group2" ], "assetGroupsIDs" : [ 1, 2 ], "assetScope" : "<some
asset scope in blob form>", "condition" : "<some condition in blob form>", "createdAt" : "2024-07-28T18:50Z",
"createdBy" : "admin", "description" : "some policy description", "disabled" : false, "evaluationModes" : [
"PERIODIC" ], "evaluationStage" : "CI", "exception" : "<some exception in blob form>", "id" : "1",
"missingInformationAction" : "ISSUE", "modifiedAt" : "2024-07-28T18:50Z", "name" : "some policy",
"remediationGuidance" : "some remediation guidance", "revision" : "3", "ruleId" : "", "severity" : "CRITICAL", "type"
: "COMPLIANCE" }'
import http.client conn = http.client.HTTPSConnection("api-yourfqdn") conn.request("PUT",
"/public_api/v1/cwp/policies/%7Bid%7D") res = conn.getresponse() data = res.read() print(data.decode("utf-8"))
require 'uri' require 'net/http' require 'openssl' url = URI("https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D") http = Net::HTTP.new(url.host, url.port) http.use_ssl = true
http.verify_mode = OpenSSL::SSL::VERIFY_NONE request = Net::HTTP::Put.new(url) response = http.request(request) puts
response.read_body
const data = null; const xhr = new XMLHttpRequest(); xhr.withCredentials = true;
xhr.addEventListener("readystatechange", function () { if (this.readyState === this.DONE) {
console.log(this.responseText); } }); xhr.open("PUT", "https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D");
xhr.send(data);
HttpResponse<String> response = Unirest.put("https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D") .asString();
import Foundation let request = NSMutableURLRequest(url: NSURL(string: "https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D")! as URL, cachePolicy: .useProtocolCachePolicy, timeoutInterval: 10.0)
request.httpMethod = "PUT" let session = URLSession.shared let dataTask = session.dataTask(with: request as
URLRequest, completionHandler: { (data, response, error) -> Void in if (error != nil) { print(error) } else { let
httpResponse = response as? HTTPURLResponse print(httpResponse) } }) dataTask.resume()
<?php $curl = curl_init(); curl_setopt_array($curl, [ CURLOPT_URL => "https://api-
yourfqdn/public_api/v1/cwp/policies/%7Bid%7D", CURLOPT_RETURNTRANSFER => true, CURLOPT_ENCODING => "",
CURLOPT_MAXREDIRS => 10, CURLOPT_TIMEOUT => 30, CURLOPT_HTTP_VERSION => CURL_HTTP_VERSION_1_1, CURLOPT_CUSTOMREQUEST
=> "PUT", ]); $response = curl_exec($curl); $err = curl_error($curl); curl_close($curl); if ($err) { echo "cURL Error
#:" . $err; } else { echo $response; }
CURL *hnd = curl_easy_init(); curl_easy_setopt(hnd, CURLOPT_CUSTOMREQUEST, "PUT"); curl_easy_setopt(hnd, CURLOPT_URL,
"https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D"); CURLcode ret = curl_easy_perform(hnd);
```

```
var client = new RestClient("https://api-yourfqdn/public_api/v1/cwp/policies/%7Bid%7D"); var request = new
RestRequest(Method.PUT); IRestResponse response = client.Execute(request);
```

Body parameters
required
application/json; charset=UTF-8

The schema of a returned policy

▼ actionstring (Enum)

One of the supported policy actions

Example:`"ISSUE"`
Allowed values:"ISSUE""PREVENT"null
▼ assetGroupsarray[string]
▼ assetGroupsIDsarray[integer]
▼ assetScopestringbyte
▼ conditionstringbyte
▼ createdAtstringdate-time
▼ createdBystring
▼ descriptionstring
▼ disabledboolean
▼ evaluationModesarray[string]
▼ evaluationStagestring (Enum)

Lists supported evaluation stages

Example:`"CI"`
Allowed values:"CI""RUNTIME""DEPLOY"
▼ exceptionstringbyte
▼ idstring
▼ missingInformationActionstring (Enum)

One of the supported policy actions

Example:`"ISSUE"`
Allowed values:"ISSUE""PREVENT"null
▼ modifiedAtstringdate-time
▼ namestring
▼ remediationGuidancestring
▼ revisioninteger
▼ rulesIdsarray[string]

The UUIDs of the rules that define the condition

▼ severitystring (Enum)

One of the supported policy severities

Example:`"CRITICAL"`
Allowed values:"INFO""LOW""MEDIUM""HIGH""CRITICAL"null
▼ typestring (Enum)

One of the supported policy types

Example:`"MALWARE"`
Allowed values:"COMPLIANCE""MALWARE""SECRET""TRUSTED_IMAGES"null
REQUEST
`[application/json; charset=UTF-8 content]`
Responses

Success

Client error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

Reported error code

RESPONSE
`[application/json; charset=UTF-8 content]`

Forbidden error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

Reported error code

RESPONSE
`[application/json; charset=UTF-8 content]`

Policy ID not found

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

Reported error code

RESPONSE
`[application/json; charset=UTF-8 content]`

Service had unexpected internal error

Body
application/json; charset=UTF-8
▼ err_msgstring

The error message

▼ metadataobjectrequired

Details for the PublicAPIError

▶ codestring (Enum)

Reported error code

RESPONSE
`[application/json; charset=UTF-8 content]`